

iPlat4J V6 产品文档

iPlatUI 用户手册

宝信软件股份有限公司 平台研究所

目 录

1 iPlatUI 整体介绍	4
1.1 概述	5
1.2 架构	5
1.3 特点	6
1.4 浏览器兼容性	7
1.4.1 说明	7
1.4.2 Kendo UI 版本	7
2 iPlatUI 开发规范	8
2.1 Javascript 开发规范	8
2.1.1 JS 通用检查项	8
2.2 JSP 开发规范	8
2.2.1 表单类页面检查项	9
2.2.2 表格类页面检查项	10
2.2.3 树表类页面检查项	11
2.2.4 EFTab 页面检查项	11
2.2.5 EFRegion 页面检查项	12
2.2.6 EFWindow 页面检查项	13
2.3 UI 通用规则【约定优先于配置】	13
2.3.1 iPlatUI 动静分离方案	13
2.3.2 iPlatUI 登录页和首页定制指南	18
2.3.3 iPlatUI 的全局配置项说明	21
2.3.4 表单输入元素由动态属性调整为 Tag 属性	26
2.3.5 iPlatUI 按钮使用规范	27
2.3.6 iPlatUI EFGGrid 组件使用说明	38
2.3.7 iPlatUI-页面通知信息规范	39
2.3.8 iPlatUI 主题定制方案	43
3 iPlatUI 开发指南	46
3.1 UI-常见开发问题汇总	46
3.1.1 如何获取控件对象	46
3.2 前端开发入门教程	46
3.2.1 网页三剑客	46
3.2.2 CSS 入门教程	47
3.2.3 HTML 入门教程	47
3.2.4 Javascript 教程	48
3.3 前端工具箱	48
3.3.1 Chrome 的调试技巧	48
3.3.2 EiCommunicator	48
3.3.3 IPLAT	49
3.3.4 jQuery 与 Underscore 常用 API 整理	54
3.3.5 JS 中字符串转数字运算规则	56
3.3.6 Kendo Format 格式化输出	57
3.3.7 Kendo Grid 常用的 API	58
3.3.8 Kendo Template 使用介绍	59
3.3.9 KendoTreeView	61
3.3.10 WindowUtil 使用说明	67
3.3.11 如何使用 spreadsheet	68
3.3.12 页面与 Iframe 数据交互	68
3.4 UI 组件库	70
3.4.1 UI 组件	71
3.4.2 布局容器	72
3.4.3 表单组件	75
3.4.4 数据表格	104
3.4.5 树和菜单	121
3.4.6 基础组件	127

3.5 示例中心.....	128
3.5.1 布局容器	128
3.5.2 表单组件	165
3.5.3 数据表格	275
3.5.4 树和菜单	444
3.5.5 基础部件	505
3.5.6 How-To 定制方案.....	512
3.5.7 纯 CSS 页面布局.....	528

1 iPlatUI 整体介绍

1.1 概述

iPlatUI 是 iPlat4J 平台的前端解决方案的统称，iPlatUI 提供了一套业务开发中实用的组件库。主要包括以下四大类组件：

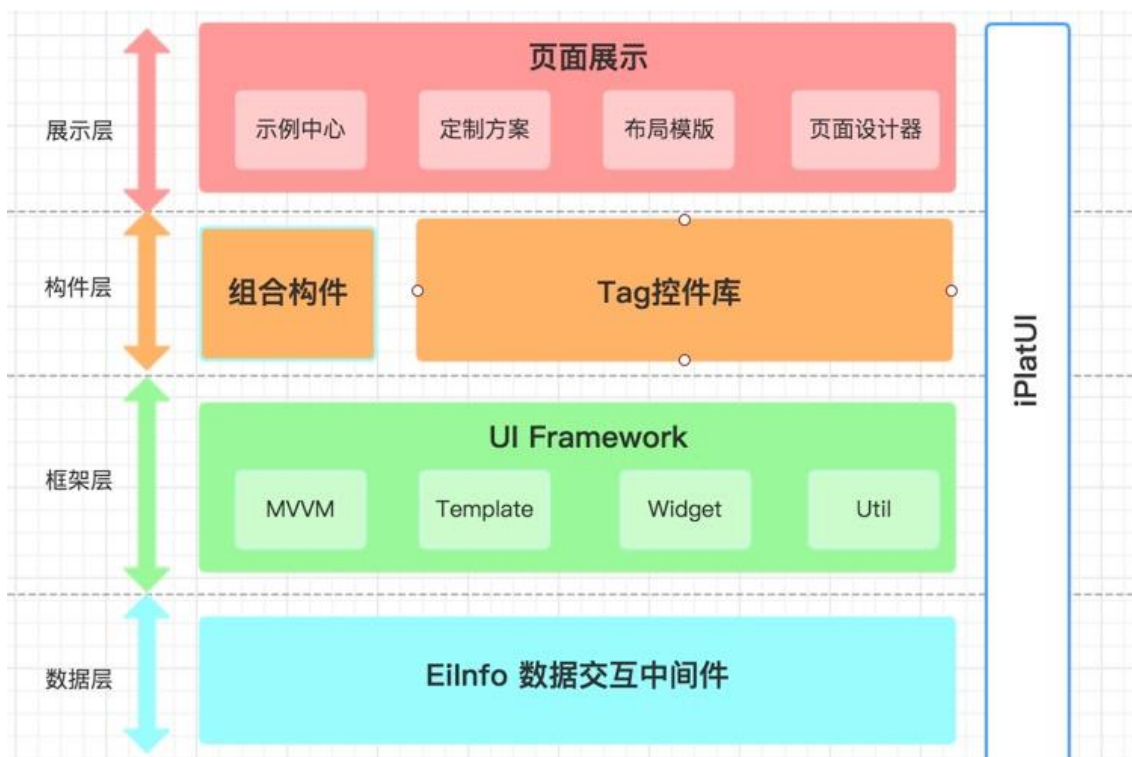
- 布局容器
- 表单组件
- 数据表格
- 树和菜单

iPlatUI 除了提供一套组件库之外，还包括了：

- 先进的开发理念
- 统一的开发规范
- 完善的扩展机制

iPlatUI 秉承了平台"make it simple"的理念，紧密围绕 EilInfo 核心数据协议，对业务开发中的常用组件进行高度封装复用，并通过 JSP Tag 2.0 完成组件化。从而降低前端开发门槛，提高整体开发效率。

1.2 架构



iPlatUI 的最底层是数据层，数据层的核心是 **EiInfo**，它是连接前后端的核心数据协议；前端页面的组件信息，完全通过 **EiInfo** 提供；

在数据层之上是框架层，框架层主要针对 **Javascript** 而言，通过使用 **jQuery**，**Underscore**，**KendoUI** 等 **Javascript** 库文件，提供了 **MVVM** 的机制，前端模版功能，**Javascript** 控件和前端常用的工具类；

在框架层之上是构件层，构件层主要针对 **JSP** 页面而言，通过 **JSP Tag 2.0** 的机制，对 **Javascript** 控件进一步定制封装，提供了 **Tag** 控件库和在 **Tag** 控件基础上，组合扩展的高级构件（组合构件是可复用的特定功能的页面小部件，例如登录框，页面反馈框等）。

在构件层之上是展示层，展示层是最终用户可见的界面。展示层的内容包括：介绍控件使用的示例中心，前端的定制化的解决方案，常用的页面布局模版，页面编辑器用于完成前端配置化，可视化的编辑设计等功能。

1.3 特点

多

组件种类多，适用的开发场景多

快

开发上手快

好

界面友好，浏览器兼容性良好

省

代码省，大量封装复用

1.4 浏览器兼容性

浏览器	支持版本	说明
Edge	20+	支持微软官方发布的版本
Chrome	49+	推荐使用官方发布的稳定版
Internet Explorer	IE8+	推荐使用IE9或更高版本，以达到最佳体验
Firefox	无版本要求	推荐使用官方发布的稳定版
Safari	6.2.6+	

1.4.1 说明

IE8+, Chrome49+ 浏览器是 iPlatUI 全面支持的浏览器，在这些浏览器出现组件类问题，可获得 iPlatUI 支持；

其他浏览器则不然。

1.4.2 Kendo UI 版本

目前使用 kendo ui 2016.3.1118 版本，是支持 IE8 的最新版。

IE的兼容性问题

由于IE8不支持HTML5和CSS3的特性，在IE8浏览器将无法达到最佳的用户体验，但会保证页面正常展示，功能完整。极度推荐用户更新使用现代的浏览器。

进一步了解，请参考：

[iPlatUI 开发规范](#)

[iPlatUI 开发指南【必读】](#)

2 iPlatUI 开发规范

2.1 Javascript 开发规范

2.1.1 JS 通用检查项

序号	问题	错误做法	正确的做法	说明
1	复制示例代码	开发js中出现 console.log , grid js出现无用的dataBound	正式开发中删除掉这些示例代码	
2	语句结束没有 ';' ;	<pre>function openParaForm(serviceId) { //设置url var url=IPLAT.JI.CONTEXT_PATH+"/web/XM02?method= var window = \$('#window'); window.data("kendoWindow").setOptions({ open: function () { window.data("kendoWindow").refresh({ url:url }); } }); window.data("kendoWindow").content(""); window.data("kendoWindow").open(); }</pre>		
3	局部变量定义使用了window	参考上图	使用page, form之类的命名; 或者使用popWindow	window和默认的window的对象容易混淆
4	if语句的代码块统一使用 {}, 即使if后只有一条语句	<pre>if (input.prev("span") && input.prev("span").hasClass("k-dropdown")) return input.prev("span"); if (input.closest("span") && input.closest("span").hasClass("k-p")) return input.closest("span"); if (input.closest("div") && input.closest("div").hasClass("k-multiselect")) return input.closest("div"); if (input.hasClass("k-popover")) return input.closest("span"); return input; catch (e) { // nodejs抛出的异常 return input; }</pre>	<pre>if(a===1) { }</pre>	代码风格统一, 方便阅读, 清晰判断if语句的作用范围
5	函数内部的变量名不要和函数的参数重复	<pre>// 阻止事件冒泡 function stopPropagation(e) { var e = e window.event; if (e && e.stopPropagation) e.stopPropagation(); else{ e.cancelBubble = true; } }</pre>	<pre>// 阻止事件冒泡 function stopPropagation(e) { var event = e window.event; if (event && event.stopPropagation) event.stopPropagation(); else{ event.cancelBubble = true; } }</pre>	

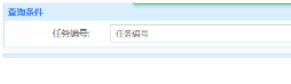
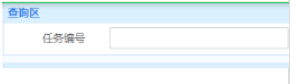
2.2 JSP 开发规范

本规范用于指导开发组, 开发整齐, 一致的页面。不是 JSP 的编码规范;
页面开发中常见且需要重视的问题:

序号	关注点	说明	正确示例	错误示例
1	注意页面对齐, 间距一致性			

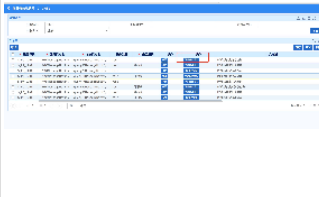
序号	关注点	说明	正确示例	错误示例
2	注意滚动条问题	1.简单的单表查询页面不应该出现纵向滚动条		
3	注意Grid中数据行的按钮大小			
4	按钮的图标			
5	页面布局的自适应性			
6	tab window region等容器嵌套显示的问题			

2.2.1 表单类页面检查项

检查项	说明	错误的例子	正确的做法	备注
1	字段的标签（label）应该右对齐，上下两行的输入框长度应保持相等；			调整ratio，具体参考 EEDM4000-EFInput数据绑定
2	input框前面的label应该统一， 没有冒号!!!			
3	页面保证对齐，统一			

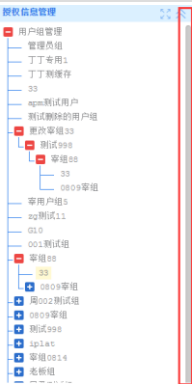
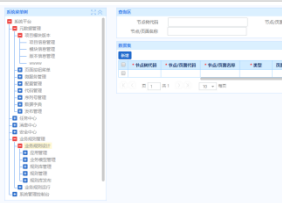
检查项	说明	错误的例子	正确的做法	备注
				

2.2.2 表格类页面检查项

检查项	说明	错误的例子	正确的做法	备注
1	右侧按钮和Grid需要对齐			
2	按钮尺寸不符合标准			具体标准见iPlatUI按钮使用规范
3	简单的单表查询不应出现滚动条			
4	多个行操作按钮:应该统一放在一个单元格里面; 按钮列推荐放在grid所有列的前面(固定列):方便操作; 按钮间间距为10px;			

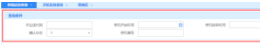
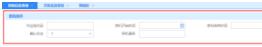
检查项	说明	错误的例子	正确的做法	备注
5	Grid中: Data Bound 无意义, 需要删除		<pre> dataBound: function() { console.log("数据渲染结束"); } </pre>	

2.2.3 树表类页面检查项

检查项	说明	错误的例子	正确的做法	备注
1	左侧树高度自适应			页面右侧不能有滚动条, 滚动条应出现在树区域内。
2	树表高度需要保证一致, 保证美观			

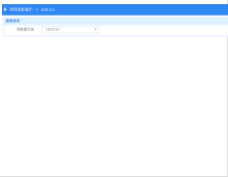
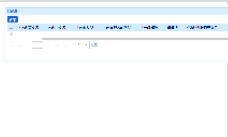
2.2.4 EFTab 页面检查项

检查项	说明	错误的例子	正确的做法	备注
1	Tab的各个标签页的高度要保持统一, 防止切换Tab页面, 高度变化造成的页面抖动			
2	block-content左右没有间距。			<pre> .row { margin-right: 15px; margin-left: 15px; } </pre> 使用了div=row, 所以需要去除外边距。

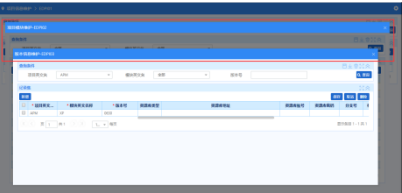
检查项	说明	错误的例子	正确的做法	备注
3	Tab内的Region没有边框			<code><div class="ui-tab" data-bbox="565 154 729 174"> <div class="ui-tab-item" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> <div class="ui-region" data-bbox="565 154 729 174"> </div> </div> </div> </div> </div> </div> </div> </div> </div> </div> </div> </div> </div></code>
4	Tab不应置于记录集中			

2.2.5 EFRegion 页面检查项

检查项	说明	错误的例子	正确的做法	备注
1	Region之间宽度需要保证一致			具体间距大小 请见 视觉设计规范
2	查询区的Region添加了无用的属性	<code><div class="ui-region" data-bbox="354 664 488 684" type="query" efRegionShowClear="true"></code>	<code><div class="ui-region" data-bbox="491 664 625 684" id="result" title="结果"></code>	只要指明id和title就可以了
3	Region右侧不可贴边			具体间距大小 请见 视觉设计规范
4	按钮大小需要规范			具体请见文档 Bootstrap 字体图标 (Glyphicons)

检查项	说明	错误的例子	正确的做法	备注	
5	Region 中包含多个图标按钮(保存, 清空, 下载), 用户根据需求选用, 而不是全部都选用。				
6	Region内Tab没有边框				
7	按钮规范: Region里有Grid时, 按钮一般放置在右上方, 而不是表尾。			具体顺序见EFGrid工具栏按钮显示顺序说明	具体顺序见 EFGrid工具栏按钮显示顺序说明

2.2.6 EFWindow 页面检查项

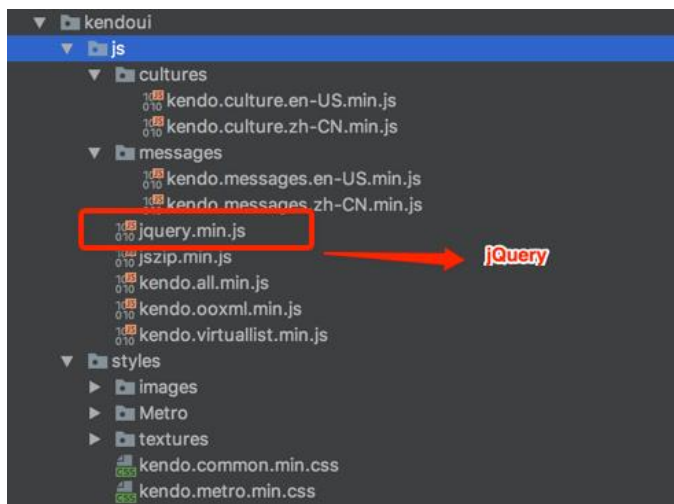
检查项	说明	错误的例子	正确的做法	备注
1	 window里禁止弹window			

2.3 UI 通用规则【约定优先于配置】

2.3.1 iPlatUI 动静分离方案

2.3.1.1 iPlatUI 静态资源目录结构说明

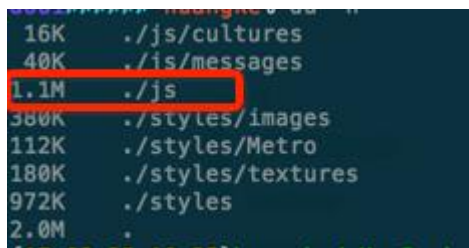
1. kendoui 的相关依赖文件



web 的根目录下 kendoui 目录（web/kendoui），包含了 kendoui 的 js 和 css 图片等静态资源；文件总大小约 2M。其中：

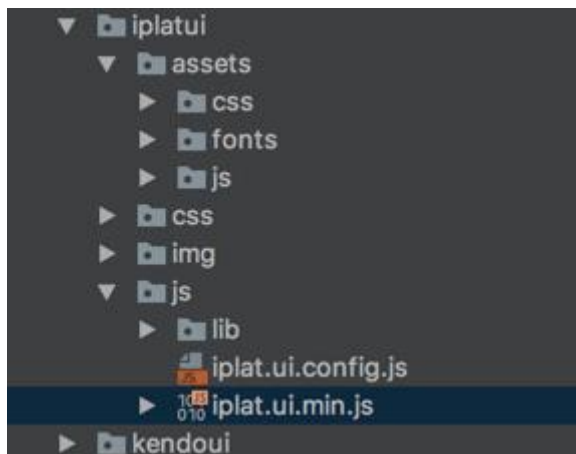
js 库约 1M；（其中 kendo.all.min.js 约 830KB，此文件已经精简，原始文件约为 2.1MB）

图片，字体文件，css 约 1M；



注意，jquery 文件在 kendoui/js 目录下，当前使用的版本为 1.12.3。

2. iplatui 的目录结构说明



iplatui 目录和 kendoui 目录相同，也在 web 的根目录下。其下分为四个子目录：

assets, css, img, js

web

|---- kendoui

|---- iplatui

|---- assets

|---- css

|---- img

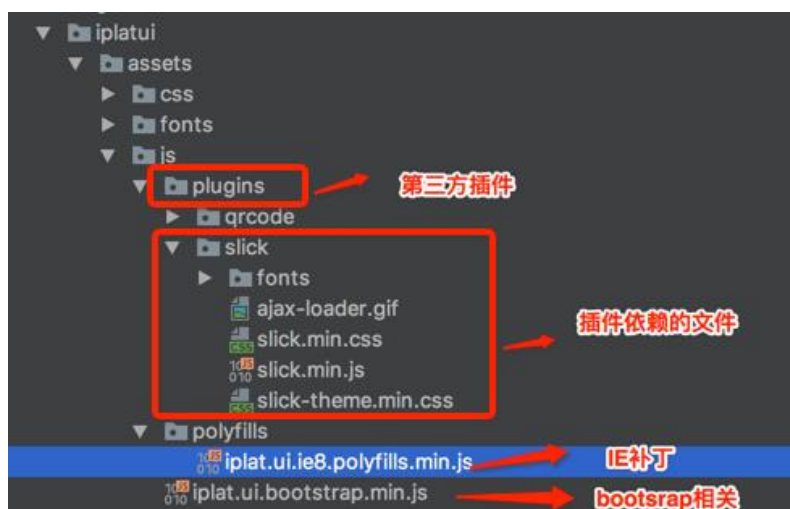
|---- js

2.1 assets 目录

此目录存放 iplatui 依赖的第三方的组件，例如 bootstrap, fontawesome 等。

子目录 css 和 fonts，存放（且仅存放）bootstrap 和 fontawesome 的样式和字体文件。

js 目录存放第三方的插件库和 IE 的补丁脚本。

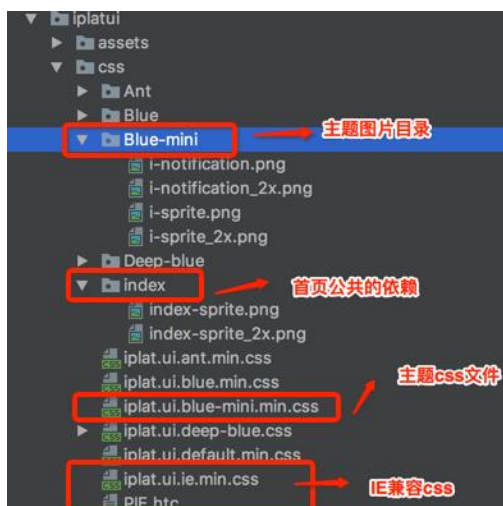


注意 bootstrap 和 fontawesome 属于 iplatui 通用的依赖，已经被压缩构建为 iplat.ui.bootstrap.min.js;

assets/js/plugins 目录，存放其他非强依赖的第三方组件库，例如二维码 轮播图等。【平台其他模块的依赖的第三方库也应放在此目录，例如工作流的 mxGraph 等】

2.2 css 目录

此目录存放 iplatui 不同主题的 css 文件和依赖的主题图片，和 IE 兼容相关的 css 文件。



☒ index 目录重构整理,主题文件重复

2.3 img 目录

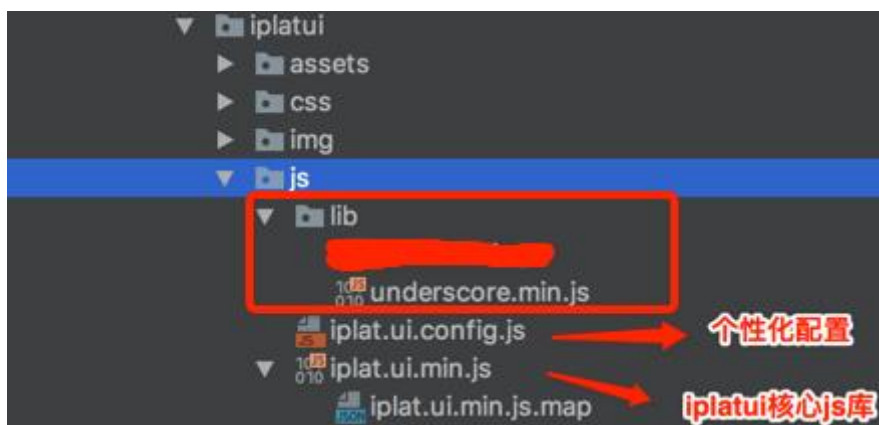
此目录存放 iplatui 公用的图片资源，例如登录页的背景图，出错页面的图片资源等

☒ 考虑和 css 中的图片重构

2.4 js 目录

此目录存放 iplatui 核心 js 库，和个性化配置相关的 iplat.ui.config.js;

js/lib 子目录，存放强依赖的 js 库（例如 underscore，注意和 assets/js/plugins 的区别）



2.3.1.2 资源文件的分离

粗犷的做法：在 EDCC03 配置， iplatui 的资源根目录。kendoui 和 iplatui 相关的资源都存放在配置的服务器下

配置项名称：

`iplat.ui.static.url`

应用自身的资源目录配置

配置项名称：

`projectName.ui.static.url`

细腻的做法：在 EDCC03 配置 iplatui 的 img，js css 目录

配置项名称：

```
projectName.ui.static.img.url
```

```
projectName.ui.static.css.url
```

```
projectName.ui.static.js.url
```

css 和 img 不建议分离配置，css 会使用 img 的资源。

2.3.1.3 页面开发中 URL 的使用规则

序号	规则	说明	正确示例	错误示例
1	js和jsp文件中不允许出现相对路径	使用类似./, ../的相对路径，增加了代码的理解难度； 页面的URL变化时，相对路径往往也需要跟着变化	【jsp代码】 "\${ctx}/web/EEDM6000"	"/DispatchAction.do?efFormEname=EEDM6000"
2			【js代码】 IPLATUI.CONTEXT_PATH + "/web/EEDM6000"	"/DispatchAction.do?efFormEname=EEDM6000"
3	js和jsp文件中不允许出现DispatchAction.do	HRA的语义化URL设计	http://10.25.103.112:8082/http://10.25.103.112:8082/iPlatDAM/web/EDFA00	http://10.25.103.112:8082/iPlatDAM/DispatchAction.do?efFormEname=EDFA00http://10.25.103.112:8082

jsp 中获取 contextPath 的说明：

JSP

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:set var="ctx" value="${pageContext.request.contextPath}"/>
```

js 中获取 contextPath 的说明：

JS

平台托管的页面（使用了EFPAGE标签的JSP页面），在页面对应的JS中，

可以直接使用`IPLATUI.CONTEXT_PATH` 获取应用的上下文；

非平台托管页面，可以在JSP的顶部插入脚本,首先还是要通过JSP获取应用上下文：

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<:set var="ctx" value="${pageContext.request.contextPath}"/>
```

在JSP中加入的js脚本

```
<script>
var ctx = "${ctx}";
</script>
```

在js中，就可以使用ctx变量，获取当前上下文了

综上所述，请将更改相关 js 和 jsp 代码

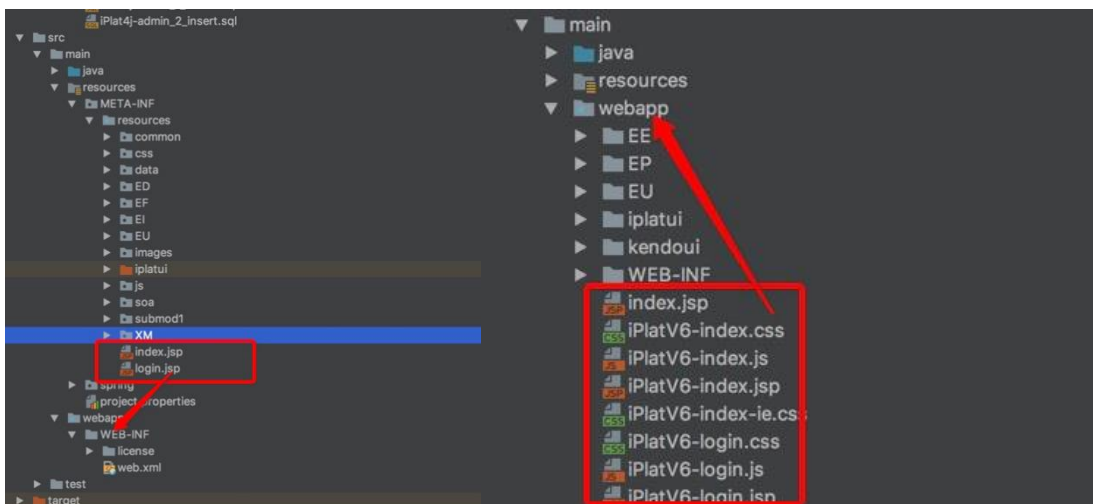
	页面中需要改动的 URL	改动后的 URL
jsp	<code>./DispatchAction.do?efFormName=</code>	<code>\${ctx}/web/</code>
js	<code>./DispatchAction.do?efFormName=</code>	<code>IPLATUI.CONTEXT_PATH + "/web/"</code>

2.3.2 iPlatUI 登录页和首页定制指南

注意事项

1. login.jsp index.jsp的来源和作用
2. 不能把login.jsp index.jsp打到jar包

login.jsp ,index.jsp 和各模块定制的 iPlatV6-* 不要打到 jar 包里面，统一挪动到 webapp 目录。



2.3.2.1 页面功能简介

本示例页面用于首页定制的简单说明。

2.3.2.1.1 首页模块分类

首页目前分为六个模块：首页轮播，左侧菜单，页面号查询，收藏页面，消息主看板，首页刷新或者关闭时，关闭所有的子窗口，如下图所示：

```
slick: true, // 首页轮播
menu: true, // 左侧菜单
pageSearch: true, // 页面号查询
favorite: true, // 收藏页面
panelBar: true, // 消息主看板
unload: true // 首页刷新或者关闭时，关闭所有的子窗口
```

2.3.2.1.2 重写首页模块方法

项目组需要重写首页模块方法，只需在自定义的首页 jsp（如 pscs-index.jsp）里引入自定义的 js（如 pscs-index.js），在 PSCS-index.js 里面实现六大模块自己的逻辑即可。

```
// 轮播图
V6Index.prototype.slick = function() {
    $('#i-slick').slick({
        slidesToShow: 1,
        slidesToScroll: 1,
        dots: true,
        autoplay: true,
        autoplaySpeed: 5000,
        appendDots: $('#i-region-dots-append'),
        dotsClass: 'slick-dots i-slick-dots'
    });
    // 解决IE下prev、next按钮图标不能正常显示的问题
    // prevArrow: '<button type="button" class="fa fa-angle-left">Previous</button>',
    // nextArrow: '<button type="button" class="fa fa-angle-right">Next</button>'
};
```

例如轮播，只需重写 js 逻辑即可自定义实现。

2.3.2.1.3 新增或修改首页模块方法

项目组要新增自定义模块，需要在首页 jsp（如 pscs-index.jsp）里增加如下代码：

```
<script>
    window.IPLAT_INDEX_WINDOW = "rootWindow";
    $(function () {
        V6Index({pscsxxx: true});
    });
</script>
```

在 V6Index 的对象里面新增自定义属性(例如：pscsxxx)，同时 value 设为 true。并在首页 js（如 pscs-index.js）里实现新增逻辑方法即可。

如需要修改方法名，只需将原有方法设置为 false，将自定义方法名加入 V6Index 的属性里面。

重写首页方法的时候，有几个地方推荐如下写法：

1.init 方法里面，需加上如下代码在尾部：

```
$iplatMenu.getNiceScroll().resize();
```

2.menu 菜单方法里面，dataBound 与 expand 的写法如下：

```
dataBound: function () {
    $("#iplat-menu").getNiceScroll().resize();
},
// expand是处理是子节点折叠起来，然后展开
// 可以通过aria-
// expanded存在这个属性表明是子节点已经获取，不需要再从后端获取节点
// 信息
expand: function (e) {
    var node = e.node;
    var expanded = $(node).attr('aria-expanded');
    if (expanded !== void 0) {
        setTimeout(function () {
            $("#iplat-menu").getNiceScroll().resize();
            // setTimeout: 200是异步执行，防止子节点没有展开完
        }, 200);
    }
}
```

3.panelBar 方法里面，如果是异步请求的回调数据，需要在 dom 节点刷新之后重新调用一次 uiLayout()方法：

```
$panel.kendoPanelBar({
```



```
dataSource: dataSource
});

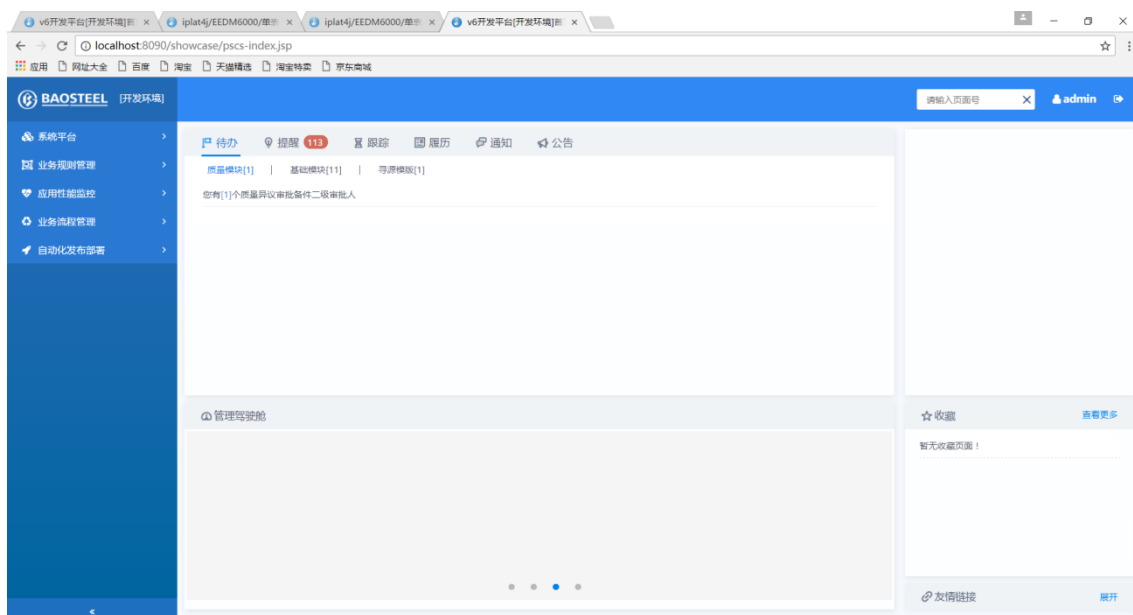
uiLayout();
```

2.3.2.1.4 登录跳转重新指向新的首页

在 index.jsp 里面，修改代码：

```
window.location.href = '${ctx}/pscs-index.jsp';
```

结果图：



2.3.3 iPlatUI 的全局配置项说明

2.3.3.1 配置文件的位置

2.3.3.1.1 平台内部

平台内部的配置文件位于 `webapp/iplatui/js/iplat.ui.config.js`。用于定义 UI 组件的个性化的全局配置；

项目组

项目组没有特定的配置文件，但是可以通过：

1. 重写覆盖 `webapp/iplatui/js/iplat.ui.config.js`
2. 在 `${prefix}-head.jsp` 文件中局部覆盖 `IPLATUI.Config` 中的原始配置。详情请见 [UI 常见开发问题 FAQ\(Q4\)](#)

2.3.3.2 配置项的含义

UI 配置项

```

extend(IPLATUI, {

    Config: {
        Layout: { // 按钮的文字 图标显示控制
            TEXT: "1", // 仅文字
            ICON: "2", // 仅图标
            ICON_AND_TEXT: "3" // 图标文字都显示
        },

        // 配置 Notification 提示
        Notification: {
            LEVEL: "INFO", // INFO: 0, DEBUG: 5, WARNING: 15, ERROR: 25 小于设定 LEVEL 的通知
            不会显示
            AUTO_HIDE_AFTER: 6000, // 配置消息展示时间
            SHOW_DETAIL_MSG: true, // 默认显示 EiInfo 中的 detailMsg
            COPY: true, // 复制按钮是否显示
            LOCK: true, // 固定按钮是否显示
            APM: true, // APM 按钮是否显示
            OFF: false, // 全局控制的开关
            WIDTH: "60%", // 整体宽度
            LEFT: "20%"
        },

        // 模态对话框
        Modal: {
            ALERT_MIN_WIDTH: 300, // 警告对话框配置的默认最小宽度 (>=300)
            CONFIRM_MIN_WIDTH: 350, // 确认对话框配置的默认最小宽度 (>=300)
            PROMPT_MIN_WIDTH: 350, // 输入对话框配置的默认最小宽度 (>=300)

            ALERT_TITLE: "警告", // 警告对话框配置的默认标题
            CONFIRM_TITLE: "确认", // 确认对话框配置的默认标题
            PROMPT_TITLE: "提示" // 输入对话框配置的默认标题
        },

        EFPage: {
            fitHeightClass: "i-fit-height", // 高度适应到底部的元素的 jQuery Class 选择
            器

            topOffset: 200, // 滚动的偏移量, 超出后将出现 region 导航和回到顶部的按钮
            scrollTop: true, // 默认显示回到顶部
            regionNav: true, // 默认显示 Region 导航
            paddingBottom: 8 // EFPage 页面内 form 的底部 padding
        },

        EFRegion: {
            borderH: 2 // EFRegion 的上下 border 的高度
        },

        EFGGrid: {
            allowCopy: true,
            autoBind: true, // 是否主动进行查询
            // height: 350,
            resizable: true,
            sortable: false, // 默认是 false, 为 true 时, 进行单列排序
            selectable: true, // 单行选中, 单元格的复制, 此种场景没有列选功能

            editable: true, // 编辑权限 incell

```

```

        headerAttributes: {
            style: "text-align:center"
        },
        pageable: {
            pageSize: 10, // DataSource 设置会覆盖此处设置
            pageSizes: [10, 20, 50, 100] // "all" // 分页配置
        },

        // IPLAT 平台封装的配置项, 非 kendoui 标准属性
        autoDraw: "yes", // Grid 中的列渲染模式
        checkMode: "multiple, cell", // 行的勾选模式
        needAuth: true, // 增删改按钮是否需要注册授权
        copyToAdd: true, // 行复制新增功能
        readonly: false, // Grid 只读控制
        enable: true, // Grid 的编辑, 提交控制
        rowNo: false, // 是否显示行序号
        rowNoText: "序号", // 行序号的列头文本
        rowNoAlign: "center", // 行序号默认居中显示

        pagerPosition: "top", // 翻页条默认配置在上方, EFGGrid 上方工具条按钮较多时,
        应用根据具体情况将翻页条放置在底部
        showCount: false, // 后台是否查询总数

        columnWidth: 120,
        minColumnWidth: 50, // 列 resize 的最小值
        rowHeight: 24,

        exportGrid: true, // 默认导出当前页面的数据
        editHelper: false, // 勾选时, 可编辑的单元格突出显示
        validateHelper: true, // 单元格编辑校验出错时, 红框突出提示
    },

    EFPopup: {
        filterPopupGrid: true, // 控制 EFPopupColumn, EFPopupInput 弹出的 Grid 的列是
        否显示过滤条件
        showCount: false
    },

    EFSelect: {
        animation: { // 下拉的动画效果 (注释掉后则使用默认的向下展开动画效果, 取消
        动画效果需将配置改为 animation: false)
            // close: {
            //     effects: "fadeOut zoom:out",
            //     duration: 300
            // },
            // open: {
            //     effects: "fadeIn zoom:in",
            //     duration: 300
            // }
        }
    },

    EFDate: {
        format: "yyyy-MM-dd",
        culture: "zh-CN",
        animation: {
            // close: {

```

```

        //      effects: "fadeOut zoom:out", //关闭时动画特效类型，多个特效用空
格隔开。
        //      duration: 400 //关闭时动画持续的时间，单位是毫秒。
        // },
        // open: {
        //      effects: "fadeIn zoom:in",
        //      duration: 400
        // }
    }
},

EFDateTime: extend({}, this.EFDate, {
    format: "yyyy-MM-dd HH:mm:ss"
}),

EFDateSpan: {
    extChar: ""
},

EFTree: {
    PID: "inqu_status-0-node", // 展开结点时，查询条件为结点的 Id，默认存储在 EiInfo
的 inqu_status-0-node（三段式）位置
    EXPAND_LEVEL: "inqu_status-0-expandLevel", // 默认展开层级会将参数存储在 EiInfo
的 inqu_status-0-expandLevel（三段式）位置
    ROOT_ID: rootId // $ 默认表示根结点的 id
},

EFTreeList: {
    ROOT_ID: rootId // $ 默认表示根结点的 id
},

EFTab: {},

EFWindow: {
    width: "80%", // 适合弹窗显示一个完整的页面 "1000", "80%", 1000 都是可接受的
    height: "80%",
    left: "10%",
    top: "10%",

    minWidth: "60%", // 适合显示局部的 div 内容
    minHeight: "60%",
    minLeft: "20%",
    minTop: "20%"
},

EFInput: {},

EFUpload: {
    initUrl: "/EU/DM/EUDM07.jsp", // 查询单个附件，或者附件组的信息 JSON 格式
    saveUrl: "/EU/DM/EUDM04.jsp", // 附件上传的 Servlet
    downloadUrl: "/EU/DM/EUDM06.jsp", // 附件下载的 Servlet
    removeUrl: "/EU/DM/EUDM08.jsp", // 附件删除的 Servlet
    downloadZip: "/EU/DM/EUDM14.jsp" // 多附件下载压缩包的 Servlet
},

EFAutoComplete: {},

```

```

// 列快捷菜单
EFColumnShortcutMenu: [
    // type => 类别, name => 名称, icon => 字体图标, invokeFn => 调用方法
    {
        type: 'sortAsc',
        name: '升序',
        icon: 'fa-sort-alpha-asc',
        invokeFn: function (gridInstance, column) {
            gridInstance.dataSource.data().sort(function (a, b) {
                return a[column] > b[column];
            });
            gridInstance.dataSource._change({action: 'sync'});
        }
    },
    {
        type: 'sortDesc',
        name: '降序',
        icon: 'fa-sort-alpha-desc',
        invokeFn: function (gridInstance, column) {
            gridInstance.dataSource.data().sort(function (a, b) {
                return a[column] < b[column];
            });
            gridInstance.dataSource._change({action: 'sync'});
        }
    },
    {
        type: 'autoFit',
        name: '宽度自适应',
        icon: 'fa-arrows-h',
        invokeFn: function (gridInstance, column) {
            gridInstance.autoFitColumn(column);
        }
    }
]
});

```

2.3.3.3 iPlatUI 依赖的配置项（EDCC03）

开发前，应首先检查 EDCC03 的 SQL 脚本和具体环境中的 EDCC03 配置

注：红色的配置项由 UI 组维护

配置项目名称	含义	使用的地方
esHost	ES分析器的地址	index页面的APM图表数据来源（ServiceEF0001#queryAPM）
IPLAT_APM_URL	iPlatAPM的部署地址	调用链路的跟踪, 查看traceId的详情
iplat.ui.static.url	iPlatUI中的静态资源的地址	引入iPlatUI的JS和CSS时使用，不配置时，则使用Web容器的Context。配置完整的静态服务器的url.(http://static.server)
iplat.service.	分布式部署时，/service,	分布式部署时，通过nginx反向代理，用统一域名进行访问。

配置项目名称	含义	使用的地方
context	/web的根上下文	单体应用的Context需要被省略. (例如PSCS系统上线的场景) 不配置时, 则使用Web容器的Context
browser	浏览器版本	IE8的提示
browser.ie8.url	IE8浏览器的下载链接	XP可升级的最高版本
browser.ie11.url	IE11浏览器的下载链接	win7可升级的最高版本
browser.edge.url	Edge浏览器的下载链接	Win8, Win10可以使用的Edge版本浏览器
browser.chrome.url	Chrome浏览器的下载链接	XP上可以使用的最高版本Chrome 49

2.3.4 表单输入元素由动态属性调整为 Tag 属性

表单输入元素的 **readonly/ disabled/ required** (只读, 禁用, 必填) 属性之前大多作为动态属性配置, 现做如下调整( 指动态属性,  指 tag 属性)

表单输入元素 Tag 名称	readonly 属性		disabled 属性		required 属性	
EFInput						
EFPopupInput						
EFTreeInput						
EFDatePicker						
EFDateSpan						
EFSelect						
EFMultiSelect						
EFCascadeSelect						

注意

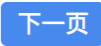
以上三个属性均是Boolean类型, 要注意配置成例如

`readonly="true", required="true"`

以前的动态属性配置如 `readonly="readonly", required="required"` 等将不起作用！！

2.3.5 iPlatUI 按钮使用规范

2.3.5.1 按钮的分类

按钮主样式		表现形式	辅助样式	使用场景	样式	按钮的 class
大按钮		正常		查询区域的按钮		i-btn-lg
	主要按钮	大边距		查询区域的按钮		i-btn-lg + i-btn-wide
	图标按钮	有图标	fa	字体图标		i-btn-lg
			i-icon	图片图标		
	次要按钮			查询区域的次要按钮, 模块对话框的取消按钮		i-btn-sec + i-btn-wide
	下拉按钮	有图标	k-icon (右侧下拉图标)	图片图标 (右侧下拉图标)		i-btn-lg + i-btn-drop
中按钮		正常		grid翻页信息上的操作按钮		i-btn
小按钮		正常		grid的行内的按钮 (Grid行内按钮一定要用i-btn-sm)  注: 1. 按钮所在的列若设置为enable="false", 即不能编辑; 2. 按钮的高度是 18px ; 3. 按钮居中显示		i-btn-sm
				日期控件 (e.g.今天) 等弹出控件上的按钮		i-btn-sm
文字按钮		正常		日期控件 (e.g.清空) 等弹出控件上的按钮		i-btn-text (可通过i-btn-sm等控制大小)

按钮主样式		表现形式	辅助样式	使用场景	样式	按钮的 class
所有按钮		禁用		按钮禁用后的样式		&:disabled
		文字				

2.3.5.2 按钮特殊布局

按钮布局的 class	使用场景	样式	备注
btn-align-right	grid单元格popupInput弹出textarea的回填按钮		
	弹出页面的右下角的确定按钮		
btn-inline-block	时，添加该class使其成为行内块		

2.3.5.3 EDFA01 按钮注册说明

● 用途

开发人员在开发平台页面的时候，需要使用按钮作为页面展示的一个组件或者通过提供按钮去触发某些事件。平台通过 **EDFA01** 这个页面对 **region** 区域中的按钮进行管理，帮助开发人员**减少按钮开发工作**和**方便权限管理**。某些开发人员不清楚 **region** 区域是指的那块区域，所谓的 **region** 区域是指 **<EF:EFRegion>.....</EF:EFRegion>**所包含的区域。还有些开发人员可能不清楚在 **region** 里面的按钮是否是一定需要通过 **EDFA01** 这个页面来注册才能使用么，平台是支持开发人员自己去定义一个 **<button>**，平台只是管理通过平台注册的按钮。

- 路径
 - 【系统平台】 - 【平台元数据管理】 - 【页面资源管理】 - 【按钮信息管理】
- 操作步骤

页面截图如下：

* 画面英文名	* 按钮英文名	* 区域标识	按钮中文	按钮描述	节点排序标识	是否分级	* 是否授权	资源地址	样式
EDCC03	QUERY	DIV:EDCC03	查询		5		授权	css:fa-search	显示文字和图标
EDCC03	INSERT	GRID:EF_GRID_RESULT	新增		1		授权		
EDCC03	SAVE	GRID:EF_GRID_RESULT	保存		2		授权		
EDCC03	DELETE	GRID:EF_GRID_RESULT	删除		3		授权		
EDCC03	CANCEL	GRID:EF_GRID_RESULT	取消		4		授权		
EDCC04	SAVE_1	GRID:EF_GRID_RESULT_DB	保存		1		授权		
EDCC04	TESTDB	GRID:EF_GRID_RESULT_DB	连接测试		2		授权		
EDCC04	SAVE_2	GRID:EF_GRID_RESULT_INL	保存		1		授权		
EDCN00	F2	DIV:EDCN00	查询		1		授权	css:fa-search	显示文字和图标
EDCN00	REFRESH	DIV:REFRESH	刷新缓存			请选择是否分级	授权		

开发人员可以通过此页面对所需要开发的页面中按钮进行注册。至于按钮注册的规则请参考: [EFGGrid 操作按钮注册规则](#)

2.3.5.3.1 按钮信息查询

1. 在查询条件区域输入查询条件
2. 点击【查询】按钮
3. 记录集区域显示符合查询条件的按钮记录

画面英文名与按钮英文名为半模糊查询

2.3.5.3.2 按钮信息新增

1. 点击记录集区域的【新增】，记录集中新增一条空记录
2. 录入信息，点击【保存】按钮

- 画面英文名为该按钮所在的页面的在页面信息管理中注册的英文名
- 按钮英文名为该按钮的英文名字
- 区域标识为按钮所在的区域
- 按钮中文
- 按钮描述为该按钮的中文描述信息
- 节点排序标识表示该region中button从前往后的排列顺序。默认是按照节点排序标识数字从小到大，而从左往右排列
- 是否授权，表示该按钮是否受到平台的权限控制，如果不授权默认所

有用户可以点击该按钮（前提是用户可以访问页面）

- 资源地址通常是标志按钮中图标的地址，如新增的图标 。（后面会针对资源地址，样式，位置进行详细说明）
- 样式为该Button所展示的形式
- 位置通产表示button所处的位置

2.3.5.3.3 按钮信息修改

1. 勾选记录集区域中需要修改的按钮记录
2. 编辑信息，点击【修改】按钮

2.3.5.3.4 按钮信息删除

1. 勾选记录集区域中需要删除的按钮记录
2. 点击【删除】按钮

2.3.5.3.5 字段属性补充说明

开发人员可能对于资源地址，样式，位置这几个属性怎么使用可能不是特别清楚，这里给大家补充说明一下

2.3.5.3.5.1 资源地址

资源地址表示按钮前面的图标，例如 ，这里图标使用的是字体图标，它的地址是 <http://fontawesome.io/icons/>，我们只是需要在此字段中填入相关字体图标样式即可。如:fa-gift, fa-globe.....等等,这样我们就能够修改 button 里面的图标样式。

2.3.5.3.5.2 样式

样式表示此 button 按钮的展示形式，里面参数包含 "默认样式"，"只显示文字"，"只显示图标"，"显示图标和文字"。顾名思义，这里表示注册的按钮是否展示文字和图标的组合。

按钮样式显示规则

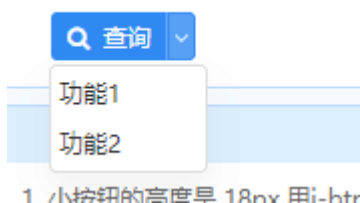
1. 样式选择 "默认样式" 或者 "只显示文字"，按钮只包含文字，无论是否配置了图标(资源地址)都不显示；
2. 样式选择 "只显示图标"，按钮只包含图标信息，图标信息统一要求的配置格式为 "css:fa-search", 若为平台的雪碧图配置的图标，则格式为 "css:i-add"；

3. 样式选择 "显示图标和文字", 那么按钮包含图标和文字信息, 图标在前。

2.3.5.3.5.3 位置

位置包含默认值、下方。我们现在一般按钮注册不允许注册在上方, 不符合规范; 如果要注册的话, 可以注册 **div** 按钮:

<http://platform.baosight.com:8090/pages/viewpage.action?pageId=12558337> ; 或者后续, 我们会在下拉按钮中解决这个需求:



2.3.5.4 EDFA01-按钮自由注册说明

EDFA01 页面可以注册按钮, 按钮通常只能注册在 **EFRegion**, **EFGrid** 上, 现在按钮可以注册到页面任意 **DIV** 中。

注册规则如下:

区域标识的注册内容 **DIV:div 的 id (大写)**

记录集										
	* 画面英文名	* 按钮英文名	* 区域标识	按钮中文	按钮描述	节点排序标识	是否分级	* 是否授权	资源地址	样式
☐	EDCC03	QUERY	DIV:EDCC03	查询		5		授权	css:fa-search	显示文字和图标
☐	EDCC03	INSERT	GRID:EF_GRID_RESULT	新增		1		授权		
☐	EDCC03	SAVE	GRID:EF_GRID_RESULT	保存		2		授权		
☐	EDCC03	DELETE	GRID:EF_GRID_RESULT	删除		3		授权		
☐	EDCC03	CANCEL	GRID:EF_GRID_RESULT	取消		4		授权		
☐	EDCC04	SAVE_1	GRID:EF_GRID_RESULT_DB	保存		1		授权		
☐	EDCC04	TESTDB	GRID:EF_GRID_RESULT_DB	连接测试		2		授权		
☐	EDCC04	SAVE_2	GRID:EF_GRID_RESULT_INL	保存		1		授权		
☐	EDCM00	F2	DIV:EDCM00	查询		1		授权	css:fa-search	显示文字和图标
☐	EDCM00	REFRESH	DIV:REFRESH	刷新缓存			请选择是否分级	授权		下

示例场景

通常在 **EFRegion** 查询区注册按钮, 按钮会独占一行空间。此时可以将按钮注册到指定的 **DIV** 中。

改进后: (注意 **JSP** 页面上的 **div** 的 **id** 是小写的, 注册在 **EDFA01** 中的 **div** 的 **id** 是大写的)

按钮自由注册

```
<EF:EFRegion id="inqu" title="查询条件" type="query" efRegionShowClear="true"
efRegionSave="true">
  <div class="row">
```

```

<EF:EFInput ename="inqu_status-0-product_id" cname="产品代号" />
<EF:EFInput ename="inqu_status-0-company_name" cname="公司名称"/>
<div class="col-md-4" style="text-align: right" id = "bbb"></div> <!--按钮注册
到 div 中-->
</div>
</EF:EFRegion>

```

div 中的按钮不会占一行，完全由开发者在 HTML 中控制。

2.3.5.5 EFGGrid 操作按钮注册规则

页面上 Grid 的按钮注册，现在需要遵循以下几条规则：

1. 页面 Grid 按钮工具栏上 新增，保存，取消，删除 4 个基础功能按钮，必须在 EDFA01 上注册后才能使用，按钮英文名(BUTTON_ENAME)必须分别对应 **"INSERT" "SAVE" "CANCEL" "DELETE"**



注意

如果要新增，需要注册 INSERT，SAVE（CANCEL 用于取消已有的操作，可选，但推荐注册）

如果要修改，需要注册 SAVE（CANCEL 用于取消已有的操作，可选，但推荐注册）

如果要删除，需要注册 DELETE（CANCEL 用于取消已有的操作，可选，但推荐注册）

2. 注册按钮时，区域标识(REGION_ID)格式形如 **"GRID:EF_GRID_RESULT"** (其中 RESULT 对应 grid 的 blockId 值), 需大写配置

☐	*画面英文名	*按钮英文名	*区域标识	按钮中文	按钮描述	排序
☐	EDDBT1	COPY	RESULT	表间挑选		5
☐	EDDBT1	UPDATE	RESULT	更新		6
☐	EDDBT101	RETURN	RESULT	返回		1
☐	EDFA00	CANCEL	GRID:EF_GRID_RESULT	取消		
☐	EDFA00	DELETE	GRID:EF_GRID_RESULT	删除		
☐	EDFA00	INSERT	GRID:EF_GRID_RESULT	新增		
☐	EDFA00	SAVE	GRID:EF_GRID_RESULT	保存		
☐	EDFA00	EXPORT	INQU	导出		1
☐	EDFA00	F2	INQU	查询		2
☐	EDFA00	SQL	RESULT	生成数据...		6

一个页面上出现多个 GRID 时，注册的按钮英文名的规则为：**按钮功能_流水号**。

例如页面上有两个 GRID，新增的按钮英文名应为：INSERT，INSERT_1

☐	* 画面英文名	* 按钮英文名	* 区域标识	按钮中文	按钮描述	节点排序标识
☒	EDFA01	DELETE	GRID:EF_GRID_RESULT	删除		
☒	EDFA01	INSERT	GRID:EF_GRID_RESULT	新增		
☒	EDFA01	UPDATE	GRID:EF_GRID_RESULT	保存		
☒	EDFA01	DELETE_1	GRID:EF_GRID_RESULT1	删除		
☒	EDFA01	INSERT_1	GRID:EF_GRID_RESULT1	新增		
☒	EDFA01	UPDATE_1	GRID:EF_GRID_RESULT1	保存		

3. 按钮显示逻辑:

首先权限配置的优先级最高，其次 IPLATUI(JS 中对 toolbarConfig 原生)的配置，优先级最低的是 Tag 中的配置(权限配置>IPLATUI>Tag)

3.1 未经注册或无权限的按钮不会显示

3.2 IPLATUI 中的配置会覆盖 Tag 中对 4 个基础功能按钮的配置，例如 Tag 中配置 toolbarConfig="{add:false, cancel:true}"

注意

此处toolbarConfig中"add" 的配置对应于Grid的新增按钮INSERT。

但是 IPLATUI 中配置 toolbarConfig:{add:true, save:false, delete:true, buttons:[{...},{...}]},

那么最终会合并成{"add:true,save:false,cancel:true, delete:true, buttons[{...},{...}]"},即会显示新增，取消，删除以及 IPLATUI 中自定义的一些按钮

3.3 IPLATUI 中若配置 toolbarConfig:{hidden:true, ...} 则隐藏工具栏，所有按钮都不可见。

IPLATUI 中 hidden 设置也可以覆盖 Tag 中 hidden 的配置。

示例代码:

JSP 示例

```
<EF:EFGrid blockId="result" autoDraw="no" toolbarConfig="{add: true, save:true, cancel: false}">
  <EF:EFColumn ename="company_name" cname="公司名称"/>
  <EF:EFColumn ename="made_country" cname="产地" />
  <EF:EFColumn ename="sale_date" cname="上市日期" type="datetime"/>
  <EF:EFColumn ename="price" cname="价格"/>
  <EF:EFColumn ename="company_page" cname="公司首页"/>
</EF:EFGrid>
```

注: Tag 中设置显示"add", "save", 默认显示"delete", 隐藏"cancel"按钮

JS 示例

```

IPLATUI.EFGrid = {
  "result": {
    toolbarConfig: {
      hidden: false, // true 不显示 toolbar
      add: false,    // 不显示新增按钮
      delete: true,  // 显示删除按钮

      buttons: [ // 不允许覆盖默认的 4 个按钮，可以设置自定义的按钮
        {
          name: "lock",
          text: "锁定",
          icon: "fa-lock",
          click: function () {
            WindowUtil({
              title: "定制",
              content: "自定义的按钮, 在工具栏的右侧"
            });
          }
        },
        {
          name: "edit",
          text: "编辑",
          icon: "fa-edit",
          layout: "3",
          click: function () {
            resultGrid.editRow(resultGrid.element.find(".k-grid-content tr.i-
state-selected"))
          }
        }
      ]
    }
  }
};

```

注:

1. IPLATUI 中设置显示"delete", 隐藏"add", 覆盖原 Tag 中"add"的显示设置
2. 结果合并应会该显示"save", "delete","cancel" 以及 buttons 内自定义的 2 个按钮, 隐藏"add"
3. 需要配置 layout: 按钮显示方式, 1 只显示文字, 2 只显示图标, 3 显示图标和文字。如果不配置, layout 默认为 1, 图标不会显示

页面效果如下:

Popup EditForm 弹出表单编辑器

10 每页 共 58 条
保存 取消 删除 锁定 编辑

	产品编号	公司名称	产地	价格	上市日期	公司类别
☐	S556	Apple	USA	¥999.00		
☐	S552	Apple	USA	¥999.00		
☐	1112	Motorola	China	¥11.00		
☐	S891	Philips	Germany	¥1,200.50		
☐	TN782	los	China	¥8.00		
☐	f5c8d8fc	宝信软件		¥10.00		
☐	K-T61812	Motorola1	13	¥15.00		
☐	K-T6181	Motorola1	13	¥15.00		
☐	Tttttt	Motorola1	13	¥15.00		
☐	T61811	los	9	¥15.00		

注： "锁定", "编辑"是在 JS 里定义的按钮

我们在 EDFA01 中自定义 Grid 按钮: 按钮 0-2 , class = "k-button k-button-icontext k-grid-" + BUTTON_ENAME 值

Popup EditForm 弹出表单编辑器

(K) < 页 1 共 6 > X 10 每页 共 58 条

按钮2 按钮0 按钮1 设置

	产品编号	公司名称	产地	价格	上市日期	公司类别
☐	S556	Apple	USA	¥999.00		
☐	S552	Apple	USA	¥999.00		
☐	1112	Motorola	China	¥11.00		
☐	S891	Philips	Germany	¥1,200.50		
☐	TN782	Ios	China	¥0.00		
☐	f5c8d8fc	宝信软件		¥10.00		
☐	K-T61812	Motorola1	13	¥15.00		
☐	K-T6181	Motorola1	13	¥15.00		
☐	Tttttt	Motorola1	13	¥15.00		
☐	T61811	Ios	9	¥15.00		

若我们在此基础上在 IPLATUI 中配置 hidden: true 如下, 则按钮工具栏被隐藏, 效果如下:

JS 示例

```

IPLATUI.EFGrid = {
  "result": {
    toolbarConfig: {
      hidden: true,    // true 不显示 toolbar
      add: false,     // 不显示新增按钮
      delete: true,   // 显示删除按钮

      buttons: [
        {...}
      ]
    }
  }
}

```

Popup EditForm 弹出表单编辑器

(K) < 页 1 共 6 > X 10 每页 共 58 条

	产品编号	公司名称	产地	价格	上市日期	公司类别
☐	S556	Apple	USA	¥999.00		
☐	S552	Apple	USA	¥999.00		
☐	1112	Motorola	China	¥11.00		
☐	S891	Philips	Germany	¥1,200.50		
☐	TN782	Ios	China	¥0.00		
☐	f5c8d8fc	宝信软件		¥10.00		
☐	K-T61812	Motorola1	13	¥15.00		
☐	K-T6181	Motorola1	13	¥15.00		
☐	Tttttt	Motorola1	13	¥15.00		
☐	T61811	Ios	9	¥15.00		

注：如若想要显示自定义导出按钮, 需开启全局配置 exportGrid: true (默认导出当前页面的数据)

页面效果如下图所示:

Popup EditForm 弹出表单编辑器

页 1 共 6 10 每页 共 58 条

	产品编号	公司名称	产地	价格	上市日期	公司类别
☐	S556	Apple	USA	¥999.00		
☐	S552	Apple	USA	¥999.00		
☐	1112	Motorola	China	¥11.00		
☐	S891	Philips	Germany	¥1,200.50		
☐	TN702	los	China	¥0.00		
☐	f5c0d8fc	宝信软件		¥10.00		
☐	K-T61812	Motorola1	13	¥15.00		
☐	K-T6181	Motorola1	13	¥15.00		
☐	Tttttt	Motorola1	13	¥15.00		
☐	T61811	Los	9	¥15.00		

2.3.5.6 EFGGrid 工具栏按钮显示顺序说明

2.3.5.6.1 Grid 按钮工具栏包含哪些按钮

Grid 按钮工具栏中可以显示三类按钮：基础功能按钮(包括"新增", "保存", "取消", "删除")，后台注册的自定义按钮，js 中设置的自定义按钮。

2.3.5.6.2 按钮排序规则

1. 基础功能按钮排列在按钮工具栏最前，严格按照 **"新增"-"保存"-"取消"-"删除"** 依次排列；
2. 后台注册的自定义按钮紧接基础功能之后，按照按钮注册列表 EDFA01 中按钮的 **NODE_SORT_ID 从小到大排列**，示例中 3 个后台自定义按钮："按钮 0"，"按钮 1"，"按钮 2"的 NODE_SORT_ID 值分别是 2,1,3，从右到左（从小到大）排列顺序则为"按钮 2"，"按钮 0"，"按钮 1"；
3. js 中设置的自定义按钮渲染在最后，根据 js 文件中 **toolbarConfig.buttons** 中定义的按钮的先后顺序排列，示例中 2 个 js 自定义按钮"锁定"，"自定义复制"按序排列。

2.3.5.6.3 示例效果

Popup EditForm 弹出表单编辑器

页 1 共 6 10 每页 共 58 条

	产品编号	公司名称	产地	价格	上市日期	公司类别
☐	S556	Apple	USA	¥999.00		
☐	S552	Apple	USA	¥999.00		
☐	1112	Motorola	China	¥11.00		
☐	S891	Philips	Germany	¥1,200.50		
☐	TN702	los	China	¥0.00		
☐	f5c0d8fc	宝信软件		¥10.00		
☐	K-T61812	Motorola1	13	¥15.00		
☐	K-T6181	Motorola1	13	¥15.00		
☐	Tttttt	Motorola1	13	¥15.00		
☐	T61811	Los	9	¥15.00		

按钮2 按钮0 按钮1

注册按钮

记录集

页 1 共 6 10 每页 共 58 条

	公司名称	产地	上市日期	价格	公司首页	公司首页1
☐	Motorola1a1j11		2007-04-27 08:31:19	⊕	ER90	
☐	Motorola1a1j		2007-03-17	⊕	EL23	
☐	Motorola1a1		2007-04-06	⊕	时段访问量统计	
☐	Samsung		2007-04-06	⊕	http://www.samsung.com	
☐	SonyEricsson		2007-04-06	⊕	http://www.sonyericsson.com	
☐	SonyEricsson		2007-04-27	⊕		
☐	Philips		2007/03/08	⊕	http://www.philips.com	
☐	SonyEricsson		2007/03/10	⊕		
☐	Motorola		2007-07-12	⊕		
☐	Motorola		2007-07-12	⊕		

基础按钮 JS按钮

2.3.5.7 EDFA01 按钮自定义样式说明

1. 通过 EDFA01 页面注册的按钮，涉及 Region 区、Div 区和 Grid 区。此前我们是配置 EDFA01 页面的 Grid 上“资源地址”字段来控制按钮图标的样式，比如配置 "css:fa-search"；现在如果有想对按钮本身样式的自定义需求，我们可以在“资源地址”字段上配置例如 "btnClass:btn_1 btn_2" 来实现。

注意

- 1."btnClass" 与 "css" 配置间以 ";" 隔开；
- 2.btnClass后可接多个自定义样式，各样式名间以空格隔开；
- 3.自定义的样式需在页面定义或引入css文件后才会生效，**样式名前需要配合当前页面号定义**，比如

```
<style>
  #EEDM3004 .btn_1 {
    background-color: yellow;
  }
  #EEDM3004 .btn_2 {
    color: #000;
  }
</style>
```

2. EDFA01 页面上配置示例

按钮信息管理 > EDFA01									
查询条件									
画面英文名	EEDM3004	按钮英文名		工程	IPLAT4J				
记录创建开始日期		记录创建截止日期							
记录集									
新增									
* 画面英文名	* 按钮英文名	* 区域标识	按钮中文	按钮描述	排序	是否分级	* 是否...	资源地址	样式
EEDM3004	BUTTON02	DIV:BTN_DIV	DIV上按钮			不分级	不授权	btnClass:btn_2	显示文字和图标
EEDM3004	INSERT	GRID:EF_GRID_RESULT	新增		0	不分级	不授权	css:k-add,btnClass:btn_1 btn_2	只显示文字
EEDM3004	BUTTON01	R1	REGION区按钮			不分级	不授权	css:fa-search,btnClass:btn_1 btn_2	显示文字和图标

3. 页面效果

按钮注册自定义样式示例 > EEDM3004													
Region和Div上按钮													
通过EDFA01页面注册的按钮，若想要自定义按钮样式(如改变按钮背景色，字体颜色等)，可以配置“资源地址”字段格式如下： css:fa-search,btnClass:btn_1 btn_2 其中，css部分与btnClass部分以“;”隔开，多个样式间以空格分开。 自定义样式需在样式名前加上当前画面英文名(如 #EEDM3004 .btn_1 {background-color: yellow;})													
DIV上按钮													
Grid上按钮													
* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字	
qqq	huawei	america	2017/11/01..	150122	baosteel	N70		baosight66..	https://st..				
1111	sumsong2	america	2017/11/01..	150122		N70		baosight66..	https://st..				
N7021	MotorolaIn..	GB,German,..	2017/11/01..	150122		N71		baosight66..	https://st..				
111EE	111	GB,German,..	2017/11/07..	11		N7011		abc@baosig..	https://st..				
test1x	AAAAA	GB,German,..	2017/12/15..	0				abc@baosig..					
N7022	MotorolaIn..	GB,German,..	2017/11/01..	150122	EEF1	N70		abc@baosig..	https://st..				
N70	oppo	GB,German,..	2017/11/22..	150122	EEE	N70		abc@baosig..	https://st..				
8251	MotorolaIn..	GB,German,..	2017/11/16..	150122	EEE	N70		abc@baosig..	https://st..				
SGH-I7161	Samsung	GB,German,..	2007-04-06	10.14	http222wwwsamsungco..	查询	13800000000	abc@baosig..	fuyimin	021-508011..	1		
T6281	SonyEricss..	Japan	2007-04-06	30	http://www.sonyeric..	查询	13800000000	abc@baosig..	label	021-508011..	1		

2.3.6 iPlatUI EFGGrid 组件使用说明

2.3.6.1 EFGGrid 字段相关说明

EFGGrid 封装的 kendoGrid，而 kendoGrid 有保留字段：id、parent、bind 等，所以 javabean 中不应出现这几个字段，否则会导致保存失败等错误。

nodeName 也是保留字段，它是 DOM 元素的基本属性。使用该字段作为列名，将引发 jQuery TAG 选择器的异常。

nodeName 属性会导致 jQuery 的 bug
<https://github.com/jquery/jquery/issues/1721>

2.3.6.2 EFGGrid 列类型设置说明

EFGGrid 类型：

列类型	说明	状态	显示说明
C	Character, 字符串, 默认的列类型		单元格中的字符串, 默认是左对齐的
N	Number, 数值类型		单元格中的数字, 默认是右对齐的
D	Date, 日期类型	开发中	
B	Boolean 布尔类型	开发中	
E	Enumerate, 枚举类型, 下拉列	开发中	

其他规则：

1. 主键列（primaryKey=true），是固定列，固定列在 grid 的左侧显示；
2. 非空列（nullable=false），列头渲染时，会有红色的 * 提示；

2.3.6.3 EFGGrid 的查询条件构造

Grid 中的列的数据类型分为四种：字符串（string），数字（number），日期（date），枚举（enums，下拉选项）

	eq	neq	isnull	isnotnull	isempty	isnotempty	startsWith	contains	doesnotcontain	endsWith
string	✓	✓	✓	✓	✓	✓	✓ like%	✓ e% %lik	✓ not %like%	✓ ke %li
number	✓	✓	✓	✓	gt >	gte >=	lt <	lte <=	✗	✗
date	✓	✓	✓	✓	gt >	gte >=	lt <	lte <=	✗	✗
enums	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗

2.3.6.4 EFComboColumn 数据列交互规则

1. 当我们需要对 EFGrid 中的下拉列给定数据源（具体几种方式见 [EEDM6002-设置表格的列类型](#)）时，约定所传下拉选项的 **key 值都必须为 string 类型**，不允许使用 int 类型数据，否则可能会出现数据保存不上的情况。
2. 数据库存储时要求是例如 int 类型时，可通过将后台 Service 接收到的 inInfo 中对应列数据转换成 int 类型后再进行存储。

2.3.7 iPlatUI-页面通知信息规范

概要说明

● 页面状态及信息的使用

页面提示信息按类型分成三类：正常信息、提示信息、异常信息，在对应的页面模板中分别显示为绿色图标、黄色闪烁图标、红色闪烁图标。

通过设置后台业务返回的 EiInfo 的 **status**，可以控制前台页面的信息显示方式。如果返回的 **status** 为零，将信息显示为正常信息；如果返回的 **status** 大于零，将信息显示为提示信息；如果返回的 **status** 小于零，将信息显示为异常信息。

除了正常的提示信息外，可以单独设置详细的提示信息。

参考：[数据交换中间件 EiInfo 的使用](#)

显示正常信息的代码示例为：

java

```
outInfo.setStatus(0);
outInfo.setMsg("查询成功！共查到 " + num + " 条记录");
```

页面效果显示如下：



显示提示信息的代码示例为：

java

```
outInfo.setStatus(1);
outInfo.setMsg("对[" + i + "]条记录执行更新操作成功!");
```

页面效果显示如下：



显示异常信息的代码示例为：

java

```
inInfo.setStatus(-1);
```

页面效果显示如下：



2.3.7.1 NotificationUtil-通知的简单应用

2.3.7.1.1 页面功能简介

本文档用于说明 **NotificationUtil** 用于通知消息的简单介绍。

2.3.7.1.2 配置和 API 详情

NotificationUtil 用于预处理消息，参数可接收 **EiInfo**.js 对象以及字符串，前台会自动取 **msg** 与 **detailMsg**，其中 **msg** 为标题，**detailMsg** 为内容。

2.3.7.1.3 页面开发步骤

2.3.7.1.3.1 参数为 EiInfo

我们在后台将 **status** 设置成异常标识位"-1",并设置 **EiInfo** 的 **msg**，最后返回此 **EiInfo**。

Service

```
public EiInfo insert(EiInfo inInfo) {
    try{
        .....
        inInfo.set("result-limit",1000);
        EiInfo queryInfo = query(inInfo);
        List efFormEnameList = queryInfo.getCol("result","form_ename");
        for(int i=0; i <efFormEnameList.size();i++ ){
            if(efFormEnameList.get(i).equals(efFormEname)) {
                throw new RuntimeException( efFormEname + "页面已在收藏列表，请勿重复收
藏! ");
            }
        }
        dao.insert("EDFA10.insert",tedfa10);

    } catch (Exception e) {
        logError("收藏信息异常：",e.getMessage());
        inInfo.setStatus(-1);
        inInfo.setMsg(e.getMessage());
    }

    return inInfo;
}
```

在前台，我们将调用 **NotificationUtil** 进行通知渲染。

JS

```
onSuccess: function (eiInfo) {
    if (eiInfo["status"] == -1) {
        NotificationUtil(
            eiInfo
            ,"error");
    }
}
```

```

    } else {
      NotificationUtil({
        msg: '收藏成功'
      });
    }
  }
}

```

页面效果如图：

NotificationUtil(eiInfo, "error") 代码段效果：

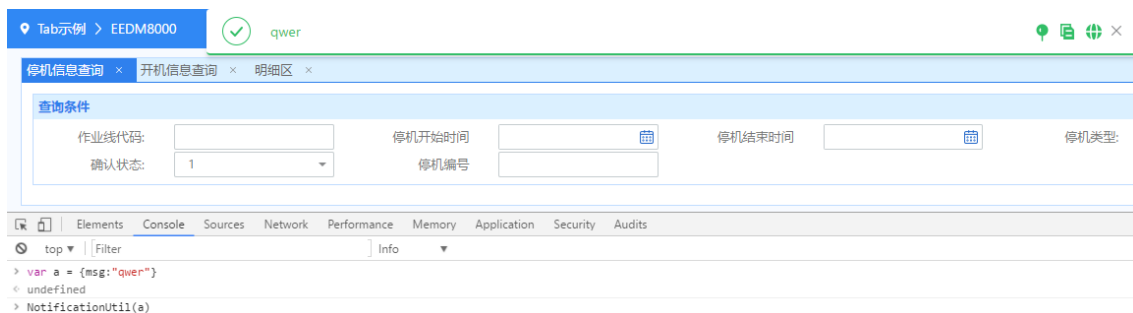


NotificationUtil({msg: '收藏成功'}) 代码段效果：



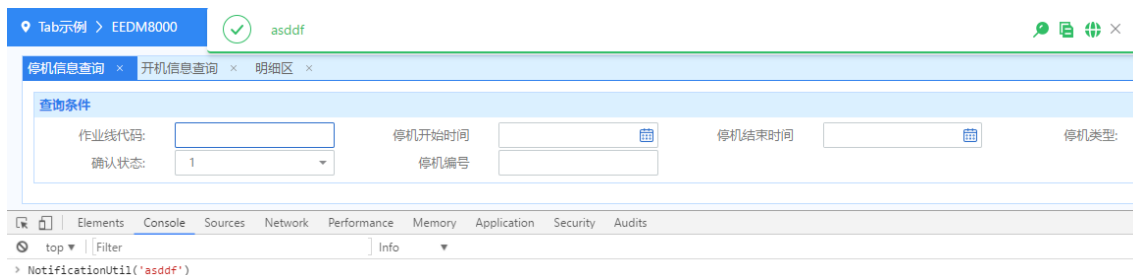
注意：只有第二个参数为"error"时通知才会显红。

2.3.7.1.3.2 参数为 js 对象

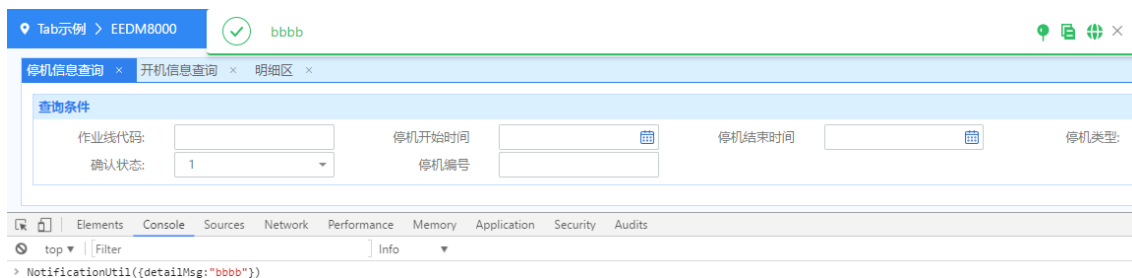


2.3.7.1.3.3 参数为字符串

直接设置字符串：



当msg与detailMsg同时存在时，msg为标题，detailMsg为内容。
当msg不存在，detailMsg存在的时候，detailMsg会取代msg做标题。



2.3.7.1.3.4 通知栏介绍



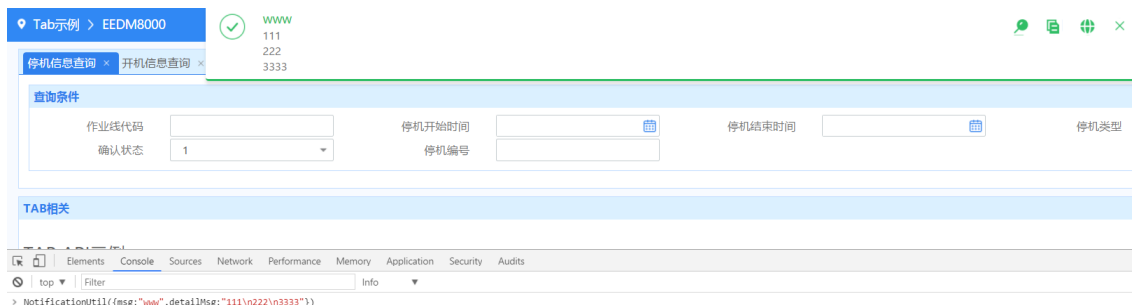
如图所示，在通知栏右边有三个小图标。

其中第一个图标为固定，如图所示的状态即固定状态，此时状态栏不定时，不覆盖会一直存在。

第二个图标为复制，该按钮会复制 `msg` 和 `detailMsg`。复制成功之后我们可将其 `ctrl+v` 拷贝出来。并且 `msg` 与 `detailMsg` 会换行显示。

注意：`msg` 的长度不应超过20个，否则会缩略显示。

当需要消息换行显示的时候，需要在 `detailMsg` 里写入 `\n`，如下图所示：



需要注意的是，`msg` 是不允许换行的。只有 `msg` 与 `detailMsg` 同时存在的时候，`detailMsg` 里的信息才会换行显示。单独写入 `msg` 或 `detailMsg` 都不会换行显示。

第三个图标为链接到 APM。

2.3.7.1.3.5 通知的四种状态

`NotificationUtil` 的第二个参数可设置 `info`, `success`, `warning`, `error` 四种状态，如图：

通知信息：



成功信息:



警告信息:



异常信息:



2.3.8 iPlatUI 主题定制方案

iPlatUI 目前支持三种主题定制方案。

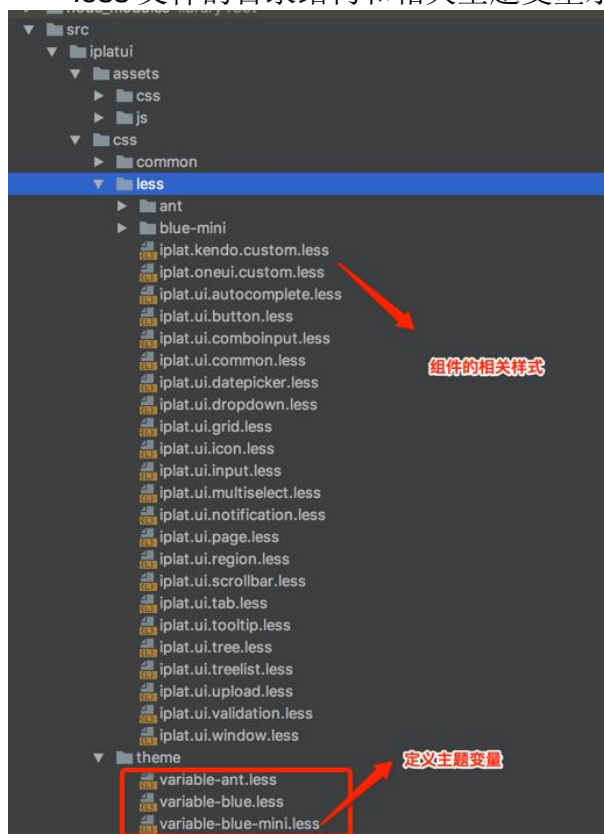
- 整体定制：当前系统中的所有页面将统一更改主题风格，目前可以使用 EDFA61 页面切换平台默认提供的主题。
- 按模块定制：当前系统中的不同模块，可以使用不同主题。某一模块下的页面使用特定的主题。
- 个性化定制：当前系统中的某些页面需要单独定制样式。

2.3.8.1 整体定制

当前平台使用 **less** 针对各组件定制主题，项目组如需整体定制：

1. 可向平台提供定制方案的设计稿，由平台开发人员根据设计稿，编写构建出项目所需的 **CSS** 主题文件；
2. 向平台申请提供 **less** 文件，由项目开发人员，本地自行编写构建出 **CSS** 主题文件。

less 文件的目录结构和相关主题变量示例：




```

@import ".../common/mixin.less";
@import ".../common/variable.less";

@baseFontSize: 16px;

@theme: @blue;
@white: #FFFFFF;
@page-header-background: #001F6B; // 页面上方的导航路径的背景色
@region-header-background: #CEE6FA; // EFRegion的标题背景色
@region-header-color: #2168C7; // EFRegion的标题文字的颜色
@region-border-color: #CEE6FA; // EFRegion的边框颜色
@active-border-color: #659700; // 按钮, 输入框active时的边框颜色

@main-container-background: #F7FAFF; // 页面的主背景色
@text-color: #666666; // 文字的颜色
@grid-text-color: @text-color; // Grid中的文字的颜色

@page-padding: 10px 15px; // 页面主容器(form, li-form)的padding
@page-bottom: 0; // 页面主容器底部的padding
@region-bottom: 9; // region之间的间距

@dropdown-menu-background: @white; //弹出的dropdown-menu或k-list-container的背景色
@dropdown-menu-shadow-color: fade(#9EA5AA, 26%); //弹出的dropdown-menu或k-list-container的阴影颜色

@region-header-padding: 7px 15px 7px;
@region-header-font-size: 16; // region标题的字体
@region-header-line-height: @region-header-font-size * 1.5;

@region-content-padding-left: 15; // region内容的padding-left
@region-content-padding-right: @region-content-padding-left; // region内容的padding-right
@region-content-padding-top: 10; // region内容的padding-top
@region-content-padding-bottom: @region-content-padding-top; // region内容的padding-bottom

@form-group-bottom: 8; // form表单中的两行之间的间距
|
@input-border-color: #C6CDD0; // 输入框正常状态的边框颜色
@input-active-border-color: #2B6DCB; // 输入框激活状态的边框颜色
@input-border-radius: 2; // 输入框的圆角

@form-dropdown-li-hover-background: #6BF3FC; //dropdown下拉选项hover状态背景色
@form-dropdown-li-selected-background: #2B6DCB; //dropdown下拉选项selected状态背景色

@form-date-picker-header-text-color: @form-date-picker-text-color; //datepicker组件顶部文本颜色
@form-date-picker-text-color: #2B6DCB; //datepicker组件中文本颜色
@form-date-picker-number-color: #89898C; //datepicker组件的日期颜色
@form-date-picker-text-other-month-color: #D0DEE1; // @datepicker组件中日期文本
@form-date-picker-text-weekend-color: #89898C;
@form-date-picker-text-hover-color: #7CD03B;

@btn-background: #2B6DCB; // 按钮背景色
@btn-color: @white; // 按钮文字颜色
@btn-border-radius: 3; // 按钮的圆角

@btn-disabled-opacity: 0.4;
@btn-link-background: #ECF2FA; // link按钮背景色
@btn-link-color: #1F69CD; // link按钮文字颜色

@btn-hover-border-color: #1F69CD;
@btn-active-border-color: @btn-hover-border-color;
@btn-active-background: #D6E7FF;

@btn-primary-hover-background: #001F6B; // 主按钮的hover状态背景色

```

2.3.8.2 按模块定制

按模块定制类似于整体定制，只是定制的主题文件的作用范围局限在某些特定的模块。

通过指明模块下页面 `EFPage` 的 `prefix` 属性，然后在相关的 `jsp` 中引用模块定制的主题文件。

2.3.8.3 个性化定制

针对个别特殊页面，可以直接（使用 `HTML link` 标签）在页面上引入定制的 `CSS` 文件。

3 iPlatUI 开发指南

3.1 UI-常见开发问题汇总

3.1.1 如何获取控件对象

iPlatUI 提供的控件包括两大类：

1. 基于 KendoUI 组件封装的控件；
2. iPlatUI 自定义开发的控件；

3.1.1.1 基于 KendoUI 组件封装的控件

通常基于 KendoUI 组件封装的控件，可以通过 `$("#id").data("kendoGrid")` 的形式获取。

其中 `$("#id")` 是控件对应的 DOM 元素。注意：**要在控件渲染结束后**，才能从 `data` 方法中，取得控件的实例对象。

- kendo + 控件名称

EF Tag 组件名称和 KendoUI 控件名称的对应关系表：

Tag 组件名称	KendoUI 控件名称
EFDatePicker	kendoDatePicker
EFSelect	kendoDropDownList
EFMultiSelect	kendoMultiSelect
EFAutoComplete	kendoAutoComplete
EFUpload	kendoUpload

3.1.1.2 iPlatUI 自定义开发的控件

EFInput

EFPopupInput

EFTreeInput

3.2 前端开发入门教程

3.2.1 网页三剑客

HTML	CSS	JS
定义：超文本标记语言——HyperText Markup Language，是构成Web世界的基石。	定义：层叠样式表 (Cascading Style Sheets)，是一	定义：是运行在浏览器端的编程语言 作用：可以更改HTML内容，更换CSS样式(style)，为网站添加交互功能（例如：游戏，交互式的地图，动态样式，动画等

HTML	CSS	JS
作用：用来结构化我们的网页内容和赋予内容含义，例如定义段落、标题、和数据表,或在页面中嵌入图片和视频。	种样式规则语言 作用：用于描述网页上的内容的外观或表现形式， 例如设置背景颜色和字体，在多个列中布局我们的内容。	)

3.2.2 CSS 入门教程

1. CSS 的历史
2. CSS 的规范
3. CSS 的基础知识
 - a. 选择器和优先级
 - b. 盒子模型 Box Model
 - c. display
 - d. 浮动和清除浮动
 - e. position 和 zindex
 - f. overflow
 - g. margin 和 padding
 - h. 伪元素和伪类
4. CSS 扩展知识
 - a. table 下的布局定位
 - b. 垂直居中水平居中
 - c. margin 为负值的使用场景
 - d. CSS3 的 flex 布局

3.2.3 HTML 入门教程

1. HTML 的历史
2. HTML 的规范
3. 基础知识
 - a. HTML 的标签
 - b. block 级元素和 inline 级元素

- c. 语义化的 HTML
- 4. 扩展知识
 - a. 什么是 domain
 - b. 什么是 cookie
 - c. localStorage
 - d. WebSocket

3.2.4 Javascript 教程

3.3 前端工具箱

3.3.1 Chrome 的调试技巧

3.3.1.1 入门使用介绍

[使用 Chrome 开发者工具](#)

3.3.1.2 Console 使用的指南

中文经验: <http://www.cnblogs.com/Wayou/p/chrome-console-tips-and-tricks.html>

官方文档: <https://developers.google.cn/web/tools/chrome-devtools/console/console-reference?hl=zh-cn#log>

3.3.2 EiCommunicator

EiCommunicator 封装前端 ajax 提交到 EiService 调用, 它是平台提供的一个前端全局变量。

EiCommunicator 提供三个静态的方法为开发人员调用, 分别是 send, \$send, sendFuncCall。

这里重点介绍一下 EiCommunicator.send 这个方法。

参数	描述
serviceName	String类型, 服务名,
methodName	String类型, 方法名
inInfo	EiInfo类型, 提交的EiInfo信息
callback	Object类型, 回调处理对象 {onSuccess:function(){},onFail:function(){}}
ajaxOptions	Object类型, 自定义的ajax选项, 覆盖jQuery ajax默认的设置

示例代码:

```

    /**JSP***/
    <EF:EFRegion id="inqu" title="查询条件">
        <div class="row">
            <EF:EFInput cname="产品代号" ename="inqu_status-0-product_id" />
            <EF:EFInput cname="公司名称" ename="inqu_status-0-company_name" />
        </div>
    </EF:EFRegion>

    /**JS***/
    <script>
    /* 下面的 JS 代码段用于示例如何使用 EiCommunicator 调用 Java 后台的 ServiceEEDM6000 的
    query 方法 */
    // 创建 EiInfo
    var inInfo = new EiInfo();

    // 将 id 为 inqu 的 EFRegion 中的输入框的内容作为查询条件，设置到 inInfo 中
    // inInfo 对应后台 ServiceEEDM6000#query 方法的入参 inInfo
    inInfo.setByNode("inqu");

    // 请求后台 Java 服务 ServiceEEDM6000 的 query 方法
    EiCommunicator.send("EEDM6000", "query", inInfo, {

    // 服务调用成功后的回调函数 onSuccess
    onSuccess: function(response) {
        console.log(response) // response 是 EiInfo 对象，对应后台 ServiceEEDM6000#query 方法
        的返回值 outInfo
    },

    // 服务调用失败后的回调函数 onFail
    onFail: function(errorMsg, status, e) { /* errorMsg 是平台格式化后的异常消息，
    status, e 的含义和$.ajax 的含义相同，请参考 jQuery 文档 */
        // 调用发生异常
        console.log(errorMsg);
    })
    </script>

```

3.3.3 IPLAT

旧版平台的 ef， efform 等工具类，统一更新到 IPLAT 名空间下。

3.3.3.1 EForm 工具类

函数	说明	变化
efform.showOverlay	弹出页面遮盖层	IPLAT.progress(\$("body"), true);
efform.hideOverlay	关闭页面遮盖层	IPLAT.progress(\$("body"), false);
efform.submit	该方法用户提交当前页面	
efform.onload	页面加载后的初始化处理函数	\$(window).load(function() { })

函数	说明	变化
efform.setFormTitle	如果页面没有设置标题， 置为统一的标题	设置html的head中的title，浏 览器标签上的显示内容。 IPLAT.setFormTitle
efform.setPageTitle	设置页面标题	设置JSP页面的header上的标 题内容
efform.hideFormHead	隐藏标题栏	隐藏页面上的header
efform.repaint	重绘页面的全部表单	N/A
efform.alertTime	增加运行时间和调试信息	N/A
efform.init	初始化方法	N/A
efform._resize	efform处理window.resize 的实现(for IE)	N/A
efform.setStatus	设置页面状态	N/A
efform.getAllGridIDs	获得页面上所有表格对象 的id	N/A
efform.addGrid	向页面中添加一个表格对 象	N/A
efform.getGrid	根据id在页面范围获取表 格对象	N/A
efform._removeGrid	删除页面中的一个表格对 象	N/A
efform.clearDiv	清除DIV层中所有输入域 的内容	IPLAT.clearDiv
efform.clearInputField	清除一个DOM结点中所有 输入域的内容	IPLAT.clearInputField
efform.hasErrorMessage	判断页面校验是否有错误 发生	IPLAT.hasErrorMessage
efform.showErrorMessage	显示校验错误信息	IPLAT.showErrorMessage
efform.setSelectDiv	设置下拉多选框所在DIV 的宽度.	IPLAT.setSelectDiv
efform.setErrorStyle efform.setErrorStyleSingleElement	设置元素的出错样式	IPLAT.setErrorStyle IPLAT.setErrorStyleSingleEle ment
efform.getErrorMessage	获取出错信息	IPLAT.getErrorMessage
efform.clearErrorMessage	清空校验错误信息	IPLAT.clearErrorMessage
efform.windowOnError	页面异常处理函数	IPLAT.windowOnError
efform.formRefresh	刷新页面	IPLAT.formRefresh
efform.formPrint	打印页面	IPLAT.formPrint
efform.getSubGridDiv	获得渲染子表格的DIV层 对象	N/A

函数	说明	变化
efform.getSubDiv	获得渲染子表格的DIV层对象	N/A
efform.createDivWindow	创建DIV层窗口	N/A
efform.showSubGridWindow	显示子表格DIV窗口	N/A
efform.showInputSubGridWindow	显示input框的子表格DIV窗口	N/A
efform.showTextAreaWindow	显示长文本编辑DIV窗口	N/A
efform.showSubTreeWindow	显示长文本编辑DIV窗口	N/A
efform.showPopupWindow efform.showPopupGridWindow efform.popGrid_save_onclick	显示长文本编辑DIV窗口	N/A
efform.subgrid_onmouseover	子表格鼠标移动的处理函数	N/A
efform.subgrid_onmouseout	子表格鼠标离开的处理函数	N/A
efform.textarea_save_onclick	长文本编辑窗口保存的处理函数	N/A
efform.subgrid_save_onclick	子表格窗口保存的处理函数	N/A
efform.subgrid_rebuild	重绘子表格窗口	N/A
efform.showMultiselectWindow	显示多选框编辑DIV窗口	N/A
efform.multiselect_save_onclick	多选框编辑窗口保存的处理函数	N/A
efform.evalFunction	eval函数调用	N/A
efform.setNewWindowStyle	设置页面弹出窗口的样式	N/A
efform.setGridCurrentRowStyle	设置表格当前行的样式	N/A
efform.getGridCurrentRowStyle	获取表格当前行的样式	N/A
efform.newWindow	在表格编辑页面中弹出模式窗口	N/A
efform.openNewForm	弹出新页面	N/A
efform.getParentWindow	获取父窗口	N/A
efform.openForm	打开页面	N/A
efform._getMessageBoxDiv	获取消息DIV层窗口	N/A
efform.showMessageBox	显示消息DIV层窗口	N/A
efform.hideDiv	隐藏DIV	<code>\$("#id").hide()</code>
efform.showDiv	显示DIV	<code>\$("#id").show()</code>

函数	说明	变化
efform.getSystemAjaxCallback	获取系统提供的ajax回调函数	N/A
efform.fillDiv	自动填充DIV层各输入域的内容	N/A
efform.fillNode	自动填充DOM结点各输入域的内容	N/A
efform._getValue	根据结点名称获取EiInfo中对应的值	N/A
efform.getNavigationBar	根据id获取页面中导航条对象	N/A
efform.getFormStatus	获取formhead的状态位	N/A
efform.getMsgKey	获取formhead的msgKey	N/A
efform.getMsg	获取formhead的msg	N/A
efform.getDetailMsg	获取formhead的DetailMsg	N/A
efform.setDetailMsg	设置formhead的DetailMsg	N/A
efform.EFPopupInput_onDeleteButtonClick	PopupInput控件删除按钮默认事件	N/A

3.3.3.2 IPLAT 相关 API

函数	参数	说明
1 IPLAT.isNull		复用underscore常用的数据类型判断函数
2 IPLAT.isUndefined		
3 IPLAT.isBoolean		
4 IPLAT.isNumber		
5 IPLAT.isString		
6 IPLAT.isDate		
7 IPLAT.isObject		
8 IPLAT.isEiColumn		EiInfo相关数据类型判断函数
9 IPLAT.isEiBlock		
10 IPLAT.isEiInfo		
11 IPLAT.isKendoModel		Kendo相关数据类型判断函数

	函数	参数	说明
1 2	IPLAT.isKendoGrid		
1 3	IPLAT.isEmptyString	str 目标字符串	判断字符串是否为空
1 4	IPLAT.isBlankString	str 目标字符串	判断trim后的字符串是否为空
1 5	IPLAT.isAvailable	obj 目标对象	判断某一对象是否为空
1 6	IPLAT.isBatchAvailable	{Array} array 目标对象数组 []	判断某一对象数组是否每个元素可用
1 7	IPLAT.getContextPath		获取应用上下文信息，应对相对路径，nginx 下端口分发等场景
1 8	IPLAT.getParentWindow		获取当前窗口的opener， 如果是iframe获取其top窗口（即浏览器的窗口）
1 9	IPLAT.getParameterByName	parameterName 参数名 url URL地址，默认为当前window的地址	根据参数名，获取URL中queryString的参数 值
2 0	IPLAT.htmlEncode	str 需被转换的字符串	将字符转换为HTML实体
2 1	IPLAT.htmlDecode	value 需被转换的HTML	将HTML实体转换为字符
2 2	IPLAT.progress	element jQuery对象 toggle showOverlay: true, hideOverlay: false	开关遮罩层，loading-image图标
2 3	IPLAT.trimString	str 目标字符串	截取字符串首尾的空格
2 4	IPLAT.getCurrentStrWidth	text 文本 font 字体大小	获取文本在dom中的长度
2 5	IPLAT.copyToClipboard	text 文本	把文本放入"剪贴板"
2 6	IPLAT.createGrid	{String} id DOM中元素ID名称 {EiInfo} ei 平台提供的EiInfo对象 {Object} options 设置kendoGrid的参数集合， 里面需要新增一 项blockId(用于指定前面EiInfo中block)	使用EiInfo生成原生的Grid
2	IPLAT.openForm	formEname 新页面的英文名	使用平台提供的API打开新窗口，并在当前窗

函数	参数	说明
7	paramList 输入参数 isMax 是否最大化	口维护所有打开的子窗口
2 8	IPLAT.createUrl formEname 新页面的英文名 paramList 输入参数 url 需要的URL路径	使用平台提供的API创建URL路径
2 9	IPLAT.postForm formEname 页面英文名 postEiInfo 大批量的EiInfo参数 params 页面URL的参数	打开新窗口时，POST大批量的EiInfo数据，在Service的initLoad方法中，可以取到传递的EiInfo参数
3 0	IPLAT.flat2Map {list} data list数据集合 {String} idField 行中主键(id)的属性名称,作为其他行的父Id {String} parentId 行中父id的属性名称 {String} rootLevel 首结点名称	把list格式转化为map格式,list数据集合是由一行行类似于[{id,pid},{id,pid}....]组成的
3 1	IPLAT.map2Flat {map} data map数据集合 {String} itemsName 子级属性名称（默认值为items）	把map格式转化为list格式,map数据集合是由一行行类似于[{id:"items:[{id:"items:[.....]}]}]组成的
3 2	IPLAT.fillNode {Object} node DOM结点对象 {EiInfo} eiInfo 数据	自动填充DOM结点各输入域的内容
3 3	IPLAT.clearNode {Object} node DOM结点对象	清空DOM结点各输入域内容
3 4	IPLAT.submitNode node jQuery selector字符串, jQuery对象, DOM对象, 用于指定需提交的区域 serviceName 服务名 methodName 方法名 callback 回调{onSuccess:function(){}, onFail:function({})}	批量提交当前页面所有Grid勾选的数据

3.3.4 jQuery 与 Underscore 常用 API 整理

jQuery 中 API	描述	Underscore 中 API	描述
--------------	----	------------------	----

jQuery 中 API	描述	Underscore 中 API	描述
\$.each()	\$.each([1,2,3], function(index, item){console.log(index+"."+item)})) <pre>=> 0:1 1:2 2:3</pre>	_.each()	_.each([1,2,3], function(item, index){console.log(index+"."+item)})) 结果与\$.each()示例一致，但注意function中对应参数的位置，是与之相反的
\$.map()	在数组的每一个元素或对象上应用一个函数并将结果映射到一个新的数组中 \$.map([2,4,6], function(val, i){return val/2;}) <pre>=> ▶ [1, 2, 3]</pre>	_.map()	_.map({one: 1, two: 2, three: 3}, function(num, key){ return num * 3; }) <pre>=> ▶ [3, 6, 9]</pre>
\$.extend()	将两个或更多对象的内容合并到第一个对象并返回第一个对象 \$.extend({name:"zhangsan"}, {name: "lisi", age:20},{age:30, tel: "123"}) <pre>=> ▼ Object {name: "lisi", age: 30, tel: "123"}</pre>	_.extend()	extend方法中参数为数组时，后一个数组将覆盖前一个数组前n位(n为后一个数组长度)，如n>=前数组长度则全部覆盖 _.extend([1,2,3], [10,20,30,40]) => <pre>▶ [10, 20, 30, 40]</pre> _.extend([1,2,3], [10,20]) => <pre>▶ [10, 20, 3]</pre> 注：目标对象（第一个参数）将被修改。如果我们想保留原对象，我们可以通过传递一个空对象作为目标对象，如_.extend({},[1,2,3], [10,20])
\$.isArray()	判断一个对象是否为数组 \$.isArray([1,2,3]) => true \$.isArray({name:"zhangsan", age:"lisi"}) => false	_.isArray()	
\$.isNumeric()	判断一个对象是否是数字	_.isNumber()	
		_.keys()	检索object拥有的所有可枚举属性的名称,返回一个数组 _.keys({name:"zhangsan", age:20, tel:"123"}) <pre>=> ▶ ["name", "age", "tel"]</pre>
		_.find()	返回第一个通过predicate迭代函数真值检测的元素值

jQuery 中 API	描述	Underscore 中 API	描述
			<code>_.find([{{name:"zhangsan", age:20, tel:"123"}}, {{name:"lisi", age:20}}, function(item){return item.age=20})</code> <code>=> ▶ Object {name: "zhangsan", age: 20, tel: "123"}</code>
		<code>_.filter</code>	返回包含所有通过 predicate 真值检测的元素值 <code>_.filter([{{name:"zhangsan", age:20, tel:"123"}}, {{name:"lisi", age:20}}, function(item){return item.age=20})</code> <code>=> [{{name:"zhangsan", age:20, tel:"123"}}, {{name:"lisi", age:20}}]</code>

3.3.5 JS 中字符串转数字运算规则

JS 中的字符串转换成数字，主要有三个方法：转换函数、强制类型转换、利用 js 变量弱类型转换。

1. 转换函数

即 `parseInt()` 和 `parseFloat()` 两个转换函数，前者将 `String` 类型数据转换成整数，后者转成浮点数。

`parseInt()` 方法可将二进制、八进制、十六进制或其他任何进制的字符串转换成整数，并且可以由第二个参数指定目标字符串的基。

例如：

```
parseInt("011", 8) => 9
```

```
parseInt("011", 16) => 17
```

```
parseInt("0x11") => 17 //以"0x"开头代表 16 进制
```

`parseFloat()` 方法中参数字符串必须是以十进制形式表示的浮点数

例如：

```
parseFloat("0xA") => NaN
```

```
parseFloat("10.23") => 10.23
```

2. 强制类型转换

强制类型转换可以访问特定的值，即使它是另一种类型的。其中 `Number()` 方法可以将给定的值转换成数字(整数或浮点数)

例如：

```
Number(" 010") => 10 //字符串首尾若有空格不影响运算
```

```
Number("0x010") => 16
```

3. 利用 js 变量弱类型转换

js 变量的弱类型指赋给变量的值是什么类型它就是什么类型。

例如：

```
var a="12.34"-0 => a=12.34
```

```
var b=" 0x12"*1 => b=18
```

```
var c=" 012"*1 => c=12 //0 开头数字字符串会被解析成十进制
```

```
var c2=012 => c2=10 //0 开头数字指八进制数
```

以上利用 js 的弱类型的特点进行算术运算，实现了字符串到数字的类型转换。

3.3.6 Kendo Format 格式化输出

这次来探讨一下 `kendo.format` 与 `kendo.toString` 这两个方法，为以后我们格式化输出一些日期，数据，文字提供了方便，下面是我的一些探究。

在 `kendo` 的实现中，`format` 调用了 `toString` 这个方法去实现格式化输出的。



上面是截取的部分代码，大家可以看出 `format` 是调用了 `toString`。那么 `toString` 是一个什么样的方法呢，`toString` 后面的实现代码层次比较复杂，我这里不详细叙述，大家感兴趣的可以自己看源码，这里只需要大家明白这层关系就可以。

我们来看一下 `toString` 这个方法。



这个是官方给出的解释，什么意思？

意思就是：在当前的文化下，提供的一个很 **数字** 与 **时间** 的格式化输出方法。我的理解就是当你设置的文化(`culture->zh-CN`) 是中国，那么输出的格式就是中国化的，你设置的文化是美国的，你输出的内容格式就是美国化的

举例如下：

你设置的文化(`culture->zh-CN`) 是中国 `kendo.toString(200,"c") => ￥200`。
当然你的文化是美国(`culture->en-US`) 那么 `kendo.toString(200,'c') => $200`

我们接着俩看 `kendo.toString` 这个方法，下面我们把 `kendo.toString` 简称为 `toString`。

`toString` 这个方法有三个参数(`value, fmt, culture`)，分别是 `value,fmt,culture`，`value` 就是你需要格式化的内容，`fmt` 是格式化输出的参数，`cultrue` 就是你指定的文化。下面给大家讲一下这三个参数。

`value` 这个只对 `number` 和 `date` 这两种类型起作用，其余类型原样输出，**这点需要注意**。而且 `number` 后面的 `fmt` 参数与 `date` 后面 `fmt` 参数是不一，至于 `toString` 后面 `fmt` 参数有哪些，`kendo` 官网上有详细的 demo：

<http://docs.telerik.com/kendo-ui/api/javascript/kendo#methods-toString>，大家可以去官网上了解和学习。还有一个参数就是 `culture`。`culture` 有两种方式可以实

现，一种是你调用 `toString` 时候作为参数传入。还有一个就是你使用 `kendo.culture` 设置全局的 `culture`。

说完 `kendo.toString` 我们来看一下 `kendo.format` 这个方法

`kendo.format` 一个功能很强大的针对字符串的格式化输出方法。下面我们把 `kendo.format` 简称为 `format`。

先从一个例子入手。



我们发现这个方法跟我们 JAVA 中的 `String.format` 差不多,使用也是差不多的,这里通过 `{Number}` 占位,`number` 必须从 0 开始 , 往后逐 1 增加。后面参数开始逐个替换。当然了, 这里也能够格式化输出一些内容。使用形式 `{number:fmt}` 这个 `fmt` 也就是 `toString` 方法的 `fmt`。

3.3.7 Kendo Grid 常用的 API

KendoGrid 常用的 API

简介

[官方文档](<http://docs.telerik.com/kendo-ui/api/javascript/ui/grid>)

KendoGrid 的方法

1. `addRow` 添加一行数据，默认在表格上方，会触发 `dataBound` 事件。
2. `autoFitColumn` 自适应列宽度，避免长的文字出现省略。类似平台以前双击列名自适应的功能，参数是列号 (Number) 或者 field 名 (String)
`autoFitColumn(1)` // 第一列，列从 1 开始计数
`autoFitColumn("form_ename")` // 列名
3. `cancelChanges` 取消 Grid 上的所有修改，会触发 `dataBound` 事件。
4. `cancelRow` 取消正在编辑状态的行 (有 bug)，新增行也是在编辑状态。不建议使用。
5. `cellIndex` 根据单元个 `td` 元素，判断其所处的列号。常用场景是，编辑某个单元格时，获取列号信息。
6. `clearSelection` 取消选中行。(使用 `select` 接口选中的行，使用 `checkbox` 自定义勾选的行，**不会取消**`checkbox` 选中状态，但是，再使用 `select` 接口，无法获取 `checkbox` 的选中行)

默认的多选操作是 `Ctrl+Click`，平台定制成为加 `checkbox` 处理。

7. `closeCell` 关闭正在编辑的单元格。
8. `collapseGroup` 折叠分组 `expandGroup` 展开分组。
9. `collapseRow` 折叠行明细 `expandRow` 展开行明细。
10. `current` 获取或者设置 选中的单元格 Gets or sets the current cell for keyboard navigation.
11. `dataItem` 根据 `tr` 元素，获取 Model 数据，Model 是可以双向绑定的

Observable 对象, `Model.toJSON()` 可以获取 JSON 对象。

12. `destroy` 去除 Grid 上的所有事件和绑定的数据, 完全作为一个静态的 DOM Table 展示。
 13. `editCell inline` 编辑模式时, 编辑单元格。
 14. `editRow inline` 编辑模式时, 编辑行。
 15. `getOptions` 获取 Grid 的 options 信息。
 16. `hideColumn` 通过设置 Column 的宽度为 0, 隐藏列。 参考 `autoFitColumn`。
 17. `items` 获取当前页面上的 tr 数组, 类似 `dataSource` 的 `view`。
 18. `lockColumn` 设置固定列, 必须要存在 `locked` 的列。
 19. `refresh` 静态刷新 grid 的数据, 不做后端提交, 会触发 `dataBound` 事件。
 20. `removeRow` 删除一行 tr 数据, `dataSource` 也会删除。需要 `sync` 才会向后台提交。
 21. `reorderColumn` 交换两列的顺序。
`grid.reorderColumn(1, grid.columns[0]);`
 22. `saveAsExcel` 前端导出 excel 文件, 需要 `jszip` 库。
 23. `saveAsPDF` 前端导出 PDF 文件。
 24. `saveChanges` 调用 `dataSource` 的 `sync` 方法 提交更改。
 25. `saveRow` 向后台提交处在编辑状态的行的数据。
 26. `select` 获取或者设置选中的行。
`grid.select("tr:eq(1), tr:eq(2)");` // 选中第一行和第二行
 27. `setDataSource` 设置 Grid 的数据源, 会刷新 Grid 的数据。使用 `EFGrid` tag 时, 不能使用此方法。
 28. `setOptions` 设置 grid 的 options, 会重写绘制 Grid。几个注意点, 不能在 Grid 事件中使用。
 29. `showColumn` 通过设置宽度, 显示列。参考 `hideColumn`。
 30. `unlockColumn` 取消固定列。
- ## Datasource 的方法。

3.3.8 Kendo Template 使用介绍

`kendo template` 是 `kendo` 提供的一个前端模版处理方法。官方的 API 地址: <http://docs.telerik.com/kendo-ui/api/javascript/kendo#methods-template>

3.3.8.1 从简单的例子开始

```
<span id="output"></span>
<script>
    var template = kendo.template("Hello, #= firstName # #= lastName #");
    var data = { firstName: "John", lastName: "Doe" };
    $("#output").html(template(data)); // => Hello, John Doe
</script>
```


`kendo.template` 接收一个模版 "Hello , `#= firstName#` `#= lastName`", 返回一个 `template` 函数。`template` 函数接收一个 `data` 对象, `data` 对象中的值替换掉模版参数中`#= #`所标识的值, 并且把结果返回。



这种模版处理方式被称为**预处理**, 编译后生成的 `template` 函数能够在其他地方运行, 提高了处理效率。

3.3.8.2 掌握模版语法

'`#`'符号是 `kendo.template` 标识符号, 模版字符串中出现'`#`'时, 需要`\#` 转义; (例如: `<a href="#" />` 在模版中需要写为`<a href="\#" />`)

- (1) `#= field #` 表达式 计算 `data` 中的 `field` 的值; (对比 JSP 理解 `<%= %>`)
- (2) `#: filed #` 表达式 计算 `data` 中的 `field` 的值时, 转义 HTML 字符;
- (3) `##` 语句 使用 JS 进行逻辑处理 (`if, else, for ....`) (对比 JSP 理解 `<% %>`)

```
<span id="output"></span>
<script>
var template = kendo.template("#if (foo) {# foo is true #}#");
var data = { foo: true };
$("#output").html(template(data));
</script>
```

`#:` 输出为转义后的 HTML 字符片段

```
<span id="output"></span>
<script>
var template = kendo.template("HTML tags are encoded: #: html #");
var data = { html: "<strong>hello</strong>" };
$("#output").html(template(data)); // &lt;strong&gt;hello&lt;/strong&gt;
</script>
```

`#= #` 输出为字符串

是否使用 **WithBlock**

在严格模式下是不能够使用 `with` 的关键字的, 至于 `with` 的问题, 大家自行百度。`kendo` 提供了支持 `with` 与不支持 `with` 两种方式, 上面提供的 `demo` 都是在 `with` 条件进行的, 大家根据自己的实际情况选择是否使用 **With** 模式。

下面给大家展示一下在非 `with` 下面如何使用。

在非 **With** 模式下面需要大家给 `##` 模版中参数指定一个参数头。

```
var template = kendo.template("<strong>#: d.name #</strong>", { paramName: "d",
  useWithBlock: false });
```


上面就需要在模版中定义好 `#: d.name #` 其中 `d` 就是定义的参数头，默认的参数头是 `data`，也就是上面可以修改为：

```
var template = kendo.template("<strong>#: data.name #</strong>", { useWithBlock: false });
```

`kendo.template` 函数中 `useWithBlock : false` 在表明是在 非 With 模式。在通常情况下，`useWithBlock` 默认值是 `true`，也就是 With 模式。

3.3.9 KendoTreeView

简介

[官方文档] <http://docs.telerik.com/kendo-ui/api/javascript/ui/treeview>

常见属性

1. `autoBind` 初始化时是否自动绑定到数据源，默认 `true`。当设置为 `false` 时，可以保证共享 `dataSource` 不会被绑定
2. `autoScroll` 是否自动收起容器元素，默认 `false`。当设为 `true` 时，在鼠标靠近树顶部的时候会自动收起容器
3. `checkBox` 是否在每个节点前添加一个复选框，默认 `false`。当赋值 `true` 或者一个 json 对象(可以设置 `checkBox` 的 `checkChildren`, `name`, `template` 等)时会在节点前追加复选框
4. `dataSource` 配置的数据源，可以是一个 js 对象或数组，也可以是一个 `kendo.data.HierarchicalDataSource` 实例
(可参考 <http://docs.telerik.com/kendo-ui/api/javascript/ui/treeview#configuration-dataSource> 官方示例)
5. `dataSpriteCssClassField` 设置给节点添加图标样式的 field 名，默认配置为 `null`
6. `dataTextField` 用于配置节点显示内容的 field 名
7. `dragAndDrop` 节点是否可拖拽配置，默认 `false` 不可拖拽
8. `template` 用于设置节点显示内容的模板，可以是一个 `String` 类型字符串或一个 `function`

常用 API

1. `append` 向 `TreeView` 对象添加一个节点，此方法也常被用来对节点进行重新排序
2. `collapse` 收起对应节点以下的所有子节点 `expand` 展开对应节点以下的所有子节点 `toggle` 改变节点的收起展开状态
3. `expandPath` 展开 `id` 包含在参数数组中的所有节点
4. `expandTo` 参数可以是某个节点的 `id`，或者某个节点对象，`expandTo` 方法可以使树展开到参数中对应的节点

5. **findByText** 根据 **dataSource** 中各节点的 **text** 值来定位节点，当多个节点具有此 **text** 可返回多个节点
6. **findById** 根据节点唯一标识 **id** 来定位，只返回一个节点对象
7. **insertAfter** 用于在一个节点之后插入另一个节点，两个参数为待插入的节点对象和树中某个节点，**insertBefore** 用于将节点插入到另一节点之前
8. **items** 返回所有树节点元素的数组 **dataItem** 可以定位到某一个节点对象，实现对单个节点的数据操作
9. **remove** 删除一个节点
10. **setDataSource** 设置数据源，不会改变 **loadOnDemand**(子节点是否懒加载)的值
11. **parent** 参数为树中某一节点对象，返回节点的父节点
12. **destroy** 销毁一棵树，但此方法不会消除树中的部件元素(只销毁了树的结构)
13. **enable** 使树节点恢复可用，当 **enable=false** 时节点不可操作

常见事件

1. **select** 节点被选中时触发
2. **change** 当被选节点改变时触发
3. **expand && collapse** 节点展开/收起之前触发
4. **check** 当节点前带有 **checkbox**，在勾选中或取消选中之后触发
5. **dragstart / drag / dragend && drop** 节点被拖拽前/中/后 和 被放下时触发
6. **dataBound** 数据源 **dataSource** 改变之后触发，如可使用此事件在加载完 **dataSource** 之后再追加一个节点

场景示例

1.checkbox 复选框

Code Block 1 JSP 示例

```
<div id="example">
  <div class="demo-section k-content">
    <div>
      <h4>带复选框的树</h4>
      <div id="treeview"></div>
    </div>
    <div style="padding-top: 2em;">
      <h4>被选中的节点</h4>
      <p id="result">无节点被选中</p>
    </div>
  </div>

  <script>
    $("#treeview").kendoTreeView({
      checkboxes: {
        checkChildren: true
      }
    });
  </script>
</div>
```

```

    },

    check: onCheck,

    dataSource: [{
        id: 1, text: "My Documents", expanded: true, spriteCssClass:
"rootfolder", items: [
            {
                id: 2, text: "Kendo UI Project", expanded: true,
spriteCssClass: "folder", items: [
                    { id: 3, text: "about.html", spriteCssClass: "html" },
                    { id: 4, text: "index.html", spriteCssClass: "html" },
                    { id: 5, text: "logo.png", spriteCssClass: "image" }
                ]
            },
            {
                id: 6, text: "New Web Site", expanded: true, spriteCssClass:
"folder", items: [
                    { id: 7, text: "mockup.jpg", spriteCssClass: "image" },
                    { id: 8, text: "Research.pdf", spriteCssClass: "pdf" },
                ]
            }
        ]
    }
    ]
});

// 获得被勾选节点的所有 id
function checkedNodeIds(nodes, checkedNodes) {
    for (var i = 0; i < nodes.length; i++) {
        if (nodes[i].checked) {
            checkedNodes.push(nodes[i].id);
        }

        if (nodes[i].hasChildren) {
            checkedNodeIds(nodes[i].children.view(), checkedNodes);
        }
    }
}

// 绑定 onCheck 事件，显示出所有被勾选节点的 id
function onCheck() {
    var checkedNodes = [],
        treeView = $("#treeview").data("kendoTreeView"),
        message;
    checkedNodeIds(treeView.dataSource.view(), checkedNodes);

    if (checkedNodes.length > 0) {
        message = "被勾选节点 id: " + checkedNodes.join(",");
    } else {
        message = "无节点被选中";
    }
    $("#result").html(message);
}
</script>

<style>
    #treeview.k-sprite {

```

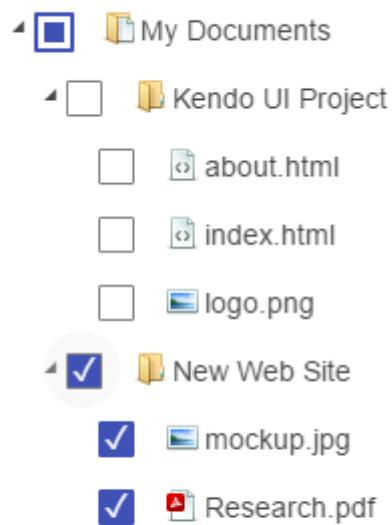
```

        background-image: url("../content/web/treeview/coloricons-sprite.png");
    }
    .rootfolder { background-position: 0 0; }
    .folder      { background-position: 0 -16px; }
    .pdf         { background-position: 0 -32px; }
    .html        { background-position: 0 -48px; }
    .image       { background-position: 0 -64px; }
</style>
</div>

```

页面效果:

带复选框的树



被选中的节点

被勾选节点id: 6,7,8

2. 添加树节点

JSP 示例

```

<div id="treeview"></div>
<script>
$("#treeview").kendoTreeView({
    dataSource: [
        { text: "root" }
    ]
});

var treeview = $("#treeview").data("kendoTreeView");
// 添加一个新节点到树根目录下
treeview.append({ text: "node0" });

```

```
// 添加一个新节点到根目录，并指定为第一个节点
treeview.append({ text: "node1" }, $("#treeview .k-item:first"));

// 在某个节点之前增加一个新节点
treeview.insertBefore({ text: "nodeBefore" }, treeview.findByText("node1"));

// 在某个节点之后增加一个新节点
treeview.insertAfter({ text: "nodeAfter" }, treeview.findByText("node1"));

// 添加一个新节点(node2)到指定节点下(node1)
treeview.append({ text: "node2" }, treeview.findByText("node1"));

// 添加多个节点到根目录下
treeview.append([
    { text: "node3" },
    { text: "node4" }
]);

</script>
```

页面效果：

```
└─ root
    └─ nodeBefore
        └─ node1
            └─ node2
                └─ nodeAfter
                    └─ node0
                        └─ node3
                            └─ node4
```

3. 设置新数据源

JSP 示例

```
<div id="treeview"></div>
<script>
$("#treeview").kendoTreeView({
    dataSource: [
        { text: "root", items: [
            { text: "node0" }
        ] }
    ]
});

var treeview = $("#treeview").data("kendoTreeView");
treeview.setDataSource(new kendo.data.HierarchicalDataSource({
    data: [
        { text: "rootNew", items: [
            { text: "node0New" },

```

```

        { text: "node1" }
      ] }
    ]
  }));
</script>

```

使用 `setDataSource` 方法可以覆盖一棵树原有 `dataSource`，页面效果如下：

```

└─ rootNew
   └─ node0New
      └─ node1

```

4. 设置树展开到某一节点

JSP 示例

```

<div id="treeview"></div>
<script>
$("#treeview").kendoTreeView({
  dataSource: [
    { id: 1, text: "node1", items: [
      { id: 2, text: "node2", items: [
        { id: 3, text: "node3" }
      ] }
    ] },
    { text: "one", items: [
      { text: "two", items: [
        { text: "three" }
      ] }
    ] }
  ]
});

var treeview = $("#treeview").data("kendoTreeView");

// 展开节点 id=3 的节点以上的全部节点
treeview.expandTo(3);

// 展开节点 text 值为"three" 的节点以上的全部节点
var three = treeview.dataItem(treeview.findByText("three"));
treeview.expandTo(three);

```

页面效果：

```

└─ node1
  └─ node2
    └─ node3
└─ one
  └─ two
    └─ three

```

3.3.10 WindowUtil 使用说明

WindowUtil 是平台提供给开发人员用于提示信息的弹出窗口。

参数	描述	默认值
title	标题名称	""
content	内容(可包含HTML)	""
modal	是否展示遮罩	true
width	content的宽度	
height	content的高度	

3.3.10.1 JS 开发

示例代码

```

<script>
.....
WindowUtil({
  title:"iPlatV6",
  content:"欢迎使用 iPlatV6",
  width: 300,
  height: 200
});
.....
</script>

```

3.3.10.2 页面效果如下所示



3.3.11 如何使用 spreadsheet

平台已经集成了 spreadsheet 的功能，调用方式如下：

spreadsheet

```
$("#spreadsheet").iplatSpreadsheet(options)
```

就可以调用 spreadsheet 的功能。iplatSpreadsheet 是对 kendoSpreadsheet 方法的封装，想了解 options，以及更好的使用 spreadsheet。请参考 <https://docs.telerik.com/kendo-ui/api/javascript/ui/spreadsheet>

3.3.12 页面与 Iframe 数据交互

□ 本文只是针对“同域”中页面和 iframe 的数据交互，如果存在跨域，则不在此讨论范围内。所以请开发人员首先判断页面中 iframe 与页面是否处于“同域”中。

在页面中存在 iframe，并且该 iframe 和此页面处于同域中时候。我们称 iframe 页面为子页面，此页面为父页面。

这里给父子页面定义一个规范，让他们之间能够相互操作。

我们给父页面引入 Message.js 这个 JS 文件。如下图：

```
<script type="text/javascript" src="{ctx}/iplatui/js/lib/underscore.min.js">
<script type="text/javascript" src="{ctx}/kendoui/js/kendo.all.min.js"></script>
<script src="{ctx}/iplatui/js/iplat.ui.min.js"></script>
<link rel="stylesheet" type="text/css" media="screen" href="{ctx}/bpm/Workfl
<script type="text/javascript" src="{ctx}/bpm/WorkflowDesigner/js/layout/jqu
<script type="text/javascript" src="{ctx}/bpm/Message.js"></script>
<script type="text/javascript" src="{ctx}/EW/PE/EWPE00.js"></script>
<script type="text/javascript">
```

当添加完这个 JS 后，会给父页面添加一个全局 Message 对象。

父页面通过监听该对象，处理子页面发送过来的消息，子页面可以通过该对象发送消息给父页面。

举例如下：

child1.jsp

```

<html>
.....
<body>
.....
<script>
.....
    // 给父页面发送信息，前提是父页面中的 iframe 页面
    if (parent && parent.Message) {
        parent.Message.send("child1", "loadComplete");
    }
</script>
</body>
</html>

```

child2.jsp

```

<html>
.....
<body>
    <div>
        <button id="save">确定</button>
    </div>
.....
<script>
.....
    $("#save").on("click", function(e) {
        // {a:1} 为测试数据
        if (parent && parent.Message) {
            parent.Message.send("child2", "save", {a:1});
        }
    })
    // 给父页面发送信息
</script>
</body>
</html>

```

父页面(main.jsp)

```

<html>
<head>
.....
    <script type="text/javascript" src="Message.js"></script>
</head>
<body>
.....
    <iframe src="child1.jsp" />
    <iframe src="child2.jsp" />
.....

<script>
Message.on("child1", "loadComplete", function(param) {
    console.log(param); // param is undefined
});

```

```

Message.on("child2", "save", function(param) {
    console.log(param); // param is {a:1}
});
</script>
</body>
</html>

```

开发人员需要注意，给父页面发送信息之前，需要判断父页面是否存在和需要确定父页面中是否存在 **Message** 对象。因为子页面是没有办法确定它自身是否处于 **iframe** 中。

3.4 UI 组件库

iPlat4J V6 封装的 **Tag** 库，基于 **JSP Tag 2.0** 的 **SimpleTag** 机制，最大程度上兼容 **3.9** 和 **V5** 的原有 **Tag** 配置和功能。针对原有设计不足的地方，也做了相应的改进。具体表现在以下几个方面：

1. 统一控件的属性配置

EiColumn 的配置是个经典的例子：

在 **Java** 代码中有 **EiColumn.java** 文件，定义了相关的属性，例如：**name**，**descName**，用来表示字段的名称和显示名称；

在 **EFColumn Tag** 中，对应的属性的名称是 **ename**，**cname**，用来表示表格中的列的字段名称和显示名称；【**iPlat4J V6 的 Tag 支持动态属性配置：04 Tag 中如何使用 attributes 动态属性**】

在 **iPlat4J V6** 之前的版本，没有在 **JavaScript** 中配置组件的属性的机制，**iPlat4J V6** 新增了在 **JS** 中配置组件属性的机制，相对于 **Tag** 标签封装的属性，**JS** 的原生配置更为灵活有效；

在 **JavaScript** 中，列对应的属性名是 **field**，**title**。

因此，存在一个属性名映射的问题。例如，**EFColumn Tag** 应该新提供 **field** 和 **title** 属性，废弃原有的 **ename** 和 **cname** 属性【当前为了兼容考虑，会保留 **ename** 和 **cname**，但优先读取 **field** 和 **title** 新属性】

现在统一配置的规则：

- 配置的优先级：**JS 的配置 > EF Tag 中的配置 > Java 代码中的配置**
- 常用的配置，推荐在 **EF Tag** 中直接配置；
- **EF Tag** 无法满足的情况下，再使用 **JS**，不能滥用 **JS** 配置；
- 极度不建议在 **Java** 代码中配置属性值；

3.4.1 UI 组件

布局容器

- [EFPage](#)
- [EFRegion](#)
- [EFTab](#)
- [EFWindow](#)

Form表单

- [EFInput](#)
- [EFPopupInput](#)
- [EFTreeInput](#)
- [EFDatePicker](#)
- [EFDateSpan](#)
- [EFSelect](#)
- [EFMultiSelect](#)
- [EFCascadeSelect](#)
- [EFAutoComplete](#)
- [EFUpload](#)
- [EFComboInput](#)

数据表格

- [EFGrid](#)
- [EFColumn](#)
- [EFComboColumn](#)
- [EFMultiSelectColumn](#)
- [EFPopupColumn](#)
- [EFTreeColumn](#)

树和菜单

- [EFTree](#)
- [EFOnceTree](#)
- [EFSwap](#)
- [EFTreeList](#)

3.4.2 1. 布局容器

布局容器主要包括：EFPAGE，EFRegion，EFTab，EFWindow 等用于定义页面结构，定义页面区域块的标签。开发者需要将具体的页面元素填充到这些标签中。

3.4.2.1 EFPAGE

概要说明

- 标签定义：EF:EFPAGE
- 标签作用：提供一个平台 JSP 页面模版，用于快速创建一个符合平台开发规范的 JSP 页面，不再需要开发者考虑如何向页面中引入的 JS，CSS 等公共的页面元素。

属性

参考 [EFPAGE 页面标签简介](#)

属性名	说明	默认值	示例页面号
title	对应HTML中的title属性，用于设置浏览器窗口上显示的标题	EDFA00中注册的画面中文名	EEDM8500
prefix	页面前缀，用于设置项目自定义的内容，参考： UI常见开发问题FAQ-Q3	""	EEDM8501
header	(不常用) 通过JSP的fragment机制，用于自定义页面head的内容	""	EEDM8500
footer	(不常用) 通过JSP的fragment机制，用于自定义页面foot的内容	""	EEDM8500

3.4.2.2 EFRegion

概要说明

- 标签定义：EF:EFRegion
- 标签说明：EFRegion 是统一信息系统平台提供的主要控件之一，用于界面的整体布局及界面内的元素组织。

EFRegion 支持折叠/展开，通过点击区域头的折叠/展开按钮可以进行折叠/展开操作。 

EFRegion 中还可包含其他图标按钮(例如清空按钮, 用以清空 Region 内的 Input 字段), 用户可根据需求选用。 

属性

参考: [Tag 中如何使用 attributes 动态属性](#)

属性名	说明	默认值	示例页面号
id	区域的Id, 在EDFA01中注册区域按钮时使用, 推荐设置	""	EEDM8400
title	标题名称	""	EEDM8400
head	是否显示标题栏, 配置head="hidden"不显示	默认显示标题栏	EEDM8400
fitHeight	当fitHeight="true", 区域高度适应到屏幕底部	默认高度不自适应	EEDM8401
showClear	是否显示清除按钮	默认不显示	EEDM8400
autoDrawButton	是否自动绘制按钮, 默认为true	true	EEDM8400
Tag中如何使用 attributes 动态属性	可选参数, 动态配置的属性 (如: class="block-opt-hidden"折叠region内容)		EEDM8400

3.4.2.3 EFTab

概要说明

- 标签定义: EF:EFTab
- 标签说明: 多页控件, 可切换查看各个页的内容, 用于处理页面内容较多时, 需要分层次展示场景。

属性

属性名	说明	默认值	示例页面号
id	tab的id		EEDM8000
tabPosition	tab标签title的位置, 可选 (top,left,bottom,right)	"top"	EEDM8008
showClose	是否显示关闭标签按钮, 默认不显示	false	EEDM8000
content	可设置为"src",此时需在包含title的div中添加data-src属性, tab标签的content会加载对应的src	"normal"	EEDM8008
active	初始化加载时选中第几个标签, 默认选中第1个	0	EEDM8000
contentType	加载内容的类型, 若设置contentType="iframe"则会延迟加载iframe页面		EEDM8001

属性名	说明	默认值	示例页面号
state	是否 动态加载 。默认false，若设置为state="true"则会在切换标签页时重新加载对应的src内容 (content设置为src时有效)	false	EEDM8008

API

API 名	参数	返回值	示例页面号
select	index	无	EEDM8000
activateTab	tab标签元素	无	
deactivateTab	tab标签元素	无	
remove	index	无	EEDM8000
contentHolder	index	tab标签内容DOM元素	
insertBefore	tab实例对象	无	EEDM8000
insertAfter	tab实例对象	无	EEDM8000
append	tab实例对象	无	EEDM8000
enable	(tab_Strip.select(), boolean)	无	EEDM8000

延迟加载说明

在开发中，有些开发人员会使用 **EFTab** 去 **iframe** 所包含的页面，当 **tab** 页面比较多，而且里面含有比较多的 **iframe** 页面时候，首次加载会比较慢。

这里平台给这种特殊的情形提供了处理方案 [EEDM8001](#)

3.4.2.4 EFWindow

概要说明

- 标签定义：EF:EFWindow
- 标签说明：弹出窗口组件，包括模态

属性

属性名	说明	默认值	示例页面号
id	Id,必须参数		
url	目标url		EEDM8010
width	window窗口的宽度（可百分比或px）	80%	
height	window窗口的高度（可百分比或px）	80%	
top	window的位置，与顶部之间的距离（可百分比或px）	10%	
left	window的位置，与左侧之间的距离（可百分比或px）	10%	

3.4.3 2. 表单组件

表单组件主要用于定义页面的输入元素，例如 `input`，`select` 等，可以和平台的 `EiInfo` 数据进行绑定。

3.4.3.1 EFInput

3.4.3.1.1 概要说明

- 标签定义：EF:EFInput
- 标签作用：用于渲染生成页面上的表单输入元素，例如 `input`、`checkbox`、`radio`、`textarea` 等；还能对渲染生成的表单输入元素提供 `EiInfo` 数据的绑定、输入内容校验、格式化等功能。

3.4.3.1.2 属性

属性名称	说明	默认值	示例页面号	备注
bindId	数据字段绑定Id			已废弃
blockId	三段式中Eiblock的名称(不推荐)			不推荐
row	三段式中的行号。使用数字字符，表示第几行(不推荐)		EEDM4000	不推荐
ename	两种形式。1.兼容老方式: 三段中的列名称(不推荐)。2.新方式: 三段式形式的连接字符串(推荐)。例如: <code>inqu_status-0-ename</code> , <code>result-1-cname</code>		EEDM4000	
cname	组件的标签，用于说明该组件		EEDM4000	
required	渲染出来的表单输入元素必须要有内容	false	EEDM4001	需要配合JS一块使用，否则不做校验，详情参考 EEDM4001
readonly	渲染出来的表单输入元素不能进行编辑，但是依然会参与校验	false		
disabled	渲染出来的表单输入元素会被禁用	false		
value	字段默认值		EEDM4000	若配置了value，value值优先；当value值没有配置时，默认使用页面initLoad中的EiInfo对应的值  checkbox 应该手动设置value，而不能绑定EiInfo中的value，否则checkbox组都会被绑定相同的EiInfo

属性名称	说明	默认值	示例页面号	备注
				值
type	元素的类型 text/radio/checkbox/textarea/hidden		EEDM4003	
ratio	input框的栅格比例（默认4:8）	4:8	EEDM4000	
colWidth	input控件的栅格宽度	4	EEDM4000	
inline	判断输入框是表单字段还是单独使用	false	EEDM4000	
format	对数字进行格式化		EEDM4002	当type属性存在的时候，format失效。
Tag中如何使用 attributes 动态属性	动态设置的属性，类似以前的etc属性 例如： data-keep="true" 可以使EFInput在EFRegion clearNode（清理按钮被点击）后，仍保留该输入框的数据。			

3.4.3.1.3 API

序号	方法名	参数	返回值	说明	示例页面号
1	IPLAT.EFInput.value(\$(element))	{String jQuery} element EFInput的id, DOM结点jQuery对象	string	获取数据	EEDM4807
2	IPLAT.EFInput.value(\$(element) , data)	{String jQuery} element EFInput的id, DOM结点jQuery对象 data String 设置的数据		设置数据	
3	IPLAT.EFInput.enable(\$(element) , state)	{String jQuery} element EFInput的id, DOM结点jQuery对象 {Boolean} state EFInput的可用状态		state: true 设置输入框可用 false 设置输入框禁用	
4	IPLAT.EFInput.readonly(\$(element) , state)	{String jQuery} element EFInput的id, DOM结点jQuery对象 {Boolean} state EFInput的可用状态		state: true 设置输入	

序号	方法名	参数	返回值	说明	示例页面号
				框只读 false 设置输入框非只读	

3.4.3.1.4 EForm 校验

3.4.3.1.4.1 注意事项

[HTML5 Form 属性](#)内置的校验规则的局限：

1. 浏览器的兼容性，旧版本的浏览器不支持；
2. 校验的提示信息样式无法自由调整；
3. 校验的类型不够丰富；

所以 EForm 完全使用 iPlatUI 框架的校验器(Validator)，目前不建议【禁止】开发中使用 HTML5 的 Form 验证。

1. 如何使用内置校验
2. 如何使用正则校验
3. 如何设置校验提示信息
4. 如何进行多重校验
5. 如何使用分组校验

3.4.3.1.4.2 错误提示信息样式调整

请参考：[UI-常见开发问题 FAQ](#) 中 Q11。

3.4.3.1.4.3 iPlatUI 平台校验机制

平台提供的前台校验方式支持 form 域，div 域的校验，和分组校验。用法简单，只需在对应的输入框配置相应的校验方式则可，可配合平台提供的 EForm 标签使用，也可单独使用。

校验分为两种：内置类别 和 正则验证。

[EEDM4001-EForm 校验](#)

3.4.3.1.4.3.1 内置校验

通过给 EForm 设置内置的校验规则，平台会根据规则对数据进行校验。对不符合的规则的数据进行拦截，并且给出提示信息。

给指定的 EForm 进行内置验证时，需要配置 data-rules，例如：data-rules="mobile_phone" placeholder="请输入手机号"。data-rules 对应规则名，placeholder 对应该规则的提示信息。需添加相应的 js 使校验生效。

校验规则名 (rules)	规则含义	校验提示信息
email	email地址，可指定最大最小长度。	不是有效的电子邮件地址
mobile_phone	11位数字，需要以13或15开头	不是有效的手机号码
postcode	6位数字	不是有效的邮政编码
phone_with_area_code	区号3位或4位数字，之后为分隔线"- "，电话号码为3-11位的数字，最后可跟分机号，电话号码和分机号之间也需要有"- "分隔，可指定最大最小长度。(如021-50801155-1004)	带区号和分机号的电话号码，格式应如：021-50801155-1004
phone_without_area_code	3-11位的数字，需以非零开头	电话号码应为3-11位的数字，需以非零开头
integer	整数，可指定最大最小长度。	必须输入整数
positive_integer	正整数，可指定最大最小长度。	必须输入正整数
non_negative_integer	非负整数，可指定最大最小长度。	必须输入非负整数
negative_integer	负整数，可指定最大最小长度。	必须输入负整数
http_url	超链接地址，需要以http://或https://开头，跟域名（可含端口），可指定最大最小长度。	需要以http://或https://开头
number	数值，包括整数和小数，可指定最大最小长度。	必须输入数字
non_negative_number	非负数值，可指定最大最小长度。	必须输入非负数字
positive_number	正数值，可指定最大最小长度。	必须输入正数字
decimal	纯小数，绝对值小于1的小数，0必须为0.0，可指定最大最小长度。	必须输入小数
label	标签，仅包含字母数字和下划线，需要以字母开头，可指定最大最小长度。	仅包含字母数字和下划线，需要以字母开头
string	英文字符串，仅包含字母数字和下划线，可指定最大最小长度。	英文字符串必须仅由字母、数字和下划线组成
chinese_string	中文字符串，仅包含中文，英文字母数字和下划线，可指定最大最小长度，其中中文字符算2位长度。	中文字符串仅包含中文，英文字母数字和下划线
not_chinese_string	字符串，不包含中文，可指定最大最小长度	包含非法的中文字符串
ip_address	4位的ip地址	非法的IP地址
mac	6位的mac地址，连接符可用":"或"- "，但不能混用。	非法的MAC地址
text	任意字符，可指定最大最小长度。	非法的字符串

3.4.3.1.4.3.2 正则校验

通过给 **EInput** 配置**正则规则**，平台会根据正则规则对数据进行校对。对符合规则的数据数据进行放行，不符合的规则的数据进行拦截，并且给出提示信息。

给指定的 **EInput** 进行正则验证时，需要给 **data-regex** 配置相对应的正则规则，如：`/(^[0-9a-zA-Z]{6,10}$)/`。**data-errorPrompt** 是该正则规则对应的提示信息，也需添加相应的 **js** 使校验生效。

3.4.3.1.4.3.3 校验提示信息

内置类别的校验有默认的提示信息，不需配置。正则校验可以配置对应的 **data-errorPrompt** 或者 **data-errorPrompt(n)** 属性，添加校验出错时的提示信息。

3.4.3.1.4.3.4 多重校验

也可以配置多条规则用于校验某一 **EInput**。添加的校验条件说明如下：

- **minLength**: 最小长度
- **maxLength**: 最大长度
- **data-regex**: 正则
- **data-errorPrompt**: **regex** 校验出错时的提示信息
- **data-regex(n)**: 多重正则，要求 **n** 从零开始，连续增加。
- **data-errorPrompt(n)**: 多重提示信息，为 **regex(n)** 校验出错时对应的提示信息。

多重校验可以通过 **data-regex(n)** 来配置正则规则，**data-errorPrompt(n)** 配置提示信息来完成，或者通过内置校验和正则校验混合的方式来完成。

3.4.3.1.4.3.5 分组校验

EInput 具有分组校验功能，通过配置 **validateGroupName** 属性进行分组，并配合 **js** 来完成分组校验功能。

在 **EInput** 中配置 **validateGroupName** 后，可在 **js** 中通过配置 **groupName** 来只校验该分组的 **EInput**(支持**单组件多分组**，如 **validateGroupName="group1,group2"**)。

若某些 **EInput** 不想进行校验也可以通过配置 **validateGroupName**，添加到未激活的分组来完成（当然也可以不配置任何校验属性来完成）。



3.4.3.2 EFPopupInput

3.4.3.2.1 概要说明

- 标签定义：EF:EFPopupInput
- 标签作用：EFPopupInput 是一个通用扩展型输入框，用于处理复杂个性化要求较高的表单输入。默认是弹出 Grid 的选择功能。也支持开发人员自定义开发弹出内容，然后进行选择。

开发人员能够用三种方式去使用 EFPopupInput:

1. 根据自己需要，自定义弹出框显示的内容
2. 配置小代码服务，渲染出 Grid
3. 根据服务名，查询方法配置服务，渲染出 Grid

3.4.3.2.2 属性

属性名称	说明	旧版本属性名	默认值	示例页面号	备注
blockId	三段式中Eiblock的名称(不推荐)	blockId			不推荐
row	三段式中的行号。使用数字字符，表示第几行(不推荐)	row			不推荐
ename	两种形式。1.兼容老方式：三段中的列名称(不推荐)。2.新方式：三段式形式的连接字符串(推荐)。例如：inqu_status-0-ename, result-1-cname	ename		EED M401 0	
cname	组件的标签, 用于说明该组件	cname		EED M401 0	
required	渲染出来的表单输入元素必须要有内容		false	EED M401 0	需要配合JS一块使用，否则不做校验，详情参考 EEDM4001
readonly	渲染出来的表单输入元素不能进行编辑，但是依然会参与校验		false	EED M401 0	
disabled	渲染出来的表单输入元素会被禁用		false		
ratio	控件的标签和input的栅格比例	无	4:8	EED M401 0	
colWidth	控件整体（标签和输入框）的栅格宽度	无	4	EED M401 0	
containe	在弹出窗口中显示的内容的div的id号，	contentDi		EED M401	

属性名称	说明	旧版本属性名	默认值	示例页面号	备注
rld	自定义弹出框内容场景下生效	vld		0	
codeName	小代码分类编号，popupType="DynamicGrid"时生效	fieldId		EED M401 0	
condition	小代码查询条件(ITEM_CODE='EM')	无			
valueField	真实值 (value) 对应EiBlock中的列名	valueProperty		EED M401 0	
textField	显示值 (label) 对应EiBlock中的列名	labelProperty		EED M401 0	
popupType	表明弹出窗口中的Grid的类型， 可选值为"DynamicGrid", "ServiceGrid" 为"DynamicGrid"时，表示弹出的Grid是通过小代码中的SQL查询渲染出来的； 为"ServiceGrid"时，表示弹出的Grid是通过服务名，方法名查询渲染出来的；			EED M401 0	
serviceName	服务名称 (popupType="ServiceGrid"时生效)	无		EED M401 0	
methodName	查询方法名称 (popupType="ServiceGrid"时生效)	无		EED M401 0	
resultId	弹出框块名称，对应dataSource请求后返回EiInfo中block(需要配置popupType="ServiceGrid")	无		EED M401 0	
columnNames	需要显示的列英文名数组	visibleColumnNames		EED M401 0	
columnCnames	需要显示的列中文名数组	visibleColumnDisplayNames		EED M401 0	
backFillColumnIds	除Input自身回填的数据外，其他需要回填的列英文名，是columnNames的子集	backFillColumnIds		EED M401 0	
backFillFieldIds	除自身回填数据的Input框外，其他需要回填数据的输入框的id	backFillFieldIds		EED M401 0	
popupTitle	弹出窗口的标题	无	'详情'	EED	

属性名称	说明	旧版本属性名	默认值	示例页面号	备注
title				M4010	
popupWidth	弹出窗口的宽度	无	620	EED M4010	
popupHeight	弹出窗口的高度			EED M4010	
save	弹出窗口是否显示确定按钮		true	EED M4010	
clear	设置控件的清除按钮		true	EED M4010	
refresh	设置再次弹出时，是否重新渲染Grid		false		
resizable	弹出Grid是否可改变窗口大小		false		
originalInput	只渲染生成一个存储数据数据Input框 (注:组件默认会生成两个input, 一个用于存储展示值, 一个用于存储实际值)。		false		1. originalInput="true" required="true": 则valueFieldInput被校验 2. originalInput="false" required="true": 则textFieldInput和valueFieldInput均被校验 (valueFieldInput校验不通过时失败msg获取不到label值)

3.4.3.2.3 API

[官方文档] <https://docs.telerik.com/kendo-ui/api/javascript/ui/popup>

序号	方法名	参数	返回值	说明	示例页面号
1	IPLAT.EFPopupInput.text(\$(element))	element jQuery对象	String 获取的显示值数据	获取显示值数据	
2	IPLAT.EFPopupInput.text(\$(element) , data)	element jQuery对象		设置显示值数据	

序号	方法名	参数	返回值	说明	示例页面号
		date String 设置的显示值数据			EEDM4012
3	IPLAT.EFPopupInput.value(\$(element))	element jQuery对象	String 获取的真实值数据	获取真实值数据	
4	IPLAT.EFPopupInput.value(\$(element) , data)	element jQuery对象 date String 设置的真实值数据		设置真实值数据	
5	IPLAT.EFPopupInput.readonly(\$(element) , state)	element jQuery对象 state EFPopupInput的只读状态 (Boolean)		设置为只读/非只读状态 state: true: 设置为只读状态 false: 设置为非只读状态	
6	IPLAT.EFPopupInput.enable(\$(element) , state)	element jQuery对象 state EFPopupInput的可用状态 (Boolean)		设置为可用/禁用状态 state: true: 设置为可用状态 false: 设置为禁用状态	
7	IPLAT.EFPopupInput.setAllFields(\$(element) , value ,text)	element jQuery对象 value String 设置的真实值数据 text String 设置的显示值数据		同时设置EFPopupInput的真实值和显示值	
8	IPLAT.EFPopupInput.clear(\$(element) , clearAll)	element jQuery对象 clearAll 是否清空其他指定回填的输入框 (Boolean)		clearAll: true: 清空其他指定回填的输入框 false: 不清空其他指定回填的输入框,只清空 所选输入框自身回填的数据 clearAll 仅适用于有其他需要回填数据的输入框	

3.4.3.2.4 事件

事件	参数	返回值	说明	示例页面号
init	e	无	「初始化弹窗中的内容时触发」，可以根据业务要求控制是否进行弹窗； 「查询渲染之前」，触发的事件，通常在此处执行自定义div内容的相关逻辑；	EEDM4203
query	无	EiInfo	「查询时触发的事件」，通常在此处，设置查询条件；(不支持自定义DIV) 注意： 必须返回一个EiInfo，作为初始查询条件； 配置了query时，默认每次打开窗口都会进行一次查询	EEDM4203
loadComplete	e		「查询完成时触发的事件」，通常在此处绑定自定义div中按钮的查询逻辑 (不支持自定义DIV)	EEDM4203
backFill	e	无	「数据回填时触发的事件」，在弹出Grid中，里面的row数据被 双击选中 或者点击 “确定” 后的回调接口 (不支持自定义DIV)	EEDM4203
clearInput	e	无	「点击清除数据按钮时触发事件」，调用此接口后， 不会触发平台原有的清除事件逻辑	EEDM4203



事件执行顺序：init --> query --> loadComplete --> backFill, clearInput

3.4.3.3 EFTreeInput

3.4.3.3.1 概要说明

- 标签定义：EF:EFTreeInput
- 标签作用：EFTreeInput 是一种特殊的输入框。需要选取弹出框中的树节点，进行数据输入。

3.4.3.3.2 属性

属性名	说明	默认值	示例页面号
blockId	数据块ID		EEDM4010
ename	字段英文名		EEDM4010
cname	字段中文名		EEDM4010
row	字段数据行信息		EEDM4010
originalInput	显示真实值 (true时显示值与真实值一致,false时显示值为show_ename)	false	EEDM4015

属性名	说明	默认值	示例页面号
serviceName	必填项, 查询树的服务名		EEDM4010
methodName	必填项, 查询树的方法名		EEDM4010
valueField	必填项, 树结点的id对应的字段名, 点击树结点时, 根据id查询子结点		EEDM4010
textField	必填项, 树结点显示值对应的字段名		EEDM4010
hasChildren	必填项, 是否显示存在子节点对应列名		EEDM4010
dataSpriteCssClassField	通过配置CSS样式来设置图标		EEDM4010
dataImageUrlField	通过配置图片URL方式, 配置图标		EEDM4010
clear	设置控件的清除按钮	true	
inline	判断输入框是表单字段还是单独使用		EEDM4010
required	字段为必填项		EEDM4015
readonly	字段为只读项,但仍可以参与校验		EEDM4015
disabled	字段为禁用项,调整样式		EEDM4015
ratio	下拉控件的标签, input的栅格比例	4:8	EEDM4010
colWidth	下拉控件的栅格宽度	4	EEDM4010
treeWidth	树的宽度	300	
treeHeight	树的高度	300	
popupTitile	弹出窗口的标题	"详情"	
onlyLeaf	是否只让叶子结点可回填	false	EEDM4010
save	弹出窗口是否显示确定按钮	true	EEDM4014
checkedBackFill	是否回填所有被勾选结点	false	EEDM4014

3.4.3.3.3 API

序号	方法名	参数	返回值	说明	示例页面号
1	IPLAT.EFTreeInput.enable(\$(element) , state)	{String jQuery} element EFTreeInput的id, DOM结点jQuery对象		state: 控制EFTreeInput的可用状态, 禁用时, 点击图标不会弹窗 true 可用 false 禁用	

序号	方法名	参数	返回值	说明	示例页面号
		{Boolean} state EFTreeInput的可用状态			EEDM4015
2	IPLAT.EFTreeInput.readonly(\$(element) , state)	{String jQuery} element EFTreeInput的id, DOM结点jQuery对象 {Boolean} state EFTreeInput的可用状态		state: 控制EFTreeInput的只读状态 true 只读 false 非只读	
3	IPLAT.EFTreeInput.value(\$(element))	{String jQuery} element EFTreeInput的id, DOM结点jQuery对象	String 获取的真实值数据	获取 EFTreeInput的真实值	
4	IPLAT.EFTreeInput.value(\$(element) , date)	{String jQuery} element EFTreeInput的id, DOM结点jQuery对象 date String 设置的真实值的数据		设置 EFTreeInput的真实值	
5	IPLAT.EFTreeInput.text(\$(element))	{String jQuery} element EFTreeInput的id, DOM结点jQuery对象	String 获取的显示值数据	获取 EFTreeInput的显示值	
6	IPLAT.EFTreeInput.text(\$(element) , date)	date String 设置的显示值的数据		设置 EFTreeInput的显示值	
7	IPLAT.EFTreeInput.setAllFields(\$(element) , value , text)	element jQuery对象 value String 设置的真实值数据 text String 设置的显示值数据		同时设置EFPopupInput的真实值和显示值	
8	IPLAT.EFTreeInput.clear(\$(element) , clearAll)	element jQuery对象 clearAll Boolean 是否清空其他指定回填的输入框		clearAll: true: 清空其他指定回填的输入框 false: 不清空其他指定回填的输入框,只清空所选输入框自身回填的数据 clearAll 仅适用于有其他需要回填数据的输入框	

3.4.3.3.4 事件

事件	参数	说明	示例页面号
backFill	e	数据回填触发事件	EEDM4014
init	e	初始化事件。	
loadComplete	e	弹出树加载完成后事件	
check	e	清除图标点击触发事件	

3.4.3.4 EFDatePicker

3.4.3.4.1 概要说明

- 标签定义：EF:EFDatePicker
- 标签作用：EFDatePicker 是用于单个日期类数据输入的一个标签，可满足绝大多数对此类数据的输入要求，例如日期的格式化，输入框的只读以及显示层次（可以设置初始弹出是年，月，日）的设置等。

3.4.3.4.2 属性

属性名	说明	默认值	示例页面号	备注
depth	日期控件可以显示到的层级		EEDM5006	无法直接设置，需要在页面js中进行配置
start	日期控件初始展示的层级	"month"	EEDM5006	无法直接设置，需要在页面js中进行配置
value	日期控件的默认值			
format	日期控件的格式(默认“yyyy-MM-dd”)			
parseFormats	解析format以外的日期格式([‘yyyy/MM/dd’])			
blockId	数据块Id		EEDM5000	
ename	字段英文名(required)		EEDM5000	
cname	字段中文名(required)		EEDM5000	
row	在数据块中的行号信息		EEDM5000	
role	控件的类型，可选填"date"和"datetime"和"time"，代表"日期控件"和"日期时间控件"和"时间控		EEDM5000	

属性名	说明	默认值	示例页面号	备注
	件"			
required	是否必须输入，默认为"false"	"false"	EEDM5000	需要配合JS一块使用，否则不做校验，详情参考 EEDM4001
readonly	字段为只读项，但可以参与校验			
disabled	字段为禁用项，调整样式			
ratio	日期控件的标签和input框的栅格比例	(4:8)	EEDM5000	
colWidth	日期控件的栅格宽度	4	EEDM5000	
Tag中如何使用attribute s 动态属性	可选参数，动态配置的属性			

3.4.3.4.3 API

3.4.3.4.3.1 日期控件（role="date"）

[官方文档] <https://docs.telerik.com/kendo-ui/api/javascript/ui/datepicker>

序号	方法名	参数	返回值	说明	示例页面号
1	<code>\$(element).data("kendoDatePicker").value()</code>	element jQuery对象	Date	获取日期 直接使用API获取到的日期格式为CST: Sun Feb 11 2018 00:00:00 GMT+0800 (CST) 所以一般需要对获取的日期进行格式化: 方式一: <code>kendo.toString(Date, "yyyy/MM/dd");</code> "2018/02/10" <code>kendo.toString(Date, "yyyy/MM/dd hh:mm:ss");</code> "2018/02/10 12:00:00" <code>kendo.toString(Date, "yyyy/MM/dd hh:mm tt");</code> "2018/02/10 12:00"	EEDM5000

序号	方法名	参数	返回值	说明	示例页面号
				上午" 方式二： 使用平台封装的日期格式 kendo.toString(Date, IPLAT.FORMAT.DATE_8_ PR); "yyyy-MM-dd"	
2	<code>\$(element).data("kendoDatePicker").value(Date)</code>	element jQuery对象 Date String 设置的日期值: 可以是Date对象, 也可以是格式化的日期字符串		设置日期	
3	<code>\$(element).data("kendoDatePicker").enable(state)</code>	element jQuery对象 state EFPopupInput的可用状态 (Boolean)		state: true: 启用日期控件 false: 禁用日期控件	
4	<code>\$(element).data("kendoDatePicker").readonly(state)</code>	element jQuery对象 state EFPopupInput的可用状态 (Boolean)		state: true: 设置日期控件为只读 false: 设置日期控件非只读	
5	<code>\$(element).data("kendoDatePicker").open()</code>	element jQuery对象		打开日期控件	
6	<code>\$(element).data("kendoDatePicker").close()</code>	element jQuery对象		关闭日期控件	

日期控件中：普通状态 只读状态 和禁用状态的区别：普通状态下日期可下拉选择，可输入编写；只读状态下只可下拉选择，不可编写；禁用状态下不可下拉，不可编写。

3.4.3.4.3.2 日期时间控件（role="datetime"）

[官方文档] <https://docs.telerik.com/kendo-ui/api/javascript/ui/datetimepicker>

序号	方法名	参数	返回值	说明	示例页面号
1	<code>\$(element).data("kendoDateTimePicker").value()</code>	element jQuery对象	Date	获取日期 格式化同上	
2	<code>\$(element).data("kendoDateTimePicker").value(Date)</code>	element jQuery对象 Date String 设置的日期值:		设置日期	

序号	方法名	参数	返回值	说明	示例页面号
		可以是Date对象，也可以是格式化的日期字符串			EEDM5000
3	<code>\$(element).data("kendoDateTimePicker").enable(state)</code>	element jQuery对象 state EFPopupInput的可用状态 (Boolean)		state: true: 启用日期控件 false: 禁用日期控件	
4	<code>\$(element).data("kendoDateTimePicker").readonly(state)</code>	element jQuery对象 state EFPopupInput的可用状态 (Boolean)		state: true: 设置日期控件为只读 false: 设置日期控件非只读	
5	<code>\$(element).data("kendoDateTimePicker").open()</code>	element jQuery对象		打开日期控件	
6	<code>\$(element).data("kendoDateTimePicker").close()</code>	element jQuery对象		关闭日期控件	

3.4.3.4.4 事件

事件	参数	说明	示例页面号
change	e	设置不同的日期会触发 change 事件	EEDM5000
open	e	打开日期选择界面，会触发 open 事件	
close	e	关闭日期选择界面，会触发 close 事件 日期控件失去焦点，也会触发 close 事件	

3.4.3.5 EFDateSpan

3.4.3.5.1 概要说明

- 标签定义：EF:EFDateSpan
- 标签作用：EFDateSpan 主要用于具有开始/结束时间关系的日期时间数据的选择输入，可灵活的根据设置的开始时间来限定结束时间的选择范围。

3.4.3.5.2 属性

属性名	说明	默认值	示例页面号
depth	日期控件可以显示的到的层级(暂未开发)		EEDCA342
start	日期控件初始展示的层级(暂未开发)	"month"	EEDCA342
format	日期控件的格式(默认“yyyy/MM/dd”)		
blockId	数据块Id		
startName	开始日期字段英文名(required)		
endName	结束日期字段英文名(required)		
startCname	开始日期字段中文名		EEDM5000
endCname	结束日期字段中文名		EEDM5000
startValue	开始时间的默认值(特殊格式需搭配parseFormats 使用)		EEDM5000
endValue	结束时间的默认值(特殊格式需搭配 parseFormats 使用)		EEDM5000
row	在数据块中的行号信息		
role	控件的类型，可选"date"和"datetime"		EEDM5000
ratio	开始日期和结束日期的栅格比例(默认4:4)	(4:4)	
startRatio	开始日期的标签和input的栅格比例(默认4:8)	(4:8)	
endRatio	结束日期的标签和input的栅格比例(默认4:8)	(4:8)	
startTimeFormat	选择完日期后，时刻的默认值	00:00:00	
endTimeFormat	选择完日期后，时刻的默认值	00:00:00	
bindWidth	将开始和结束合并成一个单元格		EEDM5000 EEDM5004
bindName	用于显示合并后的EFDateSpan的标签名，如“开始时间-结束时间”。配置了bindWidth 后 startCname 与 endCname 将只用于校验后的名称显示，而不会在label中显示。		EEDM5000 EEDM5004
bindRatio	一个单元格时候的标签名与日期的栅格比例	(4:4:4)	EEDM5000

属性名	说明	默认值	示例页面号
			EEDM5004
extChar	一个单元格的时候，日期控件中间的字符串		EEDM5000 EEDM5004
Tag中如何使用 attributes 动态属性	可选参数，动态配置的属性。如当role="datetime"时配置interval="15"，则设置时间间隔为15分钟		EEDM5000

3.4.3.6 EFSelect

3.4.3.6.1 概要说明

- 标签定义：EF:EFSelect
- 标签作用：EFSelect 标签用于产生 html 的下拉框控件，EFSelect 通过结合 EFOption、EFOptions、EFCodeOption、EFTableOption 标签来达到这一目的。其中，[EFOption](#) 标签，用于一条一条 option 的写入，[EFOptions](#) 标签则用于批量 option 的写入，[EFCodeOption](#) 标签用于指定小代码的 option 批量写入，[EFTableOption](#) 标签是一种基于数据库的下拉选择。
- [\[官方文档\] http://docs.telerik.com/kendo-ui/api/javascript/ui/dropdownlist](http://docs.telerik.com/kendo-ui/api/javascript/ui/dropdownlist)

3.4.3.6.2 属性

属性名	说明	旧版本属性名	默认值	备注	示例页面号
bindId	数据字段绑定Id	bindId		已废弃	EEDM5012
blockId	三段式中Eiblock的名称(不推荐)	blockId		不推荐	
row	三段式中的行号。使用数字字符，表示第几行(不推荐)	row		不推荐	
ename	两种形式。1.兼容老方式: 三段中的列名称(不推荐)。2.新方式: 三段式形式的连接字符串(推荐)。例如: inqu_status-0-ename, result-1-cname	ename	""		
cname	组件的标签, 用于说明该组件	cname	""		
default	字段默认值, 用于设置初始选中		第一个下拉项的value		

属性名	说明	旧版本属性名	默认值	备注	示例 页面 号
Value	项		值		
required	字段为必填项		false	需要配合JS一块使用, 否则不做校验	
readonly	字段为只读项,但仍可以参与校验		false		
disabled	字段为禁用项,调整样式		false		
serviceName	查询下拉选项的服务名				EEDM 5012
methodName	查询下拉选项查询的方法名				
resultId	查询下拉选项的数据块名。配置serviceName与methodName时生效, 表示查询结果中作为下拉数据源的数据块名				EEDM 5016
textField	显示值对应的列名				
valueField	真实值对应的列名				
ratio	下拉控件的标签和input的栅格比例	ratio	4:8		
colWidth	下拉控件的栅格宽度		4		
template	定制下拉选项显示模板 (如template="#=textField#-#=valueField#")	代替旧版本EFOption, EFOptions的conj属性	不配置时默认显示textField值		EEDM 5012
valueTemplate	定制选中后的结果的下拉选项	代替旧版本EFOption, EFOptions的conj属性	不配置时默认显示textField值		
inline	判断输入框是表单字段还是单独使用				
Tag中如何使用attributes动态属性	可选参数, 动态设置的属性, 如filter="contains"实现下拉过滤。如设置只读readonly="true"	etc			EEDM 5012

3.4.3.6.3 API

序号	方法名	参数	返回值	说明	示例页面号
1	IPLAT.EFSelect.value(<i>element</i>)	element jQuery对象 EFInput的id, DOM 结点jQuery对象	String	获取数据	EEDM5012
2	IPLAT.EFSelect.value(<i>element</i> , <i>data</i>)	element jQuery对象 EFInput的id, DOM 结点jQuery对象 data {Array String} 设置的数据		设置数据	
3	IPLAT.EFSelect.value(<i>element</i> , <i>data</i> , <i>state</i>)	element jQuery对象 EFInput的id, DOM 结点jQuery对象 data {Array String} 设置的数据 state {Boolean} 是否触发change		设置数据，并触发change	
4	IPLAT.EFSelect.dataItem(<i>element</i> , <i>index</i>)	element jQuery对象 index { jQuery Number} 指示元素的位置（最小为0）。	Object 根据索引得到的数据 如果没有数据，则返回undefined	返回指定索引处的数据项。如果未指定索引，则将使用所选项索引。	
5	IPLAT.EFSelect.setDataSource(<i>element</i> , <i>dataSource</i>)	element jQuery对象dataSource 设置的选项		设置现有DropDownList的dataSource并重新绑定它。	
6	IPLAT.EFSelect.readonly(<i>state</i>)	element jQuery对象 state {Boolean} 是否只读	无	true 设置下拉组件只读 false 解除下拉组件只读状态	
7	IPLAT.EFSelect.enable(<i>state</i>)	element jQuery对象 state {Boolean} 是否可用		true 设置下拉组件可用 false 禁用下拉组件状态	

3.4.3.6.4 EFSelect 梳理

3.4.3.6.4.1 概要说明

- 标签定义：EF:EFSelect
- 标签作用：用于下拉数据的展示，可搭配子标签 EFOption，EFOptions 和 EFCodeOption 使用

3.4.3.6.4.2 下拉项的定义

1. 配合 EFOption 标签，配置每一个下拉项

EFOption 标签(详见 [EFOption](#))

JSP 示例

```
<EF:EFSelect cname="(示例一)<br>产品名称:" template="#=valueField#-#=textField#"
    ename="inqu_status-0-product_name" required="true" defaultValue="p1">
    <EF:EFOption label="全部" value="all"/>
    <EF:EFOption label="产品 1" value="p1"/>
    <EF:EFOption label="产品 2" value="p2"/>
    <EF:EFOption label="产品 3" value="p3"/>
    <EF:EFOption label="产品 4" value="p4"/>
</EF:EFSelect>
```

2. 配合 EFOptions 标签，批量配置下拉项

EFOptions 标签(详见 [EFOptions](#))

JSP 示例

```
<EF:EFSelect cname="(示例三)<br>产地信息:" ename="inqu_status-0-product_area2"
    template="#=valueField#-#=textField#" valueTemplate="#=valueField#-
    #=textField#"
    required="true" defaultValue="China" filter="contains" readonly="true">
    <EF:EFOptions blockId="made_country_block_id" textField="country_ch_name"
    valueField="country"/>
</EF:EFSelect>
```

注：EFSelect 标签中 **template / valueTemplate** 属性值目前只可以是 "textField"和 "valueField"的组合

3. 配合 EFCodeOption 标签，使用小代码方式配置

EFCodeOption 标签(详见 [EFCodeOption](#))

JSP 示例

```
<EF:EFSelect cname="(示例二)<br>产品地址:" valueTemplate="#=valueField#-#=textField#"
    ename="inqu_status-0-product_area" required="true" defaultValue="all">
    <EF:EFOption label="全部" value="all"/>
    <EF:EFCodeOption codeName="iplat.codeStyle"/>
</EF:EFSelect>
```

注：EFCodeOption 标签中 **codeName** 为必需属性，表示代码名称，即代码分类管理页面(EDCM01)中的代码分类编号

3.4.3.6.4.3如何联动

EFCascadeSelect 标签(详见 [EFCascadeSelect](#))

JSP 示例

```
<div class="row">
    <EF:EFCascadeSelect ename="inqu_status-0-continent_code"
    valueTemplate="#=continent_code#-#=continent_name#"
```

```

        defaultValue="Europe"
        resultId="result" cname="大洲" textField="continent_name"
valueField="continent_code">
    <EF:EFOption label="大洲 1" value="c1"/>
    <EF:EFOption label="大洲 2" value="c2"/>
</EF:EFCascadeSelect>
    <EF:EFCascadeSelect ename="inqu_status-0-country_code" template="#=country_code#-
#=country_name#"
        methodName="query" cname="国家" resultId="result"
        cascadeFrom="inqu_status-0-continent_code"
        textField="country_name" valueField="country_code">
    </EF:EFCascadeSelect>
    <EF:EFCascadeSelect ename="inqu_status-0-province_code" template="#=province_code#-
#=province_name#"
        methodName="queryProvince" cname="省份" resultId="result"
        cascadeFrom="inqu_status-0-country_code"
        textField="province_name" valueField="province_code">
    </EF:EFCascadeSelect>
</div>

```

3.4.3.6.4.4 基于数据库的下拉选择

配置下拉列表中的数据时，除了使用以上下拉项定义中介绍的两种方式，我们还可以在 **EFSelect** 中定义 **EFTableOption** 标签，达到一种更为方便的下拉项配置方案。**EFTableOption** 标签是一种基于数据库的下拉选择。

详见 [EFTableOption](#)。

3.4.3.6.5 EFCodeOption

3.4.3.6.5.1 概要说明

- 标签定义：EF:EFCodeOption
- 标签作用：EFCodeOption 作为 EFSelect 的一个子标签使用。

3.4.3.6.5.2 属性

属性名	说明	默认值	示例页面号
codeName	代码类别名称，必须参数		EEDM5012
textField	显示值的列名，默认是label	"label"	
valueField	真实值的列名，默认是value	"value"	
condition	查询条件(ITEM_CODE='EM')		

3.4.3.6.6 EFOption

3.4.3.6.6.1 概要说明

- 标签定义：EF:EFOption

- 标签作用：可以在 EFSelect/ EFCascadeSelect/ EFComboColumn 标签中使用，用于设置单个静态下拉选项。

3.4.3.6.6.2 属性

属性名	说明	默认值	示例页面号
label	option的显示值，必填参数。		EEDM5012
value	option的真实值，必须参数。		EEDM5012

属性名解释

EFOption和HTML中的<option>标签含义类似；

实现细节：

EFOption标签定义的下拉选项，存储在IPLAT.EFOptions数组中，每个选项的textField取label值，valueField取value值；

{textField: "\${label}",valueField: "\${value}}")

3.4.3.6.7 EFOptions

3.4.3.6.7.1 概要说明

- 标签定义：EF:EFOptions
- 标签作用：EFOptions 是 EFSelect 的子标签，用于批量设置下拉选项。

3.4.3.6.7.2 属性

属性名	说明	旧版本属性名	默认值	示例页面号
blockId	数据对应的blockId，必须参数	blockId		EEDM5012
valueField	数据值对应的列名	valueColumn	"value"	EEDM5012
textField	显示值对应的列名	labelColumn	"label"	EEDM5012

3.4.3.6.8 EFTableOption

3.4.3.6.8.1 概要说明

- 标签定义：EF:EFTableOption
- 标签作用：EFTableOption 标签可以将数据库中指定表的指定字段提取出来，作为 EFSelect 的下拉选项。

3.4.3.6.8.2 属性

属性名	说明	默认值	示例页面号
beanName	对应的bean名称		EEDM5012
schema	数据库schema (配置beanName时生效, 解决beanName对应的bean不是平台默认数据库schema的场景)	null	
condition	条件(配置beanName或tableName时生效)		
sql	欲获取下拉内容执行的sql语句		EEDM5012
tableName	实际表名		EEDM5012
valueField	实际值对应的列名(配置tableName时生效)		
textField	显示值对应的列名(配置tableName时生效)		
orderBy	获取下拉内容时进行排序的配置(配置tableName时生效)		

3.4.3.7 EFMultiSelect

3.4.3.7.1 概要说明

- 标签定义: EF:EFMultiSelect
- 标签作用: EFMultiselect 标签用于产生 html 的下拉多选框控件, EFMultiselect 通过结合 EFOption、EFOptions、EFCodeOption、EFTableOption 标签来达到这一目的。其中, [EFOption](#) 标签, 用于一条一条 option 的写入, [EFOptions](#) 标签则用于批量 option 的写入, [EFCodeOption](#) 标签用于指定小代码的 option 批量写入, [EFTableOption](#) 标签是一种基于数据库的下拉选择。

3.4.3.7.2 属性

属性名称	说明	旧版本属性名	默认值	示例页面号
bindId	数据绑定ID			
blockId	数据块ID			
ename	字段英文名			EEDM5011
cname	字段中文名			EEDM5011
row	字段数据行信息			

属性名称	说明	旧版本属性名	默认值	示例页面号
defaultValue	字段值			EEDM5011
autoClose	选择后是否自动关闭，默认"true"		"true"	EEDM5011
required	是否必须输入		"false"	
readonly	字段为只读项，但仍可以参与校验			
disabled	字段为禁用项，调整样式			
ratio	多选下拉控件的标签和input的栅格比例(4:8)		4:8	EEDM5011
colWidth	多选下拉控件的栅格宽度		4	EEDM5011
Tag中如何使用 attributes 动态属性	动态设置的属性，如设置过滤方式 filter="contains"			EEDM5011

3.4.3.7.3 API

[官方文档] <https://docs.telerik.com/kendo-ui/api/javascript/ui/multiselect>

方法名	参数	返回值	说明	示例页面号
1 <code>\$(element).data("kendoMultiSelect").value()</code>	element jQuery对象	Array	获取数据	EEDM5011
2 <code>\$(element).data("kendoMultiSelect").value(data); \$(element).data("kendoMultiSelect").trigger("change");</code>	element jQuery对象 data {Array String} 设置的数据		设置数据 需要手动触发change事件	

3.4.3.8 EFCascadeSelect

3.4.3.8.1 概要说明

- 标签定义：EF:EFCascadeSelect
- 标签作用：EFCascadeSelect 组件用于级联下拉，可以通过标签内部属性的配置动态绑定数据并产生级联下拉的效果。

3.4.3.8.2 属性

属性名	说明	默认值	示例页面号
-----	----	-----	-------

属性名	说明	默认值	示例页面号
autoBind	boolean型,是否对默认值进行初始绑定		
cascadeFrom	级联下拉的父下拉框的id值		EEDM5010
blockId	数据块Id		EEDM5010
ename	字段英文名, 可直接使用三段式命名		EEDM5010
cname	字段中文名		EEDM5010
row	在数据块中的行号信息		
textField	显示值对应的列名 (配置serviceName生效)		EEDM5010
valueField	真实值对应的列名 (原dataField,配置serviceName生效)		EEDM5010
serviceName	查询下拉选项的服务名		EEDM5010
methodName	查询下拉选项查询的方法名(配置serviceName生效)		EEDM5010
resultId	下拉选项在EiInfo中的数据块名, 注意和blockId的区别(配置serviceName生效)		EEDM5010
required	字段为必填项	false	
readonly	字段为只读项, 但是依然会参与校验	false	
disabled	字段为禁用项,调整样式	false	
defaultValue	字段默认值, 用于设置初始选中项		EEDM5010
template	定制下拉选项显示模板, (如template= "#=ename#-#=cname#", 连接符任取, 这里取"-"; 字段名包含在"#= #"中)		EEDM5010
valueTemplate	定制选中后的结果的模版		EEDM5010
ratio	描述标签和组件的栅格比例	4:8	
colWidth	下拉选项控件的栅格宽度	4	

3.4.3.9 EFAutoComplete

3.4.3.9.1 概要说明

- 标签定义：EF:EFAutoComplete
- 标签作用：自动补全查询，该组件用于过滤筛选指定列名的数据。

3.4.3.9.2 属性

属性名	说明	默认值	示例页面号
bindId	数据字段绑定Id		EEDM4004
blockId	三段式中Eiblock的名称(不推荐)		EEDM4004
row	三段式中的行号。使用数字字符, 表示第几行(不推荐)		EEDM4004
ename	两种形式。1.兼容老方式: 三段中的列名称(不推荐)。2.新方式: 三段式形式的连接字符串(推荐)。例如: inqu_status-0-ename, result-1-cname		EEDM4004
cname	组件的标签, 用于说明该组件在页面中的作用		EEDM4004
serviceName	指定ajax调用平台后端服务		EEDM4004
queryMethod	指定ajax调用平台后端服务的方法, 和serviceName一起使用		EEDM4004
resultId	指定ajax调用平台后端服务返回的EiInfo中的EiBlock, 此EiBlock会作为数据源进行下拉展示。	result	EEDM4004
dataField	指定ajax调用平台后端服务返回的EiInfo中的EiBlock中需过滤的列, 必须参数		EEDM4004
filter	过滤形式, 参数有(startswith, endswith and contains), 默认是contains	contains	EEDM4004
noDataTemplate	无数据提示	没有数据	EEDM4004
ratio	描述标签和组件的栅格比例	(4:8)	EEDM4004
colWidth	下拉选项控件的栅格宽度	4	EEDM4004

3.4.3.10 EFUpload

3.4.3.10.1 概要说明

- 标签定义：EF:EFUpload
- 标签作用：用于附件的上传下载组件

3.4.3.10.2 属性

属性名称	说明	默认值	示例页面号
ename	上传组件所渲染生成input中的name值，用于后端获取文件流		EEDM4006
docTag	附件的标签，由业务方生成，用于附件的分组		EEDM4006
path	附件上传时的相对路径，例如：folder/sub		EEDM4006
readonly	只展示已上传的文件	false	EEDM4006

3.4.3.10.3 API 接口

这里是文件上传和下载的接口：

接口名称	接口 URL	参数	提交方法	返回值	说明
文件上传	[host]/[context]/EU/DM/EUDM04.jsp 例如10.70.12.21:8080/iplat4j/EU/DM/EUDM04.jsp	1.查询参数: ? path= [path] &docTag= [docTag] &ename= [file] 2.带文件的form表单	POST	{ // docId => 文档的唯一标识 "docId": "xxxxx", // docTag => 分组名称 "docTag": "xxx", // author => 上传用户 "author": "xxx", // uploadTime => 上传时间 "uploadTime": "xxx", // docUrl => 文件路径 "docUrl": "xxx" }	使用EFUploader.tag的时候，已经对上传接口进行封装，可以直接使用。 [host] ：端口地址 [context] ：应用上下文 [path] ：附件上传时的相对路径 [docTag] ：附件的标签，由业务方生成，用于附件的分组 [file] ：表示表单中所上传文件input的name值，用于后端获取文件流
文件查	[host]/[context]/EU/DM/EUD	1.查询参数: ? docTa	GET	{ "recRevisor": "",	此接口可以用于查询上传的分组文件

接口名称	接口 URL	参数	提交方法	返回值	说明
查询	M07.jsp 例如10.70.12.21:8080/iplat4j/EU/DM/EUD M07.jsp	g=[doc Tag]		<pre>// docName => 原文件名 "docName":"67CD4D41044F1245A CF404EBF9C73EBA.jpg", // recCreator => 上传者 "recCreator":"admin", // docId => 文档的唯一标识 "docId":"ff80808160e81acd0160e81 acde80000", "recReviseTime":"", "archiveFlag":"", // recCreateTime => 上传时间 "recCreateTime":"20180112100238" , "dirId":"297e1bed5ea84109015ea84 109f10000", // realPath => 文件路径 "realPath":"D:/lib/A", // docSize => 文件大小 "docSize":41947, // chgName => 服务端存储的文件名称 "chgName":"36158842.jpg", // docTag => 分组名称 "docTag":"qqq"}]</pre>	[host] : 端口地址 [context] : 应用上下文 [docTag]: 附件的标签, 由业务方生成, 用于附件的分组
文件下载	[host]/[context]/EU/DM/EUD M06.jsp 例如10.70.12.21:8080/iplat4j/EU/DM/EUD M06.jsp	1.查询参数: ? docId=[docId]	GET	文件	用于文件下载 [docId] : 文档的唯一标识

3.4.3.11 EFComboInput

3.4.3.11.1 概要说明

- 标签定义: EF:EFComboInput

- 标签作用：EFComboInput 标签用于产生 html 的 input 输入框与下拉框控件的结合。

3.4.3.11.2 属性

属性名	描述	旧版本 属性名	默认值	示例页面号
blockId	数据块ID	blockId		
ename	字段英文名	ename	""	
cname	字段中文名	cname	""	
defaultValue	字段值(即默认选中值)		初始时默认选中的下拉项的value值	
row	字段数据行信息	row		
required	是否必须输入		false	
serviceName	查询下拉选项的服务名			EEDM5014
methodName	查询下拉选项查询的方法名			EEDM5014
resultId	查询下拉选项的数据块名			
textField	数据块中对应显示信息的列名			
valueField	数据块中对应值的列名			
ratio	下拉控件的标签和input的栅格比例	ratio	4:8	EEDM5014
colWidth	下拉控件的栅格宽度		4	

3.4.4 3. 数据表格

3.4.4.1 EFGrid

概要说明

- 标签定义：EF:EFGrid
- 标签作用：在页面展示后台 EiBlock 中的数据，并进行增删改查等交互

属性

属性名	说明	示例 页面号	旧版本 属性名	默认 值
blockId	必须参数，指明了需要展示数据在EiInfo中的数据块（EiBlock）号。	EE DM 60 00	blockId	""
needAuth	可选参数，用于设置EFGGrid的默认按钮是否需要注册授权后显示；默认值是true，设置为false时，不受注册授权控制	EE DM 60 01	无	true
autoDraw	可选参数，用于设置EFGGrid列的展示模式。 可选值"yes"和"no"和"mixed"和"override"，默认为"yes"。 当autoDraw为"yes"时,自动渲染EiBlock块中的所有列的数据 当autoDraw为"no"时，需要有子标签EFColumn或EFComboColumn等， EFGGrid只渲染子标签列； 为"mixed"时，渲染EiBlock和EFColumn的列，列存在同名时，EFColumn的列将覆盖EiBlock中的列信息。 为"override"时，渲染EFColumn的列（类似mixed，但是只渲染EFColumn Tag指定的列，注意和no时区别，为no时，不会使用EiBlock中的列信息。）	EE DM 60 02	autoDraw	"yes"(旧版为"no")
autoBind	可选参数，表示Grid是否进行初始查询，默认为"true"，为false时，需要在Service的initLoad方法中设置Grid的EiBlockMeta信息	EE DM 60 24	无	true
autoFit	可选参数，表示grid是否自动调整单元格宽度，默认为"false"。设置为true时，会将grid的每一列的宽度设置为此列中宽度最大的单元格的宽度。	EE DM 60 01	无	
toolbarConfig	可选参数，控制Grid的工具栏增删改按钮的显示，增加自定义的按钮	EE DM 60 05	无	null
filterable	可选参数，对单列进行过滤	EE DM 60 00	无	false
readonly	可选参数，表示展示的数据是否可编辑，可取值"false"和"true"，默认为"false"。 当readonly为"true"时，该块中除刚刚新增的数据外，其他所有数据均不可编辑。会显示每行的选中框，可以对选定记录进行提交。如果想对所有的数据(包括新增)都不进行编辑操作，请使用enable	EE DM 60 33	readonly	false

属性名	说明	示例 页面号	旧版本 属性名	默认 值
enable	可选参数，表示展示的数据是否可操作，可取值false和true。 当enable为false时，该块中所有数据均不可编辑，而且不显示每行的选中框。	EE DM 60 03	enable	true
rowNo	可选参数，Grid是否开启序列号列。	EE DM 60 22	无	false
checkMode	控制checkbox的勾选模式，single只能单行勾选，row表示点击行就可以勾选，默认值是multiple，cell表示可以勾选多行，而且必须通过点击列头的checkbox勾选	EE DM 60 03 EE DM 60 08	无	"multiple, cell"
无此属性，直接使用EFGrid标签	可选参数，表示数据需要展示的div层id。 若paintId被设置，数据将展示在对应的div层中， 否则使用"ef_grid_"作为前缀加上blockId值作为展示的div层id。	无	paintId	"ef_grid_"+blockId
无此属性，统一使用ajax提交	可选参数，可取值"false"和"true"，默认为"false"。 当ajax为"true"时，表示该表格与Service的交互采用ajax方式，即不刷新整个页面。当ajax为"false"时，使用整个页面提交的方式与Service交互。	无	ajax	false
serviceName	可选参数，表示获取表格数据的Service名，默认与页面的Service名相同。	EE DM 60 01	serviceName	默认与JSP页面的Service名相同
queryMethod	可选参数，表示查询表格数据的method名，默认为"query"。 (页面上存在多个EFGrid时，需要配置不同的queryMethod)	EE DM 60 01	queryMethod	query
insertMethod	可选参数，表示新增表格数据的method名，默认为"insert"。	EE DM 60 01	无	insert
updateMethod	可选参数，表示更新表格数据的method名，默认为"update"。	EE	无	update

属性名	说明	示例页面号	旧版本属性名	默认值
eMethod		DM6001		e
deleteMethod	可选参数，表示删除表格数据method名，默认为"delete"。	EE DM6001	无	delete
showCount	可选参数，默认值为true； 为true时，EFGrid展示记录的总条数和总页数，需要在后端返回总条数； 为false时，后台将不从数据库count记录总数，EFGrid不展示总共条数，总页数。	EE DM6015	无	true
height	设置grid的高度。height应该为正整数，如:400 如果EFGrid.Tag中没有设置height属性，表格会根据当前页的记录的条数，对高度进行自适应处理。 如果设置了Height属性，EFGrid将固定高度，当记录条数过多时，会出现纵向滚动条。	EE DM6019	height	
暂无	可选参数，取值为"quickShow"，则表示grid以快速展示的方式展示grid信息，取值为""则不会快速展示而用平台通常的展示方式。默认为""。注：该方式会使grid的三个回调函数efgrid_onBeforeGridDisplay, efgrid_onCellDisplayReady, efgrid_onFixCellDisplayReady失效，同时对grid的个性化配置也不起作用。	无	showMode	""
暂无	可选参数，表示是否进行前台排序，取值为"true",则右键点击grid的行会弹出菜单，菜单包括【升序】/【降序】菜单项，可以进行grid的升序和降序排列，同时用户可以绑定自己的菜单信息，取值为"false"，则右键点击grid的行菜单不会出现。默认为"false"。		FontSort	false
sumType	可选参数，表示是否要进行小计和总计，取值为"all"则表示grid要对数字列进行小计和总计，取值为"page"则表示grid要对数字列进行小计，取值为"total"则表示grid要对数字列进行总计。其优先级要比EFColumn里的sumType优先级高。	EE DM6022	sumType	
请参考04 Tag中如何使用attributes动态属性	可选参数，是表格显示及控制的进一步配置，style属性的取值类似html标签中的style属性，目前支持以下子属性：		style	
参考toolbarConf	表示是否显示表格操作按钮（目前仅有一个新增按钮），取值为true显示操作按钮，false不显示操作按钮。	EE DM60	operationBar	true

属性名	说明	示例页面号	旧版本属性名	默认值
ig		05	r	
参考toolBarConfig	表示是否显示表格功能按钮（目前仅有一个导出成excel按钮），取值为true显示功能按钮，false不显示功能按钮。	EE DM 60 05	toolBar	false
参考toolBarConfig	表示是否显示从模板导出xls文件按钮，取值为true显示该按钮，false不显示该按钮。 initSelect:表示是否默认将当前行设置为表格的第一行，取值为true将当前行设置为表格的第一行，false不设置，默认为false。	EE DM 60 09	modelXlsBar	false
无此属性，直接按照true的逻辑处理	表示在新增行时是否默认选中新增的行，取值为true默认选中新增的行(复制新增时原行将被取消选中)，取值为false保持原来表格的选中状态。		checkNewLine	false
暂无	表示grid是否允许个性化配置，取值为true时，表示允许个性化，则该grid会在左上角显示个性化配置图标，点击该图标可弹出个性化配置信息可对grid的列属性进行配置，取值为false时表示，不允许个性化，则该图标不会出现。		personalBar	false
暂无	表示grid是否可以配置为单页查询全部记录，配置为true则表示可以单页查询在grid的翻页下拉框中会显示“全部”选项，选中后单页显示全部记录；设置为false则不支持单页查询全部。默认值为false（主要考虑可能开启会对性能有影响，因此通过此属性由应用自己控制）。		canPageAll	false
暂无	可选参数，表示EFGGrid是否支持空字段，以区别0字段与空字段，可取值“false”和“true”，默认为该属性不配置。当SupportNull为“true”时，空字段会被处理成空字符串；当SupportNull为“false”时，空字段会被处理成空格。		SupportNull	
暂无	可选参数，用户可个性配置工具栏的显示位置，属性toolBarPosition可取值“top”和“bottom”，默认为“top”。当toolBarPosition为“top”时，工具栏渲染在EFGGrid的上方；当toolBarPosition为“bottom”时，工具栏渲染在EFGGrid的下方；工具栏默认渲染在EFGGrid的上方。		toolBarPosition	"top"
暂无	可选参数，用户可根据需要控制导出xls文件的内容，属性值可取“key”、“value”，其默认值为“key”。当xlsExportMode属性值为“key”时，导出到xls文件中的数据为后台EiInfo中的Block数据，可能包含隐藏列或者丢掉界面上的构造列(界面js构造的数据列，后台EiInfo并无数据)；当xlsExportMode属性值为“value”时，导出到xls文件中的数据是界面显示的内容(即所见即所得)，不包括隐藏列也不会丢掉界面上的构造列。 平台提供全局配置，在CONFIG.js中为所有的EFGGrid配置此属性(默认为key)，属性值可取“key”、“value”；但其优先级低于EFGGrid控件上单独配置的属性。		xlsExportMode	"key"

属性名	说明	示例 页面号	旧版本 属性名	默认 值
sort	可选参数，表示表格数据的排序，有三种属性值。 当sort设置成single时，此时单列排序，可指定任意列。 当sort设置成all时，全列可排序。排序顺序与点击顺序有关，与列顺序无关。 当sort设置成setted时，指定列排序。	EE DM 60 27		
personal	可选参数，表示是否需要动态展示列。personal="true"的时候可以动态改变列的顺序及修改某些属性。	ED FA 61		
pagerPosition	可选参数，表示翻页条存在于Grid的位置。可用参数有'top'、'bottom'。top表示grid的右上角，bottom表示在Grid下面，会占一行。	EE DM 60 00		top
editHelper	标识Grid中cell是否可进行编辑，可编辑的Cell将会多套一层内框表明可编辑	EE DM 60 07		false
isFloat	当数据过多(每页100条)，且只有页面最外侧有滚动条时。推荐使用isFloat="true"来固定列头。	EE DM 60 41		false

API

API 名称	参数	返回值	说明	示例 说明
addRow	无	无	kendo原生的API，将会在Grid的头部新增一行	
addRows	mode l bottom checked	无	Grid的当前页新添加行数据，第一个参数可以传入Model对象，或者它们的集合。 bottom是表示是否在Grid的后面插入数据，默认是false，即在前面插入数据。当设置bottom = true时，将会在后面插入数据 checked是表示插入的新数据是否会被选中，默认是true，表示会被选中。当设置checked = false时，新增数据将不会被选中 1. 当checked为true，或者为空的情况下，将会触发Grid如下事件： dataBinding => onAdd => dataBound => onCheckRow => afterAdd 2. 当checked为false，将会触发Grid如下事件：dataBinding => onAdd => dataBound => afterAdd	

API 名称	参数	返回值	说明	示例说明
removeRows	list	无	删除Grid中的行数据 list 可以是单行号或者多行号的组数, 也可以是单个Model 或者Model的集合列表 当时数据被删除后, 将会触发Grid如下事件: dataBinding => dataBound	
saveChanges	无	无	kendo原生的API, 用于给后端提交变更的数据(包括: 新增数据、修改数据、删除数据)。	
cancelChanges	无	无	kendo原生的API, 用于还原还没有提交到后端的变更数据	
getDataItems	无	Array	获取当前页的 所有行 的数据的数组; 通过grid.dataItems()[index], 获取对应 一行 的数据(推荐使用); 这一行数据是kendo.data.Model对象model, 通过model.get("field"), 获取具体一列 单元格 的数据。其中field是列的英文名。	
getRows	list	Array	根据行号或行号数组, 获取对应的行数据, 返回值为kendo.data.Model的数组, 数组中的model对象, 表示选中行的数据。 参数list是可以为单个行号: 比如 0, 也可以是行号数组如[0,1,2]	
getLockedColumns	无	Array	获取Grid中的固定列的配置信息数组。 [{field: "", locked: true},{...}] 注意: 固定列中可能包括checkbox勾选列和序号列 (rowNo="true"), 这两列位于数组的头部, 可使用数组的shift方法, 从数组中移除这两列;	
getCheckedRows	无	Array	获取Grid中勾选的数据, 结果为kendo.data.Model的数组, 数组中的model对象, 表示选中行的数据。可以根据model.toJSON(), 获取JSON对象; 注意: model中的日期数据类型是Date, 在model.toJSON()时, 需要转换日期的格式	
getCheckedRowsIndex	无	Array	获取Grid已勾选的数据行的行号数组	
setCheckedRows	list	无	根据行号或行号数组, 设置Grid勾选的行 参数list是可以为单个行号: 0, 也可以是行号数组如[0,1,2] 注意: 该API会触发Grid事件onCheckRow	
checkAllRows	无	无	勾选所有的数据行, 将会触发Grid的 onCheckRow 事件	
unCheckAllRows	无	无	取消勾选所有的数据行, 将会触发Grid的 onCheckRow 事件	
getBlockData	无	EiBlock	获取Grid中当前页的所有数据, 组装成待提交到后台的EiBlock; (对比参考getDataItems)	
model2EiBlock	model	EiBlock	将kendo.data.Model对象或Model对象数组, 组装成待提交到后	

API 名称	参数	返回值	说明	示例说明
ck	Is	ock	台的EiBlock;	
getCheckedBlockData	无	EiBlock	获取Grid中勾选的数据，组装成EiBlock	
setEiInfo	EiInfo	无	使用EiInfo中的EiBlock数据，重新设置Grid当前页的数据； 1. EiInfo中应包含数据块EiBlock，其blockId应该和Grid的blockId相同； 2. EiInfo中的EiBlock数据会完全覆盖Grid当前页的数据； 3. 此方法不会提交数据到后端服务器，请调用 grid.dataSource.sync() 进行数据提交；	
addEiInfo	EiInfo bottom checked	无	将EiInfo中的EiBlock数据追加到Grid中，EiInfo中的block名字和列信息需要与Grid的block名字和列信息一致 bottom是表示是否在Grid的后面插入数据，默认是false，即在前面插入数据。当设置bottom = true时，将会在Grid的后面插入数据 checked是表示插入的新数据是否会被选中，默认是true，表示会被选中。当设置checked = false时，新增数据将不会被选中 1. 当checked为true，或者为空的情况下，将会触发Grid如下事件：dataBinding => onAdd => dataBound => onCheckRow => afterAdd 2. 当checked为false，将会触发Grid如下事件：dataBinding => onAdd => dataBound => afterAdd	
setCellValue	list field cellValue	无	根据行号或行号数组，列英文名，设置Grid中的单元格的值； 参数list是可以为单个行号：0，也可以是行号数组如[0,1,2]，也可以是Grid当前页中的 kendo.data.Model实例； field是列英文名 cellValue 是要设置的值。 注意：list为行号数组时，可以进行批量赋值	
wrapEiBlock	EiBlock	EiInfo	把EiBlock组装成EiInfo，默认将当前Grid的全部数据都放在EiInfo中；当有传入EiBlock，就会把传入的EiBlock数据全部放入EiInfo中。 此外，返回的EiInfo还会把查询区内的数据进行组装，作为一个block放入EiInfo中，一并进行返回。	
getQueryInfo	无	EiInfo	获取当前Grid的查询条件，这些条件会封装到EiInfo中进行返回	
postQuery	EiInfo	无	将会把传入的EiInfo作为参数传入后端进行查询。	

上面所指的 Model 对象是指 kendo.data.Model 的实例，详情请参考 <https://docs.telerik.com/kendo-ui/api/javascript/data/model>。

回调函数一览表:

旧版回调函数	说明	新版做法	示例页面
efgrid_onCellDisplayReady	在数据列单元格生成或刷新时调用，用于自定义单元格的显示	设置column的template属性	
efgrid_onFixCellDisplayReady	在固定列单元格生成或刷新时调用，用于自定义单元格的显示	设置column的template属性	
efgrid_onGridDisplayReady	在表格显示完成后，进行用户自定义的操作	设置Grid的dataBound事件 Grid初始化完成的回调loadComplete,	
efgrid_onDataCellClick	数据列中指定显示为链接或按钮的单元格，点击后的回调函数	onCellClick	onRowClick
efgrid_onFixCellClick	固定列中指定显示为链接或按钮的单元格，点击后的回调函数	onCellClick	onRowClick
efgrid_onRowCheckboxClicked	每一行左侧复选框点击时的回调（选中和取消选中均触发）	onCheckRow	
efgrid_afterRowCheckboxClicked	每一行左侧复选框点击后的回调（选中和取消选中均触发）	暂无，如需要提供，在当前页面评论，说明使用的场景	
efgrid_onSelectAllClicked	全选复选框点击时的回调（选中和取消选中均触发）	onCheckAllRows	
efgrid_onAddNewRow	表格创建新行按钮被点击后进行的回调	beforeAdd	
efgrid_afterAddNewRow	表格创建新行按钮被点击,新增行数据之后进行的回调	afterAdd	
efgrid_onRowClicked	表格行被点击时的回调函数（当前行改变）	onRowClick	
efgrid_onBeforeGridDisplay	表格数据渲染前的回调函数	设置Grid的dataBinding事件	
efgrid_onBeforeCellEditNodeDisplay	表格单元格进入编辑状态时的回调函数	beforeEdit, 控制单元格能否编辑	
efgrid_onCellEditNodeDisplayReady	表格单元格编辑状态渲染完毕后的回调函数	设置Grid的edit事件， 可以控制单元格的编辑类型。（下拉，日期，弹窗） 参考column的editor文档	
efgrid_onDataCellSaved	表格数据列单元格数据存储	afterEdit	

旧版回调函数	说明	新版做法	示例页面
	时的回调函数		
efgrid_onFixCellSaved	表格固定列单元格数据存储时的回调函数	afterEdit	
efgrid_onGridSubmit	表格数据提交(包括翻页, 设置显示记录数, 调用efgrid.submitForm提交数据)时的回调	onSave	
efgrid_onAjaxSubmitSuccess	表格以ajax方式提交成功后的回调函数	onSuccess	
efgrid_onRowDbClicked	表格中行双击触发的回调事件	onRowDbClick	
efgrid_onExport_modleXls		无	

3.4.4.2 EFColumn

概要说明

- 标签定义: EF:EFColumn
- 标签作用: EF:EFGrid 的子标签, 用来描述表格的列信息

属性

属性配置规则参考[属性配置规则](#)。

属性名	说明	旧版本属性名	默认值	示例页面号
ename	列字段值	ename 兼容	""	EEDM6002
cname	列名显示值	cname 兼容	""	EEDM6002
sort	可选参数, 数据列是否允许排序。(注: sort需要有EFGrid中的 sort="setted" 这样的全局配置方能使用)	sort	主键为true, 非主键为false	EEDM6027
primaryKey	表明列是否为主键, 主键列默认是固定列	无	false	EEDM600

属性名	说明	旧版本属性名	默认值	示例页面号
y				6
暂无	可选参数，该列是否允许进行可个性化配置，取值为true则可以进行个性化配置，取值为false则如果该列为可见列（固定列必然可见），则该列的固定列和隐藏列属性不可配置，如果该列为隐藏列，则该列在弹出的配置信息里不会出现。	canPersonal	true	
sumType	可选参数，取值为"all"则该列进行总计和小计，取值为"total"则该列进行总计，取值为"page"则进行小计，"none"表示不做小计总计。其优先级要比EFGGrid里的sumType优先级低。	sumType	"none"	EE DM 602 2
locked	可选参数，数据列是否为固定列。可选值为，true，false。默认值为false	fix	false（旧版本：若元数据中该列被设置为主键，取值为"yes"，否则为"no"。）已废弃	EE DM 601 0
displayType	可选参数，数据列显示类型。可选值"text"，"url"，"button"，取值为"text"时该列以文本形式展示，取值为"url"时，该列的数据被展示为超链接，取值为"button"时，该列的数据被展示为按钮。	displayType	"text"	EE DM 600 5
editType	可选参数，数据列编辑类型。可选值"text"，"date"，"textarea"，默认为"text"。取值为"text"时该列使用文本输入框形式编辑，取值为"date"时该列使用日期选择框形式编辑，取值为"textarea"时该列使用弹出文本输入域形式编辑。	editType	"text"	EE DM 600 2
dateFormat	可选参数，指定该列 日期 的显示格式。例如:yyyy-MM-dd hh:mm:ss。。该属性仅当editType取值为date或者datetime时生效。	dateFormat		EE DM 600 2
parseFormats	可选参数，指定该列 日期 的来源格式。通常使用数组形式例如 ['yyyy/MM/dd HH:mm:ss']。注意与dateFormat的区别，这里通常用于表示后端返回的数据格式，而不是实际在页面展示的格式	无		EE DM 600 2
format	可选参数，指定该列 数字 的显示格式。该属性仅当editType取值为text时生效，format可取如"{0:c2}"或"{0:n3}%"等格式。	formatString		EE DM 600 2
template	定制单元格的显示模版，注意和format的区别。template可以对 数字 ， 日期 ， 字符串 进行更灵活的配置	无		EE DM 600 3
data-rule	可选参数， 后台进行列校验时推荐同步使用 ，指定该列数据的验证类型。	validateType		EE DM 600

属性名	说明	旧版本属性名	默认值	示例页面号
s				7
data-regex	可选参数，指定该列数据的验证正则表达式。当该值为非空(非空包括不是空字符串)时，validateType设置的值无效。	validateRegex		EEDM6007
maxLength	可选参数，指定该列数据的最大长度，默认为元数据中定义的数据。	maxLength		EEDM6007
minLength	可选参数，指定该列数据的最小长度，默认为元数据中定义的数据。	minLength		EEDM6007
data-errorprompt	可选参数，指定该列数据验证错误时的提示信息，默认为元数据中定义的数据。	validateErrorPrompt		EEDM6007
required	可选参数，指定该列数据是否可为空。可选值true和false	nullable	true	EEDM6007
defaultValue	可选参数，指定该列数据在新增时的默认值。	defaultValue		EEDM6006
hidden	可选参数，指定该列数据是否为隐藏列。可选值true和false，默认为元数据中定义的数据。	visible (和hidden的配置相反)	false	EEDM6006
readonly	可选参数，指定该列数据是否为只读列。可选值"true"和"false"，默认为元数据中定义的数据。新增时可编辑列，其他状态不可编辑该列	readonly	false	EEDM6006
width	可选参数，指定该列的列宽，单位为px，默认为元数据中定义的数据。	width	150	EEDM6006
align	可选参数， 动态属性 ，指定该列的水平对齐方式，可选值"left"，"center"，"right"。	align	left	
aliases	列的别名，解决多表join时，列名冲突的问题	alias	""	EEDM640

属性名	说明	旧版本属性名	默认值	示例页面号
				0
enable	可选参数，表示该列是否可用，enable设置为false时不可编辑，包括该列新增行的数据。	enable	true	EEDM6006
copy	可选参数，表示该列是否开启批量复制功能	无	false	EEDM6021
needAuth	Grid上的新增保存删除按钮是否进行权限判断，默认是true		true	EEDM6001

3.4.4.3 EFComboColumn

概要说明

- 标签定义：EF:EFComboColumn
- 标签作用：提供数据以下拉列表编辑的方法，为 EFGrid 的子标签。内部可使用 [EFOption](#)，[EFOptions](#)，[EFCodeOption](#)，[EFTableOption](#) 子标签

属性

属性名	说明	旧版本属性名	默认值	示例页面号
ename	必须参数，数据在块中的列名	ename		EEDM6002
cname	必须参数，数据列在页面的显示名称	cname		EEDM6002
blockName	可选参数，下拉列表中数据源所在EiInfo中的块名。	blockName	""	EEDM6002
sourceName	可选参数，下拉列表中数据源所在JSP request对象中的属性名，该属性必须为java.util.List的实例，List中每个元素是一个Map对象，拥有valueField指定的属性。指定该属性后blockName属性无效。	sourceName	""	EEDM6002
valueField	必须参数，下拉列表中数据源所在EiInfo块中的列名。	valueProperty		EEDM6002
textField	可选参数，下拉列表中数据显示值所在EiInfo块中的列名。默认与valueField相同。	labelProperty		EEDM6002

属性名	说明	旧版本属性名	默认值	示例页面号
readonly	可选参数, 指定该列数据是否为只读列。可选值"true"和"false", 默认为"false"。	readonly	"false"	EEDM 6002
width	可选参数, 指定该列的列宽, 单位为px, 默认为元数据中定义的数据。	width		EEDM 6002
align	可选参数, 指定该列的水平对齐方式, 可选值"left", "center", "right", 默认为left, 默认为元数据中定义的数据。	align		
enable	可选参数, 表示该列是否可用, enable设置为false时不可编辑, 包括该列新增行的数据。默认取值为true。	enable		
defaultValue	可选参数, 指定该列数据在新增时的默认值。	default Value		EEDM 6002
locked	是否为固定列	fixed	"false"	
columnTemplate	可选参数, 指该列的单元格的显示模版。 例如该列中字段value值为"baosight", label值为"宝信", 配置属性 columnTemplate="#=valueString#-#=labelString#", 则格式化后显示为"baosight-宝信"; 当然, 可单个值显示, 配置属性 columnTemplate="#=labelString#", 则格式化后显示为"宝信"。	format String	默认与textField相同	EEDM 6002
itemTemplate	下拉框选项的显示模版, 注意和columnTemplate的区别	无		EEDM 6002
copy	可选参数, 是否开启栏批量复制功能	无	"false"	EEDM 6021
Tag中如何使用attributes动态属性	动态属性, autoWidth: 下拉项宽度是否自适应("true", "false"); optionLabel: 作为引导信息, 指定默认空值下拉项; filter: 下拉过滤配置("contains", "startswith", "endswith"); required: 必填设置("true", "false") 如设置 required="true"指定该列必填, optionLabel="请选择"指定默认空值下拉项, autoWidth="true" 设置下拉选项宽度自适应, filter="contains" 设置下拉过滤条件 (选填"contains", "startswith", "endswith")	etc		EEDM 6526

3.4.4.4 EFMultiSelectColumn

概要说明

- 标签定义: EF:EFMultiSelectColumn
- 标签作用: 提供数据以下拉列表编辑的方法, 为 EFGGrid 的子标签。内部可使用 [EFOption](#), [EFOptions](#), [EFCodeOption](#), [EFTableOption](#) 子标签

- 标签说明：EFMultiSelectColumn 和 [EFComboColumn](#) 的属性基本一致，EFMultiSelectColumn 只是能支持多个选中，EFComboColumn 只能支持每次选中一个

属性

属性名	说明	旧版本属性名	默认值	示例页面号
ename	必须参数，数据在块中的列名	ename		EEDM 6002
cname	必须参数，数据列在页面的显示名称	cname		EEDM 6002
blockName	可选参数，下拉列表中数据源所在EiInfo中的块名。	blockName	""	EEDM 6002
sourceName	可选参数，下拉列表中数据源所在JSP request对象中的属性名，该属性必须为java.util.List的实例，List中每个元素是一个Map对象，拥有valueField指定的属性。指定该属性后blockName属性无效。	sourceName	""	EEDM 6002
valueField	必须参数，下拉列表中数据源所在EiInfo块中的列名。	valueProperty		EEDM 6002
textField	可选参数，下拉列表中数据显示值所在EiInfo块中的列名。默认与valueField相同。	labelProperty		EEDM 6002
readonly	可选参数，指定该列数据是否为只读列。可选值"true"和"false"，默认为"false"。	readonly	"false"	EEDM 6002
width	可选参数，指定该列的列宽，单位为px，默认为元数据中定义的数据。	width		EEDM 6002
align	可选参数，指定该列的水平对齐方式，可选值"left", "center", "right"，默认为left，默认为元数据中定义的数据。	align		
enable	可选参数，表示该列是否可用，enable设置为false时不可编辑，包括该列新增行的数据。默认取值为true。	enable		
defaultValue	可选参数，指定该列数据在新增时的默认值。	default Value		EEDM 6002
locked	是否为固定列	fixed	"false"	
columnTemplate	可选参数，指该列的单元格的显示模版。例如该列中字段value值为"baosight"，label值为"宝信"，配置属性columnTemplate="#=valueString#-#=labelString#"，则格式化后显示为"baosight-宝信"； 配置属性columnTemplate="#=valueString#: #=labelString#"，则格式化后显示为"baosight: 宝信"； 当然，可单个值显示，配置属性columnTemplate="#=labelString#"，则格式化后显示为"宝信"。	format String	默认与textField相同	EEDM 6002
itemTemplate	下拉框选项的显示模版，注意和columnTemplate的区别	无		EEDM 6002
copy	可选参数，是否开启栏批量复制功能	无	"false"	EEDM

属性名	说明	旧版本属性名	默认值	示例页面号
				6021
Tag中如何使用attributes动态属性	动态属性，如设置 required="true"指定该列必填	etc		

3.4.4.5 EFPopupColumn

概要说明

- 标签定义：EF:EFPopupColumn
- 标签作用：EF:EFGrid 的子标签，用于在编辑时弹出 Grid 选择数据,并指定该 Grid 中几列数据回填到原 Grid 对应的几列中去。

属性

属性名称	说明	示例页面号	旧版本属性名	默认值
ename	字段英文名，表示该column的名称	EEDM6013	ename	
cname	字段中文名，表示该column的标题名称	EEDM6013	cname	
readonly	是否只读（默认设置为true）	EEDM6013	无	
containerId	在弹出窗口中显示的内容的div的id号,自定义弹出框内容场景下生效	EEDM6013	contentDivId	
codeName	小代码分类编号(popupType="DynamicGrid"时生效)	EEDM6013	fieldId	
condition	小代码查询条件(ITEM_CODE='EM')		无	
valueField	真实值（value）对应EiBlock中的列名	EEDM6013	valueProperty	
textField	显示值（label）对应EiBlock中的列名	EEDM6013	labelProperty	
popupType	表明弹出窗口中的Grid的类型， 可选值为"DynamicGrid", "ServiceGrid" 为"DynamicGrid"时，表示弹出的Grid是通过小代码中的SQL查询渲染出来的； 为"ServiceGrid"时，表示弹出的Grid是通过服务名，方法名查询渲染出来的；	EEDM6013		

属性名称	说明	示例页面号	旧版本属性名	默认值
serviceName	服务名称（popupType=ServiceGrid时生效）	EEDM6013	无	
methodName	查询方法名称（popupType=ServiceGrid时生效）	EEDM6013	无	
resultId	弹出窗口中的Grid请求后返回EiInfo中blockId（popupType="ServiceGrid"时生效）	EEDM6013	无	
columnNames	配置弹出的小代码中的可见列英文名，列名以逗号分隔	EEDM6013	visibleColumnNames	
columnCnames	配置弹出的小代码中的可见列的对应中文名，列名以逗号分隔	EEDM6013	visibleColumnDisplayNames	
backFillColumnIds	设置弹出的Grid中需要回填到页面输入框的列英文名，以逗号分隔	EEDM6013	backFillColumnIds	
backFillFieldIds	设置回填到界面输入框的Id，以逗号分隔	EEDM6013	backFillFieldIds	
popupTitle	弹出窗口的标题，默认值为详情	EEDM6013	无	
popupWidth	弹出窗口的宽度，默认值为620	EEDM6013	无	

API

API名	参数	返回值	说明
query	无	EiInfo	query方法是在调用后端Service服务之前，前端设置查询参数的接口
backFill	e	无	在弹出Grid中，里面的row数据被 双击选中 或者点击 "保存" 后的回调接口
init	e	无	当弹出框里面的内容已经渲染完毕后，将会调用此接口。通常用于处理业务逻辑的初始化

3.4.4.6 EFTreeColumn

概要说明

- 标签定义：EF:EFTreeColumn
- 标签作用：EF:EFGGrid 的子标签，用于在编辑时弹出 Tree 选择数据，勾选或点击回填到当前列中。

属性

属性名称	说明	示例页面号	默认值
ename	字段英文名，表示该column的名称，必填	EEDM6038	
cname	字段中文名，表示该column的标题名称		

属性名称	说明	示例页面号	默认值
serviceName	查询树的服务名，必填		
methodName	查询树的方法名，必填		
valueField	树结点的id对应的字段名，点击树结点时，根据id查询子结点，必填		
textField	树结点显示值对应的字段名，必填		
hasChildren	树结点类型字段，表明结点是否是叶子结点，必填		
root	根结点信息, 使用JSON格式进行配置		
dataImageUrlField	通过配置图片URL方式，配置图标		
dataSpriteCssClassField	通过配置CSS样式来设置图标		
treeWidth	树的宽度，默认值为300		300
treeHeight	树的高度，默认值为300		300
popupTitle	弹出窗口的标题，默认值为"详情"		"详情"
copy	批量复制操作栏功能是否开启，默认是false，不开启		false
onlyLeaf	是否只让叶子结点可回填(true:只有叶子结点可回填)		false
checkedBackFill	是否开启勾选回填，默认为false		false
readonly	字段为只读项，但仍可以参与校验		false

API

API 名	参数	返回值	说明
query	postEiInfo,model	EiInfo	query方法是在每次展开树结点进行查询前设置树查询条件EiInfo
backFill	e	无	自定义回填逻辑
init	e	无	当弹出框里面的内容已经渲染完毕后，将会调用此接口。通常用于处理业务逻辑的初始化

3.4.5 4. 树和菜单

3.4.5.1 EFTree

3.4.5.1.1 概要说明

- 标签定义：EF:EFTree

- 标签作用：在页面中使用树形结构逐级加载展示 EiBlock 的数据
- 标签说明：树组件是基于 kendo UI TreeView 进行开发和封装的，基本兼容 V3.9 的后端 Service 服务

属性

属性名	说明	旧版本属性名	默认值	示例页面号
bindId	树的Id名称，后期可能会弃用			
id	树表组件的唯一标识，js中的可根据此属性配置指定的组件	无		EEDM7000
root	使用 JSON格式 配置首个节点信息。配置的格式 root="{label: 'xxx', text: 'xxx'}"。表示首个节点的初始化信息。其中label为valueField 映射值， text为 textField 映射值	无	无	EEDM7000
textField	必选参数，树结点显示值对应的字段名	text		EEDM7000
valueField	树结点的id对应的字段名，点击树结点时，根据id查询子结点	id		EEDM7000
hasChildren	必选参数，用来指定节点中哪个属性用来表示是否存在子节点。比如 hasChildren="leaf" ，就表明leaf属性中的值是用来表示是否存在子节点。另外，由于平台以前版本中是使用1(不存在子节点)和2(存在子节点)两种数据来表示是否存在子节点，本EFTree.tag是兼容以前平台的服务。所以在调用后端服务时候，开发人员是需要注意返回的数据内容。	leaf	无	EEDM7000
serviceName	必选参数，调用后端Service的服务名	serviceName	无	EEDM7000
methodName	必选参数，调用后端Service的服务方法名	methodName	无	EEDM7000
dataSpriteCssClassField	可选参数，用来指定节点中那个属性用来表示此节点的图标样式。	spriteCssClass		EEDM7022
dataImageUrlField	可选参数，通过配置图片URL方式，配置图标		无	EEDM7022

3.4.5.1.2 API

方法名称	参数	返回值	说明	示例页面号
select	e	无	select 是一个EFTree提供的一个事件，它是在用户选择树中某个节点后	EEDM7001

方法名称	参数	返回值	说明	示例页面号
			触发。	
query	postEiInfo, model	无	query是提供给开发人员进行设置初始化参数的一个接口	EEDM7000

3.4.5.1.3 相关页面

- [EEDM7000-Tree 基础示例](#)
- [EEDM7001-Tree 与 Grid 联动基础示例](#)
- [EEDM7002-Tree 结点操作示例](#)
- [EEDM7003-树结点显示多选框示例](#)
- [EEDM7004-模板显示结点信息示例](#)
- [EEDM7005-过滤选择树结点示例](#)
- [EEDM7006-树结点增删改示例](#)

3.4.5.1.4 数据交互

3.4.5.1.4.1 前端数据接口

- **service**: serviceName, // 服务名称
- **method**: methodName, // 方法名称
- **eiinfo**: `ei.toJSONString()`, // eiinfo => {....
"blocks":{"inqu_status":{"attr":{},"meta":{"desc":"","attr":{},"columns":[{"name":"node"}]},"rows":[{"xxx"}]}}
- 查询参数：对应的三段式为 `inqu_status-0-node:xxxx`。xxxx 表示需要查询的父节点，`inqu_status-node` 为固定参数，后端可以通过此参数获取需要查询的父节点。

3.4.5.1.4.2 后端 Service 返回数据接口

后端返回数据接口是：

```
eiinfo => { ..... blocks: { xxxx: { ..... } } }
```

前端 `inqu_status-0-node: xxxx` 中的 xxxx 为返回数据的 block 名称，里面可以存储返回的数据。

返回的数据需要包括最基本的一些变量 (当然这些变量名称不固定，只是所带的含意如下)：

id (表明此节点的主键, 如果存在子节点的话, 是其子节点的父id), text (节点的文本名称), hasChildren (是否存在子节点, 通常使用 0 或者 1 表示, 0 =>存在, 1 =>不存在)。这些信息都可以通过解析后端返回的 eiInfo 获取。

3.4.5.2 EFOnceTree

概要说明

- 标签定义: EF:EFOnceTree
- 标签作用: 在页面中使用树形结构一次渲染 EiBlock 中数据, 一次加载所有的数据, 逐级展开不会重新加载数据
- 标签说明: 树组件是基于 kendo UI TreeView 进行开发和封装的。

属性

属性名	说明	旧版本属性名	默认值	示例页面号
bindId	可选参数, 页面中用于生成树的div块的id名称。开发人员可以通过bindId来设置id名称, 如果没有设置预设bindId, 那么将会用tree_"当前时间戳"的形式生成。如tree_1494914064459	无	"tree_" + System.currentTimeMillis()	EED M700 0
root	使用JSON格式配置首个节点信息。 配置的格式 root="{label: 'root', text: 'root', icon: 'folder'}"。 表示首个节点的初始化信息。其中label为valueField映射值, text为 textField 映射值, icon 为 dataSpriteCssClassField 映射值。	无	无	EED M700 0
textField	必选参数, 树结点显示值对应的字段名	text		EED M700 0
valueField	必选参数, 树中展示节点的唯一标识	id		EED M700 0
pid	必选参数, 用作级联关系, 是父节点的Id, 是子节点的父Id	无		EED M700 0
hasChildren	必选参数, 用来指定节点中那个属性用来表示是否存在子节点。比如hasChildren="leaf", 就表明leaf属性中的值是用来表示是否存在子节点。另外, 由于平台以前版本中是使用0(存在子节点)和1(不存在子节点)两种数据来表示是否存在子节点, 本EFTree.tag是兼容以前平台的服务。所以在调用后端服务时候, 开发人员是需要注意返回的数据内容。	leaf	无	EED M700 0
serviceName	必选参数, 调用后端Service的服务名	serviceName	无	EED M700 0
methodName	必选参数, 调用后端Service的服务方法名	methodName	无	EED M700 0

属性名	说明	旧版本属性名	默认值	示例页面号
me		e		0
dataSprite CssClass Field	可选参数，用来指定节点中那个属性用来表示此节点的图标样式。	sprite CssClass		EEDM7000
dataImage UrlField	通过配置图片URL方式，配置图标	无	无	EEDM7000

3.4.5.3 EFSwap

概要说明

- 标签定义：EF:EFSwap
- 标签作用：用于左右两棵树结点灵活移动配置场景的标签，目前只使用静态数据源的配置

属性

属性名	说明	默认值	示例页面号
blockId	可选参数，数据块Id	无	EEDM7007
row	可选参数，在数据块中的行号信息	无	EEDM7007
leftName	必选参数，左侧树英文名，与blockId row共同构成左侧树的Id	无	EEDM7007
rightName	必选参数，右侧树英文名，与blockId row共同构成右侧树的Id	无	EEDM7007
leftCname	可选参数，左侧树中文名	"左侧树"	EEDM7007
rightCname	可选参数，右侧树中文名	"右侧树"	EEDM7007
leftDataSource	必选参数，左侧树数据源	无	EEDM7007
rightDataSource	必选参数，右侧树数据源	无	EEDM7007
immediatelyRender	可选参数，布尔类型参数，是否立刻渲染生成树；若为false 则不会通过tag配置生成树	true	EEDM7007
accessAll	可选参数，布尔类型参数，是否操作所有相同ID的节点；若为true，点击结点后会将当前树具有相同ID的其他所有结点	false	

属性名	说明	默认值	示例页面号
	一并移动		
height	可选参数，设置树的高度，单位px	300	EEDM7007
width	可选参数，设置树的宽度，单位px	200	EEDM7007
ratio	左右侧树宽度的栅格比例	5:5	EEDM7007

3.4.5.4 EFTreeList

概要说明

- 标签定义：EF:EFTreeList
- 标签作用：在页面中使用树表结构，展示层级的数据。

属性

属性名	说明	旧版本属性名	默认值	示例页面号
bindId	可选参数，页面中用于生成树的div块的id名称。开发人员可以通过bindId来设置id名称，并在JS中使用该bindId，对EFTreeList配置	无		
parentId	必选参数，树结点显示值对应的字段名	parentId		EEDM7010
valueField	树结点的id对应的字段名，点击树结点时，根据id查询子结点	id		EEDM7010
hasChildren	必选参数，用来指定节点中那个属性用来表示是否存在子节点。比如hasChildren="leaf"，就表明leaf属性中的值是用来表示是否存在子节点。另外，由于平台以前版本中是使用0(存在子节点)和1(不存在子节点)两种数据来表示是否存在子节点，本EFTree.tag是兼容以前平台的服务。所以在调用后端服务时候，开发人员是需要注意返回的数据内容。	leaf	无	EEDM7010
serviceName	可选参数，调用后端Service的服务名	serviceName	当前页面对应的服务	EEDM7010
queryMethod	可选参数，调用后端Service的服务查询方法名	queryMethod	query	EEDM7010
insertMethod	可选参数，调用后端Service的服务新增方法名	queryMethod	insert	EEDM7010
deleteMethod	可选参数，调用后端Service的服务删除方法名	queryMethod	delete	EEDM7010
updateMethod	可选参数，调用后端Service的服务更新方法名	queryMethod	update	EEDM7010

属性名	说明	旧版本属性名	默认值	示例页面号
od	名	d		0

API

方法名称	参数	返回值	说明	示例页面号
------	----	-----	----	-------

示例页面: [EEDM7400-EFTreeList 组件示例](#)

3.4.6 5. 基础组件

3.4.6.1 弹窗对话框组件

3.4.6.1.1 IPLAT.alert 组件

概要说明

- 组件定义: IPLAT.alert
- 组件作用: 用于页面上弹出的警告窗口; 主要用于报错、警告操作等场景。

属性

属性名称	说明	默认值	示例页面号
title	弹出窗口的标题	“	EEDM1030
message	弹出窗口的内容文字	“	EEDM1030
okFn	点击确定按钮的回调函数	无	EEDM1030
minWidth	弹出窗口的最小宽度	300	EEDM1030

3.4.6.1.2 IPLAT.confirm 组件

概要说明

- 组件定义: IPLAT.confirm
- 组件作用: 用于页面上弹出的确认窗口; 主要用于提示是否确认操作、提供操作分支等场景。

属性

属性名称	说明	默认值	示例页面号
title	弹出窗口的标题	“	EEDM1031
message	弹出窗口的内容文字	“	EEDM1031
okFn	点击确定按钮的回调函数	无	EEDM1031
cancelFn	点击取消按钮的回调函数	无	EEDM1031
minWidth	弹出窗口的最小宽度	350	EEDM1031

3.4.6.1.3 IPLAT.prompt 组件

概要说明

- 组件定义：IPLAT.prompt
- 组件作用：用于页面上弹出的输入窗口；主要用于提示重要值的输入、数据绑定等场景。

属性

属性名称	说明	默认值	示例页面号
title	弹出窗口的标题	“	EEDM1032
message	弹出窗口的内容文字	“	EEDM1032
okFn	点击确定按钮的回调函数	无	EEDM1032
cancelFn	点击取消按钮的回调函数	无	EEDM1032
minWidth	弹出窗口的最小宽度	350	EEDM1032

3.5 示例中心

3.5.1 布局容器

3.5.1.1 EEDM8000-多页签示例

3.5.1.1.1 页面功能简介

本示例页面用于说明，EFTab 分页标签使用方式

3.5.1.1.2 配置和 API 详情

参考 [EFTab](#) 文档。

3.5.1.1.3 页面开发步骤

3.5.1.1.3.1 Tab 展示

3.5.1.1.3.1.1 JSP 页面开发

示例代码如下：

JSP 示例

```
<EF:EFTab id="info" showClose="true" active="1">
  <div title="停机信息查询">
    <EF:EFRegion id="inqu" title="查询条件">
      <div class="row">
        <div class="col-md-3">
          <div class="form-group">
```

```

        <label for="workCode" class="col-xs-5 control-label">
            作业线代码
        </label>
        <div class="col-xs-7">
            <input id="workCode" name="workCode" class="k-textbox">
        </div>
    </div>
</div>
<EF:EFDateSpan startName="sss" endName="eee" startCname="停机开始时间"
endCname="停机结束时间"
        ratio="3:3" startRatio="5:7" endRatio="5:7">
</EF:EFDateSpan>
<EF:EFSelect ename="inqu_status-0-type" cname="停机类型" ratio="5:7"
colWidth="3">
    <EF:EFOption label="1" value="2">
</EF:EFOption>
    <EF:EFOption label="2" value="3">
</EF:EFOption>
    <EF:EFOption label="3" value="4">
</EF:EFOption>
</EF:EFSelect>
</div>
<div class="row">
    <EF:EFSelect ename="inqu_status-0-confirm" cname="确认状态" ratio="5:7"
colWidth="3">
        <EF:EFOption label="1" value="2">
</EF:EFOption>
        <EF:EFOption label="2" value="3">
</EF:EFOption>
        <EF:EFOption label="3" value="4">
</EF:EFOption>
</EF:EFSelect>
        <div class="col-md-3">
            <div class="form-group">
                <label for="number" class="col-xs-5 control-label">
                    停机编号
                </label>
                <div class="col-xs-7">
                    <input id="number" name="number" class="k-textbox">
                </div>
            </div>
        </div>
    </div>
</EF:EFRegion>
</div>
<div title="开机信息查询">
    <EF:EFRegion id="result" title="记录集">
        <EF:EFGrid blockId="result">
</EF:EFGrid>
    </EF:EFRegion>
</div>
<div title="明细区">
    <EF:EFRegion id="detail" title="明细区">
        <div class="row">
            <%--<EF:EFDatePicker colWidth="3" ename="inqu_status-0-start" cname="停
机开始时间" ratio="5:7"--%>
            <%--</EF:EFDatePicker>--%>

```

```

                <%--<EF:EFDatePicker colWidth="3" ename="inqu_status-0-end" cname="停机
结束时间" ratio="5:7">--%>
                <%--</EF:EFDatePicker>--%>
                <EF:EFDateSpan startName="ss" endName="ee" startCname="停机开始时间
"
                                endCname="停机结束时间" role="date"/>

        </div>
    </EF:EFRegion>
</div>
<div title="属地管理">
    <div class="block-content query-region form-horizontal">
        <div class="row">
            <EF:EFInput ename="cname2" cname="中文名:">
            </EF:EFInput>
            <EF:EFInput ename="ename2" cname="英文名:">
            </EF:EFInput>
            <EF:EFSelect ename="legalPerson2" cname="法人代表:" ratio="4:8"
colWidth="4">
                <EF:EFOption label="1" value="2">
                </EF:EFOption>
                <EF:EFOption label="2" value="3">
                </EF:EFOption>
                <EF:EFOption label="3" value="4">
                </EF:EFOption>
            </EF:EFSelect>
        </div>
        <div class="row">
            <EF:EFInput ename="scname2" cname="中文简称:">
            </EF:EFInput>
            <EF:EFInput ename="sename2" cname="英文简称:">
            </EF:EFInput>
            <EF:EFSelect ename="company2" cname="单位区分:" ratio="4:8"
colWidth="4">
                <EF:EFOption label="1" value="2">
                </EF:EFOption>
                <EF:EFOption label="2" value="3">
                </EF:EFOption>
                <EF:EFOption label="3" value="4">
                </EF:EFOption>
            </EF:EFSelect>
        </div>
    </div>
</div>
<div title="..."></div>
<div title="..."></div>
<div title="..."></div>
<div title="..."></div>
....
</EF:ETab>

```

- 1.EFTab标签要与div配合使用
- 2.同一个EFTab内的各个div属于同一区域内的不同签页的内容
- 3.EFRegion标签包含在div内使用

3.5.1.1.3.1.2 页面展示效果



当 Tab 标签很多时：

The screenshot shows a web application interface with a top navigation bar containing multiple tabs: 产品信息查询, 开机信息查询, 明细区, 属地管理, 银行, 联系地址, 联系人, 证书, 物料对照关系, 审核履历, 通知, 提醒, 待办, 个人履历. Below the tabs is a '记录集' (Record Set) section displaying a table with columns: 产品代号, 公司名称, 产地, 上市日期, 价格, 公司首页, Google查询, 验证字段0, 验证字段1, 验证字段2, 验证字段3, 验证字段4, 验证字段5, 验证字段6. The table contains several rows of data, including entries for 'huawei', 'samsung', 'Motorola', 'testix', 'Motorola', 'oppo', 'Motorola', 'SGH-I7161', and 'SonyEric'. Below the table is a 'TAB 相关' (TAB Related) section with a 'TAB API 示例' (TAB API Example) and a '新增内容' (Add Content) section.

3.5.1.1.3.2 Tab 相关 API 示例

3.5.1.1.3.2.1 JSP 部分

按钮和 input 相关

```
<EF:EFRegion id="tabOperation" title="TAB 相关">
  <div class="row">
    <div class="col-md-12">
      <div class="form-group">
        <h4 style="font-size: 20px; margin-left: 15px;"> TAB API 示例: </h4>
      </div>
    </div>

    <div class="col-md-4">
      <div class="form-group">
        <div class="col-xs-5">
          <button id="selectByIndex" type="button" class="i-btn-lg">选择对应
            标签</button>
        </div>
        <div class="col-xs-7">
          <input id="selectIndex" class="k-textbox" value="0">
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <button id="removeSelected" type="button" class="i-btn-lg">删除所选标签
    </button>
  </div>
</div>
```

```

<hr>
<div class="row">
  <div class="col-md-4">
    <div class="form-group">
      <div class="col-xs-5">
        <button id="insertBeforeTitle" type="button" class="i-btn-lg">在所
选标签前添加</button>
      </div>
      <div class="col-xs-7">
        <input id="before" class="k-textbox" value="title">
      </div>
    </div>
  </div>
  <div class="col-md-4">
    <div class="form-group">
      <div class="col-xs-5">
        <button id="insertAfter" type="button" class="i-btn-lg">在所选标签
后添加</button>
      </div>
      <div class="col-xs-7">
        <input id="after" class="k-textbox" value="title">
      </div>
    </div>
  </div>
  <div class="col-md-4">
    <div class="form-group">
      <div class="col-xs-5">
        <button id="append" type="button" class="i-btn-lg">在最后添加标签
</button>
      </div>
      <div class="col-xs-7">
        <input class="k-textbox" id="appendI" value="title">
      </div>
    </div>
  </div>
</div>
<hr>
<div class="row">
  <div class="col-md-4">
    <div class="form-group">
      <label for="content" class="col-xs-5 control-label">
        新增内容:
      </label>
      <div class="col-xs-7">
        <input id="content" class="k-textbox" value="content">
      </div>
    </div>
  </div>
  <div class="col-md-4">
    <button id="enable" type="button" class="i-btn-lg">可选/不可选</button>
  </div>
  <div class="col-md-4">
    <button id="removeButton" type="button" class="i-btn-lg">添加删除按钮
</button>
  </div>
</div>
</EF:EFRegion>

```


3.5.1.1.3.2.2 JS 部分

JS 示例

```

$(function () {
    var tab_Strip;

    $(window).load(function () {
        // 获取 tab 实例对象 确保控件已经初始化
        tab_Strip = $("#info").data("kendoTabStrip");
    });

    // 根据 index 选择某一标签
    $("#selectByIndex").click(function () {
        var index = $("#selectIndex").val();
        tab_Strip.select(index);
    });

    // 删除所选标签
    $("#removeSelected").click(function () {
        var tab = tab_Strip.select();
        tab_Strip.remove(tab);
        tab_Strip.select(0);
    });

    // 在所选标签前添加新标签
    $("#insertBeforeTitle").click(function () {
        tab_Strip.insertBefore({
            text: $("#before").val(),
            content: $("#content").val()
        }, tab_Strip.select());
    });

    // 在所选标签后添加新标签
    $("#insertAfter").click(function () {
        tab_Strip.insertAfter({
            text: $("#after").val(),
            content: $("#content").val()
        }, tab_Strip.select());
    });

    // 在所有标签后添加
    $("#append").click(function () {
        tab_Strip.append({
            text: $("#appendI").val(),
            content: $("#content").val()
        });
    });

    // 可选/不可选
    $("#enable").click(function () {
        if (tab_Strip.select().hasClass("k-state-disabled")) {
            tab_Strip.enable(tab_Strip.select(), true);
        } else {
            tab_Strip.enable(tab_Strip.select(), false);
        }
    });
});

```

```
// 添加删除按钮
$("#removeButton").click(function () {
    var tabsElements = $("#info").find('li[role="tab"]');
    tabsElements.append('<span data-type="remove" class="k-link"><span class="k-  
icon k-i-close"></span></span>');
    tab_Strip.tabGroup.on("click", "[data-type='remove']", function (e) {
        e.preventDefault();
        e.stopPropagation();
        var item = $(e.target).closest(".k-item");
        tab_Strip.remove(item.index());
        if (tab_Strip.items().length > 0 && item.hasClass('k-state-active')) {
            tab_Strip.select(0);
        }
    });
});
```

3.5.1.1.3.2.3 页面展示效果

(1) 选择对应标签，删除对应标签：

The screenshot displays the iPlatUI interface. At the top, there is a navigation bar with various tabs like '产品信息查询', '开机信息查询', etc. Below this is a table titled '记录集' (Record Set) containing product data. The table has columns for product code, company name, origin, release date, price, company homepage, Google search, and several verification fields. Below the table, there is a section titled 'TAB 相关' (TAB Related) which includes a 'TAB API 示例' (TAB API Example) section. This section shows a form with input fields and buttons for selecting, deleting, adding, and removing tabs, along with a '新增内容' (Add Content) section.

(2) 在所选标签前（后）添加，在最后添加标签：

Tab 示例 > EEDM8000

开机信息查询 > 明细 > 属地管理 > 银行 > 联系地址 > 联系人 > 证书 > 物料对照关系 > 审核履历 > 通知 > 提醒 > 待办 > 个人履历 > 进度 >

记录集

产品代码	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	量
qqq	huawei	america	2017/11/0..	150122	baosteel	N70		baosight6..	https://s..			1		
1111	sumseng	america	2017/11/0..	150122		N70		baosight6..	https://s..			1		
N7021	Motorola1..	GB, German...	2017/11/0..	150122		N71		baosight6..	https://s..			1		
111EE	111	GB, German...	2017/11/0..	11		N7011		abc@baosi..	https://s..			1		
test1x	AAAAA	GB, German...	2017/12/1..	0				abc@baosi..				0		
N7022	Motorola1..	GB, German...	2017/11/0..	150122	EEE1	N70		abc@baosi..	https://s..			1		
N70	oppo	GB, German...	2017/11/2..	150122	EEE	N70		abc@baosi..	https://s..			20		
8251	Motorola1..	GB, German...	2017/11/1..	150122	EEE	N70		abc@baosi..	https://s..			1000		
SGH-17161	Samsung	GB, German...	2007-04-06	10.14	http222wwwsamsu...	查询	138000000..	fuyimin	021-50801..	1		2000		121
T6281	SonyEric...	Japan	2007-04-06	30	http://www.sonyer...	查询	138000000..	abc@baosi..	Label	021-50801..	1	2000		121

TAB 相关

TAB API 示例:

选择对应标签: 0 删除所选标签

在所选标签前添加: title 在所选标签后添加: title 在最后添加标签: title

新增内容: content 可选/不可选 添加删除按钮

(3) 修改添加的 TAB 的内容，TAB 的可选 / 不可选:

Tab 示例 > EEDM8000

停机信息查询 > 开机信息查询 > 明细 > 属地管理 > 银行 > 联系地址 > 联系人 > 证书 > 物料对照关系 > 审核履历 > 通知 > 提醒 > 待办 > 个人履历 >

记录集

产品代码	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	量
qqq	huawei	america	2017/11/0..	150122	baosteel	N70		baosight6..	https://s..			1		
1111	sumseng	america	2017/11/0..	150122		N70		baosight6..	https://s..			1		
N7021	Motorola1..	GB, German...	2017/11/0..	150122		N71		baosight6..	https://s..			1		
111EE	111	GB, German...	2017/11/0..	11		N7011		abc@baosi..	https://s..			1		
test1x	AAAAA	GB, German...	2017/12/1..	0				abc@baosi..				0		
N7022	Motorola1..	GB, German...	2017/11/0..	150122	EEE1	N70		abc@baosi..	https://s..			1		
N70	oppo	GB, German...	2017/11/2..	150122	EEE	N70		abc@baosi..	https://s..			20		
8251	Motorola1..	GB, German...	2017/11/1..	150122	EEE	N70		abc@baosi..	https://s..			1000		
SGH-17161	Samsung	GB, German...	2007-04-06	10.14	http222wwwsamsu...	查询	138000000..	fuyimin	021-50801..	1		2000		121
T6281	SonyEric...	Japan	2007-04-06	30	http://www.sonyer...	查询	138000000..	abc@baosi..	Label	021-50801..	1	2000		121

TAB 相关

TAB API 示例:

选择对应标签: 0 删除所选标签

在所选标签前添加: title 在所选标签后添加: title 在最后添加标签: title

新增内容: content 可选/不可选 添加删除按钮

(4) 添加删除按钮:

Tab 示例 > EEDM8000

title title2 title3 title4 title5 title6 title7 title8

content

TAB 相关

TAB API 示例:

选择对应标签: 0 删除所选标签

在所选标签前添加: title5 在所选标签后添加: title8 在最后添加标签: title

新增内容: content 可选/不可选 添加删除按钮

3.5.1.2 EEDM8001-Tab 示例 (contentType="iframe")

3.5.1.2.1 页面功能简介

本示例页面用于说明，EFTab 分页标签，contentType="iframe"，且标签页内容为 data-src 指定的页面时如何进行配置。

3.5.1.2.2 配置和 API 详情

参考 [EFTab](#) 文档。

3.5.1.2.3 页面开发步骤

3.5.1.2.3.1 JSP 页面开发

示例代码如下，

首先利用和标签，渲染出四个 Tab 标签页；

然后在<div>...</div>内配置 data-src 属性，用于指定对应标签页的内容。

当EFTab的contentType属性为iframe时，必须使用以下格式：

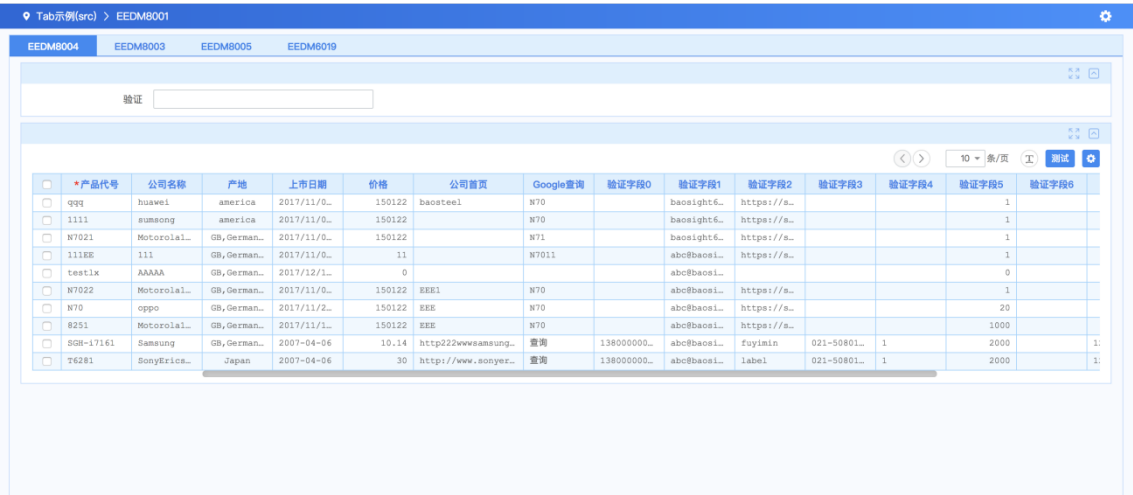
```
<ul>
<li>...</li>
<li>...</li>
</ul>
<div id="..." data-src="..."></div>
<div id="..." data-src="..."></div>
```

JSP 示例

```
<EF:EFTab id="info" contentType="iframe">
  <ul>
    <li>EEDM8004</li>
    <li>EEDM8003</li>
    <li>EEDM8005</li>
    <li>EEDM6019</li>
  </ul>
  <div id="x_03" title="EEDM8004" data-src="${ctx}/web/EEDM8004">
</div>
<div id="x_04" title="EEDM8003" data-src="${ctx}/web/EEDM8003">
</div>
<div id="x_05" title="EEDM8005" data-src="${ctx}/web/EEDM8005">
</div>
<div id="x_06" title="EEDM6019" data-src="${ctx}/web/EEDM6019">
</div>
</EF:EFTab>
```

应遵循<div>的id都不相同的原则，否则可能出现渲染问题。

3.5.1.2.3.2 页面展示效果



3.5.1.3 EEDM8002-Tab 与 Grid

3.5.1.3.1 页面功能简介

本示例页面用于说明，如何解决 EFTab 与 EFGGrid 混合使用时渲染问题。tab 每个页签内都有一个空的 grid,在查询后将数据填入到每个 grid,未显示的页签 content 中的 grid 会显示异常。

3.5.1.3.2 配置和 API 详情

参考 [EFTab](#) 文档。

3.5.1.3.3 页面开发步骤

3.5.1.3.3.1 JSP 页面开发

JSP 示例

```
<EF:EFPage title="Tab 与 grid 混合使用示例">
  <EF:EFRegion id="inqu">
    <div class="row">
      <EF:EFInput ename="inqu_status-0-product_id" cname="测试" colWidth="6"
ratio="6:6">
    </EF:EFInput>
  </div>
</EF:EFRegion>
  <EF:EFTab id="info">
    <div title="第一个 grid">
      <EF:EFGrid blockId="result" autoBind="false">
    </EF:EFGrid>
    </div>
    <div title="第二个 grid">
      <EF:EFGrid blockId="resultA" autoBind="false">
    </EF:EFGrid>
    </div>
  </EF:EFTab>
</EF:EFPage>
```

```
</EF:EFPage>
```

3.5.1.3.3.2JS 开发

渲染异常，使用 grid 的 refresh 方法来解决。在 EFTab 的 show 事件中进行处理。

JS 示例

```
$(function () {
    $("#SEARCH").click(function () {
        var ei = new EiInfo();
        ei.setByNode("inqu");
        EiCommunicator.send("EEDM6024", "query", ei, {
            onSuccess: function (ei) {
                resultGrid.setEiInfo(ei);
                resultAGrid.setEiInfo(ei);
            }, onFailure: function () {

            }
        });
    });
});

IPLATUI.EFGrid = {
    "result": {
        columns: [
            {
                field: "product_id",
                title: "产品代号",
                headerTemplate: "<span style=' color: red'>产品代号</span>",
                width: 200,
                locked: true
            },
            {
                field: "company_name",
                title: "公司名称",
                headerTemplate: "<span style=' color: blue'>公司名称</span>",
                locked: false
            }
        ],
        dataBound: function (e) {

        }
    },
    "resultA": {
        columns: [
            {
                field: "product_id",
                title: "产品代号",
                headerTemplate: "<span style=' color: red'>产品代号</span>",
                width: 200,
                locked: true
            },
            {
                field: "company_name",
                title: "公司名称",
```

```

        headerTemplate: "<span style=' color: blue'>公司名称</span>",
        locked: false
    }]
}
};
IPLATUI.EFTab = {
    "info": {
        show: function (e) {
            $(e.contentElement).find("div[data-
role=' grid' ]").data("kendoGrid").refresh()
        }
    }
}
});

```

注意：开发人员这个最好在 **show** 这个回调事件中处理 **Tab** 选中后的页面

3.5.1.3.3.3 页面展示效果

产品代号 查找

第一个grid 第二个grid

* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证
3c24f950	宝信软件	Japan;C	2018-03	10	http://j;<		13000000000	aaa@123.com	label	021-508011..	1
T6281	SonyEricss..	Japan	2007-04-06	0	http://www.sonyeric..	gg	13000000000	aaa@123.com	label	021-508011..	1
T62861	SonyEricss..	Japan	2007-04-06	0	http://www.sonyeric..	查询	13000000000	aaa@123.com	label	021-508011..	1
5GH-17161	Samsung	Korea	2007-04-06	0	http://www.samsung...	查询	13000000000	aaa@123.com	fuyimin	021-508011..	1
T6282	SonyEricss..	Japan	2007-04-06	0	http://www.sonyeric..	查询	13000000000	aaa@123.com	label	021-508011..	1
N701	NOKIA	Sweden	2007-04-06	0.12		search		abc@baosig..			
T618109hh	Los	9	20171018	15	时间访问量统计	查询	13000000000	aaa@123.com		021-508011..	1
1112	Motorola	China	2007-07-06	11	EwFM07			abc@baosig..	11	-123456789..	
5891	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13000000000	aaa@123.com	label	021-508011..	1
TN702	los		2017-10-14	0							

10 每页 显示条目 1 - 10 共 60

3.5.1.4 EEDM8007-Tab 延迟加载

3.5.1.4.1 页面功能简介

本示例页面用于说明，EFTab 多 Tab 中 Grid 的延迟（点击后）再加载。

3.5.1.4.2 配置和 API 详情

参考 [EFTab](#) 文档。

3.5.1.4.3 页面开发步骤

3.5.1.4.3.1 EFTab 中的 Grid

3.5.1.4.3.1.1 JSP 页面开发

内部为 EFGGrid 的 EFTab，可通过 Java 与 JS 来实现延迟加载。

JSP 示例

```
<EF:EFTab id="info">
  <div title="停机信息查询">
    <EF:EFGrid blockId="result">
    </EF:EFGrid>
  </div>
  <div title="开机信息查询">
    <EF:EFGrid blockId="resultB">
    </EF:EFGrid>
  </div>
</EF:EFTab>
```

3.5.1.4.3.1.2 Javascript 开发

通过 JS 配置 EFTab 的 select 事件，在选中某一标签页后，对应的 Grid 应做一次查询来完成加载，可通过判断当前 grid model 数组的 length 等方式来判断是否已加载过该 grid。

JS 示例

```
IPLATUI.EFTab = {
  "info": {
    select: function (e) {
      var grid = $(e.contentElement).find("div.k-grid").data("kendoGrid");
      if(grid.getDataItems().length === 0){
        grid.dataSource.page(1);
      }
    }
  }
}
```

3.5.1.4.3.1.3 Service 服务开发

在 initLoad 中应返回对应的 meta 信息，以便 grid 完成初始加载，将列头渲染出来。

initLoad

```
public EiInfo initLoad(EiInfo inInfo) {
  inInfo.addBlock(new EiBlock("resultB")).addBlockMeta(new EE10().eiMetadata);
  inInfo.addBlock(new EiBlock("result")).addBlockMeta(new EE10().eiMetadata);
  return inInfo;
}
```


3.5.1.4.3.1.4 页面展示效果

Tab延迟加载示例 > EEDM8007

Tab内嵌Grid

手机信息查询 开机信息查询

10 条/页

☐	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6
☐	qqq	huawei	america	2017/11/0...	150122	baosteel	N70		baosight6...	https://s...			1	
☐	1111	sumsog	america	2017/11/0...	150122		N70		baosight6...	https://s...			1	
☐	N7021	Motorola...	GB, German...	2017/11/0...	150122		N71		baosight6...	https://s...			1	
☐	111EE	111	GB, German...	2017/11/0...	11		N7011		abc@baosil...	https://s...			1	
☐	testix	AAAAA	GB, German...	2017/12/1...	0				abc@baosil...				0	
☐	N7022	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N70		abc@baosil...	https://s...			1	
☐	N70	oppo	GB, German...	2017/11/2...	150122	EEE	N70		abc@baosil...	https://s...			20	
☐	#251	Motorola...	GB, German...	2017/11/1...	150122	EEE	N70		abc@baosil...	https://s...			1000	
☐	SGH-i716i	Samsung	GB, German...	2007-04-06	10.14	http22www.samsu...	查询	138000000...	abc@baosil...	fuyimin	021-50801...	1	2000	11
☐	T6281	SonyEric...	Japan	2007-04-06	30	http://www.sonyer...	查询	138000000...	abc@baosil...	label	021-50801...	1	2000	11

3.5.1.4.3.2 EFTab 中的 iframe

3.5.1.4.3.2.1 JSP 页面开发

内部为 iframe 的 EFTab，可直接通过 tag 相关配置完成延迟加载。

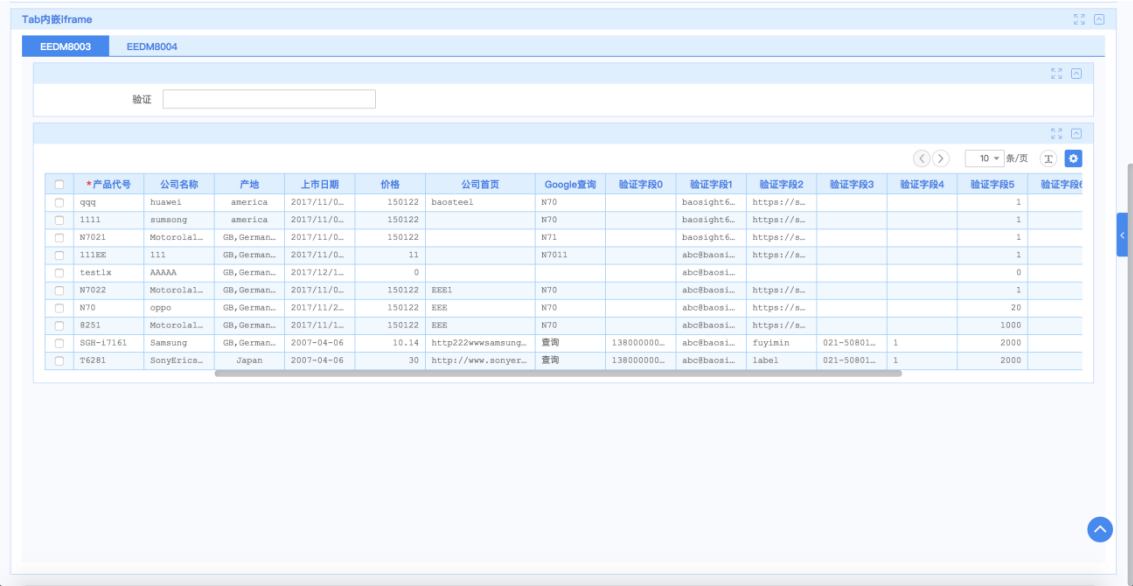
当EFTab的contentType属性为iframe时，必须使用以下格式：

```
<ul>
<li>...</li>
<li>...</li>
</ul>
<div id="..." data-src="..."></div>
<div id="..." data-src="..."></div>
```

JSP 示例

```
<EF:EFTab id="info2" contentType="iframe">
  <ul>
    <li>EEDM8003</li>
    <li>EEDM8004</li>
  </ul>
  <div id="x_03" data-src="{ctx}/web/EEDM8003">
  </div>
  <div id="x_04" data-src="{ctx}/web/EEDM8004">
  </div>
</EF:EFTab>
```

3.5.1.4.3.2.2 页面展示效果



3.5.1.5 EEDM8008-Tab 动态加载示例

3.5.1.5.1 页面功能简介

本示例页面用于说明，EFTab 如何动态加载多个页签。

3.5.1.5.2 配置和 API 详情

参考 [EFTab](#) 文档。

3.5.1.5.3 页面开发步骤

3.5.1.5.3.1JSP 页面开发

JSP 示例

```
<EF:EFTab id="info" state="true" content="src" tabPosition="left">
  <div id="x_03" title="EEDM8003" data-src="{ctx}/web/EEDM8003">
  </div>
  <div id="x_04" title="EEDM8004" data-src="{ctx}/web/EEDM8004">
  </div>
  <div id="x_05" title="EEDM8005" data-src="{ctx}/web/EEDM8005">
  </div>
</EF:EFTab>
```

- 1.state属性配置在content="src"时生效，设置state="true"时，会在切换标签页的时候重新加载对应的src内容
- 2.若设置content="src"，则EFTab组件内每一个带title的div项都要设置data-src属性
- 3.tabPosition设置标签页title的显示位置，默认tabPosition="top"显示在头部

, 可选"top","bottom","left","right"

3.5.1.5.3.2 页面展示效果



3.5.1.6 EEDM8010-EFWindow 弹出窗口示例

3.5.1.6.1 页面功能简介

本示例页面用于说明，**EFWindow** 标签介绍，以及如何使用 **EFWindow** 打开窗口并回填（**EFWindow** 可在当前页面内嵌 **iframe** 打开窗口，或者弹出新的页面以供操作）。

3.5.1.6.2 配置和 API 详情

参考 [EFWindow](#) 中的描述。

关键配置：

1. **url**: 加载页面内容的链接地址。
2. **lazyload**: 延迟加载页面，主要针对 **iframe**，默认 **false**。

3.5.1.6.3 页面开发步骤

3.5.1.6.3.1 JSP 页面开发

默认窗口配置不可见(**visible: false**)；可通过 **url** 值配置窗口中内容。

JSP 示例

```
<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<EF:EFPage title="window 示例">
  <EF:EFRegion id="region1">
    <div class="row">
      <EF:EFInput ename="GETD" cname="测试">
      </EF:EFInput>
    </div>
  </EF:EFRegion>
```

```

<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result">
    </EF:EFGrid>
  </EF:EFRegion>

<%— EEDM8010 页面加载时，不会加载 EEDM6000 (lazyload="true") --%>
<EF:EFWindow id="test" url="{ctx}/web/EEDM6000" lazyload="true">
</EF:EFWindow>

<%— EEDM8010 页面加载时，同时也会加载 EEDM6000 --%>
<EF:EFWindow id="preLoad" url="{ctx}/web/EEDM6000">
</EF:EFWindow>
</EF:EFPage>

```

3.5.1.6.3.2JS 开发

3.5.1.6.4 在当前页面内嵌 iframe 打开窗口，返回数据回填到 input 中

在打开 window 的关闭事件中处理：勾选所需数据后，关闭 iframe，回填数据。

3.5.1.6.4.1JS 开发

JS 示例

```

// 打开 EFWindow testWindow 是 kendo.ui.Window 实例，可以通过
$("#test").data("kendoWindow") 获取 kendoWindow 实例
$("#OPENW").click(function () {
  testWindow.open().center();
});

// 关闭事件
IPLATUI.EFWindow = {
  "test": {
    close: function (e) {
      var $iframe = testWindow.element.children("iframe");
      var iframejQuery = $iframe[0].contentWindow.$; // 子窗口中的 jQuery 对象

      // $iframe[0].contentWindow.resultGrid
      var dataGrid = iframejQuery("#ef_grid_result").data("kendoGrid");

      // 也可以使用如下的方式获取 dataGrid
      // iframejQuery.data($iframe.contents().find("#ef_grid_result")[0],
      "kendoGrid");

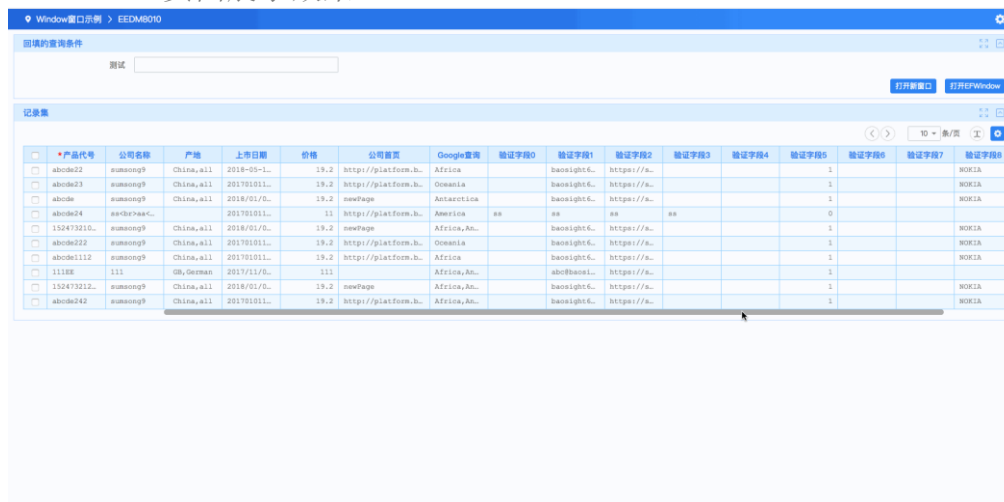
      var row = dataGrid.getCheckedRows();

      if (row.length > 0) {
        $("#GETD").val(row[0].company_name);
      } else {
        e.preventDefault(); // 没有勾选数据，将无法关闭弹窗
        NotificationUtil("没有勾选的行数据", "error");
      }
    }
  }
}

```

```
}
};
```

3.5.1.6.4.2 页面展示效果



3.5.1.6.5 打开新页面勾选 grid 中数据并返回

3.5.1.6.5.1 JS 开发

使用 `IPLAT.openNewForm` 打开指定页面

JS 示例

```
var openButton = $("#OPEN");
openButton.click(function () {
    var childWindow = IPLAT.openNewForm('EE10', "serviceName=EE10");
    childWindow.focus();
});
```

回填数据：勾选数据后，点击回填按钮，关闭窗口，原页面再将返回的数据回填到 grid 当中，并选中。

JS 示例

```
// 回填新页面返回的数据
onPopupReturn = function (winId, returnRows) {
    if (winId === "EE10") {
        var grid = $("#ef_grid_result").data("kendoGrid");

        // 回填
        grid.addRow(returnRows);
    }
};
```

3.5.1.6.5.2 页面展示效果

The screenshot shows a web application window titled 'EEDM8010'. It features a search bar at the top with the text '测试' and buttons for '打开新窗口' and '打开EFWindow'. Below the search bar is a table titled '记录集' (Record Set). The table has the following columns: 产品代号 (Product Code), 公司名称 (Company Name), 产地 (Origin), 上市日期 (Release Date), 价格 (Price), 公司网页 (Company Website), Google 查询 (Google Search), 验证字段0 (Verification Field 0), 验证字段1 (Verification Field 1), 验证字段2 (Verification Field 2), 验证字段3 (Verification Field 3), 验证字段4 (Verification Field 4), 验证字段5 (Verification Field 5), 验证字段6 (Verification Field 6), 验证字段7 (Verification Field 7), and 验证字段8 (Verification Field 8). The table contains several rows of data, including entries for 'abode2', 'abode3', 'abode', 'abode4', '132473210', 'abode222', 'abode1113', '11188', '132473212', and 'abode242'.

3.5.1.7 EEDM8011-Window 自定义标签回填示例

3.5.1.7.1 页面功能简介

本示例页面用于说明，EFWindow 内部如何使用其余标签，如何将弹出框内数据回填到指定位置。

3.5.1.7.2 配置和 API 详情

参考 [EFWindow](#) 文档。

3.5.1.7.3 页面开发步骤

3.5.1.7.3.1 JSP 页面开发

EFWindow 内部可直接使用其他标签。

JSP 示例

```
<EF:EFRegion id="region1">
  <div class="row">
    <EF:EFInput ename="GETD" cname="公司名称">
    </EF:EFInput>
  </div>
</EF:EFRegion>

<EF:EFWindow id="test">
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result"> // EFWindow 内部使用 EFGrid 标签
    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFWindow>

<EF:EFRegion id="region2">
  <EF:EFGrid blockId="blockT" needAuth="false">
  </EF:EFGrid>
```

```
</EF:EFRegion>
```

3.5.1.7.3.1.1 页面效果如下所示：

*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	验证字段7	验证字段8
abcde22	sumsong9	China,all	2018-05-1...	19.2	http://platform.b...	Africa		baosight6...	https://s...			1			NOKIA
abcde23	sumsong9	China,all	20170101L...	19.2	http://platform.b...	Oceania		baosight6...	https://s...			1			NOKIA
abcde	sumsong9	China,all	2018/01/0...	19.2	newPage	Antarctica		baosight6...	https://s...			1			NOKIA
abcde24	ssChr'sasC...		20170101L...	11	http://platform.b...	America	ss	ss	ss			0			
152473210...	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,An...		baosight6...	https://s...			1			NOKIA
abcde222	sumsong9	China,all	20170101L...	19.2	http://platform.b...	Oceania		baosight6...	https://s...			1			NOKIA
abcde1112	sumsong9	China,all	20170101L...	19.2	http://platform.b...	Africa		baosight6...	https://s...			1			NOKIA
1118E	111	GB,German	2017/11/0...	111		Africa,An...		abc@baosi...	https://s...			1			
152473212...	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,An...		baosight6...	https://s...			1			NOKIA
abcde242	sumsong9	China,all	20170101L...	19.2	http://platform.b...	Africa,An...		baosight6...	https://s...			1			NOKIA

3.5.1.7.3.2JS 开发

此页面可通过两种方式打开 Window：

第一种是通过点击 inqu 区域中的按钮。

JS 示例

```
// 打开 EFWindow 刷新 EFWindow 中的 EFGrid(注意：该场景下，resultGrid 和 blockTGrid 在同一个 window 对象中)
$("#OPENW").click(function () {
    resultGrid.dataSource.page(1);
    testWindow.open();
});
```



第二种是通过点击 grid 勾选行中的“validate_0”列，当“search”列的值为“window”时，EFWindow 会打开，其 grid 中的数据为根据该行的产品代码进行查询后的结果。

JS 示例

```
// 自定义 EFGrid 列的编辑器
IPLATUI.EFGrid = {
```

```
"blockT": {
  columns: [{
    field: "validate_0",

    // 重写 blockTGrid 中 validate_0 列的编辑器
    editor: function (container, options) {

      // 当前行的 search 列的内容是"window"时
      if (options.model.search === "window") {
        var ei = new EiInfo();
        ei.set("inqu_status", "0", "product_id", options.model.product_id);

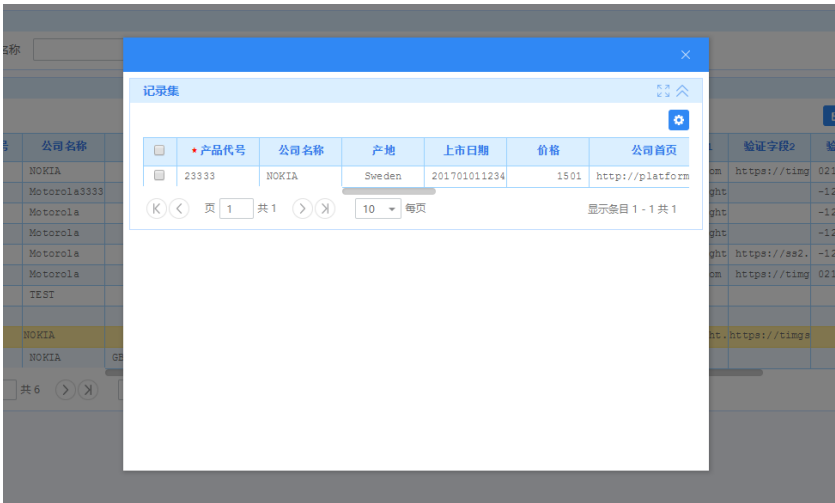
        // 发起请求
        EiCommunicator.send("EEDM8011", "query", ei, {
          onSuccess: function (ei) {
            // 请求结果回填 EWindow 中的 resultGrid
            resultGrid.setEiInfo(ei);

            testWindow.open();
          },
          onFail: function (ei) {
            NotificationUtil(ei);
          }
        });
      }
    }
  ]
}
};
```

3.5.1.7.3.2.1 页面展示效果

当 search 列为 window 时，点击验证字段 0（validate_0）即可弹出 window 窗口：

☐	产品代号	公司名称	产地	上市日期	价格	公司首页	Google 查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证号
☐	T666	NOKIA	111	201701011234	0.12	http://platform.baosight.com	查询	13800000000	aaa@123.com	https://tim	021-50801155	1	
☐	1234	Motorola3333	Rus	2017/07/04 0	2				abc@baosight		-12345678901		
☐	1231	Motorola	Canada	2007-07-12	2				abc@baosight		-12345678901		
☐	12341	Motorola	Canada	2007-07-12	2				abc@baosight		-12345678901		
☐	111	Motorola	China	2007-07-06	11				abc@baosight	https://se2.	-12345678901		
☐	XXM1	Motorola	Sweden	2007-03-17	1400	http://www.nokia.com	查询	13800000000	aaa@123.com	https://tim	021-50801155	1	
☐	666	TEST		201707040000	111	1	T6181						
☐	W7777			2017-07-19	0								
☒	23333	NOKIA	Sweden	2017010112342	1501	http://platform.baosight.com	window		abc@baosight	https://tim			
☐	scoc	NOKIA	GB,USA, Germa	2017/07/06 0	0								



3.5.1.8 EEDM8400-EFRegion 基础示例

3.5.1.8.1 页面功能简介

本示例页面用于说明，如何使用 **EFRegion** 及其基础属性配置。

3.5.1.8.2 配置和 API 详情

参考 [EFRegion](#) 中的描述。

3.5.1.8.3 页面开发步骤

3.5.1.8.3.1 JSP 页面开发

EFRegion 作为布局容器，主要用于组织页面元素，整体页面布局。

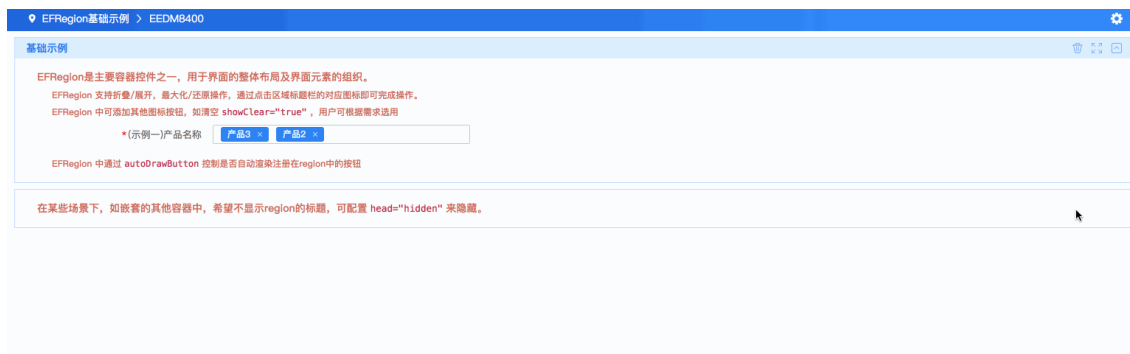
JSP 示例

```
<EF:EFRegion title="基础示例" id="inqu" showClear="true" autoDrawButton="true">
...
</EF:EFRegion>
<EF:EFRegion title="配置不显示标题" id="inqu2" head="hidden">
...
</EF:EFRegion>
```

- showClear可以控制region标题栏的清空按钮是否显示
- autoDrawButton可以控制是否自动渲染注册在region中的按钮
- head配置为hidden时可以隐藏region的标题栏

3.5.1.8.3.2 页面展示效果

1. showClear="true", head="hidden" 的展示:



2. class="block-opt-hidden"的展示：



3.5.1.9 EEDM8401-EFRegion 自适应示例

3.5.1.9.1 页面功能简介

本示例页面用于说明如何使用 **EFRegion** 进行布局。

3.5.1.9.2 配置和 API 详情

参考 [EFRegion](#) 中的描述。

3.5.1.9.3 页面开发步骤

3.5.1.9.3.1 JSP 示例

在布局的过程中会遇到需要自适应的场景，**EFRegion** 提供了 **fitHeight** 属性用于自适应到页面底部。

JSP 示例

```
<EF:EFPage>
  <div class="row">
    <div class="col-md-3">
      <EF:EFRegion id="tree" title="系统菜单树" fitHeight="true">
        <EF:EFTree id="tree01" textField="text" valueField="label"
          hasChildren="leaf"
            serviceName="EEDM05" methodName="query" style="height:100%">
        </EF:EFTree>
      </EF:EFRegion>
    </div>
```

```
<div class="col-md-9">
  <EF:EFRegion title="查询区" id="inqu">
    ...
  </EF:EFRegion>
  <EF:EFRegion title="结果集" id="result" fitHeight="true">
    <EF:EFGrid blockId="result">
      </EF:EFGrid>
    </EF:EFRegion>
  </div>
</div>
</EF:EFPage>
```

在示例中左侧为嵌套了树的region，在region配置了fitHeight="true"之后，内部的EFTree需看情况配置动态属性style="height:100%"。不配置动态属性时，tree的高度随展开的内容增加，超出region后会导致region出现纵向滚动条，如果某一树节点文字过长，树上出现横向滚动条，此时需要将region滚动条拖到下方才能拖动横向滚动条。style="height:100%"可以使树的高度固定，展开多层后纵向滚动条出现在树的div上而不是region上，避免前面的问题出现。

页面展示效果：



3.5.1.10 EEDM8500-EFPage 属性示例

3.5.1.10.1 页面功能简介

本示例页面用于说明，EFPage 标签的作用以及其属性的作用。

3.5.1.10.2 配置和 API 详情

参考 [EFPage](#)，[EFPage 页面标签简介](#)，[EFPage 中添加自定义 CSS 和 JS 中的描述](#)。

3.5.1.10.3 页面开发步骤

3.5.1.10.3.1 EFPage 的作用

作用：**EFPage** 是平台提供的统一的 JSP 页面模版，用于快速创建一个符合平台开发规范的 JSP 页面；

优势：使用 **EFPage** 标签，可使页面的布局更加简洁高效，不再需要从 `<head>` `<body>` `<foot>` 一层层往下写；也可以保证各个开发人员的页面是一致的，降低页面的维护和调整的成本；

基础使用示例：

EFPage

```
<EF:EFPage>
...
</EF:EFPage>
```

3.5.1.10.3.2 EFPage 的属性

EFPage 共有四个属性：**title**，**header**，**footer**，**prefix**。其中 **title** 用于显示页面标题，不推荐使用，默认使用 **EDFA00** 注册页面时配置。下图为 **EDFA00** 注册页面。



3.5.1.10.3.2.1 2.1 header 属性

EFPage 的作用就是平台将主题，风格样式都封装在 **EFPage.tag** 里面，不需要每个页面都去引入 **CSS** 和 **JS** 文件。那如果开发人员想自己引入一些自己的 **CSS** 和 **JS** 怎么处理呢？

开发场景：当开发者想要针对某个页面，自定义页面中某些组件的样式时，那就可以利用 **header** 属性

使用方式：`<jsp:attribute name="header"> <style> ... </style> </jsp:attribute>`

EFPage

```
<EF:EFPage>
  <jsp:attribute name="header">
    <style>
      .p-text {
        color: #FF6160;
      }
    </style>
  </jsp:attribute>
</EF:EFPage>
```

```

        font-size: 18px;
    }
</style>
</jsp:attribute>
</jsp:body>
.....
</jsp:body>
</EF:EFPage>

```



在本例中，页面引用了<jsp:attribute name="header">标签，利用这个接口，将自定义的 CSS 代码添加到此页面中。

页面效果如下所示：

Page的属性一：header

在EFPage中，我们使用header属性,把我們想在此页面自定义的样式添加进去

如：在JSP页面中引入文字的样式：`class="p-text"`，使得字体颜色为红色，大小为18px

欢迎使用iPlat!

红色字体：“欢迎使用 iPlat!” 就是使用了自定义样式。



生成后的<style>的位置，如下图所示：

```

<!--[if gt IE 9]><!-->
<link rel="stylesheet" href="/showcase/iplatui/assets/css/iplat.ui.bootstrap.min.css">
<!--<![endif]>-->
<link rel="stylesheet" href="/showcase/kendoui/styles/kendo.common.min.css">
<link href="/showcase/kendoui/styles/kendo.metro.min.css" rel="stylesheet">
<link id="themeAnchor" rel="stylesheet" href="/showcase/iplatui/css/iplat.ui.ant.css">
<link href="/showcase/iplatui/assets/js/plugins/oneui/plugins/magnific-popup/magnific-popup.c
<!--[if lte IE 8]>
<link rel="stylesheet" href="/showcase/iplatui/css/iplat.ui.ie.css">
<![endif]>-->
<script src="/showcase/kendoui/js/jquery.min.js"></script>
<script src="/showcase/iplatui/js/lib/underscore.min.js"></script>
<script src="/showcase/iplatui/js/iplat.ui.config.js"></script>
<style>
    .p-text {
        color: #FF6160;
        font-size: 18px;
    }
</style>
<link href="/showcase/itemui/css/item.common.css" rel="stylesheet">
</head>
<body class="i-theme-ant"> == $0
<div id="notification-util" data-role="notification" style="display: none;">
</div>
<div id="window-util">
</div>
<div id="help-window">

```

3.5.1.10.3.2.2 2.2 footer 属性

开发场景：当开发者想要针对某个页面，制定页面中某些功能时，那就可以利用 footer 属性。

使用方式：<jsp:attribute name="footer"> <script> ... </script> </jsp:attribute>

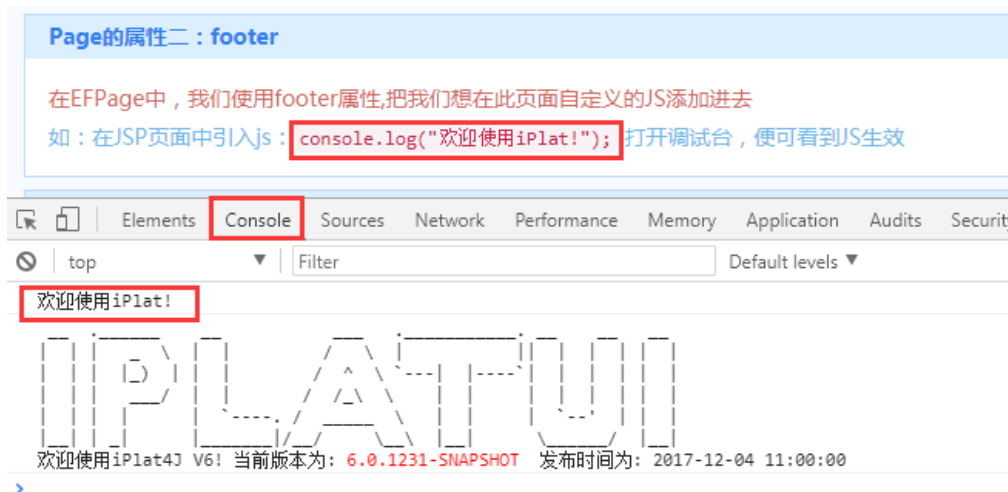
EFPage

```
<EF:EFPage>
  <jsp:body>
    .....
  </jsp:body>
  <jsp:attribute name="footer">
    <script>
      console.log("欢迎使用 iPlat!");
    </script>
  </jsp:attribute>
</EF:EFPage>
```



在本例中，页面引用了<jsp:attribute name="footer">标签，利用这个接口，将自定义的 js 代码添加到此页面中。

页面效果如下所示：



调试台中的文字：“欢迎使用 iPlat!”，就是通过自定义 js 完成的。



生成后的<script>位置如下图所示：

```
<script id="date-footer-template" type="text/x-kendo-template">...</script>
<script id="grid-pager-template" type="text/x-kendo-template">...</script>
<script id="grid-count-template" type="text/x-kendo-template">...</script>
<script id="del-template" type="text/x-kendo-template">...</script>
<script id="add-template" type="text/x-kendo-template">...</script>
<script id="alert-template" type="text/x-kendo-template">...</script>
<script id="confirm-template" type="text/x-kendo-template">...</script>
<script id="prompt-template" type="text/x-kendo-template">...</script>
<script id="msg-template" type="text/x-kendo-template">...</script>
<script id="notification-template" type="text/x-kendo-template">...</script>
<script id="sum-template" type="text/x-kendo-template">...</script>
<script id="personal-grid-template" type="text/x-kendo-template">...</script>
<div id="page-container">...</div>
<script>
  console.log("欢迎使用 iPlat!");
</script>
<script src="/showcase/itemui/js/item.common.js"></script>
<script type="text/javascript">...</script>
</body>
```



一旦使用了 `header` 或者 `footer` 属性, 通常需要注意两点

1. `attribute` 中不管是添加 `CSS` 还是 `JS` 都需要表明标签域, 也就是 `<style>.....</style>` 或者是 `<script>.....</script>`
2. 页面内容需要使用 `<jsp:body></jsp:body>` 标签将其包含起来才能使之生效, 不然页面会报错。

EFPage

```
<EF:EFPage>
  <jsp:attribute name="header">
    <style>
      .....
    </style>
  </jsp:attribute>

  <jsp:attribute name="footer">
    <script>
      .....
    </script>
  </jsp:attribute>
  <jsp:body>
    .....
  </jsp:body>
</EF:EFPage>
```

2.3 prefix 属性

开发场景: 不是某个页面需要自定义样式和 `JS`, 而是批量的页面 (模块) 需要自定义 `CSS` 和 `JS` 时, 需要用到 `prefix` 属性

使用方式: `<EF:EFPage prefix="XX">`

具体使用详情参考★: [EEDM8501-EFPage 中 prefix 配置示例](#)

3.5.1.11 EEDM8501-EFPage 中 prefix 配置示例

3.5.1.11.1 页面功能简介

本示例页面用于说明: `EFPage` 的 `prefix` 属性的用法。

`EFPage` 提供的是平台默认的页面模板, 当项目需要批量为某个模块或者某类页面统一添加公共的 `JS` 或者 `CSS` 时, 可以设置 `EFPage` 的 `prefix` 属性, 创建对应的 `jsp` 片段文件, 然后在这些 `jsp` 文件中, 引入项目所需的公共 `CSS` 和 `JS`。

3.5.1.11.2 配置和 API 详情

参考 [EFPage](#), [UI 常见开发问题 FAQ: Q3, Q4](#), [EEDM8500-EFPage 属性示例](#), [iPlatUI 主题定制方案](#) 中的描述。

3.5.1.11.3 页面开发步骤

prefix 的具体使用场景：

1. 项目定制的 CSS： 对项目的某些模块/某类页面的样式进行定制；（单个页面的定制使用 [EFPAGE](#) 的 header 和 footer 属性）
2. 项目定制的 JS： 项目中的公共的业务方法 API 等；例如，业务上封装的公共 JS 方法。

3.5.1.11.3.1 项目定制的 CSS

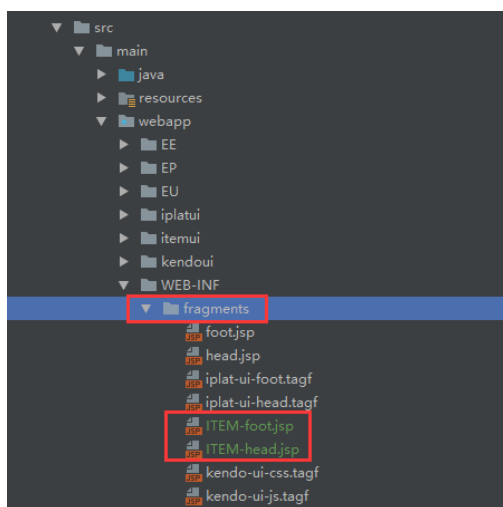
首先，设置 EFPAGE 的 prefix 属性，通常为项目代号，例如: PSCS, BSMS 等；【注意：prefix 属性大小写敏感，创建对应的 jsp 文件时，文件名的前半部分应该和 prefix 的值保持一致】

然后，在 /WEB-INF/fragments 目录下新建 \${prefix}-head.jsp 和 \${prefix}-foot.jsp 文件；【注意：必须同时创建 \${prefix}-head.jsp 和 \${prefix}-foot.jsp 文件】

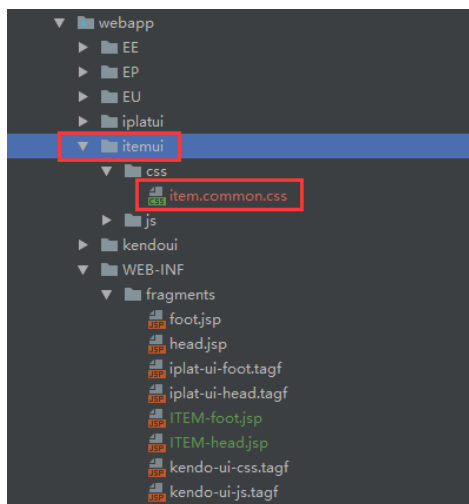
在 \${prefix}-head.jsp 中加入 CSS 文件；(注意：在 \${prefix}-head.jsp 中覆盖 IPLATUI.Config 的全局配置)

在 \${prefix}-foot.jsp 中加入业务封装的公共 JS 文件。

例如：设置 prefix 为 ITEM，在 /WEB-INF/fragments 下创建 ITEM-head.jsp, ITEM-foot.jsp，如下图所示：



创建项目公用的 CSS 文件，为了工程目录结构的清晰明了，建议在 /webapp 下创建文件夹：itemui（项目前缀+ui）来存放 CSS 文件，写入需要定制的风格。文件目录如下图所示：



开发项目自定义的 CSS, item.common.css 【示例代码，严禁直接复制】

item.common.css

```
li.circle {
  color: red;
  font-size: 14px;
}
```

将 CSS 文件引入 ITEM-head.jsp。

ITEM-head.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:set var="ctx" value="${pageContext.request.contextPath}" />
<%-- 引入项目定制的公共 CSS --%>
<link href="${ctx}/itemui/css/item.common.css" rel="stylesheet">

<%-- 少量的样式，可以在 head.jsp 中直接定义 --%>
<style>
li.square {
  color: blue;
}
</style>
```

这样，只需在 EFPage 标签中，设置 prefix="ITEM"，就可以使用定制的 CSS 样式了，从而可以覆盖平台默认的 CSS 样式。

EEDM8501

```
<EF:EFPage prefix="ITEM">
  <EF:EFRegion id="r1" title="在\${prefix}-head.jsp 中引入 CSS 文件，更改样式">
    <h5 class="text-info">自定义更改列表的样式：第一个列表为原始列表样式，第二个列表是引入 CSS 后的样式</h5>
    <ul>
      <li>苹果</li>
      <li>香蕉</li>
    </ul>
```

```

<ul class="circle"> // circle 为 css 文件中自定义的样式
  <li>苹果</li>
  <li>香蕉</li>
</ul>
</EF:EFRegion>
</EF:EFPage>

```

在本例中，第二个列表 **circle** 是自定义的 CSS 文件的样式，页面效果如下所示：

在\${prefix}-head.jsp中引入CSS文件，更改样式

考虑到开发者偶尔会有自定义定制CSS的需求，覆盖平台封装的CSS样式，推荐在webapp下创建项目目录：itemui-css文件，再引入到ITEM-head.jsp中自定义更改列表的样式：第一个列表为原始列表样式，第二个列表是引入CSS后的样式

- 苹果
- 香蕉

- 苹果
- 香蕉

3.5.1.11.3.1.1 组件的全局配置

除了定制 CSS 之外，还可以在\${prefix}-head.jsp 中对组件进行全局配置。



[iPlatUI 的全局配置项说明](#) 讲解了如何在 **head.jsp** 中进行组件的全局配置。

当页面配置了 **prefix** 属性时，**注意** 需要在\${prefix}-head.jsp 中覆盖 **IPLATUI.Config** 中的原始配置。

下面的示例为：在\${prefix}-head.jsp （ITEM-head.jsp）中，覆盖 EFGrid 组件：翻页条默认的全局配置。

ITEM-head.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:set var="ctx" value="${pageContext.request.contextPath}"/>

<%-- 引入项目定制的公共 CSS --%>
<link href="${ctx}/itemui/css/item.common.css" rel="stylesheet">

<%-- 少量的样式，可以在 head.jsp 中直接定义 --%>
<style>
...
</style>
<%-- 覆盖默认的 IPLATUI.Config 配置 --%>
<script type="text/javascript">
$.extend(true, IPLATUI.Config, { //全局配置
  EFGrid: {
    pageable: {
      input: true, //可输入页码的输入框，输入页码跳转，不可和 numeric 同时使用

```

```
numeric: false,
pageSize: 10, // 每页显示的条数, 优先级低于 DataSource 中的 pageSize
pageSizes: [5,10,20,50]
}
}
});
</script>
```

页面效果如下所示:

1 / 4页

10 条/页

共33条

☐	公司名称	产地	上市日期	价格		公司首页
☐	MotorolaInj112	China,German,GB	20070427123456222	150122	5	
☐	MotorolaInj111	China,German,GB	2017/11/01 00:00:00	150122	10 ✓	
☐	MotorolaInj111	China,German,GB	2017/11/01 00:00:00	150122	20	
☐	MotorolaInj111	China,German,GB	2017/11/01 00:00:00	150122	50	
☐	MotorolaInj111	China,German,GB	2017/11/01 00:00:00	150122	EEE1	
☐	111	USA1	2017/11/07 00:00:00	11		
☐	AAAAA	China	2017/12/15 00:00:00	0		
☐	MotorolaInj111	China,German,GB	2017/11/01 00:00:00	150122	EEE1	
☐	MotorolaInj113	China,GB	2017/11/22 00:00:00	150122	EEE	
☐	MotorolaInj111	China,GB	2017/11/16 00:00:00	150122	EEE	
☐	Samsung	Korea	2007-04-06	10.14		http222wwwsamsungcom22

在本例中，在 ITEM-head.jsp 中对 EFGGrid 进行全局配置，设置翻页条：打开可输入页码的输入框；每页显示 10 条记录；分页设置为：[5, 10, 20, 50]

3.5.1.11.3.2 项目定制的 JS

开发步骤同 CSS 顺序一致：

- 1. 在 /WEB-INF/fragments 下创建 ITEM-foot.jsp；
- 2. 在 /webapp 下创建的文件夹 itemui 中新建 JS 文件夹和项目公共的 JS 文件；
- 3. 在项目公共的 JS 文件中书写项目定制公共方法；
- 4. 在 ITEM-foot.jsp 中引入项目公共的 JS 文件或者引入第三方的 js 库；

开发项目公共的 JS 方法【示例代码，严禁直接复制】

item.common.js

```
(function ($) {
    var ITEM = window.ITEM || {};
    // 定义项目的公共 API
    ITEM.helloTest = function () {
        IPLAT.alert({
            title: '开发项目公共的 JS',
            message: 'hello world'
        });
    };
});

$.extend(window, {
    ITEM: ITEM
});
```

```
})(jQuery);
```

项目公共的 JS 文件开发结束后，引入到 **ITEM-foot.jsp** 中。

ITEM-foot.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:set var="ctx" value="${pageContext.request.contextPath}" />

<%-- 引入项目公共的 JS --%>
<script src="${ctx}/itemui/js/item.common.js"></script>

<%-- 引入第三方的 JS 库 --%>
<script src="${ctx}/iplatui/js/lib/echarts.min.js"></script>
```

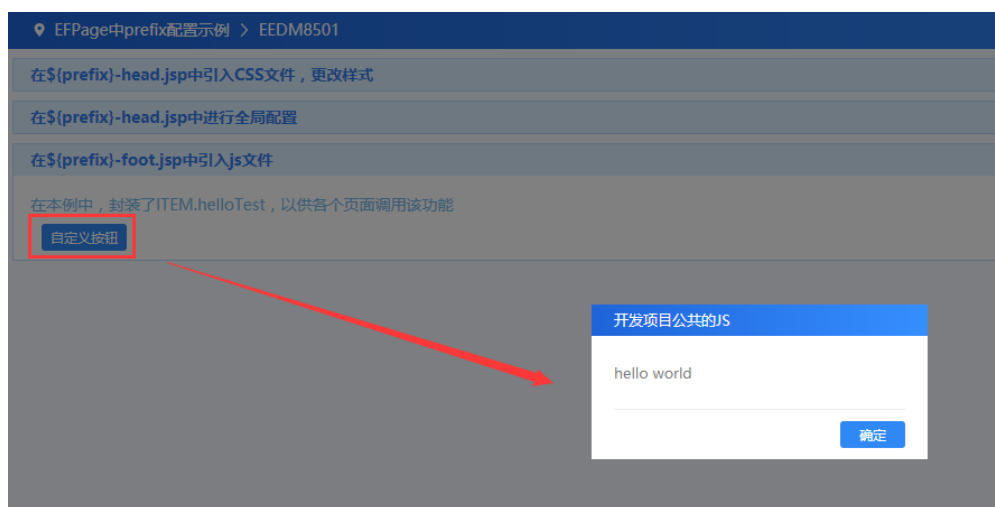
示例页面：

EEDM8501

```
<EF:EFPage prefix="ITEM">
  <EF:EFRegion id="r2" title="在\${prefix}-foot.jsp 中引入 js 文件">
    <button id="inqu_status-0-row" class="i-btn-lg">自定义按钮</button>
  </EF:EFRegion>

  <script>
  $(function() {
    $("#inqu_status-0-row").on("click", function() {
      // 调用在\${prefix}-foot.jsp 中定义的 项目所需的公共方法
      ITEM.helloTest();
    });
  });
</script>
</EF:EFPage>
```

页面效果如下所示：



3.5.1.12 EEDM9030-容器嵌套布局示例页面

3.5.1.12.1 页面功能简介

本示例页面用于说明，EFTab 内部嵌套 EFGrid 时，几种不同的布局方式。

3.5.1.12.2 配置和 API 详情

参考 [EFTab](#) , [EFGrid](#) 文档。

3.5.1.12.3 页面开发步骤

3.5.1.12.3.1 布局 1

查询区域无 EFRegion, Grid 有 EFRegion。

JSP 页面开发

布局 1: JSP 示例

```
<div title="Tab 嵌套 Grid 样式 1">
  <div class="i-space">
    <EF:EFInput cname="产品代号" ename="product_id" row="0" blockId="inqu_status"/>
    <EF:EFInput cname="公司名称" ename="inqu_status-0-company_name"/>
    <div align="right">
      <EF:EFButton ename="F1" cname="查询" layout="1" class="k-button">
    </EF:EFButton>
    </div>
  </div>
  <EF:EFRegion id="grid_region">
    <EF:EFGrid blockId="result" needAuth="false">
    </EF:EFGrid>
  </EF:EFRegion>
</div>
```

页面效果如下所示:

Tab嵌套Grid样式1

Tab嵌套Grid样式2

Tab嵌套Grid样式3

Tab内嵌套Grid样式4

产品代号

公司名称

查询

新增

1

共2

10

每页

共20 条

保存

取消

删除

* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证
N70	MotorolaIn...	china	2007-04-27...	150122	R/RB	N70		abc@baosig...	asdasdas			
8251	MotorolaIn...	china	2007-03-17	1400	EL23	查询	13800000000	aaa@123.com	验证上	021-508011...	1	
T6181	Motorola	china	2007-04-06	3550	时间访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
SGH-17161	Samsung	Korea	2007-04-06	10.14	http://www.samsun...	查询	13800000000	aaa@123.com	fuyimin	021-508011...	1	
T6281	SonyEricss...	Japan	2007-04-06	30	http://www.sonyeric...	查询	13800000000	aaa@123.com	label1	021-508011...	1	
N701	NOKIA	Sweden	2007-04-06	0.12		search		abc@baosig...				
88501	SonyEricss...	Korea	2007-04-27	0.12		1		abc@baosig...				
1112	Motorola	China	2007-07-06	11	EWF087			abc@baosig...	11	-123456789...		
S891	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13800000000	aaa@123.com	label1	021-508011...	1	
8825	SonyEricss...	Japan	2007/03/10	20000		search		abc@baosig...	11	021-508011...		

3.5.1.12.3.2 布局 2

查询区域有 Region, Grid 无 Region。

JSP 页面开发

布局 2: JSP 示例

```
<div title="Tab 嵌套 Grid 样式 2">
  <EF:EFRegion id="inqu_region">
    <div style="margin-top: 8px;margin-bottom: 20px">
      <EF:EFInput cname="产品代号" ename="product_id" row="0"
blockId="inqu_status"/>
      <EF:EFInput cname="公司名称" ename="inqu_status-0-company_name"/>
      <div align="right">
        <EF:EFButton ename="F2" cname="查询" layout="1" class="k-button">
      </EF:EFButton>
      </div>
    </div>
  </div>
  <hr>
  <EF:EFGrid blockId="resultA" needAuth="false">
  </EF:EFGrid>
</EF:EFRegion>
</div>
```

页面效果如下所示:

产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证
M70	MotorolaIn...	china	2007-04-27...	150122	RMB	M70	abc@baosig...	asdasdas				
8251	MotorolaInj	china	2007-03-17	1400	EL23	查询	13000000000	aaa@123.com	爱上海	021-508011...	1	
T6181	Motorola1	china	2007-04-06	3550	时段访问量统计	查询	13000000000	aaa@123.com		021-508011...	1	
SGH-i7161	Samsung	Korea	2007-04-06	10.14	http://www.samsu...	查询	13000000000	aaa@123.com	fuyimin	021-508011...	1	
T6281	SonyEricss...	Japan	2007-04-06	30	http://www.sonyer...	查询	13000000000	aaa@123.com	label	021-508011...	1	
N701	NOKIA	Sweden	2007-04-06	0.12		search		abc@baosig...				
88501	SonyEricss...	Korea	2007-04-27	0.12		1		abc@baosig...				
1112	Motorola	China	2007-07-06	11	ENFM07			abc@baosig...	11	-123456789...		
S891	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13000000000	aaa@123.com	label	021-508011...	1	
8825	SonyEricss...	Japan	2007/03/10	20000		search		abc@baosig...	11	021-508011...		

3.5.1.12.3.3 布局 3

查询区域和 Grid 区域都无 Region。

JSP 页面开发

布局 3: JSP 示例

```
<div title="Tab 嵌套 Grid 样式 3">
  <div class="i-space">
    <EF:EFInput cname="产品代号" ename="product_id" row="0" blockId="inqu_status"/>
    <EF:EFInput cname="公司名称" ename="inqu_status-0-company_name"/>
    <div align="right">
      <EF:EFButton ename="F3" cname="查询" layout="1" class="k-button">
    </EF:EFButton>
    </div>
  </div>
  <hr>
  <div style="padding: 8px 15px">
    <EF:EFGrid blockId="resultB" needAuth="false">
  </EF:EFGrid>
  </div>
```

```
</EF:EFGrid>
</div>
```

页面效果如下所示：

Tab嵌套Grid样式1

Tab嵌套Grid样式2

Tab嵌套Grid样式3

Tab内嵌套Grid样式4

产品代号

公司名称

查询

新增

(K) < 页 1 共 2 > (X)

10 每页 共 20 条

保存

取消

删除

* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验
N70	MotorolaIn...	china	2007-04-27...	150122	RMB	N70		abc@baosig...	asdasdas			
8251	MotorolaInj	china	2007-03-17	1400	EL23	查询	13000000000	aaa@123.com	验证上	021-508011...	1	
T6181	MotorolaI	china	2007-04-06	3550	时间访问量统计	查询	13000000000	aaa@123.com		021-508011...	1	
SGH-I7161	Samsung	Korea	2007-04-06	10.14	http://www.samsu...	查询	13000000000	aaa@123.com	fuyimin	021-508011...	1	
T6281	SonyEricss...	Japan	2007-04-06	30	http://www.sonyer...	查询	13000000000	aaa@123.com	label	021-508011...	1	
N701	NOKIA	Sweden	2007-04-06	0.12		search		abc@baosig...				
88501	SonyEricss...	Korea	2007-04-27	0.12		1		abc@baosig...				
1112	Motorola	China	2007-07-06	11	EHFM07			abc@baosig...	11	-123456789...		
5091	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13000000000	aaa@123.com	label	021-508011...	1	
8825	SonyEricss...	Japan	2007/03/10	20000		search		abc@baosig...	11	021-508011...		

3.5.1.12.3.4 布局 4

查询区域和 Grid 都有 Region。

JSP 页面开发

布局 4：JSP 示例

```
<div title="Tab 内嵌套 Grid 样式 4">
  <EF:EFRegion id="all_region1">
    <div style="margin-bottom: 0">
      <EF:EFInput cname="产品代号" ename="product_id" row="0"
blockId="inqu_status"/>
      <EF:EFInput cname="公司名称" ename="inqu_status-0-company_name"/>
      <div align="right">
        <EF:EFButton ename="F4" cname="查询" layout="1" class="k-button">
          </EF:EFButton>
      </div>
    </div>
  </EF:EFRegion>
  <EF:EFRegion id="all_region2">
    <EF:EFGrid blockId="resultC" needAuth="false">
      </EF:EFGrid>
    </EF:EFRegion>
  </div>
```

页面效果如下所示：



3.5.1.12.3.5 布局 5

Tab 内套嵌套 tab, tab 内嵌套 Grid。

JSP 页面开发

布局 5：JSP 示例

```
<div title="Tab 内嵌套 Grid 样式 5">
  <EF:EFRegion id="tab-inqu">
    <div style="margin-bottom: 0">
      <EF:EFInput cname="产品代号" ename="product_id" row="0"
      blockId="inqu_status"/>
      <EF:EFInput cname="公司名称" ename="inqu_status-0-company_name"/>
      <div align="right">
        <EF:EFButton ename="F5" cname="查询" layout="1" class="k-button">
        </EF:EFButton>
      </div>
    </div>
  </EF:EFRegion>
  <EF:EFTab id="tab-tab_grid">
    <div title="公司信息">
      <EF:EFGrid blockId="resultD" needAuth="false">
      </EF:EFGrid>
    </div>
    <div title="产品信息">
      <EF:EFGrid blockId="resultE" needAuth="false">
      </EF:EFGrid>
    </div>
  </EF:EFTab>
</div>
```

页面效果如下所示：

Tab嵌套Grid样式1Tab嵌套Grid样式2Tab嵌套Grid样式3Tab内嵌套Grid样式4Tab内嵌套Grid样式5

产品代号

公司名称

查询

公司信息

产品信息

新增

<>10条/页

保存取消删除

	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证
	N70	MotorolaIn...	china	2007-04-27...	150122	ER90	N70		abc@baosig...	asdasdas			
	8251	Motorolanj	china	2007-03-17	1400	EL23	查询	13800000000	aaa@123.com	乘上	021-508011...	1	
	T6181	Motorola1	china	2007-04-06	3550	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
	5GH-i7161	Samsung	Korea	2007-04-06	10.14	http://www.samsung...	查询	13800000000	aaa@123.com	fuyimin	021-508011...	1	
	T6281	SonyEricss...	Japan	2007-04-06	30	http://www.sonyeric...	查询	13800000000	aaa@123.com	label	021-508011...	1	
	88501	SonyEricss...	Korea	2007-04-27	0.12		1		abc@baosig...				
	5891	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13800000000	aaa@123.com	label	021-508011...	1	
	8825	SonyEricss...	Japan	2007/03/10	20000		search		abc@baosig...	11	021-508011...		
	123	Motorola	Canada	2007-07-12	2				abc@baosi...		-123456789...		
	1234	Motorola	Canada	2007-07-12	2				abc@baosig...				

 为了美观和空间利用率，tab 外层不加 Region。

3.5.2 表单组件

3.5.2.1 EEDM1002-EiColumn 动态属性示例

3.5.2.1.1 页面功能简介

本示例页面用于说明，如何对 EiColumn 进行动态属性设置。

3.5.2.1.2 配置和 API 详情

参考 [EiColumn 详解](#) 中的描述。

3.5.2.1.3 页面开发步骤

3.5.2.1.3.1 JSP 页面开发

设置的 EiColumn 动态属性值在<EF:EFInput ename="eiColumnAttr" cname="数据 3"></EF:EFInput>中获取展示，其余 EFInput 使用三段式展示值；

EFGGrid 测试后台设置的 EiColumn 动态属性(如固定列"locked"，数据校验，只读或不可编辑)在 Grid 中所起的效果。

EiColumn 动态属性示例

```
<EF:EFRegion id="inqu" title="查询条件" type="query" efRegionShowClear="true"
efRegionSave="true">
  <div class="row">
    <EF:EFInput cname="数据列 1" ename="eiColumn" row="0" blockId="test">
    </EF:EFInput>
    <EF:EFInput cname="数据列 2" ename="eiColumn2" row="0" blockId="test">
    </EF:EFInput>
  </div>
  <div class="row">
    <EF:EFInput ename="test-blockAttr" cname="二段式">
    </EF:EFInput>
    <EF:EFInput ename="eiinfoAttr" cname="一段式">
  </div>
</EF:EFRegion>
```

```

        </EF:EFInput>
        <EF:EFInput ename="eiColumnAttr" cname="数据 3">
        </EF:EFInput>
    </div>
</EF:EFRegion>

<EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" autoDraw="true" needAuth="false">
    </EF:EFGrid>
</EF:EFRegion>

// JS 代码
// 后端 set 自定义属性是否得到
var attr = __eiInfo.getBlock("test").getColumn("eiColumn")['attr'];
// EiCcolumn 自定义属性值回填
$("#eiColumnAttr").val(attr['AA']);

```

3.5.2.1.3.2 Service 服务开发

在 `ServiceEEDM1002.java` 文件中，设置列的相关动态属性。

1) 根据传回的 `EiInfo` 参数，调用 `super.query(EiInfo inInfo, String sqlName, DaoEPBase bean)` 查询出符合条件的结果 `outInfo`，并 `set` 其中部分列的动态属性，如将"price"设为固定列、"company_page"设置校验规则等。

query

```

public EiInfo query(EiInfo inInfo) {
    EiInfo outInfo = super.query(inInfo, "EE10.query", new TEE10());

    // 设置固定列
    outInfo.getBlock("result").getBlockMeta().getMeta("price").set("locked", true);

    // 校验规则

    outInfo.getBlock("result").getBlockMeta().getMeta("company_page").setValidateType("email");
    // 正则校验

    outInfo.getBlock("result").getBlockMeta().getMeta("validate_0").setRegex("/^(100|[1-9]\\d|\\d\\d)$/") ; //0-100 数字

    outInfo.getBlock("result").getBlockMeta().getMeta("validate_0").setValidateErrorPrompt("请输入 0-100 的数字");

    // 设置只读
    outInfo.getBlock("result").getBlockMeta().getMeta("validate_1").setReadonly(true);
    // 设置不可编辑
    outInfo.getBlock("result").getBlockMeta().getMeta("validate_2").set("enable", false);

    return outInfo;
}

```

2) `initLoad` 方法对 `eiColumn` 列设置自定义属性"AA"并在前端获取值，将 `query` 方法获取到的 `block` 添加到结果 `eiInfo` 中返回。

initLoad

```
public EiInfo initLoad(EiInfo inInfo) {
    EiInfo eiInfo = new EiInfo("eiInfo");
    EiBlock eiBlock = new EiBlock("test");

    EiColumn eiColumn = new EiColumn("eiColumn");
    EiColumn eiColumn2 = new EiColumn("eiColumn2");

    eiColumn.setDescName("数据列");
    eiColumn2.setDescName("数据列 2");

    eiBlock.addMeta(eiColumn);
    eiBlock.addMeta(eiColumn2);

    Map<String, String> row = new HashMap<String, String>();
    row.put("eiColumn", "数据");
    row.put("eiColumn2", "数据 2");
    eiBlock.addRow(row);

    // 后端 set 自定义属性，验证是否可以传到前端
    eiColumn.set("AA", "自定义属性 AA");

    Map<String, String> rowAttr2 = new HashMap<String, String>();
    rowAttr2.put("blockAttr", "二段");
    eiBlock.setAttr(rowAttr2);

    eiInfo.setMsg("一段");
    Map<String, String> rowAttr = new HashMap<String, String>();
    rowAttr.put("eiinfoAttr", "一段");
    eiInfo.setAttr(rowAttr);
    eiInfo.addBlock(eiBlock);

    EiInfo gridInfo = query(inInfo);
    eiInfo.addBlock(gridInfo.getBlock("result"));

    return eiInfo;
}
```

3.5.2.1.3.3 页面展示效果

The screenshot displays the iPlatUI web application interface. At the top, there are search filters for '数据列1' (Data Column 1) and '数据列2' (Data Column 2). Below these, a table titled '记录集' (Record Set) is shown. The table has columns for '产品代号' (Product Code), '公司名称' (Company Name), '价格' (Price), '产地' (Origin), '上市日期' (Release Date), '公司首页' (Company Homepage), 'Google查询' (Google Search), and several '验证字段' (Verification Fields). The table contains 10 rows of data, including entries for 'asdaf', '23444', 'N701', 'HK-T11', '1112', 'S891', '8825', 'T666', '1234', and '1231'. A red tooltip提示 '请输入0-100的数字' (Please enter a number between 0 and 100) is visible over the '验证字段0' column. At the bottom, there is a pagination bar showing '共 5 页' (Total 5 pages) and '显示条目 1 - 10 共 48' (Display items 1 - 10 of 48).

3.5.2.2 EFInput 示例

3.5.2.2.1 EEDM4000-EFInput 数据绑定

3.5.2.2.1.1 页面功能简介

本示例页面用于说明：

1. EFInput 如何进行数据绑定：具体可参考[数据交换中间件 EiInfo 的使用](#)中的三段式绑定相关。
2. 如何进行布局：了解 EFInput 的属性及其作用。
3. Input 的状态：默认状态；readonly（只读状态）；disabled（禁用状态）；required（*必填状态）。

3.5.2.2.1.2 配置和 API 详情

参考 [EFInput](#) 中的描述。

3.5.2.2.1.3 页面开发步骤

3.5.2.2.1.3.1 数据绑定

3.5.2.2.1.3.1.1 JSP 页面开发

前台页面对 EFInput 标签中 **ename** 属性值三段式定义，可以让对应数据渲染到 input 框内。

其中，“一段式绑定”直接从 **attr** 属性区获得 EiInfo 的属性值；“两段式绑定”从对应 **blockId** 的 **attr** 区获得 EiBlock 的属性值；“三段式绑定”获得对应 **blockId** 的对应列名的第 **rowNo** 行数据。

JSP 代码

```
<EF:EFRegion id="R8" title="三段式绑定 EiInfo">
  <div class="row">
    <EF:EFInput ename="info" cname="绑定 EiInfo 属性区" colWidth="6"/> //一段式
    <EF:EFInput ename="result-block" cname="绑定 EiBlock 属性区" colWidth="6"/> //二
    <EF:EFInput ename="result-0-data" cname="绑定 EiBlock 数据区" colWidth="6"/> //
    <EF:EFInput ename="result-0-data1" cname="绑定 EiBlock 数据区" colWidth="6"/>
    <EF:EFInput ename="result-1-data" cname="绑定 EiBlock 数据区" colWidth="6"/>
    <EF:EFInput ename="result-1-data1" cname="绑定 EiBlock 数据区" colWidth="6"/>
    <EF:EFInput ename="result-1-data2" x="1" y="2" class="k-state-hover"
    readonly="true" cname="默认值优先"
      value="默认值" colWidth="6"/>
    <EF:EFInput ename="result-1-data3" style="color:red;" readonly="true"
      cname="动态属性" value="红色的文字" colWidth="6"/>
  </div>
  <p class="text-info"> 前端的数据绑定到 EiInfo 对象，在 EiInfo 测试页面展示 </p>
</EF:EFRegion>
```

1. EiInfo 标签内若定义 value 属性，则其优先级会强于三段式数据绑定，input 框显示 value 的定义值。

默认值优先 默认值

2. 定义的 class 样式会默认追加 k-textbox 样式。

```
<div class="col-xs-8"> == $0
<input id="result-1-data3" name="result-1-data3" value="红色的文字" type="text" style="color:red;
class="k-textbox i-state-readonly i-input-readonly">
```

3.5.2.2.1.3.1.2 Service 服务开发

后台 service 的 initLoad 方法设置 EiInfo 对象内的数据：

Service 代码

```
public EiInfo initLoad(EiInfo inInfo) {

    EiInfo outInfo = new EiInfo("输出 outInfo");
    EiBlock result = new EiBlock("result"); // 自动创建了 EiBlockMeta

    EiColumn column = new EiColumn("data");
    column.setDescName("数据列");
    result.addMeta(column); // EiBlockMeta 添加了列信息

    column = new EiColumn("data1");
    column.setDescName("数据列 1");
    result.addMeta(column);

    Map<String, String> row = new HashMap<String, String>();
    row.put("data", "数据，三段式绑定");
    row.put("data1", "数据 1，三段式绑定");
    result.addRow(row);

    result.addRow();
    result.setCell(1, "data", "API1，三段式绑定");
    result.setCell(1, "data1", "API2，三段式绑定");

    result.set("block", "EiBlock 的属性值，两段式绑定");

    outInfo.addBlock(result);
    outInfo.set("info", "EiInfo 的属性值，一段式绑定");

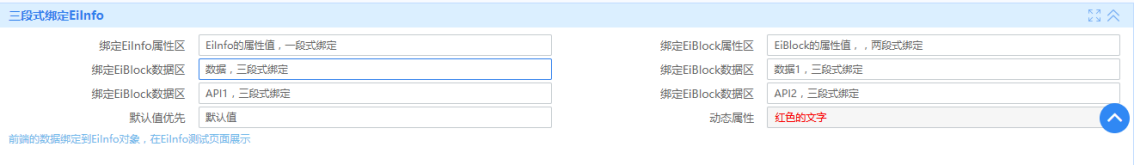
    outInfo.set("phone", "xiaomi,meizu");
    outInfo.set("lang", "Java");
    outInfo.set("bestLang", "PHP");
    outInfo.set("bestLang1", "PHP");
    return outInfo;
}
```

得到的 EiInfo 对象数据如下：

EiInfo 数据

```
{
  "name": "输出 outInfo",
  "msg": "",
  "msgKey": "",
  "detailMsg": "",
  "status": 0,
  "attr": {
    "phone": "xiaomi,meizu",
    "bestLang1": "PHP",
    "lang": "Java",
    "bestLang": "PHP",
    "info": "EiInfo 的属性值，一段式绑定"
  },
  "blocks": {
    "result": {
      "attr": {
        "block": "EiBlock 的属性值，两段式绑定"
      },
      "meta": {
        "columns": [
          {
            "pos": 0,
            "name": "data",
            "descName": "数据列"
          },
          {
            "pos": 1,
            "name": "data1",
            "descName": "数据列 1"
          }
        ]
      },
      "rows": [
        [
          "数据，三段式绑定",
          "数据 1，三段式绑定"
        ],
        [
          "API1，三段式绑定",
          "API2，三段式绑定"
        ]
      ]
    }
  }
}
```

3.5.2.2.1.3.1.3 页面展示效果



3.5.2.2.1.3.2 布局

3.5.2.2.1.3.2.1 Input 属性及作用

属性：hidden；ratio；colwidth；inline。

作用：type="hidden"可对 EForm 进行隐藏；colWidth 和 ratio 两者都属于栅格布局，可参考 [03 Bootstrap 栅格布局](#)，colWidth 用于控制整个控件的宽度，ratio 用于控制 label 和 input 框的比例；inline 可用于控制 label 是否显示。

EForm属性的作用

默认的EForm

字段标签

字段标签

字段标签

type="hidden" 的 EForm

ratio="6:6" 调整了 EForm 的标签和输入框的宽度比例

设置标签输入框比例

colwidth="9" 调整了 EForm 的宽度

设置控件宽度

colwidth="9" ratio="6:6" 既调整了 EForm 的宽度又调整了标签，输入框的宽度比例

同时设置控件宽度和标签比例

inline="true" 只显示输入框

3.5.2.2.1.3.2.2 EForm 布局分类

- 两列布局；
- 三列布局；
- 四列布局；
- 二、四列布局；
- 六列布局；
- 三、六列布局

EForm两列布局

推荐的做法 12栅格满了手动写 div.row 换行

字段标签

字段标签

字段标签

字段标签

鸡贼的做法 12栅格溢出自动换行

字段标签

字段标签

字段标签

字段标签

EForm三列布局

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

EForm四列布局

字段标签

字段标签

字段标签

字段标签

EForm二，四列布局

字段标签

字段标签

字段标签

字段标签

EForm六列布局

过于紧凑 不推荐

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

EForm四，六列布局

Bootstrap 默认 15px 的栅格 gutter 在这种紧凑的布局下就不太合适了，需要适当的缩小 gutter。

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

备注：1/18 = 1/6 * 1/3 (EForm 默认的4:8，1/3 = 4/(4+8)) label 已经小于 1/12 页面的最小栅格了，此时无法进行对齐，只能调整 EForm 的 ratio 为 6:6，1/12 = 1/6 * 1/2

字段标签

字段标签

字段标签

字段标签

字段标签

字段标签

- 171

JSP 页面开发

JSP 示例

```

<EF:EFRegion id="R2" title="EFInput 两列布局">
  <div class="row">
    <h5 class="push-30-l text-success">推荐的做法 12 栅格满了手动写
<code>div.row</code> 换行</h5>
    <EF:EFInput ename="inqu_status-0-field1" colWidth="6" cname="字段标签"/> //两
列布局, 控件宽度 (colwidth) 设为 6 (共 12 栅格), 满格后写 div.row 换行
    <EF:EFInput ename="inqu_status-0-field1" colWidth="6" cname="字段标签"/>
  </div>
  <div class="row">
    <EF:EFInput ename="inqu_status-0-field1" colWidth="6" cname="字段标签"/>
    <EF:EFInput ename="inqu_status-0-field1" colWidth="6" cname="字段标签"/>
  </div>
  <div class="row">
    <h5 class="push-30-l text-danger">鸡贼的做法 12 栅格溢出自动换行</h5>
    <EF:EFInput ename="inqu_status-0-field1" colWidth="6" cname="字段标签"/> //两
列布局, 控件宽度 (colwidth) 设为 6 (同上格), 满格后不写 div.row, 让其自动换行
    <EF:EFInput ename="inqu_status-0-field1" colWidth="6" cname="字段标签"/>
    <EF:EFInput ename="inqu_status-0-field1" colWidth="6" cname="字段标签"/>
    <EF:EFInput ename="inqu_status-0-field1" colWidth="6" cname="字段标签"/>
  </div>
</EF:EFRegion>

<EF:EFRegion id="R3" title="EFInput 三列布局">
  <div class="row">
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签"/>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签"/>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签"/>
  </div>
  <div class="row">
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签"/>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签"/>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签"/>
  </div>
</EF:EFRegion>

<EF:EFRegion id="R4" title="EFInput 四列布局">
  <div class="row">
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="3"/>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="3"/>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="3"/>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="3"/>
  </div>
</EF:EFRegion>

<EF:EFRegion id="R5" title="EFInput 二, 四列布局">
  <div class="row">
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" ratio="2:10"
colWidth="6"/> //两列布局 设置 ratio (label 和 input 框的比例), 保证上下 label 对齐, 以及
控件的宽度
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" ratio="2:10"
colWidth="6"/>
  </div>
  <div class="row">
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="3"/>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="3"/>
  </div>

```



```

        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="3"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="3"/>
    </div>
</EF:EFRegion>

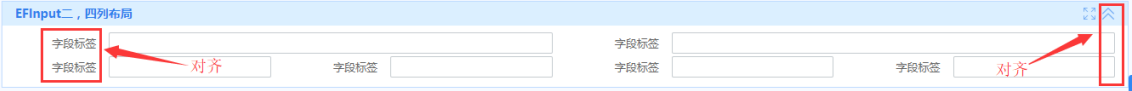
<EF:EFRegion id="R6" title="EFInput 六列布局">
    <div class="row">
        <h5 class="push-30-l text-danger">过于紧凑 不推荐</h5>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
    </div>
    <div class="row">
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" colWidth="2"/>
    </div>
</EF:EFRegion>

<EF:EFRegion id="R7" title="EFInput 四，六列布局">
    <div class="row">
        <h5 class="push-30-l text-danger">Bootstrap 默认 <code>15px</code> 的栅格
        gutter 在这种紧凑的布局下就不太合适了，
        需要适当的缩小 gutter。</h5>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" ratio="6:6"
        colWidth="2"/> //gutter: 列之间的间隔
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" ratio="6:6"
        colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" ratio="6:6"
        colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" ratio="6:6"
        colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" ratio="6:6"
        colWidth="2"/>
        <EF:EFInput ename="inqu_status-0-field1" cname="字段标签" ratio="6:6"
        colWidth="2"/>
    </div>
    <div class="row">
        <h5 class="push-30-l text-danger">备注  $1/18 = 1/6 * 1/3$  (EFInput 默认的  $4:8$ ,  $1/3 = 4/(4+8)$ )
        label 已经小于  $1/12$  页面的最小栅格了，此时无法进行对齐，只能调整 EFInput 的
        ratio 为  $6:6$ ,  $1/12 = 1/6 * 1/2$  </h5>
        <EF:EFInput ename="inqu_status-0-four_six1" cname="字段标签" colWidth="3"/>
        <EF:EFInput ename="inqu_status-0-four_six2" cname="字段标签" colWidth="3"/>
        <EF:EFInput ename="inqu_status-0-four_six3" cname="字段标签" colWidth="3"/>
        <EF:EFInput ename="inqu_status-0-four_six4" cname="字段标签" colWidth="3"/>
    </div>
</EF:EFRegion>

```

四，六列布局 HTML 片段解析：

在开发页面时，布局规范是极其重要的，表单类页面规范可看[表单类页面检查项](#)，其中列头的 label 和列尾的输入框，需要上下对齐：



在六列布局中：栅格共 12 份，六列布局，可推出每列总宽（label+输入框）占页面 2 份（ $2/12=1/6$ ）。EFInput 标签和输入框默认比例（ratio）为 4:8，但是由于太过紧凑，若 label 占 4 份，则占每列总宽为： $4/12=1/3$ ，占页面总宽为 $1/6*1/3=1/18$ ，小于一般 label 为 $1/12$ 的栅格大小，无法和其他布局的第一个 label 对齐，所以只能调整 EFInput 的 ratio 为 6:6（ $1/6*X=1/12$ ，推出 X 为 $1/2$ ，即占总宽的 6 份（ $6/12=1/2$ ），保证对齐。

12

六列布局：

1/6	1/6	1/6	1/6	1/6	1/6
-----	-----	-----	-----	-----	-----

label 输入框

4	8	1/6	1/6	1/6	1/6
---	---	-----	-----	-----	-----

12

推出： $1/6 \div 12 * 4 = 1/18$

四列布局：

1/4	1/4	1/4	1/4
-----	-----	-----	-----

四列布局正常 label

4	8	1/4	1/4
---	---	-----	-----

12

推出： $1/4 \div 12 * 4 = 1/12$

$1/18 < 1/12$ 所以 label 肯定不对齐，如下图所示：

label 输入框

4	8	1/6	1/6
---	---	-----	-----

12

推出： $1/6 \div 12 * 4 = 1/18$

四列布局：

1/4	1/4
-----	-----

四列布局正常label

4	8	1/4
---	---	-----

12

推出： $1/4 \div 12 * 4 = 1/12$

只能调整 ratio， $1/6*X=1/12$ ，推出 X 为 $1/2$ ，即占总宽的 6 份（ $6/12=1/2$ ），保证对齐。

再将其和四列布局对比：

6	6	1/6	1/6	1/6	1/6	1/6
4	8	1/4	1/4	1/4		

label 已对齐。

3.5.2.2.1.3.3 Input 的状态

3.5.2.2.1.3.3.1 页面展示效果

EFInput的状态

默认的EFInput

字段标签

readonly 的 EFInput

只读的输入框

disabled 的 EFInput

禁用的输入框

required 的 EFInput

*必填的输入框

3.5.2.2.1.3.3.2 JSP 页面开发

JSP 示例

```
<EF:EFRegion id="R0" title="EFInput 的状态">
  <div class="row">
    <h5 class="push-30-l text-info">默认的 EFInput</h5>
    <EF:EFInput ename="inqu_status-0-field1" cname="字段标签"/>
  </div>
  <div class="row">
    <h5 class="push-30-l text-info"><code>readonly</code> 的 EFInput</h5>
    <EF:EFInput ename="inqu_status-0-field1" readonly="true" cname="只读的输入框"/>
  </div>
  <div class="row">
    <h5 class="push-30-l text-info"><code>disabled</code> 的 EFInput</h5>
    <EF:EFInput ename="inqu_status-0-field1" disabled="true" cname="禁用的输入框"/>
  </div>
  <div class="row">
    <h5 class="push-30-l text-info"><code>required</code> 的 EFInput</h5>
    <EF:EFInput ename="inqu_status-0-field3" cname="必填的输入框" required="true"/>
  </div>
</div>
```

```
</EF:EFRegion>
```

3.5.2.2.2 EEDM4001-EFInput 校验

3.5.2.2.2.1 页面功能简介

本示例页面用于说明如何使用 **EFInput** 校验。包括平台内置校验、正则校验、多重校验、分组校验以及非必填与必填混合校验。

校验功能主要使用动态属性来完成 [04 Tag 中如何使用 attributes 动态属性](#)。

3.5.2.2.2.2 配置和 API 详情

IPLAT.Validator({ Id: "" }) 配置 JS 代码使页面中的 **EFInput** 能够进行校验。

IPLAT.Validator({ Id: "" }).validate() 根据已声明的验证规则验证输入元素
true: 通过验证; **false**: 验证失败。

参考 [EFInput](#) 和 [EFInput 校验](#) 中的描述。

3.5.2.2.2.3 页面开发步骤

3.5.2.2.2.3.1 内置校验

3.5.2.2.2.3.1.1 JSP 页面开发

内置验证

```
<EF:EFRegion id="region1" title="内置规则校验">
  <div class="row">
    <h5 class="push-30-l text-danger">EFSelect 和 EFDatePicker 只支持必填校验</h5>
    <EF:EFInput ename="inqu_status-0-required" cname="非空校验"
      required="true" placeholder="请填写字符"/>
    <EF:EFSelect cname="大洲" template="#=valueField#-#=textField#"
valueTemplate="#=valueField#-#=textField#"
      ename="inqu_status-0-continent" required="true" resultId="result"
      optionLabel="请选择" serviceName="EEDM5010" methodName="initLoad"
      textField="continent_name" valueField="continent_code">
      <EF:EFOption label="所有" value="all"/>
    </EF:EFSelect>

    <EF:EFDatePicker format="yyyy/MM/dd" required="true" cname="单个的日期"
ename="inqu_status-0-start">
    </EF:EFDatePicker>
  </div>

  <div class="row">
    <EF:EFInput ename="inqu_status-0-length" cname="输入长度校验"
      minLength="3" maxLength="8" placeholder="请输入长度为 3-8 的字符串"
    />
    <EF:EFInput ename="inqu_status-0-email" cname="邮箱校验"
      data-rules="email" validateGroupName="group1,group2" placeholder="
请填写邮箱"/>
```

```

        <EF:EFInput ename="inqu_status-0-url" cname="URL 校验"
            data-rules="http_url" placeholder="请填写 URL"/>
    </div>
    <div class="row">
        <EF:EFInput ename="inqu_status-0-mobile_phone" cname="手机号校验"
            data-rules="mobile_phone" placeholder="请输入手机号"/>
        <EF:EFInput ename="inqu_status-0-number" cname="数字校验"
            data-rules="positive_integer" placeholder="请输入正整数"/>
        <EF:EFInput ename="inqu_status-0-ip" cname="IP 地址校验"
            data-rules="ip_address" placeholder="请输入 IP 地址"/>
    </div>
</EF:EFRegion>

```

3.5.2.2.2.3.1.2 JS 开发

另需配置 JS 代码使页面中的 **EFInput** 能够进行校验，如下代码所示，对页面 EEDM4001 下的所有 region1 的内容都配置了校验规则。



validator1 只会校验 region1 区域中的 EFInput。

JS 示例

```

$(document).ready(function() {
    var validator1 = IPLAT.Validator({
        id: "region1"
    });
});

```

3.5.2.2.2.3.1.3 页面展示效果

1. form 表单元素校验失败时，边框会被标红
2. 鼠标 hover 时弹出红色提示消息
3. 输入的内容不符合校验时，input 框内文字会变红色。

3.5.2.2.2.3.2 正则校验

3.5.2.2.2.3.2.1 JSP 页面开发

正则验证

```

<EF:EFRegion id="region2" title="正则表达式校验">

```

```

<div class="row">
  <EF:EFInput ename="result-0-num5_101" cname="正则校验"
    data-regex="/^([5-9]|100|[1-9]\d)$/"/ data-errorPrompt="请输入 5-100
之间的数字"
    placeholder="请输入 5-100 数字"/>
  <EF:EFInput colWidth="6" ratio="6:6" ename="result-0-password" cname="密码校检
(字母, 数字, _组成 6-10 位字符)" required="true" type="password"
    data-regex="/^[0-9a-zA-Z]{6,10}$/"/ data-errorPrompt="密码格式不符合
要求"
    placeholder="字母, 数字, _组成 6-10 位字符"/>
</div>
<div class="row">
  <EF:EFInput ename="result-0-num37" cname="多重正则校验"
    data-errorPrompt='长度大于 3 的数字' data-regex="/^[0-9]{3,}$/'/
    data-errorPrompt0='长度小于 10 的数字' data-regex0="/^[0-9]{0,10}$/'/
    data-errorPrompt1='只允许出现 3 到 9 的数字' data-regex1="/^[3-
9]{0,}$/'/
    data-errorPrompt2='只允许出现 0 到 7 的数字' data-regex2="/^[0-
7]{0,}$/'/
    placeholder="长度在 3-10 只包含 3-7 的数字"/>
  <EF:EFInput colWidth="6" ratio="6:6" ename="result-0-chinese" cname="中文校检"
    data-regex="/^[u4e00-u9fa5]+$/"/ data-errorPrompt="请输入中文"
    placeholder="请输入中文"/>
</div>
</EF:EFRegion>

```

3.5.2.2.2.3.2.2 页面展示效果

正则表达式校验

正则校验	请输入 5-100 数字	密码校检 (字母, 数字, _组成 6-10 位字符)	
多重正则校验	长度在 3-10 只包含 3-7 的数字	中文校检	请输 ① 密码格式不符合要求

3.5.2.2.2.3.2.3 JS 开发

⚠ 当配置为如下代码时，对正则校验（即 id=region2）等其他未配置的区域都不会产生校验，仅会对内置规则（即，id=region1）的区域进行校验。

JS 示例

```

$(document).ready(function() {
  var validator = IPLAT.Validator({
    id: "region1" //未配置 id:"region2" 则不会对区域 2 做校验
  });
});

```

3.5.2.2.2.3.2.4 页面展示效果

正则表达式校验

正则校验	xxxxxxxx	密码校检 (字母, 数字, _组成 6-10 位字符)	xxxxxxxxxxxx
多重正则校验	1	中文校检	2

3.5.2.2.2.3.3 多重校验

3.5.2.2.2.3.3.1 JSP 页面开发

可通过 **data-regex(n)**及 **data-errorPrompt(n)**添加多个正则校验及提示信息，或与内置校验混合使用来完成多重校验。

多重校验

```
<EF:EFRegion id="region3" title="多重规则校验">
  <div class="row">
    <EF:EFInput ename="inqu_status-0-multiple1" cname="输入密码"
      data-rules="string, label"
      placeholder="同时使用 string 和 label 校验"/>
    <EF:EFInput ename="inqu_status-0-multiple2" cname="混合校验"
      required="true" minLength="3" maxLength="8" data-rules="number"
      data-errorPrompt1='只允许出现 3 到 9 的数字' data-regex1='^[3-
9]{0,}$/'
      data-errorPrompt2='只允许出现 0 到 7 的数字' data-regex2='^[0-
7]{0,}$/'
      placeholder="必须填写 3-8 位数字只包含 3-7 的数字"/>
  </div>
</EF:EFRegion>
```

3.5.2.2.2.3.3.2 页面效果展示

其中，第一个输入框是内置规则的校验；第二个输入框混合了内置校验，正则校验，placeholder 取其交集，提示校验信息。

3.5.2.2.2.3.4 分组校验

3.5.2.2.2.3.4.1 JSP 页面开发

分组校验示例

```
<EF:EFRegion id="region4" title="分组校验">
  <div class="row">
    <h5 class="push-30-l text-danger"> 通过 js 的设置:以下校验仅对分组 2 进行校验
  </h5>
    <EF:EFInput ename="inqu_status-0-multiple3" cname="分组 1"
      validateGroupName="group1"
      required="true" minLength="3" maxLength="8" data-rules="number"
      placeholder="必须填写 3-8 位数字"/>
    <EF:EFInput ename="inqu_status-0-multiple4" cname="分组 2"
      validateGroupName="group2"
      data-rules="postcode, number"
      placeholder="填写邮编，且必须是数字"/>
    <EF:EFInput ename="inqu_status-0-multiple5" cname="分组 3"
      validateGroupName="group3"
      data-roles="positive_integer"
      placeholder="必须输入正整数"/>
  </div>
</EF:EFRegion>
```

```
</div>
</EF:EFRegion>
```

3.5.2.2.3.4.2 JS 开发

示例代码中的 **validator4**，会对 **region4** 下 **group2** 的 **EFInput** 进行校验，而未配置 **validateGroupName** 或分组不为 **group2** 的则不会进行校验。

JS 示例

```
$(document).ready(function() {
    var validator4= IPLAT.Validator({
        id: "region4",
        groupName: "group2"
    });
});
```

3.5.2.2.3.4.3 页面效果展示

Tips_of_validateGroupName

1. 一个表单元素若从属于多个分组(如 group1、group2)，可配置 **validateGroupName = "group1,group2"**
2. 若要同时校验多个分组的元素(如 group1、group2)，可在js中定义validator 然后调用 **validator.validate("group1,group2")**，各分组名之间用","隔开
3. **"__all__"** 可用于validator作用范围内所有表单元素的校验：如 **validator1.validate("__all__")** 可对 **id="region1"** 的Region区内所有元素进行校验
- "__none__"**
用于validator作用范围内未配置**validateGroupName**属性的表单元素校验：
如 **validator1.validate("__none__")**

3.5.2.2.3.5 非必填与必填混合校验

3.5.2.2.3.5.1 JSP 页面开发

Code Block 2 非必填与必填混合示例

```
<EF:EFRegion id="region6" title="非必填和必填混合区域">
    <div class="row">
        <EF:EFInput ename="inqu_status-0-necessary" cname="手机号校验" required="true"
```



```

        data-rules="mobile_phone" placeholder="请填写手机号(必填)"/>
        <EF:EFInput ename="inqu_status-0-unnecessary" cname="邮箱校验"
        data-rules="email" placeholder="请填写邮箱(非必填)"/>
    </div>
    <div class="row">
        <h5 class="push-30-1 text-danger"> 非必填输入框没有设置校验规则，不校验</h5>
        <EF:EFInput ename="inqu_status-0-optional" cname="不校验情况"
        placeholder="请填写家庭住址(可不填写)"/>
    </div>
</EF:EFRegion>

// 按钮区域
<EF:EFRegion title="EFInput 校验示例区">
    <div class="col-md-4">
        <h5 class="push-15 text-danger"> 点击按钮，对非必填和必填混合区域的内容进行校验
    </h5>
    <div class="row">
        <div class="col-md-4">
            <EF:EFButton ename="validator_button2" cname="提交表单"/>
        </div>
    </div>
</div>
</EF:EFRegion>

```

3.5.2.2.2.3.5.2 JS 开发

示例代码中的 **validator4**，会对 **region4** 下 **group2** 的 **EFInput** 进行校验，而未配置 **validateGroupName** 或分组不为 **group2** 的则不会进行校验。

JS 示例

```

// 对 id 为 region6 的区域进行校验
var validator6 = IPLAT.Validator({
    id: "region6"
});

// 提交表单
$("#validator_button2").on("click", function () {
    if (validator6.validate()) { // true: 通过验证
        WindowUtil({
            "title": "校验 API 示例",
            "content": "<div class='kendo-del-message'>验证通过</div>"
        })
    } else {
        WindowUtil({
            "title": "校验 API 示例",
            "content": "<div class='kendo-del-message'>验证未通过</div>"
        })
    }
})

```

1. 若输入框为必填，且设置校验规则时：则会先验证是否已输入内容，再根

- 据验证规则来验证输入内容的合法性。
- 2.若输入框为必填，且没有设置校验规则：则只会验证是否已填。
- 3.若输入框为非必填，且设置校验规则时：若输入内容，则对提交的输入内容进行校验；若不输入内容，则不校验。
- 4.若输入框为非必填，且没有设置校验规则：不校验。

3.5.2.2.3.5.3 页面效果如下所示



3.5.2.2.3 EEDM4002-EFInput 格式化

3.5.2.2.3.1 页面功能简介

本示例页面用于说明如何对 EFInput 中内容格式化输出，主要包括货币，百分比，以及日期。

3.5.2.2.3.2 配置和 API 详情

这里需要对 kendo 的格式化原理有一定的了解，不清楚的开发人员可以参考：[Kendo Format 格式化输出](#)，里面有对格式化输出有详细的说明。此外还需要了解 [EFInput](#)，[EFDatePicker](#) 相关组件。

3.5.2.2.3.2.1 数字类格式化：

	数字	货币	百分比	货币 - 货币
不带小数点：	{0:n0}	{0:c0}	{0:p0}	{0:c0} - {1:c0}
1 位小数：	{0:n1}	{0:c1}	{0:p1}	{0:c1} - {1:c1}
2 位小数：	{0:n2}或{0:n}	{0:c2}或{0:c}	{0:p2}或{0:p}	{0:c2} - {1:c2}或{0:c} - {1:c}
3 位小数：	{0:n3}	{0:c3}	{0:p3}	{0:c3} - {1:c3}

关于“货币-货币”的说明：

```
console.log( kendo.format("{0:c} - {1:c}", 12, 24) ); // 输出 "¥12.00 - ¥24.00"
```

3.5.2.2.3.2.2 日期类格式化：

	纯日期	日期+时间	日期+时间+毫秒
--	-----	-------	----------

	纯日期	日期+时间	日期+时间+毫秒
以“-”分隔	yyyy-MM-dd	yyyy-MM-dd HH:mm:ss	yyyy-MM-dd HH:mm:ss.fff
以“/”分隔	yyyy/MM/dd	yyyy/MM/dd HH:mm:ss	yyyy/MM/dd HH:mm:ss.fff

3.5.2.2.3.3 页面开发步骤

3.5.2.2.3.3.1 货币格式化页面开发



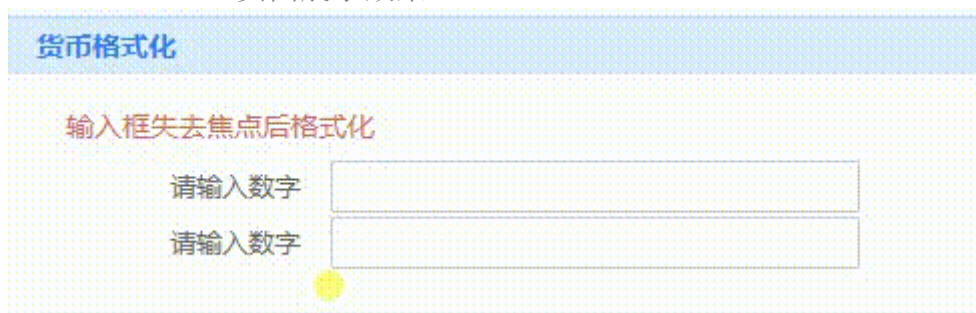
通过 EFInput 的 format 属性进行数字格式化：

3.5.2.2.3.3.1.1 JSP 页面开发

JSP 源码

```
<div class="row">
  <h5 class="push-30-l text-danger">输入框失去焦点后格式化</h5>
  <EF:EFInput ename="inqu_status-0-num" cname="请输入数字" format="{0:c}" />
  <EF:EFInput ename="inqu_status-0-num1" cname="请输入数字" format="{0: #. ##}" />
  <EF:EFInput ename="inqu_status-0-num2" cname="请输入数字" format="{0:0.00}" />
  <EF:EFInput ename="inqu_status-0-num3" cname="请输入数字" format="{0:N2}" />
</div>
```

3.5.2.2.3.3.1.2 页面展示效果



我们通过输入框，输入数据，当输入框失去焦点时（将光标移除 input 框的时候），控件会自动对数字进行格式化。

需要注意的是：

1. format 只对数字有效，当输入非数字的时候不会格式化。
2. 配置了 format 属性时，input 框的默认 type 为 text。
3. 当 format 与 type 属性同时存在时，format 属性失效。



通过 JS 进行格式化。

我们也可以通过 JS 进行格式化：点击“格式化”按钮，触发事件。在事件处理中，使用 **kendo** 的格式化接口，对数据格式化处理后，填入相对应的输入框中。

3.5.2.2.3.3.1.3 JS 开发

JS 源码

```
$(document).ready(function() {
    $("#change").on("click",function(e) {
        var val = $("#inqu_status-0-number").val();
        kendo.culture("en-US");    //格式化成美元形式
        $("#inqu_status-0-dollar").val(kendo.format("{0:c}",+val));    ////带两位小数点,
        {0:c2} 也能达到相同效果
        kendo.culture("zh-CN");    //格式化成人民币形式
        // c0 表示不带小数点,c1 表示带一个小数点
        $("#inqu_status-0-rmb").val(kendo.format("{0:c0}",+val));
    });
});
```

3.5.2.2.3.3.1.4 页面展示效果

通过js进行格式化

请输入数字

666

格式化

美元

\$666.00

人民币

¥666

通过点击“格式化”按钮，就能把输入的数字转为美元和人民币两种方式。

3.5.2.2.3.3.2 百分比格式化页面开发

同样的，我们通过输入框，获取输入数据，点击“格式化”按钮，触发事件。在事件处理中，使用 **kendo** 的格式化接口，对数据格式化处理后，填入相对应的输入框中。

3.5.2.2.3.3.2.1 JSP 页面开发

JSP 源码

```
<EF:EFRegion title="百分比格式化">
    <div class="row">
        <EF:EFInput ename="inqu_status-1-number" cname="请输入数字"/>
        <EF:EFButton ename="change1" cname="格式化" ></EF:EFButton>
    </div>
    <hr>
    <div class="row">
        <EF:EFInput ename="inqu_status-1-percent" cname="ab%"/>
        <EF:EFInput ename="inqu_status-1-percent1" cname="ab.cd%"/>
        <EF:EFInput ename="inqu_status-1-percent2" cname="ab.cdef%"/>
    </div>
</EF:EFRegion>
```

3.5.2.2.3.3.2.2 JS 开发

JS 源码

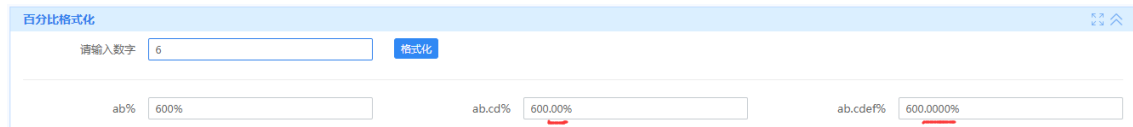
```
$(document).ready(function() {
    $("#change1").on("click",function(e){
        var val = $("#inqu_status-1-number").val();

        $("#inqu_status-1-percent").val(kendo.format("{0:p0}",+val)); //不带小数点的

        $("#inqu_status-1-percent1").val(kendo.format("{0:p}",+val)); ////带两位小数点,
        {0:p2} 也能达到相同效果

        $("#inqu_status-1-percent2").val(kendo.format("{0:p4}",+val)); //四位小数点
    });
})
```

3.5.2.2.3.3.2.3 页面效果



通过点击“格式化”按钮，就能把输入的数字转化为 不带小数点的，两位小数点，四位小数点的百分比格式。

3.5.2.2.3.3.3 日期格式化页面开发

我们通过日期输入框，获取日期数据，点击“格式化”按钮，触发事件。在事件处理中，使用 kendo 的格式化接口，对数据格式化处理后，填入相对应的输入框中。

JSP 源码

```
<EF:EFRegion title="日期格式化">
    <div class="row">
        <EF:EFDatePicker ename="inqu_status-2-number" cname="选择日期" format="yyyy-MM-dd" ></EF:EFDatePicker>
        <EF:EFButton ename="change2" cname="格式化"></EF:EFButton>
    </div>
    <hr>
    <div class="row">
        <EF:EFInput ename="inqu_status-2-date" cname="yyyy/MM/dd" ratio="6:6" />
        <EF:EFInput ename="inqu_status-2-date1" cname="yyyy/MM/dd hh:mm:ss"
ratio="6:6"/>
        <EF:EFInput ename="inqu_status-2-date2" cname="yyyy/MM/dd hh:mm tt"
ratio="6:6"/>
    </div>
</EF:EFRegion>
```

JS 源码

```
$(document).ready(function() {
    $("#change2").on("click",function(e){
        var val = $("#inqu_status-2-number").val(); //取出所选日期数据

        var argus = val.split("-"); //将其在“-”处分离，即：年 月 日，放入数组中

        var date = new Date(argus[0] + "/" + argus[1] + "/" + argus[2]); // 年/月/日

        $("#inqu_status-2-date").val(kendo.toString(date, "yyyy/MM/dd")); // 年/月/日

        $("#inqu_status-2-date1").val(kendo.toString(date, "yyyy/MM/dd hh:mm:ss")); //年/月/日 时/分/秒

        $("#inqu_status-2-date2").val(kendo.toString(date, "yyyy/MM/dd hh:mm tt")); //年/月/日 时/分 时间段
    })
})
```

效果图：

通过点击“格式化”按钮，就能把选择的日期的转化为图中展示的三种格式。

3.5.2.2.4 EEDM4003-EFInput 类型

3.5.2.2.4.1 页面功能简介

本示例页面用于说明 EFInput 的类型，其中包括有 text/radio/checkbox/textarea/hidden 等，通过 type 设置不同类型来满足不同场景的输入需求，其中 text 为默认类型。

不同类型的 EFInput 与 EiInfo 数据绑定有一定区别[数据交换中间件 EiInfo 的使用](#)。

3.5.2.2.4.2 配置和 API 详情

参考 [EFInput](#) 中的描述。

3.5.2.2.4.3 页面开发步骤

3.5.2.2.4.3.1 JSP 页面开发

3.5.2.2.4.3.2 checkbox

checkbox

应该手动设置value，而不能绑定EiInfo中的value，否则checkbox组都会被绑定相同的EiInfo值；EiInfo中的value可以用于检查checkbox是否该选中

通过在后台 Service 中，设置 value 值，进行选中：outInfo.set("phone", "xiaomi,meizu"); =》小米和魅族也被选中了

Form:2 垂直排列的checkbox

请选择手机品牌

☒ 小米

☐ 华为

☒ 魅族

JSP 代码:

JSP 示例

```
<div class="row">
  <div class="col-md-8 col-md-offset-2 form-b">
    <p class="text-info">Form:2 垂直排列的 checkbox</p>
    <div class="col-md-6">
      <div class="col-xs-offset-4 col-xs-8">
        <label>请选择手机品牌</label>
        <EF:EFInput ename="phone" value="xiaomi" cname="小米" type="checkbox"/>
        <EF:EFInput ename="phone" value="huawei" cname="华为" type="checkbox"/>
        <EF:EFInput ename="phone" value="meizu" cname="魅族" type="checkbox"/>
      </div>
    </div>
  </div>
</div>
```

若不在 JSP 中设置以上 value 值，页面效果:

Form:2 垂直排列的checkbox

请选择手机品牌

☐ 小米

☐ 华为

☐ 魅族

除通过 EInfo 中的 value 设置选中的 checkbox，也可以通过动态属性 **checked="checked"** 将 checkbox 设置为选中状态。

JSP 示例

```
<div class="row">
  <div class="col-md-8 col-md-offset-2 form-b">
```

```

<p class="text-info">Form:3 水平排列的 checkbox</p>
<label class="col-xs-4">请选择手机品牌</label>
<div class="col-xs-8">
    <EF:EInput ename="phone" inline="true" value="xiaomi" cname="小米"
type="checkbox"/> //设置 inline="true" 控制其水平排列
    <EF:EInput ename="phone" checked="checked" inline="true" value="huawei"
cname="华为" type="checkbox"/> //选中状态
    <EF:EInput ename="phone" inline="true" value="meizu" cname="魅族"
type="checkbox"/>
</div>
</div>
</div>

```

页面效果如下所示：

Form:3 水平排列的checkbox

请选择手机品牌

☒ 小米 ☒ 华为 ☒ 魅族

备注：多个EInput控件配置

inline="true"时，可使控件水平排列；不配置inline属性时默认垂直排列。

checkbox 组值是以", "隔开的字符串。可通过 setByNodeObject 等方法 set 到 EiInfo，也可以通过 get 方法，获取选中值。

示例

```

var ei = new EiInfo();
ei.setByNodeObject(document.getElementById("form1")); //setByNodeObject:把 dom 节点下面
的各种交互数据根据三段式规则填入 EiInfo。
ei.get("phone"); //=>"xiaomi,meizu,xiaomi,huawei,meizu"

```

页面效果如下所示：

checkbox示例

id='form1'

Form1: 表单中单独使用checkbox

*姓名

邮箱

自由发挥

☐ 同意

Form2: 垂直排列的checkbox

请选择手机品牌

☒ 小米 ☐ 华为 ☒ 魅族

Form3: 水平排列的checkbox

请选择手机品牌

☒ 小米 ☒ 华为 ☒ 魅族

3.5.2.2.4.3.3 radio

radio 同 checkbox 一样应手动设置 value。

JSP 示例

```
<div class="row">
  <div class="col-md-8 col-md-offset-2 form-b">
    <p class="text-info">Form:4 垂直排列的 radio</p>
    <div class="col-md-6">
      <div class="col-xs-offset-4 col-xs-8">
        <label>您使用的编程语言</label>
        <EF:EFInput ename="lang" value="PHP" cname="PHP" type="radio"/>
        <EF:EFInput ename="lang" value="Java" cname="Java" type="radio"/>
        <EF:EFInput ename="lang" value="C" cname="C" type="radio"/>
      </div>
    </div>
  </div>
</div>
```

3.5.2.2.4.3.4 textarea

若 input 框存在 value 属性值则显示 value 值，否则显示 EiInfo 中绑定的值。

JSP 示例

```
<div class="row">
  <EF:EFInput ename="text" value="1999" type="textarea" cname="示例"/>
</div>
```

3.5.2.2.4.3.5 Service 服务开发

后台 service 的 initLoad 方法设置 EiInfo 对象内的数据：

Service 代码

```
public EiInfo initLoad(EiInfo inInfo) {

    EiInfo outInfo = new EiInfo("输出 outInfo");
    EiBlock result = new EiBlock("result"); // 自动创建了 EiBlockMeta blockId 为 result

    ...
    outInfo.addBlock(result);
    outInfo.set("info", "EiInfo 的属性值，一段式绑定");

    outInfo.set("phone", "xiaomi,meizu"); // 后台设置选中值
    outInfo.set("lang", "Java");
    outInfo.set("bestLang", "PHP");
    outInfo.set("bestLang1", "PHP");
    return outInfo;
}
```

在 service 的初始化方法中得到的 EiInfo 数据，通过"一段式绑定"可以获得其 EiInfo 属性区中的内容：

EiInfo 数据

```
{
  "name": "输出 outInfo",
  "msg": "",
  "msgKey": "",
  "detailMsg": "",
  "status": 0,
  "attr": {
    "phone": "xiaomi,meizu",
    "lang": "Java",
    "bestLang": "PHP",
    "bestLang1": "PHP",
    "info": "EiInfo 的属性值，一段式绑定"
  }
  .....
}
```

3.5.2.2.4.3.6 页面展示效果

Form:2 垂直排列的checkbox

请选择手机品牌

☒ 小米

☐ 华为

☒ 魅族

Form:3 水平排列的checkbox

请选择手机品牌

☒ 小米

☒ 华为

☒ 魅族

Form:4 垂直排列的radio

您使用的编程语言

☐ PHP

☒ Java

☐ C

textarea示例

示例：

1999

3.5.2.2.5 EEDM4005-EFInput 位置问题示例

3.5.2.2.5.1 页面功能简介

本示例页面用于说明，如何解决 `type="checkbox" || "radio"` 的 EFInput 与其他类型(如`"text"`)混合使用时，出现控件显示位置前后颠倒的情况。

涉及布局相关知识 [03 Bootstrap 栅格布局](#)。

3.5.2.2.5.2 配置和 API 详情

参考 [EFInput](#) 中的描述。

3.5.2.2.5.3 页面开发步骤

3.5.2.2.5.3.1 JSP 页面开发

可以通过在 `type="checkbox" || "radio"` 的 EFInput 控件外部套上一层 `div` 解决，具体如下：

JSP 示例

```
<EF:EFRegion title="EFInput 布局">
  <div class="row">
    <div class="col-md-4">
      <EF:EFInput ename="inqu-0-phone" inline="true" value="xiaomi" cname="小米"
checked="true" type="checkbox" />
      <EF:EFInput ename="inqu-0-phone2" inline="true" value="apple" cname="苹果"
type="checkbox" />
      <EF:EFInput ename="inqu-0-phone3" inline="true" value="huawei" cname="华为"
type="checkbox" />
    </div>
    <EF:EFInput ename="inqu-0-phone4" cname="phone" row="0" blockId="oscillation"
colWidth="8" ></EF:EFInput>
  </div>
  <div class="row">
    <div class="col-md-6">
      <EF:EFInput ename="inqu-0-pc" inline="true" value="apple" cname="apple"
type="radio" />
      <EF:EFInput ename="inqu-0-pc2" inline="true" value="dell" cname="dell"
type="radio" checked="true" />
    </div>
    <EF:EFInput ename="inqu-0-pc3" cname="computer" row="0" blockId="oscillation"
colWidth="6" ></EF:EFInput>
  </div>
  <div class="row">
    <div class="col-md-3">
      <EF:EFInput ename="inqu-0-ipad" inline="true" value="apple" cname="apple"
type="radio" />
      <EF:EFInput ename="inqu-0-ipad2" inline="true" value="xiaomi"
cname="xiaomi" type="checkbox" />
    </div>
    <EF:EFInput ename="inqu-0-ipad3" value="apple3" cname="apple" type="textarea"
colWidth="3"/>
    <EF:EFInput ename="inqu-0-ipad4" cname="ipad" row="0" blockId="oscillation"
colWidth="6" ></EF:EFInput>
  </div>
</div>
```

</EF:EFRegion>

注：<div class="col-md-4">中 4 为控件所占宽度

3.5.2.2.5.3.2 页面展示效果

EFInput布局

☒ 小米

☐ 苹果

☐ 华为

☐ apple

☒ dell

☐ apple

☐ xiaomi

appleapple3

phonephone

computercomputer

ipadipad

反例：不套上 div='row'，页面效果如下所示：

EFInput布局

☒ 小米

☐ 苹果

☐ 华为

☐ apple

☒ dell

☐ apple

☐ xiaomi

computercomputer

ipadipad

appleapple3

phonephone

反例：不设置控件宽度（<div class="col-md-4">），页面效果如下所示：

EFInput布局

phonephone

☒ 小米

☐ 苹果

☐ 华为

☐ apple

☒ dell

☐ apple

☐ xiaomi

computercomputer

ipadipad

appleapple3

这些布局会导致错位，所以将输入框，checkbox，radio 组合在一起时，要对控件的宽度进行设定，并划分清楚行与行的界限。

3.5.2.3 EEDM4004-自动补全查询

3.5.2.3.1 页面功能简介

本示例页面用于说明如何使用自动补全查询。

3.5.2.3.2 配置和 API 详情

参考 [EFAutoComplete](#) 中的描述。

3.5.2.3.3 页面开发步骤

3.5.2.3.3.1原生 JS 示例

3.5.2.3.3.1.1 JSP 页面开发

JSP 示例

```
<div class="row">
  <h5 class="push-30-l text-danger">原生 JS 示例</h5>
  <div class="col-md-4">
    <div class="form-group">
      <label for="autocomplete" class="col-xs-4 control-label">
        JS 示例
      </label>
    </div>
  </div>
</div>
```

```

        <div class="col-xs-8">
            <input id="autocomplete" name="autocomplete">
        </div>
    </div>
</div>
</div>

```

3.5.2.3.3.1.2 JS 示例

JS 示例

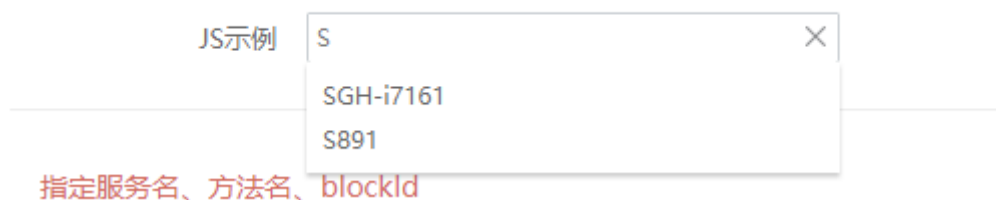
```

var dataSource = new kendo.data.DataSource({
    transport: {
        read: {
            url: IPLATUI.CONTEXT_PATH + "/service/EE10/query",
            type: 'POST',
            dataType: "json",
            contentType: "application/json"
        },
        parameterMap: function (options, operation) {
            var info = new EiInfo();
            return info.toJSONString();
        }
    },
    schema: {
        data: function (response) {
            ajaxEi = EiInfo.parseJSONObject(response);
            return ajaxEi.getBlock("result").getMappedRows();
        }
    }
});
$("#autocomplete").kendoAutoComplete({
    dataSource: dataSource,
    dataTextField: "product_id"
});

```

3.5.2.3.3.1.3 页面展示效果

原生JS示例



3.5.2.3.3.2 指定服务名，方法名，blockId

其中 **ename** 与 **dataField** 为必填参数，**ename** 为三段式定义，用于数据绑定，**dataField** 用于指定进行过滤的列；**blockId** 用于指定获取下拉数据的 **block**，**serviceName** 用于指定对应的后台服务，**queryMethod** 用于指定查询数据对应的后台处理方法。

3.5.2.3.3.2.1 JSP 页面开发

JSP 示例

```
<div class="row">
  <EF:EFAutoComplete ename="inqu_status-0-node_ename" dataField="node_ename"
cname="英文名" queryMethod="queryA"
                        serviceName="EEDM6001" resultId="blockA">
    </EF:EFAutoComplete>
</div>
```

3.5.2.3.3.2 Service 服务开发

ServiceEEDM6001

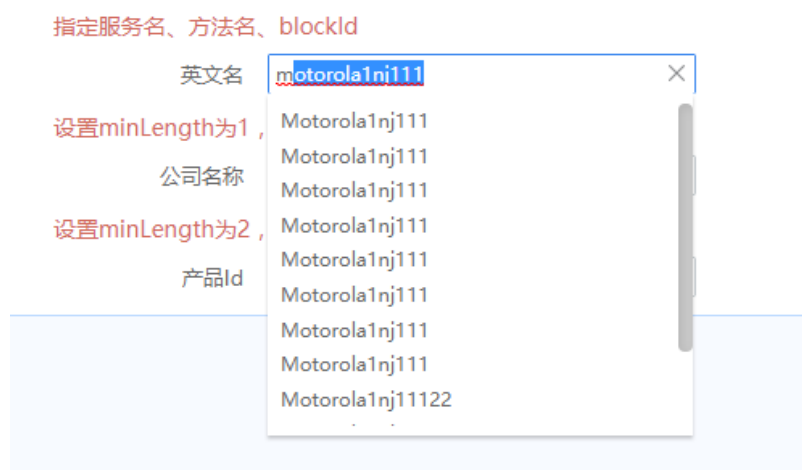
```
public EiInfo queryA(EiInfo inInfo) {
    EiBlock blockA = inInfo.getBlock("blockA");
    if (blockA!=null) {
        blockA.getBlockMeta().setBlockId("result");
        inInfo.addBlock(blockA);
    }

    EiInfo outInfo = query(inInfo, "EEDM54.query", new EEDM54());
    EiBlock block = outInfo.getBlock("result");

    block.getBlockMeta().setBlockId("blockA");
    outInfo.addBlock(block);

    return outInfo;
}
```

3.5.2.3.3.2.3 页面展示效果



3.5.2.3.3.3 设置 minLength 为 1，且 pageSize 为 15

minLength: 至少输入几位出发自动补全输入框的过滤。

pageSize: 显示信息的长度，此处显示 15 条信息（默认为 50）

3.5.2.3.3.1 JSP 页面开发

JSP 页面开发

```
<div class="row">
  <h5 class="push-30-l text-danger">设置 minLength 为 1, 且 pageSize 为 15</h5>
  <EF:EFAutoComplete ename="inqu_status-0-company_name" dataField="company_name"
cname="公司名称">
  </EF:EFAutoComplete>
</div>
```

3.5.2.3.3.2 JS 开发

可以通过设置 `serverPaging` 及 `serverFiltering` 为 `true` 来进行后台查询过滤。

JS 示例

```
IPLATUI.EFAutoComplete = {
  "inqu_status-0-product_id": {
    minLength: 2,
    dataSource: {
      pageSize: 4,
      serverPaging: true,
      serverFiltering: true
    }
  }
};
```

3.5.2.3.3.3 页面展示效果

设置minLength为1, 且pageSize为15



3.5.2.4 EEDM4006-附件上传示例

3.5.2.4.1 页面功能简介

本示例页面用于说明：

1. 如何使用 `EFUpload` 组件进行附件上传；

2. EFUpload 加载完成时的回调事件；
3. 上传附件之前的回调；
4. 上传附件或删除附件成功的回调；
5. 如何将上传的文件列表隐藏；

3.5.2.4.2 配置和 API 详情

参考 [EFUpload](#) 中的描述。

3.5.2.4.3 页面开发步骤

3.5.2.4.3.1 如何使用 EFUpload 组件进行附件上传；

3.5.2.4.3.1.1 JSP 页面开发

JSP 示例

```
<EF:EFRegion id="upload" title="上传组件示例">
  <EF:EFUpload ename="file1" docTag="hk_file1" path="A"/>
  <EF:EFUpload ename="file2" docTag="hk_file2" readonly="true" path="A/B"/>
  <EF:EFUpload ename="file3" docTag="__temp__" path="A"/>
</EF:EFRegion>
```

注意

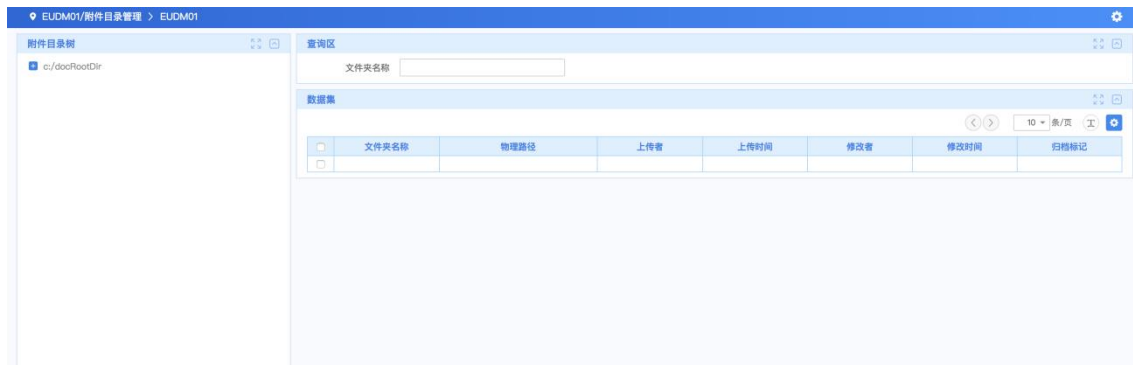
- 1.EFUpload组件中，ename，docTag，path属性必需配置；
- 2.附件上传的根目录在EDCC03页面中进行配置："配置名称"取"docRootDir"，"配置内容"结尾不能加"/"，如下：



配置环境	配置名称	配置项类别	配置项提示	配置项描述	配置内容	配置状态	创建人	创建时间
IPLAT43通用开发环境	docRootDir	初始化配置	文件管理根目录	文件管理根目录	c:/docRootDir	正常	admin	2017-0
IPLAT43通用测试环境	docRootDir	初始化配置	文件管理根目录	文件管理根目录	c:/docRootDir	正常	admin	2017-0

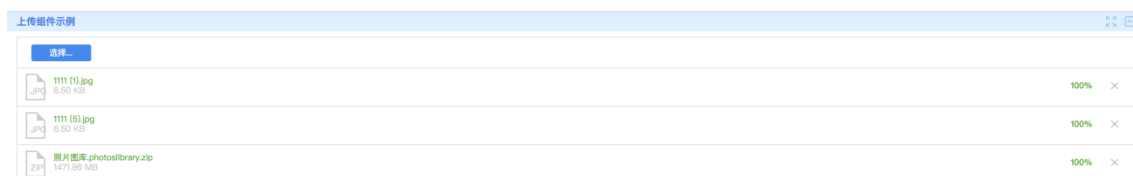
- 3.ename作为上传组件的id和name的值，同一个页面应避免取值重复；
- 4.docTag是附件标签，用于附件的分组，由业务方生成。对应于TEUDM02表的DOC_TAG列，初始化时上传组件下会默认加载对应分组的附件；
- 5.path用于配置附件上传的相对路径，相对路径的前后均不可以出现"/"；
- 6.readonly若取值"true"，则默认不允许删除附件(附件最右侧不会出现"x"供删除操作)

附件目录树和附件 分别在 EUDM01 和 EUDM02 页面中管理：

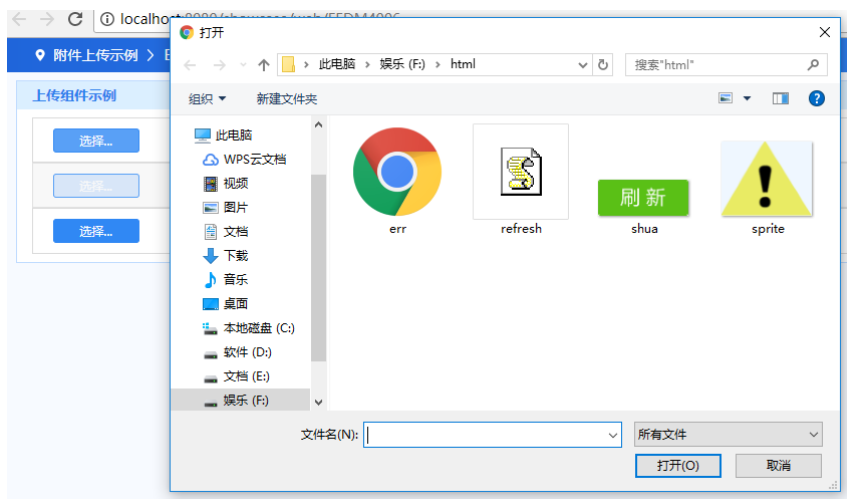


3.5.2.4.3.1.2 页面展示效果

配置 docTag="hk_file11" 后的 file1:



点击选择，选择文件打开并进行上传：



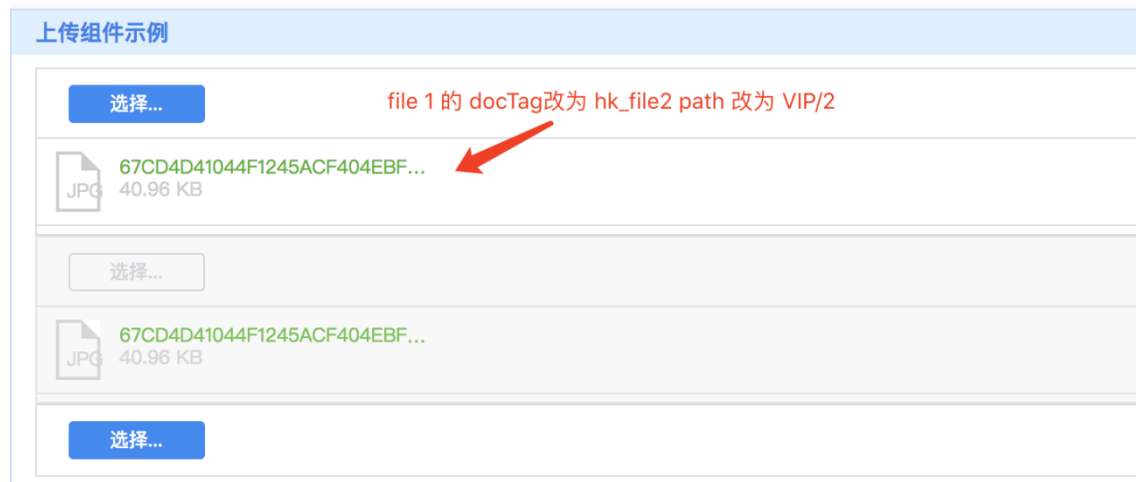
3.5.2.4.3.2 EFUpload 加载完成时的回调事件

3.5.2.4.3.2.1 JS 示例

loadComplete

```
file1: {
  loadComplete: function(e) {
    e.sender.changeSettings("hk_file2", "VIP/2"); //API: changeSettings--更改附件标
    签和相对路径。参数: docTag, path。
  }
}
```

3.5.2.4.3.2.2 页面效果如下所示



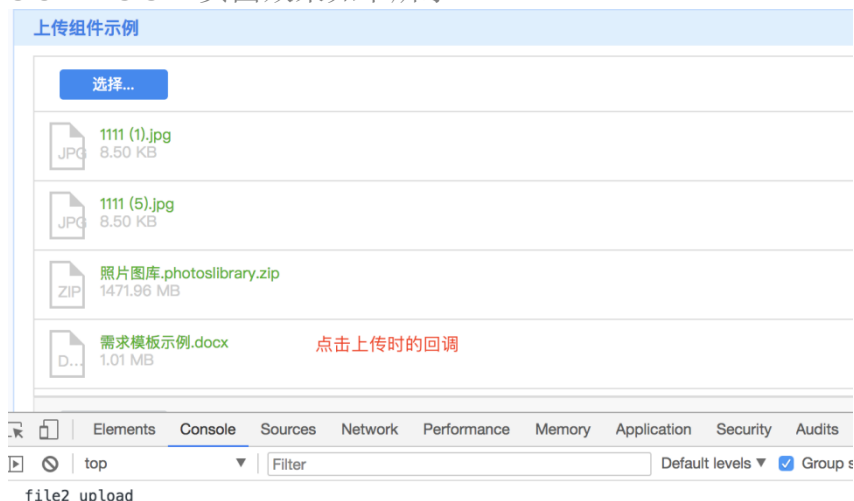
3.5.2.4.3.3 上传附件之前的回调

3.5.2.4.3.3.1 JS 示例

upload

```
IPLATUI.EFUpload = {
  file1: {
    upload: function(e) {
      console.log("file2 upload"); //点击上传时的回调
    }
  }
}
```

3.5.2.4.3.3.2 页面效果如下所示



3.5.2.4.3.4 上传附件或删除附件成功的回调

3.5.2.4.3.4.1 JS 示例

success

```

IPLATUI.EFUpload = {
  file1: {
    success: function(e) {
      console.log("file1 success docId: " + e.response.docId); //上传附件成功的回调
    }
  }
}

```

3.5.2.4.3.4.2 页面效果如下所示



3.5.2.4.3.5 如何将上传的文件列表隐藏

只需要设置 `showFileList` 属性, `showFileList: false`, 表示隐藏上传文件列表。

showFileList

```
IPLATUI.EFUpload = {
  file1: {
    showFileList: false
  }
}
```

3.5.2.4.3.5.1 页面效果如下图所示：



上图为 showFileList 为 true 时的显示效果；下图为 showFileList 为 false 时的显示效果。



当设置 **showFileList** 为 **false** 时，不仅仅是历史上传的文件会隐藏，之后上传的**所有文件都将隐藏**。若只是想隐藏历史上传记录，可以在组件 loadComplete 回调中做处理 如下：

```
IPLATUI.EFUpload = {
  file1: {
    loadComplete: function(e) {
```

```

        var uploader = e.sender.uploader;
        uploader.clearAllFiles(); // 清空所有历史上传文件
    }
};

```

3.5.2.5 EEDM4007-附件上传文件格式过滤示例

3.5.2.5.1 页面功能简介

本示例页面用于说明：在使用 **EFUpload** 组件进行附件上传时：1）如何对上传文件的格式进行过滤；2）上传的文件格式不支持时的提示信息。

在使用上传组件的过程中，我们可能想要对上传文件的格式进行校验，例如只可以上传图片格式的文件。下面主要就 **kendo** 中的原生配置方式进行说明。

3.5.2.5.2 配置和 API 详情

参考 [EFUpload](#) 中的描述。

3.5.2.5.3 页面开发步骤

3.5.2.5.3.1 对上传文件的格式进行过滤

3.5.2.5.3.1.1 JSP 页面开发

JSP 示例

```

<EF:EFRegion id="upload" title="上传组件示例">
    <EF:EFUpload ename="file2" docTag="hk_file12" path="B"/>
</EF:EFRegion>

```

3.5.2.5.3.1.2 JS 开发

JS 示例

```

IPLATUI.EFUpload = {
    file2: {
        validation: {
            allowedExtensions: [".jpg"], //文件格式过滤
            maxFileSize: 900000,
            minFileSize: 300
        }
    }
};

```

其中，**allowedExtensions** 是允许上传的文件扩展名列表，**maxFileSize** 是定义允许上传的最大文件大小（以字节为单位），**minFileSize** 是定义允许上传的最小文件大小（以字节为单位）。

3.5.2.5.3.2 上传的文件格式不支持时的提示信息

3.5.2.5.3.2.1 JS 开发

JS 示例

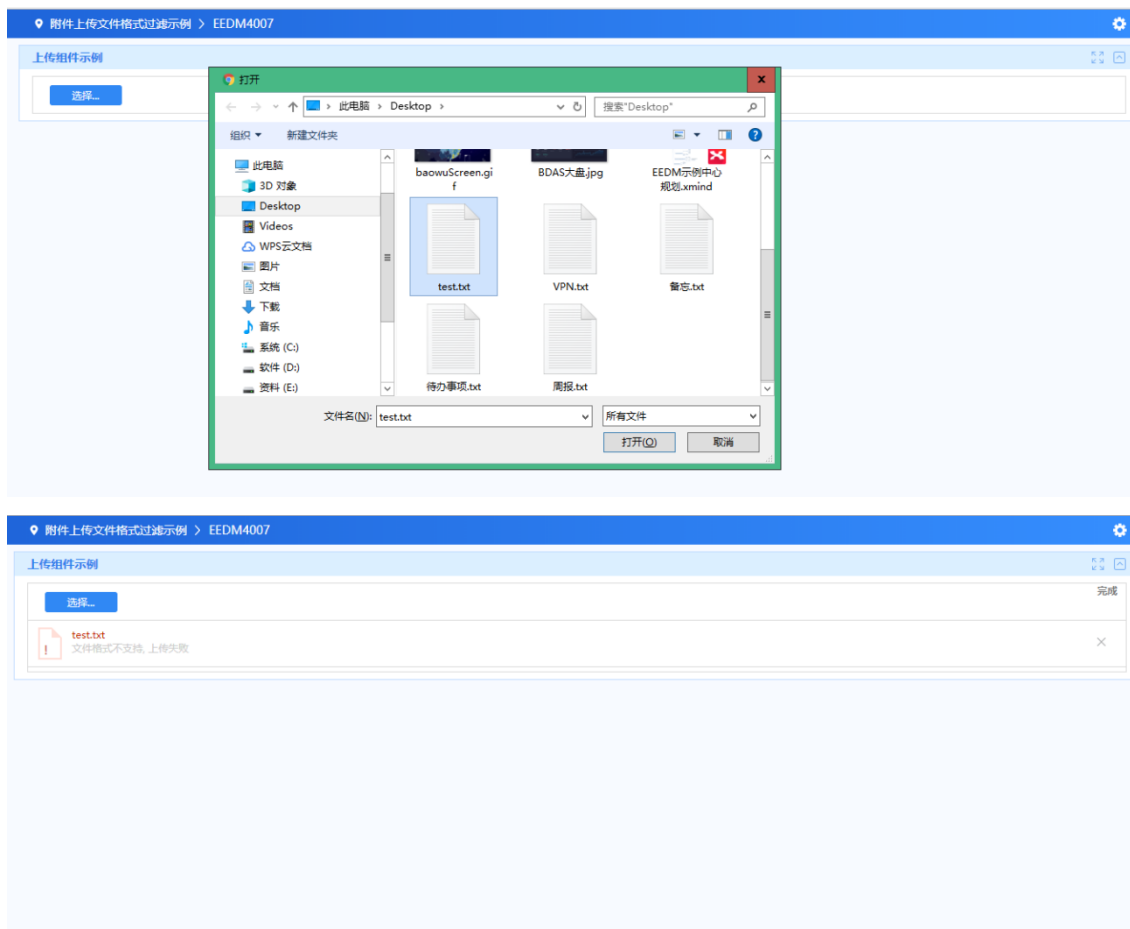
```
IPLATUI.EFUpload = {
  file2: {
    // 不支持格式提示信息
    localization: {
      invalidFileExtension: "文件格式不支持，上传失败"
    }
  }
};
```

invalidFileExtension：为无效的文件扩展名设置验证消息的文本。

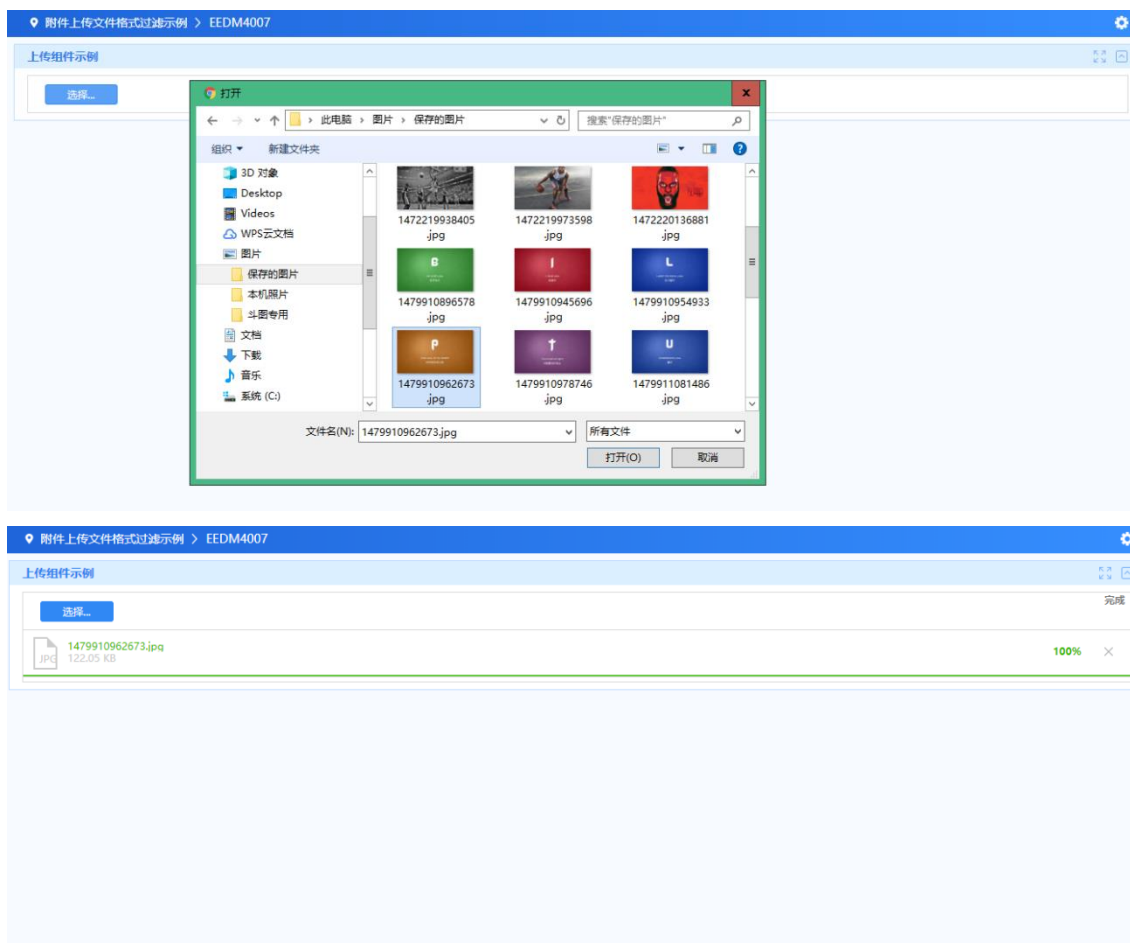
以上这两者可以当上传的文件扩展名不符合设置时结合应用。

3.5.2.5.3.3 页面效果如下所示

上传失败示例：



上传成功示例：



3.5.2.6 EEDM4010-EFPopupInput 输入示例

3.5.2.6.1 页面功能简介

本示例页面用于说明，EFPopupInput 的弹出的几种类型：下拉选择框、Grid、小代码。

3.5.2.6.2 配置和 API 详情

参考 [EFPopupInput](#)、[EFGrid](#) 和 [EFTree](#) 文档。

3.5.2.6.3 页面开发步骤

3.5.2.6.3.1 自定义弹出下拉选择框

下拉示例

```
// tag 标签
<EF:EFPopupInput ename="inqu_status-0-param2" cname="下拉选择框" readonly="true"
    containerId="ef_popup_select" textField="option"
    popupTitle="下拉选择框">
</EF:EFPopupInput>
```

```

.....
// 引用 DIV, 默认隐藏(display:none)
// EFPopupInput 中 containerId="ef_popup_select" 值是该 DIV 的 id 名称
<div id="ef_popup_select" style="display: none; height:200px; width: 200px">
    <div id="ef_popup_html" >
        </div>
    </div>

// ef_popup_html 渲染
<script>
    $("#ef_popup_html").kendoDropDownList({
        dataSource: ["选项 01", "选项 02"],
        select: function (e) {
            var item = e.item;
            var text = item.text();
            $("#inqu_status-0-param2").val(text); // 数据回填
            $("#inqu_status-0-param2_textField").val(text);

            var window = $("#ef_popup_select").data("kendoWindow");
            window.close();
        }
    });
</script>

```

使用注意：大家都知道，一个 Input 框中只能有一个 value 值。而很多时候，我们可能是以一种（value, label）形式存在的，例如：（china: 中国，japan : 日本）label 是显示给用户看的，而 value 是需要与进行后端交互的内容。在这种情况下，Tag 处理的方案是：使用两个 Input。一个用于展示 Input。还有一个是隐藏 Input，我们用于存储 value，与后端进行交互。

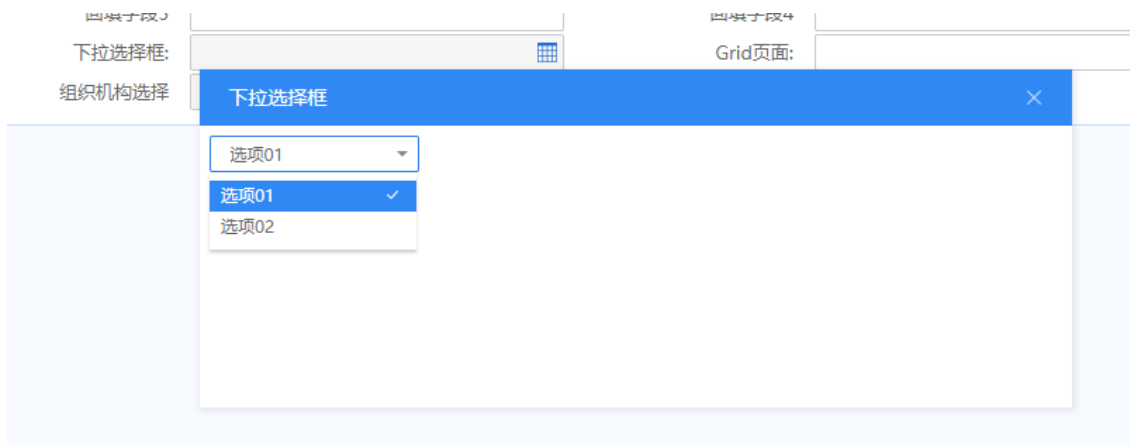
开发人员不清楚具体 Input 框时候，可以直接查看由平台渲染的页面中 dom 节点信息。

需要与后端进行交互的是第一个 Input，会在界面上隐藏，它的 Id 是由 EFPopupInput 中 ename 产生。而第二个 Input 只是用于展示信息。细心的开发人员已经发现了，它是在第二个 Input 框中都添加了一个字段_textField，就是：inqu_status-0-param1_textField。

所以 使用自定义的现实弹出内容这种方式的时候，回填数据需要回填到两个 Input 框中！！！！

就如同上面的“静态页面示例”中 JS 代码块。大家需要记得把我们需要对接的数据回写到隐藏 input 中。

示例效果如下：



3.5.2.6.3.2 自定义弹出 Grid

使用注意：在页面开发中，当我们使用 **EFGrid** 去渲染多个 **Grid** 的时候，我们需要注意，多个 **Grid** 的 **blockId** 应该是不一样的。这里被包含在 **DIV** 块中 **EFGrid** 跟我们平时开发的 **EFGrid** 一样。

我们在 **grid** 的后面添加一个 **EFButton**：确定，来作为回填的按钮。当我们点击这个按钮的时候，会把我们选中的数据回填到弹出的 **input** 框中。这里为了防止 **Grid** 中被选中多行，所以需要设置 **checkMode="single"**。

自定义弹出 **Grid** 示例

```
// tag 标签
<EF:EFPopupInput ename="inqu_status-0-param3" cname="Grid 页面" resizable="true"
    containerId="ef_popup_grid" textField="grid" popupWidth="500"
    popupTitle="Grid 页面">
</EF:EFPopupInput>

.....
// 引用 DIV, 默认隐藏(display:none)
<div id="ef_popup_grid" style="display: none">
    ...
    <EF:EFRegion id="result" title="记录集">
        <EF:EFGrid blockId="result" checkMode="single" serviceName="EEDM6000"
            autoDraw="no" > <!-- EFGrid 默认 autoDraw="yes" -->
            <EF:EFColumn ename="product_id" locked="true" cname="产品编号"/> <!-- 动态
            设置的列名 -->
            <EF:EFColumn ename="company_name" locked="true" cname="公司名称"/>
            ...
        </EF:EFGrid>
        <div class="k-window-save k-popup-save">
            <EF:EFButton cname="确定" ename="ef_popup_grid_fillback" layout="1"
            class="k-button"></EF:EFButton>
        </div>
    </EF:EFRegion>
</div>

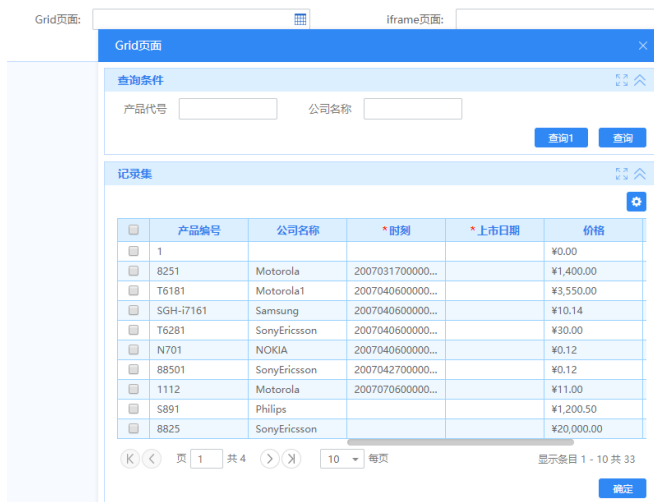
// grid 回填设置
$("#ef_popup_grid_fillback").on("click",function (e) {
    var checkRows = grid.getCheckedRows();
```

```

// checkRows 存储的是选中的列
// 获取第一列
var model = checkRows[0];
if (model) {
    $("#inqu_status-0-param3").val(model['product_id']);
    $("#inqu_status-0-param3_textField").val(model['company_name']);
    var popupGridWindow = $("#ef_popup_grid").data("kendoWindow");
    popupGridWindow.close();
} else {
    alert("没有选中数据");
}
});

```

示例效果如下：



Code Block 3 自定义弹出 Grid 事件使用示例

```

//tag 标签
<EF:EFPopupInput ename="inqu_status-0-param1" cname="显示的真实值"
serviceName="EEDM6000" methodName="dynamicQuery" resultId="result"
popupType="ServiceGrid"
valueField="product_id" textField="company_name" required="true"
clear="true"
originalInput="true" columnNames="product_id,company_name,price"
columnCNames="产品号,公司名,价格"
resizable="true" popupTitle="查询多次服务" popupWidth="500"
filterPopupGrid="false">
</EF:EFPopupInput>

// 事件执行顺序
// init --> query --> loadComplete --> backFill, clearInput

var i = 0;
IPLATUI.EFPopupInput = {
    "inqu_status-0-param1": {

        /**

```

辑;

```

* 初始化弹窗中的内容时触发，可以根据业务要求控制是否进行弹窗；
*
* 弹窗中的 EFGrid 查询之前触发的事件，通常可以在此处执行自定义 div 内容的相关逻辑；

* @param e 事件对象
* e.container 弹出窗口的 jQuery 对象
*/
init: function (e) {
    console.log("Step" + (++i) + ": ----init----");
    // 阻止弹窗
    e.preventDefault();
},

/**
* EFGrid 查询时触发的事件，通常在此处，设置查询条件
* 注意：必须返回一个 EiInfo，作为初始查询条件
* 配置了 query 时，默认每次打开窗口都会进行一次查询
*/
query: function () {
    console.log("Step" + (++i) + ": ----query----");

    // 将自定义 div 中的字段构造成查询条件 EiInfo
    var queryInfo = EiInfo.build("#ef_popup_query");
    return queryInfo;
},

// 配置了 query 时，控制打开窗口是否重新查询的开关，默认是 true
// queryRefresh: false,

/**
* EFGrid 查询完成时触发的事件，通常在此处绑定自定义 div 中 按钮的查询逻辑
*
* @param e 事件对象
* e.grid 弹窗内的 Grid 对象
*/
loadComplete: function (e) {
    console.log("Step" + (++i) + ": ----grid loadComplete----");
    var grid = e.grid;

    // 绑定查询按钮的点击事件
    $("#custom_query").click(function () {
        grid.dataSource.page(1);
    });
},

clearInput: function (e) {
    console.log("Step" + (++i) + ": ----clearInput----");
},

/**
* grid 中的数据回填时触发的事件
* @param e 回填数据事件
* e.sender 弹出窗口对象
* e.grid 弹出的 grid 对象
* e.model 选中的数据行 model
*/
backFill: function (e) {

```

```

        console.log("Step" + (++i) + ": ----backFill----");
        e.preventDefault(); // 不使用默认的回填逻辑
    }
}
};

```

3.5.2.6.3.3 小代码 SQL 查询弹出 Grid

在使用弹出小代码的输入框前，首先需要在系统中配置小代码。小代码一般都是以代码明细英文名称, 代码明细中文名称这种形式存在的。也可以配置小代码 Grid，我们在处理小代码的时候，需要对其进行 SQL 配置。

步骤如下：项目元数据中心->代码管理→代码分类管理 (EDCM00) 把我们需要进行处理的 查询的 sql 语句放入到“对应的 URL”中。比如我们这次使用的 sql 语句是：

```
select TABLE_ENAME, TABLE_CNAME, MODULE_ENAME from
iplatv5.teddbt0; 这里我们需要带上 Schema (iplatv5) 。 不同的版本，
需要对应不同的 Schema。
```

如下图：

记录集											
	*项目英文名	*代码分类编号	代码分类名称	代码分类...	代码类型	代码管理层级	操作	对应URL	国际编号	关联字段	备注
☐	IPLAT43	iplat.codeHierarchy	代码管理层级	codeHierarc...	流程类		代码明细				admin
☐	IPLAT43	iplat.codeStyle	代码类型	codeStyle			代码明细				admin
☐	IPLAT43	iplat.ed.buttonLayout	按钮样式1	buttonLayout	统计类	系统层	代码明细				limin
☐	IPLAT43	iplat.ed.buttonPosit...	按钮位置	buttonPosit...	统计类	系统层	代码明细				limin
☐	IPLAT43	iplat.ed.configType	配置项类别	config_type	流程类	外部代码	代码明细				admin
☐	IPLAT43	iplat.ed.formStyle	画面技术风格	formStyle			代码明细			100	admin
☐	IPLAT43	iplat.ed.formType	画面类型	formType			代码明细				admin
☐	IPLAT43	iplat.ed.md.domainMo...	领域模型类别	domainModel...			代码明细				admin
☐	IPLAT43	iplat.ed.mmType	中间件类型	mmType	统计类	系统层	代码明细				admin
☐	IPLAT43	iplat.ed.moduleType	模块类型	moduleType	统计类	系统层	代码明细				admin

配置好后端的小代码后，我们就可以针对这个小代码进行开发了。

小代码示例代码

```

// tag 标签使用
<EF:EFPopupInput ename="inqu_status-0-param0" cname="小代码"
resultId="result"

clear="true" popupType ="DynamicGrid"
codeName="iplatSelectWithGrid"

valueField="FORM_ENAME" textField="FORM_CNAME"

required="true"
columnEnames="FORM_ENAME, FORM_CNAME, MODULE_ENAME_1"
columnCNames="画面英文名, 画面中文名, 一级模块名"
backFillFieldIds="inqu_status-0-field1, inqu_status-0-field2"
backFillColumnIds="FORM_ENAME, FORM_CNAME" resizable="true"
popupTitle="小代码查询" readonly="true" filterPopupGrid="false">

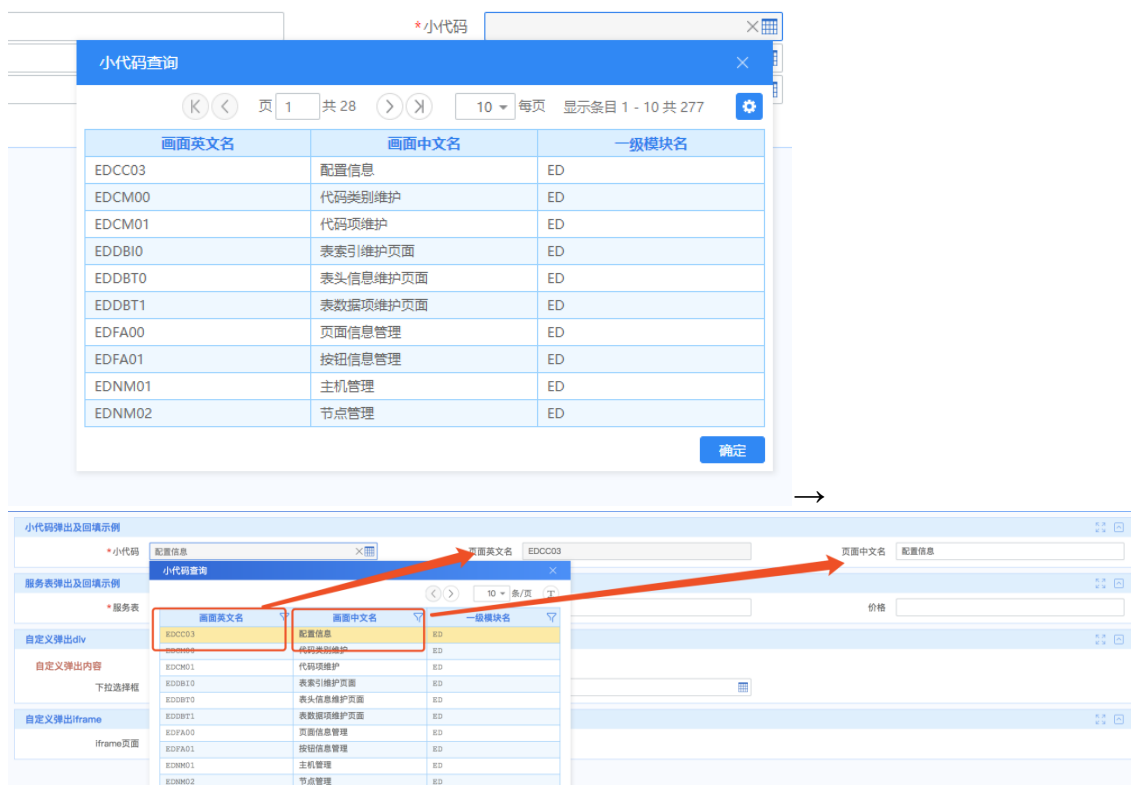
```

```

</EF:EFPopupInput>
<EF:EFInput ename="inqu_status-0-field1" cname="页面英文名" disabled="true"/>
<EF:EFInput ename="inqu_status-0-field2" cname="页面中文名"/>

```

示例效果如下：



- **backFillFieldIds**: 需要回填数据其他的输入框的 id 数组，此处为 inqu_status-0-field1, inqu_status-0-field2
- **backFillColumnIds**: 回填到界面中的列，是 columnFields 的子集，此处为 FORM_ENAME, FORM_CNAME

注意事项：

1. **textField** : 显示值对应的列名,也被用做展示 input 的 id 。
valueField : 真实值对应的列名，提交到后台。
2. **popupType = "DynamicGrid"** 为必须。
3. 后端小代码配置中，sql 查询语句查询到的 **columns** 与 **tag** 标签中 **columnEnames** 是包含关系。
4. **columnEnames** 与 **columnCnames** 是一一对应关系，且与位置相关。
5. **backFillFieldIds** 与 **backFillColumnIds** 是一一对应关系，且与位置相关。

3.5.2.6.3.4 服务名，方法名查询弹出 Grid

弹出 ServiceGrid

```

<EF:EFPopupInput ename="inqu_status-0-param1" cname="服务表"
    resultId="result" popupWidth="500"
    serviceName="EEDM6000" methodName="dynamicQuery"

    clear="true" popupType ="ServiceGrid"

    valueField="product_id" textField="company_name"

    required="true"
    columnEnames="product_id,company_name,price"
    columnCNames="产品号,公司名,价格"
    resizable="true"

    backFillFieldIds="inqu_status-0-field3, inqu_status-0-field4"
    backFillColumnIds="company_name,price"
    popupTitle="服务表带查询" filterPopupGrid="false">
</EF:EFPopupInput>
<EF:EFInput ename="inqu_status-0-field3" cname="公司名称"/>
<EF:EFInput ename="inqu_status-0-field4" cname="价格"/>

```

基本上 tag 中的参数信息可以满足我们的大部分需求，为了让开发人员能够开发一些定制化的功能，平台在 EFPopupInput 前端新增一些事件机制，详情请参考 [EFPopupInput](#) 中事件部分。

这里结合具体实例进行说明。JS 代码如下：

JS

```

IPLATUI.EFPopupInput = {
    "inqu_status-0-param1": {
        query: function () {
            var queryInfo = new EiInfo();

            if (true /*业务逻辑判断*/) {
                queryInfo.set("inqu_status1-0-custom", "custom");
                queryInfo.set("inqu_status1-0-company_name", $("#inqu_status-0-field3").val());
            } else {
                queryInfo.set("inqu_status1-0-company_name", "宝信软件");
            }
            return queryInfo;
        },
        init: function (e) {
            // console.log(e);
        },
        clearInput: function (e) {
            // console.log(e);
        }
    }
};

```

示例效果如下：



我们在前端 JS 中配置 EFPopupInput 时候，需要使用一个全局的配置名称 `IPLATUI.EFPopupInput`。使用 EFPopupInput.Tag 中的 `ename="inqu_status1-0-param1"` 来进行定位我们需要配置的 PopupInput。

query 方法是调用后端服务之前，平台提供给开发人员设置参数的一个接口，这个接口返回的是 `EiInfo`。上面的例子中，开发人员通过设置三段式的方式，对 `queryInfo` 进行设值，return 的 `EiInfo` 会在 EEDM6000 的 `query` 方法中作为 `inInfo`。

```

▶ f msg = ""
▶ f msgKey = ""
▶ f detailMsg = ""
f sys = {HashMap@7539} size = 0
▼ f blocks = {HashMap@7540} size = 2
  ▼ 0 = {HashMap$Node@7544} "inqu_status" -> "com.baosight.iplat4j.c ... View
    ▶ key = "inqu_status"
    ▼ value = {EiBlock@7547} "com.baosight.iplat4j.core.ei.EiBlock@683 ... View
      ▶ f blockMeta = {EiBlockMeta@7554} "com.baosight.iplat4j.core.ei. ... View
        ▼ f rows = {ArrayList@7555} size = 1
          ▼ 0 = {HashMap@7562} size = 2
            ▶ 0 = {HashMap$Node@7566} "custom" -> "custom"
            ▶ 1 = {HashMap$Node@7567} "company_name" ->
          f attr = {HashMap@7556} size = 0
        ▶ 1 = {HashMap$Node@7545} "result" -> "com.baosight.iplat4j.core.ei.l... View
      f status = 0
    ▶ f attr = {HashMap@7541} size = 5

```

在后端可以看到我们获取的相关 `EiInfo` 的信息。

郑重声明： 设置默认的查询 `block` 名称时候，不能设置成为 `inqu_status`，此 `block` 被平台内部使用，如果使用此 `block` 会造成外部 `block` 与平台内部 `block` 的冲突。

backFill 方法是用户在“双击选中”或者“点击保存”按钮后，平台提供给开发人员的回调方法。

`backFill` 的回调方法中，会带有一个含有 `e` 的参数。

```
e
▼ Object {grid: init, model: i, sender: i, _defaultPrevented: false, preventDefault: function...} ⓘ
  ► grid: init
  ► isDefaultPrevented: function ()
  ► model: i
  ► preventDefault: function ()
  ► sender: i
  ► _defaultPrevented: false
  ► __proto__: Object
```

- **e.grid** 表示弹出的 grid
- **e.model** 表示选中的 row 的 model 信息
- **e.preventDefault** 是平台提供给开发人员去阻止后续事件的处理，像回填数据，关闭窗口。通过 **e.preventDefault()** 都将会被阻止，当然，没有被调用，后续事件不会被阻止

此外，开发人员可以在 **backFill** 中调用 **e.sender.closeWindow();**方法来关闭弹出窗口。

3.5.2.6.3.5 弹出 Tree

在开始使用弹出 **Tree** 时候的时候，首先需要提供一个 **Tree** 相关的服务，这里采用 **EEDM05** 相关服务，不熟悉的开发人员可以参考 [EFTree](#)。

JSP

```
<EF:EFTreeInput blockId="in" row="0" ename="companyCustomizeField" cname="组织机构选择"
serviceName="EEDM05"
methodance="query" valueField="label" textFiled="text" onlyLeaf="true"
hasChildren="leaf" readonly="true">
</EF:EFTreeInput>
```



onlyLeaf="true"表示：仅回填叶子节点，而不回填根节点。

JS

```
IPLATUI.EFTree = {
  "popup_show_in-0-companyCustomizeField_tree": {
    ROOT: {label: "root", text: "地区分布", leaf: true}
  }
};
```

示例效果如下：



注意事项

1. 在 JS 中配置 ROOT: `ROOT: {label: "root", text: "地区分布", leaf: true}`
2. 在 tree 中选中后，把数据回填到相关的 input 中，这里回填需要回填到两个 input 中，一个是用于展示的 input 框，另外一个存储的实际的 value 的 input
3. select 选中后，关闭本身的 window 框

3.5.2.6.3.6 弹出 Iframe

iframe.jsp

```
// tag 标签
<EF:EFRegion id="R4" title="自定义弹出 iframe">
    <EF:EFPopupInput ename="inqu_status-0-param4" cname="iframe 页面" popupHeight="500"
    popupWidth="960"
        containerId="ef_popup_iframe" textField="iframe" resizable="true"
    draggable="true"
        popupTitle="iframe 页面">
    </EF:EFPopupInput>
</EF:EFRegion>
// 弹出的 iframe
<div id="ef_popup_iframe" style="display: none">
    <div class="row" style="height: 92%">
        <iframe frameborder="0" title="" class="k-content-frame"
        src="${ctx}/web/EEDM6010"></iframe>
    </div>
    <div class="k-window-save k-popup-save">
        <EF:EFButton ename="ef_popup_iframe_fillback" cname="确定"
            class="i-btn-wide" style="margin-right: 40px;">
        </EF:EFButton>
    </div>
</div>
```

iframe.js

```
// 关闭 iframe 时候回填数据
$("#ef_popup_iframe_fillback").on("click",function (e) {
    var checkRows = window.frames[0].resultGrid.getCheckedRows();
```

```

var model = checkRows[0];
if (model) {
    $("#inqu_status-0-param4").val(model['product_id']); // 数据回填
    $("#inqu_status-0-param4_textField").val(model['company_name']);
    var windowFrame = $("#ef_popup_iframe").data("kendoWindow");
    windowFrame.close();
} else {
    WindowUtil({title: '信息', content: "<div class='kendo-del-message'>没有选中数据</div>"});
}
});

```

选择一行，关闭 `iframe` 时回填数据。

注意点：如果一个页面有多个 `iframe` 弹出，如何获取弹出框？

首先，需要在 `iframe` 标签中使用 `name` 属性，例如是 `name="produce"`。

再改动上文的代码 `window.frames[0].resultGrid.getCheckedRows()`，改成 `window.frames["produce"].resultGrid.getCheckedRows()`。

示例效果如下：



3.5.2.7 EEDM4012-EFPopupInput API 示例

3.5.2.7.1 页面功能简介

本示例页面用于介绍：**EFPopupInput** 的使用说明：设置输入框的显示的值
为真实值或者显示值，以及其 API：`readonly`，`enable`，`text`，`value`，
`setAllFields` 和 `clear`。

3.5.2.7.2 配置和 API 详情

参考 [EFPopupInput](#) 中的描述。

3.5.2.7.3 页面开发步骤

3.5.2.7.3.1 使用说明

使用场景：EFPopupInput 组件，用于处理个性化要求较高的表单输入。

使用方式：通过弹出窗口的方式，查询，选择窗口中的数据，回填到相应的输入框内。

简单说明：通过 [EEDM4010-EFPopupInput 输入示例](#) 介绍得知，EFPopupInput 本质上是个下拉框输入，输入框中只是显示值(textField)，通常情况下不能直接编辑 EFPopupInput 输入框。

组成说明：真实值 (valueField) 输入框的 id 是: EFPopupInput 的 **ename** 属性值。

显示值 (textField) 输入框的 id 是: EFPopupInput 的 **ename** 属性值 + "_textField" (ename 后拼接 "_textField")。

3.5.2.7.3.1.1 通常情况：输入框显示显示值，隐藏真实值

3.5.2.7.3.1.2 JSP 页面开发

JSP 示例

```
<EF:EFPopupInput ename="inqu_status-0-param0" cname="隐藏的真实值"

    serviceName="EEDM6000" methodName="dynamicQuery"
    resultId="result"

    required="true" clear="true" popupType="ServiceGrid"

    valueField="product_id" textField="company_name" // 真实值 id:
product_id 显示值 id: company_name

    columnNames="product_id,company_name,price"
    columnCNames="产品号,公司名,价格"

    readonly="true"

    resizable="true" popupTitle="只读状态" popupWidth="800">
</EF:EFPopupInput>
```

3.5.2.7.3.1.3 页面效果如下所示



输入框中显示的是**显示值(textField)**，此处显示值取得是 company_name 的值，所以此处输入框的显示值为“111”，隐藏的真实值为“abcde5”。

3.5.2.7.3.1.4 设置 originalInput="true" 时：输入框显示真实值

3.5.2.7.3.1.5 JSP 页面开发

JSP 示例

```
<EF:EFPopupInput ename="inqu_status-0-param1" cname="显示的真正值"
    serviceName="EEDM6000" methodName="dynamicQuery"
    resultId="result" popupType="ServiceGrid"

    originalInput="true" // 其他不变，增加 originalInput="true"属性

    valueField="product_id" textField="company_name" //输入框显示是
valueField 的值

    required="true" clear="true"

    columnEnames="product_id,company_name,price"
    columnCnames="产品号,公司名,价格"

    resizable="true" popupTitle="查询多次服务" popupWidth="500"
    filterPopupGrid="false">
</EF:EFPopupInput>
```

当业务需要直接输入时，应该配置属性 `originalInput="true"`，则 `EFPopupInput` 只有 `valueField`，而不会显示显示值（`textField`）。

3.5.2.7.3.2PI 示例

3.5.2.7.3.2.1 设为只读/非只读---readonly

3.5.2.7.3.2.1.1 JSP 页面开发

JSP 示例

```
<EF:EFPopupInput ename="inqu_status-0-param2" cname="readonly 状态"

    serviceName="EEDM6000" methodName="dynamicQuery"
    resultId="result"

    valueField="product_id" textField="company_name"

    columnEnames="product_id, company_name, price"
    columnCNames="产品号, 公司名, 价格"

    readonly="true" originalInput="true"
    required="true" clear="true" popupType="ServiceGrid"

    resizable="true" popupTitle="只读状态" popupWidth="800">
</EF:EFPopupInput>

<div class="col-md-1 col-md-offset-2">
    <EF:EFButton ename="setReadonly" cname="设置只读" class="i-btn-wide"/>
</div>

<div class="col-md-1">
    <EF:EFButton ename="unsetReadonly" cname="设置可写" class="i-btn-wide"/>
</div>
```

3.5.2.7.3.2.1.2 JS 开发

JS 示例

```
// 设置只读
$("#setReadonly").on("click", function () {
    IPLAT.EFPopupInput.readonly("#inqu_status-0-param2", true);
});

// 设置非只读
$("#unsetReadonly").on("click", function () {
    IPLAT.EFPopupInput.readonly("#inqu_status-0-param2", false);
});
```

3.5.2.7.3.2.1.3 页面效果如下所示

如上所示，当 **EFPopupInput** 的状态是可写时，既可以回填输入，也可以手写输入；而状态是只读状态时，只能弹出选择输入，不能手写输入。

3.5.2.7.3.2.2 为禁用/非禁用---enable

3.5.2.7.3.2.2.1 JSP 页面开发

JSP 示例

```
<EF:EFPopupInput ename="inqu_status-0-param3" cname="disabled状态"
    serviceName="EEDM6000" methodName="dynamicQuery"
    resultId="result" popupType="ServiceGrid"

    valueField="product_id" textField="company_name"

    columnEnames="product_id,company_name,price"
    columnCNames="产品号,公司名,价格"

    disabled="true" required="true" clear="true"

    resizable="true" popupTitle="只读状态" popupWidth="800">
</EF:EFPopupInput>

<div class="col-md-1 col-md-offset-2">
    <EF:EFButton ename="unsetEnable" cname="设置禁用" class="i-btn-wide"/>
</div>

<div class="col-md-1">
    <EF:EFButton ename="setEnable" cname="设置可用" class="i-btn-wide"/>
</div>
```

3.5.2.7.3.2.2.2 JS 开发

JS 开发

```
// 设置禁用
$("#unsetEnable").on("click", function () {
    IPLAT.EFPopupInput.enable("#inqu_status-0-param3", false);
});
```

```
// 设置可用
$("#setEnable").on("click", function () {
    IPLAT.EFPopupInput.enable("#inqu_status-0-param3", true);
});
```

3.5.2.7.3.2.2.3 页面效果如下所示

禁用 disabled 状态的 EFPopupInput，不能弹出回填，不能直接输入

* disabled状态

设置禁用

设置可用

设为禁用后，点击按钮，将不能弹出窗口进行回填，也不能输入。

3.5.2.7.3.2.3 获取显示值/真实值数据---value, text, setAllFields

3.5.2.7.3.2.3.1 JSP 页面开发

JSP 示例

```
<EF:EFPopupInput ename="inqu_status-0-param4" cname="获取/设置数据"

    serviceName="EEDM6000" methodName="dynamicQuery"
    resultId="result" popupType="ServiceGrid"

    valueField="product_id" textField="company_name"

    columnEnames="product_id,company_name,price"
    columnCnames="产品号,公司名,价格"

    popupTitle="获取/设置数据" popupWidth="800">
</EF:EFPopupInput>
<div class="col-md-1 col-md-offset-2">
    <EF:EFButton ename="getTextField" cname="获取显示值" class="i-btn-wide"/>
</div>
<div class="col-md-1">
    <EF:EFButton ename="getValueField" cname="获取真实值" class="i-btn-wide"/>
</div>
<div class="col-md-1">
    <EF:EFButton ename="setTextField" cname="设置显示值" class="i-btn-wide"/>
</div>
<div class="col-md-1">
    <EF:EFButton ename="setValueField" cname="设置真实值" class="i-btn-wide"/>
</div>
<div class="col-md-2">
    <EF:EFButton ename="setAllFields" cname="设置真实和显示值" class="i-btn-wide"/>
</div>
</div>
```

3.5.2.7.3.2.3.2 JS 开发

JS 开发

```
// 获取显示值数据
```

```

$("#getTextField").on("click", function () {
    console.log(IPLAT.EFPopupInput.text("#inqu_status-0-param4"))
});

// 获取真实值数据
$("#getValueField").on("click", function () {
    console.log(IPLAT.EFPopupInput.value("#inqu_status-0-param4"))
});

// 设置显示值数据
$("#setTextField").on("click", function () {
    IPLAT.EFPopupInput.text("#inqu_status-0-param4", "wang")
});

// 设置真实值数据
$("#setValueField").on("click", function () {
    IPLAT.EFPopupInput.value("#inqu_status-0-param4", "miao")
});

// 同时设置真实值和显示值
$("#setAllFields").on("click", function () {
    IPLAT.EFPopupInput.setAllFields("#inqu_status-0-param4", "dog", "wang")
});

```

3.5.2.7.3.2.3.3 页面效果如下所示



3.5.2.7.3.2.4 清除所有的输入框

3.5.2.7.3.2.4.1 JSP 页面开发

JSP 示例

```

<EF:EFPopupInput ename="inqu_status-0-param5" cname="清除其他指定回填的输入框"
    resultId="result" popupType="DynamicGrid"
    codeName="iplatSelectWithGrid"

    valueField="FORM_ENAME" textField="FORM_CNAME"

    popupTitle="清除输入框"

    columnNames="FORM_ENAME, FORM_CNAME, MODULE_ENAME_1"
    columnCNames="画面英文名, 画面中文名, 一级模块名"

    backFillFieldIds="inqu_status-0-field1, inqu_status-0-field2"
    backFillColumnIds="FORM_ENAME, FORM_CNAME" resizable="true">

```



```

</EF:EFPopupInput>
<EF:EFInput ename="inqu_status-0-field1" cname="页面英文名" />

<div class="col-md-1">
  <EF:EFButton ename="clearField" cname="清除输入框" class="i-btn-wide"/>
</div>

```

3.5.2.7.3.2.4.2 JS 开发

JS 开发

```

// 清除其他指定的输入框
$("#clearField").on("click", function () {
  IPLAT.EFPopupInput.clear("#inqu_status-0-param5"), true)
});

```

3.5.2.7.3.2.4.3 页面效果如下所示



3.5.2.8 EEDM4203-EFPopupInput 事件使用示例

3.5.2.8.1 页面功能简介

本示例页面用于介绍：**EFPopupInput** 相关事件的使用说明。

3.5.2.8.2 配置和 API 详情

参考 [EFPopupInput](#) 中的描述。

3.5.2.8.3 页面开发步骤

3.5.2.8.3.1 事件使用说明

popupInput 事件使用示例

```

//tag 标签
<EF:EFPopupInput ename="inqu_status-0-param1" cname="显示的真实值"
  serviceName="EEDM6000" methodName="dynamicQuery" resultId="result"
  popupType="ServiceGrid"
  valueField="product_id" textField="company_name" required="true"
  clear="true"

```

```

        originalInput="true" columnNames="product_id,company_name,price"
columnNames="产品号,公司名,价格"
        resizable="true" popupTitle="查询多次服务" popupWidth="500"
filterPopupGrid="false">
</EF:EFPopupInput>

// 事件执行顺序
// init --> query --> loadComplete --> backFill, clearInput

var i = 0;
IPLATUI.EFPopupInput = {
    "inqu_status-0-param1": {

        /**
         * 初始化弹窗中的内容时触发, 可以根据业务要求控制是否进行弹窗;
         *
         * 弹窗中的 EFGrid 查询之前触发的事件, 通常可以在此处执行自定义 div 内容的相关逻辑;
         * @param e 事件对象
         * e.container 弹出窗口的 jQuery 对象
         */
        init: function (e) {
            console.log("Step" + (++i) + ": ----init----");
            // 阻止弹窗
            e.preventDefault();
        },

        /**
         * EFGrid 查询时触发的事件, 通常在此处, 设置查询条件
         * 注意: 必须返回一个 EiInfo, 作为初始查询条件
         * 配置了 query 时, 默认每次打开窗口都会进行一次查询
         */
        query: function () {
            console.log("Step" + (++i) + ": ----query----");

            // 将自定义 div 中的字段构造成查询条件 EiInfo
            var queryInfo = EiInfo.build("#ef_popup_query");
            return queryInfo;
        },

        // 配置了 query 时, 控制打开窗口是否重新查询的开关, 默认是 true
        // queryRefresh: false,

        /**
         * EFGrid 查询完成时触发的事件, 通常在此处绑定自定义 div 中 按钮的查询逻辑
         *
         * @param e 事件对象
         * e.grid 弹窗内的 Grid 对象
         */
        loadComplete: function (e) {
            console.log("Step" + (++i) + ": ----grid loadComplete----");
            var grid = e.grid;

            // 绑定查询按钮的点击事件
            $("#custom_query").click(function () {
                grid.dataSource.page(1);
            });
        }
    }
};

```

```

    });
  },

  clearInput: function (e) {
    console.log("Step" + (++i) + ": ----clearInput----");
  },

  /**
   * grid 中的数据回填时触发的事件
   * @param e 回填数据事件
   * e.sender 弹出窗口对象
   * e.grid 弹出的 grid 对象
   * e.model 选中的数据行 model
   */
  backFill: function (e) {
    console.log("Step" + (++i) + ": ----backFill----");
    e.preventDefault(); // 不使用默认的回填逻辑
  }
};

```

3.5.2.9 EEDM4014-EFTreeInput 勾选回填示例

3.5.2.9.1 页面功能简介

本示例页面用于说明，EFTreeInput 的普通回填和勾选（多选）回填功能。

3.5.2.9.2 配置和 API 详情

参考 [EFTreeInput](#) 和 [EFTree](#) 文档。

3.5.2.9.3 页面开发步骤

3.5.2.9.3.1 JSP 页面开发

EEDM4014.jsp

```

<EF:EFPage>
  <EF:EFRegion id="R1" title="EFTreeInput 勾选回填示例">
    <div class="row">
      <EF:EFTreeInput ename="inqu-0-selectedBackFillField" cname="单击树结点回填"
        serviceName="EEDM06" methodName="query"
        valueField="node_id" textField="node_text"
        hasChildren="node_type"
        root="{node_id: 'root', node_text: '地区分布', node_type:
        2, icon: 'folder'}"
        readonly="true" save="false"
      >
    </EF:EFTreeInput>

    <EF:EFTreeInput ename="inqu-0-checkedBackFillField" cname="勾选树结点回填"
      serviceName="EEDM06" methodName="query"
      valueField="node_id" textField="node_text"

```

```

hasChildren="node_type"
                                readonly="true" checkedBackFill="true"
                                >
                                </EF:EFTreeInput>
                                </div>
                                </EF:EFRegion>
                                </EF:EFPage>

```

3.5.2.9.3.2JS 开发

EEDM4014.js

```

$(function () {
    IPLATUI.EFTreeInput = {
        "inqu-0-checkedBackFillField": {
            /**
             * EFTreeInput 初始化事件
             *
             * @param e
             * e.container 弹出窗口的 jQuery 对象
             */
            init: function (e) {
                console.log("init", e.container);
                // e.preventDefault(); // 阻止弹窗
            },

            /**
             * 弹出树加载完成后事件
             *
             * @param e
             * e.tree Tree 对象
             */
            loadComplete: function (e) {
                console.log("loadComplete", e.tree);
            },

            /**
             * Tree 中数据回填触发事件
             *
             * @param e
             * e.tree Tree 对象
             * e.node 待回填的树结点数据对象（勾选回填时是集合）
             */
            backFill: function (e) {
                console.log("backFill", e);
                // e.preventDefault(); // 不使用默认的回填逻辑
            },

            /**
             * 清除图标点击触发事件（配置后会覆盖默认清除逻辑）
             *
             * @param e 存储 value 和 text 值的输入框 id，逗号隔开
             */
            // clearInput: function (e) {
            //     console.log("clearInput", e);

```

```

        // },

        ROOT: {node_id: 'root', node_text: '地区分布', node_type: 2, icon: 'folder'},
        check: function (e) {
            console.log("check", this.dataItem(e.node));
        }
    }
    });
});
});

```

注意:

EFTreeInput 的 treeld 的写法为: "popup_<%=cellId%>_textField_tree", <%=cellId%> 即 EFTreeInput 的 ename 值。

3.5.2.9.3.3 Service 服务开发

ServiceEEDM06.java

```

package com.baosight.iplat4j.ee.dm.service;

import com.baosight.iplat4j.core.ei.EiConstant;
import com.baosight.iplat4j.core.ei.EiInfo;
import com.baosight.iplat4j.core.service.impl.ServiceEPBase;
import com.baosight.iplat4j.ee.dm.domain.EEDM06;

public class ServiceEEDM06 extends ServiceEPBase {
    @Override
    public EiInfo initLoad(EiInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM06().eiMetadata);
        return inInfo;
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo;

        if (inInfo.getInt(EiConstant.queryBlock + "-0-expandLevel") > 0) {
            outInfo = super.query(inInfo, "EEDM06.queryByLevel", new EEDM06());
        } else {
            outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        }

        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }
}

```

```

/**
 * 一次性查询 N 层的树结点
 *
 * @param inInfo 包含层级参数
 */
public EiInfo queryByLevel(EiInfo inInfo) {
    inInfo.set("result-limit", 1000);
    EiInfo outInfo = super.query(inInfo, "EEDM06.queryByLevel", new EEDM06());

    String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
    outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
    outInfo.getBlocks().remove(EiConstant.resultBlock);

    return outInfo;
}

public EiInfo search(EiInfo inInfo) {
    inInfo.set(EiConstant.resultBlock + EiConstant.separator + EiConstant.limitStr,
1000);
    EiInfo outInfo = super.query(inInfo, "EEDM06.search", new EEDM06());
    String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
    outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
    outInfo.getBlocks().remove(EiConstant.resultBlock);
    return outInfo;
}

@Override
public EiInfo delete(EiInfo inInfo) {
    return super.delete(inInfo, "EEDM06.delete");
}

@Override
public EiInfo update(EiInfo inInfo) {
    return super.update(inInfo, "EEDM06.update");
}

@Override
public EiInfo insert(EiInfo inInfo) {
    return insert(inInfo, "EEDM06.insert");
}
}

```

EEDM06.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE sqlMap PUBLIC "-//ibatis.com//DTD SQL Map 2.0//EN"
"http://www.ibatis.com/dtd/sql-map-2.dtd">

<sqlMap namespace="EEDM06">

    <select id="count" resultClass="int">
        SELECT count(*) FROM ${platSchema}.teedm06 where l=1
        <isNotEmpty prepend=" AND " property="node">
            p_ename = #node#
        </isNotEmpty>
    </select>

```

```

        </isNotEmpty>
    </select>

    <select id="query" resultClass="java.util.HashMap">
        SELECT
            ename      as "node_id",
            cname      as "node_text",
            p_ename    as "parent_id",
            node_type  as "node_type",
            sort       as "sort",
            icon       as "icon",
            node_level as "node_level",
            node_path  as "node_path",
            node_url   as "node_url"
        FROM ${platSchema}.teedm06 where l=1
            <isNotEmpty prepend=" AND " property="node">
                p_ename = #node#
            </isNotEmpty>
        order by ename, sort asc
    </select>

    <select id="queryByLevel" resultClass="java.util.HashMap">
        SELECT
            ename      as "node_id",
            cname      as "node_text",
            p_ename    as "parent_id",
            node_type  as "node_type",
            sort       as "sort",
            icon       as "icon",
            node_level as "node_level",
            node_path  as "node_path",
            node_url   as "node_url"
        FROM ${platSchema}.teedm06 where l=1
            <isNotEmpty prepend=" AND " property="expandLevel">
                node_level <= #expandLevel#
            </isNotEmpty>
        order by ename, sort asc
    </select>

    <select id="search" resultClass="java.util.HashMap">
        SELECT
            ename      as "node_id",
            cname      as "node_text",
            p_ename    as "parent_id",
            node_type  as "node_type",
            sort       as "sort",
            icon       as "icon",
            node_level as "node_level",
            node_path  as "node_path",
            node_url   as "node_url"
        FROM ${platSchema}.teedm06 where l=1
            <isNotEmpty prepend=" AND " property="node_level">
                node_level <= #node_level#
            </isNotEmpty>
        order by ename, sort asc
    </select>

```

```

<insert id="insert">
  INSERT INTO ${platSchema}.teedm06 (
    ename,
    cname,
    p_ename
    <isNotEmpty prepend="," property="node_type">node_type</isNotEmpty>
    <isNotEmpty prepend="," property="icon">icon</isNotEmpty>
    <isNotEmpty prepend="," property="sort">sort</isNotEmpty>
    <isNotEmpty prepend="," property="node_level">node_level</isNotEmpty>
    <isNotEmpty prepend="," property="node_path">node_path</isNotEmpty>
    <isNotEmpty prepend="," property="node_url">node_url</isNotEmpty>
    <![CDATA[
      ) VALUES (
    ]]>
    #node_id#,
    #node_text#,
    #parent_id#
    <isNotEmpty prepend="," property="node_type">#node_type#</isNotEmpty>
    <isNotEmpty prepend="," property="icon">#icon#</isNotEmpty>
    <isNotEmpty prepend="," property="sort">#sort#</isNotEmpty>
    <isNotEmpty prepend="," property="node_level">#node_level#</isNotEmpty>
    <isNotEmpty prepend="," property="node_path">#node_path#</isNotEmpty>
    <isNotEmpty prepend="," property="node_url">#node_url#</isNotEmpty>
    <![CDATA[
      )
    ]]>
  </insert>

<update id="update">
  UPDATE ${platSchema}.teedm06
  <dynamic prepend="set">
    <isNotNull prepend="," property="node_id">
      ename = #node_id#
    </isNotNull>
    <isNotNull prepend="," property="node_text">
      cname = #node_text#
    </isNotNull>
    <isNotNull prepend="," property="parent_id">
      p_ename = #parent_id#
    </isNotNull>
    <isNotNull prepend="," property="node_type">
      node_type = #node_type#
    </isNotNull>
    <isNotNull prepend="," property="sort">
      sort = #sort#
    </isNotNull>
    <isNotNull prepend="," property="icon">
      icon = #icon#
    </isNotNull>
    <isNotNull prepend="," property="node_level">
      node_level = #node_level#
    </isNotNull>
    <isNotNull prepend="," property="node_path">
      node_path = #node_path#
    </isNotNull>
    <isNotNull prepend="," property="node_url">

```



```

        node_url = #node_url#
    </isNotNull>
</dynamic>
    WHERE ename = #node_id#
</update>

<delete id="delete">
    DELETE FROM ${platSchema}.teedm06
    WHERE ename = #node_id#
</delete>

<delete id="deleteBatch">
    DELETE FROM ${platSchema}.teedm06 WHERE ename in
    <iterate property="labelList" open="(" close=")" conjunction=" , ">
        #labelList[]#
    </iterate>
</delete>

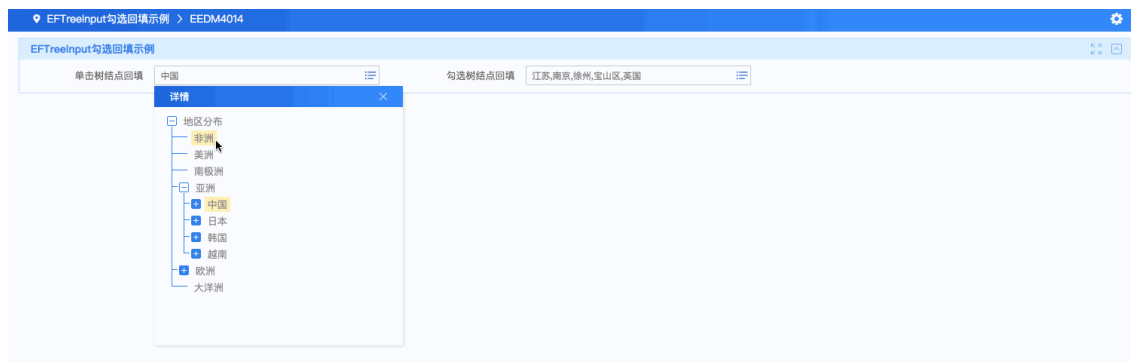
</sqlMap>

```

3.5.2.9.3.4 页面展示效果

3.5.2.10 EEDM4015-EFTreeInput API 示例

3.5.2.10.1 页面功能简介



本示例页面用于说明，EFTreeInput 的 API: originalInput, readonly 以及 disabled。

另：

EFTreeInput 的勾选（多选）回填示例详见[EEDM4014-EFTreeInput 勾选回填示例](#)

3.5.2.10.2 配置和 API 详情

参考 [EFTreeInput](#) 和 [EFTree](#) 文档。

3.5.2.10.3 页面开发步骤

3.5.2.10.3.1 JSP 页面开发

EEDM4015.jsp

```

<EF:EFPage>
  <EF:EFRegion id="R1" title="EFTreeInput 使用说明">
    <div class="row">
      <h5 class="push-30-1 text-success"><code>EFTreeInput</code> 组件，用于处理
      个性化要求较高的表单输入
      通过弹出窗口的方式，查询，选择窗口中的数据，回填到相应的输入框内。它本
      质上是个下拉框输入，输入框中只是显示值(textField)
      通常情况下不能直接编辑 <code>EFTreeInput</code> 输入框。<br>
      真实值(valueField)输入框的 id 是: <code>EFTreeInput</code> 的 ename 属性
      值 <br>
      显示值(textField)输入框的 id 是: "show_" + <code>EFTreeInput</code> 的
      ename 属性值 (ename 前拼接 "show_")
    </h5>
    <EF:EFTreeInput ename="inqu-0-param1" cname="隐藏的真实值"
      serviceName="EEDM06" methodName="query"
      valueField="node_id" textField="node_text"
      hasChildren="node_type"
      root="{node_id: 'root', node_text: '地区分布', node_type:
      2, icon: 'folder'}"
      readonly="true" required="true" save="false" popupTitle="只
      读状态"
    >
    </EF:EFTreeInput>
    </div>

    <hr>

    <div class="row">
      <h5 class="push-30-1 text-success">业务需要直接输入时，应该配置属性
      <code>originalInput="true"</code>，
      则 <code>EFPopupInput</code> 只有 valueField，显示值(textField)和真实
      值(valueField)是一致的</h5>
      <EF:EFTreeInput ename="inqu-0-param2" cname="显示的真实值"
        serviceName="EEDM06" methodName="query"
        originalInput="true"
        valueField="node_id" textField="node_text"
        hasChildren="node_type"
        root="{node_id: 'root', node_text: '地区分布', node_type:
        2, icon: 'folder'}"
        readonly="true" required="true" save="false" popupTitle="只
        读状态"
      >
      </EF:EFTreeInput>
    </div>
  </EF:EFRegion>

  <EF:EFRegion id="R2" title="API 示例">
    <div class="row">
      <h5 class="push-30-1 text-success">只读 <code>readonly</code>状态的
      <code>EFTreeInput</code>，
      只能弹窗回填，不能直接输入</h5>
      <EF:EFTreeInput ename="inqu-0-param3" cname="readonly 状态"

```

```

        serviceName="EEDM06" methodName="query"
        valueField="node_id" textField="node_text"
hasChildren="node_type"
        root="{node_id: 'root', node_text: '地区分布', node_type:
2, icon: 'folder'}"
        readonly="true" save="false"
    >
</EF:EFTreeInput>

<div class="col-md-4 i-align-right">
    <EF:EFTreeInput ename="setReadonly" cname="设置只读"/>
    <EF:EFTreeInput ename="unsetReadonly" cname="设置可写"/>
</div>
</div>

<hr>

<div class="row">
    <h5 class="push-30-1 text-success">禁用 <code>disabled</code>状态的
<code>EFTreeInput</code>,
        不能弹窗回填，不能直接输入</h5>
    <EF:EFTreeInput ename="inqu-0-param4" cname="disabled 状态"
        serviceName="EEDM06" methodName="query"
        valueField="node_id" textField="node_text"
hasChildren="node_type"
        root="{node_id: 'root', node_text: '地区分布', node_type:
2, icon: 'folder'}"
        disabled="true" save="false"
    >
</EF:EFTreeInput>

    <div class="col-md-4 i-align-right">
        <EF:EFTreeInput ename="unsetEnable" cname="设置禁用"/>
        <EF:EFTreeInput ename="setEnable" cname="设置可用"/>
    </div>
</div>
</EF:EFRegion>
</EF:EFPage>

```

3.5.2.10.3.2 JS 开发

EEDM4015.js

```

$(function () {
    var validator = IPLAT.Validator({
        id: "r1"
    });
    window.validator = validator;

    // 设置只读
    $("#setReadonly").on("click", function () {
        IPLAT.EFTreeInput.readonly("#inqu-0-param3", true);
    });

    // 设置非只读

```

```

$("#unsetReadonly").on("click", function () {
    IPLAT.EFTreeInput.readonly("#inqu-0-param3", false);
});

// 设置禁用
$("#unsetEnable").on("click", function () {
    IPLAT.EFTreeInput.enable("#inqu-0-param4", false);
});

// 设置可用
$("#setEnable").on("click", function () {
    IPLAT.EFTreeInput.enable("#inqu-0-param4", true);
});
});

```

3.5.2.10.3.3 Service 服务开发

ServiceEEDM06.java

```

package com.baosight.iplat4j.ee.dm.service;

import com.baosight.iplat4j.core.ei.EiConstant;
import com.baosight.iplat4j.core.ei.EiInfo;
import com.baosight.iplat4j.core.service.impl.ServiceEPBase;
import com.baosight.iplat4j.ee.dm.domain.EEDM06;

public class ServiceEEDM06 extends ServiceEPBase {
    @Override
    public EiInfo initLoad(EiInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM06().eiMetadata);
        return inInfo;
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo;

        if (inInfo.getInt(EiConstant.queryBlock + "-0-expandLevel") > 0) {
            outInfo = super.query(inInfo, "EEDM06.queryByLevel", new EEDM06());
        } else {
            outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        }

        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    /**
     * 一次性查询 N 层的树结点

```

```

    *
    * @param inInfo 包含层级参数
    */
    public EiInfo queryByLevel(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.queryByLevel", new EEDM06());

        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);

        return outInfo;
    }

    public EiInfo search(EiInfo inInfo) {
        inInfo.set(EiConstant.resultBlock + EiConstant.separator + EiConstant.limitStr,
1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.search", new EEDM06());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    @Override
    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.delete");
    }

    @Override
    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM06.update");
    }

    @Override
    public EiInfo insert(EiInfo inInfo) {
        return insert(inInfo, "EEDM06.insert");
    }
}

```

EEDM06.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE sqlMap PUBLIC "-//ibatis.com//DTD SQL Map 2.0//EN"
"http://www.ibatis.com/dtd/sql-map-2.dtd">

<sqlMap namespace="EEDM06">

    <select id="count" resultClass="int">
        SELECT count(*) FROM ${platSchema}.teedm06 where l=1
        <isEmpty prepend=" AND " property="node">
            p_ename = #node#
        </isEmpty>
    </select>

```

```

<select id="query" resultClass="java.util.HashMap">
  SELECT
    ename      as "node_id",
    cname      as "node_text",
    p_ename    as "parent_id",
    node_type  as "node_type",
    sort       as "sort",
    icon       as "icon",
    node_level as "node_level",
    node_path  as "node_path",
    node_url   as "node_url"
  FROM ${platSchema}.teedm06 where l=1
    <isNotEmpty prepend=" AND " property="node">
      p_ename = #node#
    </isNotEmpty>
  order by ename, sort asc
</select>

<select id="queryByLevel" resultClass="java.util.HashMap">
  SELECT
    ename      as "node_id",
    cname      as "node_text",
    p_ename    as "parent_id",
    node_type  as "node_type",
    sort       as "sort",
    icon       as "icon",
    node_level as "node_level",
    node_path  as "node_path",
    node_url   as "node_url"
  FROM ${platSchema}.teedm06 where l=1
    <isNotEmpty prepend=" AND " property="expandLevel">
      node_level <= #expandLevel#
    </isNotEmpty>
  order by ename, sort asc
</select>

<select id="search" resultClass="java.util.HashMap">
  SELECT
    ename      as "node_id",
    cname      as "node_text",
    p_ename    as "parent_id",
    node_type  as "node_type",
    sort       as "sort",
    icon       as "icon",
    node_level as "node_level",
    node_path  as "node_path",
    node_url   as "node_url"
  FROM ${platSchema}.teedm06 where l=1
    <isNotEmpty prepend=" AND " property="node_level">
      node_level <= #node_level#
    </isNotEmpty>
  order by ename, sort asc
</select>

<insert id="insert">
  INSERT INTO ${platSchema}.teedm06 (

```

```

ename,
cname,
p_ename
<isEmpty prepend=", " property="node_type">node_type</isEmpty>
<isEmpty prepend=", " property="icon">icon</isEmpty>
<isEmpty prepend=", " property="sort">sort</isEmpty>
<isEmpty prepend=", " property="node_level">node_level</isEmpty>
<isEmpty prepend=", " property="node_path">node_path</isEmpty>
<isEmpty prepend=", " property="node_url">node_url</isEmpty>
<![CDATA[
    ) VALUES (
]]>
#node_id#,
#node_text#,
#parent_id#
<isEmpty prepend=", " property="node_type">#node_type#</isEmpty>
<isEmpty prepend=", " property="icon">#icon#</isEmpty>
<isEmpty prepend=", " property="sort">#sort#</isEmpty>
<isEmpty prepend=", " property="node_level">#node_level#</isEmpty>
<isEmpty prepend=", " property="node_path">#node_path#</isEmpty>
<isEmpty prepend=", " property="node_url">#node_url#</isEmpty>
<![CDATA[
    )
]]>

</insert>

<update id="update">
UPDATE ${platSchema}.teedm06
<dynamic prepend="set">
    <isNotNull prepend=", " property="node_id">
        ename = #node_id#
    </isNotNull>
    <isNotNull prepend=", " property="node_text">
        cname = #node_text#
    </isNotNull>
    <isNotNull prepend=", " property="parent_id">
        p_ename = #parent_id#
    </isNotNull>
    <isNotNull prepend=", " property="node_type">
        node_type = #node_type#
    </isNotNull>
    <isNotNull prepend=", " property="sort">
        sort = #sort#
    </isNotNull>
    <isNotNull prepend=", " property="icon">
        icon = #icon#
    </isNotNull>
    <isNotNull prepend=", " property="node_level">
        node_level = #node_level#
    </isNotNull>
    <isNotNull prepend=", " property="node_path">
        node_path = #node_path#
    </isNotNull>
    <isNotNull prepend=", " property="node_url">
        node_url = #node_url#
    </isNotNull>

```

```

</dynamic>
WHERE ename = #node_id#
</update>

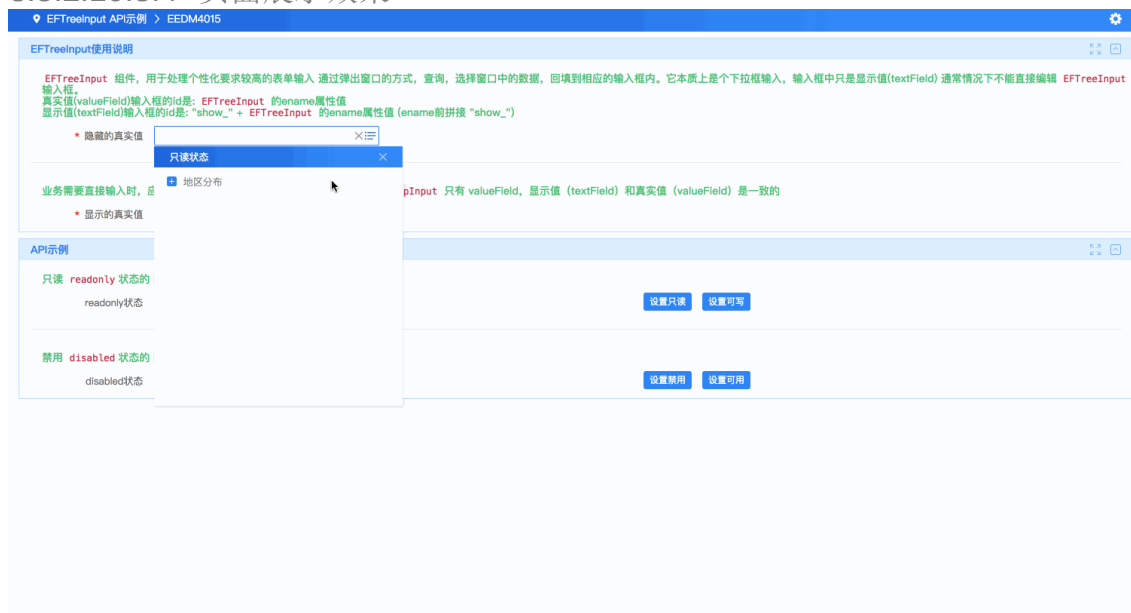
<delete id="delete">
DELETE FROM ${platSchema}.teedm06
WHERE ename = #node_id#
</delete>

<delete id="deleteBatch">
DELETE FROM ${platSchema}.teedm06 WHERE ename in
<iterate property="labelList" open="(" close=")" conjunction=" , ">
#labelList[]#
</iterate>
</delete>

</sqlMap>

```

3.5.2.10.3.4 页面展示效果



3.5.2.11 EEDM5000-日期控件示例

3.5.2.11.1 页面功能简介

本示例页面用于介绍 **EFDDatePicker** 与 **EFDDateSpan** 标签，并说明如何利用这两个标签来渲染所需的日期控件。同时，介绍相关 **API**，以供调用。

- **EFDDatePicker** 控件：可满足单个日期（时间）的选择。
- **EFDDateSpan** 控件：可满足开始日期（时间）/结束日期（时间）段的选择。
- **EFDDateSpan** 控件：可满足实现开始/结束合并成一个单元格。

3.5.2.11.2 配置和 API 详情

相关文档：参考 [EFDatePicker](#) 与 [EFDateSpan](#) 中的描述。

用法详情：

获取时间的DatePicker实例对象

```
var dp = $(selector).data("kendoDatePicker");
```

- 获取日期

var value = dp.value(); // 获取日期，没有选择日期时，value 是 null

kendo.toString(value, "d");



说明：对日期对象进行格式化。"G" 获取完整的格式，"d" 只获取年月日

举例：kendo.toString(value, "G"); // 2018/1/17 13:12:22

kendo.toString(value, "d"); // 2018/1/17

- 设置一个合法的日期

dp.value("value");

说明：此处的 value 为设置的日期值；如果设置了格式不正确的日期，日期控件的 value 会被重置成 null

举例：dp.value("2018/07/23");

- 启用日期控件

dp.enable();

- 禁用日期控件

dp.enable(false);

- 设置日期控件为只读

dp.readonly();

说明：注意 readonly 和 disabled 的区别：disabled 的字段不出现在 form 提交的参数中

- 打开日期控件

dp.open();

- 关闭日期控件

dp.close();

3.5.2.11.3 页面开发步骤

3.5.2.11.3.1 EFDatePicker

EFDatePicker 标签一般用于单个日期的输入，其中 colWidth 设置时间输入框的宽度(最大值 12)，cname 为输入框前展示值，ename 用于确定 dateId。默认日期格式 format="yyyy/MM/dd"，role="datetime" 表示使用日期时间控件；

JSP 页面开发

EFDatePicker

```
<div class="row">
  <EF:EFDatePicker ename="inqu_status-0-start" cname="单个的日期"
    format="yyyy/MM/dd" required="true">
  </EF:EFDatePicker>
  <EF:EFDatePicker ename="inqu_status-0-end" cname="日期时刻" role="datetime" //可设置
  role="datetime"显示时刻
    format="yyyy-MM-dd HH:mm:ss" required="true">
  </EF:EFDatePicker>
</div>
```

页面效果如下所示：

EFDate 时间控件，可设置role="datetime"显示时刻

* 单个的日期 2017/11/14

* 日期时刻 2017-11-13 01:30:00

注：

1. 点击右侧日历图标或输入框均可弹出日期，默认显示深度到月，输入框必填。
2. 日期底部三个功能按钮，分别具有选择当天日期、清空输入框中内容和关闭日期的功能。
3. 动态属性有：interval，详情可看文档：[EFDatePicker 与 EFDateSpan 中动态属性](#)。此处设置 interval="15"，即时间的间隔为 15 分钟。
4. 当设置 role="datetime" 时，表示使用日期时间控件；当设置 role="date" 时，仅显示日期，而不会显示具体时间

3.5.2.11.3.2 EFDateSpan

3.5.2.11.3.2.1 EFDateSpan 标签用于开始/结束日期时间段的输入

标签属性有：

- startName: 开始日期时间字段英文名；
- endName: 结束日期时间字段英文名；
- startCname: 开始日期时间字段中文名；
- endCname: 结束日期时间字段中文名
- startRatio: 开始日期的标签和 Input 的栅格比例，默认为 4:8；endRatio: 结束日期的标签和 Input 的栅格比例，默认 4:8

- **startValue**: 开始时间的默认值
- **startTimeFormat / endTimeFormat**: 选择完日期后，时刻的默认值。如：若设置了 **startTimeFormat="23:59:59"**，那么在选择完开始日期后，开始日期的时刻默认显示为 23:59:59

注：如果设置了 **startValue**，则 **startTimeFormat** 会失效；若不定义 **startCname** 和 **endCname**，则默认的 **label** 值为"开始时间"和"结束时间"。

JSP 页面开发

EFDateSpan

```
<div class="row">
  <EF:EFDateSpan startName="ss" endName="ee" startCname="开始日期" endCname="结束日期"
  "
    role="datetime" required="true" interval="15" startRatio="4:8"
    startValue="2017-09-20 12:30" parseFormats="['yyyy-MM-ddHH:mm']"/>
</div>
```

页面效果如下图所示：

说明：**startValue** 为开始时间默认值：2017-09-20 12:30



注：

1. **startName** 和 **endName** 取值应避免重复。
2. 点击右侧图标可选择日期和具体时间。
3. 开始时间任选，结束时间只能从开始时间之后选择。

EFDateSpan 时间区间控件，占两个字段输入长度，开始时间或结束时间选定后，会限制另一个控件的选择范围

请输入数字 请输入数字

*开始日期 *结束日期

EFDateSpan 时间区间控件，合并成一个字段输入长度，通常只适合显示日期

开始日期-结束日期 起止日期

EFDateSpan 时间区间控件，需要显示时刻，并且需要合并到一个输入字段输入长度，通常需

请输入数字 请输入数字

开始时间-结束时间 起止时间

十一月 2017

一 二 三 四 五 六 日

13 14 15 16 17 18 19

20 21 22 23 24 25 26

27 28 29 30 1 2 3

4 5 6 7 8 9 10

清空 关闭 今天

3.5.2.11.3.2.2 EFDateSpan 将开始/结束合并成一个单元格

设置 `bindWidth`, `bindName`, `bindRatio` 属性来实现此功能。

- `bindWidth` 属性：用来控制是否将单元格合并
- `bindName` 属性：用于显示合并后的 `EFDateSpan` 的标签名，如“开始时间-结束时间”
- `bindRatio` 属性：例如 4:4:4、3:4:5。（总数为 12）⚠ 注意：比例格式错误或者不写将采用默认比例 4:4:4。（错误格式如 6:6；2:3:4:3，或其他非比例格式）

除此以外，新增 `extChar`，可以在控件中增加字符串。

JSP 页面开发

EFDateSpan 合并开始和结束日期

```
<div class="row">
  <EF:EFDateSpan startName="s1" endName="e1" startCname="开始时间" endCname="结束时间"
    role="datetime"
    required="true" interval="15" bindWidth="6" />
  <EF:EFDateSpan startName="s2" endName="e2" startCname="开始时间" endCname="结束时间"
    role="datetime" required="true" interval="15" bindWidth="6"
    bindName="起止时间" extChar="到"/>
</div>
```



时间区间控件，需要显示时刻，并且需要合并到一个输入字段输入长度，通常需要 6 份栅格，完整示例请看文档：[EEDM5004-日期控件布局示例](#)

页面效果如下所示：

EFDateSpan 时间区间控件，需要显示时刻，并且需要合并到一个输入字段输入长度，通常需要6份栅格

请输入数字 请输入数字

开始时间-结束时间 起止时间

其中，“到”字即为日期控件中间的字符串。

3.5.2.12 EEDM5001-日期控件格式化示例

3.5.2.12.1 页面功能简介

本示例页面用于介绍：

1. 日期控件常用属性：**format** 和 **parseFormats**
2. 如何设置日期默认值，并加以解析
3. 如何设置最小日期或者最大日期。

3.5.2.12.2 配置和 API 详情

相关文档：参考 [EFDatePicker](#) 与 [EFDateSpan](#) 中的描述。

3.5.2.12.3 页面开发步骤

3.5.2.12.3.1 日期的格式

Format: EFDatePicker 日期控件的显示格式由 **format** 属性确定；

parseFormats: 日期控件额外接受的格式。使用 **parseFormats** 属性，可设置输入框和 **value** API 接受的日期字符串的格式；

JSP 页面开发

format 及 parseFormats 属性

```
<EF:EFDatePicker ename="d1" cname="日期(yyyyMMdd)" format="yyyyMMdd" //日期格式为
"yyyyMMdd"
    parseFormats="['yyyy-MM-dd', 'yyyy/MM/dd']"/>

<EF:EFDatePicker ename="d2" cname="通常的日期" role="datetime" format="yyyyMMddHHmmss"
    parseFormats="['yyyy-MM-dd HH:mm:ss', 'yyyy/MM/dd HH:mm:ss']"/>
```

页面效果如下所示：



第一个设置了 **format="yyyyMMdd"** 第二个设置了 **format="yyyyMMddHHmmss"**。

3.5.2.12.3.2 日期默认值

JSP 页面开发

后台设置默认值

```
<div class="row">
    <EF:EFDatePicker ename="startDate" cname="日期(2017/01/15)"/>
    <EF:EFDatePicker ename="endDate" cname="日期(20171025)"/>
</div>
```

此处日期控件的默认值来自后台服务 `initLoad` 中的 `EiInfo`。

Service 开发

后台设置默认值

```
...
public class ServiceEEDM5000 extends ServiceEPBase {

    @Override
    public EiInfo initLoad(EiInfo inInfo) {
        // 设置默认值
        inInfo.set("startDate", "2017/01/15");
        inInfo.set("endDate", "20171025");

        return inInfo;
    }
}
```

页面效果如下所示：

EFDatePicker 日期控件的默认值来自后台服务initLoad中的EiInfo

日期(2017/01/15) 2017-01-15 日期(20171025) 2017-10-25

EFDatePicker 会尝试解析各种常见格式的日期字符串，此处将"2017/01/15"解析成"2017-01-15"，将"20171025"解析成"2017-10-25"。

除此以外，还可以利用 `value` 属性，来设置日期控件的默认值：

JSP 页面开发

value 属性

```
<div class="row">
  <EF:EFDatePicker ename="d3" cname="格式化的日期" value="20170101"/>
  <EF:EFDatePicker ename="d4" cname="通常的日期" role="datetime" value="2015/01/02
12:23:21"
    format="yyyy-MM-dd HH:mm:ss"/>
</div>
```

页面效果如下所示：

默认值

EFDatePicker 日期控件的默认值来自 value 属性

格式化的日期 2017-01-01 通常的日期 2015-01-02 12:23:21

3.5.2.12.3.3 设置最小日期和最大日期

通过 `js`，可以指定日历可显示的最小日期，该日期之前不能选择，反之，设置最大日期亦然。

JSP 页面开发

JSP 开发

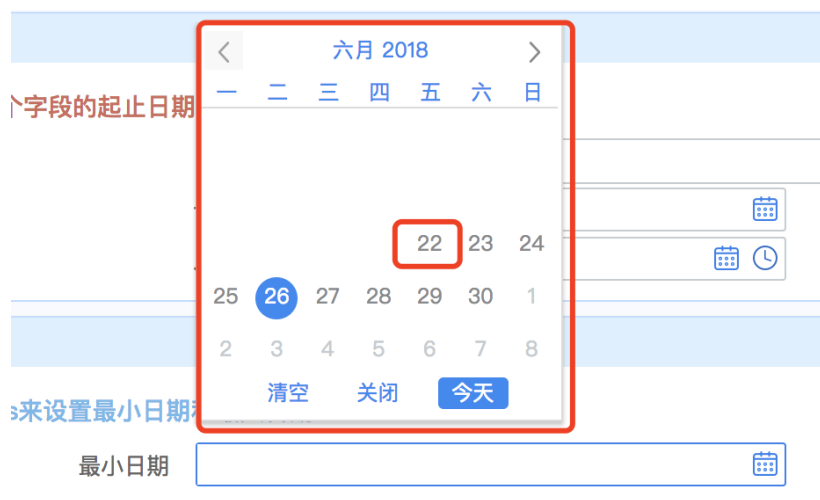
```
<div class="row">
  <EF:EFDatePicker ename="inqu_status-0-m1" cname="最小日期"/>
  <EF:EFDatePicker ename="inqu_status-0-m2" cname="最大日期"/>
</div>
```

JS 开发

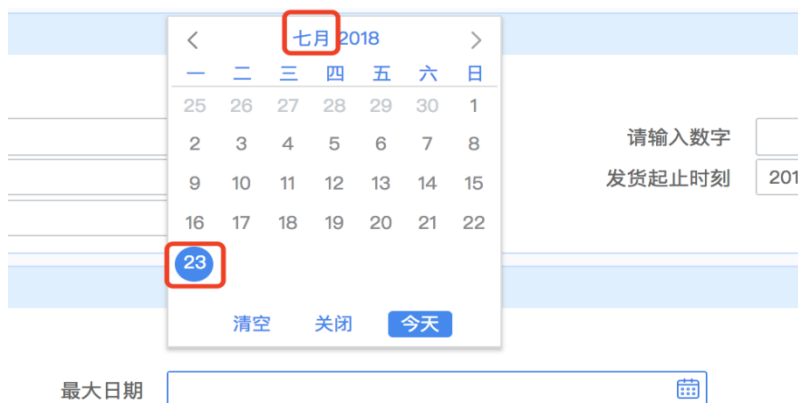
设置最大最小日期

```
IPLATUI.EFDatePicker = {
  "inqu_status-0-m1": {
    min: new Date("2018-06-22") // 设置最小日期
  },
  "inqu_status-0-m2": {
    max: new Date("2018-07-23") // 设置最大日期
  }
};
```

页面效果如下所示：



最小日期。



最大日期。

3.5.2.13 EEDM5004-日期控件布局示例

3.5.2.13.1 页面功能简介

本示例页面用于说明：如何进行布局：了解 `EFDateSpan` [的属性及其作用](#)。

3.5.2.13.2 配置和 API 详情

参考 [EFDateSpan](#)，[EEDM5000-日期控件示例](#) 中的描述。

3.5.2.13.3 页面开发步骤

首先，要先了解 `bindWidth`，`bindRatio`，`bindName` 属性。

- `bindWidth`：使用 `bindWidth`，可以合并单元格，开始时间和结束时间将在一个单元格上显示。例如：当 `bindWidth="4"` 时表明日期控件占用 4 个栅格。
- `bindRatio`：`EFDateSpan` 里的标签、开始时间、结束时间的栅格比例。例如 4:4:4、3:4:5（总数为 12），比例格式错误或者不写将采用默认比例 4:4:4。
- `bindName`：标签名字。例如：开始时间-结束时间
- `extChar`：日期控件中间的字符串，[可在 js 中进行全局配置](#)，特殊情况在组件中使用该属性配置。

了解了以上属性后，便可实现不同的日期控件布局样式，在此，栅格数推荐可以是 6，5，4。

3.5.2.13.3.1 `bindWidth="6"`

显示日期+时刻：6 份栅格；栅格比例 4:4:4；标签字数：不超过 15 个汉字

JSP 页面开发

JSP 示例

```
<div class="row">
  <EF:EFDateSpan role="datetime" required="true" startName="s6" endName="e6"
    interval="15"
      bindWidth="6" startCname="开始时间" endCname="结束时间"
      bindName="开始时间-结束时间"/>
  <EF:EFDateSpan role="date" required="true" startName="s5" endName="e5" interval="15"
    bindWidth="5" startCname="开始时间 1" endCname="结束时间 2"
    bindName="起止日期" extStyle="true" extChar="至"/>
</div>
```

页面效果如下所示：

显示日期+时刻：6 份栅格；栅格比例 2:5:5；标签字数：不超过 7 个汉字。

页面效果如下所示：

通常只适合显示日期，不显示时刻：6 份栅格；栅格比例 6:3:3；标签字数：不超过 24 个汉字。

页面效果如下所示：

The form layout for bindWidth="5" consists of two rows. The first row contains two date pickers labeled "请选择开始日期和结束日期" and "请选择开始日期和结束日期" with a "至" separator. The second row contains two text input fields labeled "请输入证件号（用户ID，身份证，护照，驾照皆可）" and "请输入公司地址（省市区具体位置）".

3.5.2.13.3.2 bindWidth="5"

显示日期+时刻：5 份栅格；栅格比例 4:4:4；标签字数：不超过 12 个汉字。

页面效果如下所示：

The form layout for bindWidth="5" consists of two rows. The first row contains two date pickers labeled "开始时间-结束时间" and "开始时间-结束时间" with a "到" separator. The second row contains two text input fields labeled "公司名称" and "公司代号".

显示日期+时刻：5 份栅格；栅格比例 2:5:5；标签字数：不超过 5 个汉字（推荐：开始-结束）。

页面效果如下所示：

The form layout for bindWidth="5" consists of two rows. The first row contains two date pickers labeled "开始-结束" and "开始-结束" with a "到" separator. The second row contains two text input fields labeled "输入价格" and "输入价格".

3.5.2.13.3.3 bindWidth="4"

显示日期+时刻：4 份栅格；栅格比例 2:5:5；标签字数：不超过 4 个汉字（推荐：开始-结束）。

页面效果如下所示：

The form layout for bindWidth="4" consists of two rows. The first row contains two date pickers labeled "开始-结束" and "开始-结束" with a "至" separator. The second row contains two text input fields labeled "公司地址" and "订单号".

通常只适合显示日期，不显示时刻：4 份栅格；栅格比例 4:4:4；标签字数：不超过 10 个汉字。

页面效果如下所示：

The form layout for bindWidth="4" consists of two rows. The first row contains two date pickers labeled "开始日期-结束日期" and "起止日期" with a "至" separator. The second row contains two text input fields labeled "公司注册日期" and "公司注册日期".

 **提醒：**考虑到小屏电脑，4 份的 bindWidth 适合显示日期，不显示时刻。

3.5.2.14 EEDM5010-级联下拉示例

3.5.2.14.1 页面功能简介

本示例页面用于说明，如何使用 EFCascadeSelect 标签实现三级级联下拉。

3.5.2.14.2 配置和 API 详情

参考 [EFCascadeSelect](#)。

3.5.2.14.3 页面开发步骤

3.5.2.14.3.1 JSP 页面开发

EFCascadeSelect 示例

```

<EF:EFRegion id="inqu" title="查询条件">
    <div class="row">
        <%--支持添加静态 EFOption; valueTemplate 定制被选中选项的显示模板, 根据查询结果字段名灵活配置; defaultValue 设置初始选中值--%>
        <EF:EFCascadeSelect ename="inqu_status-0-continent_code"
            valueTemplate="#=continent_code#-#=continent_name#"
            defaultValue="Europe"
            resultId="result" cname="大洲" textField="continent_name"
            valueField="continent_code">
            <EF:EFOption label="大洲 1" value="c1"/>
            <EF:EFOption label="大洲 2" value="c2"/>
        </EF:EFCascadeSelect>
        <%--template 定制下拉项的显示模板; methodName 设置下拉项查询的方法名, 默认 initLoad; serviceName 默认初始处理服务;
            resultId 设置下拉选项在 EiInfo 中的数据块名; cascadeFrom 设置父下拉框的 id--%>
        <EF:EFCascadeSelect ename="inqu_status-0-country_code"
            template="#=country_code#-#=country_name#"
            methodName="query" cname="国家" resultId="result"
            cascadeFrom="inqu_status-0-continent_code"
            textField="country_name" valueField="country_code">
        </EF:EFCascadeSelect>
        <EF:EFCascadeSelect ename="inqu_status-0-province_code"
            template="#=province_code#-#=province_name#"
            methodName="queryProvince" cname="省份" resultId="result"
            cascadeFrom="inqu_status-0-country_code"
            textField="province_name" valueField="province_code">
        </EF:EFCascadeSelect>
    </div>
</EF:EFRegion>

```

注：ename 推荐取值"inqu_status-0-"+valueField 值，sql 文件中查询条件 property 值也需要取 valueField 值，如下 EEDM03.xml。

EEDM03.xml

```

<sqlMap namespace="EEDM03">
    ...
    <select id="query" resultClass="java.util.HashMap">
        SELECT
            country_code      as "country_code",
            country_name      as "country_name",
            continent_code    as "continent_code"
        FROM ${platSchema}.teedm03
        <dynamic prepend="WHERE">
            <isNotEmpty prepend=" AND " property="continent_code">
                continent_code = #continent_code#
            </isNotEmpty>
        </dynamic>
        order by country_code asc
    </select>

```

```
</select>
...
</sqlMap>
```

3.5.2.14.3.2 Service 服务开发

ServiceEEDM5010

```
public class ServiceEEDM5010 extends ServiceEPBase {
    public EiInfo initLoad(EiInfo inInfo) {
        return super.query(inInfo, "EEDM02.query", new EEDM02());
    }

    public EiInfo query(EiInfo inInfo) {
        return super.query(inInfo, "EEDM03.query", new EEDM03());
    }

    public EiInfo queryProvince(EiInfo inInfo) {
        return super.query(inInfo, "EEDM07.query", new EEDM07());
    }
}
```

3.5.2.14.3.3 页面展示效果

选择大洲后，级联的国家：

The screenshot shows a web application interface with a blue header bar containing the text "级联下拉EFCascadeSelect使用示例 > EEDM5010". Below the header, there is a search condition bar labeled "查询条件". It contains three dropdown menus: "大洲" (Continent) with "Europe-欧洲" selected, "国家" (Country) with "请选择" (Please select) selected and a dropdown menu open showing "France-法国" and "UK-英国", and "省份" (Province) which is currently empty. The interface is clean and modern, with a light blue background.

选择国家后，级联的省份：

The screenshot shows the same web application interface as the previous one, but now the "国家" (Country) dropdown menu is set to "英国" (UK). Consequently, the "省份" (Province) dropdown menu now shows "伦敦" (London) as the selected option. The "大洲" (Continent) dropdown menu remains set to "Europe-欧洲". The interface is consistent with the previous screenshot, maintaining the same layout and styling.

3.5.2.15 EEDM5011-下拉多选示例

3.5.2.15.1 页面功能简介

本示例页面用于说明，如何使用 **EFMultiselect** 下拉多选标签实现下拉框中数据的多项选择。

3.5.2.15.2 配置和 API 详情

参考 [EFMultiSelect](#)。

3.5.2.15.3 页面开发步骤

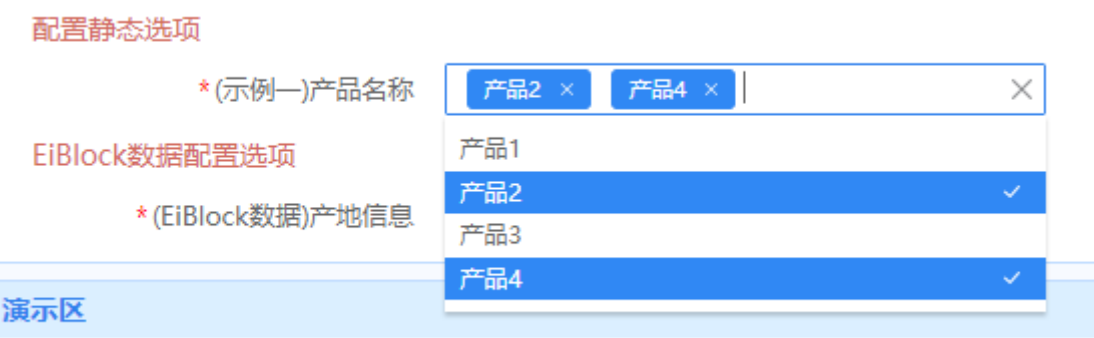
3.5.2.15.3.1 EFMultiSelect 配合 EFOption 标签渲染多选下拉组件

3.5.2.15.3.1.1 JSP 页面开发

EFMultiSelect 下拉多选示例

```
<EF:EFRegion id="inqu" title="示例展示区">
  <div class="row">
    <%--静态数据选项
      colWidth 设置多选下拉控件的栅格宽度；ratio 设置标签和 input 的栅格比例(默认 4:8)；
      defaultValue 设置初始选中项,各项间以',' 隔开；autoClose 设置控件选择一项后是否自动关闭,默认 true；
      动态属性 filter 设置下拉搜索,filter 可取值
      "startswith","endswith","contains",MultiSelect 控件默认 startswith 过滤;--%>
    <h5 class="push-30-l text-danger">配置静态选项</h5>
    <EF:EFMultiSelect ename="inqu_status-0-product_name" cname="(示例一)产品名称"
      colWidth="6" ratio="4:6" required="true" filter="contains"
      defaultValue="p2,p4" autoClose="false">
//defaultValue="p2,p4" : 默认产品 2 产品 4；配置了 autoClose="false",因此选完选项后，下拉选项框不会关闭
      <EF:EFOption label="产品 1" value="p1"/>
      <EF:EFOption label="产品 2" value="p2"/>
      <EF:EFOption label="产品 3" value="p3"/>
      <EF:EFOption label="产品 4" value="p4"/>
    </EF:EFMultiSelect>
  </div>
</EF:EFRegion>
```

3.5.2.15.3.1.2 页面效果如下所示



3.5.2.15.3.2 EFMultiSelect 配合 EFOptions 标签渲染多选下拉组件

3.5.2.15.3.2.1 JSP 页面开发

EFMultiSelect 下拉多选示例

```
<EF:EFRegion id="inqu" title="示例展示区">
  <div class="row">
    <%--EiBlock 数据选项
      blockId 数据块英文名; textField 显示值对应列名; valueField 数据值对应列
      名--%>
    <h5 class="push-30-l text-danger">EiBlock 数据配置选项</h5>
    <EF:EFMultiSelect ename="inqu_status-0-product_area" cname="(EiBlock 数据)产地
      信息"
      colWidth="6" ratio="4:6" required="true"
      defaultValue="China" > //默认选项: 中国
      <EF:EFOptions blockId="made_country_block_id" textField="country_ch_name"
        valueField="country"/>
    </EF:EFMultiSelect>
  </div>
</EF:EFRegion>
```

3.5.2.15.3.2.2 Service 服务开发

ServiceEEDM5010

```
public class ServiceEEDM5011 extends ServiceEPBase {
    private final static String MADE_COUNTRY_BLOCK_ID = "made_country_block_id";

    private void initMadeCountryBlock(EiBlock formTypeBlock) {
        EiBlockMeta metadata = new EiBlockMeta();

        EiColumn eiColumn = new EiColumn("country");
        eiColumn.setDescName("产地名称");
        metadata.addMeta(eiColumn);
        eiColumn = new EiColumn("country_ch_name");
        eiColumn.setDescName("产地名称中文");
        metadata.addMeta(eiColumn);
        formTypeBlock.setBlockMeta(metadata);

        Map<String, String> row = new HashMap<String, String>();
        row.put("country", "China");
        row.put("country_ch_name", "中国");
        formTypeBlock.addRow(row);

        row = new HashMap<String, String>();
        row.put("country", "Canada");
        row.put("country_ch_name", "加拿大");
        formTypeBlock.addRow(row);

        row = new HashMap<String, String>();
        row.put("country", "Japan");
        row.put("country_ch_name", "日本");
        formTypeBlock.addRow(row);
    }
}
```

3.5.2.15.3.2.3 页面效果如下所示



3.5.2.15.3.3 获取产品名称 value 和 label

3.5.2.15.3.3.1 JSP 页面开发

EFMultiSelect 下拉多选示例

```
<div class="row">
  <div class="col-md-6">
    <div class="form-group">
      <div class="col-xs-6">
        <span type="button" id="getProduct" class="i-btn-lg btn-inline-block"
href="javascript:;">(示例一)获取产品</span>
      </div>
      <div class="col-xs-6">
        <p class="text-white bg-info" style="padding: 5px 10px;">
          <span style="display:inline-block" id="product01"></span>
        </p>
      </div>
    </div>
  </div>
</div>
```

3.5.2.15.3.3.2 JS 示例

EEDM5011.js

```
var product_name,
    product_area;

$(window).load(function () {
    product_name = $("#inqu_status-0-product_name").data("kendoMultiSelect");
    product_area = $("#inqu_status-0-product_area").data("kendoMultiSelect");
});

// get 接口
$("#getProduct").on("click", function () {
    $("#product01").html(product_name.value().join(", ")); // 点击按钮：(示例一)获取产品
    后，获取产品 value，并用“,”分开

    console.log("产品名称 value:");
    console.log($("#inqu_status-0-product_name").data("kendoMultiSelect").value()); //
    在控制台获取相应的 value 和 label
    console.log("产品名称 label:");
    console.log(_.map($("#inqu_status-0-
product_name").data("kendoMultiSelect").dataItems(), _.iteratee("textField"))
```

```
});
```

3.5.2.15.3.3.3 页面效果如下所示



3.5.2.15.3.4 手动设置下拉输入框数据

3.5.2.15.3.4.1 JSP 页面开发

EFMultiSelect 下拉多选示例

```
<div class="row">
  <div class="col-md-6">
    <div class="form-group">
      <div class="col-xs-6">
        <span type="button" id="setProduct" class="i-btn-lg btn-inline-block"
href="javascript:;(小代码)设置产品">(示例一)设置产品</span>
      </div>
    </div>
  </div>
  <div class="col-md-6">
    <div class="form-group">
      <div class="col-xs-6">
        <span type="button" id="setArea" class="i-btn-lg btn-inline-block"
href="javascript:;(小代码)设置产地">(小代码)设置产地</span>
      </div>
    </div>
  </div>
</div>
```

3.5.2.15.3.4.2 JS 示例

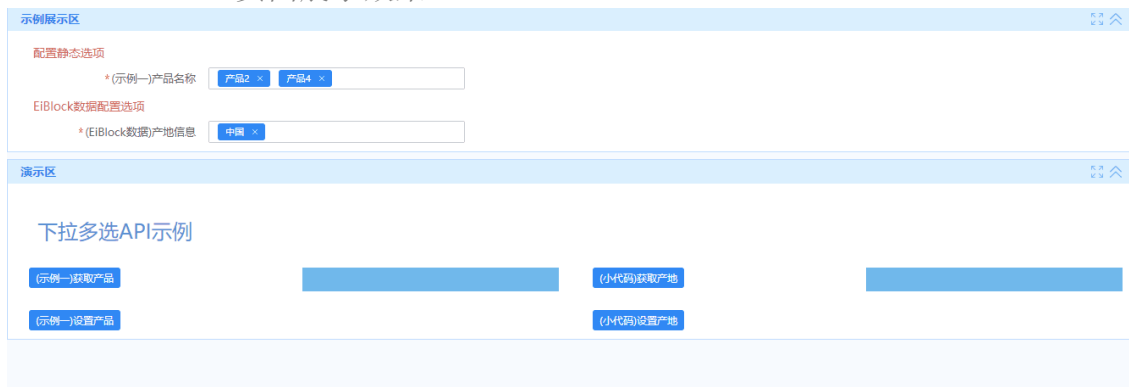
EEDM5011.js

```
// set 接口
$("#setProduct").on("click", function () {
  product_name.value("p1"); //手动设置下拉输入框数据为产品 1 (p1)
  product_name.trigger("change"); // 需要手动触发 change 事件
```

```
});

$("#setArea").on("click", function () {
    product_area.value(["Canada", "China"]);////手动设置下拉输入框数据为加拿大
    (Canada)，中国 (China)
    product_area.trigger("change"); // 需要手动触发 change 事件
});
```

3.5.2.15.3.4.3 页面展示效果



1. 点击输入框空白区域或某已选项，弹出下拉列表
2. 已选项在下拉list中会有样式上的变化，点击输入框中已选项右侧“x”号 或 下拉列表中对应已选项 可以删除对应项
3. 示例效果区显示所有已选项value值，以“,”符连接
4. EFMultiSelect获取选中项的value值数组： `$("#inqu_status-0-product_name").data("kendoMultiSelect").value()`
 EFMultiSelect获取选中项的label值数组： `_.map($("#inqu_status-0-product_name").data("kendoMultiSelect").dataItems(),
 _.iteratee("textField"))`

3.5.2.16 EEDM5012-EFSelect 标签与 EFOption 等标签配合完成下拉相关功能

3.5.2.16.1 页面功能简介

本示例页面用于说明，如何使用 **EFSelect** 标签与 **EFOption** 等标签配合完成下拉框相关功能（涉及 [EFSelect 中动态属性](#)）、如何通过设置 **DataSource** 动态隐藏或者显示 **EFOption** 标签以及 **API** 的介绍。

3.5.2.16.2 配置和 API 详情

参考 [EFSelect](#)。

3.5.2.16.3 页面开发步骤

3.5.2.16.3.1 渲染下拉框

3.5.2.16.3.2 EFOption

3.5.2.16.3.2.1 JSP 页面开发

EFOption 标签设置静态下拉选项。

EFOption

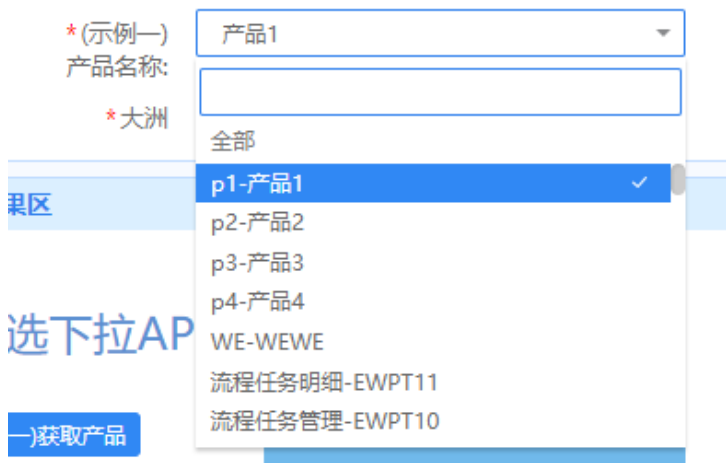
```
<%--静态的（手动设置的）数据选项，filter 表示下拉选项可以搜索，contains 是字符串
匹配 like %value% --%>
<EF:EFSelect ename="inqu_status-0-product_name" cname="(示例一)<br>产品名称"
    optionLabel="{valueField:'all', textField:'全部'}"
    template="#=valueField#-#=textField#" filter="contains" //filter: 可以下
拉搜索 contains: 字符串匹配
    required="true" defaultValue="p1">
    <%--<EF:EFOption label="全部" value="all"/>--%>
    <EF:EFOption label="产品 1" value="p1"/> //一条一条 option 的写入
    <EF:EFOption label="产品 2" value="p2"/>
    <EF:EFOption label="产品 3" value="p3"/>
    <EF:EFOption label="产品 4" value="p4"/>
    <EF:EFTableOption tableName="TEDFA00" textField="FORM_ENAME"
valueField="FORM_CNAME" //基于数据库的下拉选择
    condition="FORM_ENAME like '%EW%' " orderBy="FORM_ENAME DESC"/>
</EF:EFSelect>
```

1. EFSelect的下拉选项来源仅包括EFOption/EFOptions/EFCodeOption/EFTableOption子标签时，无需设置textField和valueField属性；
2. EFSelect的下拉选项来自serviceName，methodName查询时，**必须**设置textField和valueField属性；
3. EFSelect混合使用serviceName，methodName查询和EFOptions子标签时，**必须**设置textField和valueField属性；

注意：

- 上述的三种使用场景中，EFSelect的下拉选项模板template或valueTemplate，均固定的由"textField"和"valueField"组合而成（即使textField的值不是"textField",valueField的值不是"valueField"）（template="#=**valueField**
#-#=**textField**#"）
因为在封装过程中，EFOption/EFOptions/EFCodeOption的textField和valueField的值已经固化为"textField"和"valueField"。
- 不同于EFSelect，EFCascadeSelect(template和valueTemplate灵活可配，根据textField和valueField实际的取值来搭配)
- 不同于EFSelect，EFComboColumn(itemTemplate和columnTemplate同样可以灵活配置)
- template与valueTemplate属性不一定都需要进行配置，没有配置时，下拉选项默认显示textField值；

3.5.2.16.3.2.2 页面展示效果



3.5.2.16.3.3 EFOptions

3.5.2.16.3.3.1 JSP 页面开发

EFOptions 通过 blockId 配置下拉选项。blockId 指数据块对应 Id，textField 指显示值的字段名，valueField 指数据值的字段名。

EFOptions

```
<%--EiBlock 数据选项--%>
<EF:EFSelect ename="inqu_status-0-product_area2" cname="(示例三)<br>产地信息"
    template="#=valueField#-#=textField#" valueTemplate="#=valueField#-
    #=textField#"
    required="true" defaultValue="China" enable="false"> //enable="false":不可
    下拉
    <EF:EFOptions blockId="made_country_block_id" textField="country_ch_name"
    valueField="country"/>
</EF:EFSelect>
```

EFOptions属性变更

- 1.EFOptions标签中原先的labelColumn、valueColumn属性已被textField和valueField替换
- 2.EFOptions原先的conj属性已弃用，现在其父标签上(如配置EFSelect标签的template或valueTemplate属性)中处理

注：EFSelect 根据 ename 进行数据绑定（EiInfo）。在组件初始化时，根据 ename 的三段式设置，获取页面初始 EiInfo 中的对应值作为 EFSelect 的默认选中选项，form 元素的 id 也由它确定。上面的示例将默认选中 initLoad 中返回的 eiInfo 中 blockId="made_country_block_id"的 country 值 "China"。

```

< ▼ EiInfo {name: "", descName: "", msg: "多选下拉示例", msgKey: "", detailMsg: "", ...} ⓘ
  ▼ blocks:
    ▼ made_country_block_id: EiBlock
      colCount: 0
      ▶ extAttr: {}
      ▶ meta: EiBlockMeta {blockId: "made_country_block_id", metas: {...}, desc: "", extAttr: {...}, colCount: 2}
      ▼ rows: Array(3)
        ▼ 0: Array(2)
          0: "China"
          1: "中国"
          length: 2
          ▶ __proto__: Array(0)
        ▶ 1: (2) ["Canada", "加拿大"]
        ▶ 2: (2) ["Japan", "日本"]
          length: 3
          ▶ __proto__: Array(0)
        ▶ __proto__: AbstractEi

```

3.5.2.16.3.3.2 Service 服务开发

EOptions 配置下拉项对应的 service，构造一个 blockId 为 "made_country_block_id" 的 EiBlock。

ServiceEEDM5011

```

public class ServiceEEDM5011 extends ServiceEPBase {
    private final static String MADE_COUNTRY_BLOCK_ID = "made_country_block_id";

    private void initMadeCountryBlock(EiBlock formTypeBlock) {
        EiBlockMeta metadata = new EiBlockMeta();

        EiColumn eiColumn = new EiColumn("country");
        eiColumn.setDescName("产地名称");
        metadata.addMeta(eiColumn);
        eiColumn = new EiColumn("country_ch_name");
        eiColumn.setDescName("产地名称中文");
        metadata.addMeta(eiColumn);
        formTypeBlock.setBlockMeta(metadata);

        Map<String, String> row = new HashMap<String, String>();
        row.put("country", "China");
        row.put("country_ch_name", "中国");
        formTypeBlock.addRow(row);

        row = new HashMap<String, String>();
        row.put("country", "Canada");
        row.put("country_ch_name", "加拿大");
        formTypeBlock.addRow(row);

        row = new HashMap<String, String>();
        row.put("country", "Japan");
        row.put("country_ch_name", "日本");
        formTypeBlock.addRow(row);
    }

    public EiInfo initLoad(EiInfo inInfo) {
        EiInfo eiInfo = new EiInfo();
        initMadeCountryBlock(eiInfo.addBlock(MADE_COUNTRY_BLOCK_ID));
        eiInfo.addMsg("多选下拉示例");

        List rows = new ArrayList();
        EiInfo info = new EiInfo();
    }

```

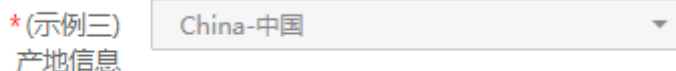
```

        info.set(EiConstant.serviceId, "S_ED_CM_00");
        info.set("codesetCode", "iplat.dynamicGird");

        info = XServiceManager.call(info);
        return eiInfo;
    }
}

```

3.5.2.16.3.3 页面展示效果



3.5.2.16.3.4 EFCodeOption

3.5.2.16.3.4.1 JSP 页面开发

EFCodeOption 通过配置小代码来设置下拉选项。**codeName** 配置小代码对应的代码类别名称。

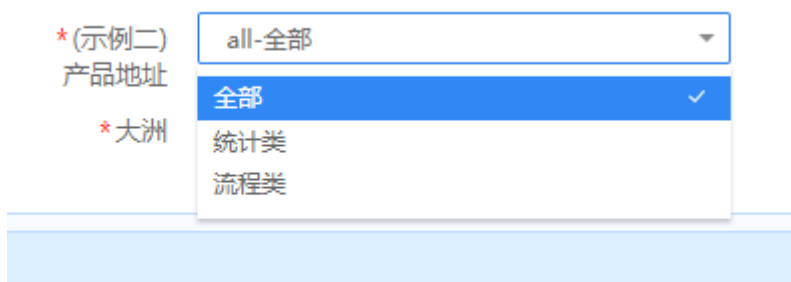
EFCodeOption

```

<!--小代码数据选项-->
<EF:EFSelect ename="inqu_status-0-product_area" cname="(示例二)<br>产品地址"
    valueTemplate="#=valueField#-#=textField#" required="true"
    defaultValue="all">
    <EF:EFOption label="全部" value="all"/>
    <EF:EFCodeOption codeName="iplat.ee.dm.validate.1"/>
    <EF:EFCodeOption codeName="iplat.codeStyle"/>
    <EF:EFCodeOption codeName="dm.hb"/>
</EF:EFSelect>

```

3.5.2.16.3.4.2 页面展示效果



3.5.2.16.3.5 EFTableOption

我们还是以示例一为例，示例一中混合使用了 **EFOption** 和 **EFTableOption** 标签。

1.必须指定beanName，sql或者tableName属性之一

2.若配置tableName，则valueField和textField也必须配置，tableName要指定到数据库

3.直接使用sql必须保证语句语法的正确性

3.5.2.16.3.5.1 页面展示效果



3.5.2.16.3.5.2 JSP 页面开发

通过服务方法名配置下拉选项，可配合 EFOption/EFOptions 等使用。optionLabel 作为引导信息，serviceName 指服务名，methodName 指方法名，textField 指数据展示值字段名，valueField 指 value 值字段名。

serviceName

```
<EF:EFSelect ename="inqu_status-0-continent" cname="大洲"
    template="#=valueField#-#=textField#" valueTemplate="#=valueField#-
    #=textField#" //下拉选中后的结果模板
    required="true" resultId="result" //resultId:下拉选项在 EiInfo 中的数据块
    名
    optionLabel="请选择"

    serviceName="EEDM5010" methodName="initLoad" textField="continent_name"
    valueField="continent_code">
    <EF:EFOption label="所有" value="all"/> //手写下拉选项
</EF:EFSelect>
```

optionLabel的使用

optionLabel是EFSelect的动态属性。一般用于提示用户字段可选。

它有两种使用场景

- 1.optionLabel是简单的String，此时EFSelect组件的value是""；
- 2.optionsLabel是一个JSON字符串，

```
optionLabel="{valueField:'all', textField:'全部'}"
```

此时EFSelect组件的value是"all";

optionLabel

不受template, valueTemplate的影响。只会渲染textField字段。



所以optionLabel可用于处理存在value是空的选项，使用template导致的显示问题。

3.5.2.16.3.5.3 Service 服务开发

通过服务方法名来配置下拉选项的对应 service。

ServiceEEDM5010

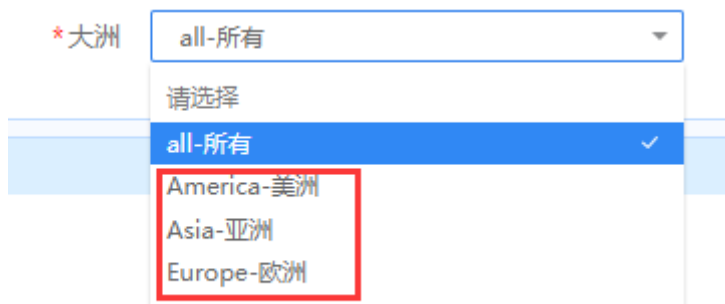
```
public class ServiceEEDM5010 extends ServiceEPBase {

    public EiInfo initLoad(EiInfo inInfo) {
        return super.query(inInfo, "EEDM02.query", new TEEDM02());
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        return super.query(inInfo, "EEDM03.query", new TEEDM03());
    }

    .....
}
```

3.5.2.16.3.5.4 页面展示效果



3.5.2.16.3.6 API

3.5.2.16.3.6.1 获取 EFSelect 标签选中项的值

JS 示例

```
...
// 获取真实值
```

```

console.log("产品名称 value:");
console.log(IPLAT.EFSelect.value($("#inqu_status-0-product_name")));
// 获取显示值
console.log("产品名称 label:");
console.log(IPLAT.EFSelect.text($("#inqu_status-0-product_name")));

```

页面效果如下所示：



3.5.2.16.3.6.2 设置 EFSelect 标签选中项的真实值 JS 示例

```
IPLAT.EFSelect.value($("#inqu_status-0-product_name"), "p2");
```

页面效果如下所示：

注：jsp代码的注释中写有相关说明

* (示例一)
产品名称

产品2

* (示例三)
产地信息

China-中国

* (示例二)
产品状态

示例效果区

下拉API示例

(示例一)获取产品名称

(示例二)设置数据

3.5.2.16.3.6.3 设置真实值，并触发 change 事件

`IPLAT.EFSelect.value($("#inqu_status-0-product_name"),"p3", true);`

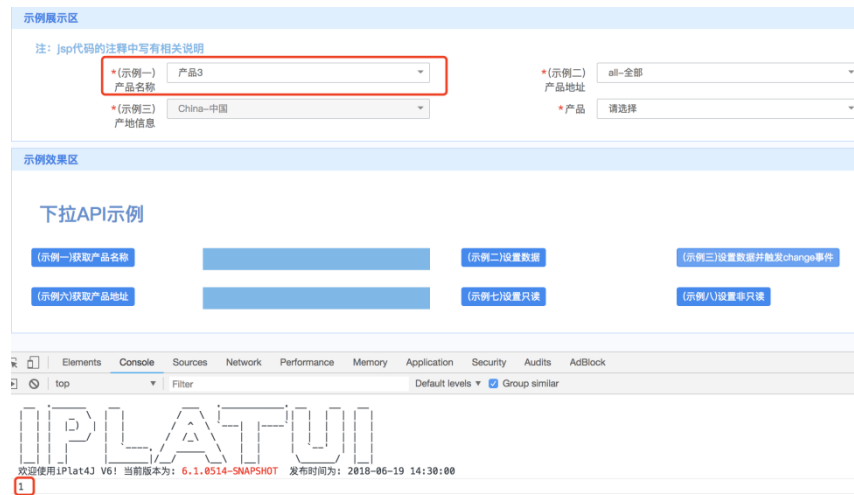
如果设置第三个参数 `true`，将触发 `change` 事件：

JS 示例

```
// 设置真实值，并触发 change
$("#setValue02").on("click", function () {
    IPLAT.EFSelect.value($("#inqu_status-0-product_name"),"p3", true);
});

IPLATUI.EFSelect = {
    "inqu_status-0-product_name": {
        // 下拉选项改变之后触发
        change: function () {
            console.log("1")
        }
    }
};
```

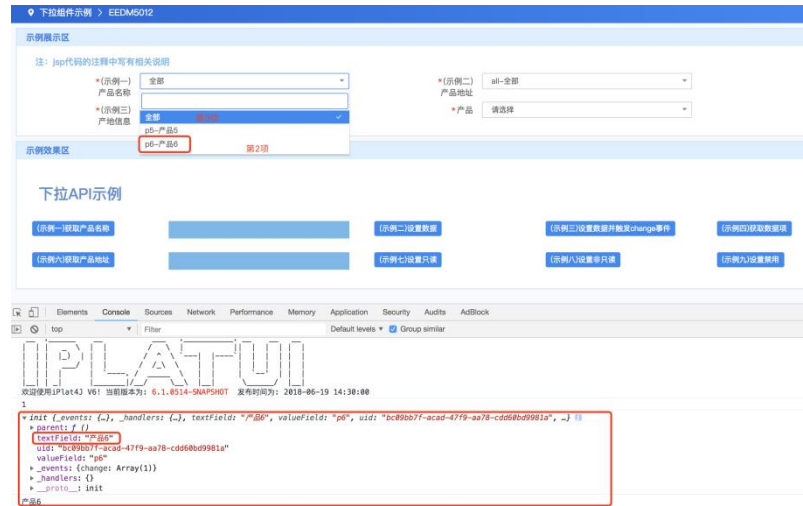
页面效果如下所示：



3.5.2.16.3.6.4 根据索引，获取数据项
JS 示例

```
// 根据索引，获取第二项的数据信息
$("#getData").on("click", function () {
    console.log(IPLAT.EFSelect.dataItem($("#inqu_status-0-product_name"),2));
    // 获取第二项数据的显示值
    console.log(IPLAT.EFSelect.dataItem($("#inqu_status-0-product_name"),2) ["textField"]);
});
```

页面效果如下所示：



3.5.2.16.3.6.5 重置 dataSource

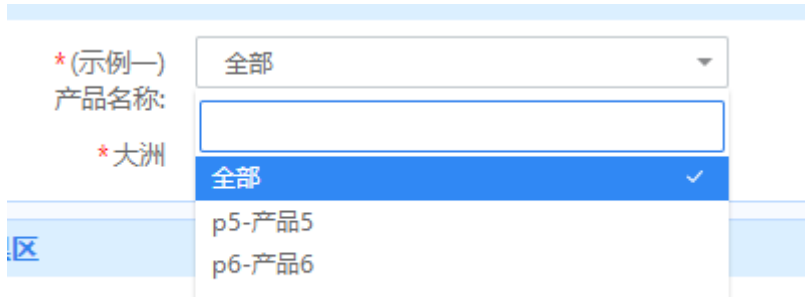
3.5.2.16.3.6.6 JS 页面开发
更改产品一的 Option 选项：

JS 示例

```
//动态选择展示或隐藏 EFOption 标签
$("#setDataSource").on("click", function (e) {
    var dataSource = new kendo.data.DataSource({
        data: [
            {textField: "产品 5", valueField: "p5"},
            {textField: "产品 6", valueField: "p6"}
        ]
    });

    IPLAT.EFSelect.setDataSource($("#inqu_status-0-product_name"), dataSource);
});
```

3.5.2.16.3.6.7 页面效果



当开发人员想要动态隐藏或显示 EFOption 标签时，可以使用此 API。

3.5.2.16.3.6.8 设置只读/非只读

JS 示例

```
// 设置输入框为只读
$("#setReadonly").on("click", function () {
    IPLAT.EFSelect.readonly($("#inqu_status-0-product_name"), true);
});
// 设置输入框非只读
$("#unsetReadonly").on("click", function () {
    IPLAT.EFSelect.readonly($("#inqu_status-0-product_name"), false);
});
```

3.5.2.16.3.6.9 设置可用/禁用

JS 示例

```
// 禁用输入框
$("#setEnable").on("click", function () {
    IPLAT.EFSelect.enable($("#inqu_status-0-product_name"), false);
});

// 设置输入框为可用
$("#unsetEnable").on("click", function () {
```

```
IPLAT.EFSelect.enable($("#inqu_status-0-product_name"), true);
});
```

3.5.2.17 EEDM5014-EFComboInput 示例

3.5.2.17.1 页面功能简介

本示例页面用于说明，如何使用 EFComboInput 实现 Input+Select 的功能。

3.5.2.17.2 配置和 API 详情

参考 [EFComboInput](#)。

3.5.2.17.3 页面开发步骤

3.5.2.17.3.1 JSP 页面开发

EFComboInput 输入框+下拉框示例

```
<EF:EFRegion id="inqu" title="EFComboinput 示例">
  <div class="row" id="exa">
    <EF:EFComboInput blockId="result" ename="0-param" cname="输入下拉选择框:"
resultId="result"
                                textField="continent_name" valueField="continent_code"
serviceName="EEDM5010" methodName="initLoad"> //指定服务名, 方法名
    </EF:EFComboInput>
  </div>
</EF:EFRegion>
</EF:EFPage>
```

3.5.2.17.3.2 JS 开发

JS 示例

```
$(function () {
  var eiInfo = new EiInfo();
  eiInfo.setByNodeObject(document.getElementById("exa"));
  console.log(eiInfo.toJSON()); // 查看数据
});
```

3.5.2.17.3.3 Service 服务开发

ServiceEEDM5010

```
public EiInfo initLoad(EiInfo inInfo) {
  return super.query(inInfo, "EEDM02.query", new EEDM02());
}
```

3.5.2.17.3.4 页面展示效果



```

{__sys__: {msg: "查询成功, 本次查询返回3条记录!", msgKey: "ep.2000", detailMsg: "", status: 0,...}, __version__: "2.0",...}
  $$remote$: "false"
  methodName: "initLoad"
  serviceName: "EEDM5010"
  serviceType: ""
  soaInvokeProtocol: "local"
  blocks: {result: {attr: {offset: 0, count: 0, limit: 10},...}}
  result: {attr: {offset: 0, count: 0, limit: 10},...}
    attr: {offset: 0, count: 0, limit: 10}
    meta: {columns: [{name: "continent_code", descName: "洲代号", primaryKey: true, nullable: false, pos: 0},...]}
    rows: [{"America", "美洲"}, {"Asia", "亚洲"}, {"Europe", "欧洲"}]
      0: ["America", "美洲"]
      1: ["Asia", "亚洲"]
      2: ["Europe", "欧洲"]
  __sys__: {msg: "查询成功, 本次查询返回3条记录!", msgKey: "ep.2000", detailMsg: "", status: 0,...}
  __version__: "2.0"

```

查看数据:

```

{__version__: "2.0", __sys__: {...}, __blocks__: {...}}
  __blocks__:
    result:
      attr: {}
      meta:
        attr: {}
        columns: Array(2)
          0: {pos: 0, name: "param"}
          1: {pos: 1, name: "param_unit"}
            length: 2
            __proto__: Array(0)
        desc: ""
        __proto__: Object
        rows: Array(1)
          0: Array(2)
            0: "12"
            1: "America"
            length: 2
            __proto__: Array(0)
          length: 1
          __proto__: Array(0)
        __proto__: Object
      __sys__: {name: "", descName: "", msg: "", msgKey: "", detailMsg: "", ...}
      version: "2.0"

```

3.5.2.18 EEDM5016-下拉框 select/change 事件绑定示例

3.5.2.18.1 页面功能简介

本示例页面用于说明，在页面 JS 文件中如何通过绑定下拉框的 select/change 事件，获取到下拉项的值。

3.5.2.18.2 配置和 API 详情

参考 [EFSelect 梳理。](#)

3.5.2.18.3 页面开发步骤

3.5.2.18.3.1 JSP 页面开发

EEDM5016.jsp

```

<EF:EFPage>
  <EF:EFRegion id="inqu" title="示例展示区">
    <div class="row">
      <EF:EFSelect ename="inqu_status-0-product" cname="产品（绑定 select 事件）"
        template="#=valueField#-#=textField#" //定制下拉的模板，如：
Nike-耐克
        ratio="5:7" optionLabel="{valueField:'all', textField:'全部
'}">
      <EF:EFOption label="耐克" value="Nike"/>
      <EF:EFOption label="阿迪" value="Adidas"/>
      <EF:EFOption label="彪马" value="Puma"/>
      <EF:EFOption label="安踏" value="Anta"/>
      <EF:EFTableOption tableName="TEDFA00" textField="FORM_ENAME"
valueField="FORM_ENAME"
        condition="FORM_ENAME like '%EW%' "
orderBy="FORM_ENAME DESC"/>
    </EF:EFSelect>

    <EF:EFSelect ename="inqu_status-0-area" cname="产地（绑定 change 事件）"
      template="#=valueField#-#=textField#"
      valueTemplate="#=valueField#-#=textField#" //定制选中后的结果
的模版，如：America-美洲
      required="true" ratio="5:7" optionLabel="请选择"
      serviceName="EEDM5010" methodName="initLoad" resultId="result"
通过服务名，方法名，数据块渲染数据
      textField="continent_name" valueField="continent_code">
      <EF:EFOption label="所有" value="all"/>
    </EF:EFSelect>
    </div>
  </EF:EFRegion>

  <EF:EFRegion title="示例效果区">
    <h3 class="block-header push-15 text-primary">下拉 select/change 事件绑定示例
</h3>
    <h5 class="text-white bg-danger" style="padding: 5px 10px; margin-left:24px;
width:700px;">
      （蓝色区域显示的是点击按钮后获取下拉框的当前值，绿色区域显示的是勾选或改变的
下拉项的值）
    </h5>
    <div class="row">
      <div class="col-md-4">
        <div class="col-xs-4">
          <span class="k-button k-long-content-btn product">产品信息</span>
        </div>
        <div class="col-xs-8">
          <p class="text-white bg-info" style="padding: 5px 10px;">
            <span style="display:inline-block" id="product_value"></span>
          </p>
          <p class="text-white bg-info" style="padding: 5px 10px;">
            <span style="display:inline-block" id="product_text"></span>
          </p>
        </div>
      </div>
    </div>
  </EF:EFRegion>

```

```

        <p class="text-white bg-success" style="padding: 5px 10px;">
            <span style="display:inline-block"
id="product_select_value"></span>
        </p>
        <p class="text-white bg-success" style="padding: 5px 10px;">
            <span style="display:inline-block"
id="product_select_text"></span>
        </p>
    </div>
</div>
<div class="col-md-4">
    <div class="col-xs-4">
        <span class="k-button k-long-content-btn area">产地信息</span>
    </div>
    <div class="col-xs-8">
        <p class="text-white bg-info" style="padding: 5px 10px;">
            <span style="display:inline-block" id="area_value"></span>
        </p>
        <p class="text-white bg-info" style="padding: 5px 10px;">
            <span style="display:inline-block" id="area_text"></span>
        </p>

        <p class="text-white bg-success" style="padding: 5px 10px;">
            <span style="display:inline-block"
id="area_change_value"></span>
        </p>
        <p class="text-white bg-success" style="padding: 5px 10px;">
            <span style="display:inline-block"
id="area_change_text"></span>
        </p>
    </div>
</div>
</div>
</EF:EFRegion>
</EF:EFPage>

```

3.5.2.18.3.2 JS 开发

EEDM5016.js

```

$(function () {
    var enames = ["product", "area"];
    $.each(enames, function (ename) {
        // 点击按钮获取下拉列 value/text 值
        $("." + ename).on("click", function ()
            var select = $("#inqu_status-0-" + ename).data("kendoDropDownList");
            $("#" + ename + "_value").html(select.value()); //真实值
            $("#" + ename + "_text").html(select.text()); //显示值
        });
    });

    IPLATUI.EFSelect = {
        "inqu_status-0-product": {
            // 点击下拉选项时触发
            select: function (e) { //获取勾选值
                var dataItem = e.dataItem;
            }
        }
    }

```

```

        $("#product_select_value").html(dataItem['valueField']);
        $("#product_select_text").html(dataItem['textField']);
    }
},
"inqu_status-0-area": {
    // 下拉选项改变之后触发
    change: function (e) { //获取改变值
        $("#area_change_value").html(this.value());
        $("#area_change_text").html(this.text());
    }
}
});

```



前者通过点击按钮获取当前下拉项的值，后者在下拉项被勾选或改变时，触发获取值的行为。

3.5.2.18.3.3 Service 服务开发

ServiceEEDM5010

```

public class ServiceEEDM5010 extends ServiceEPBase {
    public EiInfo initLoad(EiInfo inInfo) { //serviceName:EEDM5010 methodName:initLoad
        return super.query(inInfo, "EEDM02.query", new EEDM02());
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        return super.query(inInfo, "EEDM03.query", new EEDM03());
    }

    public EiInfo queryProvince(EiInfo inInfo) {
        return super.query(inInfo, "EEDM07.query", new EEDM07());
    }
}

```

EEDM02.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE sqlMap PUBLIC "-//IBATIS.com//DTD SQL Map 2.0//EN"
"http://www.ibatis.com/dtd/sql-map-2.dtd">

<sqlMap namespace="EEDM02">

    <select id="count" resultClass="int">
        SELECT count(*) FROM ${platSchema}.teedm02 where l=1
        <isNotEmpty prepend=" AND " property="continent_code">
            continent_code = #continent_code#
        </isNotEmpty>
    </select>

    <select id="query" resultClass="java.util.HashMap">
        SELECT
            continent_code      as "continent_code", //洲代号

```

```

        continent_name      as "continent_name"    //洲名称
    FROM ${platSchema}.teedm02
    order by continent_code asc
</select>

<insert id="insert">
    INSERT INTO ${platSchema}.teedm02 (
        continent_code,
        continent_name
    ) VALUES (
        #continent_code#,
        #continent_name#
    )
</insert>

<update id="update">
    UPDATE ${platSchema}.teedm02 SET
        continent_code = #continent_code#,
        continent_name = #continent_name#
    WHERE continent_code = #continent_code#
</update>

<delete id="delete">
    DELETE FROM ${platSchema}.teedm02
    WHERE continent_code = #continent_code#
</delete>

</sqlMap>

```



第二个下拉框（产地）：通过 `serviceName(服务名):EEDM5010`、`methodName(方法名):initLoad`，以及 `resultId(数据块名):result`，渲染下拉数据项。

3.5.2.18.3.4 页面展示效果

示例展示区

产品 (绑定select事件) *产地 (绑定change事件)

示例效果区

下拉select/change事件绑定示例

(蓝色区域显示的是点击按钮后获取下拉框的当前值, 绿色区域显示的是勾选或改变的下拉项的值)

产品信息

产地信息

3.5.2.19 EEDM4807-Form 表单 get/set 数据 API 示例

3.5.2.19.1 页面功能简介

本示例页面用于介绍 Form 表单获取/设置数据的 API，并举例说明如何使用 API。

3.5.2.19.2 配置和 API 详情

- 相关文档：参考 [EFormInput](#)、[EEDM4002-EFormInput 格式化](#)、[EEDM4004-自动补全查询](#)和 [EEDM4003-EFormInput 类型](#)文档。
- 适用范围：可调用此 API 的 Form 表单有：EFormInput(普通输入框)，EFormInputFormat(有格式化的输入框)，EFormAutoComplete（自动补全），checkbox/radio(复选/单选)，textarea（文本框）
- 用法详情：获取数据：IPLAT.EFormInput.value(\$(selector))
- 设置数据：IPLAT.EFormInput.value(\$(selector), value) 其中，value 为设置的值
- 重点：此文档重点介绍 EFormInputFormat(有格式化的输入框)，其余表单组件较为简单，只需直接根据示例调用 API 即可。

3.5.2.19.3 页面开发步骤

3.5.2.19.3.1 JSP 页面开发

创建 Form 表单：

EFormPage

```
<EF:EFormPage>
...
    <EF:EFormInput ename="inqu_status-0-field1" cname="输入框"/>
...
    <EF:EFormInput ename="inqu_status-0-field21" cname="请输入数字" format="{0:c}"/>
...
    <EF:EFormInput ename="inqu_status-0-field22" cname="请输入数字" format="{0:c}"/>
...
    <EF:EFormAutoComplete ename="inqu_status-0-field3" cname="自动补全"
dataField="product_id"
                        queryMethod="query" serviceName="EEDM6000"
resultId="result"/>
...
    <EF:EFormInput ename="phone" value="xiaomi" cname="小米" type="checkbox"/>
    <EF:EFormInput ename="phone" value="huawei" cname="华为" type="checkbox"/>
    <EF:EFormInput ename="phone" value="meizu" cname="魅族" type="checkbox"/>
...
    <EF:EFormInput ename="gender" cname="男" value="man" inline="true" type="radio"/>
    <EF:EFormInput ename="gender" cname="女" value="woman" inline="true"
type="radio"/>
...
```

```
<EF:EFInput ename="inqu_status-0-field6" cname="文本框" value="2018"
type="textare"/>
</EF:EFPage>
```

3.5.2.19.3.2 JS 开发

3.5.2.19.3.2.1 Input 普通输入框

具体示例

```
//获取 Input 数据
IPLAT.EFInput.value($("#inqu_status-0-field1"))
//设置 Input 数据
IPLAT.EFInput.value($("#inqu_status-0-field1"), "iPlat")
```

其中，“iPlat”为设置的值。

页面效果如下所示：

3.5.2.19.3.2.2 Input 格式化

EFInput 可以通过 `format` 属性对输入内容进行格式化。

 要注意的是：此处实际上是有两个 Input，一个显示的 Input 用于展现格式化后的值；还有一个隐藏的 Input 用于存储与后端进行交互的 `value` 值。

显而易见，两个输入框的 `id` 也肯定不同：需要与后端进行交互的 Input，会在界面上隐藏，它的 `Id` 就是 EFInput 的 `ename`；而用于展示信息的 Input 的 `Id` 则是在 `ename` 后加了一个字段： `_textField`。



因此，开发者在获取或者设置值时，可以选择获取隐藏框的值，或者显示框的值。

3.5.2.19.3.2.2.1 获取/设置隐藏框的值

具体示例

```
//获取隐藏框的值
IPLAT.EFInput.value($("#inqu_status-0-field21")) //id 即为 ename
//设置隐藏框的值
IPLAT.EFInput.value($("#inqu_status-0-field21"), "13000")
```



设置完值后，需要先点击输入框，再失去焦点后，方可显示格式化后的值。

页面效果如下所示：

Format

请输入数字

获取输入框数据

设置输入框数据

3.5.2.19.3.2.2 获取/设置显示框的值

具体示例

```
//获取显示框的值
IPLAT.EFInput.value($("#inqu_status-0-field22_textField")) //id 为 ename+_textField
//设置显示框的值
IPLAT.EFInput.value($("#inqu_status-0-field22_textField"), "13000")
```

页面效果如下所示：

请输入数字

获取输入框数据

设置输入框数据

如上图所示：隐藏框获取的是真实的 **value** 值（300），而显示框获取的则是格式化后的值（¥233.00）；隐藏框的值需要设置后，点击输入框更改值，失去焦点，才获得格式化后的值，而显示框的值只需直接设置更改即可

3.5.2.19.3.2.3 AutoComplete

具体示例

```
//获取input数据
IPLAT.EFInput.value($("#inqu_status-0-field3")) //按回车即可
//设置input数据
IPLAT.EFInput.value($("#inqu_status-0-field3"), "jungle")
```

3.5.2.19.3.2.4 checkbox

Code Block 4 具体示例

```
//获取 Input 数据
IPLAT.EFInput.value$("input[name='phone']")
//设置 Input 数据
IPLAT.EFInput.value$("input[name='phone']", "xiaomi,huawei")
```



此处的 $\$(selector)$ 为 $\$("input[name='phone'])$ ，设置的值为该选项对应的 value 值，并用“，”连接。

页面效果如下所示：

checkbox

- ☐ 小米
- ☐ 华为
- ☐ 魅族

获取多选框数据

设置checkbox数据

3.5.2.19.3.2.5 radio

具体示例

```
//获取 Input 数据
IPLAT.EFInput.value$("input[name='gender']") // $(selector) 和 checkbox 相同
//设置 Input 数据
IPLAT.EFInput.value$("input[name='gender']", "woman")
```

3.5.2.19.3.2.6 textarea

具体示例

```
//获取 Input 数据
IPLAT.EFInput.value$("#inqu_status-0-field6") //按要求调用 API 即可
//设置 Input 数据
IPLAT.EFInput.value($("#inqu_status-0-field6"), "Hello, iPlatV6!")
```

3.5.2.20 EEDM5006-depth 与 start 日期控件显示层级

3.5.2.20.1 页面功能简介

本示例页面用于介绍 EFDatPicker 与 EFDatSpan 的 start 和 depth 属性的使用方法。

3.5.2.20.2 配置和 API 详情

相关文档：参考 [EFDatePicker](#) 与 [EFDateSpan](#) 中的描述。

项目组直接于页面 js 中配置 depth 与 start 属性。

3.5.2.20.3 页面开发步骤

JSP 页面开发

EEDCA342.jsp

```
<%--EFDatePicker: depth 日期控件可以显示到的层级--%>
<%--
    日期控件的层级: [month, year, decade, century]
        月: 显示一个月中的每一天
        年: 显示一年中的十二个月
        十年: 显示十年中的每一年
        世纪: 显示一个世纪中的每十年

    默认是 month, 即显示一个月中的每一天; 当 start 比较 depth 显示的要精细时, depth 属性失效。
    例如: start 设置为 month, 但是 depth 设置为 year, 日期控件显示一个月中的每一天。
--%>
<%--详见 js--%>
<EF:EFPage>
    <EF:EFRegion id="r1" title="depth 可展示的层级 decade">
        <EF:EFDatePicker ename="inqu_status-0-start_time" cname="开始时间">
        </EF:EFDatePicker>
    </EF:EFRegion>
    <EF:EFRegion id="r2" title="depth 可展示的层级 year">
        <EF:EFDatePicker ename="inqu_status-0-end_time" cname="结束时间">
        </EF:EFDatePicker>
    </EF:EFRegion>
    <EF:EFRegion id="r3" title="EFDateSpan 中的 depth">
        <h5 class="push-30-l text-danger">DateSpan 中需要按照 startName 和 endName, 来分别配置 start 与 depth 属性</h5>
        <EF:EFDateSpan role="datetime" startName="ss" endName="ee" interval="15"
startRatio="4:8"
                                startCname="开始日期" endCname="结束日期"
                                startValue="2018-08-20 13:30" parseFormats="['yyyy-MM-dd
HH:mm']"/>
    </EF:EFRegion>
</EF:EFPage>
```

JS 配置属性

EEDCA342.js

```
// 配置 start 与 depth 属性
/**
 * 日期控件的层级: [month, year, decade, century]
 * 月: 显示一个月中的每一天
 * 年: 显示一年中的十二个月
 * 十年: 显示十年中的每一年
 * 世纪: 显示一个世纪中的每十年
```

* 默认是 month，即显示一个月中的每一天；当 start 比较 depth 显示的要精细时，depth 属性失效。

* 例如：start 设置为 month，但是 depth 设置为 year，日期控件显示一个月中的每一天。

*/

```
IPLATUI.EFDatePicker = {
  "inqu_status-0-start_time": {
    start: "century",
    depth: "decade"
  },
  "inqu_status-0-end_time": {
    start: "decade",
    depth: "year"
  }
};
```

```
IPLATUI.EFDateSpan = {
  // 开始日期
  "ss": {
    start: "century",
    depth: "decade"
  },
  // 结束日期
  "ee": {
    start: "century",
    depth: "decade"
  }
};
```

3.5.2.20.4 页面展示效果



3.5.3 数据表格

3.5.3.1.1 EEDM6000-单表增删改查

3.5.3.1.1.1 页面功能简介

本示例页面用于说明，如何使用 **EFGrid** 标签进行单表的增删改查开发。

开发人员想要在前端开发本页面所提供的功能，需要对页面进行注册和开发相应的后端服务，详情请参考[开发一个单表增删改查页面](#)。

3.5.3.1.1.2 配置和 API 详情

参考 [EFGrid](#) 中的描述。

3.5.3.1.1.3 页面开发步骤

3.5.3.1.1.3.1 页面构造

3.5.3.1.1.3.1.1 JSP 页面开发

创建 JSP:

EEDM6000.jsp

```
<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}" />

<EF:EFPage title="单表增删改查示例">
  <!--EFGrid 用于展示表格数据，通常将其放置在 EFRegion 区域内。-->
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result">
    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFPage>
```

3.5.3.1.1.3.1.2 JS 开发

创建 JS:

EEDM6000.js

```
// 使用$(function() {XXX}) 去加载我们的业务逻辑， 这样能够保证我们的业务逻辑能够在页面加载完成后执行。
$(function () {

});
```

3.5.3.1.1.3.1.3 Service 服务开发

创建 `ServiceEEDM6000.java`。在 `ServiceEEDM6000.java` 文件中，我们定义基础的增删改查方法。

ServiceEEDM6000.java

```
package com.baosight.iplat4j.ee.dm.service;

import com.baosight.iplat4j.core.ei.EiInfo;
import com.baosight.iplat4j.ee.domain.EE10;
import com.baosight.iplat4j.core.service.impl.ServiceEPBase;

/**
 * 单表增删改查示例
 * ServiceBase 有默认的 update delete insert query 的实现
 */
public class ServiceEEDM6000 extends ServiceBase {

    // 初始化服务时候加载
    public EiInfo initLoad(EiInfo inInfo) {
        return query(inInfo);
    }

    // 增加
    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EE10.update");
    }

    // 删除
    public EiInfo delete(EiInfo inInfo) {
        EiInfo outInfo = super.delete(inInfo, "EE10.delete");
        return outInfo;
    }

    // 修改
    public EiInfo insert(EiInfo inInfo) {
        return insert(inInfo, "EE10.insert");
    }

    // 查询
    public EiInfo query(EiInfo inInfo) {
        EiInfo outInfo = super.query(inInfo, "EE10.query", new EE10());
        return outInfo;
    }
}
```



后端开发详情请参考 [后台开发入门](#) 和 [开发一个单表增删改查页面](#)。



如果没有在页面注册时指定特殊的服务名，默认的服务名跟页面应该是一一对应的。

页面信息管理 > EDFAD0

查询条件

画面英文名: EEDM600 画面中文名: 记录创建开始日期: 记录创建截止日期: 画面类型: 全部 工程: PLAT4J

记录集

默认排序: 默认认为ServiceEEDM6000

画面英文名	画面中文名	画面类型	画面载入的路径	一级模块	二级模块	初始处理服务英文名	画面参数	图标	是否授权	业务类别	操作分类	操作	创建人	创建时间
EEDM6000	单表增删改显示	平台托管		EE	DM							按钮信息	nameless	2017
EEDM6001	单页多Grid展示	平台托管		EE	DM							按钮信息	admin	2017
EEDM6002	Grid列表展示	平台托管		EE	DM	EEDM6002			授权			按钮信息	admin	2017
EEDM6003	Grid列表展示	平台托管		EE	DM	EEDM6000						按钮信息	admin	2017
EEDM6004	注册Grid功能	平台托管		EE	DM	EEDM6000						按钮信息	admin	2017
EEDM6005	自定义EFGrid	平台托管		EE	DM	EEDM6000						按钮信息	admin	2017
EEDM6006	Grid列表展示	平台托管		EE	DM	EEDM6000						按钮信息	admin	2017
EEDM6007	Grid列表展示	平台托管		EE	DM	EEDM6000						按钮信息	admin	2017
EEDM6008	Grid中自定义	平台托管		EE	DM	EEDM6000						按钮信息	nameless	2017
EEDM6009	EFGrid 数据	平台托管		EE	DM	EEDM6000						按钮信息	nameless	2017

3.5.3.1.1.3.2 前情概要

ServiceEEDM6000 中的方法和数据取自于 EE10.xml 和 EE10.java。

3.5.3.1.1.3.3 SQLmap 文件 EE10.xml

EE10.sql

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE sqlMap PUBLIC "-//ibatis.com/DTD SQL Map 2.0//EN"
"http://www.ibatis.com/dtd/sql-map-2.dtd">

<sqlMap namespace="EE10">

  <select id="count" resultClass="int">
    SELECT count(*) FROM ${platSchema}.tee10 where 1=1
    <isNotEmpty prepend=" AND " property="product_id">
      product_id like ('%$product_id$')
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="company_name">
      company_name = #company_name#
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="made_country">
      made_country = #made_country#
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="price_low">
      price > #price_low#
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="price_high">
      price < #price_high#
    </isNotEmpty>
  </select>

  <select id="query" resultClass="java.util.HashMap">
    SELECT
      product_id as "product_id",
      company_name as "company_name",
      made_country as "made_country",
      sale_date as "sale_date",
      price as "price",
      company_page as "company_page",
      search as "search",
      validate_0 as "validate_0",
      validate_1 as "validate_1",
```

```

        validate_2 as "validate_2",
        validate_3 as "validate_3",
        validate_4 as "validate_4",
        validate_5 as "validate_5",
        validate_6 as "validate_6",
        validate_7 as "validate_7",
        validate_8 as "validate_8",
        validate_9 as "validate_9"
    FROM ${platSchema}.tee10 where l=1
    <isNotEmpty prepend=" AND " property="product_id">
        product_id like ('%$product_id$%')
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="company_name">
        company_name like ('%$company_name$%')
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="made_country">
        made_country like ('%$made_country$%')
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="price_low">
        price > #price_low#
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="price_high">
        price < #price_high#
    </isNotEmpty>
    <dynamic prepend="ORDER BY">
        <isNotEmpty property="orderBy">
            $orderBy$
        </isNotEmpty>
    </dynamic>

</select>

<select id="dynamicQuery" resultClass="java.util.HashMap">
    SELECT
        product_id as "product_id",
        company_name as "company_name",
        made_country as "made_country",
        sale_date as "sale_date",
        price as "price",
        company_page as "company_page",
        search as "search",
        validate_0 as "validate_0",
        validate_1 as "validate_1",
        validate_2 as "validate_2",
        validate_3 as "validate_3",
        validate_4 as "validate_4",
        validate_5 as "validate_5",
        validate_6 as "validate_6",
        validate_7 as "validate_7",
        validate_8 as "validate_8",
        validate_9 as "validate_9"
    FROM ${platSchema}.tee10 where l=1
    <isNotEmpty prepend=" AND " property="sql">
        $sql$
    </isNotEmpty>
    <dynamic prepend="ORDER BY">
        <isNotEmpty property="orderBy">

```

```

        $orderBy$
    </isNotEmpty>
</dynamic>

</select>

<select id="filter" resultClass="java.util.HashMap">
    SELECT
    product_id as "product_id",
    company_name as "company_name",
    made_country as "made_country",
    sale_date as "sale_date",
    price as "price",
    company_page as "company_page",
    search as "search",
    validate_0 as "validate_0",
    validate_1 as "validate_1",
    validate_2 as "validate_2",
    validate_3 as "validate_3",
    validate_4 as "validate_4",
    validate_5 as "validate_5",
    validate_6 as "validate_6",
    validate_7 as "validate_7",
    validate_8 as "validate_8",
    validate_9 as "validate_9"
    FROM ${platSchema}.tee10 where 1=1

    <isNotEmpty prepend=" AND " property="filter_company_name">
        $filter_company_name$
    </isNotEmpty>

    <isNotEmpty prepend=" AND " property="filter_sale_date">
        $filter_sale_date$
    </isNotEmpty>

    <isNotEmpty prepend=" AND " property="filter_price">
        $filter_price$
    </isNotEmpty>

    <dynamic prepend="ORDER BY">
        <isNotEmpty property="orderBy">
            $orderBy$
        </isNotEmpty>
    </dynamic>
</select>

<select id="sum" resultClass="java.util.HashMap">
    SELECT
    SUM(PRICE)      as "price",
    SUM(PRICE)      as "validate_5" <!-- 测试用 -->
    FROM ${platSchema}.tee10 where 1=1
    <isNotEmpty prepend=" AND " property="product_id">
        product_id like ('%$product_id$%')
    </isNotEmpty>
    <isNotEmpty prepend=" AND " property="company_name">
        company_name = #company_name#
    </isNotEmpty>

```

```

        <isEmpty prepend=" AND " property="made_country">
            made_country = #made_country#
        </isEmpty>
    </select>

    <select id="load" resultClass="java.util.HashMap">
        SELECT
        product_id as "product_id",
        company_name as "company_name",
        made_country as "made_country",
        sale_date as "sale_date",
        price as "price",
        company_page as "company_page",
        search as "search",
        validate_0 as "validate_0",
        validate_1 as "validate_1",
        validate_2 as "validate_2",
        validate_3 as "validate_3",
        validate_4 as "validate_4",
        validate_5 as "validate_5",
        validate_6 as "validate_6",
        validate_7 as "validate_7",
        validate_8 as "validate_8",
        validate_9 as "validate_9"
        FROM ${platSchema}.tee10 where product_id=#product_id#
    </select>

    <insert id="insert">
        INSERT INTO ${platSchema}.tee10 (
            product_id,
            company_name,
            made_country,
            sale_date,
            price,
            company_page,
            search,
            validate_0,
            validate_1,
            validate_2,
            validate_3,
            validate_4,
            validate_5,
            validate_6,
            validate_7,
            validate_8,
            validate_9
        ) VALUES (
            #product_id#,
            #company_name#,
            #made_country#,
            #sale_date#,
            #price#,
            #company_page#,
            #search#,
            #validate_0#,
            #validate_1#,

```

```

        #validate_2#,
        #validate_3#,
        #validate_4#,
        #validate_5#,
        #validate_6#,
        #validate_7#,
        #validate_8#,
        #validate_9#
    )
</insert>

<update id="update">
    UPDATE ${platSchema}.tee10 SET
        company_name = #company_name#,
        made_country = #made_country#,
        sale_date = #sale_date#,
        price = #price#,
        company_page = #company_page#,
        search = #search#,
        validate_0 = #validate_0#,
        validate_1 = #validate_1#,
        validate_2 = #validate_2#,
        validate_3 = #validate_3#,
        validate_4 = #validate_4#,
        validate_5 = #validate_5#,
        validate_6 = #validate_6#,
        validate_7 = #validate_7#,
        validate_8 = #validate_8#,
        validate_9 = #validate_9#
    WHERE product_id = #product_id#
</update>

<delete id="delete">
    DELETE FROM ${platSchema}.tee10
    WHERE product_id = #product_id#
</delete>

<select id="queryAlias" resultClass="java.util.HashMap">
    SELECT
        t1.product_id as "product_id",
        t1.company_name as "company_name",
        t1.made_country as "made_country",
        t1.price as "price",
        t2.ename as "validate_0",
        t2.cname as "validate_1",
        t2.node_type as "validate_2"
    FROM ${platSchema}.tee10 t1 inner join ${platSchema}.teedm06 t2
    on t1.made_country = t2.ename

    <isEmpty prepend=" AND " property="product_id">
        t1.product_id like ('%$product_id$%')
    </isEmpty>
    <isEmpty prepend=" AND " property="company_name">
        t1.company_name like ('%$company_name$%')
    </isEmpty>

    <!--配合前台排序-->

```

```

        <dynamic prepend="ORDER BY">
            <isEmpty property="orderBy">
                $orderBy$
            </isEmpty>
        </dynamic>
    </select>

</sqlMap>

```

3.5.3.1.1.3.4 domain 文件 EE10.java

EE10.java

```

package com.baosight.iplat4j.ee.domain;

import com.baosight.iplat4j.core.data.DaoEPBase;
import com.baosight.iplat4j.core.ei.EiColumn;

import java.util.HashMap;

public class EE10 extends DaoEPBase {
    public EE10() {
        initMetaData();
    }

    public void initMetaData() {
        EiColumn eiColumn = new EiColumn("product_id");
        eiColumn.setDescName("产品代号");
        eiColumn.setMaxLength(20);
        eiColumn.setNullable(false);
        eiColumn.setPrimaryKey(true);
        eiMetadata.addMeta(eiColumn);

        eiColumn = new EiColumn("company_name");
        eiColumn.setDescName("公司名称");
        eiColumn.setMaxLength(20);
        eiColumn.setNullable(true);

        eiColumn.set("locked", true);
        eiColumn.set("textField", "form");
        eiColumn.set("custom", 100);
        eiMetadata.addMeta(eiColumn);

        eiColumn = new EiColumn("made_country");
        eiColumn.setDescName("产地");
        eiColumn.setMaxLength(20);
        HashMap<String, Object> map = new HashMap<String, Object>();
        map.put("headerAlign", "center");
        eiColumn.setAttr(map);
        eiColumn.setAlign("center");
        eiColumn.setNullable(true);
        eiMetadata.addMeta(eiColumn);

        eiColumn = new EiColumn("sale_date");
        eiColumn.setDescName("上市日期");
        eiColumn.setNullable(true);
    }
}

```

```

eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("price");
eiColumn.setDescName("价格");
eiColumn.setType("N");
eiColumn.setNullable(true);
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("company_page");
eiColumn.setDescName("公司首页");
eiColumn.setMaxLength(255);
eiColumn.setWidth(150);
eiColumn.setNullable(true);
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("search");
eiColumn.setDescName("Google 查询");
eiColumn.setMaxLength(20);
eiColumn.setNullable(true);
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_0");
eiColumn.setDescName("验证字段 0");
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_1");
eiColumn.setDescName("验证字段 1");
eiColumn.set("sort", true);
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_2");
eiColumn.setDescName("验证字段 2");
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_3");
eiColumn.setDescName("验证字段 3");
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_4");
eiColumn.setDescName("验证字段 4");
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_5");
eiColumn.setDescName("验证字段 5");
eiColumn.setType("N");
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_6");
eiColumn.setDescName("验证字段 6");
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_7");
eiColumn.setDescName("验证字段 7");
eiMetadata.addMeta(eiColumn);

eiColumn = new EiColumn("validate_8");
eiColumn.setDescName("验证字段 8");

```

```

        eiMetadata.addMeta(eiColumn);

        eiColumn = new EiColumn("validate_9");
        eiColumn.setDescName("验证字段 9");
        eiMetadata.addMeta(eiColumn);
    }
}

```

页面展示的效果如下：

产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证
N701	Motorola...	China, Ger...	2017/11/0...	150122	EEE	N70		abc@baosi...	asdasdas				0
N7011	Motorola...	China, Ger...	2017/11/0...	150122	EEE	N70		abc@baosi...	asdasdas				0
N7012	Motorola...	China, Ger...	2017/11/0...	15012	EEE	N70		abc@baosi...	asdasdas				0
N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE	N70		abc@baosi...	asdasdas				0
111EE	111	USA	2017/11/0...	11		N7011							0
N1	Sony	China	2017/11/1...	0	Asia								0
ttt		China	2017/11/0...	0									0
N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	asdasdas				0
#251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	asdasdas				0
T6181	Motorola...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...	021-50801...	1			2000

3.5.3.1.1.3.5 基础功能

3.5.3.1.1.3.5.1 查询

“查询”在业务中是一个很基础操作。在页面中，通常需要两个 **EFRegion**，上面存放查询区，下面存放 **EFGrid**，现在我们在之前代码的基础上添加查询区。

EFGrid 示例

```

<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}" />
<EF:EFPage title="单表增删改查示例">
    <!-- 查询区 -->
    <EF:EFRegion id="inqu" title="查询条件">
        <div class="row">
            <EF:EFInput ename="inqu_status-0-product_id" cname="产品代号"/>
            <EF:EFInput ename="inqu_status-0-company_name" cname="公司名称"/>
        </div>
    </EF:EFRegion>

    <!-- Grid 区域 -->
    <EF:EFRegion id="result" title="记录集">
        <EF:EFGrid blockId="result">
        </EF:EFGrid>
    </EF:EFRegion>
</EF:EFPage>

```


查询区代码现在只是添加了输入框，用于输入查询的条件。除此之外，我们还需要一个查询按钮，将我们的查询条件提交到后端，然后更新我们 Grid 的数据，于是开发者需要给查询区域注册一个按钮。



当然直接在上面添加一个 Button 做查询是很简单的，不过不推荐这样做，因为这样很可能就失去了权限控制。详细请参考 [EDFA01 按钮注册说明](#)。

我们通过 EDFA01 给查询区域注册了一个 cname 为"查询"、ename 为"F1"的 Button。在实际的页面渲染结束后，按钮的在页面中的 id 是跟 ename 同名，显示名称跟 cname 同名，分别是"F1"和"查询"。

页面效果如下图：

在 JS 中完成查询功能：

EEDM6000.js

```
$(function () {
    $("#F1").on("click", function () {
        resultGrid.dataSource.page(1); // 点击查询按钮，从第 1 页开始查询
    });
});
```



"resultGrid"是什么？

平台为了方便开发人员对 Grid 进行操作，给开发人员提供了被注入的 Grid 的对象。这个 Grid 对象名称是由（EFGrid 中 BlockId + Grid）组成，可以在 JS 页面中直接调用，而不用去声明。这个对象是 kendo.ui.Grid 的实例，可以使用 kendoGrid 的原生方法和 iPlatUI 的扩展方法。具体请参考 [EFGrid](#) 文档中的 API 和 kendo.ui.Grid 相关文档。例如：像上面的 EFGrid 中的 blockId 是 result。所以这里我们能够使用 resultGrid 这个对象去操作 Grid。

resultGrid.dataSource.page(1)：这个是 kendo 提供的一个可以把查询区域里面的数据提交到后端 (ServiceEEDM6000) 的方法，提交到后端后，会调用 query 这个方法。

query 方法会对数据进行过滤、排序、并且把结果放在 EiInfo 里面，然后把这个 EiInfo 返回给前台。Grid 就能展示相关的数据。这个过程就是一次查询。

3.5.3.1.1.3.5.2 添加、删除、修改

想要实现这些功能，就需要对应的操作按钮。设置 needAuth="false"，显示 EFGGrid 的默认按钮。

EFGGrid 示例

```
...
<EF:EFRegion id="result" title="记录集">
  <EF:EFGGrid blockId="result" needAuth="false">
    </EF:EFGGrid>
  </EF:EFRegion>
</EF:EFPage>
```

页面效果如下所示：



至于平台是怎么是怎么把我们(增删改查)操作的数据定位到 ServiceEEDM6000 方法里面，请参考 [EFGGrid](#) 中 serviceName、queryMethod、insertMethod、updateMethod、deleteMethod 中属性说明。

3.5.3.1.1.3.5.3 翻页条位置

上面可以看到，分页条默认存在于 Grid 的右上角的位置：



除了可以存在于右上角位置，我们也可以将其放置于 Grid 的底部，修改 EEDM6000.jsp，在 Grid 中添加 pagerPosition="bottom"。

EEDM6000.jsp

```

<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}" />
<EF:EFPage title="单表增删改查示例">
    <!-- 查询区 -->
    <EF:EFRegion id="inqu" title="查询条件">
        <div class="row">
            <EF:EFInput ename="inqu_status-0-product_id" cname="产品代号"/>
            <EF:EFInput ename="inqu_status-0-company_name" cname="公司名称"/>
        </div>
    </EF:EFRegion>
    <EF:EFRegion id="result" title="记录集">
        <EF:EFGrid blockId="result" needAuth="false" pagerPosition="bottom">
        </EF:EFGrid>
    </EF:EFRegion>
</EF:EFPage>

```

效果如下：

☐	*产品代号	覆盖公司名	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证
☐	wewewe	Motorola...	GB, German...	2017/12/1...	150122	EEEE	N70		abc@baosi...	https://a...				1
☐	qqq	Motorola...	GB, German...	2017/11/0...	150122	EEEE	N70		baosight6...	https://a...				1
☐	1111	Motorola...	GB, German...	2017/11/0...	150122	EEEE	N70		baosight6...	https://a...				1
☐	N7021	Motorola...	GB, German...	2017/11/0...	150122	EEEE	N71		baosight6...	https://a...				1
☐	111EE	111	GB, German...	2017/11/0...	11		N7011		abc@baosi...	https://a...				1
☐	testlx	AAAAA	GB, German...	2017/12/1...	0				abc@baosi...					0
☐	N7022	Motorola...	GB, German...	2017/11/0...	150122	EEEE	N70		abc@baosi...	https://a...				1
☐	N70	Motorola...	GB, German...	2017/11/2...	150122	EEE	N70		abc@baosi...	https://a...				20
☐	8251	Motorola...	GB, German...	2017/11/1...	150122	EEE	N70		abc@baosi...	https://a...				1000
☐	SGH-I7161	Samsung	GB, German...	2007-04-06	10.14	http222wwwsam...	查询	1380000000...	abc@baosi...	fuyimin	021-50801...	1		2000

这样分页条就被放置于 Grid 的底部。

3.5.3.1.1.3.5.4 单列过滤 filterable

filterable 是平台提供给开发人员的一个便捷过滤界面，接收需要过滤条件并且把条件传递到 **queryMethod** 对应的后端接口，后端接收到条件，对数据进行过滤，返回需要的数据。

如果使用了前端的便捷过滤界面，而并没有在后端相应的接口中对数据进行过滤，是无法实现过滤的。

修改 EEDM6000.jsp:

EEDM6000.jsp

```

<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}" />
<EF:EFPage title="单表增删改查示例">

```

```

<!--查询区-->
<EF:EFRegion id="inqu" title="查询条件">
  <div class="row">
    <EF:EFInput ename="inqu_status-0-product_id" cname="产品代号"/>
    <EF:EFInput ename="inqu_status-0-company_name" cname="公司名称"/>
  </div>
</EF:EFRegion>
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" needAuth="false" pagerPosition="bottom"
filterable="true">
    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFPage>

```

这样我们就开启了 **filterable** 的功能，效果如下：

记录集													
新增 保存 取消 删除													
☐	*产品代号	覆盖公...	产地	上市日期	价格	公司首页	Google...	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5
☐	wewewe	Motorola...	GB, German...	2017/12/1...	150122	EEE1	N70		abc@baosi...	https://s...			1
☐	qqq	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N70		baosight6...	https://s...			1
☐	1111	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N70		baosight6...	https://s...			1
☐	N7021	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N71		baosight6...	https://s...			1
☐	111EE	111	GB, German...	2017/11/0...	11		N7011		abc@baosi...	https://s...			1
☐	test1x	AAAAA	GB, German...	2017/12/1...	0				abc@baosi...				0
☐	N7022	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...			1
☐	N70	Motorola...	GB, German...	2017/11/2...	150122	EEE	N70		abc@baosi...	https://s...			20
☐	8251	Motorola...	GB, German...	2017/11/1...	150122	EEE			abc@baosi...	https://s...			1000
☐	SGH-i7161	Samsung	GB, German...	2007-04-06	10.14	http222wwwsamsu...	查询	138000000...	abc@baosi...	fuyimin	021-50801...	1	2000

当用户点击  小图标的时候，会弹出快捷过滤界面：

	覆盖公...	产地	上市日期
☐	Motorola...		2/1...
☐	Motorola...		1/0...
☐	Motorola...		1/0...
☐	Motorola...		1/0...
☐	111		1/0...
☐	AAAAA	GB, German...	2017/12/1...

显示符合以下条件的行

包含

11

过滤 清除

输入需要过滤的字符，点击过滤，平台会把上面的字符和列信息作为条件传 入到 **queryMethod** 对应的后端方法里面。在后端，开发人员根据条件进行过滤， 然后返回过滤后的数据即可。

EFGGrid 中 **filterable = "true"**会对所有列开启 **便捷过滤界面**功能，实际中，我们 可能不需要把所有列进行过滤。那么我们可以通过 **JS** 的配置去关闭某些列 **filterable** 功能。

添加 EEDM6000.js:

EEDM6000.js

```
$(function () {
```

```

IPLATUI.EFGrid = {
  "result": {
    columns: [
      { // 覆盖 EFColumn 中的 company_name
        field: "company_name",
        filterable: false
      }
    ]
  }
};

```

上面我们关闭了"company_name" 的 filterable 功能，效果如下：

产品代号	公司名称	产地	上市日期	价格	公司首页	Google...	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证
wewewe	Motorola...	GB, German...	2017/12/1...	150122	EEE1	N70	abc@baosi...	https://s...					1
qqq	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N70	baosight6...	https://s...					1
1111	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N70	baosight6...	https://s...					1
N7021	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N71	baosight6...	https://s...					1
111EE	111	GB, German...	2017/11/0...	11		N7011	abc@baosi...	https://s...					1
testlk	AAAAA	GB, German...	2017/12/1...	0			abc@baosi...						0
N7022	Motorola...	GB, German...	2017/11/0...	150122	EEE1	N70	abc@baosi...	https://s...					1
N70	Motorola...	GB, German...	2017/11/2...	150122	EEE	N70	abc@baosi...	https://s...					20
8251	Motorola...	GB, German...	2017/11/1...	150122	EEE	N70	abc@baosi...	https://s...					1000
SGH-17161	Samsung	GB, German...	2007-04-06	10.14	http222wwwsamsu...	查询	138000000...	abc@baosi...	fuyimin	021-50801...	1		2000

3.5.3.1.1.3.6 事件

EFGrid 事件补充说明：

针对 [EFGrid](#) 中的“升级指南”进行说明：

- column 中 template 属性，是可以配置模版。也是可以配置函数，当配置函数时候，return 的值作为这列的值。在配置模版的时候，需要注意，不要出现单个#这样的字符，不然模版解析时候，会出错。比如 **template: "#:company_name#"**。这样会造成模版解析出错。
- dataBound 与 dataBinding。autoBind 为 false 情况下，首次不会触发 dataBound 和 dataBinding 事件。
- onCheckRow, onRowClick, onCheckAllRows 事件关系
 - onCheckAllRows: 由全选复选框进行触发，同时也会触发次页面中所有的 row 选中事件 onCheckRow（以前选中也会重新触发一次）
 - onRowClick: 点击 Grid 中一行进行触发，同时也会触发次行的 onCheckRow 事件。只能让 row 从未选中到选中，不能从选中返回未选中。选中后能进行编辑
 - onCheckRow: 通过点击每一列前面的复选框触发，也可以通过 Grid API 中 setCheckedRows 方法进行触发
- beforeAdd 与 afterAdd 事件 beforeAdd 是在没有生成新增行时候触发，可以用来判断当前行需要不需要增加（阻止新增可以调用 **e.preventDefault()**）afterAdd 事件是在新增结束后，触发，可以通过这个事件来修改当前行的数据信息。（现没有提供当前新增行的信息）

5. **onSave** 事件取消提交可以使用 **grid.cancelchanges** 方法，处理的事件是保存按钮的 **click** 事件，暂时不清楚是新增，修改，还是删除操作提交到后端
6. **beforeEdit, edit, afterEdit, editor** 事件当用户开始编辑 Grid 中某个 Cell 时候，事件运行的步骤是 **beforeEdit => column.editor => edit => afterEdit**。
beforeEdit 可以判断当前 Cell 是否需要编辑（阻止编辑可以调用 **e.preventDefault()**）**column.editor** 可以用于处理下拉，日期，以及自定义弹出框。**edit** 也是可以用于处理下拉，日期，自定义弹出框。（不过通过测试，数据没有能够绑定到 Grid 中，例如：都是使用 JS 创建的下拉选择框，**editor** 里面数据能够回填到 **grid** 中，而 **edit** 不能）下面是针对 **EEDM6000** 这个页面进行的对比代码。

edit 代码

```
edit: function(e) {
    var container = e.container;
    var input = $('<input />');
    input.attr("name", "company_name");
    input.appendTo(container);

    input.kendoDropDownList({
        dataSource: {
            data: ["One", "Two"]
        }
    });
    // console.log("edit: " + i++);
},
```

editor

```
field: "company_name",
title: "覆盖公司名",
headerTemplate: "<span style='color: red'>覆盖公司名</span>",
editor: function (container,options) {
    var input = $('<input />');
    input.attr("name", options.field);
    input.appendTo(container);
    input.kendoDropDownList({
        dataSource: {
            data: ["One", "Two"]
        }
    });
},
width: 200
```

这里发现点击“company_name”这列时候，**editor** 中下拉框选中的数据能够回填到 **Grid** 中，**edit** 却不能回填到 **grid** 中。

事件示例

这里是基于 **EEDM6000** 进行扩展说明：

- **loadComplete** 事件

loadComplete 是表示此 Grid 完全加载后，提供给开发人员的回调，此方法只执行一次。方法里面提供给开发一个回调参数 grid。grid 是此 grid 的 kendoGrid 的对象实例。

loadcomplete

```
IPLATUI.EFGrid = {
  "result": {
    loadComplete: function (grid /* kendo.ui.Grid */) { // EFGrid 渲染成功的回调
      事件,
      // grid 为 kendoGrid 的实例对象，如何使用 Grid 请参考: kendoGrid
      // grid.element 为此 Grid 的 jquery 对象，开发人员可以通过此对象，进行相
      关 dom 操作处理
      console.log(grid);
      // 举例：监听每一行 row 的点击事件
      grid.element.on("click", "tr[role=row]", function (e) {
        alert(e)
      })
    }
  }
}
```

● page 事件

page 事件是表示 Grid 中页面翻页事件。

page

```
IPLATUI.EFGrid = {
  "result": {
    /**
     * 翻页事件，点翻页按钮，输入页数跳转时触发
     * @param e 事件对象
     * e.sender Grid 对象
     * e.page 期望翻到的页数
     */
    page: function (e) {
      e.preventDefault(); // 禁止翻页
      // console.log("翻页" + e.page);
    }
  }
}
```

开发人员可以通过 e.preventDefault 阻止翻页事件，e.page 能够获取期望翻到的页面 (用户交互时候指定的页面)。

● beforeAdd 事件

beforeAdd 事件是指 Grid 中新增行之前触发。

beforeAdd

```
IPLATUI.EFGrid = {
  "result": {
    /**
```

```

    * EFGGrid 新增行之前触发的事件，可以根据业务逻辑控制是否进行新增
    * @param e 事件对象
    * e.sender Grid 对象
    */
    beforeAdd: function (e) {
        var logic = false;
        if (logic) { // 通过业务逻辑判断，控制是否进行新增
            // 不会再新增
            e.preventDefault();
        }
        console.log("beforeAdd");
    }
}

```

beforeAdd 是 Grid 在新添加 row 之前触发，开发人员人员可以通过逻辑判断 **新增此 row** 是否符合自己的业务逻辑，如果不符合自己的业务逻辑，可以通过 **e.preventDefault()** 阻止后续事件的发生。

● onAdd 事件

EFGGrid 新增行时触发的事件，此时数据行 tr 元素还未渲染。

onAdd

```

IPLATUI.EFGGrid = {
    "result": {
        /**
        * EFGGrid 新增行时触发的事件，此时数据行 tr 元素还未渲染
        * @param e 事件对象
        * e.sender Grid 对象
        * e.preventDefault 不会触发 dataBound，不渲染 tr
        * e.index 插入的行号
        * e.items 将要插入的数据行数组([Model,Model, ...])
        */
        onAdd: function (e) {
            //新增的数据集合，
            // 单个新增的情况下，获取新增的 model 是 e.items[0]
            // 批量新增的情况下，可以通过 e.items 来获得新增的集合
            console.log(e.items);
            //给新增行添加默认数据，给 xxx 设置“xxxx”值
            var model = e.items[0];
            model['xxx'] = "xxxx" ;
        }
    }
}

```

● afterAdd 事件

EFGGrid 新增后触发的事件，此时数据行 tr 元素已经渲染。

afterAdd

```

IPLATUI.EFGGrid = {
    "result": {

```



```

    /**
    * EFGrid 新增后触发的事件，此时数据行 tr 元素已经渲染
    * @param e 事件对象
    * e.sender Grid 对象
    * e.preventDefault 不会触发 dataBound，不渲染 tr
    * e.index 插入的行号
    * e.items 将要插入的数据行数组([Model,Model, ...])
    */
    afterAdd: function (e) {
        $.each(e.items, function (index, item) {
            // var trIndex = index + e.index;
            // 定制新增行 tr 的背景色
            e.sender.element.find("tr[data-uid=" + item.uid + "]").css({
                background: "green",
                color: "yellow"
            });
        });
    });
}
}

```

新增 row 的事件执行流程：

beforeAdd =》 **dataBinding** =》 **onAdd** =》 **dataBound** =》 **afterAdd**。

1. beforeAdd 通常主要是判断能否进行新增
2. onAdd 可以用来设置 model 的默认数据，或者修改复制新增中 model 的值
3. afterAdd 主要用来处理 row 新增完成后，对 dom 的处理

● beforeEdit 事件

勾选行后，点击单元格准备编辑时的事件。

beforeEdit

```

IPLATUI.EFGrid = {
    "result": {
        /**
        * EFGrid 新增后触发的事件，此时数据行 tr 元素已经渲染
        * @param e 事件对象
        * e.sender Grid 对象
        * e.preventDefault 不会触发 dataBound，不渲染 tr
        * e.index 插入的行号
        * e.items 将要插入的数据行数组([Model,Model, ...])
        */
        beforeEdit: function (e) {
            $.each(e.items, function (index, item) {
                // var trIndex = index + e.index;
                // 定制新增行 tr 的背景色
                e.sender.element.find("tr[data-uid=" + item.uid + "]").css({
                    background: "green",
                    color: "yellow"
                });
            });
        });
    }
}

```

```
}
}
```

3.5.3.1.2 EEDM6001-单页多表操作

3.5.3.1.2.1 页面功能简介

本示例页面用于说明，当一个页面中出现两个或者多个 EFGrid 时，如何使用 queryMethod, insertMethod, updateMethod, deleteMethod 属性区分 EFGrid 对应的后台操作函数。讲解了：

1. [EFGrid 操作按钮注册规则](#)
2. needAuth 属性的作用
3. Kendo MVVM 双向绑定的基础使用方式。
4. autoFit 的基础功能

3.5.3.1.2.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。开发人员想实际开发本页面所提供的功能，需要对页面进行注册和开发相应的服务，详情请参考[开发一个单表增删改查页面](#)。

3.5.3.1.2.3 页面开发步骤

3.5.3.1.2.3.1 JSP 页面开发

当一个 JSP 页面中出现多个 EFGrid 时，首个 EFGrid 可使用默认配置，其余的 EFGrid 可通过：

- queryMethod 指定查询方法，
- insertMethod 指定插入方法，
- updateMethod 指定更新方法，
- deleteMethod 指定删除方法。

□使用注意

EEDM6001

```
// 在页面的初始方法 initLoad 中，注意要指明每个 EFGrid 对应的 EiBlock 信息。
// 也就是说 initLoad 返回的 EiInfo 中要包含多个 EiBlock，依次对应 EFGrid 中的 blockId。

public EiInfo initLoad(EiInfo inInfo) {
    inInfo.addBlock(EiConstant.resultBlock); // EiConstant.resultBlock => "result"
    inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM05().eiMetadata);

    inInfo.addBlock("blockA");
    inInfo.getBlock("blockA").addBlockMeta(new EE10().eiMetadata);
}
```

```

    return inInfo;
}

```

JSP 代码示例:

JSP 示例

```

<EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" autoDraw="yes" autoFit="true">
        </EF:EFGrid>
    </EF:EFRegion>

    <EF:EFRegion id="regionA" title="另一个 EFGrid">
        <EF:EFGrid blockId="blockA" needAuth="false" queryMethod="queryA">
            </EF:EFGrid>
        </EF:EFRegion>
    </EF:EFRegion>

```

needAuth 的使用说明:

needAuth的使用

□ 第二个EFGrid使用了属性needAuth="false"。
Grid默认的操作按钮是需要EDFA01注册授权后才可以显示的。开发中，可以使用needAuth="false"跳过按钮注册授权的控制，在Grid上方显示默认的操作按钮。所以第一个EFGrid只显示EDFA01中注册过的按钮，第二个EFGrid则会显示4个默认的按钮。

3.5.3.1.2.3.1.1 页面效果

页面效果如下所示，needAuth="true":

记录集											
产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4
1236	Motorola	Canada	2007-07-12	2			abc@baosight.com			-12345678901234	0
8251	Motorola	china	2007-03-17	1400	EL23	查询	13800000000	aaa@123.com	label	021-50801155	1
T6181	Motorola	china	2007-04-06	3550	时间访问量统计	查询	13800000000	aaa@123.com		021-50801155	1
5GH-17161	Samsung	Korea	2007-04-06	10.14	http://www.samsung...	查询	13800000000	aaa@123.com	fuyimin	021-50801155	1
T6281	SonyEricsson	Japan	2007-04-06	30	http://www.sonyeric...	查询	13800000000	aaa@123.com	label	021-50801155	1
N701	NOKIA	Sweden	2007-04-06	0.12		search		abc@baosight.com			0
88501	SonyEricsson	Korea	2017010112342356	0.12	http://platform.bao...	1		abc@baosight.com			0
1112	Motorola	China	2007-07-06	11	EWFM07			abc@baosight.com	11	-12345678901234	0
5891	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13800000000	aaa@123.com	label	021-50801155	1
8825	SonyEricsson	Japan	2007/03/10	20000		search		abc@baosight.com	11	021-50801155	0

needAuth="false":

另一个EFGrid			
节点ID	节点显示名	父节点ID	是否为叶子节点
1	1	1	1

autoFit 的使用说明:

autoFit的使用

□第一个EFGGrid使用了属性autoFit="true", 此属性用来调整grid里面信息不能完全展示的问题。autoFit属性设置为true的时候, grid的单元格宽度默认取此列里面最宽的单元格的宽度。需要注意的是, 如果单元格已经设置了宽度, autoFit属性是无效的。

3.5.3.1.2.3.1.2 页面效果

页面效果如下所示, autoFit="false":

记录集

* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证
1236	Motorola	Canada	2007-07-12	2				abc@baosig...		-123456789...		
8251	Motorola	china	2007-03-17	1400	EL23	查询	13800000000	aaa@123.com	label	021-508011...	1	
T6181	Motorola	china	2007-04-06	3550	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
SGH-I7161	Samsung	Korea	2007-04-06	10.14	http://www.samsun...	查询	13800000000	aaa@123.com	fuyimin	021-508011...	1	
T6281	SonyEricss...	Japan	2007-04-06	30	http://www.sonyeric...	查询	13800000000	aaa@123.com	label	021-508011...	1	
N701	NOKIA	Sweden	2007-04-06	0.12		search		abc@baosig...				
88501	SonyEricss...	Korea	2017010112...	0.12	http://platform.bao...	1		abc@baosig...				
1112	Motorola	China	2007-07-06	11	EWFM07			abc@baosig...	11	-123456789...		
5891	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13800000000	aaa@123.com	label	021-508011...	1	
8825	SonyEricss...	Japan	2007/03/10	20000		search		abc@baosig...	11	021-508011...		

10 每页

autoFit="true":

记录集

* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证
1236	Motorola	Canada	2007-07-12	2				abc@baosight.com		-12345678901234		0
8251	Motorola	china	2007-03-17	1400	EL23	查询	13800000000	aaa@123.com	label	021-50801155	1	2000
T6181	Motorola	china	2007-04-06	3550	时间段访问量统计	查询	13800000000	aaa@123.com		021-50801155	1	2000
SGH-I7161	Samsung	Korea	2007-04-06	10.14	http://www.samsun...	查询	13800000000	aaa@123.com	fuyimin	021-50801155	1	2000
T6281	SonyEricsson	Japan	2007-04-06	30	http://www.sonyeric...	查询	13800000000	aaa@123.com	label	021-50801155	1	2000
N701	NOKIA	Sweden	2007-04-06	0.12		search		abc@baosight.com				0
88501	SonyEricsson	Korea	2017010112342356	0.12	http://platform.bao...	1		abc@baosight.com				0
1112	Motorola	China	2007-07-06	11	EWFM07			abc@baosight.com	11	-12345678901234		0
5891	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13800000000	aaa@123.com	label	021-50801155	1	2000
8825	SonyEricsson	Japan	2007/03/10	20000		search		abc@baosight.com	11	021-50801155		0

10 每页

3.5.3.1.2.3.2 Service 服务开发

下面的代码片段仅仅用于演示如何在后台添加 id 为 blockA 的 EiBlock, 禁止在项目中复制使用。

JAVA 代码示例

```
public EiInfo queryA(EiInfo inInfo) { // 仅仅用于演示如何在后台添加 id 为 blockA 的
    EiBlock, 禁止在项目中使用的 EiBlock
    EiBlock blockA = inInfo.getBlock("blockA");
    if (blockA!=null) {
        blockA.getBlockMeta().setBlockId("result");
        inInfo.addBlock(blockA);
    }

    EiInfo outInfo = query(inInfo, "EDFA00.query", new TEDFA00());
    EiBlock block = outInfo.getBlock("result");

    block.getBlockMeta().setBlockId("blockA");
    outInfo.addBlock(block);

    return outInfo;
}
```

```
}

```

3.5.3.1.2.3.3 初步使用双向绑定

通过双向绑定，可以将 Grid 中的一行数据，根据列名将数据绑定到对应的 EFInput 输入框中。编辑 EFInput 输入框的值，EFGGrid 中的行数据也会自动更新（未提交到后台）。编辑 EFGGrid 的行数据，EFInput 的值也会自动更新。

3.5.3.1.2.3.4 设置 EFInput 的动态属性

（data-bind）实现数据绑定。data-bind="value: node_ename"说明 EFInput 绑定了 EFGGrid 中的 node_ename 列。

JSP 示例

```
<div id="detail" style="margin-top: 20px;" class="row">
    <h3 class="text-danger push-15 push-15-1">和 grid 中选中的行进行双向绑定</h3>
    <EF:EFInput ename="node_ename" cname="节点英文名" data-bind="value: node_ename" />
    <EF:EFInput ename="node_cname" cname="节点中文名" data-bind="value: node_cname" />
</div>

```

点击 EFGGrid 的数据行时，在 JS 中进行双向绑定。

JS 示例

```
IPLATUI.EFGGrid = {
    "blockA": {
        change: function(e) {
            var selectedRow = this.select(); // 获取当前点击的行，kendo 的原生方法
            var item = this.dataItem(selectedRow); // 获取行数据
            kendo.bind($("#detail"), item); // 双向绑定
        }
    }
};

```

效果如下：



3.5.3.1.3 EEDM6002-设置表格的列类型

3.5.3.1.3.1 页面功能简介

本示例页面用于说明：

- 如何在 EFGrid 中使用 [EEDM6002-设置表格的列类型](#)，[EEDM6002-设置表格的列类型](#)
- 讲解了 autoDraw 属性的使用规则；
- 标签：[EEDM6002-设置表格的列类型](#)，[EEDM6002-设置表格的列类型](#)，[EEDM6002-设置表格的列类型](#)的基础用法。
- [EEDM6002-设置表格的列类型](#)（列头和单元格的模版）
- [EEDM6002-设置表格的列类型](#)（数字列的格式化，日期的格式化）
- Grid 渲染前的事件：dataBinding

3.5.3.1.3.2配置和 API 详情

参考 [EFGrid 表格](#)中的描述。

3.5.3.1.3.3页面开发步骤

3.5.3.1.3.3.1 属性：autoDraw

使用背景：在使用 EFGrid 开发的过程中，我们可能会遇到类似“不需要显示 EiBlock 中全部列信息”或者“需要对 EiBlock 中的某几列进行定制”的情况，此时应该使用 EFGrid 标签的 autoDraw 属性。

属性简介：

- autoDraw 有四个可选值，默认是"yes"；
- autoDraw="yes:" 只渲染 EiBlock 中的列，忽略所有的 EFColumn Tag；
- autoDraw="override": 要求 EFGrid 必须有 EFColumn 子标签。只渲染自定义列 EFColumn Tag 中的列，覆盖 EiBlock；
- autoDraw="no": 要求 EFGrid 必须有 EFColumn 等子标签。并且只渲染自定义列 EFColumn Tag 中的列，忽略 EiBlock；
- autoDraw="mixed": 渲染 EiBlock 中的列，根据 EFColumn Tag 的配置，灵活渲染：
 1. EFColumn Tag 中的 ename 如果存在于 EiBlock 中，则使用 EFColumn 的配置覆盖 EiBlock 中的列设置。
 2. EFColumn Tag 中的 ename 如果不存在于 EiBlock 中，则将 EFColumn 追加到最后一列。

autoDraw使用注意事项

在JS中配置的

IPLATUI.EFGrid中的columns的信息，优先级最高，无论autoDraw配置为何值，JS中的配置都会覆盖EiBlock和EFColumn标签中的列配置信息。

3.5.3.1.3.3.2 列的多种类型

在开发过程中，我们不仅仅会使用普通的数据列，还会经常使用到日期列，下拉列，弹出列等多种列类型。下面的示例代码用于说明，如何使用 EFGrid 的多种列类型。

3.5.3.1.3.3.2.1 EFColumn 标签（普通数据列）

EFColumn 是一个普通的列标签，详情请参考 [EEDM6002-设置表格的列类型](#)

JSP 页面开发

JSP 示例

```
<c:set var="pid" value="product_id"/>

<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" needAuth="false" autoDraw="override"> //
  autoDraw="override": 要求 EFGrid 必须有 EFColumn 子标签。
  <EF:EFColumn ename="{pid}" locked="true" cname="产品编号"/>          <%-- 动态设置
  的列名 --%>
  <EF:EFColumn ename="company_name" locked="true" editType="textarea" cname="公司名称"
  "/>
  </EF:EFGrid>
</EF:EFRegion>
```

分析：autoDraw="override"，因此只渲染自定义列 EFColumn Tag 中的列，覆盖 EiBlock；needAuth 属性请看 [EEDM6001](#)；locked 属性请看 [EEDM6010](#)。

3.5.3.1.3.3.2.2 EFComboColumn 标签（下拉数据列）

标签简介：EFComboColumn 是 EFGrid 的子 Tag，用于处理 Grid 中的下拉列的信息，和表单中的 **EFSelect** 的功能类似；

属性简介：valueField：真实值对应的列名，提交到后台，必填参数；
textField：显示值对应的列名，默认和 valueField 相同，可选参数；

选项来源：

1. 使用子 Tag: EFOption（用于一条一条 option 的写入），EFOptions（批量 option 的写入），EFCodeOption（指定小代码的 option 批量写入），EFTableOption（基于数据库的下拉选择）；
2. 后台的 EiBlock（设置 blockName），
3. request.setAttribute("sourceName", list)

备注

不能同时设置 blockName，sourceName，如果同时设置，sourceName 的优先级高于 blockName；

当 blockName 和 sourceName 都没有设置时，处理子 Tag 中的数据（来源1）

3.5.3.1.3.3.2.2.1 与子 Tag 标签搭配使用

与 EFOption, EFOptions, EFCodeOption, EFTableOption 标签搭配使用。

JSP 页面开发

子 Tag

```
<EF:EFComboColumn ename="company_page" cname="小代码选项"
    columnTemplate="#=textField#-#=valueField#"
itemTemplate="#=textField#-#=valueField#" // columnTemplate: 选完选项后该列单元格的显示模版。 itemTemplate: 下拉选项的模版
    textField="textField" valueField="valueField">

    <EF:EFOption label="全部" value="all"/>

    <EF:EFOptions blockId="country" valueField="countryEname"
textField="countryCname"/>

    <EF:EFCodeOption codeName="iplat.ed.resType"/>
    <EF:EFCodeOption codeName="iplat.codeStyle"/>

    <EF:EFTableOption tableName="TEDFA00" textField="FORM_ENAME"
valueField="FORM_CNAME"
        condition="FORM_ENAME = 'EEDM6002'"/>
</EF:EFComboColumn>
```

注意

1.

当且仅当下拉选项来源于子标签时，**EFComboColumn**标签的属性：textField="textField"

valueField="valueField"是必然的，不能修改。itemTemplate或columnTemplate，也均固定的由"textField"和"valueField"组合而成，即

columnTemplate="#=textField#-#=valueField#"

itemTemplate="#=textField#-#=valueField#"。

因为在封装过程中，EFOption/EFOptions/EFCodeOption/EFTableOption的textField和valueField的值已经固化为"textField"和"valueField"，如若根据textField和valueField实际的取值来搭配，无法渲染下拉选项。

2.子标签若有属性textField和valueField，直接根据实际情况取值即可，无需定为textField="textField" valueField="valueField"。如上EFOptions标签的属性valueField="countryEname" textField="countryCname"。

A. 与 EFOption 标签搭配使用

属性简介：

- label: 必填参数，option 的显示值
- value: 必填参数，option 的真实值

textField	valueField	说明
label	value	EFOption标签定义的下拉选项，存储在IPLAT.EFOptions数组中，每个选项的textField取label值，valueField取value值； ({textField: "\${label}", valueField: "\${value}"})

此处的 textField 取属性 label 的值“全部”，valueField 取属性 value 的值“all”，下拉显示模版和选择后的列单元格的显示模版都为：“全部-all”。

B. 与 EFOptions 标签搭配使用

EFOptions 通过 blockId 配置下拉选项。blockId 指后台配置的数据块对应的 Id。

Service 服务开发

ServiceEEDM6002

```
public EiInfo query(EiInfo inInfo) {
    EiInfo outInfo = super.query(inInfo, "EE10.query", new EE10());

    // 额外的 EiBlock country
    outInfo.addBlock(MockDataUtil.getCountryBlock("country", 6));

    return outInfo;
}
```

blockId 为 country，共两列：countryEname，countryCname，共六行数据，如下图所示：



此处的 textField 为 countryCname，valueField 为 countryEname，下拉显示模版和选择后的列单元格的显示模版如：“中国-China”，“美国-USA”等。

C. 与 EFCodeOption 标签搭配使用

调用对应小代码（小代码的 codeName 一定要写正确）



在 EDCM01- 代码项维护中有代码分类编号

代码项维护 > EDCM01

查询条件: 项目英文名: IPLAT4J, 代码项编号: , 代码分类编号:

记录集:

*项目英文名	*代码分类编号	*代码项编号	代码项中文名称	代码项英文名称	*状态	顺序号	备注	创建人	创建时间	修改人
IPLAT4J	iplat.codeHierarchy	system	系统类	00	显示			admin	2012-02-07 07...	
IPLAT4J	iplat.codeHierarchy	headquarters	母公司层	05	显示			admin	2012-02-07 07...	
IPLAT4J	iplat.codeHierarchy	sub	子公司层	10	显示			admin	2012-02-07 07...	
IPLAT4J	iplat.codeHierarchy	branch	分公司层	20	显示			admin	2012-02-07 07...	
IPLAT4J	iplat.codeHierarchy	outer	外部代码	99	显示			admin	2012-02-07 07...	
IPLAT4J	iplat.codeHierarchy	test1	测试类1		不显示			admin	2017-09-22 10...	admin
IPLAT4J	iplat.codeStyle	workflow	流程类	1	显示			admin	2012-02-07 07...	
IPLAT4J	iplat.codeStyle	caculate	统计类	2	显示			admin	2012-02-07 07...	
IPLAT4J	iplat.ed.buttonlay...	2	只显示图标		显示			hubing	2013-05-04 22...	
IPLAT4J	iplat.ed.buttonlay...	1	只显示文字		显示			hubing	2013-05-04 22...	

代码项维护 > EDCM01

查询条件: 项目英文名: IPLAT4J, 代码项编号: , 代码分类编号: codeStyle

记录集:

*项目英文名	*代码分类编号	*代码项编号	代码项中文名称	代码项英文名称	*状态	顺序号	备注	创建人	创建时间	修改人
IPLAT4J	iplat.codeStyle	workflow	流程类	1	显示			admin	2012-02-07 07...	
IPLAT4J	iplat.codeStyle	caculate	统计类	2	显示			admin	2012-02-07 07...	

EFCodeOption 标签中的 textField 和 valueField 可不配置，默认 textField 取 label，valueField 取 value。而 label 默认取 ITEM_CNAME（代码项中文名称），value 取 ITEM_CODE（代码项编号）。

所以此处的 textField 为代码项中文名称，valueField 为代码项编号，下拉显示模版和选择后的列单元格的显示模版如：统计类-caculate。

D. 与 EFTableOption 标签搭配使用

EFTableOption 标签可以将数据库中指定表（TEDFA00）的指定字段提取出来，通过 condition 属性可筛选所需选项。

若配置 tableName，则 valueField 和 textField 也必须配置，tableName 要指定到数据库。

此处的 textField 为 FORM_ENAME，valueField 为 FORM_CNAME，下拉显示模版和选择后的列单元格的显示模版为：EEDM6002-Grid 列类型示例。

页面效果如下所示：

子Tag选项

全部-all

美国-USA

英国-GB

德国-German

法国-France

俄罗斯-Rus

GIT-GIT

SVN-SVN

统计类-caculate

流程类-workflow

3.5.3.1.3.3.2.2.2 设置 blockName JSP 页面开发

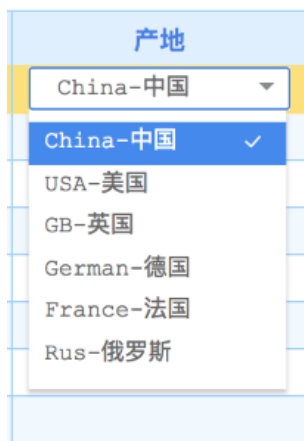
blockName

```
<%-- 使用 EiBlock 作为下拉数据源 --%>
<EF:EFComboColumn ename="made_country_combo" cname="产地" blockName="country"
    columnTemplate="#=countryEname#-#=countryCname#"
    itemTemplate="#=countryEname#-#=countryCname#"
    textField="countryCname" valueField="countryEname"/>
```

使用EiBlock作为下拉数据源时，EFComboColumn标签的属性：textField和valueField可以根据实际的取值来搭配，无需定为textField="textField" valueField="valueField"，如上：textField="countryCname" valueField="countryEname"即可。

itemTemplate和columnTemplate可以根据textField和valueField实际的取值来搭配。

页面效果如下所示：



3.5.3.1.3.3.2.2.3 使用 List 下拉数据源

首先，在 JSP 中定义一个 List，作为下拉列中的下拉选项来源。

JSP 上定义的 List

```
<%
    List<Map> customList = new ArrayList<>();

    for (int i = 0; i < 8; i++) {
        Map<String, String> map = new HashMap<>();
        map.put("name", "item" + i);
        map.put("desc", "选项" + i);
        customList.add(map);
    }

    request.setAttribute("list", customList);
%>
```

JSP 页面开发

JSP 示例

```
<%-- 使用 List<Map>作为下拉数据源 --%>
<EF:EFComboColumn ename="items" cname="List 选项" itemTemplate="#=name#-#=desc#"
    required="true"
    sourceName="list" textField="desc" valueField="name" />
```

分析：在 JSP 页面中自定义 List 数据，设置到 request 对象的属性中。再通过 sourceName 使用 List 下拉数据源（sourceName 指的是 request 区中设置的 list 属性值，textField 和 valueField 分别指该数据源的显示列名和值列名）【静态的数据选项建议使用 EFOption，在 JSP 页面定义 List 仅作参考，应该在后台设置 List，然后放在 request 对象中】

页面效果如下所示：



3.5.3.1.3.3.2.3 EFMultiSelectColumn 标签（多选下拉数据列）

标签简介：[EEDM6002-设置表格的列类型](#)

使用说明：EFMultiSelectColumn 与 EFComboColumn 两者的属性基本一致，区别在于 EFMultiSelectColumn 支持多个选中，EFComboColumn 只能支持每次选中一个。

JSP 页面开发

多选下拉列

```
<EF:EFMultiSelectColumn ename="made_country_multi" cname="产地"
    columnTemplate="#=countryEname#-#=countryCname#"
    blockName="country" textField="countryCname"
    valueField="countryEname" copy="true"> // blockName:下拉数据来源于 EiBlock, block 的数据由后台进行添加。
    <EF:EFOption label="全部 111" value="all"/>
    <EF:EFOption label="test" value="2"/>
</EF:EFMultiSelectColumn>
```

页面效果如下所示：

产地
全部111 × 美国 × 德国 ×
China-中国,all-全部111
adsffa
GB-英国,German-德国
GB-英国,German-德国,France-法国
GB-英国,German-德国,France-法国
america
GB-英国,German-德国,France-法国
1231

3.5.3.1.3.3.2.4 EFColumn 标签(日期时间列)

使用方法：配置 **editType** 属性值，在编辑单元格时，使用日期控件(“date”)或者日期时间控件(“datetime”)

动态属性：**interval** 动态属性可指定日期时间控件的时间间隔(单位:分钟)

如果存在后端时间列的格式与前端展示格式不一致的时候，请注意使用displayType这个属性值。

displayType(date|datetime)会对后端传入的时间列进行前端自定义展示。

展示是按照dateFormat格式进行展示的。与此同时，开发人员需要配置parseFormats第一个值为后端时间显示的格式。

属性简介：

1. **parseFormats** 表示除了标准格式（editType="date"时是"yyyy-MM-dd"; editType="datetime"时是"yyyy-MM-dd HH:mm:ss"）之外还可以解析的日期格式，此属性主要是解决数据库中已经存储了非标准格式（例如：时间戳）的日期数据，在编辑列的时候，可以正确地弹出日期控件。【注意：毫秒的通配符是 fff，不是 SSS】
2. **dateFormat** 属性用于指定编辑或新增状态下的时间格式，可以将标准格式 && **parseFormats** 中定义的时间格式数据，解析成 **dateFormat** 格式并传到后台。

JSP 页面开发

日期时间列

```
<%-- 设置日期列时间的间隔为 15 分钟 --%>
<EF:EFColumn ename="sale_date" cname="时刻" required="true" editType="datetime"
interval="15"
parseFormats="[' yyyy/MM/dd HH:mm:ss']" dateFormat="yyyyMMdd HH:mm:ss"
displayType="datetime"/>
<EF:EFColumn ename="on_date" cname="上市日期" required="true" editType="date"
displayType="date">
```

```
dateFormat="yyyyMMdd"/>
```

页面效果如下所示：

* 时刻	* 上市日期
20180315 01:15:00	20180329
20180103 12:00:00	

3.5.3.1.3.3.3 设置模版

我们可以在页面对应的 JS 文件中，设置列头模版和单元格内容的模版。

JS 开发

JS 示例

```
$(function () {
    IPLATUI.EFGrid = {
        "result": {
            columns: [{
                field: "company_name",
                title: "覆盖公司名",
                headerTemplate: "<span style='color: blue'>覆盖公司名</span>", // 列头模版:
// 为 "company_name" 的列: 列头名称为 "覆盖公司名", 颜色为蓝色
                template: "<span style='color: blue'>公司: #=company_name#</span>", // 列单元
// 格模版: 显示值为: "公司: XXX", XXX 取 company_name 对应值, 颜色为蓝色
                width: 200 // 列宽度为 200
            }]
        }
    };
});
```

分析：

1. "result"指 JSP 示例代码中 EFGrid 的 blockId 值；
2. field 指定字段英文名（EFColumn Tag 中的 ename），title 指定字段中文名的值，在 JS 中配置会覆盖 JSP 中 EFColumn 标签内 cname 属性值；
3. headerTemplate 和 template 分别是列头的模板和列内容的模板, 如何使用 kendo 模板请参考 [EEDM6002-设置表格的列类型](#)；
4. JS 中通过这种配置的方式，能够帮助开发人员去实现一些通过 [EEDM6002-设置表格的列类型](#)，[EEDM6002-设置表格的列类型](#)，[EEDM6002-设置表格的列类型](#) 以及后端 java 不能实现的功能，比如上面的列头部信息，列内容的特殊定制。开发人员还能通过配置 JS 回调事件的方式，能够把握 [EEDM6002-设置表格的列类型](#)（具体内容请查看 EFGrid 的"升级指南"）中的各个生命周期，从而实现自己的业务逻辑。

页面效果如下所示：

覆盖公司名
公司: 111
公司: sumsong9
公司: dfasfasdf
公司: 111
公司: AAAAA
公司: Samsung
公司: sumsong2
公司: Motorola1nj111
公司: 12312
公司: oppo

我们可以通过设置列模版，使得数据呈 URL 的形式：

JS 示例

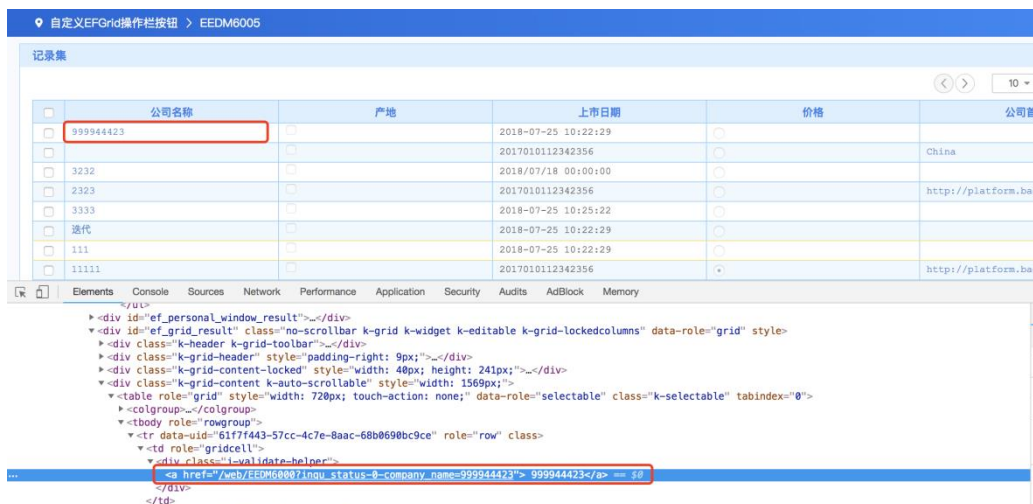
```
$(function () {
    IPLATUI.EFGrid = {
        "result": {
            columns : [{
                field: "company_name",
                template: "<a href='\"+ ctx + \"/web/EEDM6000?inqu_status=0-company_name=#=company_name#'> #=company_name#</a>" //拼接 url 字符串
            }]
        }
    }
});
```

页面效果如下所示：

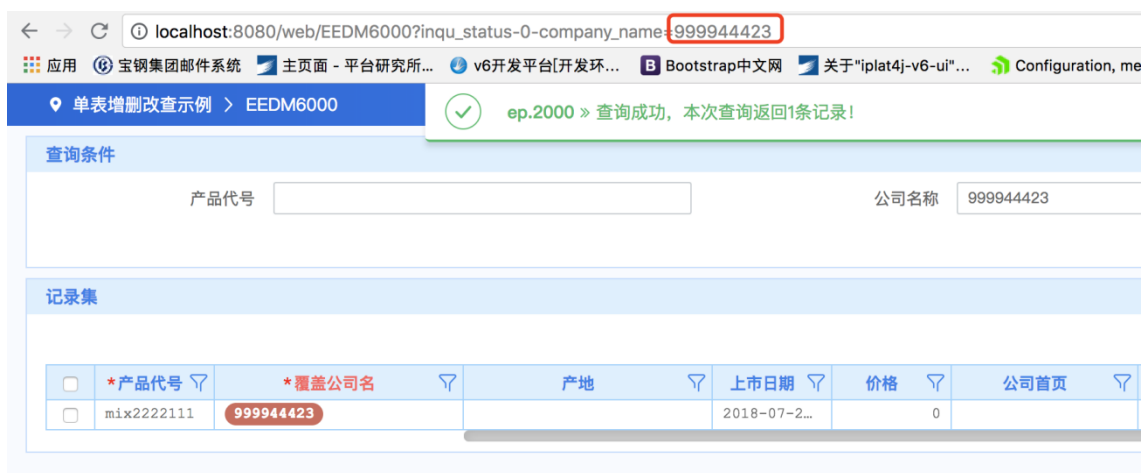
公司名称	产地	上市日期	价格	公司首页	公司首页1
999944423		2018-07-25 10:22:29			
3232		2017010112342356		China	
2323		2018/07/18 00:00:00			
3333		2017010112342356		http://platform.baosight.com	
逸代		2018-07-25 10:25:22			
111		2018-07-25 10:22:29			
11111		2017010112342356		http://platform.baosight.com	
3322		2018-07-25 09:51:51			
88		2018-05-14 16:16:36			

设置列模版后数据样式：

公司名称	产地	上市日期	价格	公司首页	公司首页1
999944423		2018-07-25 10:22:29			
3232		2017010112342356		China	
2323		2018/07/18 00:00:00			
3333		2017010112342356		http://platform.baosight.com	
逸代		2018-07-25 10:25:22			
111		2018-07-25 10:22:29			
11111		2017010112342356		http://platform.baosight.com	
3322		2018-07-25 09:51:51			
88		2018-05-14 16:16:36			



跳转页面:



3.5.3.1.3.3.4 数字格式化

背景介绍：往往列中储存的值会是数字的形式，有时我们需要将数字格式化成我们想要显示的样子，那就要用到属性：**format**。

属性简介：格式化日期和数字单元格信息；

JSP 页面开发

格式化

```
<%-- 格式化数字类型数据 注意要在 js 中 dataBinding 转换数据类型 --%>
<EF:EFColumn ename="price" cname="价格" format="{0:c2}" />
```



对数字列进行格式化，注意要确保 **EiBlock** 中存储的数据是正确的**数字**，而不是字符串。例如，正确的数值应该是`{price: 11}`，而不是`{price: "11"}`。

如下所示，后台返回的数据是字符串：


```
> EInfo (name: "", descName: "", msg: "查询成功, 本次返回记录0条记录!", msgKey: "ep-2008", detailMsg: "", -)
vblocks:
  > country: EBlockMeta {meta: EBlockMeta, rows: Array(6), colCount: 0, extAttr: {...}}
  > result: EBlockMeta {meta: EBlockMeta, rows: Array(6), colCount: 0, extAttr: {...}}
    > country: EBlockMeta {meta: EBlockMeta, rows: Array(6), colCount: 0, extAttr: {...}}
      > extAttr: {offset: 0, count: 0, limit: 10}
      > meta: EBlockMeta {blockId: "result", meta: {...}, desc: "...", extAttr: {...}, colCount: 17}
        > rows: Array(18)
          > 0: [7] ["abcde5", "111", "2", "2.88%", "...", "...", "...", "...", "...", "...", "...", "...", "...", "...", "...", "...", "...", "..."]
          > 1: [7] ["abced5", "sumsong", "China,all", "2018/01/03 12:00:00", "119.80%", "newPage", "Africa,America", "...", "baosight6666666666.cc", "https://static.bootcss.com/www/assets/img/typescript.png", "...", "1%", "..."]
          > 2: [7] ["FSDKcc", "redge", "USA", "2018/01/03 12:00:00", "119.80%", "newPage", "Asia,Tasg", "sadf", "-1", "asdf", "afds", "fat", "..."]
          > 3: [7] ["11EE", "1", "GB,German", "2017/11/07 08:00:00", "111.80%", "...", "America", "...", "abc@baosight", "https://static.bootcss.com/www/assets/img/typescript.png", "...", "...", "...", "..."]
          > 4: [7] ["test1", "AAAAA", "GB,German,France", "2017/12/15 08:00:00", "8.80%", "...", "abc@baosight", "...", "...", "...", "..."]
          > 5: [7] ["5H-1762", "Samung", "GB,German,France", "2017/10/28", "111.80%", "http22wsamsungcomcn2", "三星", "13808000000", "abc@baosight", "fuyinjin", "021-50881155", "1", "2008", "...", "127.8.0.1", "中文", "EFDAA0"]
          > 6: [7] ["15181474320", "sumsong", "America", "2018/01/03 12:00:00", "28.80%", "newPage", "Africa,America", "...", "baosight6666666666.cc", "https://static.bootcss.com/www/assets/img/typescript.png", "...", "1%", "..."]
          > 7: [7] ["W822", "Motorola11", "GB,German,France", "2017/11/07 08:00:00", "111.80%", "EETL", "N70", "...", "abc@baosight", "https://static.bootcss.com/www/assets/img/typescript.png", "...", "1%", "...", "NOKIA"]
          > 8: [7] ["15181474320", "12312", "1231", "213", "11.80%", "11323", "America", "123", "123", "21", "123", "21", "123", "123", "123", "123", "123", "123"]
          > 9: [7] ["N70", "apple", "GB,German,France", "2017/11/22 08:00:00", "11323.80%", "EETL", "N70", "...", "abc@baosight", "https://static.bootcss.com/www/assets/img/typescript.png", "...", "20", "...", "NOKIA", "..."]
        ]
      ]
    ]
  ]
```



💡 那么需要使用 `dataBinding` 方法在 `Grid` 数据渲染之前，做数据预处理，进行类型转换，转换完成后属性 `format` 才会生效，得到正确的格式。

JS 开发

格式化

```
// 数据绑定，将要格式化的数字数据转成 float，否则属性 format 无效
dataBinding: function (e) {
    for (var i = 0, length = e.items.length; i < length; i++) {
        if(isAvailable(e.items[i].price)){
            e.items[i].price = parseFloat(e.items[i].price);
        }
    }
}
```

页面效果如下所示:

* 结点名称	价格	搜索
	¥2.00	
	¥19.00	Africa, Am...
	¥1.00	Oceania
	¥11.00	America
	¥0.00	
	¥10.14	查询
	¥20.00	Africa, Am...
	¥150,122.00	N70
	¥1.00	America
	¥150,122.00	N70

3.5.3.1.4 EEDM6003-EFGrid 的列模版示例

3.5.3.1.4.1 页面功能简介

本示例页面用于说明，如何在 EFGrid 中使用列的模版。

在开发前，请先阅读：[Kendo Template 使用介绍](#)

3.5.3.1.4.2 配置和 API 详情

参考 [EFGGrid](#) 中的描述。

3.5.3.1.4.3 页面开发步骤

3.5.3.1.4.3.1 JSP 页面开发

在 **Grid** 渲染之前，我们可能需要对某些列的数据做展示处理，例如，将某列的文字显示成红色。如果不使用列模版功能，我们通过会在 **Grid** 渲染之前的回调函数中，编写列单元格的渲染逻辑。这通常导致回调函数的代码臃肿，逻辑混乱。现在我们可以使用列模版功能，对 **Grid** 中的数据列进行定制化展示。

在开发之前，我们需要知道什么是 **template**（[Kendo Template 使用介绍](#)），开发过程中，自定义列模版完全遵循上述的模版语法。

示例代码如下：

EEDM6003.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid needAuth="false" blockId="result" autoDraw="no">
    <EF:EFColumn ename="price" cname="价格" defaultValue="998"/>
    <EF:EFColumn ename="validate_5" cname="数量" defaultValue="100"/>
    <EF:EFColumn ename="totalPrice" enable="false" cname="总价"/>
  </EF:EFGrid>
</EF:EFRegion>
```

我们通过对 [EFGGrid](#) 进行基本的配置，首先可以看到页面效果如下：

	价格	数量	总价
☐	15012	0	
☐	150122	0	
☐	11	0	
☐	150122	0	
☐	150122	0	
☐	3550	2000	
☐	10.14	2000	
☐	30	2000	
☐	0.12	0	
☐	1200.5	2000	

这是一个基本 **Grid** 表格，实现了基础(增删改查功能)功能。但是我们如果想对列进行定制化的展示，比如为了突出价格，前面添加货币符号，数量超过 1000 时字体加粗，总价能够自动计算出来等需求。

这就会使用到我们的列模板技术。

kendo 在给 **Grid** 渲染列内容的时候，给我们开发人员提供了一个可以操控的接口，这个接口就是 **template**，**template** 的值可以是字符串，也可以是 **function**。所以列模板的实现通常来说，有两种实现方式，**模板**和**回调函数**。

3.5.3.1.4.3.1.1 模板

模板嘛，其实和前面的 [Kendo Template](#) 有着一样使用方式。下面我们就使用模板来给价格添加货币符号并且标红：

EEDM6003.js

```
$(function () {
    IPLATUI.EFGrid = {
        "result": {
            columns: [{
                field: "price",
                headerTemplate: "价格",
                template: "<span style='color: red'>¥#=price#</span>",
            }]
        }
    };
});
```

效果如下：

	价格	数量	总价
☐	¥1501.2	0	
☐	¥1501.22	0	
☐	¥11	0	
☐	¥1501.22	0	
☐	¥1501.22	0	
☐	¥9550	2000	
☐	¥10.14	2000	
☐	¥30	2000	
☐	¥0.12	0	
☐	¥1200.5	2000	

这里我们通过模板的方式，给价格列进行了定制化展示(添加货币符号以及标红)。注：#=price# 会获取列中字段 price 的数值。

3.5.3.1.4.3.1.2 回调函数

上面是介绍了模板方式去定制我们的列内容，但是这样灵活度不够。像后面这样一个需求，当数量超过 1000 时，把字体加粗。使用模板方式就比较麻烦，我们开始使用回调方法去实现这样的需求。

EEDM6003.js

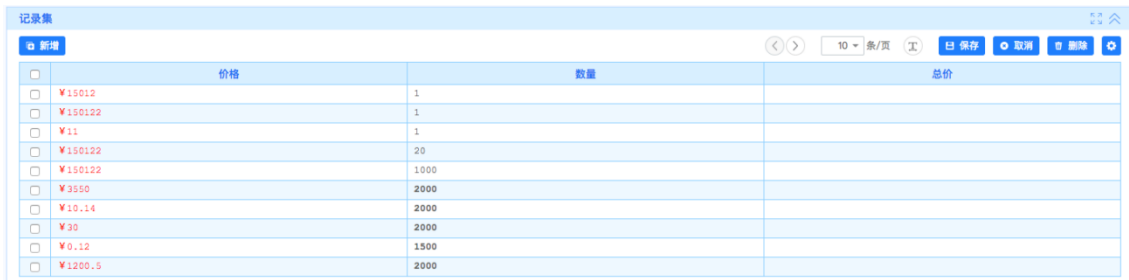
```
$(function () {
    IPLATUI.EFGrid = {
        "result": {
            columns: [{
                field: "price",
                headerTemplate: "价格",
                template: "<span style='color: red'>¥#=price#</span>"
            }, {
                field: "validate_5",
                headerTemplate: "数量",
                // 渲染列的时候，会调用此方法，返回的内容将会作为列内容
                template: function (item) {
                    if ($.isNumeric(item['validate_5'])) {
                        // + 能够强制把其他类型转为数字类型
                        if (+item['validate_5'] > 1000 ) {
                            return "<span style='font-weight: 600'>" + item['validate_5'] +
                                "</span>";
                        } else {
```

```

        return item['validate_5'];
    }
    } else {
        // 防止 validate_5 不存在
        return item['validate_5'] || '';
    }
    }
    ]]
    }
    });
});

```

效果如下图：



	价格	数量	总价
☐	¥15012	1	
☐	¥150122	1	
☐	¥11	1	
☐	¥150122	20	
☐	¥150122	1000	
☐	¥3550	2000	
☐	¥10.14	2000	
☐	¥30	2000	
☐	¥0.12	1500	
☐	¥1200.5	2000	

前面完成了价格和数量的定制显示，接下来准备处理总价。这里我们使用模板方式去实现"总价列"的头信息定制，使用回调函数方式去实现总价计算。

代码如下：

EEDM6003.js

```

$(function () {
    IPLATUI.EFGGrid = {
        "result": {
            columns: [{
                field: "price",
                headerTemplate: "价格",
                template: "<span style='color: red'>¥#=price#</span>"
            }, {
                field: "validate_5",
                headerTemplate: "数量",
                template: function (item) {
                    if ($.isNumeric(item['validate_5'])) {
                        // + 能够强制把其他类型转为数字类型
                        if (+item['validate_5'] > 1000 ) {
                            return "<span style='font-weight: 600'>" + item['validate_5'] +
                            "</span>";
                        }
                    } else {
                        return item['validate_5'];
                    }
                }
            }, {
                field: "totalPrice",

```

```

        headerTemplate: "<span style='color: red'>总价</span>",
        template: function (item) {
            return kendo.format("{0:c}", (+item['price']) * (+item['validate_5']));
        }
    }
}
});
});

```

效果如下：

	价格	数量	总价
☐	¥15012	1	¥15,012.00
☐	¥150122	1	¥150,122.00
☐	¥11	1	¥11.00
☐	¥150122	20	¥3,002,440.00
☐	¥150122	1000	¥150,122,000.00
☐	¥3550	2000	¥7,100,000.00
☐	¥10.14	2000	¥20,280.00
☐	¥30	2000	¥60,000.00
☐	¥0.12	1500	¥180.00
☐	¥1200.5	2000	¥2,401,000.00

当我们修改"数量"或者"价格", 并且保存完毕。总价也会跟着变化。

JS配置原理

在JS中, 通过IPLATUI.EFGrid可以对EFGrid进行个性化的配置。熟悉Kendo UI 的开发者会知道, kendo 可以通过options设置组件的初始配置。这里采取了同样的方式, 只不过是把options的值存放在IPLATUI.EFGrid["result"]对象下面。这里的result是对应的EFGrid的blockId。这样JS中定义的options 属性, 就可以覆盖 (extend) 到的EFGrid中的属性。从而达到了既支持Kendo原生配置, 又能支持EFGrid标签配置的效果。

因此, 我们只需要对columns 进行自定义设置, 就能实现我们的目的。要了解更多的kendoGrid的columns 的配置信息, 请参考官网。

3.5.3.1.5 EEDM6004-注册 EFGrid 功能按钮

3.5.3.1.5.1 页面功能简介

本示例页面用于说明, 如何在 EFGrid 中注册按钮, 并编写按钮的业务逻辑。

请先阅读文档:

[EDFA01 按钮注册说明](#)

[EFGrid 操作按钮注册规则](#)

3.5.3.1.5.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.3.1.5.3 页面开发步骤

3.5.3.1.5.3.1 在 EDFA01 注册按钮

首先，通过配置管理中心的 **EDFA01** 页面，进行按钮信息注册。在按钮注册过程中，需要注意区域标识的设置，可以将按钮注册到 **Grid** 默认位置的操作栏上，也支持将按钮注册到 **EFGrid** 所在的 **EFRegion** 区域中（兼容 3.9 之前的做法，~~但不推荐~~）。

3.5.3.1.5.3.1.1 按钮注册到 Grid 默认位置的操作栏

首先在 **EDFA01** 页面按照上方的 [EFGrid 操作按钮注册规则](#)，[EDFA01 按钮注册说明](#) 中的说明，注册相关的功能按钮。

例如下面截图表示：在示例页面 **EEDM6004** 的 **EFGrid**（blockId 为 result）的默认位置注册了一个自定义的按钮，按钮的 id 是 **CX02**。

查询条件

画面英文名: EEDM6004 按钮英文名: 工程: IPLAT4J

记录创建开始日期: 记录创建截止日期:

记录集

新增 保存 取消 删除

	*画面英文名	*按钮英文名	*区域标识	按钮中文	按钮描述	排序	是否分级	*是否授权	资源地址	样式
☐	EEDM6004	INSERT	GRID:EF_GRID_RESULT	新增	Grid的新增按钮	0	不分级	不授权	css:k-add	显示文字和图标
☐	EEDM6004	SAVE	GRID:EF_GRID_RESULT	保存	Grid的保存按钮	1	不分级	不授权	css:k-update	显示文字和图标
☐	EEDM6004	CX01	GRID:EF_GRID_RESULT	注册	注册的示例按钮1	2	不分级	不授权	css:fa-user	显示文字和图标
☐	EEDM6004	CX02	GRID:EF_GRID_RESULT	复制	注册的示例按钮2	3	不分级	不授权	css:fa-copy	显示文字和图标
☐	EEDM6004	CANCEL	GRID:EF_GRID_RESULT	取消	Grid的取消按钮	4	不分级	不授权	css:k-cancel	显示文字和图标

然后进入到示例页面 **EEDM6004**，可以看到我们注册的 **CX02** 按钮 复制，出现在 **Grid** 的默认位置的工具条上了。

记录集

公司名称: 复制到选中的列

新增 保存 取消 注册 复制 设置

默认位置

	公司名称	产地	上市日期	价格	公司首页
☐	Apple	USA	2017/10/04 00:00:00	999	
☐	Motorola	China	2007-07-06	11	http://www.philips.com
☐	Philips	Germany	2007/03/08	1200.5	
☐	los	China	2017-10-14	0	
☐	宝信软件		2017-10-10	10	
☐	Motorola1	13	2007-04-06	15	时间访问量统计
☐	Motorola1	13	2007-04-06	15	时间访问量统计
☐	Motorola1	13	2017-10-09	15	时间访问量统计
☐	Los	9	2017-10-20	15	时间访问量统计

测试

注册Grid功能按钮 > EEDM6004

记录集

公司名称: 复制到选中的列

新增 保存 取消 注册 复制 设置

	公司名称	产地	上市日期	价格	公司首页
☐	Apple	USA	2017/10/04 00:00:00	999	
☐	Motorola	China	2007-07-06	11	http://www.philips.com
☐	Philips	Germany	2007/03/08	1200.5	
☐	los	China	2017-10-14	0	
☐	宝信软件		2017-10-10	10	
☐	Motorola1	13	2007-04-06	15	时间访问量统计

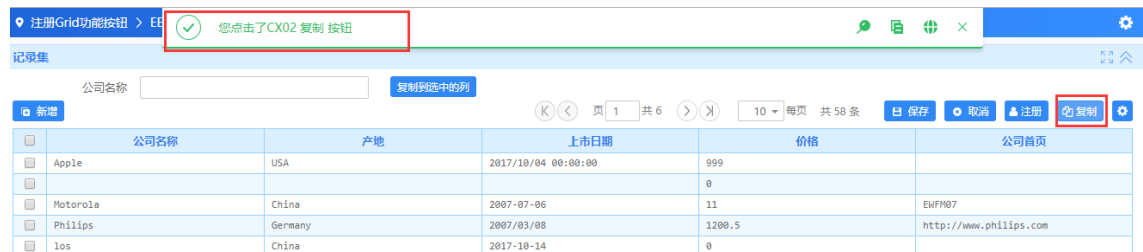
接下来，我们可以为这个注册的 **CX02** 按钮定义事件，在 **EEDM6004.js** 文件中，代码如下：

Grid 按钮事件定义

```
$(function () {
    IPLATUI.EFGrid = {
        "result": {
            loadComplete: function(grid) { // 在 Grid 加载完成后, 才能给 Grid 上的按钮绑定事件

                $("#CX02").on("click", function () {
                    NotificationUtil("您点击了 CX02 复制 按钮");
                });
            }
        }
    };
});
```

点击 CX02 按钮后, 我们将会看到:



3.5.3.1.5.3.1.2 按钮注册到 EFRegion 中的场景

当 EFRegion 中存在 EFGrid 时, 不推荐在 EFRegion 上注册按钮。

下面的截图, 用于说明按钮注册到 EFRegion 中的场景。(区域标识对应于 [EFRegion](#) 中的 id 属性):

	* 画面英文名	* 按钮英文名	* 区域标识	按钮中文
	EEDM6004	CUSTOM	RESULT	测试

按钮注册授权后, 在该 region 中, 将渲染出相对应的 button。如下图所示:



在 JS 中为按钮绑定事件:

Grid 按钮事件

```
$("#CUSTOM").on("click", function () {
```

```
resultGrid.dataSource.page(1);
});
```

3.5.3.1.5.3.2 在 JSP 中配置 EFButton

我们也可以在 jsp 中渲染 button，并在 JS 中为其绑定事件。

jsp 页面开发：

Grid 按钮渲染

```
<EF:EFRegion id="result" title="记录集">
    <div class="row">
        <EF:EFInput ename="company_name" cname="公司名称"/>
        <EF:EFButton ename="c" cname="复制到选中的列"/>
    </div>
    ...
</EF:EFRegion>
```

js 示例：

Grid 按钮事件定义

```
$("#c").on("click", function () {
    var rows = resultGrid.getCheckedRows(); // EFGrid 列的批量赋值功能
    $.each(rows, function (index, item) {
        resultGrid.setCellValue(item, "company_name", $("#company_name").val())
    });
});
```



#CUSTOM（测试）是通过 EDFA01 注册的按钮（位置为 grid 下方），**#c**（复制到选中的列）是 JSP 页面 EFButton 渲染的按钮（位置为输入框后面），其余的 INSERT（新增），SAVE（保存），CX01（注册），CX02（复制），CANCEL（取消）皆通过 EDFA01 注册的按钮（位置为默认位置）



【注意】

在 3.9 版本之前，我们通过如下的代码片段，来编写按钮的点击事件。

```
/*JS*/
function button_F1_onclick() {
    .....
}
```

3.5.3.1.5.3.2.1 注意事项

现在，我们已经把这种方式废弃掉了！！！！。统一采用 jQuery 的事件绑定方式。请大家以后开发过程中，对 BUTTON 进行事件处理的时，千万注意！！！！


```

/*JS*/
$("#F1").on("click", function (e) {
    .....
});

```

3.5.3.1.6 EEDM6005-自定义 EFGrid 的工具栏

3.5.3.1.6.1 页面功能简介

本示例页面用于说明，如何在 JS 中设置 EFGrid 的工具栏的按钮显示和点击事件。

请先阅读文档：

[EDFA01 按钮注册说明](#)

[EFGrid 操作按钮注册规则](#)

3.5.3.1.6.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.3.1.6.3 页面开发步骤

3.5.3.1.6.3.1 JSP 页面开发

在开发中，当 EFGrid 默认的增删改按钮不能满足业务需求时，通常要在 EFGrid 的工具栏上自定义一些业务按钮，还有一种场景，需要自由控制 EFGrid 上默认的功能按钮的显示和隐藏。

自定义工具栏有两种方式：

一种是在 EFGrid 标签中设置 toolbarConfig 属性；（通常用于自由控制 EFGrid 上默认的功能按钮的显示和隐藏。）

另一种是在页面 js 中，通过设置 toolbarConfig 属性来自定义 button。（通常用于添加自定义的功能按钮）

1.EFGrid 标签中设置 toolbarConfig，控制默认的按钮显示：

JSP 示例

```

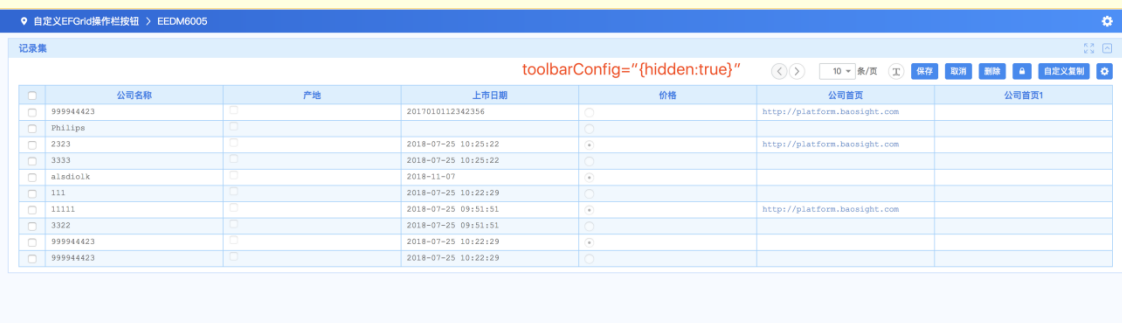
<EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" autoDraw="no" toolbarConfig="{add: false, cancel:
true, save: true}" needAuth="false">
        <EF:EFColumn ename="company_name" cname="公司名称"/>
        <EF:EFColumn ename="made_country" cname="产地" editType="checkbox"
displayType="checkbox" />
        <EF:EFColumn ename="sale_date" cname="上市日期" editType="datetime"/>
        <EF:EFColumn ename="price" cname="价格" editType="radio"
displayType="radio"/>
        <EF:EFColumn ename="company_page" displayType="url" cname="公司首页"/>
        <EF:EFColumn ename="company_page1" displayType="url" cname="公司首页1"/>
    </EF:EFGrid>
</EF:EFRegion>

```

```
</EF:EFGGrid>
</EF:EFRegion>
```

EFGGrid配置项toolbarConfig相关

- 1.toolbarConfig用于配置EFGGrid工具栏的"新增", "保存", "取消"和"删除"按钮的显示, 分别对应 **add save cancel delete** (EDFA01按钮注册页面中按钮英文名为"**INSERT**" "**SAVE**" "**CANCEL**" "**DELETE**")
- 2.JS中toolbarConfig的配置优先级高于tag配置
3. 隐藏全部基础功能按钮(不会隐藏 **自定义按钮/工具栏最右侧setting导出按钮/ 翻页信息**), 可以配置 toolbarConfig={hidden:true}, 默认false; (此时Grid的操作栏还在, 包括翻页和setting导出按钮)
隐藏Grid操作栏上所有内容, 可以配置 toolbarConfig={hidden:'all'}; (此时整个Grid操作栏都不会显示, EDFA01注册的自定义按钮当然也不会显示。注意 '**all**' 要配置成字符串形式)
- 4.默认显示全部按钮, 若要隐藏某个按钮(如"新增"按钮), 可以配置 add:false



toolbarConfig="{hidden:'all'}"

	单价	数量	总价	覆盖公司名	验证码0	搜索
☐	2	41	82	Motorola	测试 多个按钮	
☐	1400	2000	2800000	Motorola	测试 多个按钮	查询
☐	3550	2000	7100000	Motorola1	测试 多个按钮	查询
☐	10.14	2000	20280	Samsung	测试 多个按钮	查询
☐	30	2000	60000	SonyEricsson	测试 多个按钮	查询
☐	0.12	0	0	NOKIA	测试 多个按钮	search
☐	0.12	0	0	SonyEricsson	测试 多个按钮	1
☐	11	0	0	Motorola	测试 多个按钮	
☐	1200.5	2000	2401000	Phillips	测试 多个按钮	search
☐	20000	0	0	SonyEricsson	测试 多个按钮	search

2.Javascript 开发

在 JS 文件中, 配置 IPLATUI.EFGGrid 的 toolbarConfig 属性, 在 Grid 的工具栏自定义按钮。

js 代码

```
IPLATUI.EFGGrid = {
```

```

"result": {
  ...
  toolbarConfig: {
    hidden: false, // true 时, 不显示功能按钮, 但保留 setting 导出按钮
    // add: false, // 不显示新增按钮
    // cancel: false, // 不显示取消按钮
    // save: false, // 不显示保存按钮
    delete: true, // 显示删除按钮

    buttons: [ // 不允许覆盖默认的 4 个按钮, 可以设置自定义的按钮
      {
        name: "lock",
        text: "锁定",
        layout: "2",
        icon: "css:fa-lock",
        click: function () {
          WindowUtil({
            title: "定制",
            content: "<div class='kendo-del-message'>自定义的按钮, 在工具栏的右侧
</div>"
          });
        }
      },
      {
        name: "custom",
        text: "自定义复制",
        layout: "3",
        icon: "fa-file",
        click: copy
      }
    ]
  }

  function copy() {
    WindowUtil({title:'信息', content: "<div class='kendo-del-message'>复制
</div>"});
  }
}
};

```

注:

1.toolbarConfig.buttons 中配置 **button**: **name** 为按钮的唯一标识, **text** 为按钮显示名, **icon** 为按钮图标, **click** 中定义按钮的点击事件

2.自定义按钮的 **name** 值不要取 **"add"**, **"save-changes"**, **"cancel-changes"** 和 **"delete"**, 避免与 **EFGGrid** 中默认按钮("新增","保存","取消"和"删除")的 **name** 值冲突, 覆盖了默认的按钮功能。

3. layout 取 **2** 时, 只显示图标, 取 **3** 时, 显示图标和文字

页面效果如下所示:

记录集

10 每页 共 57 条

保存 取消 删除 自定义规则

☐	公司名称	产地	上市日期	价格	公司首页	公司首页1
☐	Apple	☐	2017/10/04 00:00:00	⊕		
☐	Motorola	☐	2007-07-06	⊕	EMF007	
☐	Philips	☐	2007/03/08	⊕	http://www.philips.com	
☐	los	☐	2017-10-14	⊕		
☐	宝信软件	☐	2017-10-10	⊕		
☐	Motorola1	☒	2007-04-06	⊕	时间段访问量统计	
☐	Motorola1	☒	2007-04-06	⊕	时间段访问量统计	
☐	Motorola1	☒	2017-10-09	⊕	时间段访问量统计	
☐	Los	☒	2017-10-20	⊕	时间段访问量统计	
☐	Motorola	☐	2007-03-17	⊕	http://www.nokia.com	

3.5.3.1.7 EEDM6006-列的默认值和编辑状态

3.5.3.1.7.1 页面功能简介

本示例页面用于说明，如何在 EFGrid 中设置列的默认值和列的编辑状态；讲解内容包括：

- defaultValue 的使用；
- enable 和 readonly 的使用区别；
- 隐藏列 hidden 的使用；
- primaryKey 的主键的使用。

3.5.3.1.7.2 配置和 API 详情

参考 [EFGrid](#) 中的描述。

3.5.3.1.7.3 页面开发步骤

3.5.3.1.7.3.1 JSP 页面开发

在开发过程中，我们通常会遇到如下的场景：

- 1) 新增一行数据时，需要设置某些列的默认值；例如，添加用户时，默认的部门信息是“平台研究所”。
- 2) 列永远不可以编辑。例如，用户只有查看权限的列信息。
- 3) 新增时可以编辑，除此之外不允许编辑。例如，新增数据时，可以在前台设置主键信息，但不允许修改已有的主键信息。

针对上述场景，我们通过对 [EFColumn](#)，[EFComboColumn](#)，[EFMultiSelectColumn](#) 的学习和使用，去解决我们上面的需求。

3.5.3.1.7.3.1.1 defaultValue

defaultValue 存在于 [EFColumn](#)，[EFComboColumn](#)，[EFMultiSelectColumn](#) 等 tag 属性中。主要的作用是在 Grid 新增时，给行添加默认值。

针对第 1) 需求，我们可以这样进行处理：

EEDM6006.jsp

```
<%
```

```

        Date date = new Date();
        SimpleDateFormat simpleDateFormat = new SimpleDateFormat("yyyy-MM-dd");
        pageContext.setAttribute("now", simpleDateFormat.format(date));
    %>

<jsp:useBean id="currentTime" class="java.util.Date" />
<fmt:formatDate type="time" value="${currentTime}" pattern="yyyy-MM-dd HH:mm:ss"
var="formatDate"/>

<EF:EFPage title="Grid 列的默认值和编辑状态">
    <EF:EFRegion id="result" title="记录集">
        <EF:EFGrid blockId="result" needAuth="false" autoDraw="no">
            <EF:EFColumn ename="product_id" primaryKey="true" readonly="true"
locked="true" width="120" cname="产品 Id" />
            <EF:EFColumn ename="company_name" cname="公司名" defaultValue="宝信软件" />
            <EF:EFColumn ename="sale_date1" width="150" cname="上市日期"
defaultValue="${now}" editType="date"/>
            <%--defaultValue 只能设置静态的数据，类似时间动态的内容，需要再后台 java 处
理--%>
            <EF:EFColumn ename="sale_date" width="350" required="true" cname="上市时刻"
defaultValue="${formatDate}" editType="datetime"/>
        </EF:EFGrid>
    </EF:EFRegion>
</EF:EFPage>

```

效果如下：

产品Id	覆盖公司名	上市日期	上市时刻
	宝信软件	2017-11-30	2017-11-30 16:32:57
N7021	MotorolaInj111		2017/11/01 00:00:00
111EE	111		2017/11/07 00:00:00
N7022	MotorolaInj111		2017/11/01 00:00:00
N70	MotorolaInj111		2017/11/22 00:00:00
B251	MotorolaInj111		2017/11/16 00:00:00
T6181	Motorola		2007-04-06
SGH-i7161	Samsung		2007-04-06
T6281	SonyEricsson		2007-04-06
B8501	SonyEricsson		2007-04-27
B891	Philips		2007/03/08

当我们点击"新增"按钮后，会给 Grid 新增一行数据，里面除了"产品 id"外，其他列都存在一个默认值。这个默认值就是我们在 EFColumn 中设置的 defaultValue，这样我们就解决了我们的第一个需求。

3.5.3.1.7.3.1.2 enable

enable 属性存在于 [EFColumn](#)，[EFComboColumn](#)，[EFMultiSelectColumn](#) 等 tag 属性中。设置 tag 中属性 enable = false 时，整列都不能被编辑(包括新增和修改)。

针对第 2) 需求，我们可以这样进行处理：

EEDM6006.jsp

```

<c:set var="ctx" value="${pageContext.request.contextPath}" />
<%
    Date date = new Date();
    SimpleDateFormat simpleDateFormat = new SimpleDateFormat("yyyy-MM-dd");

```

```

    pageContext.setAttribute("now", simpleDateFormat.format(date));
%>

<jsp:useBean id="currentTime" class="java.util.Date" />
<fmt:formatDate type="time" value="${currentTime}" pattern="yyyy-MM-dd HH:mm:ss"
var="formatDate"/>

<EF:EFPage title="Grid 列的默认值和编辑状态">
    <EF:EFRegion id="result" title="记录集">
        <EF:EFGrid blockId="result" needAuth="false" autoDraw="no">
            <EF:EFColumn ename="product_id" locked="true" width="120" cname="产品 Id" />
            <EF:EFColumn ename="company_name" cname="公司名" defaultValue="宝信软件" />

            <EF:EFColumn ename="sale_date" width="150" cname="上市日期"
defaultValue="${now}" editType="date"/>
            <%--defaultValue 只能设置静态的数据，类似时间动态的内容，需要再后台 java 处
理--%>
            <EF:EFColumn ename="sale_date" width="350" required="true" cname="上市时刻"
defaultValue="${formatDate}" editType="datetime"/>

            <EF:EFColumn ename="company_page" cname="公司首页" enable="false"/>
        </EF:EFGrid>
    </EF:EFRegion>
</EF:EFPage>

```

现在我们添加新列 `<EF:EFColumn ename="company_page" cname="公司首页" enable="false"/>`，设置其 `enable = "false"`，这样此列就不能被编辑。

效果如下：

记录集					
产品Id	覆盖公司名	上市日期	*上市时刻	公司首页	
☐ N7021	Motorola	111	2017/11/01 00:00:00	EEE1	
☐ 111EE	Motorola	111	2017/11/07 00:00:00	EEE1	
☐ N7022	Motorola	111	2017/11/01 00:00:00	EEE1	
☐ N70	Motorola	111	2017/11/22 00:00:00	EEE	
☐ 8251	Motorola	111	2017/11/16 00:00:00	EEE	
☐ T6181	Motorola		2007-04-06	时间段访问量统计	
☐ SGH-i7161	Samsung		2007-04-06	http://www.samsung.com	
☐ T6281	SonyEricsson		2007-04-06	http://www.sonyericsa...	
☐ 88501	SonyEricsson		2007-04-27		
☐ S891	Philips		2007/03/08	http://www.philips.com	

"公司首页" 这列就不能被编辑。

3.5.3.1.7.3.1.3 readonly

`readonly` 属性存在于 [EFColumn](#)，[EFComboColumn](#)，[EFMultiSelectColumn](#) 等 tag 属性中。设置 tag 中属性 `readonly = true` 时，新增时能够编辑此列，当新增完成并且点击保存后，此列就不能再编辑了。

针对第 3) 需求，我们可以这样进行处理：

EEDM6006.jsp

```

<c:set var="ctx" value="${pageContext.request.contextPath}" />
<%
    Date date = new Date();

```

```

SimpleDateFormat simpleDateFormat = new SimpleDateFormat("yyyy-MM-dd");
pageContext.setAttribute("now", simpleDateFormat.format(date));
%>

<jsp:useBean id="currentTime" class="java.util.Date" />
<fmt:formatDate type="time" value="${currentTime}" pattern="yyyy-MM-dd HH:mm:ss"
var="formatDate"/>

<EF:EFPage title="Grid 列的默认值和编辑状态">
    <EF:EFRegion id="result" title="记录集">
        <EF:EFGrid blockId="result" needAuth="false" autoDraw="no">
            <EF:EFColumn ename="product_id" primaryKey="true" readonly="true"
locked="true" width="120" cname="产品 Id" />
            <EF:EFColumn ename="company_name" cname="公司名" defaultValue="宝信软件" />
            <EF:EFColumn ename="sale_date1" width="150" cname="上市日期"
defaultValue="${now}" editType="date"/>
            <%--defaultValue 只能设置静态的数据，类似时间动态的内容，需要再后台 java 处
理--%>
            <EF:EFColumn ename="sale_date" width="350" required="true" cname="上市时刻"
defaultValue="${formatDate}" editType="datetime"/>

            <EF:EFColumn ename="company_page" cname="公司首页" enable="false"/>
        </EF:EFGrid>
    </EF:EFRegion>
</EF:EFPage>

```

设置 "product_id" 列的 `readonly = "true"`, `primaryKey="true"`。 `primaryKey = "true"` 表示此列被设置成为主键，`readonly = "true"` 是表示此列只能新增时候进行编辑，保存完毕后，不能再继续编辑。

3.5.3.1.7.3.1.4 hidden

`hidden` 属性存在于 [EFColumn](#)，[EFComboColumn](#)，[EFMultiSelectColumn](#) 等 tag 属性中。设置 tag 中属性 `hidden = "true"` 时，此列就会被隐藏。

我们针对上面的代码，做出如下修改：

EEDM6006.jsp

```

<c:set var="ctx" value="${pageContext.request.contextPath}" />
<%
    Date date = new Date();
    SimpleDateFormat simpleDateFormat = new SimpleDateFormat("yyyy-MM-dd");
    pageContext.setAttribute("now", simpleDateFormat.format(date));
%>

<jsp:useBean id="currentTime" class="java.util.Date" />
<fmt:formatDate type="time" value="${currentTime}" pattern="yyyy-MM-dd HH:mm:ss"
var="formatDate"/>

<EF:EFPage title="Grid 列的默认值和编辑状态">
    <EF:EFRegion id="result" title="记录集">
        <EF:EFGrid blockId="result" needAuth="false" autoDraw="no">
            <EF:EFColumn ename="product_id" primaryKey="true" readonly="true"
locked="true" width="120" cname="产品 Id" />
            <EF:EFColumn ename="company_name" cname="公司名" defaultValue="宝信软件" />

```

```

<EF:EFColumn ename="sale_date1" width="150" cname="上市日期"
defaultValue="{now}" editType="date"/>
<%-defaultValue 只能设置静态的数据，类似时间动态的内容，需要再后台 java 处
理-%>
<EF:EFColumn ename="sale_date" width="350" required="true" cname="上市时刻"
defaultValue="{formatDate}" editType="datetime"/>

<EF:EFColumn ename="company_page" cname="公司首页" enable="false"
hidden="true"/>
</EF:EFGGrid>
</EF:EFRegion>
</EF:EFPage>

```

当我们给 `<EF:EFColumn ename="company_page" cname="公司首页" enable="false" hidden="true"/>` 列添加属性 `hidden = "true"`，这样此列就已经被隐藏了。

产品id	覆盖公司名	上市日期	上市时刻
☐ N7021	MotorolaInjill		2017/11/01 00:00:00
☐ 111EE	111		2017/11/07 00:00:00
☐ N7022	MotorolaInjill		2017/11/01 00:00:00
☐ N70	MotorolaInjill		2017/11/22 00:00:00
☐ 8251	MotorolaInjill		2017/11/16 00:00:00
☐ T6181	MotorolaI		2007-04-06
☐ SGH-I7161	Samsung		2007-04-06
☐ T6281	SonyEricsson		2007-04-06
☐ 88501	SonyEricsson		2007-04-27
☐ S891	Philips		2007/03/08

3.5.3.1.7.3.2 使用总结

1. `defaultValue` 属性设置某一列的默认值，可以是字符串(如: "宝信软件")或者日期时间(`{now}`)指页面 `request` 区设置的一个日期类型的属性，即当前日期
2. 当配置 `readonly="true"`，那么这一列只在新增时可编辑，保存后只读，默认为 `false`
3. 当配置 `enable="false"`，则这一列永远不可编辑(包括在新增时)，可以与 `defaultValue` 一起使用，默认为 `true`
4. 在 `js` 中配置 `hidden="true"` 时，即该列不可见，在页面上不会显示此列

3.5.3.1.8 EEDM6007-EFGGrid 列校验示例

3.5.3.1.8.1 页面功能简介

本示例页面用于说明，如何对 `Grid` 内的列信息进行校验，包括对 `EFColumn` (包括日期列) 和 `EFComboColumn` 的校验；

列的校验的规则，参考 [EFCInput](#) 中的描述；

讲解内容包括：`EFGGrid` 中 `EFColumn` 的校检使用。

3.5.3.1.8.2 配置和 API 详情

参考 [EiColumn](#) 中的描述。

3.5.3.1.8.3 页面开发步骤

主场景主要是针对 **Grid** 中列字段信息进行校验。包括必填校验、邮箱验证、数字、等校验具体详情请查看 [EiColumn](#)。下面列举一些使用场景。

3.5.3.1.8.3.1 JSP 页面开发

JSP 示例

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" autoDraw="no" needAuth="false" editHelper="true">
    <EF:EFColumn ename="product_id" primaryKey="true" readonly="true" cname="编号
[固定列]" />

    <EF:EFColumn ename="sale_date" cname="日期" editType="date" required="true" />
    <EF:EFComboColumn ename="company_page" cname="下拉" required="true"
      columnTemplate="#=textField#-#=valueField#"
      itemTemplate="#=textField#-#=valueField#"
      textField="textField" valueField="valueField" optionLabel="请
选择">
      <EF:EFOption label="p1" value="1" />
      <EF:EFOption label="p2" value="2" />
      <EF:EFOption label="p3" value="3" />
    </EF:EFComboColumn>

    <EF:EFColumn ename="field1" data-rules="email" cname="电子邮箱" />
    <EF:EFColumn ename="field2" minLength="3" cname="最小长度" />
    <EF:EFColumn ename="field3" cname="必填" required="true" />
    <EF:EFColumn ename="field4" cname="正则表达式"
      valueType = "N"
      data-regex="/^([5-9]|100|[1-9]\\d)$/" data-errorprompt="请输入 5-
100 之间的数字"
      required="false" />
  </EF:EFGrid>
</EF:EFRegion>
```

上面中的的我们通过 `required = "true"` 能够实现列的必填校验，`data-rules = "email"` 判断是否为邮箱，`minLength = "3"` 来判断列的最小长度是 3。规则详情请参考 [EiColumn](#)。

数值校验-正则表达式

`data-regex="/^([5-9]|100|[1-9]\\d)$/"`

注意：

数字的正则表达式是 `\d`，在 `data-regex` 中的应转义为 `\\d`，同理 `s`、`w` 等也应该转义为 `\\s`、`\\w`

3.5.3.1.8.3.2 页面效果

新增一条数据：

记录集							
新增 10 条/页 保存 取消 删除							
☐	编号[固定列]	*日期	*下拉	电子邮箱	最小长度	*必填	*正则表达式
☒							
☐	www	2017/11/01 00:00:00	EEE1				
☐	N7021	2017/11/01 00:00:00	EEE1				
☐	111EE	2017/11/01 00:00:00					
☐	N7022	2017/11/01 00:00:00	EEE1				
☐	N70	2017/11/22 00:00:00	EEE				
☐	8251	2017/11/16 00:00:00	EEE				
☐	F6181	2007-04-06	时间段访问量统计				
☐	SGH-17161	2007-04-06	http://www.samsung.com				
☐	T6281	2007-04-06	http://www.sonyericsson...				
☐	88501	2007-04-27					

3.5.3.1.9 EEDM6008-EFGrid 中自定义选中项和选中项获取示例

3.5.3.1.9.1 页面功能简介

本示例页面用于说明，如何使用 JS 让 Grid 进行选中、获取选中后的行信息。

讲解内容包括：

- EFGrid 中 checkMode 属性的使用；
- EFGrid 中 setCheckedRows 方法使用；
- EFGrid 中 getCheckedRows 方法使用；
- [WindowUtil](#) 使用；
- EFGrid 中 fillNode 方法使用。

3.5.3.1.9.2 配置和 API 详情

参考 [EFGrid](#) 中的描述。

3.5.3.1.9.3 页面开发步骤

3.5.3.1.9.3.1 JSP 页面开发

3.5.3.1.9.3.1.1 checkMode 的使用

checkMode 是 EFGrid 提供的一个控制 Grid 选中方式的属性，配置参数分为两个字段，中间使用"," 进行分割。

第一个属性的选项值有两个，"single"和"multiple"，表示选中一行还是多行。顾名思义，single 表示 Grid 只能选中一行，multiple 表示 Grid 能够选中多行。默认是表示能够选中多行，即"multiple"。

示例代码如下：

EEDM6008.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result">
    </EF:EFGrid>
  </EF:EFRegion>
```

效果如下：

记录集													
☐	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5
☒	wewewe	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdadas			1
☒	qqq	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdadas			1
☒	N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdadas			1
☐	111EE	111	USA	2017/11/0...	11		N7011						1
☐	N7022	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdadas			1
☐	N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	asdadas			20
☐	8251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	asdadas			1000
☐	T6181	Motorola...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...		021-50801...	1	2000
☐	SGH-i7161	Samsun...	Korea	2007-04-06	10.14	http://www.samsun...	查询	138000000...	aaa@123.c...	fuyimin	021-50801...	1	2000
☐	T6281	SonyEric...	Japan	2007-04-06	30	http://www.sonyer...	查询	138000000...	aaa@123.c...	label	021-50801...	1	2000

能够选中多行的数据。

当我们在 EFGGrid 中添加 `checkMode = "single, cell"`，代码如下：

EEDM6008.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGGrid blockId="result" checkMode="single, cell">
  </EF:EFGGrid>
</EF:EFRegion>
```

效果如下：

记录集													
☒	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5
☒	wewewe	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdadas			1
☐	qqq	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdadas			1
☐	N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdadas			1
☐	111EE	111	USA	2017/11/0...	11		N7011						1
☐	N7022	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdadas			1
☐	N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	asdadas			20
☐	8251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	asdadas			1000
☐	T6181	Motorola...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...		021-50801...	1	2000
☐	SGH-i7161	Samsun...	Korea	2007-04-06	10.14	http://www.samsun...	查询	138000000...	aaa@123.c...	fuyimin	021-50801...	1	2000
☒	T6281	SonyEric...	Japan	2007-04-06	30	http://www.sonyer...	查询	138000000...	aaa@123.c...	label	021-50801...	1	2000

我们对 EFGGrid 进行勾选，全选会默认选中最后一行，单独选择只能选中一行，不能选择多行。

第一个参数讲解完了，现在来看看第二个参数，第二个参数的选项也有两个，分别是"cell"和"row"。默认是 cell, cell 表示 **只能是使用 checkbox 才能勾选中数据**。前面的例子已经展示效果，这里不额外展示。

现在来看看"row"的使用，代码如下：

EEDM6008.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGGrid blockId="result" checkMode="single, row">
  </EF:EFGGrid>
</EF:EFRegion>
```

效果如下：

☐	产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证
☐	wewewe	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas				1
☐	qqq	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas				1
☐	N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas				1
☐	111EE	111	USA	2017/11/0...	11		N7011							1
☒	N7022	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas				1
☐	N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	asdasdas				20
☐	#251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	asdasdas				1000
☐	T6181	Motorola	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...		021-50801...	1		2000
☐	SGH-i7161	Samsung	Korea	2007-04-06	10.14	http://www.samsun...	查询	138000000...	aaa@123.c...	fuyimin	021-50801...	1		2000
☐	T6281	SonyEric...	Japan	2007-04-06	30	http://www.sonyer...	查询	138000000...	aaa@123.c...	label	021-50801...	1		2000

随机点击一行，此行就能够被选中。

3.5.3.1.9.3.1.2 setCheckedRows 的使用

前面是通过手动去实现 Grid 中行被选中，我们其实也是可以通过使用 JS 去控制选中行信息的。这个 API 就是 `setCheckedRows`。示例代码如下：

EEDM6008.jsp

```
<EF:EFRegion id="example" title="Grid 中自定义选中项和选中项获取示例">
  <div class="row">
    <EF:EFSelect cname="选中数据行" ename="selectList" bindId="selectList">
      <EF:EFOption label="第一行" value="0"/>
      <EF:EFOption label="第二行" value="1"/>
      <EF:EFOption label="第三行" value="2"/>
      <EF:EFOption label="第四行" value="3"/>
      <EF:EFOption label="第五行" value="4"/>
      <EF:EFOption label="第六行" value="5"/>
      <EF:EFOption label="第七行" value="6"/>
      <EF:EFOption label="第八行" value="7"/>
      <EF:EFOption label="第九行" value="8"/>
    </EF:EFSelect>
  </div>
</EF:EFRegion>

<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" checkMode="single, cell">
  </EF:EFGrid>
</EF:EFRegion>
```

我们使用了两个 **Region**，一个是下拉列表，一个是 **Grid**，通过对下拉列表行的选择，相应的 **Grid** 中行将会被选中。

现在是不能把两者联动 **d** 在一起的，需要使用 **JS** 去实现两者的联动。

JS 实现如下：

EEDM6008.js

```
$(function() {
  IPLATUI.EFGrid = {
    "result": {
      // Grid 加载完成后的回调函数
      loadComplete: function () {
        var select = $("#selectList").data("kendoDropDownList");

        select.bind("change", function () {
```

```

        // 设置选中行
        resultGrid.setCheckedRows(this.value());

        // 获取选中行
        var selectedRows = resultGrid.getCheckedRows();

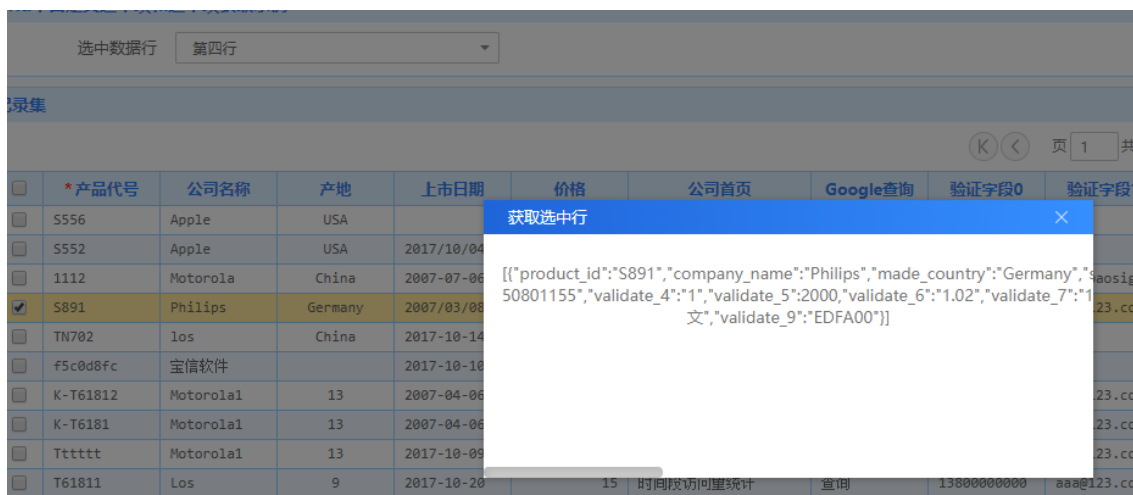
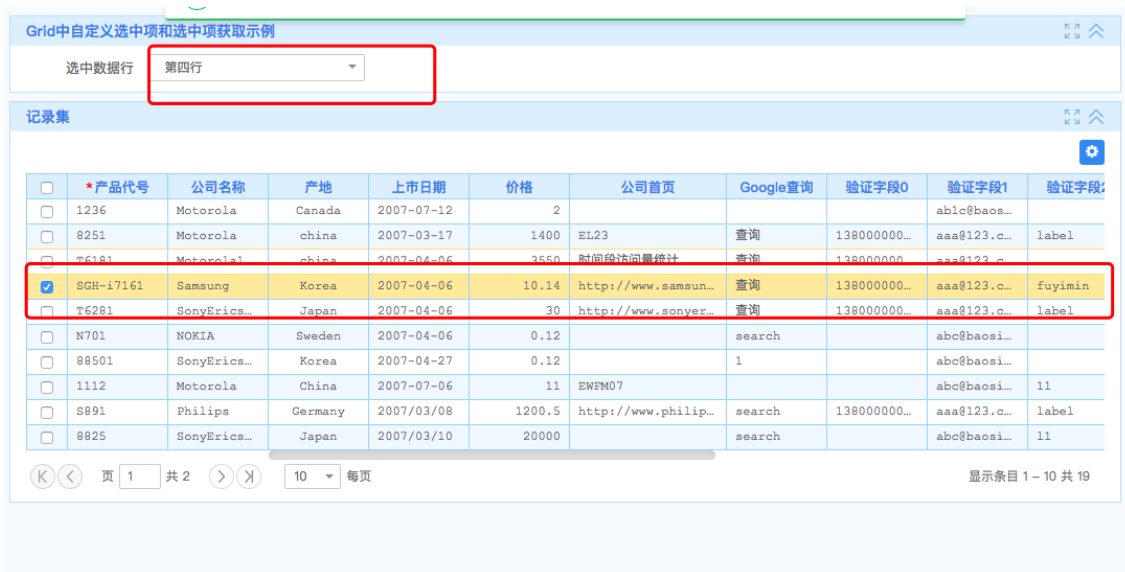
        WindowUtil({
            "title": "获取选中行",
            "content": "<div class='kendo-del-
message'>" + JSON.stringify(rows) + "</div>", //JSON 格式化输出 rows
            height: "200px",
            width: "500px"
        });
    });
}
});

```

- `loadComplete` 表示 Grid 加载完成，实现的回调方法，只会执行一次。
- `select = $("#selectList").data("kendoDropDownList")` 会获取下拉选择框的 `kendo` 实例 ([dropdownlist](#))。我们通过绑定其 `change` 事件，就能监听下拉框发生变化后的事件。
- `setCheckedRows` 是一个能够 Grid 中行选中的 API，参数可以为单个数字或者字符如：0 或者 "0"，也可以是包含数字的数组如 [0,1,2] 或者 [0,1,"2"]
- `getCheckedRows` 是能够获取选中行 Model 的 API，返回一个 list Model 的集合。

1. EFSelect使用 \$("#xxx").data("kendoDropDownList")获取kendo实例; 通过bind方法绑定change事件
2. 使用 setCheckedRows可以设置第几行被选中， getCheckedRows 获取选中行的Model 信息

页面效果，下拉选中后：



3.5.3.1.9.3.1.3 fillNode 的使用

双击 grid 中某一行，将 grid 这一行记录信息显示到对应的字段。

IPLAT.fillNode

```
IPLATUI.EFGrid = {
  "result": {
    onRowDbClick: function (e) {
      var ei = new EiInfo();
      var block = resultGrid.model2EiBlock(e.model); // 获取 resultGrid 中的信息
      ei.addBlock(block);

      IPLAT.fillNode(document.getElementById("result1"), ei);
    }, // 显示到对应 result1 的 block 中
  }
}
```

3.5.3.1.10 EEDM6009-EFGrid Excel 数据导出

kendo 能够支持前端导出 Excel 数据。

在导出前端 Excel 文本之前，需要确认项目中是否引入 **jszip.min.js**，导出 Excel 需要依赖它。如果项目中没有引入 **jszip.min.js**，可以通过加载 **iplat4j-ui-kendo** 模块进行添加。

添加路径为：

```
<c:set var="ctx" value="${pageContext.request.contextPath}" />
<script src="${ctx}/kendoui/js/jszip.min.js"></script>
```

3.5.3.1.10.1 了解什么是 OOXML

OOXML 是由[微软](#)公司为 Office 2007 产品开发的技术规范，现已成为国际文档格式标准，兼容前国际标准[开放文档格式](#)和中国文档标准“标文通”（外语简称：[UOF](#)）。

于 2006 年 12 月成为 [ECMA](#) 标准。

kendo 已经支持 OOXML，也就是说 **kendo** 能够生成 Excel 的标准文档。

这里主要针对 **EFGGrid** 情景下，如何导出 Excel 数据。

3.5.3.1.10.2 当前页所有行导出

```
var _beforeInit = function(dataItems,needColumns) {
    var data = [];

    var mate = __eiInfo.getBlock("result").getBlockMeta().getMetas();
    var _titleCells = [], index = 0, cells,_value ;

    // 处理标题信息
    for(var key in mate) {
        if (mate.hasOwnProperty(key) ) {
            if (!!needColumns) {
                if (needColumns.indexOf(key) >= 0) {
                    var _obj = mate[key];
                    _value = {value: _obj['descName']};
                    _titleCells[index++] = _value;
                }
            } else {
                var _obj = mate[key];
                _value = {value: _obj['descName']};
                _titleCells[index++] = _value;
            }
        }
    }
    data.push({cells: _titleCells});

    // 处理 grid 数据
    $.each(dataItems, function(index, item) {
        index = 0;
        cells = [];
        for (var key in mate) {
            if (mate.hasOwnProperty(key)) {
                if (!!needColumns) {
                    if (needColumns.indexOf(key) >= 0) {
```

```

        _value = {value:item[key]};
        cells[index++] = _value;
    }
} else {
    _value = {value:item[key]};
    cells[index++] = _value;
}
}
}
data.push({cells: cells})
});

return data;
};

var _exportData = function(data,name) {
    // 生成 workbook
    var workbook = new kendo.ooxml.Workbook({
        sheets: [
            {
                columns: [ { width: 100 }, { width: 200 } ],
                title: "Orders",
                rows: data
            }
        ]
    });

    // 导出 excel
    kendo.saveAs({
        dataURI: workbook.toDataURL(),
        fileName: name
    });
};

// 获取当前页的导出数据
$("#EXPORT").on("click",function() {
    var dataItems = resultGrid.getDataItems();

    var data = _beforeInit(dataItems);

    _exportData(data,"当前页.xlsx");

});

```

导出 Excel 如下:

产品代号	公司名称	产地	上市日期	价格	公司网页	Google查询	验证字段0	验证字段1	验证字段2	验证字
1111111 0	22222 22222	22222.33333.5555 22222.33333.5555	2017/08/15 00:00:00 2017-08-08	15.21 15.21	http://platform.baosight.com				https://imgsa.baidu.com/im g?image&quality=80&size=b 9999_10000&sec=150181864 3303&se=602775507&aduc06 e7cdea326e9368&imgtype=0 &src=http%3A%2F%2Fimg1 cache.msn.com%2Fcac hpic%2F8%2F89%2F8978E C67AC4531E1A58052F073E 08018.jpg	021-50801155
T6181	NOKIA	test	2017/07/11 12:34:2356	0.12	http://platform.baosight.com	查询	13800000000	aaa@123.com	https://imgsa.baidu.com/im g?image&quality=80&size=b 9999_10000&sec=150181871 9095&se=54773c1f5bc062b0f 52e7e1ba3d752&imgtype=0 &src=http%3A%2F%2Fimg1 eikaj.com%2Fresource%2F1 mg%2F9024a02928b4499a60 edc3969e31f5f.jpg	021-50801155
SGH-i7161	NOKIA	test	2017/07/11 01:45:00	2	http://www.samsung.com	查询	13800000000	aaa@123.com	https://imgsa.baidu.com/im g?image&quality=80&size=b 9999_10000&sec=150181877 2173&se=32a0b087a0c08b47 802c6507b472010&imgtype =0&src=http%3A%2F%2Fim g4.taobao.com%2Fbae %2Fupload%2F%2F%2F15 VWDEBaXXXXXX%2521%2 5210-Item_pc.jpg	021-50801155
T6281	NOKIA	China,German,USA,GB	2017/07/11 01:00:00	30	http://www.sonyericsson.com	查询	13800000000	aaa@123.com		

3.5.3.1.10.3 当前页选中行导出

```

.....
$("#EXPORT01").on("click",function() {
    var dataItems = resultGrid.getCheckedRows(), data = [];
    var data = _beforeInit(dataItems);
    _exportData(data,"Test.xlsx")

});

```

页面选中如下:

记录集

K<

页 1 共 6

>X

10 ▾

每页

显示条目 1 - 10 共 58

⚙

☐	* 产品代码	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证
☒	S556	Apple	USA		999		S556						
☐	S552	Apple	USA	2017/10/04_	999		S556						
☒	1112	Motorola	China	2007-07-06	11	EWfN07			ebc@baosig-	11	-123456789-		
☐	S891	Philips	Germany	2007/03/08	1200.5	http://www.philips...	search	13800000000	aaa@123.com	label	021-508011...	1	
☐	TN702	los	China	2017-10-14	0								
☐	F5C0d0Fc	宝信软件		2017-10-10	10		111						
☒	K-T61812	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
☐	K-T6181	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
☐	Tttttt	Motorola1	13	2017-10-09	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
☐	T61811	Los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	

导出 Excel 如下:

产品代号	公司名称	产地	上市日期	价格	公司首页	Google查	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	验证字段7
S556	Apple	USA		999	S556								0	
1112	Motorola	China	2007-07-0	11	EWF07		abc@baosii11		-12345678					
K-T61812	Motorola	13	2007-04-0	15	时间段访问查询	1380000000	aaa@123.c		021-508011			2000		127.0.0

3.5.3.1.10.4 当前页指定列导出

```

.....
$("#EXPORT02").on("click",function() {
    // 需要导出的 excel 列名
    var _needColumns = ["company_name","made_country","price"];

    var dataItems = resultGrid.getDataItems();

    var data = _beforeInit(dataItems,_needColumns);

    _exportData(data,"Test.xlsx")
});

```

导出 Excel 如下:

公司名称	产地	价格
Apple	USA	999
Apple	USA	999
Motorola	China	11
Philips	Germany	1200.5
los	China	0
宝信软件		10
Motorola1	13	15
Motorola1	13	15
Motorola1	13	15
Los	9	15

3.5.3.1.10.5 前端自定义导出

```

IPLATUI.EFGrid = {
    "result": {
        "exportGrid": {
            exportEiInfo: getExportEiInfo // 点击导出时，执行函数，产生导出的 EiInfo
        }
    }
};

function getExportEiInfo() {
    // 执行具体的业务逻辑，在前端产生要导出的 EiInfo
    var eiInfo = new EiInfo();
    eiInfo.addBlock(resultGrid.getBlockData());
    return eiInfo;
}

```

3.5.3.1.10.6 后端自定义导出

3.5.3.1.10.6.1 根据服务名和导出方法名的后端导出 后端导出

```

// 后端导出
IPLATUI.EFGrid = {
    "result": {
        height: IPLAT.Util.getGridFullScreenHeight({selector: "#result", isScrollX:
true, isPagerPositionTop: true, toolbar: true}),
        "exportGrid": {
            /**
             * 导出前的事件
             *
             * @param gridInstance kendoGrid 对象
             * @return {boolean} 是否执行导出的逻辑
             */
            beforeExport: function(gridInstance) {
                return gridInstance.getDataItems().length > 5; // false 不执行导出，true
时执行导出
            },
            // 如果配置了导出服务名和导出方法名，将调用对应的服务名和方法名，在后台产生
要导出的 EiInfo
            exportServiceName: "EEDM6000", // 配置导出服务名，默认和 grid 的 serviceName
相同
            exportMethodName: "query", // 配置导出方法名，默认和 grid 的 queryMethod
相同

            // exportFileName: "result2", // 默认值和 blockId 相同
            exportFileName: function(gridInstance) {
                // 导出的文件名包含时间戳 yyyyMMddHHmmss
                return "result" + kendo.toString(new Date(), IPLAT.FORMAT.DATE_14);
            },
            exportFileType: "xls", // 默认值是 xls，可选值为 pdf[规划中]
            exportBlockId: "result", // 默认值和 blockId 相同，导出的 EiInfo 中的指定数
据块被导出

```

```
// 如果不配置导出服务名和导出方法名，将在前端提供导出的 EiInfo;
// 如果既配置了导出服务名和导出方法名，又在前端配置了 exportEiInfo，
// 前端配置的 exportEiInfo 无效，采用后端服务生成的 EiInfo

// exportEiInfo: eiInfo, 直接指明要导出的 EiInfo
exportEiInfo: getExportEiInfo // 点击导出时，执行函数，产生导出的 EiInfo 此
处已配置服务名和导出方法名，所以前端配置的 exportEiInfo 无效
    }
};
```

页面效果:

Excel 数据导出示例 > EEDM0009													
记录集													
☐	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5
☐	abcde22	samsung9	China,all	20170101..	19.2	https://platform.h.	Africa	bascnift6.	https://a.				1
☐	abcde23	samsung9	China,all	20170101..	19.2	https://platform.h.	Oceania	bascnift6.	https://a.				1
☐	abcde	samsung9	China,all	2016/01/0.	19.2	newPage	Antarctica	bascnift6.	https://a.				1
☐	abcde24	sgscr3aac.		20170101..	11	https://platform.h.	America	as	as	as	as		0
☐	152473210.	samsung9	China,all	2016/01/0.	19.2	newPage	Africa,A.m.	bascnift6.	https://a.				1
☐	abcde222	samsung9	China,all	20170101..	19.2	https://platform.h.	Oceania	bascnift6.	https://a.				1
☐	111EE	111	GB,German	2017/11/0.	111		Africa,A.m.	abrBbaasi.	https://a.				1
☐	152473212.	samsung9	China,all	2016/01/0.	19.2	newPage	Africa,A.m.	bascnift6.	https://a.				1
☐	abcde242	samsung9	China,all	20170101..	19.2	https://platform.h.	Africa,A.m.	bascnift6.	https://a.				1
☐	2131		Rus	2016/04/2.	1 123		Africa,A.m.	123	123	13	13	31	0

导出 Excel 效果:

125%
显示 收藏

📄 文件

📊 表格

📈 图表

📝 文本

🖨 打印

📺 媒体

📌 批注

文件名称包含时间戳

🔗 协作

🌐 格式 翻译与拼写检查

产品代码	公司名称	产地	上市日期	价格	公司官网	Google位置	链接url0	链接url1	链接url2	链接url3
abcde22	sumsung@	China.all	2017/10/11242356	19.2	http://platform.basosight.com	Africa	bao\$hg\$6666666666666666 cc	https://static.bootcss.com/www/assets/img/generic.jpg	https://static.bootcss.com/www/assets/img/generic.jpg	
abcde23	sumsung@	China.all	2017/10/11242356	19.2	http://platform.basosight.com	Oceania	bao\$hg\$6666666666666666 cc	https://static.bootcss.com/www/assets/img/generic.jpg	https://static.bootcss.com/www/assets/img/generic.jpg	
abcde	ab\$cd\$e\$w\$or\$b\$nc\$e	China.all	2018/01/03 12:00:00	19.2 newPage	Antarctica America	Asia	bao\$hg\$6666666666666666 cc	https://static.bootcss.com/www/assets/img/generic.jpg	https://static.bootcss.com/www/assets/img/generic.jpg	Asia
15d473210816		China.all	2018/01/03 12:00:00	19.2 newPage	Africa,Antarctica		bao\$hg\$6666666666666666 cc	https://static.bootcss.com/www/assets/img/generic.jpg	https://static.bootcss.com/www/assets/img/generic.jpg	
abcde22	sumsung@	China.all	2017/10/11242356	19.2	http://platform.basosight.com	Oceania	bao\$hg\$6666666666666666 cc	https://static.bootcss.com/www/assets/img/generic.jpg	https://static.bootcss.com/www/assets/img/generic.jpg	
111EE	111	GB,German	2017/11/07 00:00:00	11.1		Africa,Antarctica	abc@basosight	https://static.bootcss.com/www/assets/img/generic.jpg	https://static.bootcss.com/www/assets/img/generic.jpg	
15d4732125204	sumsung@	China.all	2018/01/03 12:00:00	19.2 newPage	Africa,Antarctica		bao\$hg\$6666666666666666 cc	https://static.bootcss.com/www/assets/img/generic.jpg	https://static.bootcss.com/www/assets/img/generic.jpg	
abcde42	sumsung@	Russia	2017/10/11242356	19.2	http://platform.basosight.com	Africa,Antarctica	bao\$hg\$6666666666666666 cc	https://static.bootcss.com/www/assets/img/generic.jpg	https://static.bootcss.com/www/assets/img/generic.jpg	
2131		Russ	2018/04/08 00:00:00	1 123						31

根据方法名, 服务名导出数据

exportServiceName: "EEDM6000";
exportMethodName: "query"

根据方法名，服务名导出数据

3.5.3.1.10.6.2 根据提供的自定义查询条件，导出数据

```
// 后端导出
IPLATUI.EFGrid = {
    "result": {
        height: IPLAT.Util.getGridFullScreenHeight({selector: "#result", isScrollX:
true, isPagerPositionTop: true, toolbar: true}),
        "exportGrid": {
            // 如果配置了导出服务名和导出方法名，将调用对应的服务名和方法名，在后台产生
            要导出的 EiInfo
            exportServiceName: "EEDM6000", // 配置导出服务名，默认和 grid 的 serviceName
            相同
            exportMethodName: "query",    // 配置导出方法名，默认和 grid 的 queryMethod
            相同

            /**
             * 进行后台导出时，可以提供自定义的查询条件
            */
        }
    }
}
```

```

    * 默认使用当前 Grid 的查询条件
    * @return {EiInfo}
    */
    queryEiInfo: function (gridInstance) {
        // gridInstance.getEiInfo();
        var queryInfo = new EiInfo();
        queryInfo.set("inqu_status-0-product_id", "2131"); // 导出 product_id 为
“2131” 的数据
        return queryInfo;
    },

    // exportFileName: "result2", // 默认值和 blockId 相同
    exportFileName: function(gridInstance) {
        // 导出的文件名包含时间戳 yyyyMMddHHmmss
        return "result" + kendo.toString(new Date(), IPLAT.FORMAT.DATE_14);
    },
    exportFileType: "xls", // 默认值是 xls, 可选值为 pdf[规划中]
    exportBlockId: "result", // 默认值和 blockId 相同, 导出的 EiInfo 中的指定数
据块被导出

    }
};
```

页面效果:

记录集															
☐	产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	验证字段7
☐	abode22	sumsong9	China,all	201701011..	19.2	https://platform.b..	Africa		baosight6..	https://s..			1		NOKIA
☐	abode23	sumsong9	China,all	201701011..	19.2	https://platform.b..	Oceania		baosight6..	https://s..			1		NOKIA
☐	abode	sumsong9	China,all	2018/01/0..	19.2	newPage	Antarctica		baosight6..	https://s..			1		NOKIA
☐	abode24	sp-Chr>as6..		201701011..	11	https://platform.b..	America	ss	ss	ss	ss		0		
☐	152473210..	sumsong9	China,all	2018/01/0..	19.2	newPage	Africa,An..		baosight6..	https://s..			1		NOKIA
☐	abode222	sumsong9	China,all	201701011..	19.2	https://platform.b..	Oceania		baosight6..	https://s..			1		NOKIA
☐	111EE	111	GB,German	2017/11/0..	111		Africa,An..		abc@baosel..	https://s..			1		
☐	152473212..	sumsong9	China,all	2018/01/0..	19.2	newPage	Africa,An..		baosight6..	https://s..			1		NOKIA
☐	abode242	sumsong9	China,all	201701011..	19.2	https://platform.b..	Africa,An..		baosight6..	https://s..			1		NOKIA
☒	2131		Rus	2018/04/2..	1	123	Africa,An..	123	123	13	13	31	0		

导出 Excel 效果:

result									
A	B	C	D	E	F	G	H		
1	产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	
2	2131		Rus	2018/04/26 00:00:00	1	123	Africa,Antarctica	123	123
3									
4									
5									
6									
7									
8									
9									
10									

3.5.3.1.10.6.3 自定义导出异常

自定义导出异常

```
// 后端导出
IPLATUI.EFGrid = {
    "result": {
```

```

        height: IPLAT.Util.getGridFullScreenHeight({selector: "#result", isScrollX:
true, isPagerPositionTop: true, toolbar: true}),
        "exportGrid": {
            // 如果配置了导出服务名和导出方法名，将调用对应的服务名和方法名，在后台产生
            // 要导出的 EiInfo
            exportServiceName: "EEDM6000", // 配置导出服务名，默认和 grid 的 serviceName
            相同
            exportMethodName: "query", // 配置导出方法名，默认和 grid 的 queryMethod
            相同

            /**
             * 进行后台导出时，可以提供自定义的查询条件
             * 默认使用当前 Grid 的查询条件
             * @return {EiInfo}
             */
            queryEiInfo: function (gridInstance) {
                // gridInstance.getEiInfo();
                var queryInfo = new EiInfo();
                queryInfo.setStatus(-200);
                queryInfo.setMsg("EFTab 自定义导出异常!"); //设置异常信息
                queryInfo.set("inqu_status-0-product_id", "2131");
                return queryInfo;
            },

            // exportFileName: "result2", // 默认值和 blockId 相同
            exportFileName: function(gridInstance) {
                // 导出的文件名包含时间戳 yyyyMMddHHmmss
                return "result" + kendo.toString(new Date(), IPLAT.FORMAT.DATE_14);
            },
            exportFileType: "xls", // 默认值是 xls，可选值为 pdf[规划中]
            exportBlockId: "result", // 默认值和 blockId 相同，导出的 EiInfo 中的指定数
            据块被导出

        }
    }
};

```

导出效果:

产品代号	公司名称	产地	上市日期	价格	公司网页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	验证字段7	验证字段8
abcde22	sumsong9	China,all	20170101L..	19.2	http://platform.h..	Africa		baonight6..	https://s..			1			NOKIA
abcde23	sumsong9	China,all	20170101L..	19.2	http://platform.h..	Oceania		baonight6..	https://s..			1			NOKIA
abcde	sumsong9	China,all	2018/01/0..	19.2	newPage	Antarctica		baonight6..	https://s..			1			NOKIA
abcde24	ss aa<..		20170101L..	11	http://platform.h..	America	ss	ss	ss	ss		0			
152473210..	sumsong9	China,all	2018/01/0..	19.2	newPage	Africa,A...		baonight6..	https://s..			1			NOKIA
abcde222	sumsong9	China,all	20170101L..	19.2	http://platform.h..	Oceania		baonight6..	https://s..			1			NOKIA
111EE	111	GB,German	2017/11/0..	111		Africa,A...		abrBbaa1..	https://s..			1			NOKIA
152473232..	sumsong9	China,all	2018/01/0..	19.2	newPage	Africa,A...		baonight6..	https://s..			1			NOKIA
abcde242	sumsong9	China,all	20170101L..	19.2	http://platform.h..	Africa,A...		baonight6..	https://s..			1			NOKIA
2131		Rus	2018/04/2..	1	123	Africa,A...	123	123	13	13	31	0			

3.5.3.1.10.6.4 自定义导出的几个注意点

1. 自定义导出需开启全局配置 `exportGrid: true`，全局配置方法请参考 [UI 常见开发问题 FAQ\[Q4\]](#)
2. 如果既配置了导出服务名和导出方法名，又在前端配置了 `exportEiInfo`，前端配置的 `exportEiInfo` 无效，采用后端服务生成的 `EiInfo`

3. 页面的服务名与方法名可与导出的服务名方法名不一致，导出的数据是以前/后端返回的 **EiInfo** 里 **block** 的数据为准。
4. 后端自定义导出时，会默认带上页面的查询条件，但如果想自己设置查询条件，可参照以下代码写入 **js**。并且，如果页面查询字段与 **js** 里自己设置的查询字段重复，查询值将以 **js** 里设置的为准。

```
queryEiInfo: function () {
    var queryInfo = new EiInfo();
    queryInfo.set("inqu_status-0-product_id", "T6181");
    return queryInfo;
}
```

3.5.3.1.11 EEDM6010-EFGrid 固定列

3.5.3.1.11.1 页面功能简介

本示例页面用于说明，如何在 **EFGrid** 中设置固定列。

固定列的约定

1. 固定列在Grid的最左侧显示；
2. EiBlock中的主键列，默认是固定列。（EiColumn中设置了primaryKey为true）
3. 固定列和数据列分别在两个table中显示；（lockedTable和contentTable）
4. Grid中列开头展示的checkbox和序列号默认都是固定列
5. 如果Grid中配置有 detailInit 是不能存在固定列的

3.5.3.1.11.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.3.1.11.3 页面开发步骤

3.5.3.1.11.3.1 JSP 页面开发

EFGrid 中对列进行固定的设置很简单。可以通过配置 **EFColumn** 标签，也可以通过设置 **JS** 中的 **columns**。 **locked : true**

3.5.3.1.11.3.1.1 EFColumn

JSP

```
<EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" autoDraw="no" needAuth="false">
```

```

<EF:EFColumn ename="product_id" cname="编号[固定列]" defaultValue="101"/>
<EF:EFColumn ename="company_name" cname="公司名称" locked="true"
editType="textarea" readonly="true"/>
<EF:EFColumn ename="made_country" cname="产地"/>
<EF:EFColumn ename="sale_date" cname="上市日期" editType="datetime"/>
<EF:EFColumn ename="price" cname="价格"/>
<EF:EFColumn ename="company_page" cname="公司首页"/>
</EF:EFGGrid>
</EF:EFRegion>

```

效果如下：

公司名称	编号(固定列)	产地	上市日期	价格	公司首页
Apple	5552	USA,China	2017/10/04 00:00:00	999	
Motorola	1112	China	2007-07-06	11	EWFM07
Philips	5891	Germany	2007/03/08	1200.5	http://www.philips.com
ios	TN702	China	2017-10-14	0	
宝信软件	F5c0d8fc		2017-10-10	10	
Motorola1	K-T61812	13	2007-04-06	15	时段访问量统计
Motorola1	K-T6181	13	2007-04-06	15	时段访问量统计
Motorola1	Tttttt	13	2017-10-09	15	时段访问量统计
Los	T61811	9	2017-10-20	15	时段访问量统计

其中 `company_name` 设置了 `locked= true`，`company_name` 列被固定在表格的左侧（虽然它定义在 `product_id` 之后）。

3.5.3.1.11.3.2 Javascript 开发

通过 JS，配置 `column` 的 `locked` 属性

EEDM6010.js

```

$(function() {
    IPLATUI.EFGGrid = {
        "result": {
            columns: [
                {
                    field: "product_id",
                    title: "产品代号",
                    locked: true,
                    width: 200
                },
                {
                    field: "search",
                    title: "搜索",
                    locked: true
                }
            ]
        }
    };
})

```

页面效果如下：

记录集							
☐	产品代号	公司名称	搜索	产地	上市日期	价格	公司首页
☐	wewewe	MotorolaInj111	N70	China,German,GB	2017/11/01 00:00:00	150122	EEEE
☐	qqqq	MotorolaInj111	N70	China,German,GB	2017/11/01 00:00:00	150122	EEEE
☐	1111	MotorolaInj111	N70	China,German,GB	2017/11/01 00:00:00	150122	EEEE
☐	N7021	MotorolaInj111	N70	China,German,GB	2017/11/01 00:00:00	150122	EEEE
☐	111EE	111	N7011	USA	2017/11/07 00:00:00	11	
☐	N7022	MotorolaInj111	N70	China,German,GB	2017/11/01 00:00:00	150122	EEEE
☐	N70	MotorolaInj111	N70	China,GB	2017/11/22 00:00:00	150122	EEE
☐	8251	MotorolaInj111	N70	China,GB	2017/11/16 00:00:00	150122	
☐	T6181	Motorola	查询	china	2007-04-06	3550	时间段访问量统计
☐	SGH-I7161	Samsung	查询	Korea	2007-04-06	10.14	http://www.samsung.com

product_id, search 这两列在 js 中也设置了 locked 为 true。它们也被固定的放置在 Grid 的左侧。

非常不推荐的做法，在后台的 Java 代码中设置列的 locked 属性。这段示例代码，用于说明 EiColumn 支持动态的属性设置。

在JavaBean中设置固定列信息

```

eiColumn = new EiColumn("company_name");
eiColumn.setDescName("公司名称");
eiColumn.setNullable(true);

eiColumn.set("locked", true); // 动态的属性, bool类型
eiColumn.set("textField", "form"); // 自定义的其它属性, 字符串类型
eiColumn.set("custom", 100); // 自定义的其它属性, 数值类型

```

3.5.3.1.12 EEDM6011-EFGrid 列头合并

3.5.3.1.12.1 页面功能简介

本示例页面用于说明，如何在 EFGrid 中进行列头合并。

3.5.3.1.12.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.3.1.12.3 页面开发步骤

3.5.3.1.12.3.1 JSP 页面开发

合并列头的使用场景，将两列，或者多列合并成为一个大的列（虚拟的列，列名不能和实际的列名重复），然后在这个合并后的列中，分别展示各个子列。

我们先添加一个简单的 EFGrid:

EEDM6011.jsp

```

<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" autoDraw="no">
    </EF:EFGrid>
  </EF:EFRegion>

```

效果如下：

记录集													
☐	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5
☐	wewewe	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	qqq	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	1111	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	wewewe1	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N71		abc@baosi...	asdasdas			1
☐	111EE	111	USA	2017/11/0...	11		N7011						1
☐	N7022	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	asdasdas			20
☐	8251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	asdasdas			1000
☐	T6181	Motorola...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...		021-50801...	1	2000

为了进行列头合并，需要在 JS 中配置 IPLATUI.EFGrid 的 columns 属性。

复合列

```
$(function () {
    IPLATUI.EFGrid = {
        result: {
            columns: [
                {
                    field: "test", // 需要设置一个 field 名称
                    title: "公司信息",
                    columns: [
                        {
                            field: "company_name",
                            title: "公司名称",
                            width: 200
                        },
                        {
                            field: "company_page",
                            title: "公司首页",
                            width: 200
                        }
                    ]
                },
                {
                    field: "product_id",
                    title: "产品代号",
                    locked: true,
                    lockable: true,
                    width: 200
                },
                {
                    field: "search",
                    title: "搜索",
                    locked: true
                }
            ]
        }
    };
});
```

通过 columns 中嵌套 columns，进行列头合并处理。

页面效果：

记录集

10 条/页

	*产品代号	公司名称	搜索	证字段4	证字段5	证字段6	证字段7	证字段8	证字段9	公司信息	
										公司名称	公司首页
☐	wewewe	MotorolaL...	N70		1			NOKIA		MotorolaInj111	EEE1
☐	qqq	MotorolaL...	N70		1			NOKIA		MotorolaInj111	EEE1
☐	1111	MotorolaL...	N70		1			NOKIA		MotorolaInj111	EEE1
☐	wewewe1	MotorolaL...	N70		1			NOKIA		MotorolaInj111	EEE1
☐	N7021	MotorolaL...	N71		1			NOKIA		MotorolaInj111	EEE1
☐	111EE	111	N7011		1					111	
☐	N7022	MotorolaL...	N70		1			NOKIA		MotorolaInj111	EEE1
☐	N70	MotorolaL...	N70		20			NOKIA		MotorolaInj111	EEE
☐	0251	MotorolaL...	N70		1000			NOKIA		MotorolaInj111	EEE
☐	T6181	MotorolaL...	查询		2000		127.0.0.1	中文	EDFA00	MotorolaL	时间段访问量统计

3.5.3.1.13 EEDM6012-EFGrid 级联 Column 示例

在 EFGrid 级联示例中，我们并没有做相关对应的 Tag 封装，而是作为一个特殊场景进行处理。这里给开发人员提供两种级联方式，供开发人员选择。

3.5.3.1.13.1 纯 JS 实现

整个过程将分为三步。分为是数据源准备，column 自定义实现下拉，父子 column 级联。

1. 数据源准备

首先我们需要准备给级联的子 column 和父 column 准备好相关的 dataSource 数据集合。下边我通过对 kendo 原生实现 dropDownList 进行举例说明。

```
/*kendo drop*/
<input id="dropdownlist" />
<script>
    $("#dropdownlist").kendoDropDownList({
        dataSource: [
            { Name: "Parent1", Id: 1 },
            { Name: "Parent2", Id: 2 }
        ],
        dataTextField: "Name",
        dataValueField: "Id"
    });
</script>
```

我们在处理 dropdownlist 时候需要设置它的数据源也就是 dataSource。对于平台来说，我们设置 dropdownlist (下拉选择框)数据源通常是来自于 eiInfo，当然也可以是开发人员手动配置如上图。

我们初始化平台 Service 时候，通常会把数据源设置在 block 中。然后通过前端的 __eiInfo 获取相关的数据源。

```
/**java Service**/
public EiInfo initLoad(EiInfo inInfo) {
    EiInfo outInfo = query(inInfo);
    // generateOtherBlocks(outInfo);
    outInfo.addBlock(MockDataUtil.getProvinces("province"));
    outInfo.addBlock(MockDataUtil.getCounty("county"));
}
```

```

    return outInfo;
}

```

前端 eiInfo 数据如下:

```

__eiInfo
▼ EiInfo ⓘ
  ▼ blocks: Object
    ► county: EiBlock
    ► province: EiBlock
    ► result: EiBlock
    ► __proto__: Object
    descName: ""
    ► extAttr: Object
      msg: "查询成功, 本次查询返回4条记录! "
      msgKey: "ep.2000"
      name: ""
      status: 0
    ► __proto__: AbstractEi

```

__eiInfo 中 blocks 里面的 county 与 province 就是我们在后端设置的好的数据源。

__eiInfo 中 block 是不能被 kendo dropdownlist 所使用的，所以我们需要对其进行数据转换。

__eiInfo.getBlock('xxx').getMappedRows() 是平台提供给开发人员的把 block 转化为基本的 object 数组的方法。

```

__eiInfo.getBlock("province").getMappedRows()
▼ Array[4] ⓘ
  ▼ 0: Object
    provinceCName: "广东"
    provinceENAME: "guangDong"
    ► __proto__: Object
  ► 1: Object
  ► 2: Object
  ► 3: Object
  length: 4
  ► __proto__: Array[0]

```

在数据源的数据上, 父级 column 与子级 column 是不同的, 父级数据源是可以只是有显示值(text)和实际值 (value), 如上,"广东"就是显示给用户看的, "guangDong" 就是实际值, 用来做数据交互使用。

子级 column 是需要添加与父级的关系。如下图:

```

__eiInfo.getBlock("county").getMappedRows();
▼ Array[10]
  ▼ 0: Object
    countyCName: "深圳"
    countyENAME: "shenZheng"
    province: "guangDong"
    ▶ __proto__: Object
  ▶ 1: Object
  ▶ 2: Object
  ▶ 3: Object
  ▶ 4: Object
  ▶ 5: Object
  ▶ 6: Object
  ▶ 7: Object
  ▶ 8: Object
  ▶ 9: Object
  length: 10
  ▶ __proto__: Array[0]

```

3.5.3.1.13.1.1 column 自定义实现下拉

我们准备完数据源后，就开始考虑第二步，如何实现自定义下拉。

column

```

/**** JSP ****/
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" autoDraw="false">
    <EF:EFColumn ename="productId" cname="产品编号"/>
    <EF:EFColumn ename="productName" cname="产品名称"/>

    <EF:EFColumn ename="province" cname="省份" />

    <EF:EFColumn ename="county" cname="城市" />

    <EF:EFColumn ename="city" cname="详细地址信息" />
  </EF:EFGrid>
</EF:EFRegion>

/**** JS ****/
<script>
/**** 获取省名称 ****/
function provinceName(province) {
  var provinces = __eiInfo.getBlock("province").getMappedRows();
  for (var i = 0; i < provinces.length; i++) {
    if (provinces[i]['provinceENAME'] == province) {
      return provinces[i]['provinceCName'] || "";
    }
  }
  return "";
}

/**** 获取城市名称 ****/
function countyName(county) {
  var countyList = __eiInfo.getBlock("county").getMappedRows();
  for (var i = 0; i < countyList.length; i++) {
    if (countyList[i]['countyENAME'] == county) {

```

```

        return countyList[i]['countyCName'] || "";
    }
}
return "";
}

/** 过滤 */
function _filter(arr, field, value) {
    if ($.isArray(arr)) {
        return arr.filter(function(item) {
            return item[field] == value
        });
    }
    return [];
}

/** 设置 column */
IPLATUI.EFGrid = {
    "result": {
        columns: [
            {
                // 定义父级 column
                field: "province",
                title: "省份",
                // 重写 editor 方法,
                editor: function (container, options) {
                    var input = $('<input />');
                    input.attr("name", options.field);
                    input.appendTo(container);

                    // 获取__eiInfo 中 block 为 province 中获取省级信息数据源
                    var provinces = __eiInfo.getBlock("province").getMappedRows();

                    // 在 grid 的 cell 中创建下拉选择框
                    input.kendoDropDownList({
                        valuePrimitive: true,
                        dataTextField: "provinceCName",
                        dataValueField: "provinceEName",
                        dataSource: provinces
                    }).appendTo(container);
                },
                // 通过模版方法, 可以修改 cell 中的展示值
                template: function(dataItem) {
                    var provinceEName = dataItem['province'];
                    return provinceName(provinceEName);
                }
            }, {
                // 定义子级 column
                field: "county",
                title: "城市",
                // 重写 editor 方法
                editor: function (container, options) {
                    // container 表示正在编辑的 cell(td)
                    var tr = container.parent();

                    // resultGrid 是平台提供的一个全局的 Grid 对象, 开发人员需要根据不同
                    // 的 Grid 去获取不同的全局对象

```

```

// 这里定义的是 result，所以可以使用 resultGrid 这个全局对象
// item 表示此行的 model
var item = resultGrid.dataItem(tr);

// 获取此 model 的父级信息
// 这里是处理 “省=>市” 这种层级关系，所以“市”对应的上级一级就是“省”
var _province = item['province'];

// 获取__eiInfo 中 block 为 county 的城市信息
var countyList = __eiInfo.getBlock("county").getMappedRows();

// 过滤出来省下面对应的“市”信息
var _data = _filter(countyList, "province", _province);

// 创建好 Input 框
var input = $('<input />');

// _data[0] 表示获取的第一个元素信息
// 修改 model 中 county 值
item['county'] = _data[0]['countyEName'];
input.attr("name", options.field);
input.appendTo(container);

// 创建 dropDownList
var _dropDownList = input.kendoDropDownList({
    //valuePrimitive: true,
    // 关闭自定绑定是必须，不然后面设置的 value 与 text 值无效
    // value 与 text 表示默认值
    autoBind: false,
    dataTextField: "countyCName",
    dataValueField: "countyEName",
    dataSource: _data,
    value: _data[0]['countyEName'],
    text: _data[0]['countyCName']
}).data("kendoDropDownList");
},
template: function(dataItem) {
    var countyEName = dataItem['county'];
    return countyName(countyEName);
}
}
]
}
};
</script>

```

上边的代码是如何实现自定下拉，在这里给后面开发人员提供以下注意点：

1. JSP 中 EFGGrid 中配置的是 EFColumn
2. 在 JS 中配置 column 时候，需要重写 editor 方法
3. 在 editor 实现的方法中，子级 column 在需要对自己的数据进行过滤，过滤的条件是父级的值
4. 在子级 column 中进行 kendoDropDownList 时候 dataSource 中数据是过滤后的子级数据源

5. template 方法是让 column 的值进行转换后展示的方式

父子 column 级联

上面能够实现父级 column 与 子级 column 的下拉，不过在父级变动的时候，子级不能做出相应的变化。所以，我们最后还需要对父子进行联动。

父子级联

```

/** JS **/
<script>
IPLATUI.EFGrid = {
  "result": {
    columns: [
      .....
    ],
    loadComplete: function(grid) {
      // 此 grid 对象
      // 处理父子级联动, 通过监听 change 事件, 判断父级节点是否发生变化
      grid.dataSource.bind("change", function(e) {
        // 判断父级节点是否发生变化
        if (e.field == "province") {
          // cell_label 表示子级节点, that 表示此 grid
          var cell_label = "county", that = grid;

          // locked 表示是否为固定列
          var locked = that.isCellLocked(cell_label);
          // tr 表示 locked 和非 locked 的行, index 表示此行的第几列
          var tr, index;
          // 获取此 model 元素信息
          var item = e.items[0];
          var _uid = item.uid;
          if (locked) {
            tr = $(".k-grid-content-locked tr[data-uid='"+ _uid +"]");
            index = $(".th[data-field='"+cell_label+"']").data("index");
          } else {
            tr = $(".k-grid-content tr[data-uid='"+ _uid +"]");
            index = parseInt($(".th[data-
field='"+cell_label+"']").data("index")) - that.lockedHeader.find("th").length;
          }

          // 获取子 cell(td)
          var td = tr.children("td:eq("+index+")");

          // 触发 td.click 事件,
          td.trigger("click");
        }
      });
    }
  }
};
</script>

```

3.5.3.1.13.2 EFComboColumn 与 JS 结合

3.5.3.1.13.2.1 渲染 Grid

使用 EFGGrid 把 Grid 渲染出来，但是需要保证其中有级联关系下拉的两列中 block 数据都是全的。

示例如下：

EEDM6012.jsp

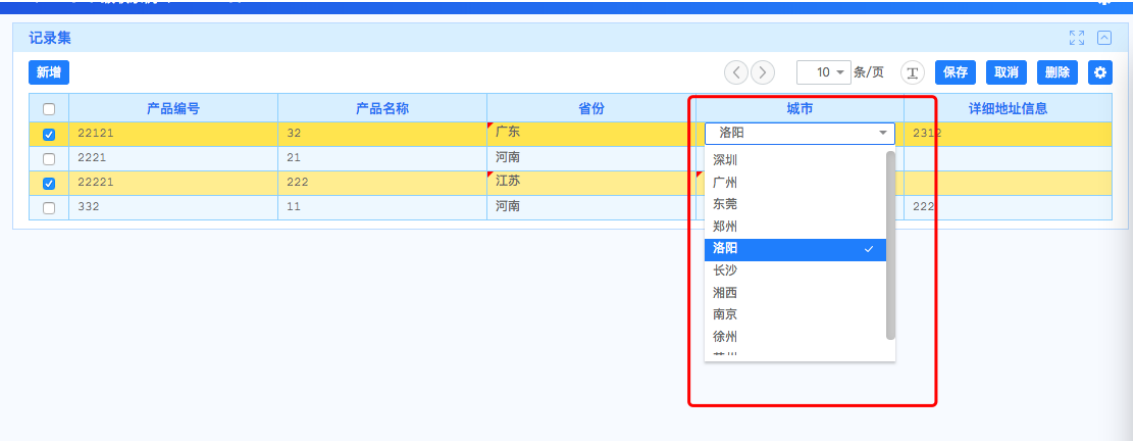
```
<EF:EFPage title="EFGGrid 级联示例">
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGGrid blockId="result" autoDraw="no" needAuth="false">
      <EF:EFColumn ename="productId" cname="产品编号"/>
      <EF:EFColumn ename="productName" cname="产品名称"/>

      <EF:EFComboColumn blockName="province" valueField="provinceEName"
textField="provinceCName" ename="province" cname="省份" />

      <EF:EFComboColumn blockName="county" valueField="countyEName"
textField="countyCName" ename="county" cname="城市" />

      <EF:EFColumn ename="city" cname="详细地址信息" />
    </EF:EFGGrid>
  </EF:EFRegion>
</EF:EFPage>
```

效果如下：



因为这里级联的列还不会进行条件的过滤，所以能够把所有的列选项都展示出来。

3.5.3.1.13.2.2 对级联列数据选项进行过滤

我们应该渲染完 Grid 了，现在需要根据条件对级联列数据选项进行过滤，这部分代码我们在 JS 中进行处理。

EEDM6012.js

```
$(function() {
  IPLATUI.EFGGrid = {
```



```

    "result": {
      columns: [
        {
          // 定义子级 column
          field: "county",
          title: "城市",
          filter: function (options) {
            var province = options.model['province'];
            if(province) {
              // 返回我们过滤后的数据集
              return _.filter(options.values, function (item) {
                return item['province'] === province;
              })
            } else {
              return [];
            }
          }
        }
      ]
    }
  };
});

```

这里需要调用平台 **filter** 接口，**filter** 接口是提供给开发人员进行对下拉选项进行数据过滤的回调方法。里面存在一个参数 **options** 对象。

在 **chrome** 中可以看出其中包含的参数信息。

```

console.log(options)
▼ {field: "county", editor: f, values: Array(10), model: i}
  ► editor: f (container, options)
    field: "county"
  ► model: i {_events: {...}, _handlers: {...}, productId: "22121", productName: "32", province: "huNan", ...}
  ► values: (10) [{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}]
  ► __proto__: Object
undefined

```

field: county 表明是哪个字段。**model** 表示是哪个 **model** 对象，方便获取本行数据。**values** 是指我们将要过滤的数据集合。

在上面例子中，处理级联数据列思路也很清晰，就是在 **values** 中把我们需要的数据过滤出来并且返回去。判断的条件是看是否具有同样的 **"province"**。

3.5.3.1.14 EEDM6013-EFGrid 弹出列示例

3.5.3.1.14.1 页面功能简介

本示例页面用于说明，如何使用 **Grid** 的弹出功能。

3.5.3.1.14.2 配置和 API 详情

参考 [EFPopupColumn](#) 和 [EFGrid](#)。

3.5.3.1.14.3 页面开发步骤

3.5.3.1.14.3.1 服务名，方法名 / 小代码弹出窗口

3.5.3.1.14.3.1.1 JSP 页面开发

EFPopupColumn 示例

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" autoDraw="no" needAuth="false">
    <EF:EFColumn ename="${pid}" primaryKey="true" required="true" locked="true"
readonly="true" cname="产品编号"/>

    <EF:EFColumn ename="company_name" cname="公司名称" required="true" />

    <EF:EFComboColumn ename="category" cname="类别" defaultValue="big"
textField="textField" valueField="valueField">
      <EF:EFOption label="大类" value="big"/>
      <EF:EFOption label="中类" value="middle"/>
      <EF:EFOption label="小类" value="small"/>
    </EF:EFComboColumn>

    <%--服务方法名查询数据
      popupType="ServiceGrid"会根据配置的 serviceName(服务名)、methodName(方法
      名)、
      columnEnames(需显示的列英文名数组)、columnCNames(需显示的列中文名数组) 查询
      并渲染出弹出的 Grid;
      backFillColumnIds 设置选中一行数据后需要回填的列数组，已逗号隔开;
      backFillFieldIds 设置回填到界面输入框的 Id，以逗号隔开;
      valueField 设置真实值对应的列名,用于向后台提交;
      resultId 设置通过本地服务获取的数据的 blockId;
      popupTitle 设置弹出窗口的标题; popupWidth 设置弹出窗口的宽度,默认 600-->
    <EF:EFPopupColumn ename="made_country" cname="产地[ServiceGrid]"
popupType="ServiceGrid" resultId="result"
      popupTitle="弹出服务 Grid" popupWidth="400"
      serviceName="EEDM6013" methodName="query"
      columnEnames="company_name,made_country,price"
      columnCNames="公司名称,产地,价格"
      backFillFieldIds="company_name,made_country,price"
      backFillColumnIds="company_name,made_country,price"
      valueField="made_country"
      copy="true" resizable="true"
      filterPopupGrid="false"/>
    <EF:EFColumn ename="price" cname="价格" />

    <EF:EFColumn ename="validate_3" cname="自定义 DIV 弹出 Grid" readonly="readonly"
  />
```

```

        <EF:EFColumn ename="validate_4" cname="自定义DIV 弹出 iframe"
readonly="readonly" />

        <%--小代码查询数据
        popupType="DynamicGrid" 会根据配置的 codeName (小代码类别名称) 查询并渲染出弹出
        的 Grid,
        columnEnames/columnCnames/backFillFieldIds/backFillColumnIds 等属性含义与
        popupType="ServiceGrid" 时相同--%>
        <EF:EFPopupColumn ename="company_page" cname="公司首页[DynamicGrid]"
            textField="FORM_CNAME"
            valueField="FORM_ENAME"
            codeName="iplatSelectWithGrid"
            columnEnames="FORM_ENAME, FORM_CNAME, MODULE_ENAME_1"
            columnCnames="画面英文名, 画面中文名, 一级模块名"

            backFillFieldIds="company_name,made_country"
            backFillColumnIds="FORM_ENAME, FORM_CNAME"
            popupType="DynamicGrid"/>

        <EF:EFColumn ename="sale_date" cname="时刻" editType="datetime" interval="15"
required="true"
            parseFormats="['yyyy-MM-dd', 'yyyy/MM/dd']"/>
    </EF:EFGrid>
</EF:EFRegion>

```

serviceName、**methodName**、**resultId** 这三个参数能够定位到服务返回的 **block** 数据信息。**columnEnames** 是表明在 **block** 中的哪些列需要进行展示，**columnCnames** 表明它们的中文名称，都是使用“,” 对它们进行分割。这里需要让 **columnEnames** 与 **columnCnames** 中的字段名称一一对应。这里涉及到 [EFPopupColumn](#) 比较多的属性。**请开发人员使用前一定要好好阅读 [EFPopupColumn](#)**

3.5.3.1.14.3.1.2 Service 服务开发 ServiceEEDM6013

```

public class ServiceEEDM6013 extends ServiceBase {

    public EiInfo initLoad(EiInfo inInfo) {

        return query(inInfo);
    }

    public EiInfo query(EiInfo inInfo) {
        EiInfo outInfo = super.query(inInfo, "EE10.query", new EE10());

        // 额外的 EiBlock country
        outInfo.addBlock(MockDataUtil.getCountryBlock("country", 6));

        return outInfo;
    }
    .....
}

```

3.5.3.1.14.3.1.3 页面展示效果

popupType = "ServiceGrid".

* 公司名称	类别	* 产地[ServiceGrid]	价格	自定义DIV弹
Philips		Germany	1200.5	
ss222				
Apple				
Motorola				
Philips				
los				
宝信软件				
Motorola1				
Motorola1				

弹出服务Grid		
(K) (L)	页 1	共 6
(M) (N)	10	每页
公司名称	产地	价格
ss222		222
ss222		222
Apple	USA,China	999
		0
Motorola	China	11
Philips	Germany	1200.5
los	China	0
宝信软件		10
Motorola1	13	15
Motorola1	13	15

此时，backFillColumnIds 是弹出 grid 的字段，backFillFieldIds 是对应的 grid 的需回填字段，这两个属性一一对应。例如：

```
backFillFieldIds="company_name,made_country,price"
backFillColumnIds="company_name,made_country,price"
```

popupType = "DynamicGrid".

* 公司名称	类别	* 产地[ServiceGrid]	价格	自定义DIV弹出G...	自定义DIV弹出if...	公司首页IDyna...	* 时刻
ED0810		索引维护页面	1200.5			索引维护页面	2017-10-26 00:00...
ss222			222				
Apple		USA,China	999				
Motorola		China	11				
Philips		Germany	1200.5				
los		China	0				
宝信软件			10				
Motorola1		13	15				
Motorola1		13	15				

详情		
(K) (L)	页 1	共 41
(M) (N)	10	每页
显示条目 1 - 10 共 410		
画面英文名	画面中文名	一级模块名
EDCC03	配置信息	ED
EDCM00	代码类别维护	ED
EDCM01	代码项维护	ED
ED0810	索引维护页面	ED
ED08T0	表头信息维护页面	ED
ED08T1	表数据项维护页面	ED
EDFA00	页面信息管理	ED
EDFA01	按钮信息管理	ED
EDNM01	主机管理	ED
EDNM02	节点管理	ED

3.5.3.1.14.3.2 自定义弹出 Grid

3.5.3.1.14.3.2.1 JSP 页面开发

EFPopupColumn 示例

```
//渲染弹出的 Grid
<div id="ef_popup_grid" style="display: none">
  <EF:EFRegion id="inquA" title="查询条件">
    <div class="row">
      <EF:EFInput ename="inqu_status-0-product_id" cname="产品代号" />
      <EF:EFInput ename="inqu_status-0-company_name" cname="公司名称" />
    </div>
  </EF:EFRegion>
</div>
```

```

    <EF:EFRegion id="resultA" title="记录集">
        <EF:EFGrid blockId="resultA" checkMode="single" serviceName="EEDM6019"
autoDraw="false" > <!-- EFGrid 默认 autoDraw="yes" -->
            <EF:EFColumn ename="product_id" locked="true" cname="产品编号"/> <!-- 动态
设置的列名 -->
            <EF:EFColumn ename="company_name" locked="true" cname="公司名称"/>

            <!-- 设置日期列时间的间隔为 15 分钟 -->
            <EF:EFColumn ename="sale_date" cname="时刻" required="true"
editType="datetime" interval="15"
                        parseFormats="['yyyy-MM-dd','yyyy/MM/dd']"
dateFormat="yyyyMMddHHmmssfff" />
            <EF:EFColumn ename="on_date" cname="上市日期" required="true"
editType="date" dateFormat="yyyyMMdd"/>

            <!-- 格式化数字类型数据 注意要在 js 中 dataBinding 转换数据类型 -->
            <EF:EFColumn ename="price" cname="价格" format="{0:c2}" />
            <EF:EFColumn ename="search" cname="搜索"/>
        </EF:EFGrid>
        <div class="k-window-save k-popup-save">
            <EF:EFButton cname="确定" ename="ef_popup_grid_fillback" layout="1"
class="k-button"></EF:EFButton>
        </div>
    </EF:EFRegion>
</div>

```

3.5.3.1.14.3.2.2 js 示例

Grid 设置以及按钮的事件绑定

```

//设置 Grid
"result": {
columns:[{
    field: "validate_3", // popupColumn, 暂不支持 DynamicGrid
    enable: true,
    readonly: true,
    hidden: false,
    locked: false,
    editor: function (container,param) {
        // 设置产生弹框 model
        if (container.hasClass("fake-edit")) {
            container.removeClass("fake-edit");
        } else {
            editorModel = param.model;
            IPLAT.Popup.popupContainer({
                containerId: "ef_popup_grid",
                textElement: $(container),
                width: 600,
                title: "详细说明"
            });
        }
    }
}]
}

```

```

    }
    }
    ...

//绑定 Grid 中“确定”按钮事件
"resultA": {
    loadComplete: function (grid) {
        // grid 完全加载
        $("#ef_popup_grid_fillback").on("click",function (e) {
            var checkRows = grid.getCheckedRows();
            // 获取第一列
            var model = checkRows[0];
            if (model) {
                // 重置 resultAGrid 中数据
                resultAGrid.dataSource.page(1);

                var popupGridWindow = $("#ef_popup_grid").data("kendoWindow");
                // 关闭 window
                popupGridWindow.close();

                //回填数据

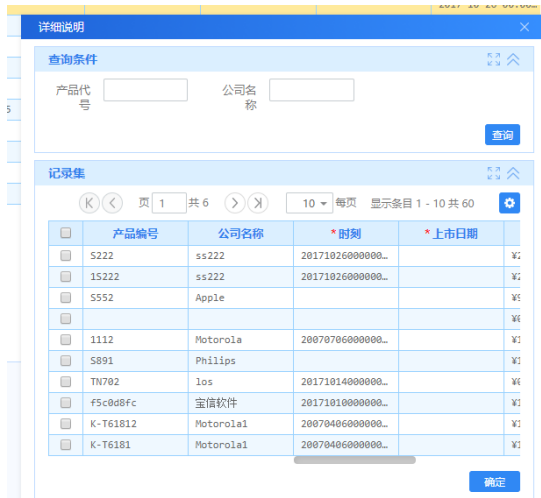
resultGrid.setCellValue(editorModel,"validate_3",model['company_name']);

            } else {
                WindowUtil({
                    "title": "回填数据",
                    "content": "<div class='kendo-del-message'>没有选中数据</div>"
                });
            }
        });
    }
}
};

```

Grid 中设置了“确定”按钮，勾选一行后，点击“确定”后，即可回填到相应的列中（此时 `checkMode="single"`）。

3.5.3.1.14.3.2.3 页面效果如下所示



3.5.3.1.14.3.3 自定义弹出 iframe

3.5.3.1.14.3.3.1 JSP 页面开发

EFPopupColumn 示例

```
//渲染 iframe
<EF:EFWindow url="${ctx}/web/EEDM6010" id="ef_popup_iframe"
modal="false"></EF:EFWindow>
```

3.5.3.1.14.3.3.2 js 示例

iframe 设置

```
//设置 iframe
"result": {
    {
        field: "validate_4", // popupColumn, 暂不支持 DynamicGrid
        enable: true,
        readonly: true,
        hidden: false,
        locked: false,
        editor: function (container,param) {
            if (container.hasClass("fake-edit")) {
                container.removeClass("fake-edit");
            } else {
                // 设置产生弹框 model
                editorModel = param.model;
                // 弹出弹出框
                IPLAT.Popup.popupContainer({
                    containerId: "ef_popup_iframe",
                    textElement: $(container),
                    width: 600,
                    title: "详细说明"
                });
            }
        }
    }
}

...
//关闭时回填数据
"result": {
    loadComplete: function(grid){
        var windowFrame = $("#ef_popup_iframe").data("kendoWindow");
        // 绑定 Frame 的关闭事件, 把数据进行回填
        windowFrame.bind("close",function(e) {
            var checkRows = window.frames[0].resultGrid.getCheckedRows();
            var model = checkRows[0];
            if (model) {
                resultGrid.setCellValue(editorModel, "validate_4", model['company_name']);
            }
        });
    }
}
```

```
}

```

勾选一行后，关闭窗口，即可回填到相应列

3.5.3.1.14.3.3 页面效果如下所示



3.5.3.1.15 EEDM6015-EFGrid 后端无 Count 分页

3.5.3.1.15.1 页面功能简介

本示例页面针对平台中后端没有 Count 返回的情况下，前端自定义实现分页的情形。

3.5.3.1.15.2 配置和 API 详情

参考 [EFGrid](#) 中 showCount 属性。

3.5.3.1.15.3 页面开发步骤

3.5.3.1.15.3.1 当 showCount 为“false”

JSP 页面开发：

EEDM6015.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" showCount="false"> <!-- EFGrid 默认的查询方法是 query -->
  </EF:EFGrid>
</EF:EFRegion>
```

注：我们需要在 EFGrid 中设置 showCount 属性为 false，这样前端不会向后端请求 count 总数。在这种情况下，前端 Grid 页面是不会展示总记录数和总页数的。页面只能通过 [上一页](#)，[下一页](#) 去实现分页，也就无法直接定位到最后一页。

页面展示效果：

记录集													
☐	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5
☐	wewewe	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	qqq	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	1111	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	wewewe1	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N71		abc@baosi...	asdasdas			1
☐	111EE	111	USA	2017/11/0...	11		N7011						1
☐	N7022	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	asdasdas			20
☐	8251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	asdasdas			1000
☐	T6181	Motorola...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...	021-50801...	1		2000

3.5.3.1.15.3.2 当 showCount 为“true”时： JSP 页面开发：

EEDM6015.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" showCount="true"> <!-- EFGrid 默认的查询方法是 query -->
  </EF:EFGrid>
</EF:EFRegion>
```

我们设置 showCount 为 true 时，需要注意：当我们在页面初始化数据，也就是在 initLoad 方法里面初始化数据的时候，需要手动的在相应的 block 块的 attr 中添加 showCount = "true"和 count=xxx

EEDM6015.java

```
public EiInfo initLoad(EiInfo inInfo) {
    EiInfo outInfo = new EiInfo();
    EiBlock resultBlock = query(inInfo).getBlock("result");
    resultBlock.getAttr().put("count", 32); //32 是模拟的一个数字，表示页面中 Grid 展示的总条数，实际开发中，需要开发人员自己去获取
    resultBlock.getAttr().put("showCount", "true");
    outInfo.setBlock(resultBlock);
    return outInfo;
}
```

页面效果如下所示：

记录集													
☐	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5
☐	wewewe	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	qqq	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	1111	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	wewewe1	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N71		abc@baosi...	asdasdas			1
☐	111EE	111	USA	2017/11/0...	11		N7011						1
☐	N7022	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	asdasdas			1
☐	N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	asdasdas			20
☐	8251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	asdasdas			1000
☐	T6181	Motorola...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...	021-50801...	1		2000

3.5.3.1.16 EEDM6016-EFGrid 自定列可配置显示的位置

3.5.3.1.16.1 页面功能简介

本示例页面针对平台中自定列设置位置，将 columns 从左向右排列位置。

3.5.3.1.16.2 页面开发步骤

3.5.3.1.16.2.1 JSP 页面开发

EEDM6016.jsp

```
<EF:EFRegion id="result" title="记录集" >
  <EF:EFGrid blockId="result" autoDraw="yes">
    </EF:EFGrid>
  </EF:EFRegion>
```

3.5.3.1.16.2.2 JS 开发

EEDM6016.js

```
$(function () {
  IPLATUI.EFGrid = {
    "result": {
      columns: [{
        field: "btn",
        title: "行操作按钮前置",
        locked: true, // 注意固定列存在的时候，locked 列才会前置
        position: "front",
        template: "<button class='custom i-btn-lg'>自定义的按钮</button>"
      }, {
        field: "btn1",
        title: "行操作按钮在后",
        template: "<button class='custom i-btn-lg'>自定义的按钮</button>"
      }
    ]
  };
});
```

我们在JS中给通过给column配置 **position: front** 这个属性来调节此column的位置，此column会根据它在JS中配置顺序从左往右进行排列。

如上面的示例代码，按钮：“行操作按钮前置”配置了position: "front"，因此按钮前置。

如果给按钮：“行操作按钮在后”也配置 locked: true，则此按钮会在固定列里出现，排在固定列的最后一列，否则就排在非固定列的最后一列

此属性在不能改变固定列和非固定列的左右排列顺序。

3.5.3.1.16.2.3 页面展示效果

行操作按钮前置	产品代号	公司名称	段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	验证字段7	验证字段8	验证字段9	行操作按钮在后
自定义的按钮	weewewe	Motorola...		abc@baosi...	asdasdas			1			NOKIA		自定义的按钮
自定义的按钮	qqq	Motorola...		abc@baosi...	asdasdas			1			NOKIA		自定义的按钮
自定义的按钮	1111	Motorola...		abc@baosi...	asdasdas			1			NOKIA		自定义的按钮
自定义的按钮	weewew1	Motorola...		abc@baosi...	asdasdas			1			NOKIA		自定义的按钮
自定义的按钮	N7021	Motorola...		abc@baosi...	asdasdas			1			NOKIA		自定义的按钮
自定义的按钮	111EE	111						1					自定义的按钮
自定义的按钮	N7022	Motorola...		abc@baosi...	asdasdas			1			NOKIA		自定义的按钮
自定义的按钮	N70	Motorola...		abc@baosi...	asdasdas			20			NOKIA		自定义的按钮
自定义的按钮	8251	Motorola...		abc@baosi...	asdasdas			1000			NOKIA		自定义的按钮
自定义的按钮	T6181	Motorola...	000...	aaa@123.c...		021-50801...	1	2000		127.0.0.1	中文	EDFA00	自定义的按钮

3.5.3.1.17 EEDM6017-EFGrid 自定义查询

3.5.3.1.17.1 页面功能简介

本示例页面用于说明，如何去自定义控制查询和查询参数。

3.5.3.1.17.2 配置和 API 详情

参考 [EFGGrid](#) 介绍。

3.5.3.1.17.3 页面开发步骤

3.5.3.1.17.3.1 业务场景说明

大家都知道，平台的前后端交互的数据格式是 **EiInfo**。前面的示例页面中，前端都是使用平台封装的方式去组装查询查询，现在给大家介绍一个平台提供的可供查询的接口。

query 是一个平台给开发人员提供的一个组装查询参数的 JS 接口。配置在 **IPLATUI.EFGrid** 对象的 **JS** 中。

3.5.3.1.17.3.2 页面开发

我们创建一个带有查询区和数据区的页面 **EEDM6017.jsp**，并且在查询区注册一个“查询”按钮，其 **id="F1"**，不清楚的小伙伴请参考：[EEDM6004-注册 EFGrid 功能按钮](#)。

EEDM6017.jsp

```
<EF:EFPage title="单表增删改查示例">
  <EF:EFRegion id="inqu" title="查询条件">
    <div class="row">
      <EF:EFInput cname="公司名称" ename="inqu_status-0-company_name" />
      <EF:EFSelect cname="产品代号" ename="inqu_status-0-product_id"
defaultValue="">
        <EF:EFOption label="全部" value="" />
        <EF:EFTableOption tableName="tee10" valueField="product_id"
textField="product_id"/>
      </EF:EFSelect>
    </div>
  </EF:EFRegion>
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result">
    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFPage>
```

```
</EF:EFRegion>
</EF:EFPage>
```

效果如下：



现在我们点击"查询"，页面没有任何的反应。我们接着来开发我们的 JS，我们创建 EEDM6017.js。

EEDM6017.js

```
$(function () {
    IPLATUI.EFGrid = {
        "result": {
            loadComplete: function (grid) {
                $("#F1").on("click", function () {
                    grid.dataSource.page(1);
                });
            }
        }
    };
});
```

我们在 `loadComplete` (不懂得小伙伴请参考:[EEDM6000-单表增删改查](#)中升级指南)方法中，给"查询"绑定事件，回调方法中调用 `Grid` 的查询 `grid.dataSource.page(1)`。这种方式是基于平台的封装。我们只能在查询区域中的 `form` 表单中填写内容，这些内容将会作为查询条件去后端过滤数据。这样能够适应大部分的场景，但是灵活性太差，没法实现定制化的需求。

我们使用平台提供 `query` 接口去实现定制化的一些需求。

EEDM6017.js

```
$(function () {
    IPLATUI.EFGrid = {
        "result": {
            query: function () {
                var ei = new EiInfo();
                return ei;
            }
        }
    };
});
```

```

    },
    loadComplete: function (grid) {
        $("#F1").on("click", function () {
            grid.dataSource.page(1);
        });
    }
};
});

```

我们在 `resultGrid` 中配置了 `query` 接口，我们当我们向后端发送查询请求时，会把 `query` 回调函数里面返回的 `EiInfo` 作为唯一的参数返回给后端服务，作为查询条件。

所以我们调整一下上面的 JS 代码，就能实现查询。

EEDM6017.js

```

$(function () {
    IPLATUI.EFGrid = {
        "result": {
            query: function () {
                var ei = new EiInfo();
                ei.set('inqu_status-0-company_name', $('#inqu_status-0-company_name').val());
                ei.set('inqu_status-0-product_id', $('#inqu_status-0-product_id').val());
                return ei;
            },
            loadComplete: function (grid) {
                $("#F1").on("click", function () {
                    grid.dataSource.page(1);
                });
            }
        }
    };
});

```

效果如下：

单表增删改查示例 > EEDM6017

查询条件

公司名称: Motorola 产品代号: 全部

查询

记录集

*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证
vwewew	Motorola	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baoci...	asdasdas				1
9999	Motorola	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baoci...	asdasdas				1
1111	Motorola	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baoci...	asdasdas				1
vwewew1	Motorola	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baoci...	asdasdas				1
N7021	Motorola	China, Ger...	2017/11/0...	150122	EEE1	N71		abc@baoci...	asdasdas				1
N7022	Motorola	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baoci...	asdasdas				1
N70	Motorola	China, GB	2017/11/2...	150122	EEE	N70		abc@baoci...	asdasdas				20
8251	Motorola	China, GB	2017/11/1...	150122	EEE	N70		abc@baoci...	asdasdas				1000
T6181	Motorola	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...		021-50801...	1		2000
123	Motorola	Canada	2007-07-12	2				abio@baoc...		-12345678...			0

3.5.3.1.18 EEDM6018-EFGrid 自定义按钮事件处理示例

3.5.3.1.18.1 页面功能简介

本示例页面用于说明，如何对平台中自定义的按钮添加点击事件。

3.5.3.1.18.2 页面开发步骤

3.5.3.1.18.2.1 JSP 页面开发

JSP 示例

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result">
    </EF:EFGrid>
  </EF:EFRegion>
```

3.5.3.1.18.2.2 JS 开发

JS 示例

```
$(function () {

  IPLATUI.EFGrid = {
    "result": {
      columns: [
        {
          field: "company_name",
          title: "覆盖公司名",
          headerTemplate: "<span style='color: red'>覆盖公司名</span>",
          nullable: false,
          width: 200
        },
        {
          field: "validate_0",
```

```

        title: "验证字段 0",
        enable: false,
        template: "<span><button class='i-btn-sm test'>测试</button></span> "
    },
    {
        field: "search",
        title: "搜索",
        nullable: false,
        headerTemplate: function () {
            return "<span style='color: blue'>搜索</span>";
        },
        template: "<span style='color: blue'>#=search#</span> "
    },
    onCellClick: function (e) {
        if (e.field === 'validate_0') {
            WindowUtil({
                "title": "测试",
                "content": "<div class='kendo-del-message'>点击" + (e.row+1) + "行</div>"
            })
        }
    }
};
});

```

注意：开发人员在 **td** 中使用 **button** 时候，**button** 的高度不要超过 24px，不然会造成 **Grid** 的高度溢出。

onCellClick 是平台提供，用于监听 **Grid** 中单元格的点击事件。

3.5.3.1.18.2.3 页面展示效果

记录集									
	* 产品代号	覆盖公司名	产地	上市日期	价格	公司首页	搜索	验证字段0	验证字段1
☐	15222	ss222		2017-10-26...	222			测试	
☐	S552	Apple	USA,China	2017/10/04...	999		S556	测试	
☐	1112	Motorola	China	2007-07-06	11	EWFM07		测试	abc@baosig-
☐	TN702	los	China	2017-10-14	0			测试	
☐	f5c0d8fc	宝信软件		2017-10-10	10		111	测试	
☐	K-T61812	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	测试	aaa@123.com
☐	K-T6181	Motorola1	13	2007-04-06			查询	测试	aaa@123.com
☐	Tttttt	Motorola1	13	2017-10-09			查询	测试	aaa@123.com
☐	T61811	Los	9	2017-10-20			查询	测试	aaa@123.com
☐	XXX1	Motorola	Sweden	2007-03-17			查询	测试	aaa@123.com

3.5.3.1.19 EEDM6019-高度自定义示例

3.5.3.1.19.1 页面功能简介

开发过程中，可能会遇到要修改 **Grid** 的高度的情况，本示例页面就如何自定义 **Grid** 的高度，提供两种方式：**tag** 配置和 **js** 配置。

3.5.3.1.19.2 配置和 API 详情

参考 [EFGGrid 表格](#) 中 **height** 属性的描述。

3.5.3.1.19.3 页面开发步骤

3.5.3.1.19.3.1 JSP 页面开发

Tag 修改 grid 高度

```
<EF:EFPage title="Grid 高度自定义示例">
  <EF:EFRegion id="result" title="记录集(自适应)">
    <EF:EFGrid blockId="result">
    </EF:EFGrid>
  </EF:EFRegion>

  <EF:EFRegion id="resultA" title="记录集(高度=300)">
    <EF:EFGrid blockId="resultA" queryMethod="query" height="300"
toolbarConfig="{hidden:true}">
    </EF:EFGrid>
  </EF:EFRegion>

  <EF:EFRegion id="resultB" title="记录集(高度=500)">
    <EF:EFGrid blockId="resultB" queryMethod="query" height="500"
toolbarConfig="{hidden:true}">
    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFPage>
```

3.5.3.1.19.3.2 JS 开发

JS 修改 grid 高度

```
/**JSP**/
<EF:EFPage title="Grid 高度自定义示例">
  <EF:EFRegion id="result" title="记录集(自适应)">
    <EF:EFGrid blockId="result">
    </EF:EFGrid>
  </EF:EFRegion>

  <EF:EFRegion id="resultA" title="记录集(高度=300)">
```



```

    <EF:EFGGrid blockId="resultA" queryMethod="query" toolbarConfig="{hidden:true}">
    </EF:EFGGrid>
</EF:EFRegion>

<EF:EFRegion id="resultB" title="记录集(高度=500)">
    <EF:EFGGrid blockId="resultB" queryMethod="query" toolbarConfig="{hidden:true}">
    </EF:EFGGrid>
</EF:EFRegion>
</EF:EFPage>

/**JS***/
<script>
IPLATUI.EFGGrid = {
    "resultA": {
        height: 300
    },
    "resultB": {
        height: 500
    }
};
</script>

```

3.5.3.1.19.3.3 页面展示效果

Grid 高度自适应:

记录集(自适应)												
* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证?
15222	ss222		2017-10-26...	222								
5552	Apple	USA,China	2017/10/04...	999		5556						
1112	Motorola	China	2007-07-06	11	EMFN07			abc@baosig...	11	-123456789...		
TN702	los	China	2017-10-14	0								
f5cd8fc	宝信软件		2017-10-10	10		111						
K-T61812	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
K-T6181	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
Tttttt	Motorola1	13	2017-10-09	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T61811	los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
XXX1	Motorola	Sweden	2007-03-17	1400	http://www.nokia.com	查询	13800000000	aaa@123.com	lab	021-508011...	1	

Grid (height=300):

记录集(高度=300)												
* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证?
15222	ss222		2017-10-26...	222								
5552	Apple	USA,China	2017/10/04...	999		5556						
1112	Motorola	China	2007-07-06	11	EMFN07			abc@baosig...	11	-123456789...		
TN702	los	China	2017-10-14	0								
f5cd8fc	宝信软件		2017-10-10	10		111						
K-T61812	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
K-T6181	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
Tttttt	Motorola1	13	2017-10-09	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T61811	los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
XXX1	Motorola	Sweden	2007-03-17	1400	http://www.nokia.com	查询	13800000000	aaa@123.com	lab	021-508011...	1	

Grid (height=500):

记录集(高度=500)

*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证
15222	ss222		2017-10-26...	222								
5552	Apple	USA,China	2017/10/04...	999		5556						
1112	Motorola	China	2007-07-06	11	EwFM07			abc@easig...	11		-123456789...	
TN702	los	China	2017-10-14	0								
f5c0d8fc	宝信软件		2017-10-10	10		111						
K-T61812	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
K-T6181	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
Tttttt	Motorola1	13	2017-10-09	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T61811	Los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
XXX1	Motorola	Sweden	2007-03-17	1400	http://www.nokia.com	查询	13800000000	aaa@123.com	lab	021-508011...	1	

3.5.3.1.20 EEDM6020-EFGrid 图片展示

3.5.3.1.20.1 页面功能简介

本示例页面针对在 **Grid** 中某些需要展示图片的列，提供两种场景，具体介绍如下。

3.5.3.1.20.2 页面开发步骤

3.5.3.1.20.2.1 column 中展示图片

column 表示 **Grid** 中的列，列里面除了可以存放 **text**, **button** 等常规元素外，我们也可以存放图片。

创建 EEDM6020.jsp:

EEDM6020.jsp

```
<EF:EFRegion id="result" title="通用图片展示">
  <EF:EFGrid blockId="result" autoDraw="no">
    <EF:EFColumn ename="product_id" cname="产品编号" copy="true"/>
    <EF:EFColumn ename="company_name" cname="公司名称" />
    <EF:EFColumn ename="made_country" cname="产地" />
    <EF:EFColumn ename="validate_2" cname="产品图标" readonly="true" /> //
    validate_2 字段中存储图片的 URL 路径
    <EF:EFColumn ename="sale_date" cname="时刻" editType="datetime"
      parseFormats="['yyyy-MM-dd','yyyy/MM/dd']"/>
    <EF:EFColumn ename="sale_date1" cname="上市日期" editType="date"/>
    <EF:EFColumn ename="price1" cname="价格 1"/>
  </EF:EFGrid>
</EF:EFRegion>
```

展示效果如下，“产品图标”展示的是图片的 **URL** 路径。

通用图片展示

产品编号	公司名称	产地	产品图标	时刻	上市日期	价格1
wewewe	MotorolaInj111	China, German, GB	https://static.bootcss.com/www/assets/img/typescript.png			
qqq	MotorolaInj111	China, German, GB	https://static.bootcss.com/www/assets/img/typescript.png			
1111	MotorolaInj111	China, German, GB	https://static.bootcss.com/www/assets/img/typescript.png			
wewewe1	MotorolaInj111	China, German, GB	https://static.bootcss.com/www/assets/img/typescript.png			
N7021	MotorolaInj111	China, German, GB	https://static.bootcss.com/www/assets/img/typescript.png			
111EE	111	USA	https://static.bootcss.com/www/assets/img/typescript.png			
N7022	MotorolaInj111	China, German, GB	https://static.bootcss.com/www/assets/img/typescript.png			
N70	MotorolaInj111	China, GB	https://static.bootcss.com/www/assets/img/typescript.png			
8251	MotorolaInj111	China, GB	https://static.bootcss.com/www/assets/img/typescript.png			
T6181	MotorolaInj111	china	https://static.bootcss.com/www/assets/img/typescript.png	2007-04-06 00:00...		

这个不是我们想要的效果。

现在，我们需要通过使用 `template`(请参考: [EFGGrid 的列模版示例](#))的方式，来展示图片。

创建 EEDM6020.js:

EEDM6020.js

```
(function () {
  IPLATUI.EFGGrid = {
    "result": {
      columns: [
        {
          field: "validate_2",
          title: "产品图标",
          template: "<span><img src='#:validate_2#' width='150'
height='75'></span>",
          width: 150
        }
      ]
    }
  };
})();
```

把 `validate_2` 作为图片的 `src` 路径，这样就能在 `Grid` 中展示图片。效果如下：

通用图片展示

☐	产品编号	公司名称	产地	产品图标	时刻	上市日期	价格1
☐	wewewe	MotorolaInj111	China, German, GB	TypeScript 中文手册			
☐	qeqq	MotorolaInj111	China, German, GB	TypeScript 中文手册			
☐	1111	MotorolaInj111	China, German, GB	TypeScript 中文手册			
☐	wewewe1	MotorolaInj111	China, German, GB	TypeScript 中文手册			
☐	N7021	MotorolaInj111	China, German, GB	TypeScript 中文手册			
☐	111EE	111	USA	TypeScript 中文手册			
☐	N7022	MotorolaInj111	China, German, GB	TypeScript 中文手册			
☐	N70	MotorolaInj111	China, GB	TypeScript 中文手册			

3.5.3.1.20.2.2 column 弹出图片

上面的示例页面中，我们是把图片作为一个单独的资源存放在 column 中，但是图片占用的页面中面积比较大，导致页面不美观。

通过设置缩略图，然后使用弹出框对图片进行展示，是一个不错的选择。

对 EEDM6020.js 进行修改：

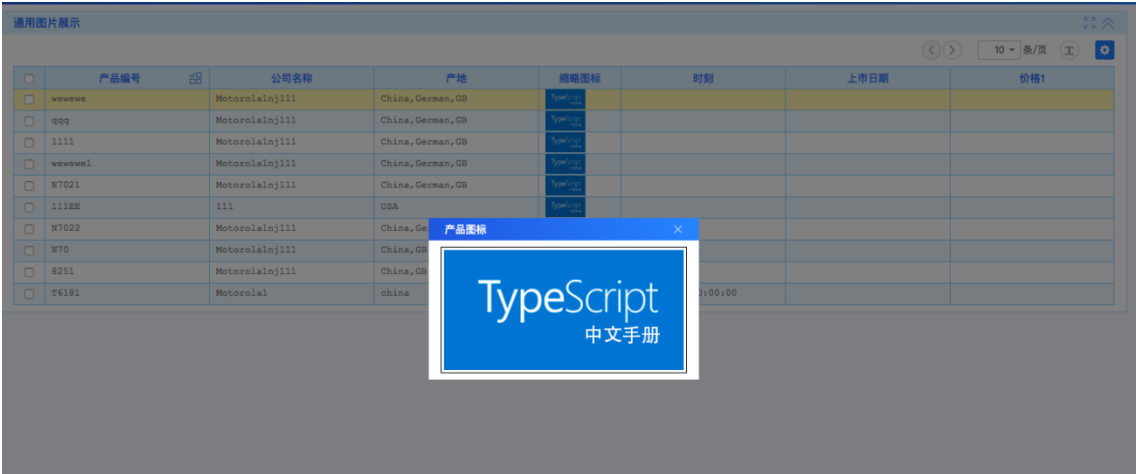
EEDM60020.js

```
$(function () {
    IPLATUI.EFGrid = {
        "resultA": {
            onCellClick: function (e) {
                e.preventDefault();
                if (e.field === "validate_2") {
                    var img = e.model['validate_2'];
                    WindowUtil({
                        title: "产品图标",
                        draggable: false,
                        content: "<span style='padding:3px;display:inline-block;border:1px solid black;'></span>"
                    });
                }
            },
            columns: [
                {
                    field: "validate_2",
                    title: "缩略图标",
                    template: "<span ><img src='#:validate_2#' width='50' height='25'></span>",
                }
            ]
        }
    }
});
```

```
        width:60
      }
    ]
  }
};
});
```

通过使用 onCellClick 事件，我们能够对 Grid 中 Cell 的点击事件进行监听。
通过对"validate_2"列进行点击，我们就能够查看弹出的图片信息。

效果如下：



3.5.3.1.21 EEDM6021-EFGrid 栏批量赋值

3.5.3.1.21.1 页面功能简介

栏批量赋值功能是在平台提供的一个快捷拷贝功能，它能快速的赋值多条数据。

开发人员可以通过对 EFColumn.tag、EFComboColumn.tag、EFPopupColumn 或者在 JS 的 column 配置中设置 copy: true 就可以开启批量赋值功能。

本示例页面用于说明，如何通过 tag 和 JS 来开启栏批量赋值功能。

3.5.3.1.21.2 配置和 API 详情

参考 [EFColumn](#)、[EFComboColumn](#)、[EFPopupColumn](#) 中的描述。

3.5.3.1.21.3 页面开发步骤

3.5.3.1.21.3.1 页面开发

栏批量赋值是一个基础的操作功能，在 [EFColumn](#)、[EFComboColumn](#)、[EFPopupColumn](#) 都可以使用使用 copy = "true" 来开启。创建 EEDM6021.jsp:

EEDM6021.jsp

```

<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" autoDraw="no" needAuth="false">
    <EF:EFColumn ename="${pid}" primaryKey="true" locked="true" readonly="true"
cname="产品编号"/>

    <EF:EFComboColumn ename="made_country" cname="产地"
columnTemplate="#=countryEname#-#=countryCname#"
      blockName="country" textField="countryCname"
valueField="countryEname" >
      <EF:EFOption label="全部" value="all"/>
      <EF:EFOption label="test" value="test"/>
    </EF:EFComboColumn>

    <EF:EFColumn ename="price" cname="价格" />
    <EF:EFColumn ename="validate_5" cname="时刻" editType="datetime"/>
    <EF:EFColumn ename="sale_date" cname="日期" editType="date"
      parseFormats="['yyyy-MM-dd','yyyy/MM/dd']"/>
  </EF:EFGrid>
</EF:EFRegion>

```

效果如下：



产品编号	产地	价格	时刻	日期
wewewe	China, German, GB	150122	1	
qqqq	China, German, GB	150122	1	
1111	China, German, GB	150122	1	
wewewe1	China, German, GB	150122	1	
N7021	China, German, GB	150122	1	
111EE	USA-美国	11	1	
N7022	China, German, GB	150122	1	
N70	China, GB	150122	20	
8251	China, GB	150122	1000	
T6181	china	3550	2000	2007-04-06

上面是一个普通的 Grid 示例页面，我们通过使用 `copy = "true"` 来开启它的批量赋值功能。

代码变更如下：

EEDM6021.jsp

```

<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" autoDraw="no" needAuth="false">
    <EF:EFColumn ename="${pid}" primaryKey="true" locked="true" readonly="true"
cname="产品编号"/>

    <EF:EFComboColumn ename="made_country" cname="产地"
columnTemplate="#=countryEname#-#=countryCname#"
      blockName="country" textField="countryCname"
valueField="countryEname" >
      <EF:EFOption label="全部" value="all"/>
      <EF:EFOption label="test" value="test"/>
    </EF:EFComboColumn>

    <EF:EFColumn ename="price" cname="价格" />
    <EF:EFColumn ename="validate_5" cname="时刻" editType="datetime"/>
    <EF:EFColumn ename="sale_date" cname="日期" editType="date"
      parseFormats="['yyyy-MM-dd','yyyy/MM/dd']"/>
  </EF:EFGrid>
</EF:EFRegion>

```

```

valueField="countryEname" copy="true">
  <EF:EFOption label="全部" value="all"/>
  <EF:EFOption label="test" value="test"/>
</EF:EFComboColumn>
<EF:EFColumn ename="price" cname="价格" copy="true" />
<EF:EFColumn ename="validate_5" cname="时刻" editType="datetime" copy="true"/>
<EF:EFColumn ename="sale_date" cname="日期" editType="date" copy="true"
  parseFormats=['yyyy-MM-dd','yyyy/MM/dd']/>
</EF:EFGrid>
</EF:EFRegion>

```

通过让 [EFColumn](#)、[EFComboColumn](#) 设置 `copy = "true"`，就开启了批量赋值的功能。

注意：当 `column` 设置有 `readonly = true` 和 `enable = false` 的情况下，此时 `column` 已经存在的值是不能被修改，为了避免不必要的逻辑冲突，批量赋值功能将会关闭，不被打开。

产品编号	产地	价格	时刻	日期
wwwwww	USA-美国	150122	1	
qqqq	USA-美国	150122	1	
1111	USA-美国	150122	1	
wwwwww1	USA-美国	150122	1	
W7021	USA-美国	150122	1	
111EE	USA-美国	11	1	
W7022	USA-美国	150122	1	
W70	USA-美国	150122	20	
8251	USA-美国	150122	1000	
76181	USA-美国	3550	2000	2007-04-06

除了使用 `tag` 去开启批量赋值功能，我们也可以使用 `JS` 去开启批量赋值功能。平台还提供 `backFill` 去实现自定义回填。

我们添加 `EEDM6021.js` 去了解 `backFill` 回调函数。

JS 代码

```

$(function () {
  IPLATUI.EFGrid = {
    "result": {
      columns: [{
        field: "validate_5",
        title: "时刻",
        backFill: function (option) {
          if (option.copy === true) {
            WindowUtil({
              "title": "提示",
              "content": "<div class='kendo-del-message'>手动回填</div>"
            });
            option.window.close();
          }
        }
      }
    ]
  }
});

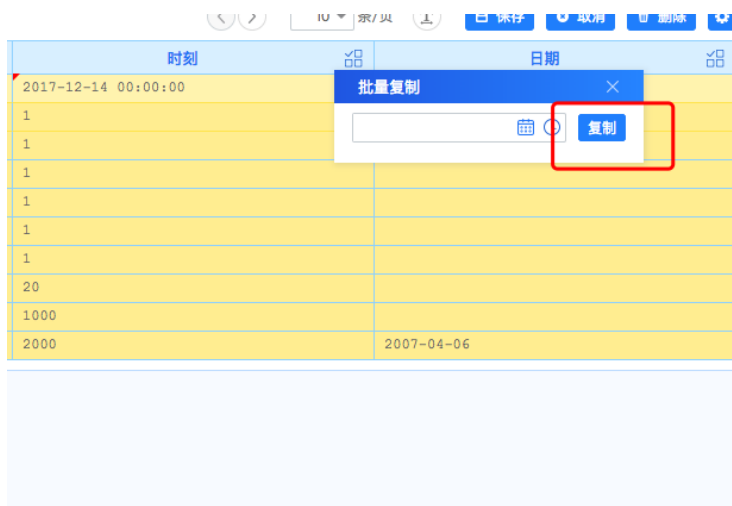
```

```

    }
  }
}
};
});

```

对 `validate_5`，中文名称为“时刻”的列在 JS 中添加 `backFill` 回调函数，当我们在页面中点击：



“复制”按钮的时候，平台会调用 `backFill` 这个方法。我们可以在此方法中去实现批量赋值。赋值完毕后，是不会关闭弹出赋值框，我们需要手动去实现关闭，使用平台提供的 `option.window.close()`；

3.5.3.1.22 EEDM6022-Grid 序列号展示与小计总计示例

3.5.3.1.22.1 页面功能简介

平台给开发人员提供一个可选参数 `rowNo`，能够在 `grid` 中提供一个序列号列。平台提供给开发人员一个小计和总计开发示例。

本示例页面用于说明，如何通过 `EFGrid.tag` 或者 JS 中配置 `rowNo` 来开启或者关闭序列号列。

3.5.3.1.22.2 配置和 API 详情

参考 [EFGrid 表格](#) 中 `rowNo`，`sumType` 属性的描述；[EFColumn](#) 中的 `sumType` 属性的描述。

3.5.3.1.22.3 序列号页面开发步骤

3.5.3.1.22.3.1 JSP 页面开发

创建页面 `EEDM6022.jsp`，在 `EFGrid` 中添加 `rowNo = "true"`，开始序列号功能。

EEDM6022.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" rowNo="true" enable="false"> <!-- 配置 rowNo: true 就能开
  启序列号列 -->
  </EF:EFGrid>
</EF:EFRegion>
```

效果如下:

序号	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证
1	wewewe	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
2	qqq	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
3	1111	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
4	wewewe1	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
5	N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N71		abc@baosi...	https://s...				1
6	111EE	111	USA	2017/11/0...	11		N7011			https://s...				1
7	N7022	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
8	N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	https://s...				20
9	8251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	https://s...				1000
10	T6181	Motorola...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...	https://s...	021-50801...	1		2000

我们也可以通过使用 JS 去配置，使用方式如下:

JS 配置 rowNo

```
IPLATUI.EFGrid = {
  "result": {
    rowNo: true
  }
};
```

3.5.3.1.22.4 小计总计页面开发步骤

3.5.3.1.22.4.1 JSP 页面开发

上面我们使用 rowNo 开启了序列号的功能, 在序列号功能的基础上, 我们添加小计与总计的功能。

JSP 页面

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" sumType="all" enable="false" autoDraw="yes">
    <EF:EFColumn ename="price" cname="价格" format="{0:c}" />
  </EF:EFGrid>
</EF:EFRegion>
```

小计总计功能是可以配置 sumType 来设置是否展示小计和总计数据。

sumType 有"page, total, all"三个可选值。“page”是展示小计，“total”是展示总计，“all”是展示小计和总计。

"page"小计是计算当前页中列中数据的总和。

"total"总计是计算此 Grid 中列中**所有数据**(不止当前页)的总和。此数据需要在后端进行设置。

配置说明

1. 只有数字列才可以小计，总计。（数据列就是JavaBean中设置了type为"N"的列）；
2. EFGrid配置了sumType, 将自动应用到所有的数字列；
3. EFGrid不配置sumType, 可以直接在EFColumn上配置sumType, 对指定的列进行小计，总计；
4. EFGrid需要配置autoDraw="override"来渲染EFColumn的列。

3.5.3.1.22.4.2 xml 开发

XML 配置

```
<select id="sum" resultClass="java.util.HashMap">
  SELECT
    SUM(PRICE) as "price",
    SUM(PRICE) as "validate_5" <!-- 测试用 -->
  FROM ${platSchema}.tee10
</select>
```

这里是对数据表中“price”和“validate_5”两个列数据进行总计。

3.5.3.1.22.4.3 Service 服务开发

ServiceEEDM6000

```
public EiInfo query(EiInfo inInfo) {
    EiInfo outInfo = super.query(inInfo,"EE10.query" );

    List sum = dao.query("EE10.sum", null); // 后台数据库的 sum 结果

    // 总计的一行 Map 信息，设置到 resultBlock 的扩展属性区 columnTotalSum
    // EiConstant.resultBlock => result , EiConstant.COLUMN_TOTAL_SUM =>
    columnTotalSum
    // sum.get(0) = > HashMap{price:'xxxxxx',validate_5:'xxxxxx'}
    outInfo.getBlock(EiConstant.resultBlock).set(EiConstant.COLUMN_TOTAL_SUM,
    sum.get(0));

    return outInfo;
}
```

```
}

```

这里把计算出来的总计数据以 **HashMap** 的形式存放到 **block** 的 **attr** 中。开发人员需要注意，总计的数据是需要现在后端计算出来，用 **Map** 的形式存放在 **block** 的 **attr** 区中。

3.5.3.1.22.4.4 页面展示效果

序号	公司名称	* 产品编号	* 时刻	* 上市日期	价格	搜索	v5	产地	* List选项	company_page
1	JJJJJJ JJJJ	JJJJ			¥0.000000		0	HK9K		
2	SonyEricss...	T6281x	2007040600000000...		¥0.000000	EE	2000	Japan		http://www.sonyeric...
3	Motorola	1112	2007070600000000...		¥11.000000		0	China		EWFM07
4	Motorola	1112Y	2007070600000000...		¥11.000000		0	China		EWFM07
5	Motorola	1112a	2007070600000000...		¥11.000000		0	China		EWFM07
6	Motorola	1112b	2007070600000000...		¥11.000000		0	China		EWFM07
7	NOKIA	12	2017101000000000...		¥0.120000		0	Sweden		
8	Motorola	12361	2007071200000000...		¥2.000000		0	Canada		
9	Motorola	123610	2007071200000000...		¥2.000000		0	Canada		
10	Motorola	1236a	2007071200000000...		¥2.000000		0	Canada		
小计					¥50.120000		2000.00			
总计					¥59,870.620000		59870.62			

3.5.3.1.23 EEDM6023-多区域提交

3.5.3.1.23.1 页面功能简介

本示例页面用于说明如何一次性提交页面上单记录区域及多记录区域勾选数据到指定后台方法。

3.5.3.1.23.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.3.1.23.3 页面开发步骤

3.5.3.1.23.3.1 JSP 开发

页面包含多个 **region** 区域，示例代码如下。

JSP 示例

```
<EF:EFPage title="单记录及多记录区域数据提交">
  <EF:EFRegion id="inqu" title="查询条件" type="query" efRegionShowClear="true"
  efRegionSave="true">
    <div class="row">
      <EF:EFInput cname="产品代号" ename="product_id" row="0"
      blockId="inqu_status"/>
      <EF:EFInput cname="公司名称" ename="company_name" row="0"
      blockId="inqu_status"/>
    </div>
  </EF:EFRegion>
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result"> <!-- EFGGrid 默认 autoDraw="yes" -->
    </EF:EFGrid>
  </EF:EFRegion>

```

```

    <EF:EFRegion id="inqu2" title="查询条件" type="query" efRegionShowClear="true"
efRegionSave="true">
        ...
    </EF:EFRegion>
    <EF:EFRegion id="result2" title="另一个 EFGrid">
        <EF:EFGrid blockId="blockT" queryMethod="queryAnother"> <!-- 另一个 EFGrid 的查
询方法设置为 queryAnother-->
        </EF:EFGrid>
    </EF:EFRegion>
</EF:EFPage>

```

3.5.3.1.23.3.2 JS 部分

调用 `IPLAT.submitNode` 将指定节点下 **单记录区域** 及 **多记录区域** 勾选的数据提交到 `ServiceEEDM6023` 下的 `testForSubmit` 方法。

JS 示例

```

$(function () {
    $("#SUBMIT").click(function () {
        var btnNode = $(this);
        btnNode.attr("disabled", true);
        //grid 区域
        var node = $("#EEDM6023");
        IPLAT.submitNode(node, "EEDM6023", "testForSubmit", {
            onSuccess: function (ei) {
                btnNode.attr("disabled", false);
                console.log(ei);
                // onFail 表示失败回掉函数
            }, onFail: function (ei) {
                // 发生异常
                btnNode.attr("disabled", false);
                console.log(ei);
            }
        });
    });
});

```

3.5.3.1.23.3.3 页面展示效果

The screenshot displays a web application interface with a table of records and search filters. The table has columns for product code, company name, origin, release date, price, company homepage, Google search, and various verification fields. Below the table is a search section with input fields for data series 1, 2, 3, and 4, and a search button.

*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证
15222	ss222		2017-10-26...	222								
S552	Apple	USA, China	2017/10/04...	999		S556						
1112	Motorola	China	2007-07-06	11	ENFM07			abc@baosig...	11	-123456789...		
TN702	los	China	2017-10-14	0								
f5c0d0fc	宝信软件		2017-10-10	10		111						
K-T61812	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13000000000	aaa@123.com		021-508011...	1	
K-T6181	Motorola1	13	2007-04-06	15	时间段访问量统计	查询	13000000000	aaa@123.com		021-508011...	1	
Tttttt	Motorola1	13	2017-10-09	15	时间段访问量统计	查询	13000000000	aaa@123.com		021-508011...	1	
T61811	Los	9	2017-10-20	15	时间段访问量统计	查询	13000000000	aaa@123.com		021-508011...	1	
XXXX	Motorola	Sweden	2007-03-17	1400	http://www.nokia.com	查询	13000000000	aaa@123.com		021-508011...	1	

查询条件

数据列1: 数据列2: 数据列3:

数据列4: 二段式: 一段式:

点击提交后会将单记录区域及多记录区域勾选的数据组装到 EiInfo, 如图所示。

```

▼ EiInfo {name: "", msg: "测试成功", msgKey: "", status: 0, descName: ""...} ⓘ
  ▼ blocks: Object
    ► blockT: EiBlock
    ► inqu_status: EiBlock
    ► result: EiBlock
    ► test: EiBlock
    ► __proto__: Object
  descName: ""
  ► extAttr: Object
    msg: "测试成功"
    msgKey: ""
    name: ""
    status: 0
  ► __proto__: AbstractEi

```

3.5.3.1.24 EEDM6024-autoBind 相关

3.5.3.1.24.1 页面功能简介

autoBind 是一个比较特殊的属性，它主要是辅助于 EFGGrid 进行初始化。

1. 当后端 initLoad 返回的 eiInfo 中存在页面 Grid 渲染所需要的 Block 时，它是失效的，不管设置其值为 true 或者 false。页面都会根据 initLoad 所返回的 eiInfo 中相对应的 block 进行渲染 Grid
2. 当后端 initLoad 返回的 eiInfo 中不存在页面 Grid 渲染所需要的 Block 时，只有当它为 true，他会调用 queryMethod 所配置的方法，向后端请求数据。否则会报错。

3.5.3.1.24.2 配置和 API 详情

参考 [EFGGrid 表格](#) 中的描述。

3.5.3.1.24.3 页面开发步骤

3.5.3.1.24.3.1 initLoad 返回 Grid 所需要的 Block

initLoad 返回的 Eilnfo 中存在页面 Grid 所需要的 Block 时候, 逻辑比较简单的, 页面 Grid 会根据返回的 Eilnfo 中 block 进行渲染。

创建服务端文件 EEDM6024.java:

EEDM6024.java

```
public Eilnfo initLoad(Eilnfo inInfo) {

    EiBlockMeta blockMeta = new EE10().eiMetadata;
    EiBlock eiblock = new EiBlock("result");
    eiblock.addBlockMeta(blockMeta);
    Eilnfo eilnfo = new Eilnfo();
    eilnfo.addBlock(eiblock);
    return eilnfo;
}
```

在编写 initLaod 方法时候, 对返回的 Eilnfo 添加"result"的 block 数据块。

我们也创建前端文件 EEDM6024.jsp:

EEDM6024.jsp

```
<EF:EFRegion id="result" >
    <EF:EFGrid blockId="result" autoBind="false">
        </EF:EFGrid>
    </EF:EFRegion>
```

这里不管设置 autoBind=(true|false), Grid 都会根据 InitLoad 方法中返回的 Eilnfo 中"result"的 block 来进行渲染。所以当开发人员知道在 initLoad 方法里面有返回的 EiBlock 是 EFGrid 所需要的时候, 前端继续配置 autoBind = (true|false) 是没有意义的。

效果如下:



因为在初始化时候没有给 block 添加数据, 所以这里数据集是空的。

我们给 EEDM6024.java 添加上初始化数据:

EEDM6024.java

```
public Eilnfo initLoad(Eilnfo inInfo) {
```

```
return query(inInfo, "EE10.query", new EE10());
}
```

效果如下：

*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证
wewewe	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
qqq	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
1111	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
wewewe1	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
N7021	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N71		abc@baosi...	https://s...				1
111EE	111	USA	2017/11/0...	11		N7011			https://s...				1
N7022	Motorola...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
N70	Motorola...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	https://s...				20
8251	Motorola...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	https://s...				1000
T6181	Motorola...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...	https://s...	021-50801...	1		2000

3.5.3.1.24.3.2 initLoad 没有返回 Grid 所需要的 Block

我们修改 EEDM6024.java, 让其返回的 eiInfo 中不存在 Grid 所需要的 block。

EEDM6024.java

```
public EiInfo initLoad(EiInfo inInfo) {
    return inInfo;
}

public EiInfo query(EiInfo inInfo) {
    EiInfo outInfo = query(inInfo, "EE10.query", new EE10());
    return outInfo;
}
```

同时我们也修改 EEDM6024.jsp:

EEDM6024.jsp

```
<EF:EFRegion id="result" >
    <EF:EFGrid blockId="result" autoBind="true" autoDraw="yes" queryMethod="query">
    </EF:EFGrid>
</EF:EFRegion>
```

我们设置 EFGrid 的 autoBind="true", queryMethod="query" 这样 Grid 会自动向 query 方法去请求数据，渲染 Grid。

效果如下：

☐	*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证
☐	wewewe	MotorolaL...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
☐	qqq	MotorolaL...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
☐	1111	MotorolaL...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
☐	wewewe1	MotorolaL...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
☐	N7021	MotorolaL...	China, Ger...	2017/11/0...	150122	EEE1	N71		abc@baosi...	https://s...				1
☐	111EE	111	USA	2017/11/0...	11		N7011			https://s...				1
☐	N7022	MotorolaL...	China, Ger...	2017/11/0...	150122	EEE1	N70		abc@baosi...	https://s...				1
☐	N70	MotorolaL...	China, GB	2017/11/2...	150122	EEE	N70		abc@baosi...	https://s...				20
☐	8251	MotorolaL...	China, GB	2017/11/1...	150122	EEE	N70		abc@baosi...	https://s...				1000
☐	T6181	MotorolaL...	china	2007-04-06	3550	时间段访问量统计	查询	138000000...	aaa@123.c...	https://s...	021-50801...	1		2000

3.5.3.1.25 EEDM6026-URL 里拼接过长参数的处理方法

3.5.3.1.25.1 页面功能简介

此页面用于展示当需要在 URL 里拼接过长参数的处理方法。

3.5.3.1.25.2 配置和 API 详情

(开发版本 6.0.0707-SNAPSHOT 之后，才有该 API)

IPLAT.postForm(ars1,args2,args3)。其中：

- postForm()的第一个参数是要打开的页面号；
- 第二个参数为需要 POST 提交的大批量数据，组装成的 EilInfo；
- 第三个参数为可选参数，少部分出现在 URL 中的 parameter；

3.5.3.1.25.3 页面开发步骤

3.5.3.1.25.3.1 JS 开发

这是 EEDM6026.js，在这段 js 里，我们在 EilInfo 里设置了大数据，假定为“huge data”，通过 IPLAT.postForm()的方法，将页面重定向到 EEDM6002 页面。

JS 示例

```
$(function () {
    var postEilInfo = new EilInfo();
    postEilInfo.set("post-0-data", "huge data");

    IPLAT.postForm("EEDM6002", postEilInfo, "p1=100&p2=mysql");
});
```

在 EEDM6002 页面对应的 ServiceEEDM6002 的 initLoad 的 inInfo 中，将会包含这个 postEilInfo。

3.5.3.1.26 EEDM6027-Grid 排序示例

3.5.3.1.26.1 页面功能简介

前台可以设置 sort 属性，有三种选择，分别为 single，all 和 setted。

当 sort="single"时，grid 根据点击的任意一列进行排序；

当 sort="all"时，grid 可进行多列排序，order by 的顺序取决于点击顺序；

当 sort="setted"时，grid 指定列排序，非指定列不能排序。

3.5.3.1.26.2 配置和 API 详情

参考 [EFGGrid 表格](#) 中的描述。

3.5.3.1.26.3 页面开发步骤

3.5.3.1.26.3.1 JSP 页面开发

此时第一个 grid，设置 sort="single";第二个 grid，设置 sort="all";第三个 grid，设置 sort="setted"。

JSP 示例

```
<EF:EFRegion id="result" title="单列排序">
  <EF:EFGGrid blockId="blockA" queryMethod="queryA" sort="single"/>
</EF:EFRegion>

<EF:EFRegion id="all" title="所有列均可排序">
  <EF:EFGGrid blockId="blockB" queryMethod="queryB" sort="all"/>
</EF:EFRegion>

<EF:EFRegion id="setted" title="指定列可排序">
  <EF:EFGGrid blockId="blockC" queryMethod="queryC" sort="setted" autoDraw="mixed">
    <EF:EFGColumn ename="company_name" cname="公司名" sort="true" />
  </EF:EFGGrid>
</EF:EFRegion>
```

3.5.3.1.26.3.2 sort 的三种属性

当 sort 设置成 single 时，此时单列排序，可指定任意列。

单列排序													
*产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证	
15222	ss222		2017-10-26	222									
f5c0d8fc	宝信软件		2017-10-10	10		111							
18de8321	宝信软件		2017-10-10	10		111							
373f9022	宝信软件		2017-10-10	10		111							
22378463	宝信软件		2017-10-10	10		111							
4204df24	宝信软件		2017-10-10	10		111							
fe157050	宝信软件		2017-10-13	10		111							
K-T61812	Motorola1	13	2007-04-06	15	时间访问量统计	查询	13800000000	aaa@123.com		021-508011...	1		
Tttttt	Motorola1	13	2017-10-09	15	时间访问量统计	查询	13800000000	aaa@123.com		021-508011...	1		
K-T6181	Motorola1	13	2007-04-06	15	时间访问量统计	查询	13800000000	aaa@123.com		021-508011...	1		

当 sort 设置成 all 时，全列可排序。排序顺序与点击顺序有关，与列顺序无关。

所有列均可排序

* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证?
	苹果	USA		0								
1112	Motorola	China	2007-07-06	11	ENFM07			abc@baosig...	11	-123456789...		
1112Y	Motorola	China	2007-07-06	11	ENFM07			abc@baosig...	11	-123456789...		
1112a	Motorola	China	2007-07-06	11	ENFM07			abc@baosig...	11	-123456789...		
1112b	Motorola	China	2007-07-06	11	ENFM07			abc@baosig...	11	-123456789...		
12	NOKIA	Sweden	2017-10-10	0.12								
12361	Motorola	Canada	2007-07-12	2				ab1c@baosi...		-123456789...		
123610	Motorola	Canada	2007-07-12	2				ab1c@baosi...		-123456789...		
1236a	Motorola	Canada	2007-07-12	2				ab1c@baosi...		-123456789...		
1236b	Motorola	Canada	2007-07-12	2				ab1c@baosi...		-123456789...		

来看一下 Header 信息:

```

▼ __blocks__: {blockB: {, ...}}
  ▼ blockB: {, ...}
    ▶ attr: {orderBy: "company_name asc,made_country asc,sale_date desc", limit: 10, offset: 0, showCount: "true"}
    ▶ meta: {desc: "", attr: {, ...}}
    rows: []

```

可以看出, 当我们依次点击公司名称, 产地, 上市日期时, **order by** 的顺序是我们的点击顺序。此时若再点击公司名称会发生什么呢, 让我们来看一下。

```

▼ __blocks__: {blockB: {, ...}}
  ▼ blockB: {, ...}
    ▶ attr: {orderBy: "made_country asc,sale_date desc,company_name desc", limit: 10, offset: 0, showCount: "true"}
    ▶ meta: {desc: "", attr: {, ...}}
    rows: []

```

可以看出, 此时公司名称又成为了 **order by** 的最后一个字段。所以, 全列排序的时候 **order by** 顺序只与点击顺序有关。

当 **sort** 设置成 **setted** 时, 指定列排序, **column** 的 **sort** 属性需设置成 **true**。

指定列可排序

* 产品代号	公司名	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证?
S552	Apple	USA, China	2017/10/04...	999		S556						
S551	Apple	USA	2017/10/11...	999								
S5561	Apple	USA		999								
2221	EDFA01	按钮信息管理	2017-09-05	11	EDFA01					SonyEricss...	Motorola	
T61811	Los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T61819	Los	9	20171018	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T61814	Los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T618110	Los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
1112	Motorola	China	2007-07-06	11	ENFM07			abc@baosig...	11	-123456789...		
8251a	Motorola	china	2007-03-17	1400	EL23	查询	13800000000	aaa@123.com	label1	021-508011...	1	

此时只有 **column** 设置成 **sort="true"** 的公司名称列才可进行排序。

指定列可排序

* 产品代号	公司名	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证?
S552	Apple	USA, China	2017/10/04...	999		S556						
S551	Apple	USA	2017/10/11...	999								
S5561	Apple	USA		999								
2221	EDFA01	按钮信息管理	2017-09-05	11	EDFA01					SonyEricss...	Motorola	
T61811	Los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T61819	Los	9	20171018	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T61814	Los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
T618110	Los	9	2017-10-20	15	时间段访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
XXX1	Motorola	Sweden	2007-03-17	1400	http://www.nokia.com	查询	13800000000	aaa@123.com		021-508011...	1	
8251b	Motorola	china	2007-03-17	1400	EL23	查询	13800000000	aaa@123.com	label1	021-508011...	1	

callhost:8080/showcase/web/EEDM6027#

然而我们发现, 验证字段 1 也可进行排序, 发生这种情况的原因是因为在 **javabeen** 中, 我们将此字段设置成了可排序的。

```
eiColumn = new EiColumn( name: "validate_1");
eiColumn.setDescName("验证字段1");
eiColumn.set("sort", true);
eiMetadata.addMeta(eiColumn);
```

不过如果仅是为了排序，我们不推荐这种方式，应当优先使用 `sort` 属性进行排序处理，除非是对 `autoDraw` 有一定需求。

3.5.3.1.26.3.3 single,all 时设置不可排序/setted 时设置可排序

关于 `sort` 属性的，可在 `js`，`tag`，以及 `javabean` 里面设置，这三者的优先级为 `js` 中 `sortable` > `column` 中 `sort` > `attr(java)` 中 `sort`。

sort 属性，js 中是设置 `sortable`，javabean 与 tag 中是设置 `sort`。

在 `single` 与 `all` 的情况下，要设置某一列不能排序，需设置 `sort` 属性为 `false`；在 `setted` 情况下，要设置某列能排序，需设置 `sort` 属性为 `true`。

3.5.3.1.27 EEDM9028-主从表功能示例(子表无 CRUD)

3.5.3.1.27.1 页面功能简介

本示例页面用于说明主从表的简单应用，子表只取数据，不做 `CRUD`。

此示例主表为老师表，子表为课程表，表之间是一对多的关系。

3.5.3.1.27.2 配置和 API 详情

参考 [EFGGrid](#) 中的描述。

了解 `kendoUI` 原生 [detailInit](#) 的用法。

3.5.3.1.27.3 页面开发步骤

3.5.3.1.27.3.1 JSP 页面开发

JSP 示例

```
<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<EF:EFPage>

  <EF:EFRegion id="inqu" title="查询条件">
    <div class="row">
      <EF:EFInput ename="inqu_status-0-teacher_id" cname="老师编号"/>
      <EF:EFInput ename="inqu_status-0-teacher_name" cname="老师姓名"/>
    </div>
  </EF:EFRegion>
</EF:EFPage>
```

```

        <EF:EButton ename="query" cname="查询"></EF:EButton>
        <EF:EButton ename="getChecked" cname="获取数据"></EF:EButton>
    </div>
</EF:EFRegion>

<EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" needAuth="false" pagerPosition="top"
        autoDraw="no" enable="false" > <!-- EFGGrid 默认 autoDraw="yes" -->
        <EF:EColumn ename="teacher_id" cname="老师编号"
primaryKey="true"></EF:EColumn>
        <EF:EColumn ename="teacher_name" cname="老师姓名"></EF:EColumn>
        <EF:EColumn ename="teacher_sex" cname="老师性别"></EF:EColumn>
        <EF:EColumn ename="teacher_age" cname="老师年龄"></EF:EColumn>
        <EF:EColumn ename="hiretime" cname="入校时间"></EF:EColumn>
    </EF:EFGrid>
</EF:EFRegion>

</EF:EFPage>

```

3.5.3.1.27.3.2 前端 JS 配置

JS 示例

```

$(function () {

    $("#query").on("click", function () {
        resultGrid.dataSource.page(1);
    });

    var extend = $.extend,
        __ctx__ = IPLATUI.CONTEXT_PATH;

    IPLATUI.EFGrid = {
        "result": {
            columns: [
                {
                    field: "checkbox",
                    headerTemplate: "<input type='checkbox' class='check-all' style='text-align:
center' />",

```

```

        template: '<input type="checkbox" value=#=uid# class="kendo-check-box
check-one" style="text-align: center" />',
        resizable: false,
        enable: false,
        readonly: true,
        position: "front",
        width: "30px"
    }
],
detailInit: function (e) {
    var gridId = "childBlock-" + kendo.guid();
    var tagId = gridId + "tag";
    var teacherId = e.data['teacher_id'];

    $("<div id='ef_grid_' + gridId + '>").appendTo(e.detailCell);

    IPLAT.EFGrid[tagId] = {
        columns: [],
        readonlyColumns: []
    };

    var defaultOption = {
        autoBind: true,
        autoDraw: "false",
        needAuth: false,
        copyToAdd: false,
        readonly: false,
        enable: false,
        rowNo: false,
        showCount: true,
        // checkMode: "hidden,row",
        url: __ctx__ + "/service",
        eiInfo: __eiInfo // 要注意初始化的时候主子表都得存在 block
    };

    var childOptions = extend({}, defaultOption, {
        tagId: tagId,
        blockId: "child",
        gridId: "ef_grid_" + gridId,
        pageable: false,
        toolbarConfig: {hidden: "all"},
        queryMethod: 'query',
        insertMethod: 'insertChild',
    });

```

```

updateMethod: 'updateChild',
deleteMethod: 'deleteChild',
query: function () {
    var inInfo = new EiInfo();
    inInfo.set("inqu_status-0-teacher_id", teacherId);
    return inInfo;
},
columns: [
    {
        field: "checkbox",
        headerTemplate: "",
        template: '<input type="checkbox" value=#=uid# class="kendo-check-box check-one" style="text-align: center" />',
        width: '30px',
        resizable: false,
        enable: false,
        locked: false,
        readonly: true
    },
    {field: "course_id", title: "课程编号", primaryKey: true},
    {
        field: "course_name",
        title: "课程名称"
    },
    {
        field: "opentime",
        title: "开课时间",
        parseFormats: ["yyyy-MM-dd"],
        dateFormat: "yyyy-MM-dd",
        editType: "date",
        template: function (e) {
            return kendo.toString(e.opentime, "yyyy-MM-dd");
        }
    },
    {field: "course_rank", title: "课程优先级"},
    {field: "gpa", title: "课程绩点"}
],

//当数据绑定到子表时触发, dataBouding 是在绑定数据之前触发
dataBound: function (e) {
    var grid = e.sender;
    var trs = grid.table.find("tr");

```

```

        $.each(trs, function (i, tr) {
            var courseName = e.sender.dataItems()[i].course_name;
            if (courseName.indexOf("A") > 0) {
                tr.style.background = "#FFB3FF";
            } else {
                tr.style.background = "#99FFFF";
            }
            // tr.style.color="white"
        });
    }
});

IPLAT.Grid(childOptions);
},

//点击主表的 checkbox 将映射对应子表的所有 checkbox
onCheckRow: function (e) {
    var kendoGrid = e.tr.next().find("[data-role=grid]").data("kendoGrid");
    if (kendoGrid) {
        var length = kendoGrid.getDataItems().length;
        var arr = [];
        for (var i = 0; i < length; i++) {
            arr.push(i);
        }
        if (e.checked) {
            kendoGrid.setCheckedRows(arr);
        } else {
            kendoGrid.setUnCheckedRows(arr);
        }
    }
},

//页面加载结束所有子表全展开
loadComplete: function (grid) {
    $.each($(grid.element.find("tr.k-master-row")).find(".k-icon.k-i-expand"),
        function (i, element) {
            element.click();
        });
    $(".k-header.k-grid-toolbar").hide();
}
}
};

```

```

var getCheckedData = function (parentGrid, childGridName) {
    var childRows = [],
        parentRows = [],
        eiInfo = new EiInfo(),
        eiColumn,
        eiBlockMeta;

    eiInfo.addBlock(new EiBlock(childGridName));
    eiBlockMeta = eiInfo.getBlock(childGridName).getBlockMeta();

    //获取主表的选中数据，添加到 parentRows 里面
    $.each(parentGrid.element.find(".k-grid-content tr.k-master-row.i-state-selected"),
        function (i, element) {
            parentRows.push(parentGrid.dataItem(element));
        });

    //获取子表的选中数据，添加到 childRows 里面
    $.each(parentGrid.element.find("tr.k-detail-row").find(".k-grid-content tr.i-state-
selected"),
        function (i, element) {
            //$(element.closest(".k-detail-row")).prev()是为了找到子表所在的主表 row 的 DOM
            //对象
            var childGrid = $(element.closest(".k-detail-row")).prev().next().find("[data-
role=grid]").data("kendoGrid");
            childRows.push(childGrid.dataItem(element));
        });

    //如果未选中数据，返回空数组
    if (childRows.length == 0) {
        return [];
    }

    //构建 meta 信息
    for (var name in childRows[0].toJSON()) {
        eiColumn = new EiColumn("" + name);
        eiBlockMeta.addMeta(eiColumn);
    }

    //构建前台 block 的二维数组信息，这里将 JSON 格式的一行数据，拉平成一行数组数据 {key:
value} ==> [value]
    for (var i = 0; i < childRows.length; i++) {
        var _childRows = [];
        for (var name in childRows[i].toJSON()) {

```



```
        _childRows.push(childRows[i][name]);
    }
    childRows[i] = _childRows;
}

eIInfo.getBlock(childGridName).addRow(childRows);

eIInfo.addBlock(parentGrid.model2EiBlock(parentRows));

return eIInfo;

};

$("#getChecked").on("click", function () {
    console.log(getCheckedData(resultGrid, "child"));
    return getCheckedData(resultGrid, "child");
});

});
```

detailInit 存在的时候，主表不能有固定列。

3.5.3.1.27.3.3 页面展示效果

主从示例页面及操作 > EEDM9028

查询条件

老师编号老师姓名

查询获取数据

记录集

10 每页 共 5 条

☐	☐	老师编号	老师姓名	老师性别	老师年龄	入职时间
☐	☐	1	谭金莲	0	23	2007-03-17
	☐	课程编号	课程名称	开课时间	课程优先级	课程绩点
	☐	1	语文A班	2007-03-17	High	3.91
	☐	2	语文B班	2007-03-17	Medium	2.02
☐	☐	2	梅知慧	0	18	2007-03-17
	☐	课程编号	课程名称	开课时间	课程优先级	课程绩点
	☐	3	物理B班	2007-03-17	Medium	1.63
☒	☐	3	武松	1	36	2007-03-17
☒	☐	4	1213	0	23	2007-03-17
☒	☐	5	1213	0	23	2007-03-17

3.5.3.1.27.3.4 getCheckedData()方法说明

getCheckedData()方法为获取勾选的主子表数据并塞进各自的 block 里面，从而组成一个新的 EIInfo。

- 389

3.5.3.1.27.3.5 展示效果

教师编号	教师姓名	教师性别	教师年龄	入校时间
1	潘金莲	0	23	2007-03-17

课程编号	课程名称	开课时间	课程优先级	课程终点
1	语文A班	2007-03-17	High	3.91
2	语文B班	2007-03-17	Medium	2.02

课程编号	课程名称	开课时间	课程优先级	课程终点
2	物理班	0	18	2007-03-17
3	物理班	2007-03-17	Medium	1.63
3	武松	1	36	2007-03-17
4	1213	0	23	2007-03-17

3.5.3.1.27.3.6 页面加载结束所有子表全展开
只需找到点击的 icon，触发 click 事件即可。

```
loadComplete: function (grid) {
    $.each($(grid.element.find("tr.k-master-row")).find(".k-icon.k-i-expand"),
        function (i, element) {
            element.click();
        });
    $(".k-header.k-grid-toolbar").hide();
}
```

3.5.3.1.27.3.7 根据班级类型设置行背景色
通过 dataBound 事件，找到行信息，根据条件设置不同样式属性。

```
dataBound: function (e) {
    var grid = e.sender;
    var trs = grid.table.find("tr");
    $.each(trs, function (i, tr) {
        var courseName = e.sender.dataItems()[i].course_name;
        if (courseName.indexOf("A") > 0) {
            tr.style.background = "#FFB3FF";
        } else {
            tr.style.background = "#99FFFF";
        }
        // tr.style.color="white"
    });
}
```

页面效果：

记录集						
		老师编号	老师姓名	老师性别	老师年龄	入职时间
		1	潘金莲	0	23	2007-03-17
		课程编号	课程名称	开课时间	课程优先级	课程练点
		1	语文A班	2007-03-17	High	3.91
		2	语文B班	2007-03-17	Medium	2.82
		2	物理A班	0	18	2007-03-17
		课程编号	课程名称	开课时间	课程优先级	课程练点
		3	物理B班	2007-03-17	Medium	1.63
		3	数学	1	36	2007-03-17
		课程编号	课程名称	开课时间	课程优先级	课程练点
		5	健美操A班	2007-03-17	High	4.63
		4	1213	0	23	2007-03-17
		课程编号	课程名称	开课时间	课程优先级	课程练点
		5	1213	0	23	2007-03-17
		课程编号	课程名称	开课时间	课程优先级	课程练点

3.5.3.1.28 EEDM6029-加密显示

3.5.3.1.28.1 页面功能简介

本示例页面用于说明，如何在 EFGrid 中显示加密的密码。

3.5.3.1.28.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.3.1.28.3 页面开发步骤

3.5.3.1.28.3.1 JSP 页面开发

JSP 示例

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" needAuth="false" enable="true"> <!-- autoDraw="no" 只渲染 EFGrid 的标签列 --%>
  </EF:EFGrid>
</EF:EFRegion>
```

3.5.3.1.28.3.2 Javascript 开发

我们在 js 中配置密码列，设置 template 为 *****

JS 示例

```
$(function () {
  IPLATUI.EFGrid = {
    "result": {
      columns: [
        {
          field:"company_name",
          template:"*****",
          title:"密码",
          editor: function (container, options) {
            var input = $('<input type="password"/>');
            input.attr("name", options.field);
          }
        }
      ]
    }
  }
});
```

```
        input.attr("id", options.field);
        input.appendTo(container);
    }
}
}
};

});
```

3.5.3.1.28.3.3 页面效果如下所示

记录集

新增

共 6 页 10 条 共 51 条 保存 取消 删除

☐	* 产品代号	密码	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字
☒	15222	*****		2017-10-26...	222								
☐		*****	USA		0								
☐	5552	*****	USA,China	2017/10/04...	999		5556						
☐	1112	*****	China	2007-07-06	11	EWFM07			abc@baosig...	11	-123456789...		
☐	002	*****	USA		0								
☐	7H702	*****	China	2017-10-14	0								
☐	f5c0d8fc	*****		2017-10-10	10		111						
☐	K-T61812	*****	13	2007-04-06	15	时间访问量统计		13800000000	aaa@123.com		021-508011...	1	
☐	K-T6181	*****	13	2007-04-06	15	时间访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	
☐	Tttttt	*****	13	2017-10-09	15	时间访问量统计	查询	13800000000	aaa@123.com		021-508011...	1	

多区域提交

3.5.3.1.29 EEDM6030-提取一列进行数据汇总，横表展示

3.5.3.1.29.1 页面功能简介

将数据库表进行结构转换，在页面展示。

数据库表结构：

*	TES11_ID	MATERIAL	SUPPLIER	NAME	PRICE
1	1	A	1	1A	10.22
2	2	A	2	2A	13.22
3	3	A	3	3A	22.22
4	4	B	1	1B	22.00
5	5	B	2	2B	22.33
6	6	B	3	3B	111.00
7	7	C	1	1C	23.50
8	8	C	2	2C	32.30
9	9	C	3	3C	4.00
10	10	D	1	1D	20.50
11	11	D	2	2D	22.66

提取 material 一列，其他列循环展示。页面效果：

记录集											
	material	name_1	supplier_1	price_1	name_2	supplier_2	price_2	name_3	supplier_3	price_3	
A	1A	1	10.22	2A	2	13.22	3A	3	22.22		
B	1B	1	22.00	2B	2	22.33	3B	3	111.00		
C	1C	1	23.50	2C	2	32.30	3C	3	4.00		
D	1D	1	20.50	2D	2	22.66					

3.5.3.1.29.2 配置和 API 详情

Map, List, EiInfo 的基本知识。

3.5.3.1.29.3 页面开发步骤

3.5.3.1.29.3.1 JSP 页面开发

JSP 示例

```
<EF:EFRegion id="inqu" title="查询条件" type="query" efRegionShowClear="true"
efRegionSave="true">
  <div class="row">
    <EF:EFInput cname="物料" ename="inqu_status-0-material" />
  </div>
</EF:EFRegion>
<EF:EFRegion id="result" title="记录集">
  <EF:EFInput ename="inqu_status-0-continent_code" type="hidden" />
  <div class="k-content" style="overflow: auto">
    <div id="ef_window_result"></div>
    <div id="ef_grid_result"></div>
  </div>
```

3.5.3.1.29.3.2 JS 开发

js

```
$(function () {
  $("#QUERY").on("click", function (e) {
    var ei = EiInfo.build($("#inqu"));
    EiCommunicator.send("ES11", "query", ei, {
      onSuccess: function (ei) {
        initGrid(ei)
      },

      onFail: function () {
        alert("请求失败");
      }
    })
  });
  initGrid(_eiInfo);
});
```

```

function initGrid(ei) {
    var tagId = "result-" + kendo.guid();

    $("#ef_grid_result").empty();

    $("<div id='" + tagId + "'></div>").appendTo($("#ef_grid_result"));
    var extend = $.extend,
        __ctx__ = IPLATUI.CONTEXT_PATH;

    var defaultOption = {
        serviceName: "ES11",
        autoBind: true,
        autoDraw: "yes",
        needAuth: false,
        copyToAdd: false,
        readonly: false,
        enable: true,
        rowNo: false,
        showCount: true,
        // checkMode: "hidden,row",
        url: __ctx__ + "/service",
        eiInfo: ei
    };

    IPLAT.EFGrid[tagId] = {
        columns: [],
        readonlyColumns: []
    };

    var masterOptions = extend({}, defaultOption, {
        tagId: tagId,
        blockId: "result",
        gridId: tagId
    });
    window['resultGrid'] = IPLAT.Grid(masterOptions);
}

```

3.5.3.1.29.3.3 Service 开发

查询：

query

```

public EiInfo query(EiInfo inInfo) {
    EiInfo materialInfo = super.query(inInfo, "ES11.queryMaterial"); // distinct 物料, 即转换
    成几行数据
    EiBlock materialBlock = materialInfo.getBlock("result");
    String mater = "";
    if (!inInfo.getBlocks().isEmpty()) {
        if (StringUtils.isNotEmpty(inInfo.getBlock("inqu_status").getCellStr(0, "material"))) {
            mater = (String) inInfo.getBlock("inqu_status").getCellStr(0, "material");
        }
    }
    List list = new ArrayList(); // list 存放转换后的数据结构
    initMetaList();

    for (int i = 0; i < materialBlock.getRowCount(); i++) {
        Map<String, Object> map = new LinkedHashMap<>(); // map 代表每一行数据
        String material = materialBlock.getCellStr(i, "material"); // 物料代码
        inInfo.set("inqu_status-0-material", material);
        inInfo.set("result-offset", 0);
        EiInfo info = super.query(inInfo, "ES11.query"); // 根据物料代码查询所有供应商
        EiBlock infoBlock = info.getBlock("result");

        map.put("material", material);

        for (int j = 0; j < infoBlock.getRowCount(); j++) { // 循环处理所有的供应商数据
            Map<String, Object> row = infoBlock.getRow(j);
            row.remove("material");

            for (Map.Entry<String, Object> entry : row.entrySet()) { // 将处理过的数据合并至一
                个 map
                map.put(entry.getKey() + "__" + (j + 1), entry.getValue());
            }
        }

        list.add(map);
    }

    EiInfo newInfo = new EiInfo();
    EiBlock transformBlock = newInfo.addBlock("result"); // 转换后的 EiBlock

    EiInfo count = new EiInfo();
    count.set("result-limit", 1000);
}

```

```

count.set("inqu_status-0-material", mater);
transformBlock.set("count", super.query(count,
"ES11.queryMaterial").getBlock("result").getRowCount());

EiColumn eiColumn = new EiColumn("material");
eiColumn.setDescName("material");
eiColumn.setPrimaryKey(true);
eiColumn.setNullable(true);
eiColumn.setReadonly(true);
eiColumn.set("enable", false);
transformBlock.addMeta(eiColumn);

for (int i = 0; i < list.size(); i++) {
//    Map<String, String> newMap = (Map) list.get(i);
    LinkedHashMap<String, String> newMap = (LinkedHashMap) list.get(i);
    for (Map.Entry<String, String> entry : newMap.entrySet()) {
        eiColumn = new EiColumn("" + entry.getKey());
        eiColumn.setDescName("" + entry.getKey());
        String column = entry.getKey().split("_")[0];
        if (column.equals((String) metaList.get(metaList.size() - 1))) { // 如果是 id, 则隐藏
            eiColumn.setVisible(false);
        }
        transformBlock.addMeta(eiColumn);
    }
}
transformBlock.addRows(list);

Map<String, Object> metas = inInfo.getBlock("result").getBlockMeta().getMetas();
//    metas.clear();
Map<String, Object> newMetas =
newInfo.getBlock("result").getBlockMeta().getMetas();
List hideList = new LinkedList();

for (String key : metas.keySet()) {
    if (newMetas.get(key) == null) {
        inInfo.getBlock("result");
        hideList.add(key);
    }
}

newInfo.set("hideList", hideList);
return newInfo;
}

```


新增:

insert

```

public EiInfo insert(EiInfo inInfo) {
    initMetaList();
    EiBlock resultBlock = inInfo.getBlock("result");
    String material = "";
    for (int i = 0; i < resultBlock.getRowCount(); i++) {
        Map<String, String> resultMap = resultBlock.getRow(i); // 取出前台数据
        material = (String) resultMap.get(metaList.get(0));

        inInfo.set("inqu_status-0-material", material);
        EiInfo query = super.query(inInfo, "ES11.query");
        if (query.getBlock("result").getRowCount() > 0) {
            throw new RuntimeException("插入异常: 重复物料! ");
        }

        resultMap.remove(metaList.get(0));
//      tes11_id = resultMap.get(metaList.get(metaList.size() - 1)); 保留, 另一种截取方式
//      resultMap.remove(metaList.get(metaList.size() - 1));
//      metaList.remove(0);
        int colCount = metaList.size() - 1; // 循环, 需要每次插入的字段个数
        int insertRows = resultMap.size() / colCount; // 一共插入的数据条数

        List insertList = new ArrayList(); // 插入表中的数据, 带分隔符"_"
        for (int j = 1; j < insertRows + 1; j++) { // 一共插入 insertRows 条数据, 则需循环
            insertRows 次
                Map insertMap = new HashMap();
                for (int k = 1; k < colCount + 1; k++) {
                    String column = (String) metaList.get(k) + "_" + j;
                    insertMap.put(column, resultMap.get(column));
                }
                insertList.add(insertMap);
        }
        for (int m = 0; m < insertList.size(); m++) { // 插入表中的数据, 最终数据
            int isInsert = 0;
            Map<Object, String> map = (Map) insertList.get(m);
            Map<Object, String> endMap = new HashMap<>(); // 存放最终调用 SQLMap 的数
            据
            for (Map.Entry<Object, String> entry : map.entrySet()) {
                String key = ((String) entry.getKey()).split("_")[0];
                endMap.put(key, entry.getValue());
                if (StringUtils.isEmpty(entry.getValue())) {

```

```

        isInsert = 1;
    }
    if ((key.equals(metaList.get(1)) && !(StringUtils.isEmpty(endMap.get(key)))))
    { // metaList.get(1)为报价，因为是 decimal，需在为空的时候进行置零处理
        endMap.put(key, "0");
    }
    //      map.remove(entry.getKey());
    }
    endMap.put(metaList.get(0), material);
    endMap.put(metaList.get(metaList.size() - 1),
    UUIDHexIdGenerator.generate().toString());

    if (isInsert == 1) { // 第二种方案，属性为空，则不插入这条数据
        if (!StringUtils.isEmpty(endMap.get(3)))
            dao.insert("ES11.insert", endMap);

    }
}

inInfo.set("inqu_status-0-material", material);

}
return query(inInfo);
}

```

修改：

update

```

public EiInfo update(EiInfo inInfo) {
    initMetaList();
    EiBlock resultBlock = inInfo.getBlock("result");
    String material = "";
    for (int i = 0; i < resultBlock.getRowCount(); i++) {
        Map<String, String> resultMap = resultBlock.getRow(i); // 取出前台数据
        material = (String) resultMap.get(metaList.get(0));

        resultMap.remove(metaList.get(0));
        int colCount = metaList.size() - 1; // 循环，需要每次处理的字段个数
        int insertRows = resultMap.size() / colCount; // 一共处理的数据条数

        List insertList = new ArrayList(); // 这个 list 存放除 material 以外的属性 map
        for (int j = 1; j < insertRows + 1; j++) {
            Map insertMap = new HashMap();

```

```

        for (int k = 1; k < colCount + 1; k++) {
            String column = (String) metaList.get(k) + "_" + j;
            insertMap.put(column, resultMap.get(column));
        }
        insertList.add(insertMap);
    }
    for (int m = 0; m < insertList.size(); m++) {
        int isUpdate = 0;
        Map<Object, String> map = (Map) insertList.get(m);
        Map<Object, String> endMap = new HashMap<>();
        for (Map.Entry<Object, String> entry : map.entrySet()) {
            String key = ((String) entry.getKey()).split("_")[0];
            endMap.put(key, entry.getValue());

            if (StringUtils.isEmpty(entry.getValue())) { // 只要有一个属性有值，则这条数据
就得进行更新操作，防止空数据进行更新(无 uuid)
                isUpdate = 1;
            }
            if ((key.equals(metaList.get(1)) && !StringUtils.isEmpty(endMap.get(key))))
            { // metaList.get(1)为报价，因为是 decimal，需在为空的时候进行置零处理
                endMap.put(key, "0");
            }
        }
        endMap.put(metaList.get(0), material);
        int updateFlag = 1;
        if (!StringUtils.isEmpty(endMap.get(metaList.get(metaList.size() - 1)))) {
            updateFlag = 0;
        }
        //      endMap.put(metaList.get(metaList.size() - 1),
        UUIDHexIdGenerator.generate().toString());

        if (isUpdate == 1) {
            if (updateFlag == 1) {
                dao.update("ES11.update", endMap);
            }

            if (updateFlag == 0) { // 当页面属性由无变有进行更新，实际上纵表是进行插入的操作
(第二种方案，属性为空则不插入这条数据)
                endMap.put(metaList.get(metaList.size() - 1),
                UUIDHexIdGenerator.generate().toString());
                dao.insert("ES11.insert", endMap);
            }
        }
    }
}

```

```

        inInfo.set("inqu_status-0-material", material);

    }
    return query(inInfo);

}

```

3.5.3.1.29.3.4 Service 开发说明

首先，定义一个 **metaList**，用来存放列名，以适用于多种数据表。

```

private List metaList;

public List initMetaList(){

    metaList = new ArrayList(); // 以后增加新字段需放在price后, tes11_id前

    metaList.add("material"); // 物料放第一位,固定位
    metaList.add("price"); // 价格放第二位, 固定位
    metaList.add("supplier");
    metaList.add("name");
    metaList.add("tes11_id"); // id放最后一位, 固定位
    return metaList;
}

```

编者习惯，**material** 与 **id** 分别放在第一位和最后一位，固定位，便于获取。**price** 由于是 **BigDecimal** 类型，后续代码需进行取出判断，固也设为固定位(第 2 位)。

3.5.3.1.29.3.4.1 查询代码

queryMaterial 的 SQL 是查出 **distinct** 物料。定义一个 **list** 用来存放转换后的数据结构。

```

@Override
public EiInfo query(EiInfo inInfo) {
    EiInfo materialInfo = super.query(inInfo, sqlName: "ES11.queryMaterial"); // distinct物料, 即转换成几行数据
    EiBlock materialBlock = materialInfo.getBlock( blockId: "result");

    List list = new ArrayList(); // list存放转换后的数据结构
    initMetaList();
}

```

循环每一个 **material**，以此为 **where** 条件，查出数据库中此物料对应的所有供应商数据。循环供应商数据，将列名当成 **key**，**key** 值通过“__”加上递增数字来区别，便于之后的取值。列值做 **value**，存放进 **map** 里面。此时需要注意的是，公共列名 **material** 需单独处理。

```

for (int i = 0; i < materialBlock.getRowCount(); i++) {
    Map<String, Object> map = new LinkedHashMap<>(); // map代表每一行数据
    String material = materialBlock.getCellStr(i, columnName: "material"); // 物料代码
    inInfo.set("ingr_status-0-material", material);
    inInfo.set("result-offset", 0);
    EiInfo info = super.query(inInfo, sqlName: "ES11.query"); // 根据物料代码查询所有供应商
    EiBlock infoBlock = info.getBlock( blockId: "result");

    map.put("material", material);

    for (int j = 0; j < infoBlock.getRowCount(); j++) { // 循环处理所有的供应商
        Map<String, Object> row = infoBlock.getRow(j);
        row.remove( key: "material");

        for (Map.Entry<String, Object> entry : row.entrySet()) { // 将处理过的数据合并至一个map
            map.put(entry.getKey() + "__" + (j + 1), entry.getValue());
        }
    }

    list.add(map);
}

```

可以进断点看一下此时 list 里存放的数据结构正是需求所要的：

```

ServiceES10.java  ES12.js  Evaluate Expression
ServiceES11 query()
    map.put("material", material);
    for (int j = 0; j < infoBlock.ge
    Map<String, Object> row = in
    row.remove( key: "material");
    for (Map.Entry<String, Objec
    map.put(entry.getKey() +
    }
    }
    list.add(map); list: size = 4
    }
    EiInfo newInfo = new EiInfo();
    EiBlock transformBlock = newInfo.add

    EiInfo count = new EiInfo();
    count.set("result-limit", 1000);
    count.set("result-limit", 1000);

Result:
list
Result:
result = [ArrayList@8247] size = 4
  0 = (LinkedHashMap@8248) size = 13
    0 = (LinkedHashMap$Entry@8308) "material" -> "A"
    1 = (LinkedHashMap$Entry@8309) "tes11_id_1" -> "1"
    2 = (LinkedHashMap$Entry@8310) "supplier_1" -> "1"
    3 = (LinkedHashMap$Entry@8311) "name_1" -> "1A"
    4 = (LinkedHashMap$Entry@8312) "price_1" -> "10.00"
    5 = (LinkedHashMap$Entry@8313) "tes11_id_2" -> "2"
    6 = (LinkedHashMap$Entry@8314) "supplier_2" -> "2"
    7 = (LinkedHashMap$Entry@8315) "name_2" -> "2A"
    8 = (LinkedHashMap$Entry@8316) "price_2" -> "20.00"

```

将 list 里的数据加进 EiColumn 里面，从而在页面渲染。

```

EiInfo newInfo = new EiInfo();
EiBlock transformBlock = newInfo.addBlock( blockId: "result"); // 转换后的EiBlock

EiInfo count = new EiInfo();
count.set("result-limit", 1000);
transformBlock.set("count", super.query(count, sqlName: "ES11.queryMaterial").getBlock( blockId: "result").getRowCount());

EiColumn eiColumn = new EiColumn( name: "material");
eiColumn.setDescName("material");
eiColumn.setPrimaryKey(true);
eiColumn.setNullable(true);
eiColumn.setReadOnly(true);
eiColumn.set("enable", false);
transformBlock.addMeta(eiColumn);

for (int i = 0; i < list.size(); i++) {
    Map<String, String> newMap = (Map) list.get(i);
    for (Map.Entry<String, String> entry : newMap.entrySet()) {
        eiColumn = new EiColumn( name: "" + entry.getKey());
        eiColumn.setDescName("" + entry.getKey());
        String column = entry.getKey().split( regex: "[_]" )[0];
        if (column.equals((String)metaList.get(metaList.size() - 1))) { // 如果是id, 则隐藏
            eiColumn.setVisible(false);
        }
        transformBlock.addMeta(eiColumn);
    }
}
transformBlock.addRows(list);

return newInfo;

```

3.5.3.1.29.3.4.2 插入代码

从前台插入数据，首先取出前台需插入的数据，得出每次需插入的字段数以及需转换插入的数据条数。同理，material 需单独处理。

```
public EiInfo insert(EiInfo inInfo) {
    initMetalist();
    EiBlock resultBlock = inInfo.getBlock( blockId: "result");
    String material = "";
    for (int i = 0; i < resultBlock.getRowCount(); i++) {
        Map<String, String> resultMap = resultBlock.getRow(i); // 取出前台数据
        material = (String) resultMap.get(metalist.get(0));

        inInfo.set("inqu_status-0-material", material);
        EiInfo query = super.query(inInfo, sqlName: "ES11.query");
        if (query.getBlock( blockId: "result").getRowCount() > 0) {
            throw new RuntimeException("插入异常: 重复物料!");
        }

        resultMap.remove(metalist.get(0));
        // tes11_id = resultMap.get(metalist.get(metalist.size() - 1)); 保留, 另一种截取方式
        // resultMap.remove(metalist.get(metalist.size() - 1));
        // metalist.remove(0);
        int colCount = metalist.size() - 1; // 循环, 需要每次插入的字段个数
        int insertRows = resultMap.size() / colCount; // 一共插入的数据条数
    }
}
```

通过拼接的 column 取出 key-value，存放在 list 里面。

```
List insertList = new ArrayList(); // 插入表中的数据, 带分隔符"_"
for (int j = 1; j < insertRows + 1; j++) { // 一共插入insertRows条数据, 则需循环insertRows次
    Map insertMap = new HashMap();
    for (int k = 1; k < colCount + 1; k++) {
        String column = (String) metalist.get(k) + "_" + j;
        insertMap.put(column, resultMap.get(column));
    }
    insertList.add(insertMap);
}
```

循环 list，根据分隔符“_”切分出原始 key，并根据原始 key 组建一个新的 map，此 map 的 key 值即与数据库字段对应。这时候我们可进行插入操作了。这里对 price 做了特殊处理，由于不填值传入后台为空，而 price 为 decimal 类型，固当为填入 price 值的时候，我们置零处理。

```
for (int m = 0; m < insertList.size(); m++) { // 插入表中的数据, 最终数据
    int isInsert = 0;
    Map<Object, String> map = (Map) insertList.get(m);
    Map<Object, String> endMap = new HashMap<>(); // 存放最终调用SQLMap的数据
    for (Map.Entry<Object, String> entry : map.entrySet()) {
        String key = ((String) entry.getKey()).split( regex: "[_]" )[0];
        endMap.put(key, entry.getValue());
        if (StringUtils.isEmpty(entry.getValue())) {
            isInsert = 1;
        }
        if ((key.equals(metalist.get(1)) && !StringUtils.isEmpty(endMap.get(key)))) { // metalist.get(1)为报价, 因为是decimal, 需在为空的时候进行置零处理
            endMap.put(key, "0");
        }
        map.remove(entry.getKey());
    }
    endMap.put(metalist.get(0), material);
    endMap.put(metalist.get(metalist.size() - 1), UUIDHexStringGenerator.generate().toString());

    if (isInsert == 1) { // 第二种方案, 属性为空, 则不插入这条数据
        if (!StringUtils.isEmpty(endMap.get(3)))
            dao.insert( "ES11.insert", endMap);
    }

    inInfo.set("inqu_status-0-material", material);
}
return query(inInfo);
}
```

3.5.3.1.29.3.4.3 更新代码

更新的代码与插入的差不多，这里不再赘述。只是在一个地方需要注意，当页面属性由无到有的时候，数据库表实际上是新增了一条数据。当然，我们在插入的时候也可以将逻辑写成属性是无的时候依然插入数据，仅有 material 单列显示。

8a99d9615d4fb017015d4fe476cd0006	F	1	1F	11.00
8a99d9615d4fb017015d4fe476db0007	F	2	2F	22.00
8a99d9615d4fb017015d4fe476eb0008	F			0.00

记录集									
新增									
保存 取消 删除									
material	supplier_1	name_1	price_1	supplier_2	name_2	price_2	supplier_3	name_3	price_3
A	1	1A	10.00	2	2A	20.00	3	3A	30.00
B	1	1B	100.00	2	2B	200.00	3	3B	300.00
C	1	1C	11.00	2	2C	22.00	3	3C	33.00
D	1	222	0.00	2	333	0.00	3	444	0.00
F	1	1F	11.00	2	2F	22.00			0.00

此代码是为了防止开发人员直接在数据库增数据，但同一 material 并未增加至等同于 insertRows 的数据，固之后更新的时候更新不到。

```
int updateFlag = dao.update( $: "ES11.update", endMap);
if(updateFlag == 0){ // 当页面属性由无变有进行更新，实际上纵表是进行插入的操作（第二种方案，属性为空则不插入这条数据）
    endMap.put(metalist.get(metalist.size() - 1), UUIDHexStringGenerator.generate().toString());
    dao.insert( $: "ES11.insert", endMap);
}
```

此时缺少一条 F 的数据，若我们直接在页面进行更新将更新不了(缺少 UUID)，故加入 updateFlag 进行判断，当出现此种情况的时候我们做 insert 操作。

8a99d9615d4fb017015d4fd8dc40003		F	1	1F	11.00				
8a99d9615d4fb017015d4fd8dcf70005		F	2	3	0.00				
☐ B	1	1B	100.00	2	2B	200.00	3	3B	300.00
☐ C	1	1C	11.00	2	2C	22.00	3	3C	33.00
☐ D	1	222	0.00	2	333	0.00	3	444	0.00
☐ F	1	1F	11.00			0.00			

3.5.3.1.29.3.4.4 删除代码

比较简单，根据 material 删除所有数据即可，不再赘述。

```
public EiInfo delete(EiInfo inInfo) {
    EiInfo outInfo = new EiInfo();
    outInfo = super.delete(inInfo, sqlName: "ES11.delete");
    return outInfo;
}
```

注：由于是特殊场景，且改造空间较小，此种方式较为繁琐。一般情况可在前台用 kendoUI 框架进行 groupBy 处理。

3.5.3.1.30 EEDM6033-EFGrid 的 readonly

3.5.3.1.30.1 页面功能简介

本示例页面用于说明，如何在 EFGrid 中使用 readonly。

3.5.3.1.30.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.3.1.30.3 页面开发步骤

3.5.3.1.30.3.1 场景分析

`readonly` 是平台提供给开发人员用于控制列内容的可编辑属性，当 `readonly = true` 时，表示此列内容除了新增外，其余都内容是都是不可编辑。

readonly 通常被开发人员用于 [EFColumn](#)，[EFComboColumn](#) 等 column 属性中，除了能够被用于 column 字段中外，其实也是可以用于 Grid 中，表示整个 Grid 中的列除新增数据外，其余数据均不可进行编辑。

3.5.3.1.30.3.2 页面开发

我们创建页面 EEDM6033.jsp

EEDM6033.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" autoDraw="no" needAuth="false">
    <EF:EFColumn ename="company_name" cname="公司名称" copy="true"/>
    <EF:EFColumn ename="sale_date" cname="上市日期" editType="datetime"/>
    <EF:EFColumn ename="company_page" displayType="url" cname="公司首页"/>
  </EF:EFGrid>
</EF:EFRegion>
```

效果如下:

记录集

新增

<>

10 条/页

工

保存

取消

删除

⚙

✓	公司名称	日期	上市日期	公司首页
✓	MotorolaInj111	2017/11/01 00:00:00		首页1
✓	MotorolaInj111	2017/11/01 00:00:00		首页1
✓	MotorolaInj111			首页1
✓	MotorolaInj111	2017/11/01 00:00:00		首页1
✓	MotorolaInj111	2017/11/01 00:00:00		首页1
✓	111	2017/11/07 00:00:00		
✓	MotorolaInj111	2017/11/01 00:00:00		首页1
✓	MotorolaInj111	2017/11/22 00:00:00		首页2
✓	MotorolaInj111	2017/11/16 00:00:00		首页2
✓	Motorolal	2007-04-06		时间按访问量统计

现在我们是随意编辑里面的列内容。

我们把 EEDM6033.jsp 进行修改，添加上 readonly=true 的属性。

EEDM6033.jsp

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" readonly="true" autoDraw="no" needAuth="false">
    <EF:EFColumn ename="company_name" cname="公司名称" copy="true"/>
    <EF:EFColumn ename="sale_date" cname="上市日期" editType="datetime"/>
    <EF:EFColumn ename="company_page" displayType="url" cname="公司首页"/>
  </EF:EFGrid>
</EF:EFRegion>
```

效果如下:

记录集			
新增	公司名称	上市日期	公司首页
☒	MotorolaInj111	2017/11/01 00:00:00	EED1
☒	MotorolaInj111	2017/11/01 00:00:00	EED1
☒	MotorolaInj111	2017/11/01 00:00:00	EED1
☒	MotorolaInj111	2017/11/01 00:00:00	EED1
☒	MotorolaInj111	2017/11/01 00:00:00	EED1
☒	111	2017/11/07 00:00:00	EED1
☒	MotorolaInj111	2017/11/01 00:00:00	EED1
☒	MotorolaInj111	2017/11/22 00:00:00	EED1
☒	MotorolaInj111	2017/11/16 00:00:00	EED1
☒	Motorolai	2007-04-06	时间段访问量统计

我们无法修改现存的列。表明在 `readonly = true` 的情况下，现存的列是无法被修改，给 Grid 新增一行数据。

记录集			
新增	公司名称	上市日期	公司首页
☒	222	2017-12-14 00:00:00	22
☐	MotorolaInj111	2017/11/01 00:00:00	EED1
☐	MotorolaInj111	2017/11/01 00:00:00	EED1
☐	MotorolaInj111	2017/11/01 00:00:00	EED1
☐	MotorolaInj111	2017/11/01 00:00:00	EED1
☐	MotorolaInj111	2017/11/01 00:00:00	EED1
☐	111	2017/11/07 00:00:00	EED1
☐	MotorolaInj111	2017/11/01 00:00:00	EED1
☐	MotorolaInj111	2017/11/22 00:00:00	EED1
☐	MotorolaInj111	2017/11/16 00:00:00	EED1
☐	Motorolai	2007-04-06	时间段访问量统计

于是我们就能够在新增的行中进行编辑。

3.5.3.1.31 EEDM6035-EFGrid 的行高亮显示

3.5.3.1.31.1 页面功能简介

本示例页面用于说明，如何展示 Grid 的 **行高亮**。

开发人员想要在前端开发本页面所提供的功能，需要对页面进行注册和开发相应的后端服务，详情请参考[开发一个单表增删改查页面](#)。

3.5.3.1.31.2 配置和 API 详情

参考 [EFGrid](#) 中的描述。

3.5.3.1.31.3 页面开发步骤

3.5.3.1.31.3.1 JSP 页面开发

我们创建好页面 `EEDM6035.jsp`，给 `EEDM6035.jsp` 添加一个基本的 `EFGrid`。

`EEDM6035.jsp`

```
<EF:EFPage>
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" autoDraw="no" needAuth="false" >
      <EF:EFColumn ename="product_id" cname="产品 Id" />
      <EF:EFColumn ename="company_name" cname="公司名称"/>
      <EF:EFColumn ename="made_country" cname="产地" />
      <EF:EFColumn ename="price" cname="价格"/>
      <EF:EFColumn ename="company_page" cname="公司首页"/>
    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFPage>
```

```
</EF:EFRegion>
</EF:EFPage>
```

效果如下：

产品Id	公司名称	产地	价格	公司首页
999	huawei	america	150122	EEE1
1111	MotorolaInj111	GB, German, France	150122	EEE1
N7021	MotorolaInj111	GB, German, France	150122	
111EE	111	GB, German, France	11	
test1x	AAAA	GB, German, France	0	
N7022	MotorolaInj111	GB, German, France	150122	EEE1
N70	MotorolaInj113	GB, German, France	150122	EEE
8251	MotorolaInj111	GB, German, France	150122	EEE
SGH-i7161	Samsung	GB, German, France	10.14	http222wwwsamsungcom22
T6281	SonyEricsson	Japan	30	http://www.sonyericsson.com

上面展示的是一个基本的 Grid 页面。

现在我们想给 Grid 指定列(company_page)的值不为空时，该行进行高亮展示，那么如何去处理？

我们可以把这个过程分为两步。

- 第一步：找到 **company_page** 值不为空的 tr
- 第二步：给找到的 tr 添加高亮的 class

这两步我们在 EEDM6035.js 中完成，创建好 EEDM6035.js

EEDM6035.js

```
$(function () {
    // 定义高亮行的 class
    var HIGH_LIGHT_TR = 'high-light-tr';

    // 给 Grid 中 model 所渲染的行添加 Class
    function addClass2Tr(grid, model, className) {
        var trList = grid.element.find("tr[data-uid=" + model.uid + "]");
        if (trList.length > 0 && className) {
            trList.addClass(className);
        }
    }

    // 给 Grid 中 model 所渲染的行移除 Class
    function removeClass2Tr(grid, model, className) {
        var trList = grid.element.find("tr[data-uid=" + model.uid + "]");
        if (trList.length > 0 && className) {
            trList.removeClass(className);
        }
    }

    IPLATUI.EFGrid = {
```

```

"result":{
  dataBound: function (e) {
    // 获取全部的 item 列表
    var items = e.sender.getDataItems(), item;
    for (var i = 0; i < items.length; i++) {
      item = items[i] || {};
      // 筛选出符合条件的数据
      // 过滤出'company_page'列中不为空的行
      if (!IPLAT.isEmptyString(item['company_page'])) {
        // e.sender 表示此 Grid 的实例
        addClass2Tr(e.sender, item, HIGH_LIGHT_TR);
      } else {
        removeClass2Tr(e.sender, item, HIGH_LIGHT_TR)
      }
    }
  },
  // cell 修改后, 不会触发 dataBound 事件, 需要点击保存才会触发
  // 如需要更改后立即发生变化, 可使用 afterEdit 事件作为补充
  afterEdit: function (e) {
    var item = e.model || {};
    if (!IPLAT.isEmptyString(item['company_page'])) {
      addClass2Tr(e.sender, item, HIGH_LIGHT_TR);
    } else {
      removeClass2Tr(e.sender, item, HIGH_LIGHT_TR)
    }
  }
}
}
})

```

效果如下:

记录集					
新增					
	产品id	公司名称	产地	价格	公司首页
☐	999	huawei	america	150122	EEEL
☐	1111	MotorolaInj111	GB,German,France	150122	EEEL
☐	N7021	MotorolaInj111	GB,German,France	150122	
☐	111EE	111	GB,German,France	11	
☐	testlx	AAAA	GB,German,France	0	
☐	N7022	MotorolaInj111	GB,German,France	150122	EEEL
☐	N70	MotorolaInj113	GB,German,France	150122	EEEL
☐	8251	MotorolaInj111	GB,German,France	150122	EEEL
☐	SCH-17161	Samsung	GB,German,France	10.14	http222wwwsamsungcom22
☐	T6281	SonyEricsson	Japan	30	http://www.sonyericsson.com

查看 tr 的元素信息:

```

<div id="ef_grid_result" class="no-scrollbar k-grid k-widget k-editable k-grid-lockedcolumns" data-role="grid" style>
  <div class="k-header k-grid-toolbar">...</div>
  <div class="k-grid-header" style="padding-right: 9px;">...</div>
  <div class="k-grid-content-locked" style="width: 40px; height: 241px;">...</div>
  <div class="k-grid-content k-auto-scrollable" style="width: 1329px;">
    <table role="grid" style="width: 600px; touch-action: none;" data-role="selectable" class="k-selectable" tabindex="0">
      <colgroup>...</colgroup>
      <tbody role="rowgroup">
        <tr data-uid="2dd0b4f1-f1e8-478d-8633-30f06a67c161" role="row" class="high-light-tr">...
          <td role="gridcell">...</td>
          <td role="gridcell">...</td>
          <td role="gridcell">
            <div class="i-validate-helper">america</div>
          </td>
          <td role="gridcell">...</td>
          <td role="gridcell">...</td>
        </tr>
        <tr class="k-alt high-light-tr" data-uid="51e450f8-a7f0-41f8-8cba-958378423763" role="row">...</tr>
        <tr data-uid="d74f642d-2835-44c7-82f4-72bb636dd9e8" role="row" class="k-alt">...</tr>
        <tr class="k-alt" data-uid="8eb81863-d9d3-46d3-a079-399e494ac485" role="row">...</tr>
        <tr data-uid="aded9667-4d1d-4e32-8493-2fa1593f772a" role="row" class="k-alt">...</tr>
        <tr class="k-alt high-light-tr" data-uid="06647d5b-b9ad-4366-a639-4d2af0169947" role="row">...</tr>
        <tr data-uid="h277027h-5831-4a2a-80c7-hcr8e73fd738" role="row" class="k-alt">...</tr>
      </tbody>
    </table>
  </div>
</div>

```

给 `company_page` 不为空的 `tr` 添加上 `'high-light-tr'` 这个 `class` 样式。虽然 `class` 样式添加上去了，但是还没有完成我们高亮的效果。我们现在通过本页面自定义 `CSS` 来实现这样的效果。

修改 `EEDM6035.jsp`:

EEDM6035.jsp

```

<EF:EFPage>
  <jsp:attribute name="header">
    <style>
      #result .high-light-tr {
        background: #ec0303;
      }
    </style>
  </jsp:attribute>
  <jsp:body>
    <EF:EFRegion id="result" title="记录集">
      <EF:EFGrid blockId="result" autoDraw="no" needAuth="false" >
        <EF:EFColumn ename="product_id" cname="产品 Id" />
        <EF:EFColumn ename="company_name" cname="公司名称"/>
        <EF:EFColumn ename="made_country" cname="产地" />
        <EF:EFColumn ename="price" cname="价格"/>
        <EF:EFColumn ename="company_page" cname="公司首页"/>
      </EF:EFGrid>
    </EF:EFRegion>
  </jsp:body>
</EF:EFPage>

```

效果如下：

记录集					
新增 10 条/页 保存 取消 删除					
☐	产品Id	公司名称	产地	价格	公司首页
☒	001	Dupont	Germany	120122	001
☒	111	Motorola111	GB,German,France	150122	001
☐	N7021	Motorola111	GB,German,France	150122	
☐	111EE	111	GB,German,France	11	
☐	test1x	AAAA	GB,German,France	0	
☒	0022	Motorola111	GB,German,France	120122	001
☒	003	Motorola111	GB,German,France	120122	001
☒	004	Motorola111	GB,German,France	120122	001
☒	005-111	Shanghai	GB,German,France	120122	001?param=005-111
☒	006	Motorola111	GB,German,France	120122	001?param=006

3.5.3.1.32 EEDM6036-EFGrid 数据筛选

3.5.3.1.32.1 页面功能简介

本示例页面用于说明，如何使用 **filterable** 属性对列数据进行筛选。

3.5.3.1.32.2 配置和 API 详情

参考 [EFGrid](#) 中的描述。

3.5.3.1.32.3 页面开发步骤

3.5.3.1.32.3.1 JSP 页面开发

EFGrid 示例

```
<EF:EFGrid blockId="result" autoDraw="override" needAuth="false" filterable="true"
pagerPosition="top">
  <EF:EFColumn ename="product_id" cname="产品代号"/>
  <EF:EFColumn ename="company_name" cname="公司名称"/>

  <%-- 数字 --%>
  <EF:EFColumn ename="price" cname="价格" format="{0:c2}"/>

  <%-- 单选下拉列 --%>
  <EF:EFColumn ename="made_country" cname="产地"/>

  <%-- 日期时间 --%>
  <EF:EFColumn ename="sale_date" cname="上市日期" format="yyyy-MM-dd"
    editType="date" displayType="date" type="date"/>

  <%-- 无过滤 --%>
  <EF:EFColumn ename="company_page" cname="公司首页"/>
</EF:EFGrid>
```

3.5.3.1.32.3.2 filterable 构造

3.5.3.1.32.3.2.1 设置操作语句

JS 开发：

JS 示例

```
filterable: {  
  operators: {  
    number: {  
      contains: "包含", // 设置 operators  
      eq: "等于",  
      neq: "不等于",  
      gt: "大于",  
      lt: "小于",  
      gte: "大于等于",  
      lte: "小于等于"  
    },  
    date: {  
      gt: "之后",  
      lt: "之前"  
    },  
    string: {  
      contains: "包含",  
      eq: "等于",  
      neq: "不等于"  
    }  
  }  
}
```

页面效果如下所示：



价格	产地
¥19.	
¥11.	
¥0.	
¥10.	
¥30.	
¥1,200.5	
¥20,000.0	
¥11.0	
¥1,400.0	
¥150,122.0	

上市日期	公司
2018-01-03	
2017-11-07	
2017-12-15	
2017-10-28	
2007-04-06	
2007-03-08	http://www.philips.com
2007-03-10	
2007-07-06	
2007-03-17	http://www.nokia.com
2007-04-27	EEE1

3.5.3.1.32.3.2.2 设置输入框类型

JS 页面开发：

JS 示例

```
dataSource: {
  schema: {
    model: {
      fields: {
        price: {type: "number"}, // 设置 filterable 的类型
        sale_date: {type: "date"} // 设置 sale_date 列为日期类型
      }
    }
  }
}
```

页面效果如下所示：

价格	产地
¥19.	
¥11.	
¥0.	
¥10.	
¥30.	
¥1,200.50	Germany
¥20,000.00	Japan
¥11.00	China
¥1,400.00	Sweden
¥150,122.00	China, German, GB

显示符合以下条件的行

包含

¥0.00

过滤

清除

上市日期	公司首页
2018-01-03	
2017-11-07	
2017-12-15	
2017-10-28	
2007-04-06	on.com
2007-03-08	http://www.philips.com
2007-03-10	
2007-07-06	
2007-03-17	http://www.nokia.com
2007-04-27	EEE1

显示符合以下条件的行

之后



过滤

清除

3.5.3.1.32.3.2.3 渲染 Input 框

输入框格式化

JS 示例

```

{
  field: "price", // 输入框格式化
  filterable: {
    ui: function (element) {
      element[0].id = "text"; // 设置输入框的 id

      $("#text").blur(function () { // 输入框失去焦点后
        var val = $("#text").val();
        $("#text").val(kendo.format("{0:c2}", +val)); // 格式化输入框内的值
      });
    }
  }
}

```


页面效果如下所示：

	价格	产地
	¥19.	
	¥11.	
	¥0.	
	¥10.	
son	¥30.	
	¥1,200.5	
son	¥20,000.0	
	¥11.0	
	¥1,400.0	

显示符合以下条件的行

包含

包含

等于

不等于

大于

小于

大于等于

小于等于

渲染 DropDownList 下拉数据选项

JS 示例

```
{
  field: "made_country", // 单选下拉过滤
  filterable: {
    ui: function (element) {
      console.log(element);
      element.kendoDropDownList({
        dataSource: {
          data: [
            {label: "中国", value: "China"} // 设置下拉选项
          ]
        },
        dataTextField: "label",
        dataValueField: "value"
      });
    }
  }
}
```

页面效果如下所示：

产地	上市日期
China,all	
GB,German	
GB,German,France	
GB,German,France	
Japan	
Germany	2007-03-08
Japan	2007-03-10
China	2007-07-06
Sweden	2007-03-17
China,German,GB	2007-04-27

JS 示例

```
{
  field: "sale_date", // 日期时间
  filterable: {
    ui: function (element) {
      element.kendoDateTimePicker();
    }
  }
}
```

页面效果如下所示：

上市日期	公司首页
2018-01-03	
2017-11-07	
2017-12-15	
2017-10-28	
2007-04-06	
2007-03-08	http://www.philips.com
2007-03-10	
2007-07-06	
2007-03-17	http://www.nokia.com
2007-04-27	EEE1

3.5.3.1.32.3.2.4 关闭过滤功能

JS 开发：

JS 示例

```
{
```

```

field: "product_id", // 不设置过滤
filterable: false
}

```

页面效果如下所示：

☐ *产品代号	公司名称 ▾	价格 ▾	产地 ▾	上市日期 ▾	公司首页 ▾
☐ abcde	samsung9	¥19.00	China,all	2018-01-03	newPage
☐ 111EE	111	¥11.00	GB,German	2017-11-07	
☐ test1x	AAAAA	¥0.00	GB,German,France	2017-12-15	
☐ SGH-i7162	Samsung	¥10.00	GB,German,France	2017-10-28	http22wwwsamsungcom22
☐ T6281	SonyEricsson	¥30.00	Japan	2007-04-06	http://www.sonyericsson.com
☐ S891	Philips	¥1,200.50	Germany	2007-03-08	http://www.philips.com
☐ 8825	SonyEricsson	¥20,000.00	Japan	2007-03-10	
☐ 111	Motorola	¥11.00	China	2007-07-06	
☐ XXX1	Motorola	¥1,400.00	Sweden	2007-03-17	http://www.nokia.com
☐ weewe001	MotorolaInj1	¥150,122.00	China,German,GB	2007-04-27	EEE1

3.5.3.1.32.3.2.5 针对某一列设置不同的操作语句

JS 示例

```

{
  field: "company_name",
  filterable: {
    operators: {
      string: {
        contains: "包含",
        eq: "等于",
        neq: "不等于",
        startWith: "开始于",
        endwith: "结束于"
      }
    }
  }
}

```

页面效果如下所示：

公司名称	价格
sumsong9	¥1
111	¥1
AAAAA	¥
Samsung	¥1
SonyEricss	¥3
Philips	¥20
SonyEricss	¥00
Motorola	¥1
Motorola	¥1,40
Motorolalnjl	¥150,12

显示符合以下条件的行

包含

包含

等于

不等于

开始于

结束于

3.5.3.1.32.3.3 功能实现

Service 服务开发:

filterable

```

package com.baosight.iplat4j.ee.dm.service;

import com.baosight.iplat4j.core.ei.EiInfo;
import com.baosight.iplat4j.core.service.impl.ServiceEPBase;
import com.baosight.iplat4j.core.util.StringUtils;
import com.baosight.iplat4j.ee.domain.EE10;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.HashMap;

/**
 * 数据筛选示例
 */
public class ServiceEEDM6036 extends ServiceEPBase {

    private HashMap<String, String> operationList = null;

    public ServiceEEDM6036() {
        this.operationList = new HashMap<>();
        operationList.put("gt", "%s > '%s'");
        operationList.put("gte", "%s >= '%s'");
        operationList.put("eq", "%s = '%s'");
        operationList.put("neq", "%s <> '%s'");
        operationList.put("lt", "%s < '%s'");
        operationList.put("lte", "%s <= '%s'");
    }

```

```

        operationList.put("contains", "%s like '%%s%%'");
        operationList.put("startswith", "%s like '%%s%%'");
        operationList.put("endswith", "%s like '%%s%'");
    }

    private static String getDateWithTimestamp(String timestamp, String timeFormat) throws
    ParseException {
        SimpleDateFormat format = new SimpleDateFormat(timeFormat);
        Long time = new Long(timestamp);
        return format.format(time);
    }

    private String findOperation(String operation) {
        return this.operationList.get(operation);
    }

    // 生成过滤 SQL, 存在于 inqu_status-0-filter_field(field => 字段名称)字段中, 需要在 XML 中
    进行配置

    private void buildFilterSQL(EiInfo info, String field, String fieldValue) {
        if (!StringUtils.isEmpty(fieldValue)) {
            fieldValue = info.getCellStr("inqu_status", 0, field);
        }
        String fieldOperation = info.getCellStr("inqu_status", 1, field);
        if (StringUtils.isEmpty(fieldValue) && StringUtils.isEmpty(fieldOperation)) {
            String operation = findOperation(fieldOperation);
            if (StringUtils.isEmpty(operation)) {
                info.set(String.format("inqu_status-0-filter_%s", field), String.format(operation,
                field, fieldValue));
            }
        }
    }

    private void buildFilterSQL(EiInfo info, String field) {
        String fieldValue = info.getCellStr("inqu_status", 0, field);

        buildFilterSQL(info, field, fieldValue);
    }

    private void buildFilterSQL(EiInfo info, String field, String fieldValue, String format)
    throws ParseException {
        if (!StringUtils.isEmpty(fieldValue)) {
            fieldValue = info.getCellStr("inqu_status", 0, field);

```

```

    }
    String fieldOperation = info.getCellStr("inqu_status", 1, field);
    if (!StringUtils.isEmpty(format)) {
        return;
    }

    if (StringUtils.isEmpty(fieldValue) && StringUtils.isEmpty(fieldOperation)) {
        try {
            fieldValue = getDateWithTimestamp(fieldValue, format);
        } catch (ParseException ex) {
            throw ex;
        }
        buildFilterSQL(info, field, fieldValue);
    }
}

@Override
public EilInfo initLoad(EilInfo inInfo) {
    return query(inInfo);
}

@Override
public EilInfo query(EilInfo inInfo) {
    // 生成日期(sale_date)过滤条件
    try {
        buildFilterSQL(inInfo, "sale_date", null, "yyyy/MM/dd");
    } catch (Exception e) {
        e.printStackTrace();
    }

    // 生成价格(price)过滤条件
    buildFilterSQL(inInfo, "price");

    // 生成公司名称(company_name) 过滤条件
    buildFilterSQL(inInfo, "company_name");

    return super.query(inInfo, "EE10.filter", new EE10());
}

@Override
public EilInfo delete(EilInfo inInfo) {
    return super.delete(inInfo, "EE10.delete");
}

```

```

@Override
public EilInfo update(EilInfo inInfo) {
    return super.update(inInfo, "EE10.update");
}

@Override
public EilInfo insert(EilInfo inInfo) {
    return insert(inInfo, "EE10.insert");
}

}

```

EE10.xml:

EE10.xml

```

<select id="filter" resultClass="java.util.HashMap">
    SELECT
    product_id as "product_id",
    company_name as "company_name",
    made_country as "made_country",
    sale_date as "sale_date",
    price as "price",
    company_page as "company_page",
    search as "search",
    validate_0 as "validate_0",
    validate_1 as "validate_1",
    validate_2 as "validate_2",
    validate_3 as "validate_3",
    validate_4 as "validate_4",
    validate_5 as "validate_5",
    validate_6 as "validate_6",
    validate_7 as "validate_7",
    validate_8 as "validate_8",
    validate_9 as "validate_9"
    FROM ${platSchema}.tee10 where 1=1

    <isEmpty prepend=" AND " property="filter_company_name">
        $filter_company_name$
    </isEmpty>

    <isEmpty prepend=" AND " property="filter_sale_date">
        $filter_sale_date$
    </isEmpty>

```

```

</isEmpty>

<isEmpty prepend=" AND " property="filter_price">
    $filter_price$
</isEmpty>

<dynamic prepend="ORDER BY">
    <isEmpty property="orderBy">
        $orderBy$
    </isEmpty>
</dynamic>
</select>

```

3.5.3.1.33 EEDM6037-EFGrid 前端导出显示值示例

3.5.3.1.33.1 页面功能简介

此前 [EEDM6009-EFGrid Excel 数据导出](#) 示例介绍了如何对 EFGrid 内各列真实值数据进行前后端导出，本示例页面用于说明 如何在 **前端导出 EFGrid 当前页中各列的显示值数据**（不包含序号列、隐藏列）

3.5.3.1.33.2 配置和 API 详情

参考示例 [EEDM6009](#) 中的描述。

3.5.3.1.33.3 页面开发步骤

3.5.3.1.33.3.1 JSP 页面开发

EEDM6037.jsp

```

<EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" autoDraw="mixed" autoFit="true">
        <EF:EFColumn ename="product_id" cname="产品代号【主键】" primaryKey="true"
readonly="true" locked="true"/>
        <EF:EFColumn ename="company_name" cname="公司名【固定列】" locked="true"/>
        <EF:EFColumn ename="made_country" cname="产地【隐藏列】" hidden="true"/>

        <EF:EFColumn ename="sale_date" cname="上市日期【日期列】" editType="date"
displayType="date"
            dateFormat="yyyy/MM/dd hh:mm:ss"/>
        <EF:EFColumn ename="search" cname="查询【文本列】" editType="textarea"/>

        <EF:EFComboColumn ename="price" cname="价格【下拉列】"
columnTemplate="#=textField#-#=valueField#"

```



```

        itemTemplate="#=textField#-#=valueField#"
        textField="textField" valueField="valueField" optionLabel="请选择">
        <EF:EFOption label="价格" value="10"/>
        <EF:EFOption label="价格" value="20"/>
        <EF:EFOption label="价格" value="30"/>
    </EF:EFComboColumn>

    <EF:EFPopupColumn ename="company_page" cname="公司首页【弹出列】"
    popupType="ServiceGrid" resultId="result"
        popupTitle="弹出服务 Grid" popupWidth="400"
        serviceName="EEDM6013" methodName="query"
        columnEnames="company_page,made_country,price"
        columnCnames="公司首页,产地,价格"
        backFillFieldIds="company_page,made_country,price"
        backFillColumnIds="company_page,made_country,price"
        valueField="company_page"
        copy="true" resizable="true"
        filterPopupGrid="false"/>
</EF:EFGrid>
</EF:EFRegion>

```

3.5.3.1.33.3.2 JS 开发

EEDM6037.js

```

$(function () {
    IPLATUI.EFGrid = {
        "result": {
            "exportGrid": {
                exportEiInfo: function () {
                    return resultGrid.getDisplayEiInfo() // 点击导出时，执行函数，产生导出的 EiInfo
                }
            }
        }
    };
});

```

3.5.3.1.33.3.3 页面效果

记录集									
☐	*产品代号【主键】	公司名【固定列】	上市日期【日期列】	价格【下钻列】	公司首页【弹出式...】	查询【文本列】	验证码0	验证码1	验证码字符串2
☐	abcde22	sunaong9	2017/01/01 12:34:356	19.2	http://platform.b...	Africa	baoeight666666666666.cc	https://static.bootcss.com/assets/img/typescript.png	导出数据
☐	abcde23	sunaong9	2017/01/01 12:34:356	19.2	http://platform.b...	Oceania	baoeight666666666666.cc	https://static.bootcss.com/www/assets/img/typescript.png	
☐	abcde	sunaong9	2018/01/03 12:00:00	19.2	newAge	Antarctica	baoight666666666666.cc	https://static.bootcss.com/www/assets/img/typescript.png	
☐	abcde24	as<chr>aa<chr>b c c<	2017/01/01 12:34:356	11	http://platform.b...	America	ss	ss	ss
☐	1524732108916		2018/01/03 12:00:00	19.2	newAge	Africa,Antarctica	baoight666666666666.cc	https://static.bootcss.com/www/assets/img/typescript.png	
☐	abcde222	sunaong9	2017/01/01 12:34:356	19.2	http://platform.b...	Oceania	baoeight666666666666.cc	https://static.bootcss.com/www/assets/img/typescript.png	
☐	111EE	111	2017/11/07 09:00:00	111		Africa,Antarctica	abcfbaosight	https://static.bootcss.com/www/assets/img/typescript.png	
☐	1524732125294	sunaong9	2018/01/03 12:00:00	19.2	newAge	Africa,Antarctica	baoight666666666666.cc	https://static.bootcss.com/www/assets/img/typescript.png	
☐	abcde242	sunaong9	2017/01/01 12:34:356	19.2	http://platform.b...	Africa,Antarctica	baoight666666666666.cc	https://static.bootcss.com/www/assets/img/typescript.png	
☐	2131		2018/04/26 00:00:00	1	123	Africa,Antarctica	123	123	13

3.5.3.1.33.3.4 导出 Excel 效果

产品代码【主键】	公司名称【主键列】	上市日期【主键列】	价格【主键列】	公司网页【主键列】	番薯【主键列】	验证码0	验证码1	验证码2	验证码3
abdc22	sumsung1	2017/10/11242356	19.2	http://platform.baosight.com	Africa	baosight66666666666666	https://static.bootcss.com/view/asset/img/hypercentr.png		
abdc23	sumsung1	2017/10/11242356	19.2	http://platform.baosight.com	Oseania	baosight66666666666666	https://static.bootcss.com/view/asset/img/hypercentr.png		
abdc24	sumsung1	2018/1/03 12:00:00	19.2	newPage	Antarctica	baosight66666666666666	https://static.bootcss.com/view/asset/img/hypercentr.png		
abdc24	sa-bz-aa-bb-bb-bc-cc	2017/10/11242356	11	http://platform.baosight.com	America	ss	ss	ss	ss
1524732108916	sumsung1	2018/1/03 12:00:00	19.2	newPage	Africa,Antarctica	baosight66666666666666	https://static.bootcss.com/view/asset/img/hypercentr.png		
abdc222	sumsung1	2017/10/11242356	19.2	http://platform.baosight.com	Oseania	baosight66666666666666	https://static.bootcss.com/view/asset/img/hypercentr.png		
111EE	111	2017/11/07 00:00:00	111		Africa,Antarctica	abc@baosight	https://static.bootcss.com/view/asset/img/hypercentr.png		
1524732125294	sumsung1	2018/01/03 12:00:00	19.2	newPage	Africa,Antarctica	baosight66666666666666	https://static.bootcss.com/view/asset/img/hypercentr.png		
abdc242	sumsung1	2017/10/11242356	19.2	http://platform.baosight.com	Africa,Antarctica	baosight66666666666666	https://static.bootcss.com/view/asset/img/hypercentr.png		

3.5.3.1.34 EEDM6062-EFGrid 时间控件的校验

3.5.3.1.34.1 页面功能简介

此页面用于展示 EFGrid 里的时间控件需要进行时间校验的功能，如开始时间不能晚于结束时间。

3.5.3.1.34.2 配置和 API 详情

参考 [EFGGrid](#) 表格中的描述。

3.5.3.1.34.3 页面开发步骤

3.5.3.1.34.3.1 JSP 页面开发

Grid 里存在 UUID，开始时间，结束时间三列。

JSP 示例

```
<EF:EFPage title="EFGrid 时间控件的校验">
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" needAuth="false" autoDraw="no">
      <EF:EFColumn ename="uuid" locked="true" width="120" cname="uuid"
hidden="true"/>
      <EF:EFColumn ename="start" width="100" cname="开始时间" required="true"
defaultValue="${now}" editType="datetime"/>
      <EF:EFColumn ename="end" width="100" cname="结束时间"
defaultValue="${formatDate}" editType="datetime"/>
    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFPage>
```

3.5.3.1.34.3.2 JS 开发

JS

```

$(function () {

    IPLATUI.EFGrid = {
        "result": {
            columns: [
                {
                    field: "start",
                    editor: function (container, options) {

                        var input = $('<input />');
                        input.attr("name", options.field);
                        input.attr("id", options.field);
                        input.appendTo(container);

                        var dateOptions = {
                            datelId: "start",
                            format: "yyyy-MM-dd HH:mm:ss",
                            role: "datetime"
                        };
                        date(dateOptions,options);
                    }
                },
                {
                    field:"end",
                    editor:function(container,options){
                        var input = $('<input />');
                        input.attr("name", options.field);
                        input.attr("id", options.field);
                        input.appendTo(container);
                        var dateOptions = {
                            datelId: "end",
                            format: "yyyy-MM-dd HH:mm:ss",
                            role: "datetime"
                        };
                        date(dateOptions,options);
                    }
                }
            ]
        }
    };
});

```

```

var date = function (options,model) {

    var dateInstance, $date = $("#" + options.datelId);

    if ("datetime" === options.role) {

        dateInstance = $date.kendoDateTimePicker({
            open: function (e) {
                if(options["datelId"] == "start"){
                    var endDate = new Date(model["model"]["end"]);
                    this.max(endDate);
                }
                if(options["datelId"] == "end"){
                    var startDate = new Date(model["model"]["start"]);
                    this.min(startDate);
                }
            }
        })
    } else {
        dateInstance = $date.kendoDatePicker(options).data("kendoDatePicker");
    }

    return dateInstance;

};

```

在 js 里面，定位到时间列，使用 **columns.editor** 事件，在 **kendoDateTimePicker.open** 事件里处理弹出渲染的逻辑。

当点击开始时间控件的时候，去取到结束时间的值，将弹出的时间控件里的最大时间设为结束时间。

当点击结束时间控件的时候，去取到开始时间的值，将弹出的时间控件里的最小时间设为开始时间。

3.5.3.1.35 EEDM6400-Grid 列别名使用示例

3.5.3.1.35.1 页面功能简介

Grid 中数据来自多张表连接时，前台可以设置列的 **alias** 属性来避免列名冲突，具体使用如下。

3.5.3.1.35.2 配置和 API 详情

参考 [EFGGrid 表格](#) 中的描述。

3.5.3.1.35.3 页面开发步骤

3.5.3.1.35.3.1 JSP 页面开发

JSP 示例

```
// EEDM6400.jsp
<EF:EFPage>
  <EF:EFRegion id="inqu" title="查询条件">
    <div class="row">
      <EF:EFInput ename="inqu_status-0-product_id" cname="产品代号"/>
      <EF:EFInput ename="inqu_status-0-company_name" cname="公司名称"/>
    </div>
  </EF:EFRegion>

  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" checkMode="single cell" autoDraw="no" sort="all">
      <%--开启 sort 排序时, 此处 alias 别名会替换掉 sql 中原列名,
        其中't1'是 sql 中给当前列所在表取的别名,具体见 sql 语句--%>
      <EF:EFColumn ename="product_id" locked="true" cname="产品编号"
alias="t1.product_id"/>
      <EF:EFColumn ename="company_name" cname="公司名称"
alias="t1.company_name"/>
      <EF:EFColumn ename="made_country" cname="产地" alias="t1.made_country"/>

      <EF:EFColumn ename="validate_0" cname="结点英文名" alias="t2.ename"/>
      <EF:EFColumn ename="validate_1" cname="结点中文名" alias="t2.cname"/>

    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFPage>

// EEDM6400.js
$(function () {
  $("#F1").on("click", function () {
    resultGrid.dataSource.page(1);
  });
});
```

3.5.3.1.35.3.2 Service 开发

Service 示例

```
/**
 * ServiceEEDM6400 列别名示例
```

```

*/
public class ServiceEEDM6400 extends ServiceEPBase {

    public EilInfo initLoad(EilInfo inInfo) {

        return query(inInfo);
    }

    @Override
    public EilInfo query(EilInfo inInfo) {
        return super.query(inInfo, "EE10.queryAlias", new EE10());
    }

    public EilInfo delete(EilInfo inInfo) {
        return super.delete(inInfo, "EE10.delete");
    }

    public EilInfo update(EilInfo inInfo) {
        return super.update(inInfo, "EE10.update");
    }

    public EilInfo insert(EilInfo inInfo) {
        return insert(inInfo, "EE10.insert");
    }
}

```

3.5.3.1.35.3.3 sql 语句

sql

```

...
<select id="queryAlias" resultClass="java.util.HashMap">
    SELECT
    t1.product_id as "product_id",
    t1.company_name as "company_name",
    t1.made_country as "made_country",
    t1.price as "price",
    t2.ename as "validate_0",
    t2.cname as "validate_1",
    t2.node_type as "validate_2"
    FROM ${platSchema}.tee10 t1 inner join ${platSchema}.teedm06 t2
    on t1.made_country = t2.ename

```

```

<isEmpty prepend=" AND " property="product_id">
    t1.product_id like ('%$product_id$')
</isEmpty>
<isEmpty prepend=" AND " property="company_name">
    t1.company_name like ('%$company_name$')
</isEmpty>

<!--配合前台排序-->
<dynamic prepend="ORDER BY">
    <isEmpty property="orderBy">
        $orderBy$
    </isEmpty>
</dynamic>
</select>
...

```

3.5.3.1.35.3.4 页面效果

查询编号包含"1"，且根据产地名升序排序结果：



The screenshot shows a web interface for EEDM6009-EFGrid. At the top, there's a search bar with '产品编号' (Product ID) set to '1' and '公司名称' (Company Name) empty. Below the search bar is a table of results. The table has columns: '产品编号' (Product ID), '公司名称' (Company Name), '产地' (Origin), '结点英文名' (Node English Name), and '结点中文名' (Node Chinese Name). The results are sorted by origin (China, Germany, Japan, USA).

产品编号	公司名称	产地	结点英文名	结点中文名
111	Motorola	China	China	中国
S891	Philips	Germany	Germany	德国
T6281	SonyEricsson	Japan	Japan	日本
HK1000	11122	USA	USA	美国
MIX111	11122	USA	USA	美国

此时后台输出查询语句：

```

%2018-04-10 14:08:39 INFO [http-nio-8080-exec-10] c.b.i.c.d.i.d.SqlMapDaoLogProxy - SELECT t1.product_id as "product_id",
t1.company_name as "company_name", t1.made_country as "made_country", t1.price as "price", t2.ename as "validate_0",
t2.cname as "validate_1", t2.node_type as "validate_2" FROM IPLAT.tee10 t1 inner join IPLAT.teedm06 t2 on t1
.made_country = t2.ename AND t1.product_id like ('%1%') ORDER BY
t1.made_country asc fetch first 10 rows only

```

3.5.3.1.36 EEDM6513-EFGrid 后台导出自定义列

3.5.3.1.36.1 页面功能简介

此前 [EEDM6009-EFGrid Excel 数据导出](#) 示例介绍了如何对 EFGrid 的数据进行前后端导出，本示例页面用于说明 如何在后台导出自定义列。

3.5.3.1.36.2 配置和 API 详情

参考示例 [EEDM6009](#) 中的描述。

3.5.3.1.36.3 页面开发步骤

3.5.3.1.36.3.1 JSP 页面开发

渲染 Grid

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" autoDraw="yes" needAuth="false" checkMode="single"
autoBind="false">
  </EF:EFGrid>
</EF:EFRegion>
```

3.5.3.1.36.3.2 JS 开发

配置后台查询方法名，服务名

```
$(function () {

  IPLATUI.EFGrid = {
    "result": {
      "exportGrid": {
        /**
         * 导出前的事件
         *
         * @param gridInstance kendoGrid 对象
         * @return {boolean} 是否执行导出的逻辑
         */
        beforeExport: function(gridInstance) {
          return true; // false 不执行导出，true 时执行导出
        },

        // 如果配置了导出服务名和导出方法名，将调用对应的服务名和方法名，在后台产生要导
        出的 EiInfo
        exportServiceName: "EEDM6513", // 配置导出服务名，默认和 grid 的 serviceName
        相同
        exportMethodName: "query", // 配置导出方法名，默认和 grid 的 queryMethod 相
        同

        exportFileName: function(gridInstance) {
          // 导出的文件名包含时间戳 yyyyMMddHHmmss
          return "result" + kendo.toString(new Date(), IPLAT.FORMAT.DATE_14);
        },

        exportFileType: "xls", // 默认值是 xls，可选值为 pdf[规划中]
```



```

        exportBlockId: "result" // 默认值和 blockId 相同，导出的 EiInfo 中的指定数据块
    }
}
};
});

```

此处配置了 `exportServiceName: "EEDM6513"` 以及 `exportMethodName: "query"`，那么会在后台产出要导出的 `EiInfo`。

3.5.3.1.36.3.3 Service 开发

Service 服务

```

package com.baosight.iplat4j.ee.dm.service;

import com.baosight.iplat4j.core.ei.EiBlock;
import com.baosight.iplat4j.core.ei.EiColumn;
import com.baosight.iplat4j.core.ei.EiConstant;
import com.baosight.iplat4j.core.ei.EiInfo;
import com.baosight.iplat4j.core.service.impl.ServiceEPBase;
import com.baosight.iplat4j.ee.domain.EE10;

import java.util.*;

/**
 * 后台导出自定义列名及顺序
 */
public class ServiceEEDM6513 extends ServiceEPBase {

    public EiInfo initLoad(EiInfo inInfo) {

        //    return inInfo;
        EiInfo outInfo = super.query(inInfo, "EE10.query", new EE10());
        return outInfo;
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        EiInfo outInfo = super.query(inInfo, "EE10.query", new EE10());

        outInfo.getBlocks().put("detail", outInfo.getBlock(EiConstant.resultBlock)); // 增加
        block:detail
        //    outInfo.getBlocks().put(EiConstant.resultBlock,

```

```

outInfo.getBlock(EiConstant.resultBlock));
    outInfo.getBlocks().remove(EiConstant.resultBlock); // 移除 block:result

    List list = new LinkedList();
    List list2 = new LinkedList();
    Map map = new LinkedHashMap();
    list.add("公司首页");
    list.add("验证字段 0");
    list.add("公司名称");
    list.add("产品代号");
    list.add("自定义列");

    for (int i = 0; i < list.size(); i++) {
        map.put(list.get(i), i);
        list2.add(map);
    }

    EiBlock transformBlock = outInfo.addBlock("result");
    for (int i = 0; i < list2.size(); i++) {
        //      Map<String, String> newMap = (Map) list.get(i);
        HashMap<String, String> newMap = (HashMap) list2.get(i);
        for (Map.Entry<String, String> entry : newMap.entrySet()) {
            EiColumn eiColumn = new EiColumn("" + entry.getKey());
            eiColumn.setDescName("" + entry.getKey());
            transformBlock.addMeta(eiColumn); // 添加列
        }
    }

    transformBlock.addRows(list2); // 增加行数据

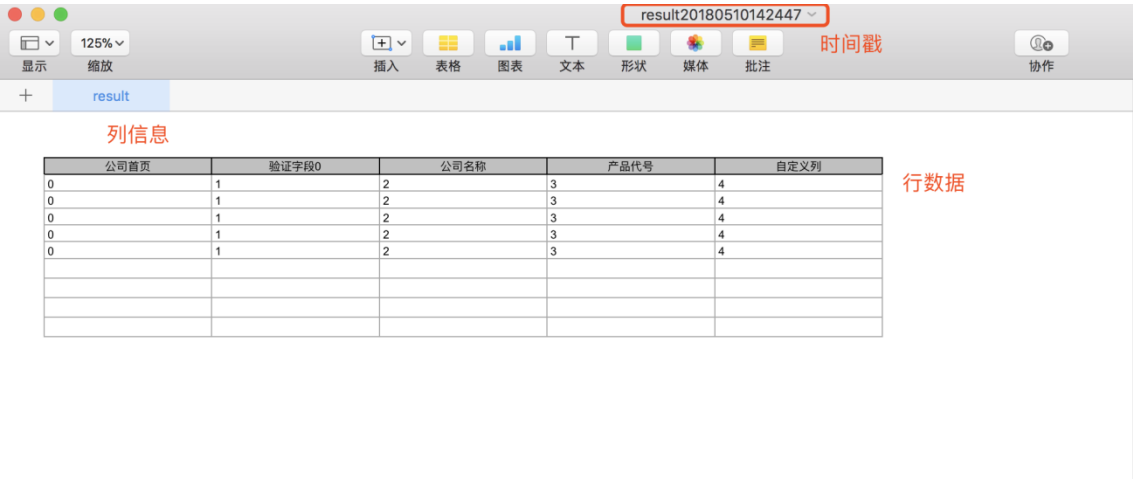
    return outInfo;
}
}

```

3.5.3.1.36.3.4 页面效果如下图所示



3.5.3.1.36.3.5 导出 Excel 效果



3.5.3.1.37 EEDM6531-自定义删除、添加数据行

3.5.3.1.37.1 页面功能简介

本示例页面用于说明，如何使用 [dataSource](https://docs.telerik.com/kendo-ui/api/javascript/ui/grid/configuration/datasource) <https://docs.telerik.com/kendo-ui/api/javascript/ui/grid/configuration/datasource> 选项进行添加、删除数据项。

3.5.3.1.37.2 配置和 API 详情

参考 [EFGGrid](#) 中的描述。

方法名	参数	返回值	说明
1 resultGrid.dataSource.at(Number)	index Number 索引	有数据 返回指定的数据 无数据 undefined	返回指定索引处的数据，索引从0开始
2 resultGrid.dataSource.remove(dataItem)	dataItem 需要被移除的数据		从dataSource中删除数据
3 resultGrid.dataSource.sync()		Promise 同步操作的最终状态(完成或失败)	保存任何数据的更改 * 在数据被增删改的时候，需要执行该方法
4 resultGrid.setCellValue(li	list 单行号或者多行号		设置Grid中一行或多列记录的

方法名	参数	返回值	说明
st, field, cellValue)	, 也可以是某行数据的Model对象 field 列字段英文名 cellValue 设置列的值		字段值（批量赋值）
5 resultGrid.setCheckedRows(list)	list 可以为单个数字或者字符如：0 或者"0", 也可以是包含数字的数组如 [0,1,2] 或者 [0,1,"2"]		设置Grid中勾选的行 * 如果数组中出现数字与非数字, 会对数字标识的行进行选中

3.5.3.1.37.3 页面开发步骤

3.5.3.1.37.3.1 JSP 页面开发

EFGrid 示例

```

<EF:EFRegion id="inqu" title="自定义按钮区">
  <div class="row">
    <%-- 自定义的按钮 --%>
    <div class="col-md-3">
      <h5 class="text-white bg-danger" style="padding: 5px 10px; margin-left:10px; width:300px;">
        自定义删除一行数据(当前页第 5 行数据)
      </h5>
      <span class="i-btn-lg deleteRow5">自定义删除行</span>
    </div>

    <div class="col-md-3">
      <h5 class="text-white bg-primary" style="padding: 5px 10px; margin-left:10px; width:330px;">
        自定义修改一行数据(第 1 行 company_page 列值)
      </h5>
      <span class="i-btn-lg updateOneRow">自定义修改行</span>
    </div>

    <div class="col-md-3">
      <h5 class="text-white bg-success" style="padding: 5px 10px; margin-left:10px; width:300px;">
        自定义复制新增前 3 行数据
      </h5>

```

```

        <span class="i-btn-lg insertRows">自定义添加行</span>
    </div>

    <div class="col-md-3">
        <h5 class="text-white bg-success" style="padding: 5px 10px; margin-left:10px; width:300px;">
            EiCommunicator 复制新增前 3 行数据
        </h5>
        <span class="i-btn-lg insertRows2">EiCommunicator 添加行</span>
    </div>
</EF:EFRegion>

<!-- Grid -->
<EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" showCount="false" personal="true" needAuth="false"
        filterable="true">
    </EF:EFGrid>
</EF:EFRegion>

```

页面效果如下所示：

自定义删除 添加Grid数据行示例 > EEDM8531

自定义按钮区

自定义删除一行数据(当前页第5行数据) 自定义修改一行数据(第1行company_page列值) 自定义复制新增前3行数据 EiCommunicator复制新增前3行数据

自定义删除行 自定义修改行 自定义添加行 EiCommunicator添加行

记录集

产品代码	公司名称	产地	上市日期	价格	公司网页	Google...	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	验证字段7	验证字段8
abcde	eumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...		baosight6...	https://s...						
111EE	111	GB,German...	2017/11/0...	111		America		abc8baos1...	https://s...						
test1x	AAAAA	GB,German...	2017/12/1...	201	98.2			abc8baos1...	https://s...						
2131		Rus	2018/04/2...	1	123	111EE	123	123	13	13	31				
abcd2e2	eumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...		baosight6...	https://s...						
444rrr	GB		2018/04/1...	445	France	test1x									
76281	SonyErica...	Japan	2007-04-06	100.01	http://www.sonyer...	查询	138000000...	abc8baos1...	Label	021-50801...	1	2000		127.0.0.1	中文
152472248...	eumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...		baosight6...	https://s...						
152472248...	111	GB,German...	2017/11/0...	111		America		abc8baos1...	https://s...						
8891	Philips	Germany	2007/03/08	98.2	http://www.philip...	search	138000000...	aaa8123.c...	Label	021-50801...	1	2000	1.02	127.0.0.1	中文

3.5.3.1.37.3.2 JS 开发

自定义删除一行数据

e.g. 此处删除当前页第五行数据。

JS 示例

```

// 自定义删除当前页第 5 行数据
$(".deleteRow5").on("click", function () {
    var row5DataItem = resultGrid.dataSource.at(4); // 选中第 5 行数据
    resultGrid.dataSource.remove(row5DataItem); // 从 dataSource 中删除该行
    resultGrid.dataSource.sync(); // 提交删除

```

```
});
```

解析:

- `resultGrid.dataSource.at(4)` 返回第五行数据（索引从 0 开始）
 - `resultGrid.dataSource.remove(row5DataItem)`：删除第五行数据
 - `resultGrid.dataSource.sync()`：保存「删除」这一更改

页面效果如下所示：

自定义删除 添加Grid数据行示例 > EEDM6531

自定义按钮区

自定义删除一行数据(当前页第5行数据) 自定义修改一行数据(第1行company_page列值) 自定义复制新增前3行数据 EICommunicator复制新增前3行数据

自定义删除行 自定义修改行 自定义添加行 EICommunicator添加行

记录集

新增	产品代号	公司名称	产地	上市日期	价格	公司首页	Google...	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	验证字段7	验证字段8
☐	abode	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...	baonight6...	https://s...				1			NOKIA
☐	152473215...	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...	baonight6...	https://s...				1			NOKIA
☐	11188	111	GB,German	2017/11/0...	111		America	abofbaos1...	https://s...				1			
☐	152473212...	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...	baonight6...	https://s...				1			NOKIA
☐	2131		Rua	2018/04/2...	1	123	11188	123	123	13	13	31	0			
☐	152473263...	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...	baonight6...	https://s...				1			NOKIA

自定义修改一行数据

e.g. 此处修改第 1 行 `company_page` 列值。

JS 示例

```
// 自定义修改一行数据
$(".updateOneRow").on("click", function () {
    resultGrid.setCellValue(0, "company_page", "newPage");
    resultGrid.setCheckedRows(0);
    resultGrid.dataSource.sync();
});
```

解析:

- `resultGrid.setCellValue(0, "company_page", "newPage")`：设置第一行，列英文名为：“`company_page`”的值，值为“`newPage`”
- `resultGrid.setCheckedRows(0)`：设置选中行，即第一行 □ 保存修改

页面效果如下所示：

自定义删除 添加Grid数据行示例 > EEDM6531

自定义按钮区

自定义删除一行数据(当前页第5行数据) 自定义修改一行数据(第1行company_page列值) 自定义复制新增前3行数据 EICommunicator复制新增前3行数据

自定义删除行 自定义修改行 自定义添加行 EICommunicator添加行

记录集

新增	产品代号	公司名称	产地	上市日期	价格	公司首页	Google...	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5	验证字段6	验证字段7	验证字段8
☐	abode	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...	baonight6...	https://s...				1			NOKIA
☐	152473215...	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...	baonight6...	https://s...				1			NOKIA
☐	11188	111	GB,German	2017/11/0...	111		America	abofbaos1...	https://s...				1			
☐	152473212...	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...	baonight6...	https://s...				1			NOKIA
☐	2131		Rua	2018/04/2...	1	123	11188	123	123	13	13	31	0			
☐	152473263...	sumsong9	China,all	2018/01/0...	19.2	newPage	Africa,A...	baonight6...	https://s...				1			NOKIA

自定义添加数据

e.g. 此处复制新增前三行数据。

JS 示例

```
// 自定义复制新增 3 行数据
$(".insertRows").on("click", function () {
    var list = [0, 1, 2],
        time = new Date().getTime();
    resultGrid.addRows(resultGrid.getRows(list)); // 复制前 3 行数据
    for (var i = 0; i < list.length; i++) {
        resultGrid.setCellValue(i, "product_id", time + i); // 改变主键值
    }
    // sync()同步 Grid 数据更新,会向 Service 发起请求,保存 Grid 中所有修改;
    // 操作结束后提示信息自动显示
    resultGrid.dataSource.sync(); // 提交新增;
});
```

解析:

- **resultGrid.getRows(list):** 根据行号或行号数组（[0,1,2]），获取对应的行数据（前三行数据）
resultGrid.addRows(resultGrid.getRows(list)): Grid 的当前页新添加获取的行数据（添加前 3 行数据）
- **resultGrid.setCellValue(i, "product_id", time + i):** 改变前三行，列英文名为“product_id”的值
 提交「新增」这一操作

页面效果如下所示:



EiCommunicator 添加行

e.g. 此处 EiCommunicator 复制新增前 3 行数据。

JS 示例

```
// EiCommunicator 方式复制新增 3 行数据
$(".insertRows2").on("click", function () {
    var ei = new EiInfo(),
        block = new EiBlock("result");
    ei.addBlock(block);
    block.setBlockMeta(__eiInfo.getBlock("result").getBlockMeta());
```

```

block.setRows(_eiInfo.getBlock("result").getRows().slice(0, 3)); // 取_eiInfo 前 3 行数据
// 修改数据主键值
var time = new Date().getTime();
for (var i = 0; i < block.getRows().length; i++) {
    ei.getBlock("result").setCell(i, "product_id", (time + i).toString());
}

EiCommunicator.send("EEDM6000", "insert", ei, {
    onSuccess: function (eiInfo) {
        // 采用 EiCommunicator 异步请求方式,操作结束后需手动触发提示信息
        NotificationUtil(eiInfo, 'success');
    }, onFailure: function (data) {
        // 发生异常
        console.log(data);
        NotificationUtil(data, 'error');
    }
});
});

```

解析:

- 取_eiInfo 的前 3 行数据,修改英文名为"product_id"的值,并采用 EiCommunicator 异步请求方式(调用 Java 后台的 ServiceEEDM6000 的 insert 方法),操作结束后触发提示成功/失败信息。

页面效果如下所示:



3.5.3.1.38 EEDM6533-根据条件动态获取下拉框的值

3.5.3.1.38.1 页面功能简介

此页面用于说明 EFGrid 里的下拉框需要根据行中的某项值动态展示下拉选项的问题。

3.5.3.1.38.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.3.1.38.3 页面开发步骤

3.5.3.1.38.3.1 JSP 页面开发

重点关注两列，`company_name` 和 `itemA`，其中 `company_name` 是我们的动态条件，`itemA` 需要根据这个条件动态展示下拉选项。

JSP 示例

```
<EF:EFRegion id="result" title="记录集">
    .....

    <EF:EFColumn ename="company_name" locked="true" cname="公司名称"
required="true"/>
    .....
    <EF:EFColumn ename="itemA" cname="List 选项"/>
    .....
</EF:EFGrid>
</EF:EFRegion>
```

3.5.3.1.38.3.2 JS 开发

在 js 里面，通过 `itemA` 的 `editor` 事件，动态获取到 `company_name` 这一列的值，将这个值当做查询条件由 `Ajax` 传到后台查询并返回一个 `EiInfo`，将我们需要的数据在下拉框里渲染。

JS

```
$(function () {
    var globalData = [];

    IPLATUI.EFGrid = {
        "result": {
            columns: [{
                field: "itemA",
                template: function (dataItem) {
                    if (globalData.length) {
                        var newArr = globalData.find(function (item, index) {
                            return item['product_id'] === dataItem['itemA']
                        });
                        return newArr['product_id'];
                    }
                    return "";
                }
            }],
        },
    },
```

```

editor: function (container, options) {
    var grid = container.closest(".k-grid").data("kendoGrid");
    var input = $('<input />');
    var dataSource;

    input.attr("name", options.field);
    input.attr("id", options.field);
    input.appendTo(container);

    var eiInfo = new EiInfo();
    eiInfo.set("inqu_status-0-company_name", options.model["company_name"]);

    EiCommunicator.send("EEDM6000", "query", eiInfo, {
        onSuccess: function (ei) {
            dataSource = ei.getBlock("result").getMappedRows();
            globalData = dataSource;
        }, onFail: function (ei) {
        }
    }, {async: false});

    input.kendoDropDownList({
        valuePrimitive: true,
        dataTextField: "company_name",
        dataValueField: "product_id",
        dataSource: dataSource,
        template: "#=product_id#-#=company_name#"
    });
}
}
}
};
});

```

我们定义一个 EiInfo，通过 options.model["company_name"] 取到条件值，放到“inqu_status”里面。

```

var eiInfo = new EiInfo();
eiInfo.set("inqu_status-0-company_name",
options.model["company_name"]);

```

通过 Ajax，将带有查询条件的 EiInfo 传入对应的 Service，调用 query 方法，取到我们想要的值，回调给前台，这个回调值就保存在“ei”里面。

将回调值 ei 转换成 json 数组，其中 globalData 是一个全局变量。

```
EiCommunicator.send("EEDM6000", "query", eiInfo, {
    onSuccess: function (ei) {
        dataSource = ei.getBlock("result").getMappedRows();
        globalData = dataSource;
    }, onFail: function (ei) {
    }
}, {async: false});
```

将 datasource 配置进 kendoDropDownList，其中 dataTextField 是显示值，dataValueField 是真实值 template 为下拉框里展示的数据结构。

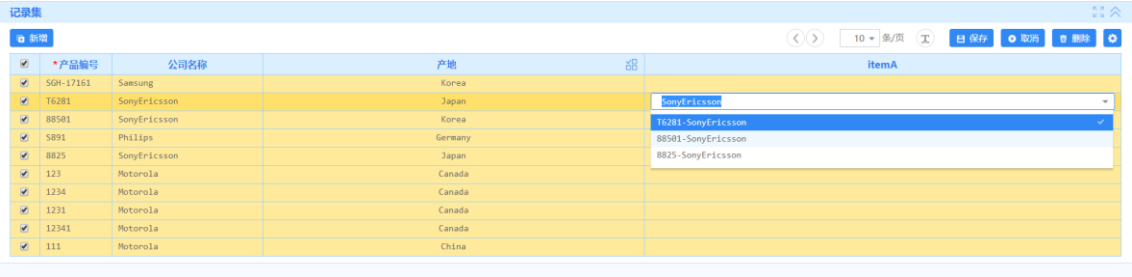
```
input.kendoDropDownList({
    valuePrimitive: true,
    dataTextField: "company_name",
    dataValueField: "product_id",
    dataSource: dataSource,
    template: "#=product_id#-#=company_name#"
});
```

最后配置 columns 下的 template 用来展示回填值，取到我们下拉的点击项 (值为真实值 dataValueField)，循环遍历 dataSource 里的各项，定位到我们所点击的那一项，取出想要展示的值回填到 input 框。这里我们回填了 product_id。

```
template: function (dataItem) {
    if (globalData.length) {
        var newArr = globalData.find(function (item, index) {
            return item['product_id'] === dataItem['itemA']
        });
        return newArr['product_id'];
    }
    return "";
}
```

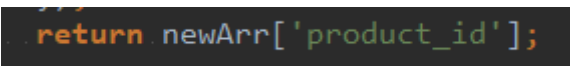
3.5.3.1.38.3.3 页面效果

此时我们点击 88501-SonyEricsson 这项。



产品编号	公司名称	产地
504-17161	Samsung	Korea
16281	SonyEricsson	Japan
88501	SonyEricsson	Korea
5891	Philips	Germany
8825	SonyEricsson	Japan
123	Motorola	Canada
1234	Motorola	Canada
1231	Motorola	Canada
12341	Motorola	Canada
111	Motorola	China

The dropdown menu 'ItemA' is open, showing a list of items with '16281-SonyEricsson' selected.

由  回填代码可知，我们回填了 product_id，即 88501,如图所示。

☒	T6281	SonyEricsson	Japan	88501
-------------------------------------	-------	--------------	-------	-------

当然我们点击不同列，会展示不同的下拉选项，由此，开发人员只需要修改条件项即可动态展示下拉选项内容。

记录集				
☒	*产品编号	公司名称	产地	ItemA
☒	SGH-17161	Samsung	Korea	
☒	T6281	SonyEricsson	Japan	88501
☒	88501	SonyEricsson	Korea	
☒	5891	Philips	Germany	
☒	8825	SonyEricsson	Japan	
☒	123	Motorola	Canada	Motorola
☒	1234	Motorola	Canada	123-Motorola
☒	1231	Motorola	Canada	1234-Motorola
☒	12341	Motorola	Canada	1231-Motorola
☒	111	Motorola	China	12341-Motorola
				111-Motorola
				XXXX-Motorola

3.5.3.1.39 EDFA61-GRID 动态列展示

3.5.3.1.39.1 页面功能简介

本篇文档用于说明在 grid 中如何动态展示列的顺序及修改列的某些属性。
实现此功能需要我们在 grid 中配置 `personal="true"` 属性。

3.5.3.1.39.2 配置和 API 详情

参考 [EFGGrid 表格](#) 中的描述。

3.5.3.1.39.3 页面开发步骤

3.5.3.1.39.3.1 JSP 页面开发

JSP 示例

```
<EF:EFGGrid blockId="edfa00" personal="true" serviceName="EF0001"
queryMethod="queryForEDFA61">
</EF:EFGGrid>
```

当配置了 `personal="true"` 属性之后，grid 里会多一个按钮，如图：

主题预览

1

共 42

10

每页

显示条目 1 - 10 共 417

自定义数据列

导出数据

☐	画面英文名	画面中文名	画面载入的...	画面类型	一级模块英...	二级模块英...	初始处理服...	是否设置分...	是否授权	画面参数	画面技术风格	记录创建表...	自定义数据列 导出数据
☐	EDCC01	测试1	0	ED	CC		0	1				admin	
☐	EDCC02	配置环境	0	ED	CC		0	1				admin	
☐	EDCC03	配置信息	0	ED	CC			1			00	admin	2017032810...
☐	EDCC04	初始化配置页...	0	ED	CC			1				admin	2017070511...
☐	EDC900	代码类别维护	0	ED	CH		0	1				admin	2009071900...
☐	EDC901	代码项维护	0	ED	CH		0	1				admin	2009071900...
☐	ED0810	表索引维护页...	0	ED	DB		0	1				admin	2008111915...
☐	ED0810	表头信息维护...	0	ED	DB		0	1				admin	2008111214...
☐	ED081001	模板输出页面	0	ED	DB		0	1				admin	2008112120...
☐	ED0811	表数据项维护...	0	ED	DB		0	1				admin	2008111809...

点击此按钮，出现自定义数据列按钮，继续点击，会弹出此 grid 的列信息，如图：

自定义列信息

↑

↓

默认配置

自定义配置

⏪

⏩

页 1 共 1

⏪

⏩

20 每页 共 17 条

保存并刷新

取消

⚙

☐	列英文名	列中文名	是否为固定列	是否为隐藏列	列宽	配置状态
☐	form_ename	画面英文名	☒	☐	96	可自定义
☐	form_cname	画面中文名	☐	☐	96	可自定义
☐	form_load_path	画面载入的路径	☐	☐	96	可自定义
☐	form_type	画面类型	☐	☐	96	可自定义
☐	module_ename_1	一级模块英文名	☐	☐	96	可自定义
☐	module_ename_2	二级模块英文名	☐	☐	96	可自定义
☐	init_load_service_ename	初始处理服务英文名	☐	☐	96	可自定义
☐	is_grade	是否设置分级授权	☐	☐	96	可自定义
☐	is_auth	是否授权	☐	☐	96	可自定义
☐	form_param	画面参数	☐	☐	96	可自定义
☐	form_style	画面技术风格	☐	☐	96	可自定义
☐	rec_creator	记录创建责任人	☐	☐	96	可自定义
☐	rec_create_time	记录创建时刻	☐	☐	96	可自定义

我们选中一列，在左上角有两个箭头，通过此箭头可重新部署列的排序。

自定义列信息

↑

↓

默认配置

自定义配置

(K) <

页 1 共 1

> (J)

20

每页

共 17 条

保存并刷新

取消

⚙

☐	列英文名	列中文名	是否为固定列	是否为隐藏列	列宽	配置状态
☐	form_name	画面英文名	☒	☐	96	可自定义
☐	form_name	画面中文名	☐	☐	96	可自定义
☒	form_load_path	画面载入的路径	☐	☐	96	可自定义
☒	form_type	画面类型	☐	☐	96	可自定义
☐	module_name_1	一级模块英文名	☐	☐	96	可自定义
☐	module_name_2	二级模块英文名	☐	☐	96	可自定义
☐	init_load_service_name	初始处理服务英文名	☐	☐	96	可自定义
☐	is_grade	是否设置分级授权	☐	☐	96	可自定义
☐	is_auth	是否授权	☐	☐	96	可自定义
☐	form_param	画面参数	☐	☐	96	可自定义
☐	form_style	画面技术风格	☐	☐	96	可自定义
☐	rec_creator	记录创建责任者	☐	☐	96	可自定义
☐	rec_create_time	记录创建时刻	☐	☐	96	可自定义

需要注意的是，非固定列不能移到固定列上方。

自定义列信息

↑

↓

默认配置

自定义配置

⌕

<

页 1

共 1

>

⌕

20

每页

共 17 条

保存并刷新

取消

⚙

☐	列英文名	列中文名	是否为固定列	是否为隐藏列	列宽	配置状态
☐	form_name	画面英文名	☒	☐	96	可自定义
☒	form_load_path	画面载入的路径	☐	☐	96	可自定义
☒	form_type	画面类型	☐	☐	96	可自定义
☐	form_cname	画面中文名	☐	☐	96	可自定义
☐	module_name_1	一级模块英文名	☐	☐	96	可自定义
☐	module_name_2	二级模块英文名	☐	☐	96	可自定义
☐	init_load_service_name	初始处理服务英文名	☐	☐	96	可自定义
☐	is_grade	是否设置分级授权	☐	☐	96	可自定义
☐	is_auth	是否授权	☐	☐	96	可自定义
☐	form_param	画面参数	☐	☐	96	可自定义
☐	form_style	画面技术风格	☐	☐	96	可自定义
☐	rec_creator	记录创建责任人	☐	☐	96	可自定义
☐	rec_create_time	记录创建时刻	☐	☐	96	可自定义

我们也可以设置列的其他信息，如是否为固定列，是否为隐藏列，列宽等，设置完点击保存并刷新按钮，则 `grid` 的列会重新部署。

重新排列后的 grid:

	画面英文名	画面载入的...	画面类型	画面中文名	一级模块英...	二级模块英...	初始处理服...	是否设置分...	是否授权	画面参数	画面技术风格	记录创建表...	记录创建时刻	
☐	EDCC01		0	测试1	ED	CC		0	1			admin	2017101914...	a
☐	EDCC02		0	配置环境	ED	CC		0	1			admin	2017101210...	a
☐	EDCC03		0	配置信息	ED	CC			1	00		admin	2017032810...	
☐	EDCC04		0	初始化配置页...	ED	CC			1			admin	2017070511...	
☐	EDCM00		0	代码类别维护	ED	CM		0	1			admin	2009071900...	
☐	EDCM01		0	代码项维护	ED	CM		0	1			admin	2009071900...	
☐	EDDBT0		0	表索引维护页...	ED	DB		0	1			admin	2008111915...	
☐	EDDBT0		0	表头信息维护...	ED	DB		0	1			admin	2008111214...	
☐	EDDBT001		0	模板弹出页面	ED	DB		0	1			admin	2008112120...	
☐	EDDBT1		0	表数据项维护...	ED	DB		0	1			admin	2008111809...	

3.5.3.1.40 EEDM6526-Grid 下拉列动态属性配置

3.5.3.1.40.1 页面功能简介

本示例页面用于说明 如何对 **grid** 中的下拉列进行动态属性配置。

3.5.3.1.40.2 配置和 API 详情

参考 [EFComboColumn](#) 中的动态属性的描述。

3.5.3.1.40.3 页面开发步骤

3.5.3.1.40.3.1 JSP 页面开发

EEDM6526.jsp

```
<EF:EFPage title="Grid 下拉列动态属性配置">
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" autoDraw="no" needAuth="false">
      <EF:EFColumn ename="product_id" primaryKey="true" readonly="true" cname="产
      品编号"/>
      <EF:EFColumn ename="company_name" cname="公司名称[默认值]" defaultValue="
      宝信软件"/>
      <!-- Grid 中下拉列动态属性配置说明:
        autoWidth: 下拉项宽度是否自适应("true", "false");
        optionLabel: 作为引导信息, 指定默认空值下拉项;
        filter: 下拉过滤配置("contains", "startswith", "endswith");
        required: 必填设置("true", "false")
      --%>
      <EF:EFComboColumn ename="made_country" cname="产地[下拉列]"
        columnTemplate="#=textField#-#=valueField#"
        itemTemplate="#=textField#-#=valueField#"
        textField="textField" valueField="valueField"
        autoWidth="true" optionLabel="请选择" filter="contains"
        required="true">
        <EF:EFOption label="产地 1" value="1"/>
        <EF:EFOption label="产地 2" value="2"/>
        <EF:EFOption label="产地 3" value="3"/>
      </EF:EFComboColumn>
      <EF:EFColumn ename="made_country" hidden="true" cname="产地"/>
    </EF:EFGrid>
  </EF:EFRegion>
</EF:EFPage>
```

```
<EF:EFColumn ename="price" format="{0:c}" cname="价格[只读]"
defaultValue="100" readonly="true"/>
<EF:EFColumn ename="sale_date" cname="日期" editType="date"/>
</EF:EFGGrid>
</EF:EFRegion>
</EF:EFPage>
```

3.5.3.1.40.3.2 页面效果如下图所示



搜索框内输入“产地 3”：



3.5.3.1.41 EEDM6041-EFGGrid 列头悬浮固定

3.5.3.1.41.1 页面功能简介

本示例页面用于说明，如何在 EFGGrid 无滚动条，但因 Grid 内数据过多（每页 100 条）导致页面出现滚动条时，来使列头固定。

3.5.3.1.41.2 配置和 API 详情

参考 EFGGrid 表格中的描述。

3.5.3.1.41.3 页面开发步骤

3.5.3.1.41.3.1 JSP 页面开发

EEDM6041.jsp

```
<EF:EFPage>
  <%--设置 isFloat="true", 数据过多时, 列头悬浮。--%>
  <EF:EFRegion id="result" title="记录集">
    <EF:EFGrid blockId="result" isFloat="true">
      </EF:EFGrid>
    </EF:EFRegion>
  </EF:EFPage>
```

3.5.3.1.41.3.2 页面效果展示

EFGrid 列头悬浮示例 > EEDM6041

记录集

	* 产品代号	公司名称	产地	上市日期	价格	公司首页	Google查询	验证字段0	验证字段1	验证字段2	验证字段3	验证字段4	验证字段5
☐	mix2222111	999944423		2018-07-19	0	newPage							0
☐	A4	3232	China	2018/07/1...	0								0
☐	cscsaa	2323	GB	2018-07-2...	777	http://platform.b...							0
☐	xxxxxxx	3333		2018-07-2...	0								0
☐	mix2	111		2018-07-2...	0								0
☐	mix1111	11111	China, all	2018-07-2...	19.2	http://platform.b...	Africa		baosight6...	https://s...			1
☐	mix2222	3322		2018-07-2...	0								0
☐	153318105...	999944423		2018-07-2...	44								0
☐	153318104...	999944423		2018-07-2...	0								0
☐	153318104...			2018-07-2...	0	China							0
☐	A123	as		2018-05-1...	22								0
☐	2131		Rus	2018/04/2...	1	123	Africa, Am...	123	123	13	13	31	0
☐	152473263...	sumsong9	China, all	2018/01/0...	19.2	newPage	Africa, Am...		baosight6...	https://s...			1
☐	152473263...	111	GB, German	2017/11/0...	111		America		abc@baosi...	https://s...			1
☐	152473266...	sumsong9	China, all	2018/01/0...	19.2	newPage	Africa, Am...		baosight6...	https://s...			1
☐	152473266...	sumsong9	China, all	2018/01/0...	19.2	newPage	Africa, Am...		baosight6...	https://s...			1
☐	152473266...	111	GB, German	2017/11/0...	111		America		abc@baosi...	https://s...			1
☐	152473273...	sumsong9	China, all	2018/01/0...	19.2	newPage	Africa, Am...		baosight6...	https://s...			1
☐	152473273...	sumsong9	China, all	2018/01/0...	19.2	newPage	Africa, Am...		baosight6...	https://s...			1
☐	abcd2e2	sumsong9	China, all	2018/01/0...	19.2	newPage	Africa, Am...		baosight6...	https://s...			1
☐	444rrr		GB	2018/04/1...	445	France	test1x						0
☐	152473273...	111	GB, German	2017/11/0...	111		America		abc@baosi...	https://s...			1
☐	T6281	SonyErica...	Japan	2007-04-06	100.01	http://www.sonye...	查询	138000000...	abc@baosi...	label	021-50801...	1	2000
☐	152473331...	sumsong9	China, all	2018/01/0...	19.2	newPage	Africa, Am...		baosight6...	https://s...			1
☐	152472248...	sumsong9	China, all	2018/01/0...	19.2	newPage	Africa, Am...		baosight6...	https://s...			1
☐	152472248...	111	GB, German	2017/11/0...	111		America		abc@baosi...	https://s...			1
☐	S891	Philips	Germany	2007/03/08	98.2	http://www.philip...	search	138000000...	aaa@123.c...	label	021-50801...	1	2000

3.5.4 树和菜单

3.5.4.1 EFTree 示例

3.5.4.1.1 EEDM7000-Tree 基础示例

3.5.4.1.1.1 页面功能简介

本示例页面用于说明，如何使用 EFTree 和 EFOnceTree 组件进行树的基本展示。

3.5.4.1.1.2 配置和 API 详情

参考 [EFTree](#) 和 [EFOnceTree](#) 中的描述。

3.5.4.1.1.3 页面开发步骤

3.5.4.1.1.3.1 JSP 页面开发

EFTree 和 EFOnceTree 用于展示数据，通常将其放置在 EFRegion 区域内，示例代码如下。

EEDM7000.jsp

```
<EF:EFPage>
  <EF:EFRegion id="R1" title="分层动态查询加载树(JS 配置根结点)">
    <EF:EFTree id="tree01" textField="text" valueField="label" hasChildren="leaf"
      dataSpriteCssClassField="icon"
      serviceName="EEDM05" methodName="query">
    </EF:EFTree>
  </EF:EFRegion>

  <EF:EFRegion id="R2" title="一次加载树(JS 配置首结点)" >
    <EF:EFOnceTree id="tree02" textField="text" valueField="label" pid="parent"
      dataSpriteCssClassField="icon"
      hasChildren="hasChildren" serviceName="EEDM7000"
methodName="onceTree">
    </EF:EFOnceTree>
  </EF:EFRegion>

  <EF:EFRegion id="R3" title="分层动态查询加载树(Tag 配置根结点)">
    <EF:EFTree id="tree03" textField="text" valueField="label" hasChildren="leaf"
      dataSpriteCssClassField="icon" root="{label: 'root', text: 'root', icon: 'folder'}"
      serviceName="EEDM05" methodName="query">
    </EF:EFTree>
  </EF:EFRegion>

  <EF:EFRegion id="R4" title="一次加载树(Tag 配置首结点)" >
    <EF:EFOnceTree id="tree04" textField="text" valueField="label" pid="parent"
      dataSpriteCssClassField="icon" root="{label: '0', text: 'root', icon: 'folder'}"
      hasChildren="hasChildren" serviceName="EEDM7000"
methodName="onceTree">
    </EF:EFOnceTree>
  </EF:EFRegion>
</EF:EFPage>
```

3.5.4.1.1.3.2 Service 服务开发

EFTree 中我们调用的是 ServiceEEDM05，代码如下。

ServiceEEDM05.java

```
public class ServiceEEDM05 extends ServiceEPBase {
    public EiInfo initLoad(EiInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM05().eiMetadata);
        return inInfo;
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        EiInfo outInfo = super.query(inInfo, "EEDM05.query", new EEDM05());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM05.delete");
    }

    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM05.update");
    }

    public EiInfo insert(EiInfo inInfo) {
        return insert(inInfo, "EEDM05.insert");
    }
}
```

EFOnceTree 中我们调用的是 ServiceEEDM7000。

ServiceEEDM7000.java

```
public class ServiceEEDM7000 extends ServiceEPBase {

    public EiInfo onceTree(EiInfo inInfo){
        EiInfo outInfo = new EiInfo();
        // EiConstant.queryBlock => inqu_status,表示通用查询 blockId 名称
        // "node" => 表示一次加载结点名称，在前端的 ROOT 中定义其值
        String blockId = inInfo.getString(EiConstant.queryBlock + EiConstant.separator+"0-
node");
```

```

// 返回的的 blockId 的名称是传入的结点其值
// 如: inqu_status-0-node => "$", 则返回的的 block 名称是"$"
outInfo.addBlock(MockDataUtil.getCascade(blockId));
return outInfo;
}
}

```

这里我们在开发EFOnceTree后端服务中需要注意

1. 在后端服务中，**我们需要获取的block是inqu_status，返回的block名称是我们前端传入的树结点Id。**
2. 约定在后端获取前端传入的树结点三段式格式是：**inqu_status-0-node**
3. 开发人员把多行数据存储在block中，每一行里面通常需要包含带有value Field, pid关系的属性。

3.5.4.1.1.3.3 JS 开发

EEDM7000.js

```

IPLATUI.EFTree = {
  "tree01": {
    // ROOT: "root", // 虚拟结点不渲染，仅作为初始查询条件

    ROOT: {label: "root", text: "地区分布", leaf: true, icon: "folder"},
    query: function (postEiInfo, model) {
      if (model) {
        console.log(model.text);
      }
    }
  },

  "tree02": {
    ROOT: {label: "0", text: "联级菜单", leaf: true},
    /**
     *
     * @param e
     */
    onSuccess: function (e) {
      console.log(e);
    },
  },
}

```

```

/**
 *
 * @param e
 */
onFail: function (e) {
    console.log(e);
}
};

```

- 1.EFTree 是逐级加载数据，EFOnceTree 是一次数据加载形成树
- 2.EFTree.tag中ROOT配置可以参考 [UI-常见开发问题FAQ](#) 中Q5

kendo 中树的默认数据格式。

```

var schema = {
    schema: {
        model: {
            id: 'node_id', // 结点的 Id 字段名
            children: 'movies', // 结点的 Id 字段名 默认是"items"
            hasChildren: 'node_type'// 结点类型 判断是否还有子结点
        }
    }
};

var data = {
    "node_id": "1", "node_text": "根", "node_type": 2,
    "movies": [
        {"node_id": "2", "node_text": "叶子 1", "node_type": 2},
        {"node_id": "3", "node_text": "中间结点", "node_type": 2,
        "movies": [
            {"node_id": "4", "node_text": "叶子", "node_type": 1},
            {"node_id": "5", "node_text": "叶子", "node_type": 1}
        ]
    }
]
};

```

平台还提供了将 list 格式数据转化为 map 格式数据的方法
IPLAT.flat2Map(listData, idField, parentField, rootId)。

其中 listData 指 list 数据集合，idField 指 id 的属性名(会作为其它数据的父 Id)，parentField 指作为父结点 Id 的属性名，rootId 指首结点名；

IPLAT.flat2Map()

```

var listData = [
    {parent_id: "root", node_text: "a", node_id: "a"},
    {parent_id: "a", node_text: "a1", node_id: "a1"},
    {parent_id: "a1", node_text: "a11", node_id: "a11"},
    {parent_id: "a", node_text: "a2", node_id: "a2"},
    {parent_id: "a2", node_text: "a21", node_id: "a21"},
    {parent_id: "a2", node_text: "a22", node_id: "a22"},
    {parent_id: "a2", node_text: "a23", node_id: "a23"}
];
var mapData = IPLAT.flat2Map(listData, "node_id", "parent_id", "root");

// mapData:
//  {
//    parent_id: "root", node_text: "a", node_id: "a",
//    items: [
//      {
//        parent_id: "a", node_text: "a1", node_id: "a1",
//        items: [
//          {parent_id: "a1", node_text: "a11", node_id: "a11"}
//        ]
//      },
//      {
//        parent_id: "a", node_text: "a2", node_id: "a2",
//        items: [
//          {parent_id: "a2", node_text: "a21", node_id: "a21"},
//          {parent_id: "a2", node_text: "a22", node_id: "a22"},
//          {parent_id: "a2", node_text: "a23", node_id: "a23"}
//        ]
//      }
//    ]
//  }

```

相应的还提供了将 **map** 数据转成 **list** 的方法 **IPLAT.map2Flat(mapData, itemName)**。

其中 **mapData** 指 **map** 数据集合，**itemName** 指子集属性名称(默认为 "items")

IPLAT.map2Flat()

```

var mapData = {
    parent_id: "root", node_text: "a", node_id: "a",
    items: [

```

```

    {
      parent_id: "a", node_text: "a1", node_id: "a1",
      items: [
        {parent_id: "a1", node_text: "a11", node_id: "a11"}
      ]
    },
    {
      parent_id: "a", node_text: "a2", node_id: "a2",
      items: [
        {parent_id: "a2", node_text: "a21", node_id: "a21"},
        {parent_id: "a2", node_text: "a22", node_id: "a22"},
        {parent_id: "a2", node_text: "a23", node_id: "a23"}
      ]
    }
  ]
};
var listData = IPLAT.map2Flat(mapData, "items");

// listData:
// [
//   {parent_id: "root", node_text: "a", node_id: "a"},
//   {parent_id: "a", node_text: "a1", node_id: "a1"},
//   {parent_id: "a1", node_text: "a11", node_id: "a11"},
//   {parent_id: "a", node_text: "a2", node_id: "a2"},
//   {parent_id: "a2", node_text: "a21", node_id: "a21"},
//   {parent_id: "a2", node_text: "a22", node_id: "a22"},
//   {parent_id: "a2", node_text: "a23", node_id: "a23"}
// ]

```

3.5.4.1.1.3.4 页面展示效果



3.5.4.1.2 EEDM7001-Tree 与 Grid 联动基础示例

3.5.4.1.2.1 页面功能简介

本示例页面用于说明，如何进行树表联动，点击 **EFTree** 中某个结点，更新 **Grid** 中表格数据。

3.5.4.1.2.2 配置和 API 详情

参考 [EFGGrid](#) 与 [EFTree](#) 中的描述。

3.5.4.1.2.3 页面开发步骤

3.5.4.1.2.3.1 JSP 页面开发

EEDM7001.jsp

```
<EF:EFPage>
  <!--使用 splitter 布局，需要为 left-pane/right-pane 设置高度。--%>
  <!--当 EFRegion 的 fitHeight="true" 时，则须为 splitter/left-pane/right-pane 加上
class="i-fit-height"。--%>
  <div id="splitter" class="i-fit-height">
    <div id="left-pane" class="i-fit-height">
      <EF:EFRegion id="tree" title="系统菜单树" fitHeight="true">
        <EF:EFTree id="tree01" valueField="node_id" textField="node_text"
hasChildren="node_type"
```

```

        serviceName="EEDM06" methodName="query">
    </EF:EFTree>
    <ul id="tree01menu" style="width:85px; display: none">
        <li data-type="edit"><span style="font-size: 16px; width: 16px; vertical-align:
middle; margin: -7px 2px -5px;" class="fa fa-pencil"></span>编辑</li>
        <li data-type="delete"><span style="font-size: 16px; width: 16px; vertical-
align: middle; margin: -7px 2px -5px;" class="fa fa-trash"></span>删除</li>
    </ul>
    </EF:EFRegion>
</div>

<div id="right-pane" class="i-fit-height">
    <EF:EFRegion id="inqu" title="查询区">
        <div class="row">
            <EF:EFInput ename="inqu_status-0-node" cname="目录代码" colWidth="6"/>
        </div>
    </EF:EFRegion>
    <EF:EFRegion id="result" title="数据集" fitHeight="true">
        <EF:EFGrid blockId="result" autoDraw="no" serviceName="EEDM7001"
needAuth="false">
            <EF:EFColumn ename="node_id" cname="英文名称"/>
            <EF:EFColumn ename="node_text" cname="中文名称"/>
            <EF:EFColumn ename="parent_id" cname="父结点名称"/>
            <EF:EFComboColumn ename="node_type" cname="结点类型"
valueField="valueField" textField="textField">
                <EF:EFOption label="有子结点" value="2">
                </EF:EFOption>
                <EF:EFOption label="无子结点" value="1">
                </EF:EFOption>
            </EF:EFComboColumn>
        </EF:EFGrid>
    </EF:EFRegion>
</div>
</EF:EFPage>

```

- 1.这个页面是左右结构，我们通过bootstrap的栅格对EFTree和EFGrid进行定位，树和表所在EFRegion区配置fitHeight="true"使Region高度适应到屏幕底部
- 2.EFTree中所点击结点的node_id值作为查询区EFInput中的Value值。

3.5.4.1.2.3.2 JS 开发

EEDM7001.js

```

/**
 * EFTree 和 EFGGrid 的联动示例
 *
 * 1. 掌握"左树右表"的布局
 * 2. 掌握 EFRegion fitHeight 属性的用法
 * 3. 掌握 EFTree select 事件
 * 4. 掌握 dataItem API
 */

// EEDM7001
$(function () {
    IPLATUI.EFTree = {
        "tree01": {
            root: "root:地区分布",

            /**
             * 选中树结点时触发的事件，注意点击树 折叠/展开 的图标不会触发此事件
             *
             * @param e
             * e.sender kendoTree 对象, tree01Tree (EFTree 的 id + "Tree")
             * e.node 点击的节点 DOM 元素
             */
            select: function (e) {
                // e.sender === this 都是 kendoTree 对象

                // dataItem 方法，传入节点 DOM 对象，获取结点上的数据对象 (kendo.data.Node)
                // 结点的数据对象包括 valueField, textField 等后台传过来的字段值
                var nodeData = this.dataItem(e.node);

                // 更新查询条件到 EFInput 输入框 nodeData[valueField]
                $("#inqu_status-0-node").val(nodeData.node_id);

                // 重新查询 EFGGrid
                resultGrid.dataSource.page(1);
            }
        }
    };

    $("#F1").on("click", function () {
        resultGrid.dataSource.page(1);
    });
});

```

```
});
```

- 1.kendo原生方法监听树结点的select事件，联动渲染出Grid数据;
- 2.ROOT定义树的根结点

3.5.4.1.2.3.3 Service 服务开发

- 1) EFTree 的后端服务调用 EEDM06，这个服务平台已经提供。
- 2) EFGrid 是调用 EEDM7001 中的基本的增，删，改，查服务。

ServiceEEDM06.java & ServiceEEDM7001.java

```
// ServiceEEDM06
public class ServiceEEDM06 extends ServiceEPBase {
    public EiInfo initLoad(EiInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM06().eiMetadata);
        return inInfo;
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    public EiInfo search(EiInfo inInfo) {
        inInfo.set(EiConstant.resultBlock + EiConstant.separator + EiConstant.limitStr, 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.search", new EEDM06());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.delete");
    }
}
```

```

    }

    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM06.update");
    }

    public EiInfo insert(EiInfo inInfo) {
        return insert(inInfo, "EEDM06.insert");
    }
}

// ServiceEEDM7001, 提供基本的增, 删, 改, 查服务
public class ServiceEEDM7001 extends ServiceEPBase {

    public EiInfo initLoad(EiInfo inInfo) {
        EiBlock block = new EiBlock(EiConstant.resultBlock);
        block.addBlockMeta(new EEDM06().getMetaData());
        EiInfo outInfo = new EiInfo();
        outInfo.addBlock(block);
        outInfo.setMsg("欢迎来到树表联动示例");
        return outInfo;
    }

    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        return outInfo;
    }

    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.delete");
    }

    public EiInfo deleteBatch(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.deleteBatch");
    }

    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM06.update");
    }
}

```

```
public EilInfo insert(EilInfo inInfo) {  
  
    return insert(inInfo, "EEDM06.insert");  
  
}  
  
}
```

3.5.4.1.2.3.4 页面展示效果



3.5.4.1.3 EEDM7002-Tree 结点操作示例

3.5.4.1.3.1 页面功能简介

本示例页面用于说明：

- 1. 如何实现树结点可拖拽更新功能；
- 2. 如何实现树结点右击增删改操作功能；
- 3. Cookies 保持树结点展开状态；

3.5.4.1.3.2 配置和 API 详情

参考 [EFTree](#) 和 [EFGGrid 表格](#) 中的描述。

3.5.4.1.3.3 页面开发步骤

3.5.4.1.3.3.1 JSP 页面开发

EEDM7002.jsp

```
<EF:EFPage>  
    <div class="row">  
        <div class="col-md-3">
```

```

<EF:EFRegion id="tree" title="系统菜单树（可右键编辑）" fitHeight="true">
  <EF:EFTree id="menu" valueField="node_id" textField="node_text"
hasChildren="node_type"
    dataSpriteCssClassField="icon"
    serviceName="EEDM06" methodName="query" style="height:400px;" >
</EF:EFTree>
<ul id="handleMenu" style="display: none;">
  <li data-type="add"><span class="fa fa-plus"></span>添加</li>
  <li data-type="edit"><span class="fa fa-pencil"></span>编辑</li>
  <li data-type="delete"><span class="fa fa-trash"></span>删除</li>
</ul>
<EF:EFWindow id="addWnd" width="40%" height="40%" title="在所选结点下添加
子结点">
  <EF:EFRegion id="add_node_region" title="添加结点" style="margin-bottom:
0;">
    <!-- edit form -->
    <div id="addNodeDiv">
      <div class="row">
        <EF:EFInput ename="add-text" cname="中文名" colWidth="10"
required="true"/>
      </div>
      <div class="row">
        <EF:EFInput ename="add-label" cname="英文名" colWidth="10"
required="true"/>
      </div>
    </div>
    <div class="k-window-save k-popup-save">
      <EF:EFButton ename="addNode" cname="确定" layout="1" class="i-btn-
wide"/>
    </div>
  </EF:EFRegion>
</EF:EFWindow>

  <EF:EFWindow id="editWnd" width="40%" height="40%" title="编辑所选结点">
    <EF:EFRegion id="edit_node_region" title="编辑结点" style="margin-bottom:
0;">
      <!-- edit form -->
      <div id="editNodeDiv">
        <div class="row">
          <EF:EFInput ename="edit-text" cname="中文名" colWidth="10"
required="true"/>
        </div>
        <div class="row">

```

```

        <EF:EInput ename="edit-label" cname="英文名" colWidth="10"
required="true" class="i-input-readonly i-state-readonly"/>
    </div>
</div>
<div class="k-window-save k-popup-save">
    <EF:EButton ename="editNode" cname="确定" layout="1" class="i-btn-
wide"/>
</div>
</EF:EFRegion>
</EF:EFWindow>

</EF:EFRegion>
</div>

<div class="col-md-9">
    <EF:EFRegion id="inqu" title="查询区">
        <div class="row">
            <EF:EInput ename="inqu_status-0-node" cname="目录代码"/>
        </div>
    </EF:EFRegion>
    <EF:EFRegion id="result" title="数据集" fitHeight="true">
        <EF:EFGrid blockId="result" autoDraw="no" serviceName="EEDM7001">
            <EF:EFColumn ename="node_id" cname="英文名称" readonly="true"/>
            <EF:EFColumn ename="node_text" cname="中文名称"/>
            <EF:EFColumn ename="parent_id" cname="父结点名称"/>
            <EF:EFComboColumn ename="node_type" cname="类型"
valueField="valueField" textField="textField">
                <EF:EFOption label="有子结点" value="2">
                </EF:EFOption>
                <EF:EFOption label="无子结点" value="1">
                </EF:EFOption>
            </EF:EFComboColumn>
            <EF:EFColumn ename="icon" cname="图标"/>
        </EF:EFGrid>
    </EF:EFRegion>
</div>
</div>
</EF:EFPage>

```

1.EFTree 的基本使用请参考 [EFTree](#)

2.出现树结点过多超出页面高度的情况，可以在EFTree标签外层EFRegion上配置fitHeight="true"出现滚动条；同时还可以通过动态属性设置树高度；

3.5.4.1.3.3.2 JS 开发

EEDM7002.js

```

$(function () {
    // 定义结点的各属性名
    var NODE = {
        nodeIdField: "node_id",
        nodeTextField: "node_text",
        parentIdField: "parent_id",
        nodeTypeField: "node_type"
    };

    IPLATUI.EFTree = {
        "menu": {
            ROOT: {node_id: "root", node_text: "地区分布", node_type: 2, icon: "folder"},

            /**
             * 树结点内容点击触发回调函数(点击结点内容前展开收起符号不会触发)
             * @param e
             */
            select: function (e) {
                var _data = this.dataItem(e.node);
                $("#inqu_status-0-node").val(_data[NODE.nodeIdField]);
                resultGrid.dataSource.page(1);
            },

            /**
             * 开启结点拖拽功能,默认 dragAndDrop = false 不开启,
             * 若未开启拖拽,不会触发 dragstart 和 dragend 事件
             */
            dragAndDrop: true,

            /**
             * 结点拖拽结束后的回调函数
             * @param e
             *   e.sourceNode: 被拖拽的结点
             *   e.destinationNode: 释放被拖拽结点的结点(新的父结点)
             *   e.dropPosition: 释放被拖拽结点的位置 over, before, or after
             */
            dragend: function (e) {
                var tree = this;
                var dropPosition = e.dropPosition;
            }
        }
    }

```

```

var ei = new EiInfo(), key, json;
var model = this.dataItem(e.sourceNode);

// 同步后端数据库
// 提交一次请求, 修改父亲结点信息
/* 根据 drop 类别处理 */
if (dropPosition === "over") {
    var parentModel = this.dataItem(e.destinationNode);
    model.set(NODE.parentIdField, parentModel[NODE.nodeIdField]);
    json = model.toJSON();
    json[NODE.nodeTypeField] = +(!json[NODE.nodeTypeField]);
    for (key in json) {
        ei.set("result", "0", key, json[key] || "");
    }
    if (parentModel[NODE.nodeIdField] !== "root") {
        parentModel.set(NODE.nodeTypeField, 2);
        json = parentModel.toJSON();
        for (key in json) {
            ei.set("result", "1", key, json[key] || "");
        }
    }
} else {
    var brotherModel = this.dataItem(e.destinationNode);
    model.set(NODE.parentIdField, brotherModel[NODE.parentIdField]);
    json = model.toJSON();
    json[NODE.nodeTypeField] = +(!json[NODE.nodeTypeField]);
    for (key in json) {
        ei.set("result", "0", key, json[key] || "");
    }
}

EiCommunicator.send("EEDM7001", "update", ei, {
    onSuccess: function () {
        NotificationUtil("【" + model[NODE.nodeTypeField] + "】" + "结点调整成功");
        tree.reload(model[NODE.parentIdField]); // 重新加载父结点
    }, onFail: function (ei) {
        // 调用异常
        NotificationUtil("调用失败, 原因[" + ei + "], "error");
    }
});
},

/**

```



```

* 树加载完成后的回调函数
* @param options: 树的配置项
*/
loadComplete: function (options) {
    // 保持结点展开状态
    var expanded = Cookies.get('expanded');
    if (expanded) {
        Cookies.remove('expanded');
        expanded = JSON.parse(expanded);
        $("#menu").data("kendoTreeView").expandPath(_keys(expanded));
    }

    // 树结点右击增删改
    var _tree = this;
    $("#handleMenu").kendoContextMenu({
        target: "#" + options.treeld,
        filter: ".k-in",
        select: function (e) {
            var node = e.target;

            // 当前操作结点的 Model,包含该结点的一切属性信息
            var model = _tree.dataItem(node);
            var model2Ei = model2EiInfo(model);

            // 当前操作结点的父结点拥有的子结点个数
            var pNodeChildrenCount = _tree.parent(e.target).find('.k-item').length;

            /**
             * 根据右击菜单选项类型进行结点操作
             * add: 新增, delete: 删除, edit: 编辑
             */
            var selectedType = $(e.item).data("type");
            if (selectedType === "add") {
                addWndWindow.open().center();
                $("#addNode").unbind('click').click(function () {
                    var validator = IPLAT.Validator({
                        id: "addWnd"
                    });
                    if (validator.validate()) {
                        var eiInfo = new EiInfo();
                        var pId = model[NODE.parentIdField];

                        eiInfo.set("result-0-node_id", $("#add-label").val());
                    }
                });
            }
        }
    });
}

```

```

        eiInfo.set("result-0-node_text", $("#add-text").val());
        eiInfo.set("result-0-parent_id", model[NODE.nodeIdField]);

        EiCommunicator.send("EEDM7001", "insert", eiInfo, {
            onSuccess: function () {
                // 若操作结点为叶子结点, 更新其 node_type 值
                if (model2Ei.get("result-0-node_type") === "1") {
                    model2Ei.set("result-0-node_type", "2");
                    EiCommunicator.send("EEDM7001", "update", model2Ei, {
                        onSuccess: function () {
                            _tree.reload(model[NODE.parentIdField]);
                        },
                        onFail: function (ei) {
                            NotificationUtil("调用失败, 原因[" + ei + "]", "error");
                        }
                    });
                }
                NotificationUtil("【" + $("#add-text").val() + "】" + "结点添加成功");
            },
            onFail: function (ei) {
                NotificationUtil("调用失败, 原因[" + ei + "]", "error");
            }
        });
    }
});
}

if (selectedType === "delete") {
    var deleteHTML = kendo.template($("#del-template").html());
    {message: "确认删除此结点和它所有的子结点?", ok: '确定', cancel: '取消'}
});

// 弹出警告框, 确定是否删除结点
WindowUtil({
    title: "警告",
    content: deleteHTML,
    ok: function () {
        EiCommunicator.send("EEDM7001", "query", new EiInfo(), {
            onSuccess: function (eiInfo) {

```

```

        // 获取所有被删结点(包括子结点)的 node_id 集合
        var data = eiInfo.getBlock("result").getMappedRows();
        var idDeletedList = getChildrenIds(data, NODE.nodeIdField,
NODE.parentIdField, model[NODE.nodeIdField], "items");

        // 批量删除所有结点
        var info = new EiInfo();
        info.set("result-0-labelList", idDeletedList);
        EiCommunicator.send("EEDM7001", "deleteBatch", info, {
            onSuccess: function () {

                // 若是唯一子结点,更新父结点 node_type 值
                if (pNodeChildrenCount === 1) {
                    var pEi = new EiInfo();
                    pEi.set("result-0-node_id", model[NODE.parentIdField]);
                    pEi.set("result-0-node_type", "1");

                    EiCommunicator.send("EEDM7001", "update", pEi);
                }

                NotificationUtil("【" + model[NODE.nodeTextField] + "】"
+ "结点删除成功");

                _tree.reload(model[NODE.parentIdField]); // 删除成功, 重新
加载父结点

            }, onFail: function (ei) {
                NotificationUtil("调用失败, 原因[" + ei + "]", "error");
            }
        }, {async: false});
        }, onFail: function (eiInfo) {
            NotificationUtil("调用失败, 原因[" + eiInfo + "]", "error");
        }
    });
    this.data("kendoWindow").close();
},
cancel: function () {
    this.data("kendoWindow").close(); // 关闭警告框
}
});
}

if (selectedType === "edit") {
    editWndWindow.open().center();
    $("#edit-label").val(model[NODE.nodeIdField]);
}

```

```

$("#edit-text").val(model[NODE.nodeTextField]);

$("#editNode").unbind('click').click(function () {
    var validator = IPLAT.Validator({
        id: "editWnd"
    });
    if (validator.validate()) {
        model2Ei.set("result-0-node_id", $("#edit-label").val());
        model2Ei.set("result-0-node_text", $("#edit-text").val());

        EiCommunicator.send("EEDM7001", "update", model2Ei, {
            onSuccess: function () {
                // 重新加载父结点信息
                _tree.reload(model[NODE.parentIdField]);
                NotificationUtil("【" + model[NODE.nodeTextField] + "】" + "结
点更新成功");

                // 清除编辑面板 div 信息
                IPLAT.clearNode(document.getElementById("editNodeDiv"));
                editWndWindow.close();
            },
            onFail: function (ei) {
                // 调用异常
                NotificationUtil("调用失败, 原因[" + ei + "]", "error");
            }
        });
    }
});
}
}
});
},

/**
 * 数据源改变后回调,增删改结点信息都会触发
 * @param e
 *   e.node: 被操作结点的父结点
 */
dataBound: function (e) {
    console.log("DataBound", e.node);
},

/**

```

```

    * expand(e): 结点展开前回调函数, e.node 指被展开的结点;
    * collapse(e): 结点收起前回调函数, e.node 指被收起的结点;
    * saveExpanded(): 结点展开和收起时, 记录结点展开状态
    */
    expand: function () {
        saveExpanded();
    },
    collapse: function () {
        saveExpanded();
    }
}
};

// Cookies 保存结点展开状态 Fn
function saveExpanded() {
    var treeview = $("#menu").data("kendoTreeView");
    var expandedItemsIds = {};
    treeview.element.find(".k-item").each(function () {
        var item = treeview.dataItem(this);
        if (item.expanded) {
            expandedItemsIds[item.id] = true;
        }
    });
    Cookies.set('expanded', kendo.stringify(expandedItemsIds));
}

IPLATUI.EFGrid = {
    "result": {
        loadComplete: function (grid) {
            grid.dataSource.bind("requestEnd", function (e) {
                var response = e.response;
                var type = e.type; // 事件类型
                if ("create" === type || "update" === type || "destroy" === type) {
                    // 获取 kendo 的树
                    var tree = $("#menu").data("kendoTreeView");
                    var ajaxEi = EiInfo.parseJSONObject(response);
                    var result = ajaxEi.getBlock("result");
                    var rows = result.getMappedRows();
                    // rows 表示修改的结点信息
                    for (var i = 0; i < rows.length; i++) {
                        var node = rows[i];
                        tree.reload(node[NODE.parentIdField]); // 刷新树结点信息
                    }
                }
            });
        }
    }
};

```

```

        }
    })
}
}
};

// 用于删除结点时获取其所有子结点 id 的方法
function getChildrenIds(data, idField, parentId, rootId, itemName) {
    var mapList = IPLAT.flat2Map(data, idField, parentId, rootId);

    var list = [];
    list.push(rootId);
    var _map2Array = function (arrayList) {
        $.each(arrayList, function (i, array) {
            if (array[itemName] && array[itemName].length > 0) {
                _map2Array(array[itemName]);
            }
            list.push(array[idField]);
            delete array[itemName];
        })
    };
    _map2Array(mapList);
    return list;
}

function model2EiInfo(model) {
    var ei = new EiInfo(), key;
    var json = model.toJSON();
    for (key in json) {
        if (typeof json[key] === "string") {
            ei.set("result", "0", key, json[key]);
        }
        if (typeof json[key] === "boolean") {
            ei.set("result", "0", key, json[key] ? "2" : "1");
        }
    }
    return ei;
}

$("#F1").on("click", function () {
    resultGrid.dataSource.page(1);
});
});

```

注：EEDM7002 支持树结点拖拽功能，以及右击树结点增删改功能。

3.5.4.1.3.3.3 Service 服务开发

ServiceEEDM06.java & ServiceEEDM7001.java

```
// ServiceEEDM06
public class ServiceEEDM06 extends ServiceEPBase {
    public EilInfo initLoad(EilInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM06().eiMetadata);
        return inInfo;
    }

    @Override
    public EilInfo query(EilInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EilInfo outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    public EilInfo search(EilInfo inInfo) {
        inInfo.set(EiConstant.resultBlock + EiConstant.separator + EiConstant.limitStr, 1000);
        EilInfo outInfo = super.query(inInfo, "EEDM06.search", new EEDM06());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    public EilInfo delete(EilInfo inInfo) {
        return super.delete(inInfo, "EEDM06.delete");
    }

    public EilInfo update(EilInfo inInfo) {
        return super.update(inInfo, "EEDM06.update");
    }

    public EilInfo insert(EilInfo inInfo) {
        return insert(inInfo, "EEDM06.insert");
    }
}
```

```

}

// ServiceEEDM7001
public class ServiceEEDM7001 extends ServiceEPBase {

    public EiInfo initLoad(EiInfo inInfo) {
        EiBlock block = new EiBlock(EiConstant.resultBlock);
        block.addBlockMeta(new EEDM06().getMetaData());
        EiInfo outInfo = new EiInfo();
        outInfo.addBlock(block);
        outInfo.setMsg("欢迎来到树表联动示例");
        return outInfo;
    }

    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        return outInfo;
    }

    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.delete");
    }

    public EiInfo deleteBatch(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.deleteBatch");
    }

    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM06.update");
    }

    public EiInfo insert(EiInfo inInfo) {
        return insert(inInfo, "EEDM06.insert");
    }
}

```


3.5.4.1.3.3.4 页面展示效果

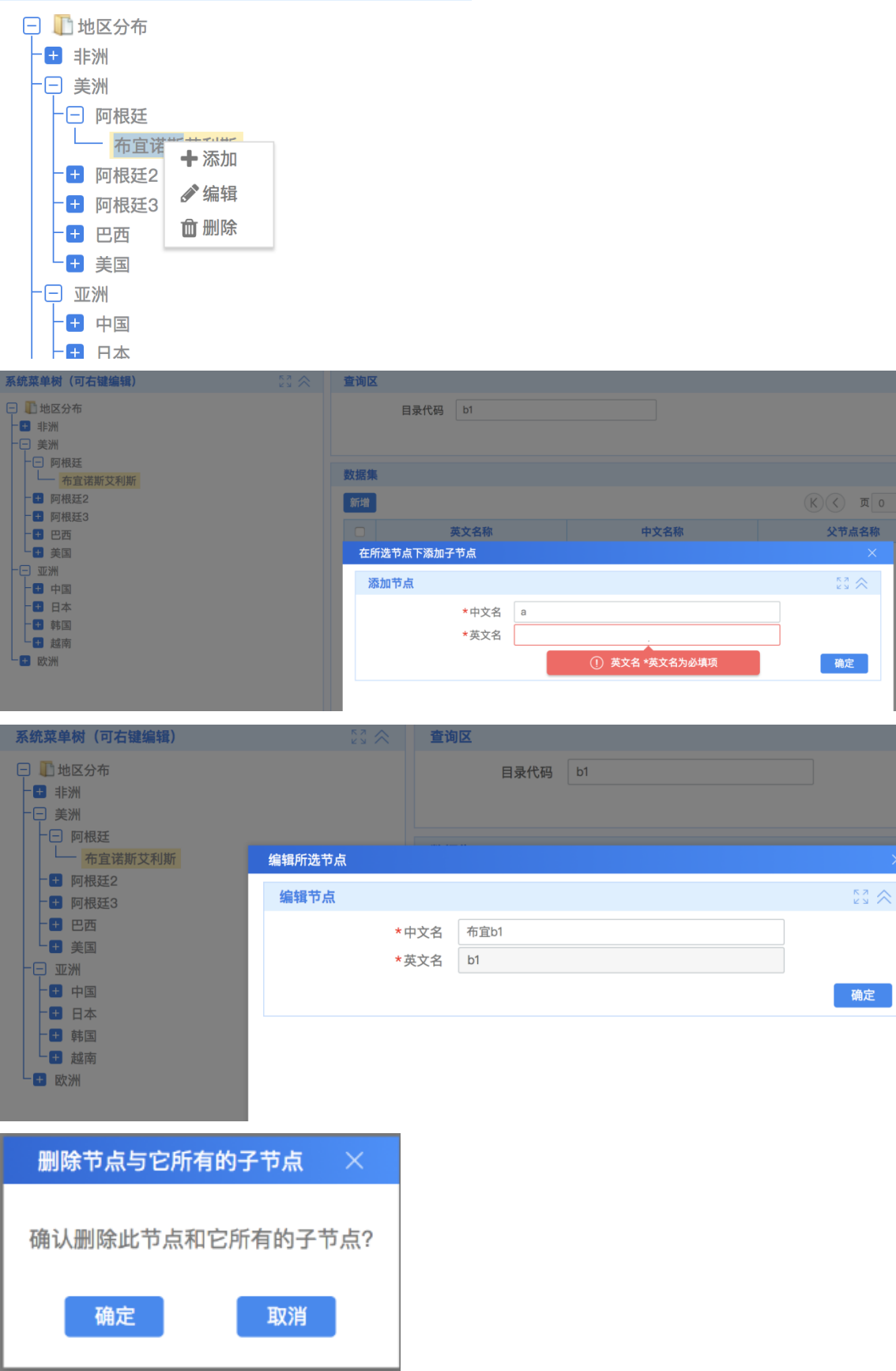


3.5.4.1.3.3.4.1 拖拽功能



3.5.4.1.3.3.4.2 右击树结点增删改功能

系统菜单树（可右键编辑）



3.5.4.1.4 EEDM7003-树结点显示多选框示例

3.5.4.1.4.1 页面功能简介

在使用 **EFTree** 标签时，可能会有需要显示多选框进行结点勾选的需求，我们可以在 JS 文件中配置 **kendoTreeView** 的原生属性 **checkboxes**.

3.5.4.1.4.2 配置和 API 详情

参考 [EFGGrid](#) 与 [EFTree](#) 中的描述。

3.5.4.1.4.3 页面开发步骤

3.5.4.1.4.3.1 JSP 页面开发

EEDM7003.jsp

```
<EF:EFPage>
  <div class="row">
    <div class="col-md-3">
      <EF:EFRegion id="tree" title="系统菜单树" fitHeight="true">
        <EF:EFTree id="menu" valueField="node_id" textField="node_text"
hasChildren="node_type"
          serviceName="EEDM06" methodName="query" style="height:500px;">
        </EF:EFTree>
      </EF:EFRegion>
    </div>

    <div class="col-md-9">
      <EF:EFRegion id="inqu" title="查询区">
        <div class="row">
          <EF:EFInput ename="inqu_status-0-node" cname="当前勾选结点 ID"
ratio="5:7"/>
          <EF:EFInput ename="inqu_status-0-tree_all" cname="所有勾选结点 ID"
colWidth="8"/>
        </div>
      </EF:EFRegion>
      <EF:EFRegion id="result" title="数据集" fitHeight="true">
        <EF:EFGGrid blockId="result" autoDraw="no" serviceName="EEDM7001">
          <EF:EFColumn ename="node_id" cname="英文名称"/>
          <EF:EFColumn ename="node_text" cname="中文名称"/>
          <EF:EFColumn ename="parent_id" cname="父结点名称"/>
          <EF:EFComboColumn ename="node_type" cname="结点类型"
valueField="valueField" textField="textField">
            <EF:EFOption label="有子结点" value="2">
            </EF:EFOption>
            <EF:EFOption label="无子结点" value="1">
            </EF:EFOption>
          </EF:EFComboColumn>
        </EF:EFGGrid>
      </EF:EFRegion>
    </div>
  </div>
```

```

        </EF:EFComboColumn>
    </EF:EFGrid>
</EF:EFRegion>
</div>
</div>
</EF:EFPage>

```

3.5.4.1.4.3.2 JS 开发

EEDM7003.js

```

$(function () {
    IPLATUI.EFTree = {
        "menu": {
            ROOT: {node_id: "root", node_text: "地区分布", node_type: 2},
            checkboxes: { // 配置结点前多选框,可直接配置 checkboxes: true
                checkChildren: true // 是否连同子结点一起勾选
            },
            check: onCheck // 勾选触发事件配置
        }
    };

    IPLATUI.EFGrid = {
        "result": {
            loadComplete: function (grid) {
                grid.dataSource.bind("requestEnd", function (e) {
                    var response = e.response;
                    if (e.type == "create" || e.type == "update" || e.type == "destroy") {
                        var tree = $("#menu").data("kendoTreeView");
                        var ajaxEi = EiInfo.parseJSONObject(response);
                        var result = ajaxEi.getBlock("result");
                        var rows = result.getMappedRows();

                        for (var i = 0; i < rows.length; i++) {
                            var node = rows[i];
                            // 找到父亲结点
                            var pNode = tree.dataSource.get(node["tree_ename"]);

                            if (pNode) {
                                // 重新设置 flag , 这样才能重新使其到服务器获取数据
                                pNode.loaded(false);
                                // 重新加载
                                pNode.load();
                            }
                        }
                    }
                });
            }
        }
    };

```

```

        }
    }
}
})
}
}
};

$("#F1").on("click", function () {
    resultGrid.dataSource.page(1);
    return false;
});

function checkedNodeIds(nodes, checkedNodes) {
    for (var i = 0; i < nodes.length; i++) {
        if (nodes[i].checked) {
            checkedNodes.push(nodes[i].id);
        }

        if (nodes[i].hasChildren) {
            checkedNodeIds(nodes[i].children.view(), checkedNodes);
        }
    }
}

// onCheck 事件，获得所有被选中的结点 id 数组，以","隔开
function onCheck(e) {
    // 联动显示 Grid
    var _data = this.dataItem(e.node);
    $("#inqu_status-0-node").val(_data.id);
    resultGrid.dataSource.page(1);

    var checkedNodes = [],
        treeView = $("#menu").data("kendoTreeView"),
        checkedIds;

    checkedNodeIds(treeView.dataSource.view(), checkedNodes);

    if (checkedNodes.length > 0) {
        checkedIds = checkedNodes.join(",");
    } else {
        checkedIds = "无结点被勾选";
    }
}

```

```

        $("#inqu_status-0-tree_all").val(checkedIds);
    }
});

```

3.5.4.1.4.3.3 Service 服务开发

ServiceEEDM06.java & ServiceEEDM7001.java

```

// ServiceEEDM06
public class ServiceEEDM06 extends ServiceEPBase {
    public EiInfo initLoad(EiInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM06().eiMetadata);
        return inInfo;
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    public EiInfo search(EiInfo inInfo) {
        inInfo.set(EiConstant.resultBlock + EiConstant.separator + EiConstant.limitStr, 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.search", new EEDM06());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.delete");
    }

    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM06.update");
    }
}

```

```

    }

    public EiInfo insert(EiInfo inInfo) {
        return insert(inInfo, "EEDM06.insert");
    }

}

// ServiceEEDM7001
public class ServiceEEDM7001 extends ServiceEPBase {

    public EiInfo initLoad(EiInfo inInfo) {
        EiBlock block = new EiBlock(EiConstant.resultBlock);
        block.addBlockMeta(new EEDM06().getMetaData());
        EiInfo outInfo = new EiInfo();
        outInfo.addBlock(block);
        outInfo.setMsg("欢迎来到树表联动示例");
        return outInfo;
    }

    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        return outInfo;
    }

    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.delete");
    }

    public EiInfo deleteBatch(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.deleteBatch");
    }

    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM06.update");
    }

    public EiInfo insert(EiInfo inInfo) {

        return insert(inInfo, "EEDM06.insert");
    }
}

```

```
}
```

3.5.4.1.4.3.4 页面展示效果



3.5.4.1.5 EEDM7004-模板显示结点信息示例

3.5.4.1.5.1 页面功能简介

本示例页面用于说明，如何通过 js 配置树原生的 `template` 属性模板显示结点信息。

3.5.4.1.5.2 配置和 API 详情

参考 [EFGGrid](#) 与 [EFTree](#) 中的描述。

3.5.4.1.5.3 页面开发步骤

3.5.4.1.5.3.1 JSP 页面开发

EEDM7004.jsp

```
<EF:EFPage>
  <div class="row">
    <div class="col-md-3">
      <EF:EFRegion id="tree" title="系统菜单树">
        <EF:EFTree id="menu" textField="text" valueField="label" hasChildren="leaf"
          serviceName="EEDM05" methodName="query" style="height:500px;">
        </EF:EFTree>
      </EF:EFRegion>
    </div>

    <div class="col-md-9">
```



```

    <EF:EFRegion id="inqu" title="查询区">
      <div class="row">
        <EF:EFInput ename="inqu_status-0-node" cname="当前勾选结点 ID"
ratio="5:7"/>
        <EF:EFInput ename="inqu_status-0-tree_all" cname="所有勾选结点 ID"
colWidth="8"/>
      </div>
    </EF:EFRegion>
    <EF:EFRegion id="result" title="数据集">
      <EF:EFGrid blockId="result" serviceName="EEDM7001" autoDraw="false">
        <EF:EFColumn ename="text" cname="中文名称"/>
        <EF:EFColumn ename="pId" cname="父结点名称"/>
        <EF:EFComboColumn ename="leaf" cname="类型" valueField="valueField"
textField="textField">
          <EF:EFOption label="有子结点" value="0">
            </EF:EFOption>
          <EF:EFOption label="无子结点" value="1">
            </EF:EFOption>
        </EF:EFComboColumn>
      </EF:EFGrid>
    </EF:EFRegion>
  </div>
</div>
</EF:EFPage>

```

3.5.4.1.5.3.2 JS 开发

EEDM7001.js

```

$(function() {
  IPLATUI.EFTree = {
    "menu": {
      ROOT: {label: "root", text: "地区分布", leaf: true},
      template: "#= item.text # (#= item.id #)", // 配置树结点的模板
      checkboxes: {
        checkChildren: true
      },
    },
    check: onCheck
  }
};
.....
});

```

1. JS中使用kendoTreeView的原生属性template，可以对结点信息进行自定义显示
2. 模板中要显示的值xxx，可以写成"#=item.xxx#"。具体可见 [Kendo Template 使用介绍](#)

3.5.4.1.5.3.3 Service 服务开发

ServiceEEDM7001.java

```
public class ServiceEEDM7001 extends ServiceEPBase {

    public EiInfo initLoad(EiInfo inInfo) {
        EiBlock block = new EiBlock(EiConstant.resultBlock);
        block.addBlockMeta(new EEDM05().getMetaData());
        EiInfo outInfo = new EiInfo();
        outInfo.addBlock(block);
        outInfo.setMsg("欢迎来到树表联动示例");
        return outInfo;
    }

    public EiInfo query(EiInfo inInfo) {
        EiInfo outInfo = super.query(inInfo, "EEDM05.query", new EEDM05());
        return outInfo;
    }

    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM05.delete");
    }

    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM05.update");
    }

    public EiInfo insert(EiInfo inInfo) {

        return insert(inInfo, "EEDM05.insert");
    }

}
```

3.5.4.1.5.3.4 页面展示效果



3.5.4.1.6 EEDM7005-过滤选择树结点示例

3.5.4.1.6.1 页面功能简介

当树结点过多时，我们可以在 JS 中配合使用 `kendoAutoComplete` 下拉筛选出结点并生成以之为首结点的一棵树，实现树结点的过滤选择，具体如下。

3.5.4.1.6.2 配置和 API 详情

参考 [EFGGrid](#) 与 [EFTree](#) 中的描述。

3.5.4.1.6.3 页面开发步骤

3.5.4.1.6.3.1 JSP 页面开发

EEDM7005.jsp

```
<EF:EFPage>
  <div class="row">
    <div class="col-md-3">
      <EF:EFRegion id="tree" title="系统菜单树">
        <div class="row">
          <div class="col-md-4">
            <span style="line-height: 25px">过滤器</span>
          </div>
          <div class="col-md-8">
            <input id="filterText" name="filterText" />
          </div>
        </div>

        <div id="menu" style="height:550px;margin-top: 20px">
          <EF:EFTree id="menu01" textField="text" valueField="label"
            hasChildren="leaf"
              serviceName="EEDM05" methodName="query" >
        </EF:EFTree>
        </div>
      </EF:EFRegion>
    </div>
  </div>
</EF:EFPage>
```

```

        </div>
    </EF:EFRegion>
</div>
</div>
<script>
    var ctx = "${ctx}";
</script>
</EF:EFPage>

```

3.5.4.1.6.3.2 JS 开发 EEDM7005.js

```

$(function () {
    IPLATUI.EFTree = {
        "menu01": {
            ROOT: {label: "root", text: "地区分布", leaf: true},
            template: "#= item.text # (#= item.id #)"
        }
    };

    var dataSource = new kendo.data.DataSource({
        transport: {
            read: {
                url: IPLATUI.CONTEXT_PATH + "/service/EEDM05/search",
                type: 'POST',
                dataType: "json",
                contentType: "application/json"
            },
            parameterMap: function (options, operation) {
                var info = new EiInfo();
                info.set("inqu_status-0-cname",$("#filterText").val());
                return info.toJSONString();
            }
        },
        schema: {
            data: function (response) {
                ajaxEi = EiInfo.parseJSONObject(response);
                return ajaxEi.getBlock("result").getMappedRows();
            }
        },
        pageSize: 10,
        serverFiltering: true
    });

```

```

});

var options = {
    url: ctx,
    serviceName: "EEDM05",
    methodName: "query",
    textField: "text",
    valueField: "label",
    hasChildren: "leaf"
};

$("#filterText").kendoAutoComplete({
    dataSource: dataSource,
    minLength: 1,
    enforceMinLength: true,
    select: function(e) {
        // 清空以前的 DIV 块
        $("#menu").empty();

        // 获取新的 DIV
        var div = document.createElement("div");
        var id = "menu"+kendo.guid();
        div.id = id;
        $("#menu").append(div);

        // 构建一个新的 Tree
        var initOptions = $.extend({},options,{treeId: id},{ROOT:e.dataItem});
        IPLAT.TreeView(initOptions);
    },
    dataTextField: "text"
});
});

```

3.5.4.1.6.3.3 Service 服务开发

ServiceEEDM05.java

```

public class ServiceEEDM05 extends ServiceEPBase {
    public EiInfo initLoad(EiInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM05().eiMetadata);
        return inInfo;
    }
}

```

```
@Override
public EilInfo query(EilInfo inInfo) {
    EilInfo outInfo = super.query(inInfo, "EEDM05.query", new EEDM05());
    String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
    outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
    outInfo.getBlocks().remove(EiConstant.resultBlock);
    return outInfo;
}

public EilInfo search(EilInfo inInfo) {
    return super.query(inInfo, "EEDM05.search", new EEDM05());
}

public EilInfo delete(EilInfo inInfo) {
    return super.delete(inInfo, "EEDM05.delete");
}

public EilInfo update(EilInfo inInfo) {
    return super.update(inInfo, "EEDM05.update");
}

public EilInfo insert(EilInfo inInfo) {
    return insert(inInfo, "EEDM05.insert");
}
}
```

3.5.4.1.6.3.4 页面展示效果



3.5.4.1.7 EEDM7006-树结点增删改示例

要想对树结点进行增删改操作，可以在 **JS** 文件中使用 **Tree** 的原生方法进行操作。

JSP 示例

```
<EF:EFRegion title="系统菜单树" id="tree">
  <div class="row">
    <EF:EFInput ename="text" colWidth="8" inline="true"/>
    <EF:EFButton ename="add" efClass="fa fa-plus" layout="2"/>
    <EF:EFButton ename="remove" efClass="fa fa-trash-o" layout="2"/>
    <EF:EFButton ename="edit" efClass="fa fa-pencil" layout="2"/>
  </div>
  <EF:EFTree bindId="tree01" textField="text" valueField="label" hasChildren="leaf"
    serviceName="EEDM05" methodName="query">
  </EF:EFTree>
</EF:EFRegion>
```

JS 示例

```
IPLATUI.EFTree = {
  "tree01": {
    change: function (e) {
      var treeview = $("#tree01").data("kendoTreeView");
      // 新增结点
```

```

$( "#add").on("click", function(){
    var textArray = $("#text").val().trim().split(","); // id,text 形式
    var selectedNode = treeview.select();           // 当前选中结点
    if(textArray.length > 1 ){
        treeview.append({                          // append()方法, 会将新增结点添加为
selectedNode 的子结点
            id: textArray[0],
            text: textArray[1]
        }, selectedNode);
    }
});

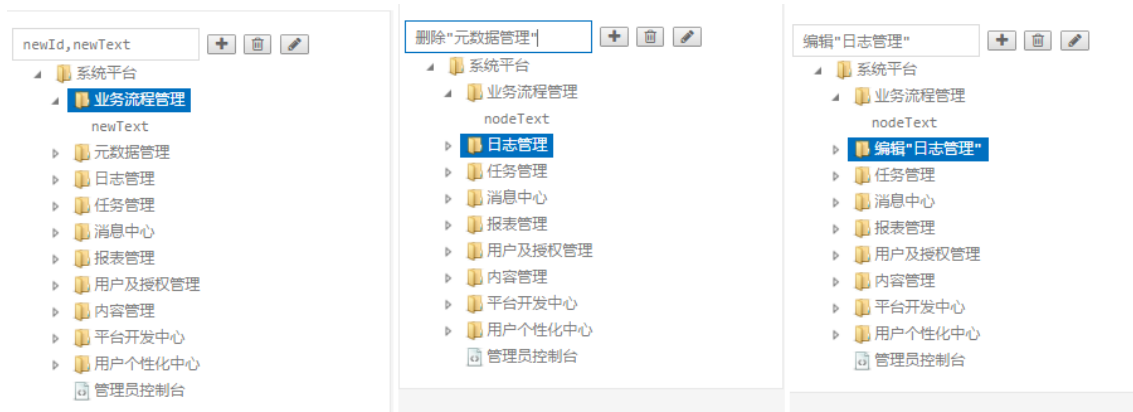
// 删除结点
$("#remove").on("click", function(){
    var selectedNode = treeview.select();
    treeview.remove(selectedNode);                 // remove()方法删除结点, 如有子结点
会被一并删除
});

// 编辑修改选中结点 text
$("#edit").on("click", function(){
    var text = $("#text").val().trim();
    var selectedNode = treeview.select();
    treeview.text(selectedNode, text);             // text()方法给编辑结点 text 值
});

resultGrid.dataSource.query();
},
ROOT:"root"
}
};

```

页面效果：



3.5.4.1.8 EEDM7022-EFTree 配置结点图标

3.5.4.1.8.1 页面功能简介

本示例页面用于说明如何通过 EFTree 上的属性配置树的结点图标：

1. 配置 **dataSpriteCssClassField** 属性 ( 推荐, dataSpriteCssClassField 字段值对应的样式会被添加到结点前, 当然也可自定义样式图片)
2. 配置 **dataImageUrlField** 属性 ( 不推荐, 需要将 dataImageUrlField 字段值对应的图片添加到 iplatui/img 目录下)

3.5.4.1.8.2 配置和 API 详情

参考 [EFTree](#) 中的描述。

3.5.4.1.8.3 页面开发步骤

3.5.4.1.8.3.1 JSP 页面开发

功能简介所述两种配置结点图标方式代码如下：

EEDM7022.jsp

```
<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<style>
    .icon-flower { /*dataSpriteCssClassField 属性配置树结点图标 根结点自定义样式*/
        background-image: url("../iplatui/img/icon_1.png")
    }
</style>

<EF:EFPage>
    <EF:EFRegion id="R1" title="1.dataSpriteCssClassField 属性配置树结点图标">
        <h5 class="text-success">
            推荐, dataSpriteCssClassField 字段值对应的样式会被添加到结点前, 当然也可自定义样式图片
        </h5>
        <EF:EFTree id="tree01" valueField="node_id" textField="node_text"
            hasChildren="node_type"
            root="{node_id: 'root', node_text: '地区分布', icon: 'icon-flower'}"
            dataSpriteCssClassField="icon"
            serviceName="EEDM06" methodName="query">
        </EF:EFTree>
    </EF:EFRegion>
</EF:EFPage>
```

```

</EF:EFRegion>

<EF:EFRegion id="R2" title="2.dataImageUrlField 属性配置树结点图标">
  <h5 class="text-danger">
    不推荐，需要将 dataImageUrlField 字段值对应的图片添加到 webapp/iplatui/img 目录下
  </h5>
  <EF:EFTree id="tree02" valueField="node_id" textField="node_text"
hasChildren="node_type"
    root="{node_id: 'root', node_text: 'root', icon: '../iplatui/img/icon_2.png'}"
    dataImageUrlField="icon"
    serviceName="EEDM06" methodName="query">
  </EF:EFTree>
</EF:EFRegion>

</EF:EFPage>

```

3.5.4.1.8.3.2 Service 服务开发

EFTree 中我们调用的是 ServiceEEDM06，代码如下。

ServiceEEDM06.java

```

public class ServiceEEDM06 extends ServiceEPBase {
    @Override
    public EiInfo initLoad(EiInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM06().eiMetadata);
        return inInfo;
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo;

        if (inInfo.getInt(EiConstant.queryBlock + "-0-expandLevel") > 0) {
            outInfo = super.query(inInfo, "EEDM06.queryByLevel", new EEDM06());
        } else {
            outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        }

        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
    }
}

```

```

    return outInfo;
}

```

添加 EEDM06.xml:

EEDM06.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE sqlMap PUBLIC "-//iBATIS.com//DTD SQL Map 2.0//EN"
"http://www.ibatis.com/dtd/sql-map-2.dtd">

<sqlMap namespace="EEDM06">

    <select id="count" resultClass="int">
        SELECT count(*) FROM ${platSchema}.teedm06 where 1=1
        <isEmpty prepend=" AND " property="node">
            p_ename = #node#
        </isEmpty>
    </select>

    <select id="query" resultClass="java.util.HashMap">
        SELECT
            ename    as "node_id",
            cname    as "node_text",
            p_ename  as "parent_id",
            node_type as "node_type",
            sort     as "sort",
            icon     as "icon",
            node_level as "node_level",
            node_path as "node_path",
            node_url  as "node_url"
        FROM ${platSchema}.teedm06 where 1=1
        <isEmpty prepend=" AND " property="node">
            p_ename = #node#
        </isEmpty>
        order by ename, sort asc
    </select>

    <select id="queryByLevel" resultClass="java.util.HashMap">
        SELECT
            ename    as "node_id",
            cname    as "node_text",
            p_ename  as "parent_id",
            node_type as "node_type",

```

```

sort    as "sort",
icon    as "icon",
node_level as "node_level",
node_path as "node_path",
node_url  as "node_url"
FROM ${platSchema}.teedm06 where 1=1
<isEmpty prepend=" AND " property="expandLevel">
  node_level &lt;= #expandLevel#
</isEmpty>
order by ename, sort asc
</select>

<select id="search" resultClass="java.util.HashMap">
  SELECT
  ename    as "node_id",
  cname    as "node_text",
  p_ename  as "parent_id",
  node_type as "node_type",
  sort     as "sort",
  icon     as "icon",
  node_level as "node_level",
  node_path as "node_path",
  node_url  as "node_url"
  FROM ${platSchema}.teedm06 where 1=1
  <isEmpty prepend=" AND " property="node_level">
    node_level &lt;= #node_level#
  </isEmpty>
  order by ename, sort asc
</select>

<insert id="insert">
  INSERT INTO ${platSchema}.teedm06 (
    ename,
    cname,
    p_ename
    <isEmpty prepend="," property="node_type">node_type</isEmpty>
    <isEmpty prepend="," property="icon">icon</isEmpty>
    <isEmpty prepend="," property="sort">sort</isEmpty>
    <isEmpty prepend="," property="node_level">node_level</isEmpty>
    <isEmpty prepend="," property="node_path">node_path</isEmpty>
    <isEmpty prepend="," property="node_url">node_url</isEmpty>
    <![CDATA[
      ) VALUES (

```

```

]]>
#node_id#,
#node_text#,
#parent_id#
<isEmpty prepend="," property="node_type">#node_type#</isEmpty>
<isEmpty prepend="," property="icon">#icon#</isEmpty>
<isEmpty prepend="," property="sort">#sort#</isEmpty>
<isEmpty prepend="," property="node_level">#node_level#</isEmpty>
<isEmpty prepend="," property="node_path">#node_path#</isEmpty>
<isEmpty prepend="," property="node_url">#node_url#</isEmpty>
<![CDATA[
    )
]]>

</insert>

<update id="update">
UPDATE ${platSchema}.teedm06
<dynamic prepend="set">
  <isNotNull prepend="," property="node_id">
    ename = #node_id#
  </isNotNull>
  <isNotNull prepend="," property="node_text">
    cname = #node_text#
  </isNotNull>
  <isNotNull prepend="," property="parent_id">
    p_ename = #parent_id#
  </isNotNull>
  <isNotNull prepend="," property="node_type">
    node_type = #node_type#
  </isNotNull>
  <isNotNull prepend="," property="sort">
    sort = #sort#
  </isNotNull>
  <isNotNull prepend="," property="icon">
    icon = #icon#
  </isNotNull>
  <isNotNull prepend="," property="node_level">
    node_level = #node_level#
  </isNotNull>
  <isNotNull prepend="," property="node_path">
    node_path = #node_path#
  </isNotNull>

```

```

    <isNotNull prepend="," property="node_url">
        node_url = #node_url#
    </isNotNull>
</dynamic>
WHERE ename = #node_id#
</update>

<delete id="delete">
    DELETE FROM ${platSchema}.teedm06
    WHERE ename = #node_id#
</delete>

<delete id="deleteBatch">
    DELETE FROM ${platSchema}.teedm06 WHERE ename in
    <iterate property="labelList" open="(" close=")" conjunction="," ">
        #labelList[]#
    </iterate>
</delete>

<!--
EEDM7100 树菜单示例:
    queryA: 查询模块项
    queryB: 查询一级菜单项(可以来自不同表)
    queryC: 查询二级菜单项(可以来自不同表)

    '_A' 用于标识当前结点类型, 在 Service 中调用对应查询方法
-->
<select id="queryA" resultClass="java.util.HashMap">
    SELECT
    CONCAT(ename, '_A')    as "node_id",
    cname    as "node_text",
    p_ename  as "parent_id",
    node_type as "node_type",
    sort     as "sort",
    icon     as "icon",
    node_level as "node_level",
    node_path as "node_path",
    node_url  as "node_url"
    FROM ${platSchema}.teedm06 where 1=1
    <isNotEmpty prepend=" AND " property="node">
        p_ename = #node#
    </isNotEmpty>
    order by ename, sort asc

```

```

</select>

<select id="queryB" resultClass="java.util.HashMap">
  SELECT
  CONCAT(ename, '_B')   as "node_id",
  cname   as "node_text",
  p_ename as "parent_id",
  node_type as "node_type",
  sort    as "sort",
  icon    as "icon",
  node_level as "node_level",
  node_path as "node_path",
  node_url  as "node_url"
  FROM ${platSchema}.teedm06 where 1=1
  <isEmpty prepend=" AND " property="node">
    CONCAT(p_ename, '_A') = #node#
  </isEmpty>
  order by ename, sort asc
</select>

<select id="queryC" resultClass="java.util.HashMap">
  SELECT
  CONCAT(ename, '_C')   as "node_id",
  cname   as "node_text",
  p_ename as "parent_id",
  node_type as "node_type",
  sort    as "sort",
  icon    as "icon",
  node_level as "node_level",
  node_path as "node_path",
  node_url  as "node_url"
  FROM ${platSchema}.teedm06 where 1=1
  <isEmpty prepend=" AND " property="node">
    CONCAT(p_ename, '_B') = #node#
  </isEmpty>
  order by ename, sort asc
</select>

</sqlMap>

```

3.5.4.1.8.3.3 数据库中字段 icon 取值

	ENAME	CNAME	P_ENAME	NODE_TYPE	SORT	ICON	NODE_LEVEL
1	Asia	亚洲	root	2		fa fa-home	1
2	America	美洲	root	2		fa fa-search	1
3	Europe	欧洲	root	2			1
4	USA	美国	America	1			2
5	Brazil	巴西	America	2			2
6	BA	布宜诺斯艾利斯	Argentina	1			3
7	China	中国	Asia	2			2
8	Vietnam	越南	Asia	1			2
9	Korea	韩国	Asia	1			2
10	Japan	日本	Asia	1			2
11	Germany	德国	Europe	1			2
12	France	法国	Europe	1			2
13	England	英国	Europe	1			2
14	jiangsu	江苏	China	2			3
15	shanghai	上海	China	2			3
16	nanjing	南京	jiangsu	1			4
17	xuzhou	徐州	jiangsu	2			4
18	baoshan	宝山区	shanghai	1			4
19	jingan	静安区	shanghai	2			4
20	Africa	非洲	root	1		../iplatui/img/icon_1.png	1
21	Argentina	阿根廷	America	2			2
22	Canada	加拿大	America	1			3
23	Rio	里约热内卢	Brazil	1			1

1. fa fa-xx 为fontawesome字体样式，无需自行定义，具体使用可参考[fontawesome图标库](#)

2.使用dataImageUrlField属性需注意图片路径配置正确

3.5.4.1.8.3.4 页面展示效果



3.5.4.1.9 EEDM7102-树结点多次点击触发事件示例

3.5.4.1.9.1 页面功能简介

kendo 的树组件原生 select 事件只会在树结点首次被点击时触发，本示例页面用于说明 **如何使用模板 template 实现树上结点多次点击可触发**。

3.5.4.1.9.2 配置和 API 详情

参考 [Kendo Template 使用介绍](#) 中的说明。

3.5.4.1.9.3 页面开发步骤

3.5.4.1.9.3.1 JSP 页面开发

EEDM7102.jsp

```
<EF:EFPage>
  <div class="col-md-4">
    <EF:EFRegion id="R1" title="树结点多次点击触发事件示例" fitHeight="true">
      <EF:EFTree id="tree01" valueField="node_id" textField="node_text"
        root="{node_id: 'root', node_text: '根结点'}"
        hasChildren="node_type" serviceName="EEDM06" methodName="query">
      </EF:EFTree>
    </EF:EFRegion>
  </div>
</EF:EFPage>
```

3.5.4.1.9.3.2 JS 开发

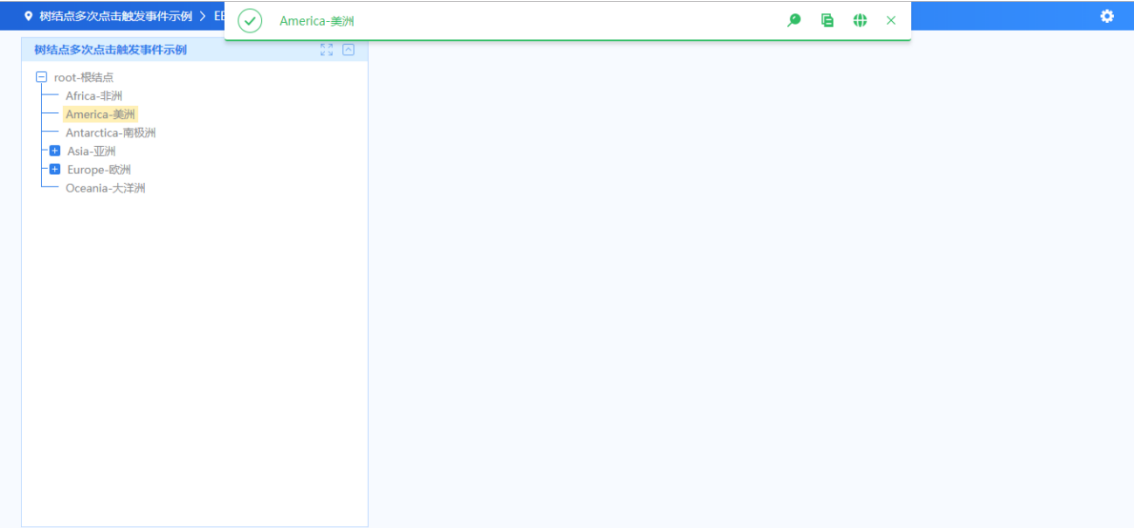
EEDM7102.js

```
$(function () {
  IPLATUI.EFTree = {
    "tree01": {
      template: "<span onClick='onNodeClick(this);'># item.node_id #-#=
item.node_text # </span>"
    }
  };
});

/**
 * 树的原生 select 事件，在多次点击同一结点只触发一次
 * 在 template 里将树结点用 span 包裹并给一个 onClick 事件，可解决此问题
 *
 * @param e 当前点击的树结点
 */
function onNodeClick(e) {
  var tree = $("#tree01").data("kendoTreeView");
  var node = tree.dataItem(e);

  NotificationUtil(node["node_id"] + "-" + node["node_text"], "success");
}
```

3.5.4.1.9.3.3 页面效果



3.5.4.2 EEDM7400-EFTreeList 组件示例

3.5.4.2.1 页面功能简介

本示例页面用于说明，如何使用 **EFTreeList** 标签进行树表展示。**EFTreeList** 本质上是一颗树，只是将树结点的其他属性字段，通过表格的形式展现出来了。

3.5.4.2.2 配置和 API 详情

参考 [EFTreeList](#) 中的描述。

3.5.4.2.3 页面开发步骤

3.5.4.2.3.1JSP 页面开发

EFTreeList 用于展示层级的数据，通常将其放置在 **EFRegion** 区域内，示例代码如下：

JSP 示例

```
<EF:EFRegion id="R1" title="查询">

</EF:EFRegion>

<EF:EFRegion id="R2" title="地区">
  <EF:EFTreeList id="area" valueField="node_id" pid="parent_id" root="root"
hasChildren="node_type" serviceName="EEDM06">
    <EF:EFColumn ename="node_id" cname="节点编号"/>
    <EF:EFColumn ename="node_path" cname="根路径"/>
    <EF:EFColumn ename="sort" cname="排序"/>
    <EF:EFColumn ename="node_level" cname="层级"/>
  </EF:EFTreeList>
</EF:EFRegion>
```

```
</EF:EFRegion>
```

3.5.4.2.3.2 Service 服务开发

EFTreeList 中我们调用的是 ServiceEEDM06，和逐级加载的 Tree 没有区别，代码如下：

```
public class ServiceEEDM06 extends ServiceEPBase {
    @Override
    public EiInfo initLoad(EiInfo inInfo) {
        inInfo.addBlock(EiConstant.resultBlock);
        inInfo.getBlock(EiConstant.resultBlock).addBlockMeta(new EEDM06().eiMetadata);
        return inInfo;
    }

    @Override
    public EiInfo query(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo;

        if (inInfo.getInt(EiConstant.queryBlock + "-0-expandLevel") > 0) {
            outInfo = super.query(inInfo, "EEDM06.queryByLevel", new EEDM06());
        } else {
            outInfo = super.query(inInfo, "EEDM06.query", new EEDM06());
        }

        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    /**
     * 一次性查询 N 层的树结点
     *
     * @param inInfo 包含层级参数
     */
    public EiInfo queryByLevel(EiInfo inInfo) {
        inInfo.set("result-limit", 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.queryByLevel", new EEDM06());
    }
}
```

```

        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);

        return outInfo;
    }

    public EiInfo search(EiInfo inInfo) {
        inInfo.set(EiConstant.resultBlock + EiConstant.separator + EiConstant.limitStr, 1000);
        EiInfo outInfo = super.query(inInfo, "EEDM06.search", new EEDM06());
        String pEname = outInfo.getCellStr(EiConstant.queryBlock, 0, "node");
        outInfo.getBlocks().put(pEname, outInfo.getBlock(EiConstant.resultBlock));
        outInfo.getBlocks().remove(EiConstant.resultBlock);
        return outInfo;
    }

    @Override
    public EiInfo delete(EiInfo inInfo) {
        return super.delete(inInfo, "EEDM06.delete");
    }

    @Override
    public EiInfo update(EiInfo inInfo) {
        return super.update(inInfo, "EEDM06.update");
    }

    @Override
    public EiInfo insert(EiInfo inInfo) {
        return insert(inInfo, "EEDM06.insert");
    }
}

```

在后端服务中，需要根据查询的 **inqu_status** 数据块中的查询条件，获取下一级数据。返回的 **block** 名称是前端传入的树节点 **Id**。后端服务中需要注意：

1. 约定在后端获取前端传入的树节点三段式格式是：**inqu_status-0-node**
2. 开发人员把多行数据存储在 **block** 中，每一行里面通常需要包含带有 **valueField**, **pid** 关系的属性。

3.5.4.2.3.3 前端 JS 配置

```
// 暂无
```

注意：EFTreeList 逐级加载数据。

3.5.4.3 EFSwap 示例

3.5.4.3.1 EEDM7007-EFSwap 基础示例

3.5.4.3.1.1 页面功能简介

本示例页面用于说明，如何使用 EFSwap 标签灵活配置左右两棵树的子节点，并在如下示例中点击"打印"按钮返回两棵树的 list 集合。

3.5.4.3.1.2 配置和 API 详情

参考 [EFSwap](#) 中的描述。

3.5.4.3.1.3 页面开发步骤

3.5.4.3.1.3.1 JSP 页面开发

EEDM7007.jsp

```
<% // 配置左右两棵树的静态数据源
    String leftDataSource = "[{text: 'a', items: [{ text: 'a1', items: [{ text: 'a11'}, { text: 'a12'}]},
    { text: 'a2',items:[{text:'a21'},items:[{text:'a211'},{text:'a212'}]},{text:'a22'}]},{text:'a3'}]}, { text:
    'b'}]";
    String rightDataSource = "[{text: 'A', items: [{ text: 'A1', items: [{ text: 'A11'}, { text: 'A12'}]},
    { text: 'A2'}]}, { text: 'B'}]";

    request.setAttribute("leftDataSource", leftDataSource);
    request.setAttribute("rightDataSource", rightDataSource);
%>

<EF:EFPage>
    <div class="row">
        <div class="col-md-6">
            <EF:EFRegion id="tree" title="静态示例树">
                <div class="row">
                    <h5 class="push-30-l text-info">注： 点击按钮打印出两棵树的 dataSource 集合
                </h5>
            </div>
            <EF:EFSwap blockId="result" row="0" leftName="tree1" rightName="tree2"
            immediatelyRender="false"
                leftDataSource="{leftDataSource}" rightDataSource="{rightDataSource}"
            height="300"
```

```

        width="200"/>
    <br/>
    <br/>
    <div style="text-align: right; margin: 0; padding-top: 10px;">
        <button id="btn" type="button" class="i-btn-lg i-btn-wide">打印</button>
    </div>
</EF:EFRegion>
</div>
</div>
</EF:EFPage>

```

1. 设置左右两棵树的静态数据leftDataSource和rightDataSource
2. leftName和rightName设置左右两棵树的英文名（tree1和tree2）
3. height和width设置左右树的高度和宽度，默认height="300" width="200"
4. immediatelyRender="false"，不会立即渲染tag上配置的数据源

3.5.4.3.1.3.2 Service 开发 ServiceEEDM7007.java

```

public class ServiceEEDM7007 extends ServiceEPBase {

    public EiInfo initLoad(EiInfo inInfo) {
        return mockData();
    }

    // 提供模拟数据
    public EiInfo mockData() {
        EiInfo outInfo = new EiInfo();
        String jsonA = "[{text: 'a', id: 'a', pid: 'root'}, {text: 'a1', id: 'a1', pid: 'a'}, {text: 'a11', id: 'a11', pid: 'a1'}, {text: 'a12', id: 'a12', pid: 'a1'}, {text: 'a2', id: 'a2', pid: 'a'}, {text: 'a21', id: 'a21', pid: 'a2'}, {text: 'a21', id: 'a21', pid: 'a1'}, {text: 'a211', id: 'a211', pid: 'a21'}, {text: 'a212', id: 'a212', pid: 'a21'}, {text: 'a22', id: 'a22', pid: 'a2'}, {text: 'a3', id: 'a3', pid: 'a'}, {text: 'b', id: 'b', pid: 'root'}}]";
        String jsonB = "[{text: 'A', id: 'A', pid: 'root'}, {text: 'A1', id: 'A1', pid: 'A'}, {text: 'A11', id: 'A11', pid: 'A1'}, {text: 'A12', id: 'A12', pid: 'A1'}, {text: 'A2', id: 'A2', pid: 'A'}, {text: 'B', id: 'B', pid: 'root'}}]";

        EiBlock resultA = MockDataUtil.createMockTree("resultA", jsonA);
        EiBlock resultB = MockDataUtil.createMockTree("resultB", jsonB);

        outInfo.addBlock(resultA);
    }
}

```

```

        outInfo.addBlock(resultB);
        return outInfo;
    }
}

```

3.5.4.3.1.3.3 JS 开发 EEDM7007.js

```

$(function () {
    // 获取左右两棵树的数据
    var resultA = __eiInfo.getBlock('resultA'),
        resultB = __eiInfo.getBlock('resultB'),
        listAData = resultA.getMappedRows(),
        listBData = resultB.getMappedRows();

    // 把 list 数据转化为 tree 数据
    var treeDataA = IPLAT.flat2Map(listAData, 'id', 'pid', 'root');
    var treeDataB = IPLAT.flat2Map(listBData, 'id', 'pid', 'root');

    IPLAT.TreeViews({
        leftId: "result-0-tree1",
        rightId: "result-0-tree2",
        leftDataSource: treeDataA,
        rightDataSource: treeDataB,
        accessAll: false
    });

    // 点击按钮打印出两棵树的 dataSource 集合
    $("#btn").on("click", function () {
        var _tree1 = $("#result-0-tree1").data("kendoTreeView");
        console.log(IPLAT.map2Flat(_tree1.dataSource.data()));
        var _tree2 = $("#result-0-tree2").data("kendoTreeView");
        console.log(IPLAT.map2Flat(_tree2.dataSource.data()));
    });
});

```

js中调用IPLAT.flat2Map(data, idField, parentId, rootLevel)方法得到的treeDataA，格式如下：

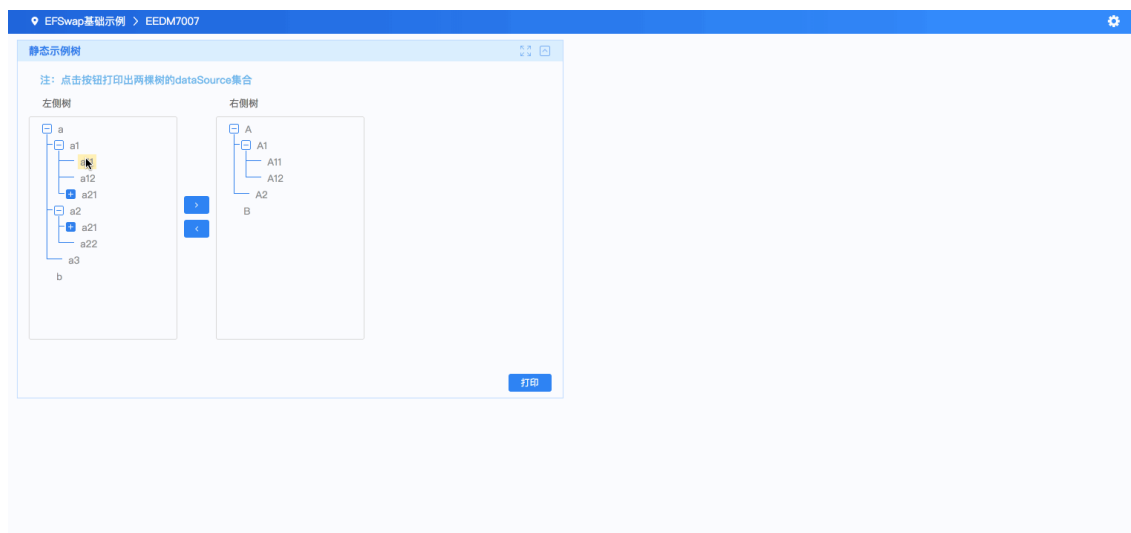
```

> treeDataA
< ▼ (2) [{...}, {...}] ⓘ
  ▼ 0:
    id: "a"
    ▼ items: Array(3)
      ▼ 0:
        id: "a1"
        ▶ items: (3) [{...}, {...}, {...}]
        pid: "a"
        text: "a1"
        ▶ __proto__: Object
      ▶ 1: {pid: "a", text: "a2", id: "a2", items: Array(2)}
      ▶ 2: {pid: "a", text: "a3", id: "a3", items: Array(0)}
      length: 3
      ▶ __proto__: Array(0)
    pid: "root"
    text: "a"
    ▶ __proto__: Object
  ▶ 1: {pid: "root", text: "b", id: "b", items: Array(0)}
    length: 2

```

3.5.4.3.1.3.4 页面展示效果

1. 双击左侧 **a11** 节点(或者选中 **a11** 节点并点击">"按钮), **a11** 节点添加至右侧树中并且左侧不再显示 **a11**
2. 同理选中右侧 **A1** 节点, 则 **A1** 节点被添至左侧树中



3. 点击"打印"按钮, 输出的右侧树数据集合如下:


```

▼ Array(6) ⓘ
  ▶ 0: {pid: "A", text: "A2", id: "A2", index: 1}
  ▶ 1: {pid: "root", text: "A", id: "A", index: 0, expanded: true}
  ▶ 2: {pid: "root", text: "B", id: "B", index: 1}
  ▶ 3: {id: "a11", text: "a11", index: 0}
  ▶ 4: {id: "a1", text: "a1", index: 0, expanded: true}
  ▶ 5: {id: "a", text: "a", index: 2, expanded: true}
    length: 6
  ▶ __proto__: Array(0)

```

3.5.4.3.2 EEDM7500- EFSwap 同 ID 结点示例

3.5.4.3.2.1 页面功能简介

本示例页面用于说明，如何使用 **EFSwap** 标签配置树中的同名节点，并在如下示例中点击“确定”按钮返回两棵树的 list 集合。

3.5.4.3.2.2 配置和 API 详情

参考 [EFSwap](#) 中的描述。

3.5.4.3.2.3 页面开发步骤

3.5.4.3.2.3.1 JSP 页面开发

EEDM7500.jsp

```

<%
    String leftDataSource = "[{id:'root_a',text: '同名结点', items: [{id:'a_a1', text: '同名结点',
items: [{id:'a1_a11', text: '同名结点'}, {id:'a1_a12', text: 'a12'}]}, {id:'a_a2', text:
'a2',items:[{id:'a2_a21',text:'a21',items:[{id:'a21_a211',text:'a211'},{id:'a21_a212',text:'a212'}]},{i
d:'a2_a22',text:'a22'}]}, {id:'a_a3',text:'a3'}]}, {id:'b', text: 'b'}]";

    String rightDataSource = "[{id:'root_A',text: 'A', items: [{id:'A_A1', text: 'A1', items:
[{id:'A1_A11', text: 'A11'}, {id:'A1_A12', text: 'A12'}]}, {id:'A_A2', text: 'A2'}]}, {id:'B', text: 'B'}]";

    request.setAttribute("leftDataSource", leftDataSource);
    request.setAttribute("rightDataSource", rightDataSource);
%>

<EF:EFPage>
  <div class="row">
    <div class="col-md-6">
      <EF:EFRegion title="静态示例树" id="tree">
        <EF:EFSwap blockId="result" row="0" leftName="tree1" rightName="tree2"
immediatelyRender="false"
          leftDataSource="${leftDataSource}" rightDataSource="${rightDataSource}"
height="300"
width="200"/>

```

```

        <div class="row" style="text-align: right; margin: 0; padding-top: 10px;">
            <button id="btn" type="button" class="i-btn-lg i-btn-wide">确定</button>
        </div>
    </EF:EFRegion>
</div>
</div>
</EF:EFPage>

```

3.5.4.3.2.3.2 Service 开发

ServiceEEDM7500.java

```

public class ServiceEEDM7500 extends ServiceEPBase {

    public EiInfo initLoad(EiInfo inInfo) {
        return mockData();
    }

    // 提供模拟数据
    private EiInfo mockData() {
        EiInfo outInfo = new EiInfo();
        String jsonA = "[{text: 'a', label: 'a', pld: 'root'}, {text: 'a1', label: 'a1', pld: 'a'}, {text: '同名节点', label: 'SAME', pld: 'a1'}, {text: '同名节点', label: 'SAME', pld: 'a2'}, {text: 'a2', label: 'a2', pld: 'a'}, {text: 'a21', label: 'a21', pld: 'a2'}, {text: 'a211', label: 'a211', pld: 'a21'}, {text: '同名节点', label: 'SAME', pld: 'a21'}, {text: 'a22', label: 'a22', pld: 'a2'}, {text: 'a3', label: 'a3', pld: 'a'}, {text: '同名节点', label: 'SAME', pld: 'root'}]";
        String jsonB = "[{text: 'A', label: 'A', pld: 'root'}, {text: 'A1', label: 'A1', pld: 'A'}, {text: 'A11', label: 'A11', pld: 'A1'}, {text: 'A12', label: 'A12', pld: 'A1'}, {text: 'A2', label: 'A2', pld: 'A'}, {text: 'B', label: 'B', pld: 'root'}]";

        EiBlock resultA = MockDataUtil.createMockTree("resultA", jsonA);
        EiBlock resultB = MockDataUtil.createMockTree("resultB", jsonB);

        outInfo.addBlock(resultA);
        outInfo.addBlock(resultB);
        return outInfo;
    }
}

```

3.5.4.3.2.3.3 JS 开发

EEDM7500.js

```

$(function () {
    // 获取左右两棵树的数据
    var resultA = __eiInfo.getBlock('resultA'),
        resultB = __eiInfo.getBlock('resultB'),
        listAData = resultA.getMappedRows(),
        listBData = resultB.getMappedRows();

    // 把 list 数据转化为 tree 数据(pld,label 为联合主键)
    var treeDataA = flat2MapX(listAData, 'label', 'pld', 'root', 'pld,label');
    var treeDataB = flat2MapX(listBData, 'label', 'pld', 'root', 'pld,label');

    function flat2MapX(data, idField, parentId, rootLevel, primaryKey) {
        var map = {};
        if (!$.isArray(data)) {
            return map;
        }

        for (var i = 0; i < data.length; i++) {
            var item = data[i];
            var id = item[idField];
            var pid = item[parentId];

            map[id] = map[id] || [];
            map[pid] = map[pid] || [];

            item.items = map[id]; // schema model 中的 children: "items"

            // Diff with IPLAT.flat2Map(data, idField, parentId, rootLevel)
            if (!_isEmpty(primaryKey)) { // 联合主键
                var PKSplit = primaryKey.split(",");
                var compositedId = item[PKSplit[0]];
                PKSplit.shift();
                _.each(PKSplit, function (key) {
                    compositedId += "_" + item[key];
                });
                item['id'] = compositedId;
            }

            map[pid].push(item);
        }
    }
}

```

```

        return map[rootLevel];
    }

    IPLAT.TreeViews({
        leftId: "result-0-tree1",
        rightId: "result-0-tree2",
        leftDataSource: treeDataA,
        rightDataSource: treeDataB,
        accessAll: true
    });

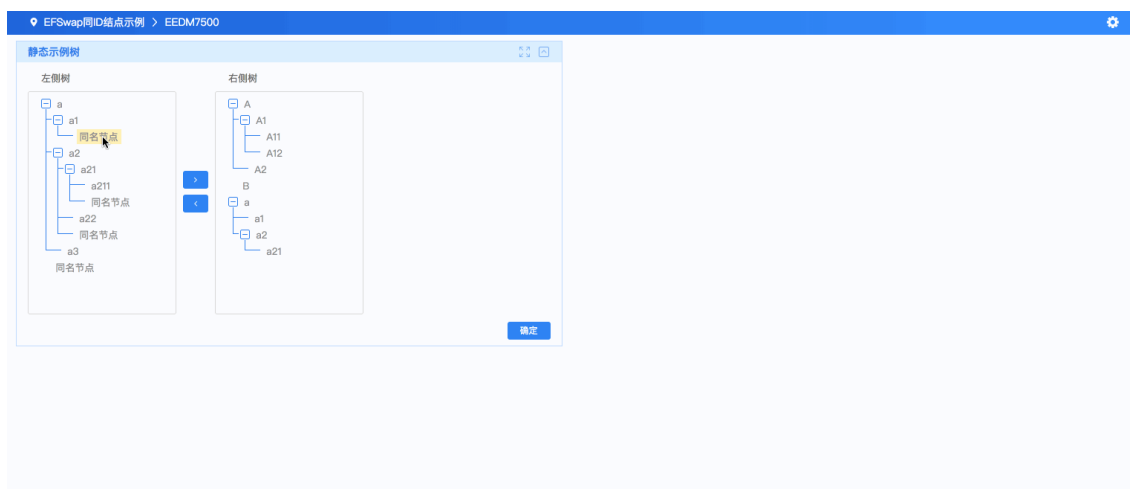
    $("#btn").on("click", function () {
        var _tree1 = $("#result-0-tree1").data("kendoTreeView");
        console.log(IPLAT.map2Flat(_tree1.dataSource.data()));
        var _tree2 = $("#result-0-tree2").data("kendoTreeView");
        console.log(IPLAT.map2Flat(_tree2.dataSource.data()));
    });

});

```

3.5.4.3.2.3.4 页面展示效果

1. 双击左侧“同名节点”节点(或者选中“同名节点”节点并点击">"按钮)，与“同名节点”相同 id 的节点将被添加至右侧树中，并且左侧不再显示 id 为“同名节点”的节点。
2. 同理选中右侧“同名节点”节点，则右侧 id 为“同名节点”的节点都被添至左侧树中。



3. 点击"确定"按钮，输出的左侧树数据集合如下：

```

▼ (11) [{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}] ⓘ
  ▶ 0: {id: "a1_SAME", text: "同名节点", index: 0}
  ▶ 1: {pId: "a", text: "a1", label: "a1", id: "a_a1", index: 0, ...}
  ▶ 2: {pId: "a21", text: "a211", label: "a211", id: "a21_a211", index: 0}
  ▶ 3: {id: "a21_SAME", text: "同名节点", index: 1}
  ▶ 4: {pId: "a2", text: "a21", label: "a21", id: "a2_a21", index: 1, ...}
  ▶ 5: {pId: "a2", text: "a22", label: "a22", id: "a2_a22", index: 2}
  ▶ 6: {id: "a2_SAME", text: "同名节点", index: 2}
  ▶ 7: {pId: "a", text: "a2", label: "a2", id: "a_a2", index: 1, ...}
  ▶ 8: {pId: "a", text: "a3", label: "a3", id: "a_a3", index: 2}
  ▶ 9: {pId: "root", text: "a", label: "a", id: "root_a", index: 0, ...}
  ▶ 10: {id: "root_SAME", text: "同名节点", index: 1}
  length: 11
  ▶ __proto__: Array(0)

```

3.5.5 基础部件

3.5.5.1 EEDM1030-Alert 组件示例

3.5.5.1.1 页面功能简介

本示例页面用于说明如何使用 IPLAT.alert 警告对话框组件。

3.5.5.1.2 配置和 API 详情

参考 [IPLAT.alert 组件](#) 中的描述。

可接收 **object** 与 **string** 这两种参数。

3.5.5.1.3 页面开发步骤

3.5.5.1.3.1 JSP 页面开发

EEDM1030.jsp

```

<EF:EFRegion title="普通 alert 示例" id="common">
  <div class="row">
    <div class="col-md-3">
      <EF:EFButton ename="inqu_status-0-btn" cname="IPLAT.alert(string)" layout="3"
uri="css:fa-exclamation-triangle">
      </EF:EFButton>
    </div>

    <div class="col-md-3">
      <EF:EFButton ename="inqu_status-1-btn" cname="IPLAT.alert(object)" layout="3"
uri="css:fa-exclamation-triangle">
      </EF:EFButton>
    </div>
  </div>

```

```

</EF:EFRegion>

<EF:EFRegion title="自定义宽度的 alert 示例" id="customWidth">
  <div class="row">
    <div class="col-md-3">
      <EF:EFButton ename="inqu_status-2-btn" cname="IPLAT.alert(string) (minWidth:
350)" layout="3" uri="css:fa-exclamation-triangle">
        </EF:EFButton>
      </div>

      <div class="col-md-3">
        <EF:EFButton ename="inqu_status-3-btn" cname="IPLAT.alert(object) (minWidth:
400)" layout="3" uri="css:fa-exclamation-triangle">
          </EF:EFButton>
        </div>
      </div>
    </EF:EFRegion>

```

3.5.5.1.3.2 JS 开发

EEDM1030.js

```

$(function () {
  $('#inqu_status-0-btn').on('click', function () {
    IPLAT.alert('<b>警告：该操作可能影响表结构!</b>', function(e) {}, ' '); // title 为"或空格
    时，取 config.js 中的默认值
  });

  $('#inqu_status-1-btn').on('click', function () {
    IPLAT.alert({
      message: '<b>警告：该操作可能影响表结构!</b>',
      okFn: function (e) {},
      title: '警告框自定义 title'
    });
  });

  $('#inqu_status-2-btn').on('click', function () {
    IPLAT.alert("&<b>警告：该操作可能影响表结构!</b>", // message
      function(e) {}, // okFn
      "警告框自定义 title", // title
      350 // minWidth
    );
  });
});

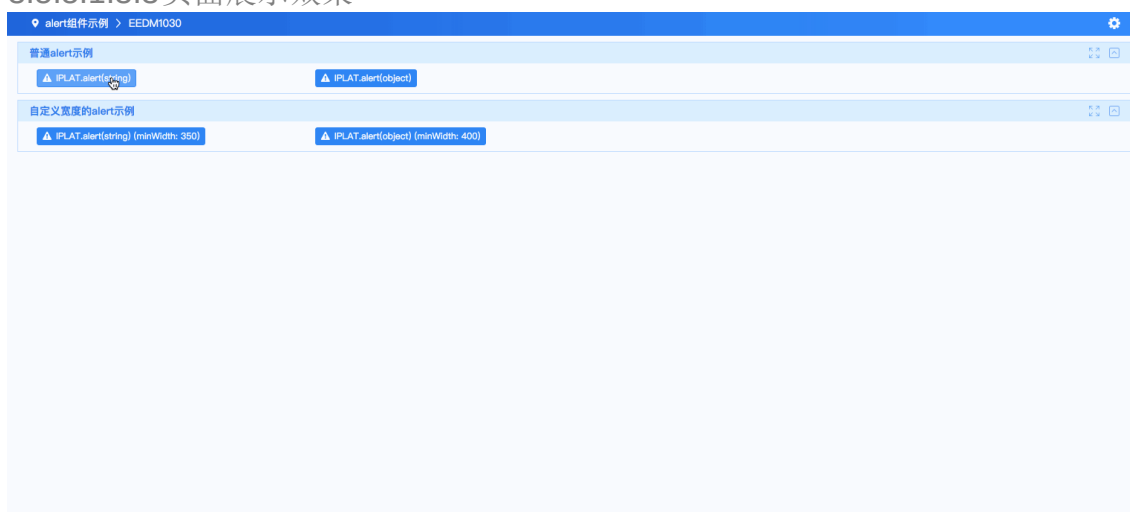
```

```

$('#inqu_status-3-btn').on('click', function () {
    IPLAT.alert({
        message: '<b>警告：该操作可能影响表结构!</b>',
        okFn: function (e) {},
        title: '警告框自定义 title',
        minWidth: 400
    });
});
});

```

3.5.5.1.3.3 页面展示效果



3.5.5.2 EEDM1031-Confirm 组件示例

3.5.5.2.1 页面功能简介

本示例页面用于说明如何使用 `IPLAT.confirm` 确定对话框组件。

3.5.5.2.2 配置和 API 详情

参考 [IPLAT.confirm 组件](#) 中的描述。

可接收 **object** 与 **string** 这两种参数。

3.5.5.2.3 页面开发步骤

3.5.5.2.3.1 JSP 页面开发

EEDM1030.jsp

```

<EF:EFRegion title="普通 confirm 示例" id="common">
    <div class="row">

```

```

<div class="col-md-3">
  <EF:EButton ename="inqu_status-0-btn" cname="IPLAT.confirm(string)"
layout="3" uri="css:fa-check-circle">
    </EF:EButton>
</div>

<div class="col-md-3">
  <EF:EButton ename="inqu_status-1-btn" cname="IPLAT.confirm(object)"
layout="3" uri="css:fa-check-circle">
    </EF:EButton>
</div>

<div class="col-md-3">
  <EF:EButton ename="inqu_status-2-btn" cname="IPLAT.confirm(object)+返回取消"
layout="3" uri="css:fa-search">
    </EF:EButton>
</div>

<div class="col-md-3">
  <EF:EButton ename="inqu_status-3-btn" cname="IPLAT.confirm(string)+返回标题"
layout="3" uri="css:fa-search">
    </EF:EButton>
</div>
</EF:ERegion>

<EF:ERegion title="自定义宽度的 confirm 示例" id="customWidth">
  <div class="row">
    <div class="col-md-6">
      <EF:EButton ename="inqu_status-4-btn" cname="IPLAT.confirm(object)+返回标题
(minWidth: 450)" layout="3" uri="css:fa-search">
        </EF:EButton>
      </div>
    </div>
  </EF:ERegion>

```

3.5.5.2.3.2JS 开发

EEDM1030.js

```

$(function () {
  $('#inqu_status-0-btn').on('click', function () {
    IPLAT.confirm(

```



```

        '<b>确定执行该操作吗? </b> \n\n <i style="font-size: 12px;">注: 请优先通知管理
        员。</i>', // message
        function (e) { IPLAT.NotificationUtil('确定该操作'); }, // okFn
        function (e) {}, // cancelFn
        ' ' // title, title 为"或空格时, 取 config.js 中的默认值
    );
});

$('#inqu_status-1-btn').on('click', function () {
    IPLAT.confirm({
        message: '<b>确定执行该操作吗? </b> \n\n <i style="font-size: 12px">注: 请优先通
        知管理员。</i>',
        okFn: function (e) { IPLAT.NotificationUtil('确定该操作'); },
        cancelFn: function (e) {}
    });
});

$('#inqu_status-2-btn').on('click', function () {
    IPLAT.confirm({
        message: '<b>确定执行该操作吗? </b> \n\n <i style="font-size: 12px">注: 请优先通
        知管理员。</i>',
        okFn: function (e) { IPLAT.NotificationUtil('确定该操作'); },
        cancelFn: function (e) { IPLAT.NotificationUtil('取消该操作'); }
    });
});

$('#inqu_status-3-btn').on('click', function () {
    IPLAT.confirm(
        '<b>确定执行该操作吗? </b> \n\n <i style="font-size: 12px">注: 请优先通知管理
        员。</i>',
        function (e) { IPLAT.NotificationUtil('确定该操作'); },
        function (e) { IPLAT.NotificationUtil('取消该操作'); },
        '确认对话框自定义 title'
    );
});

$('#inqu_status-4-btn').on('click', function () {
    IPLAT.confirm({
        message: '<b>确定执行该操作吗? </b> \n\n <i style="font-size: 12px">注: 请优先通
        知管理员。</i>',
        okFn: function (e) { IPLAT.NotificationUtil('确定该操作'); },
        cancelFn: function (e) { IPLAT.NotificationUtil('取消该操作'); },
        title: '确认对话框自定义 title',
    });
});

```

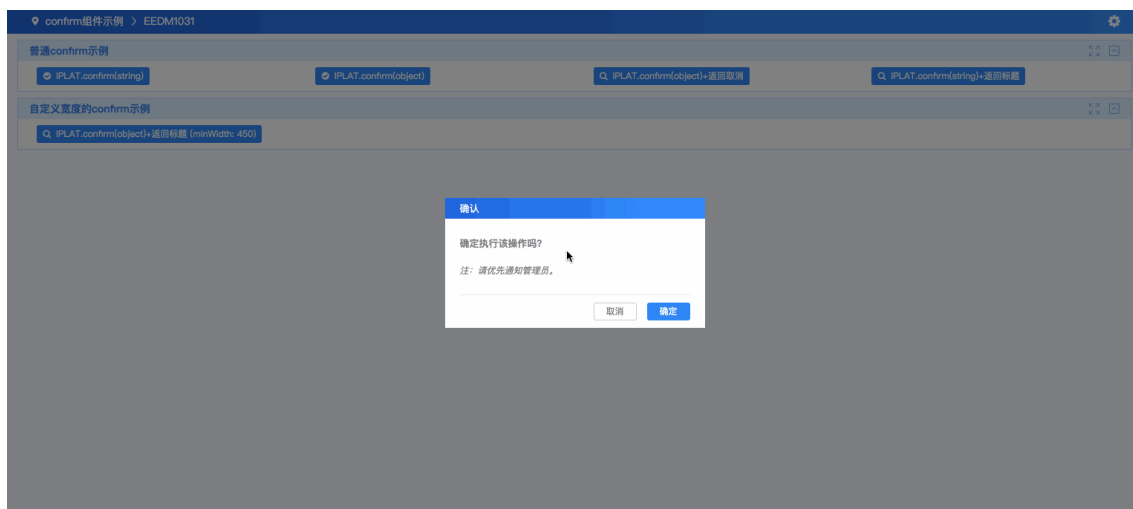
```

        minWidth: 450
    });
});
});

```

3.5.5.2.3.3 页面展示效果

1. 弹出对话框；
2. Notification 组件显示已确认该操作。



3.5.5.3 EEDM1032-Prompt 组件示例

3.5.5.3.1 页面功能简介

本示例页面用于说明如何使用 `IPLAT.prompt` 输入对话框组件。

3.5.5.3.2 配置和 API 详情

参考 [IPLAT.prompt 组件](#) 中的描述。

可接收 **object** 与 **string** 这两种参数。

3.5.5.3.3 页面开发步骤

3.5.5.3.3.1 JSP 页面开发

EEDM1030.jsp

```

<EF:EFRegion title="普通 prompt 示例" id="common">
    <div class="row">
        <div class="col-md-4">

```

```

        <EF:EButton ename="inqu_status-1-btn" cname="IPLAT.prompt(object)+返回输入
值" layout="3" uri="css:fa-search">
        </EF:EButton>
    </div>

    <div class="col-md-4">
        <EF:EButton ename="inqu_status-2-btn" cname="IPLAT.prompt(string)+返回输入值
+标题" layout="3"
            uri="css:fa-search">
        </EF:EButton>
    </div>
</div>
</EF:EFRegion>

<EF:EFRegion title="自定义宽度的 confirm 示例" id="customWidth">
    <div class="row">
        <div class="col-md-6">
            <EF:EButton ename="inqu_status-3-btn" cname="IPLAT.prompt(object)+返回输入
值+标题 (minWidth: 450)" layout="3"
                uri="css:fa-search">
            </EF:EButton>
        </div>
    </div>
</EF:EFRegion>

```

3.5.5.3.3.2JS 开发

EEDM1030.js

```

$(function () {
    $('#inqu_status-1-btn').on('click', function () {
        IPLAT.prompt({
            message: '请输入:',
            okFn: function (e) { IPLAT.NotificationUtil('输入的值为: ' + e); },
            cancelFn: function (e) {}
        });
    });

    $('#inqu_status-2-btn').on('click', function () {
        IPLAT.prompt(
            '请输入:', // message
            function (e) { IPLAT.NotificationUtil('输入的值为: ' + e); }, // okFn
            function (e) {}, // cancelFn
        );
    });

```

```

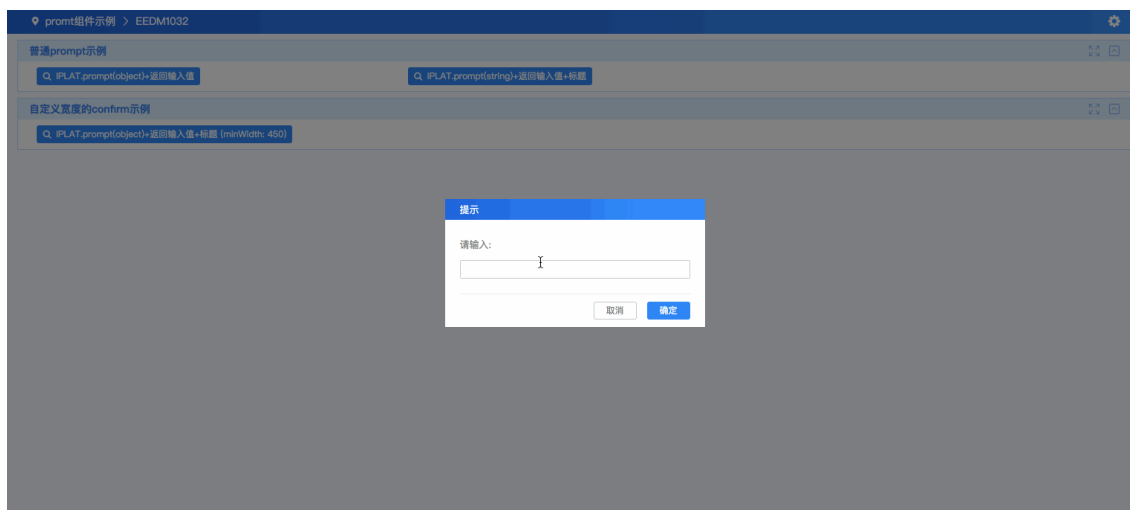
        '输入对话框自定义 title' // title
    );
});

$('#inqu_status-3-btn').on('click', function () {
    IPLAT.prompt({
        message: '请输入:',
        okFn: function (e) { IPLAT.NotificationUtil('输入的值: ' + e); },
        cancelFn: function (e) {},
        title: '输入对话框自定义 title',
        minWidth: 450
    });
});
});

```

3.5.5.3.3 页面展示效果

1. 弹出对话框；
2. 输入值，点击确定；
3. Notification 组件显示返回的值。



3.5.6 How-To 定制方案

3.5.6.1 EEDM9000-单列多行输入

针对单列多行输入场景，如大批量单列输入框。

3.5.6.2 EEDM9001-多行多列输入

针对多行多列输入场景，如大批量多行多列输入框。

3.5.6.3 EEDM9002-单表查询

针对单表查询一类场景的布局，上方查询条件，下方查询结果。

3.5.6.4 EEDM9003-多行多列布局

针对多个数据区域场景，可以上下或者左右划分数据区域。

3.5.6.5 EEDM9004-上中下三行布局

上中下三行布局，上方查询条件，中间查询结果，下方第三块为明细，可维护（主从）。

3.5.6.6 EEDM9005-上中下+Tab

上中下+tab 上方查询条件,中间单条明细,下方多 tab 页（tab 里可能是单条明细，也可能是多条结果）。

3.5.6.7 EEDM9006-树表布局

树表布局，上面查询条件，下方左边多记录（可以是树），右边多记录或单条明细（与左边主从）。

3.5.6.8 EFGrid 里面使用 JS 去实现下拉列

3.5.6.8.1 页面功能简介

此页面用于说明 EFGrid 里的下拉框需要根据行中的某项值动态展示下拉选项的问题。

3.5.6.8.2 配置和 API 详情

参考 [EFGrid 表格](#) 中的描述。

3.5.6.8.3 页面开发步骤

3.5.6.8.3.1 JSP 页面开发

重点关注两列，company_name 和 itemA，其中 company_name 是我们的动态条件，itemA 需要根据这个条件动态展示下拉选项。

JSP 示例

```
<EF:EFRegion id="result" title="记录集">
    .....

    <EF:EFColumn ename="company_name" locked="true" cname="公司名称"
required="true"/>
    .....
    <EF:EFColumn ename="itemA" cname="List 选项"/>
    .....
```

```
</EF:EFGrid>
</EF:EFRegion>
```

3.5.6.8.3.2JS 开发

在 js 里面，通过 itemA 的 editor 事件，动态获取到 company_name 这一列的值，将这个值当做查询条件由 Ajax 传到后台查询并返回一个 EiInfo,将我们需要的数据在下拉框里渲染。

JS

```
$(function () {

    var globalData = [];

    .....

    IPLATUI.EFGrid = {
        "result": {
            columns: [{

                .....

                {
                    field: "itemA",
                    template: function(dataItem) {
                        for (var i = 0 ; i < globalData.length ; i++) {
                            if ( globalData[i]['product_id'] === dataItem['itemA']) {
                                return globalData[i]['product_id'];
                            }
                        }
                    }
                }
            ]
        },
        editor: function (container, options) {
            var grid = container.closest(".k-grid").data("kendoGrid");
            var cellIndex = grid.cellIndex(container);
            var input = $('<input />');
            input.attr("name", options.field);
            input.attr("id", options.field);
            input.appendTo(container);

            var eiInfo = new EiInfo();
```

```

        eiInfo.set("inqu_status-0-company_name",
options.model["company_name"]);
        var dataSource;

        EiCommunicator.send("EEDM6000", "query", eiInfo, {
            onSuccess: function (ei) {
                dataSource = ei.getBlock("result").getMappedRows();
                globalData = dataSource;
            }, onFailure: function (ei) {
            }
        }, {async: false});

        input.kendoDropDownList({
            valuePrimitive: true,
            dataTextField: "company_name",
            dataValueField: "product_id",
            dataSource: dataSource,
            template: "#=product_id#-#=company_name#"
        });

        .....

    }

}

],

.....

}

}

};

});

```

我们定义一个 EiInfo，通过 options.model["company_name"] 取到条件值，放到“inqu_status”里面。

```

var eiInfo = new EiInfo();
eiInfo.set("inqu_status-0-company_name",
options.model["company_name"]);

```

通过 Ajax，将带有查询条件的 EiInfo 传入对应的 Service，调用 query 方法，取到我们想要的值，回调给前台，这个回调值就保存在“ei”里面。

将回调值 `ei` 转换成 `json` 数组，此时这个 `json` 数组就是我们所需要的数据源 `dataSource`，其中 `globalData` 是一个全局变量。

```
EiCommunicator.send("EEDM6000", "query", eiInfo, {
    onSuccess: function (ei) {
        dataSource = ei.getBlock("result").getMappedRows();
        globalData = dataSource;
    }, onFailure: function (ei) {
    }
}, {async: false});
```

将 `datasource` 配置进 `kendoDropDownList`，其中 `dataTextField` 是显示值，`dataValueField` 是真实值 `template` 为下拉框里展示的数据结构。

```
input.kendoDropDownList({
    valuePrimitive: true,
    dataTextField: "company_name",
    dataValueField: "product_id",
    dataSource: dataSource,
    template: "#=product_id#-#=company_name#"
});
```

最后配置 `columns` 下的 `template` 用来展示回填值，这里我们采用 `for` 循环的方法，简单点说，就是取到我们下拉的点击项(值为真实值 `dataValueField`)，循环遍历 `dataSource` 里的各项，定位到我们所点击的那一项，取出想要展示的值回填到 `input` 框。这里我们回填了 `product_id`。

```
template: function(dataItem) {
    for (var i = 0 ; i < globalData.length ; i++) {
        if ( globalData[i]['product_id'] === dataItem['itemA']) {
            return globalData[i]['product_id'];
        }
    }
    return "";
}
```

3.5.6.8.3.3 页面效果

此时我们点击 `N701-NOKIA` 这项。

* 产品编号	* 商家公司名	产地	List选项	* 小代码选项	* 时刻	* 上市日期	价格	搜索
T6181	公司: NOKIA	China222	NOKIA	http://platform.baosig...			¥0.12	查询
SGH-i7161	公司: Motorola	Canada	T6181-NOKIA	http://www.samsun...	20170711 01:45:00		¥2.00	查询
T6281	公司: NOKIA	China-中国,German...	T6281-NOKIA	http://www.sonyeric...	20170711 01:00:00		¥30.00	查询
N701	公司: NOKIA	China-中国,German...	N701-NOKIA				¥0.12	search
88501	公司: NOKIA	China-中国	88501-NOKIA		20170711 01:00:00		¥0.12	1
1112	公司: NOKIA	China-中国,USA-美...	1112-NOKIA	EWFM07	20170711 01:00:00		¥11.00	
S891	公司: NOKIA	China-中国	S891-NOKIA	http://www.philips.c...	20170711 00:15:00		¥1,200.50	search
8825	公司: NOKIA	China-中国	8825-NOKIA		20170711 01:00:00		¥20,000.00	search
1234	公司: Motorola	Canada	23333-NOKIA		20170704 00:30:00		¥2.00	
1231	公司: Motorola	Canada	sccc-NOKIA				¥2.00	

由 `return globalData[i]['product_id'];` 回填代码可知，我们回填了 `product_id`，即 N701，如图所示。

T6181	公司: NOKIA	China222	N701	http://platform.baosig...			¥0.12	查询
-------	-----------	----------	------	---------------------------	--	--	-------	----

当然我们点击不同列，会展示不同的下拉选项，由此，开发人员只需要修改条件项即可动态展示下拉选项内容。

* 产品编号	* 商家公司名	产地	List选项	* 小代码选项	* 时刻	* 上市日期	价格	搜索
T6181	公司: NOKIA	China222	N701	http://platform.baosig...			¥0.12	查询
SGH-i7161	公司: Motorola	Canada		http://www.samsun...	20170711 01:45:00		¥2.00	查询
T6281	公司: NOKIA	China-中国,German...	SGH-i7161-Motorola	http://www.sonyeric...	20170711 01:00:00		¥30.00	查询
N701	公司: NOKIA	China-中国,German...	1234-Motorola		20170711 01:00:00		¥0.12	search
88501	公司: NOKIA	China-中国	1231-Motorola		20170711 01:00:00		¥0.12	1
1112	公司: NOKIA	China-中国,USA-美...	1111-Motorola	EWFM07	20170711 01:00:00		¥11.00	
S891	公司: NOKIA	China-中国	XXX1-Motorola	http://www.philips.c...	20170711 00:15:00		¥1,200.50	search
8825	公司: NOKIA	China-中国			20170711 01:00:00		¥20,000.00	search
1234	公司: Motorola	Canada			20170704 00:30:00		¥2.00	
1231	公司: Motorola	Canada					¥2.00	

3.5.6.8.3.4 构造 dataSource 的其他方式

以上方法为根据后台查询的数据动态展示下拉框的值，此时的 `dataSource` 为后台传来的 `EiInfo` 转换成的 `json` 数组。当然，我们也可以用其他方法来构建我们所需要的 `dataSource`，下面就简单介绍一下。

3.5.6.8.3.4.1 第一种：将 request 域里的数据作为下拉数据源

```
List<Map> customList = new ArrayList<>();

for (int i = 0; i < 8; i++) {
    Map<String, String> map = new HashMap<>();
    map.put("name", "item" + i);
    map.put("desc", "选项" + i);
    customList.add(map);
}
request.setAttribute("list", customList);
```

我们自定义一个 List 集合，将集合加入 request 域对象，在 jsp 里面使用 EFComboColumn 标签里的 sourceName 属性来配置数据源。

```
<%-- 使用List<Map>作为下拉数据源 --%>
<EF:EFComboColumn ename="items" cname="List选项"
itemTemplate="#=name#-#=desc#"
required="true" sourceName="list" textField="desc" valueField="name"/>
```

页面效果：

产品编号	厂商公司名	产地	List选项	小代码选项	时刻	上市日期	价格	搜索
T6181	公司: NOKIA	China222	item0-选项0	http://platform.baosight.com			¥0.12	查询
SGH-i7161	公司: Motorola	Canada	item0-选项0	http://www.samsung.com	20170711 01:45:00		¥2.00	查询
T6281	公司: NOKIA	China-中国, Germa...	item1-选项1	http://www.sonyericsson.com	20170711 01:00:00		¥30.00	查询
N701	公司: NOKIA	China-中国, Germa...	item2-选项2				¥0.12	search
88501	公司: NOKIA	China-中国	item3-选项3		20170711 01:00:00		¥0.12	1
1112	公司: NOKIA	China-中国, USA-美...	item4-选项4	EWFM07	20170711 01:00:00		¥11.00	
5891	公司: NOKIA	China-中国	item5-选项5	http://www.philips.com	20170711 00:15:00		¥1,200.50	search
8825	公司: NOKIA	China-中国	item6-选项6		20170711 01:00:00		¥20,000.00	search
1234	公司: Motorola	Canada	item7-选项7		20170704 00:30:00		¥2.00	
1231	公司: Motorola	Canada					¥2.00	

3.5.6.8.3.4.2 第二种：通过小代码来配置数据源

在 jsp 里配置小代码，其中 itemTemplate 为下拉选项的模版信息，columnTemplate 为单元格的显示模版，非编辑状态时的显示，类似 3.9 版本的 formatString。

```
<%-- 使用小代码作为下拉数据源 --%>
<EF:EFComboColumn ename="company_page" cname="小代码选项"
columnTemplate="#=textField#-#=valueField#"
itemTemplate="#=textField#-#=valueField#"
textField="textField" valueField="valueField">
  <EF:EFOption label="全部" value="all"/>
  <EF:EFCCodeOption codeName="iplat.ed.resType"/>
  <EF:EFCCodeOption codeName="iplat.codeStyle"/>
</EF:EFComboColumn>
```

页面效果：

产品编号	厂商公司名	产地	List选项	小代码选项	时刻	上市日期	价格	搜索
T6181	公司: NOKIA	China222	选项0	GIT-0			¥0.12	查询
SGH-i7161	公司: Motorola	Canada	全部-all		20170711 01:45:00		¥2.00	查询
T6281	公司: NOKIA	China-中国, Germa...	✓ GIT-0		20170711 01:00:00		¥30.00	查询
N701	公司: NOKIA	China-中国, Germa...	SVN-1				¥0.12	search
88501	公司: NOKIA	China-中国	统计类-caculate		20170711 01:00:00		¥0.12	1
1112	公司: NOKIA	China-中国, USA-美...	流程类-workflow		20170711 01:00:00		¥11.00	
5891	公司: NOKIA	China-中国		http://www.philips.com	20170711 00:15:00		¥1,200.50	search
8825	公司: NOKIA	China-中国			20170711 01:00:00		¥20,000.00	search
1234	公司: Motorola	Canada			20170704 00:30:00		¥2.00	
1231	公司: Motorola	Canada					¥2.00	

3.5.6.8.3.4.3 第三种：在 js 里自己拼装数据源

这种方式是在 js 里自己任意拼装所需的数据源方式，例如 dataSource：
[{ name: "Jane Doe", age: 30 }, { name: "John Doe", age: 33 }], 这种方式与前端数据展示框一样，需要手动将 dataSource 配置在 kendoDropDownList 里面

3.5.6.9 首页菜单的配置

菜单的配置在 EDPI10 菜单信息管理里面进行配置的。

3.5.6.9.1 配置根节点

首页菜单的根节点在 EDCC03 中配置 menuRoot。



3.5.6.9.2 菜单的连接图标，参数的含义和配置

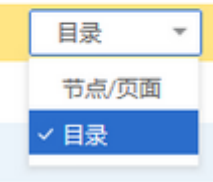
在系统菜单的页面，我们可以看到系统菜单树的节点信息与我们系统平台首页显示的信息是一致的。





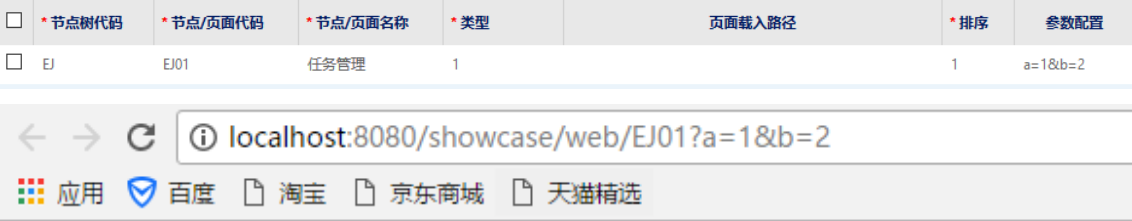
对于每个节点配置项，“节点树代码”为父节点，“节点/页面代码”为子节点，其中顶层的“节点树代码”为根节点。

“节点/页面名称”为显示的节点名称，在类型里面配置的是是否为目录节点，即是否可以点击  进行展示。



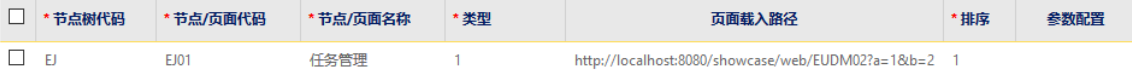
排序为节点展示顺序，默认从上至下按大小排列，数字越小，，排列越靠上。
页面载入路径只有叶子结点配置有效，点击叶子结点打开新页面。

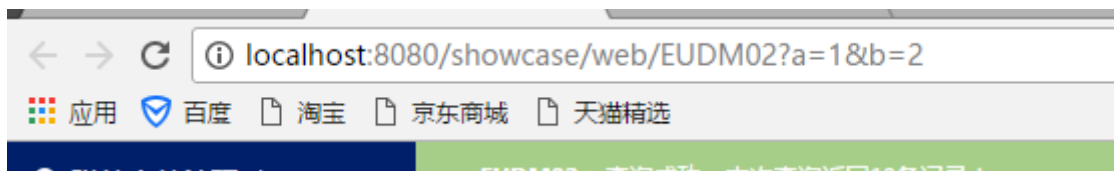
参数配置里配置字符串参数，如“a=1”,配置多个参数需加“&”连接，如“a=1&b=2”。此项配置用于打开叶子结点页面时在 url 上拼接参数设置。如图所示：



图标路径为节点前显示的图标项。

需要注意的是，当已经配置了页面载入路径项后，再进行参数设置是无效的，此时应该把参数直接拼接在载入路径。如图所示：





3.5.6.10 editHelper-可编辑的单元格突出显示

本文档用于说明全局配置 `editHelper` 属性的作用与效果。

在 `ui` 的全局配置文件 `iplat.ui.config.js` 中，我们在 `EFGGrid` 里配置了 `editHelper` 这个属性。

```
editHelper: true, // 勾选时，可编辑的单元格突出显示
```

此属性是作用于 `grid` 的单元格中，用来突显编辑状态的单元格的样式。如图：

产品代号	厂商公司名	价格	Google查...	公司首页	上市日期	验证字段0	验证字段1	产地	验证字段2	验证字段
11112	2222	0		http://platform.baosight.cc	20170101123423	13800000000	aaa@123.com	112		
T6181	NOKIA	0.12	查询	http://platform.baosight.cc	20170101123423	13800000000	aaa@123.com	China2222222	https://timgsa.bj	021-50801
SGH-i7161	Motorola	2	查询	http://www.samsung.com	2017/07/11 01:04	13800000000	aaa@123.com	Canada	https://timgsa.bj	021-50801
T6281	NOKIA	30	查询	http://www.sonyericsson.co	2017/07/11 01:0	13800000000	aaa@123.com	China,German,USA,GB	https://timgsa.bj	021-50801
N701	NOKIA	0.12	search		2017-07-11		abc@baosight.cc	China,German	https://timgsa.bj	
88501	NOKIA	0.13	1	http://platform.baosight.cc	20170101123423		abc@baosight.cc	China	https://ss0.bdsta	
11112	NOKIA	11		EWFM07	2017/07/11 01:0		abc@baosight.cc	China,USA,GB	https://ss0.bdsta	-12345678
S891	NOKIA	1200.5	search	http://www.philips.com	2017/07/11 00:1	13800000000	aaa@123.com	China	https://timgsa.bj	021-50801
8825	NOKIA	20000	search		2017/07/11 01:0		abc@baosight.cc	China	https://timgsa.bj	021-50801
T666	NOKIA	0.12	查询	http://platform.baosight.cc	20170101123423	13800000000	abc@baosight.com	china	https://timgsa.bj	021-50801

将 `editHelper` 设置为 `false`，取消此功能，如图：

产品代号	厂商公司名	价格	Google查...	公司首页	上市日期	验证字段0	验证字段1	产地	验证字段2	验证字段
11112	2222	0		http://platform.baosight.cc	20170101123423	13800000000	aaa@123.com	112		
T6181	NOKIA	0.12	查询	http://platform.baosight.cc	20170101123423	13800000000	aaa@123.com	China2222222	https://timgsa.bj	021-50801
SGH-i7161	Motorola	2	查询	http://www.samsung.co...	2017/07/11 01:04	13800000000	aaa@123.com	Canada	https://timgsa.bj	021-50801
T6281	NOKIA	30	查询	http://www.sonyericsson...	2017/07/11 01:0	13800000000	aaa@123.com	China,German,USA,GB	https://timgsa.bj	021-50801
N701	NOKIA	0.12	search		2017-07-11		abc@baosigh...	China,German	https://timgsa.bj	
88501	NOKIA	0.13	1	http://platform.baosight...	20170101123423		abc@baosigh...	China	https://ss0.bd...	
11112	NOKIA	11		EWFM07	2017/07/11 01:0		abc@baosigh...	China,USA,GB	https://ss0.bd...	-12345678
S891	NOKIA	1200.5	search	http://www.philips.com	2017/07/11 00:1	13800000000	aaa@123.com	China	https://timgsa.bj	021-50801
8825	NOKIA	20000	search		2017/07/11 01:0		abc@baosigh...	China	https://timgsa.bj	021-50801
T666	NOKIA	0.12	查询	http://platform.baosight...	20170101123423	13800000000	aaa@123.com	china	https://timgsa.bj	021-50801

可以看出，配置 `editHelper="true"` 的时候，编辑状态的单元格会呈现描边状态，效果更为明显。

对项目组而言，`iplat.ui.config.js` 是不可见的，需要修改 `head.jsp`，参考 [UI-常见开发问题 FAQ](#) 中的问题 Q4。

可编辑框的提示

```
IPLATUI.Config.EFGGrid.editHelper = false; // 默认是 true，勾选时，可编辑的单元格突出显示。
设置为 false 时，不提示
```

3.5.6.10.1 如何在 editHelper 新增列中展示描边状态的解决方案

开发中，**新增**的数据行中配置 `readonly = true` 这个属性，默认是不展示描边状态，但是可以进行编辑处理。这样对用户展示是不友好的，下面是针对这种情况给出的处理方案，有描边状态又可以编辑处理。

```
.....
columns: [
  .....
  // 对 column 进行配置
  {
    field: 'company_name', // 列名
    title: '覆盖公司名',
    //设置后面的 template 之前，请先确认 tag 中 readonly = true 是否存在
    template: function (model) {
      // editDiv 表示 editHelper 编辑时候展示模版 #:company_name# 中需要根据不同的列名
      进行变化
      var editDiv = "<div class='i-validate-helper i-edit-
      helper'>#:company_name#</div>",
      // value 表示值
      value = model['company_name'] || "";
      //model.isNew() === true 表示新增 调用 editHelper 的展示模版， 否则就直接返回值
      if (model.isNew()) {
        return kendo.template(editDiv)(model);
      } else {
        return value;
      }
    },
    width: 200
  }
  .....
],
.....
```

3.5.6.11 EUDM02-url 拼接参数动态显示 grid 的文件列表

本文档用于说明如何通过 url 的参数动态显示我们所需的列表内容，此示例是用于文件上传下载页面。

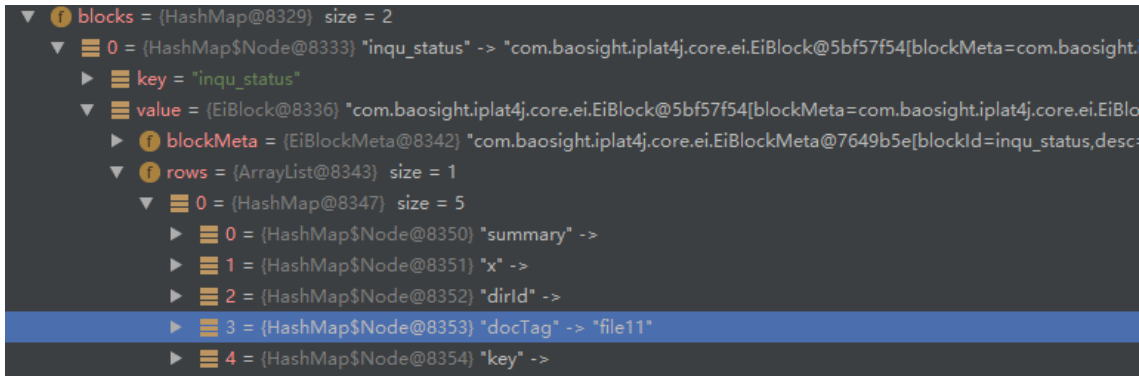
3.5.6.11.1 jsp 页面开发

在 jsp 中，我们增加一个隐藏框，name 与 id 为三段式查询条件，如“`inqu_status-0-docTag`”。

jsp

```
<input type="hidden" name="inqu_status-0-docTag" id="inqu_status-0-docTag" />
```

此参数会传值到后台，进行 query 查询，并返回所需的 EiInfo。



3.5.6.11.2 js 开发

在 js 里面，首先我们通过 GetQueryString() 取到 url 里的所需参数值，例如“docTag”，判断此参数是否存在。

js

```
function GetQueryString(name)
{
    var reg = new RegExp("(^|&)" + name + "=(^[&]*)(&|$)");
    var r = window.location.search.substr(1).match(reg);
    if(r!=null)return unescape(r[2]); return null;
}
var docTag = GetQueryString("docTag");
if(!((docTag == "") || (docTag == null))) {
    $("#tree").closest(".col-md-3").hide();
    $("#inqu").hide();
    $(".col-md-9").attr("class", "col-md-12");
}
$("#inqu_status-0-docTag").val(docTag);
```

当 docTag 存在且不为空的时候，我们将除附件信息 grid 以外的模块隐藏。

```
$("#tree").closest(".col-md-3").hide(); //将左边tree隐藏
$("#inqu").hide(); // 将查询区域隐藏
```

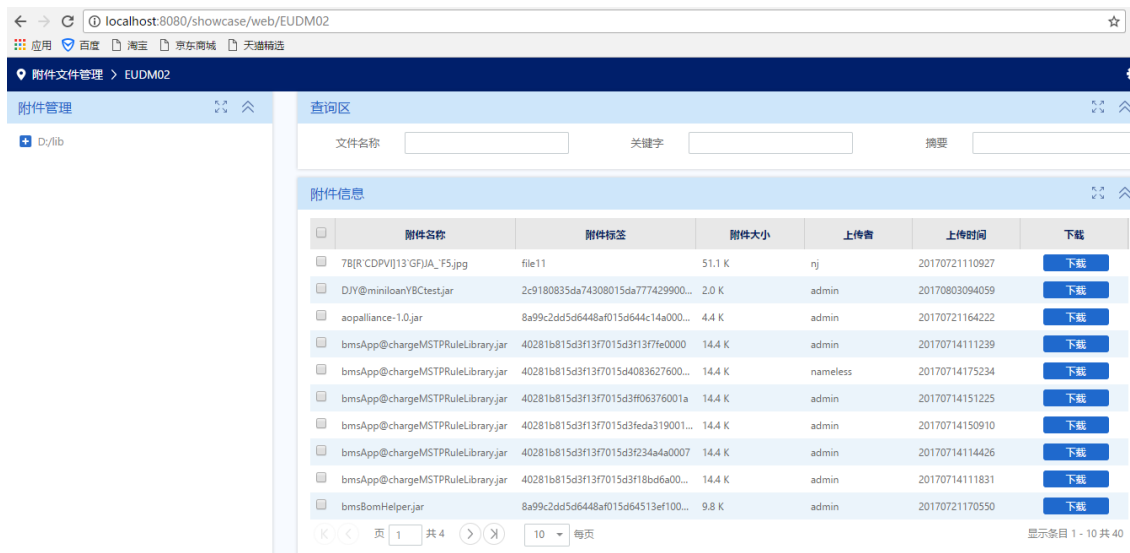
隐藏其他模块后，将最后用于展示模块正常适应屏幕即可。

```
$(".col-md-9").attr("class","col-md-12");
```

此处为栅型布局，故设置展示的 grid 的 class 为 col-md-12。

3.5.6.11.3 页面效果

不加参数：



拼接参数：docTag=file11



3.5.6.12 pagerPosition-grid 翻页条位置

3.5.6.12.1 页面功能简介

本文档用于说明 pagerPosition 属性的翻页样式。

3.5.6.12.2 配置和 API 详情

参考 [EFGGrid](#)。

3.5.6.12.3 页面开发步骤

3.5.6.12.3.1 JSP 页面开发

pagerPosition

```
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result" needAuth="false" autoDraw="override"
    pagerPosition="top" personal="true"
    enable="true">
    .....
  </EF:EFGrid>
</EF:EFRegion>
```

当我们设置 `pagerPosition` 属性为 `top` 的时候，翻页条将显示在 `grid` 右上方。

* 产品编号	商家公司名	* List选项	* 上市日期	* 时刻	产地	价格	小代码选项	搜索
1111111	公司: 22222		undefined	20170815 00:00:00	22222,33333,5555	¥15.21	http://platform.baosight.com	
0	公司: 22222		undefined		22222,33333,5555	¥15.21		

当grid上方无按钮时，翻页条将整行显示。如下图：

* 产品编号	商家公司名	* List选项	* 上市日期	* 时刻	产地	价格	小代码选项	搜索
1111111	公司: 22222		undefined	20170815 00:00:00	22222,33333,5555	¥15.21	http://platform.baosight.com	
0	公司: 22222		undefined		22222,33333,5555	¥15.21		

另外需要注意的是，只有 `pagerPosition='top'` 翻页条才会在grid上面展示，不设 `pagerPosition` 属性或者 `pagerPosition` 属性为其他值翻页条都将在grid下面展示。

1112	公司: NOKIA		undefined	20170711 01:00:00	China-中国,USA-美...	¥11.00	EWFM07	
S891	公司: NOKIA		undefined	20170711 00:15:00	China-中国	¥1,200.50	http://www.philips.com	search
8825	公司: NOKIA		undefined	20170711 01:00:00	China-中国	¥20,000.00		search

3.5.6.13 EFUpload 如何用弹出框的方式实现上传文件类型错误的提示

3.5.6.13.1 页面功能简介

此页面用于说明 **EFUpload** 控件如何用弹出框的方式实现上传文件类型错误提示的问题。

3.5.6.13.2 配置和 API 详情

参考 [EFUpload](#) 中的描述。

3.5.6.13.3 页面开发步骤

3.5.6.13.3.1 JS、JSP 页面开发

代码示例

```
<input type="file" name="files" id="photos" />

<script>
    $("#photos").kendoUpload({
        ...
        upload: onUpload
    });

    function onUpload(e) {
        // files 是一个包含上传文件信息的数组
        var files = e.files;

        // 检查每个上传的文件的后缀，如果不是.jpg 格式的话弹出信息提示框并且阻止上传
        $.each(files, function () {
            if (this.extension.toLowerCase() != ".jpg") {
                alert("Only .jpg files can be uploaded!");
                e.preventDefault();
            }
        });
    }
}
</script>
```

注意：要在 **upload** 事件中用弹出框的方式实现上传附件类型错误判断的话，就不能同时使用 **validation+invalidFileExtension** 的上传文件类型判断方法，否则弹出框方式不生效。

3.5.6.14 EEDM8020-按钮防重复点击提交

3.5.6.14.1 页面功能简介

本示例页面用于说明，防止按钮被重复点击。

3.5.6.14.2 页面开发步骤

3.5.6.14.2.1 JSP 页面开发

EEDM8020.jsp

```
<EF:EFPage title="按钮状态变更">
    <EF:EFRegion id="inqu" title="查询条件">
```

```

<div class="row">
  <EF:EFInput ename="inqu_status-0-product_id" cname="产品代号"/>
  <EF:EFInput ename="inqu_status-0-company_name" cname="公司名称"/>
  <EF:EFButton regionId="QUERY" cname="查询" ename="QUERY"></EF:EFButton>
</div>
</EF:EFRegion>
<EF:EFRegion id="result" title="记录集">
  <EF:EFGrid blockId="result"> <!-- EFGGrid 默认 autoDraw="yes" -->
</EF:EFGrid>
</EF:EFRegion>
</EF:EFPage>

```

3.5.6.14.2.2 JS 开发

EEDM8020.js

```

$(function () {
  $("#QUERY").click(function () {
    var btnNode = $(this);
    btnNode.attr("disabled", true);
    // 创建 EiInfo
    var ei = new EiInfo();
    ei.setByNodeObject(document.body);
    // 获取 数据
    // 调用请求 onSuccess 成功回调函数
    EiCommunicator.send("EE10", "query", ei, {
      onSuccess: function (ei) {
        resultGrid.setEiInfo(ei);
        btnNode.attr("disabled", false);
        // onSuccess 表示失败回调函数
      }, onFailure: function (ei) {
        // 发生异常
        btnNode.attr("disabled", false);
        NotificationUtil(ei);
      }
    });
  });
});

```

3.5.6.14.2.3 页面效果



3.5.6.14.2.4 原理说明

当点击上图所示的按钮"查询"时候，会触发它的执行逻辑，但是当它的执行逻辑还没有运行完毕之前，又发生了点击，这样又会触发一次上面的执行逻辑。当在上一次逻辑还没有执行完毕之前，再次触发上面的逻辑，这样会造成资源浪费。为了避免这种情况。可以通过控制 button 的 `disabled` 属性来防止重复点击。设置 `disabled = true`，按钮被点击，也不会触发后面的执行逻辑。

3.5.7 纯 CSS 页面布局

3.5.7.1 EEDM0910-0912 两栏布局（高度自适应）

3.5.7.1.1 页面功能简介

EEDM0910-0912 示例页面用于说明，用 CSS2 进行两栏布局的几种方式。

3.5.7.1.2 页面开发步骤

3.5.7.1.2.1 EEDM0910 两栏且无 header 的布局

示例代码如下：

EEDM0910.jsp

```
<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<!--页面布局：两栏无 header 的布局-->
<EF:EFPage>
    <style>
        body {
            overflow: hidden;
```

```
}

.demo-container {
  padding: 0 15px;
}

#right-part-container, .iplat-aside, .iplat-main {
  box-sizing: border-box;
}

.aside-wrapper {
  overflow: hidden;
  display: inline;
  position: fixed;
  float: left;
  width: 300px;
  padding-top: 42px;
  padding-bottom: 8px;
  top: 0;
  bottom: 0;
  z-index: 1000;
}

.iplat-aside {
  background-color: #D3DCE6;
  color: #333;
  height: 100%;
  line-height: 50px;
  padding: 0 20px;
  overflow-y: scroll;
}

.iplat-aside b {
  font-size: 16px;
}

#right-part-container {
  overflow: hidden;
  position: absolute;
  display: inline;
  float: left;
  width: 100%;
  padding-top: 42px;
  padding-right: 15px;
```

```

padding-bottom: 8px;
padding-left: 315px;
top: 0;
bottom: 0;
right: 0;
}

/* 滚动条在内部 */
.iplat-main {
background-color: #E9EEF3;
color: #333;
line-height: 200px;
padding: 20px;
font-size: 24px;
text-align: center;
height: 100%;
overflow-y: scroll;
}

</style>

<div id="demo-container" class="demo-container">

<div class="row">
<div class="aside-wrapper">
<aside id="aside" class="iplat-aside">
<b>Aside Menu</b> <br>
HTML<br>
CSS<br>
JavaScript<br>
高度自适应<br>
高度自适应<br>
高度自适应<br>
HTML<br>
CSS<br>
JavaScript<br>
高度自适应<br>
高度自适应<br>
高度自适应<br>
HTML<br>
CSS<br>
JavaScript<br>
高度自适应<br>

```

```

        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript
    </aside>
</div>

<div id="right-part-container">
    <main id="main" class="iplat-main">
        功能区 1<br>
        滚动条在内部<br>
        功能区 3<br>
        滚动条在内部<br>
        功能区 5
    </main>
</div>

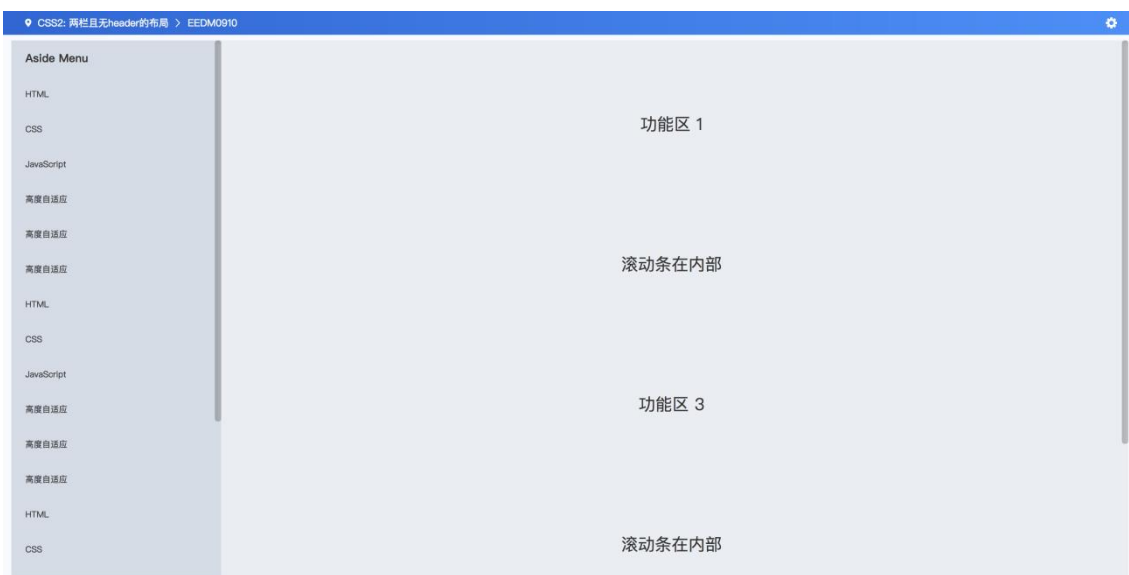
</div>

</div>

</EF:EFPage>

```

页面展示效果：



高度自适应：

```
.wrapper {
```

```

overflow-y: hidden;
position: absolute;
}
.content {
overflow-y: scroll;
height: 100%
}

```

外层的wrapper的位置固定，内层content高度为100%时，可实现高度自适应。

3.5.7.1.2EEDM0911 两栏且有 header 的布局

示例代码如下：

EEDM0911.jsp

```

<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<%--页面布局：两栏且有 header 的布局--%>
<EF:EFPage>
    <style>
        body {
            overflow: hidden;
        }

        .demo-container {
            padding: 0 15px;
        }

        #right-part-container, .main-area-wrapper, .iplat-header, .iplat-aside, .iplat-main {
            box-sizing: border-box;
        }

        .iplat-header {
            background-color: #B3C0D1;
            color: #333;
            line-height: 60px;
            padding: 0 20px;
            font-size: 18px;
            text-overflow: ellipsis;
        }
    </style>

```



```
        white-space: nowrap;
        overflow: hidden;
    }

    .aside-wrapper {
        overflow: hidden;
        display: inline;
        position: fixed;
        float: left;
        width: 300px;
        padding-top: 102px;
        padding-bottom: 8px;
        top: 0;
        bottom: 0;
        z-index: 1000;
    }

    .iplat-aside {
        background-color: #D3DCE6;
        color: #333;
        height: 100%;
        line-height: 50px;
        padding: 0 20px;
        overflow-y: scroll;
    }

    .iplat-aside b {
        font-size: 16px;
    }

    #right-part-container {
        overflow: hidden;
        position: absolute;
        display: inline;
        float: left;
        width: 100%;
        padding-top: 102px;
        padding-right: 6px;
        padding-bottom: 8px;
        padding-left: 315px;
        top: 0;
        bottom: 0;
        right: 0;
    }
```

```

.main-area-wrapper {
    height: 100%;
    overflow-y: scroll;
}

/* 滚动条在外部 */
.iplat-main {
    background-color: #E9EEF3;
    color: #333;
    line-height: 200px;
    padding: 20px;
    font-size: 24px;
}

.iplat-header, .iplat-main {
    text-align: center;
}

</style>

<div id="demo-container" class="demo-container">

    <header id="header" class="iplat-header row">
        页面布局：两栏且有 header 的布局
    </header>

    <div class="row">
        <div class="aside-wrapper">
            <aside id="aside" class="iplat-aside">
                <b>Aside Menu</b> <br>
                HTML<br>
                CSS<br>
                JavaScript<br>
                高度自适应<br>
                高度自适应<br>
                高度自适应<br>
                HTML<br>
                CSS<br>
                JavaScript<br>
                高度自适应<br>
                高度自适应<br>
                高度自适应<br>
            </aside>
        </div>
    </div>

```

```

        HTML<br>
        CSS<br>
        JavaScript<br>
        高度自适应<br>
        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript
    </aside>
</div>

<div id="right-part-container">
    <div id="main-area-wrapper" class="main-area-wrapper">
        <main id="main" class="iplat-main">
            功能区 1<br>
            滚动条在外部<br>
            功能区 3<br>
            滚动条在外部<br>
            功能区 5
        </main>
    </div>

</div>

</div>

</div>

</EF:EFPage>

```

页面展示效果：



3.5.7.1.2.3EEDM0912 两栏且有 header、footer 的布局

示例代码如下：

EEDM0912.jsp

```
<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<!--页面布局：两栏且有 header、footer 的布局-->
<EF:EFPage>
    <style>
        body {
            overflow: hidden;

            .demo-container {
                padding: 0 15px;

                #right-part-container, .main-area-wrapper, .iplat-header, .iplat-aside, .iplat-
main, .iplat-footer {
                    box-sizing: border-box;

                    .iplat-header, .iplat-footer {
                        background-color: #B3C0D1;
                        color: #333;
```

```
    line-height: 60px;
    padding: 0 20px;
    text-overflow: ellipsis;
    white-space: nowrap;
    overflow: hidden;
}

.iplat-header {
    font-size: 18px;
}

.aside-wrapper {
    overflow: hidden;
    display: inline;
    position: fixed;
    float: left;
    width: 300px;
    padding-top: 102px;
    padding-bottom: 8px;
    top: 0;
    bottom: 0;
    z-index: 1000;
}

.iplat-aside {
    background-color: #D3DCE6;
    color: #333;
    height: 100%;
    line-height: 50px;
    padding: 0 20px;
    overflow-y: scroll;
}

.iplat-aside b {
    font-size: 16px;
}

#right-part-container {
    overflow: hidden;
    position: absolute;
    display: inline;
    float: left;
    width: 100%;
    padding-top: 102px;
```

```

padding-right: 6px;
padding-bottom: 68px;
padding-left: 315px;
top: 0;
bottom: 0;
right: 0;
}

.main-area-wrapper {
height: 100%;
overflow-y: scroll;
}

/* 滚动条在外部 */
.iplat-main {
background-color: #E9EEF3;
color: #333;
line-height: 200px;
padding: 20px;
font-size: 24px;
}

.iplat-footer {
position: relative;
width: 100%;
margin-bottom: 8px;
margin-left: -9px;
padding-left: 9px;
}

.iplat-header, .iplat-main, .iplat-footer {
text-align: center;
}

</style>

<div id="demo-container" class="demo-container">

<header id="header" class="iplat-header row">
  页面布局：两栏且有 header 和 footer 的布局
</header>

<div class="row">

```

```

<div class="aside-wrapper">
  <aside id="aside" class="iplat-aside">
    <b>Aside Menu</b> <br>
    HTML<br>
    CSS<br>
    JavaScript<br>
    高度自适应<br>
    高度自适应<br>
    高度自适应<br>
    HTML<br>
    CSS<br>
    JavaScript<br>
    高度自适应<br>
    高度自适应<br>
    高度自适应<br>
    HTML<br>
    CSS<br>
    JavaScript<br>
    高度自适应<br>
    高度自适应<br>
    高度自适应<br>
    HTML<br>
    CSS<br>
    JavaScript
  </aside>
</div>

<div id="right-part-container">
  <div id="main-area-wrapper" class="main-area-wrapper">
    <main id="main" class="iplat-main">
      功能区 1<br>
      滚动条在外部<br>
      功能区 3<br>
      滚动条在外部<br>
      功能区 5
    </main>
  </div>

  <footer id="footer" class="iplat-footer">
    ©上海宝信软件股份有限公司 Copyright @1998-2018 BAOSIGHT Corporation. All
    Rights Reserved
  </footer>
</div>

```

```

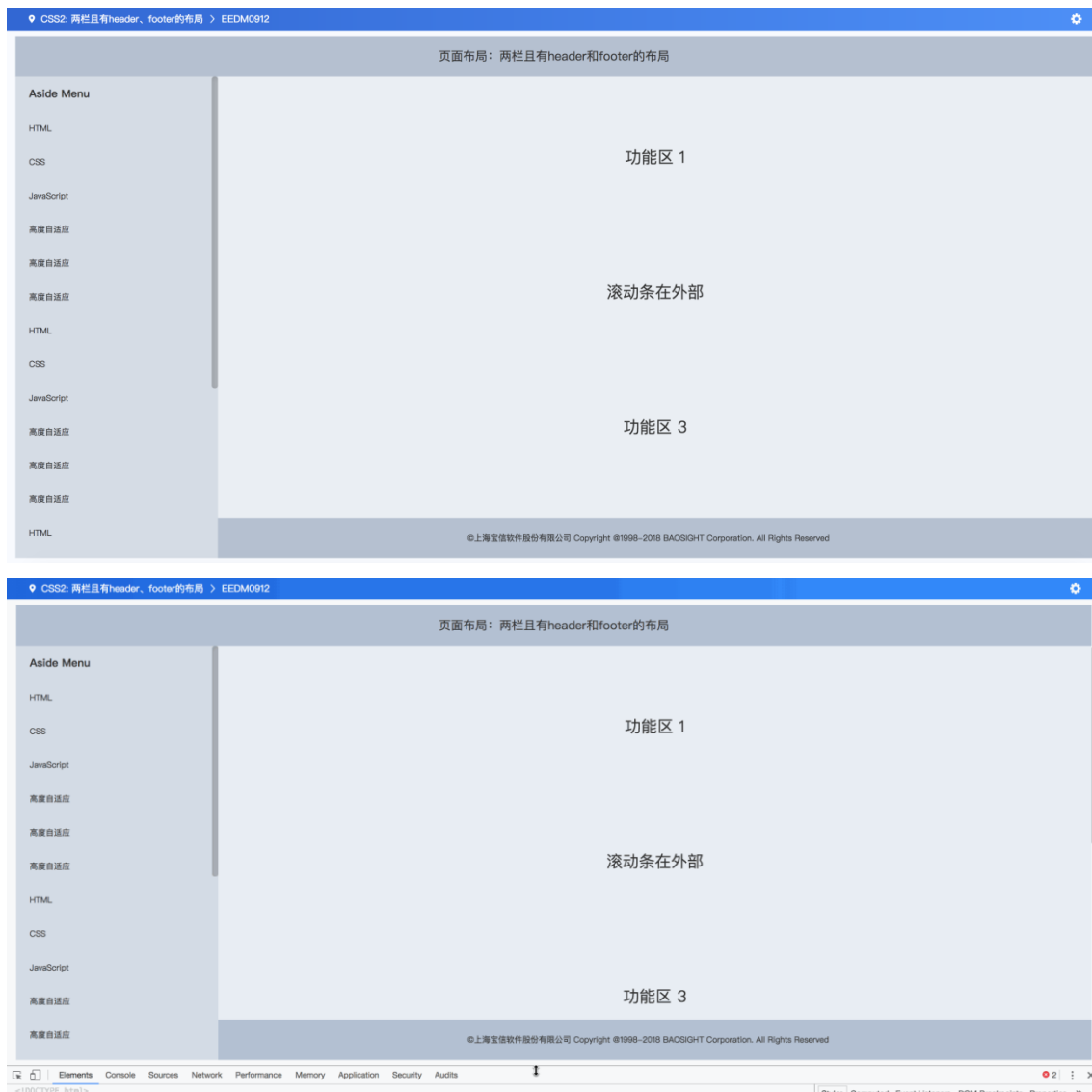
</div>

</div>

</EF:EFPage>

```

页面展示效果：



3.5.7.2 EEDM0913-0915 两栏布局（flex）

3.5.7.2.1 页面功能简介

EEDM0913-0915 示例页面用于说明，采用 Flex 布局实现常见的两栏页面布局。

3.5.7.2.2 Flex 布局

Flex 是 Flexible Box 的缩写，意为“弹性布局”，是一种可以灵活、简洁、快速布局的属性。

Flex 功能十分强大，有许多优势，如：独立的高度控制与对齐；独立的元素顺序；指定元素之间的关系；灵活的尺寸和对齐方式。

但兼容性方面不是非常理想，尤其是 IE 低版本的显示情况，下图为浏览器支持情况。

Current aligned	Usage relative	Show all								
IE	Edge	Firefox	Chrome	Safari	Opera	iOS Safari	Opera Mini	Android Browser	Chrome for Android	
			31							
			42							
			45							
			47							
			49							
			50							
8			51					4.3		
9		1	52					4.4		
10	13	6				9.3		4.4.4		
11	14	49	53	10	39	10	all	52	51	
		50	54	TP	40					
		51	55		41					
		52	56							

3.5.7.2.3 Flex 主要属性



3.5.7.2.4 页面开发步骤

3.5.7.2.4.1 EEDM0913 两栏且无 header 的布局

3.5.7.2.4.1.1 示例代码

EEDM0913.jsp

```
<EF:EFPage>
  <jsp:attribute name="header">
    <style>
      body {
        overflow: hidden;
      }

      /* 排列方向: row □ */
      .container {
        display: flex;
        flex-direction: row;
        width: 100%;
      }

      /* 左侧部分 */
      .aside-wrapper {
        width: 300px;
        flex: none;
      }

      .iplat-aside {
        overflow-y: scroll;
        height: 72%;
        background-color: #D3DCE6;
        color: #333;
        line-height: 50px;
        padding: 0 20px;
      }

      .iplat-aside b {
        font-size: 16px;
      }

      /* 主要部分, flex: 1: 按需分配 */
      .main-area-wrapper {
        flex: 1;
      }
    </style>
  </jsp:attribute>
</EF:EFPage>
```

```

.iplat-main {
    overflow-y: scroll;
    height: 72%;
    background-color: #E9EEF3;
    color: #333;
    line-height: 200px;
    padding: 20px;
    font-size: 24px;
    text-align: center;
}
</style>
</jsp:attribute>

<jsp:body>
    <div class="parent">
        <div class="row">
            <!--左侧部分-->
            <div class="container">
                <div class="aside-wrapper">
                    <aside id="aside" class="iplat-aside">
                        <b>Aside Menu</b> <br>
                        HTML<br>
                        CSS<br>
                        JavaScript<br>
                        HTML<br>
                        CSS<br>
                        JavaScript<br>
                        HTML<br>
                        CSS<br>
                        JavaScript<br>
                        HTML<br>
                        CSS<br>
                        JavaScript<br>
                        HTML<br>
                        CSS<br>
                        JavaScript<br>
                        HTML<br>
                        CSS<br>
                        JavaScript<br>
                        HTML<br>
                        CSS<br>
                        JavaScript

```

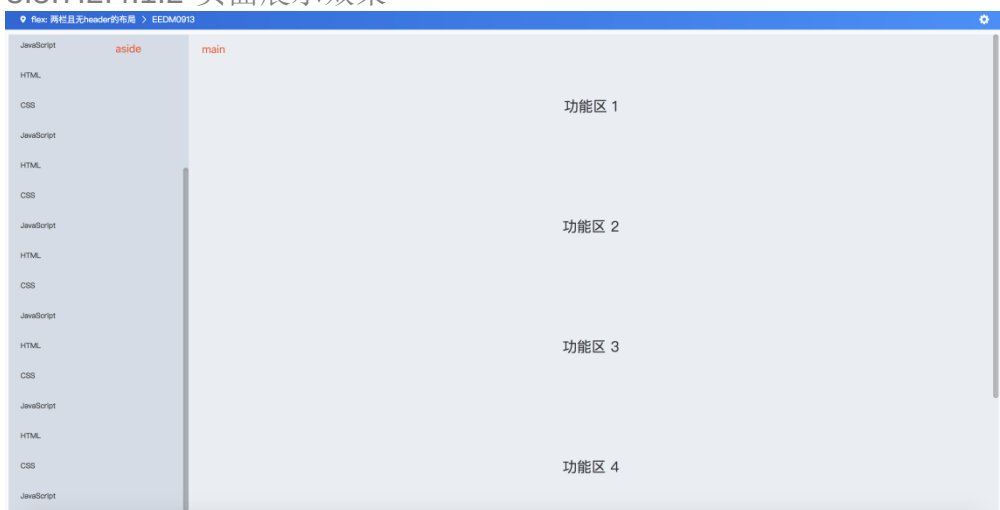
```

        </aside>
    </div>

    <!--主要内容-->
    <div id="main-area-wrapper" class="main-area-wrapper">
        <main id="main" class="iplat-main">
            功能区 1<br>
            功能区 2<br>
            功能区 3<br>
            功能区 4<br>
            功能区 5
        </main>
    </div>
</div>
</div>
</jsp:body>
</EF:EFPage>

```

3.5.7.2.4.1.2 页面展示效果



3.5.7.2.4.2 EEDM0914 两栏且有 header 的布局

3.5.7.2.4.2.1 示例代码

EEDM0914.jsp

```

<EF:EFPage>
    <jsp:attribute name="header">
        <style>
            body {

```

```

        overflow: hidden;
    }

    /* 排列方向: column □ */
    .parent {
        display: flex;
        flex-direction: column;
    }

    .iplat-header {
        flex: 0 0 50px;
        background-color: #B3C0D1;
        color: #333;
        line-height: 60px;
        padding: 0 20px;
        font-size: 18px;
        text-align: center;
    }

    /* 排列方向: row □, 依次为左侧内容和右侧主体内容 */
    .container {
        display: flex;
        flex-direction: row;
        flex: 1;
        width: 100%;
    }

    .aside-wrapper {
        width: 300px;
        flex: none;
    }

    .iplat-aside {
        overflow-y: scroll;
        height: 67%;
        background-color: #D3DCE6;
        color: #333;
        line-height: 50px;
        padding: 0 20px;
    }

    .iplat-aside b {
        font-size: 16px;
    }

```

```

    }

    .main-area-wrapper {
        flex: 1;
    }

    .iplat-main {
        overflow-y: scroll;
        height: 67%;
        background-color: #E9EEF3;
        color: #333;
        line-height: 200px;
        padding: 20px;
        font-size: 24px;
        text-align: center;
    }
</style>
</jsp:attribute>

<jsp:body>

<div class="parent">
    <!--头部-->
    <header id="header" class="iplat-header">
        页面布局：两栏且有 header 的布局
    </header>

    <div class="row">
        <div class="container">
            <div class="aside-wrapper">
                <aside id="aside" class="iplat-aside">
                    <b>Aside Menu</b><br>
                    HTML<br>
                    CSS<br>
                    JavaScript<br>
                    HTML<br>
                    CSS<br>
                    JavaScript<br>
                    HTML<br>
                    CSS<br>
                    JavaScript<br>
                    HTML<br>
                    CSS<br>
                </aside>
            </div>
        </div>
    </div>

```

```

        JavaScript<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        HTML<br>
        CSS<br>
        JavaScript
    </aside>
</div>

<div id="main-area-wrapper" class="main-area-wrapper">
    <main id="main" class="iplat-main">
        功能区 1<br>
        功能区 2<br>
        功能区 3<br>
        功能区 4<br>
        功能区 5
    </main>
</div>
</div>
</div>
</div>

</jsp.body>
</EF:EFPage>

```

3.5.7.2.4.2.2 页面展示效果



3.5.7.2.4.3EEDM0912 两栏且有 header、footer 的布局

3.5.7.2.4.3.1 示例代码

EEDM0915.jsp

```
<EF:EFPage>
  <jsp:attribute name="header">
    <style>
      body {
        overflow: hidden;
      }

      .parent {
        display: flex;
        flex-direction: column;
        height: 740px;
      }

      /* flex: 0 0 60px header 和 footer 不放大不缩小, 占据 60px */
      .iplat-header, .iplat-footer {
        height: 20%;
        flex: 0 0 60px;
        background-color: #B3C0D1;
        color: #333;
        line-height: 60px;
        padding: 0 20px;
        text-align: center;
      }

      .iplat-header {
        font-size: 18px;
      }

      .container {
        height: 100%;
        display: flex;
        flex-direction: row;
        flex: 1;
        width: 100%;
      }

      .aside-wrapper {
        flex: 0 0 300px;
```



```

    }

    .iplat-aside {
        height: 100%;
        overflow-y: scroll;
        background-color: #D3DCE6;
        color: #333;
        line-height: 50px;
        padding: 0 20px;
    }

    .iplat-aside b {
        font-size: 16px;
    }

    .right {
        display: flex;
        flex-direction: column;
        flex: 1;
    }

    .main-area-wrapper {
        flex: 1;
        height: 80%;
    }

    .iplat-main {
        height: 100%;
        overflow-y: scroll;
        background-color: #E9EEF3;
        color: #333;
        line-height: 200px;
        padding: 20px;
        font-size: 24px;
        text-align: center;
    }
</style>
</jsp:attribute>

<jsp:body>
    <div class="parent">
        <header id="header" class="iplat-header">
            页面布局：两栏且有 header 和 footer 的布局

```

```

</header>

<div class="row" style="height: 100%;">
  <div class="container">
    <!--left-->
    <div class="aside-wrapper">
      <aside id="aside" class="iplat-aside">
        <b>Aside Menu</b><br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        HTML<br>
        CSS<br>
        JavaScript
      </aside>
    </div>

    <div class="right">
      <div id="main-area-wrapper" class="main-area-wrapper">
        <main id="main" class="iplat-main">
          功能区 1<br>
          功能区 2<br>
          功能区 3<br>
          功能区 4<br>
          功能区 5
        </main>
      </div>
    </div>
  </div>
</div>

```

```

        <footer id="footer" class="iplat-footer">
            ©上海宝信软件股份有限公司 Copyright @1998-2018 BAOSIGHT
            Corporation. All Rights Reserved
        </footer>
    </div>

</div>
</div>
</div>
</jsp:body>
</EF:EFPage>

```

3.5.7.2.4.3.2 页面展示效果



3.5.7.3 EEDM0916-0918 三栏布局（高度自适应）

3.5.7.3.1 页面功能简介

EEDM0916-0918 示例页面用于说明，用 CSS2 进行三栏布局的几种方式。

3.5.7.3.2 页面开发步骤

3.5.7.3.2.1 EEDM0916 三栏且无 header 的布局

示例代码如下：

EEDM0916.jsp

```

<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

```

```

<c:set var="ctx" value="{pageContext.request.contextPath}"/>
<%--页面布局：三栏无 header 的布局--%>
<EF:EFPage>
  <style>

    body {
      overflow: hidden;
    }

    /* 高度自适应 */
    .main-view {
      position: absolute;
      width: 100%;
      height: 100%;
      left: 0;
      top: 0;
      padding-top: 42px;
      padding-right: 265px;
      padding-bottom: 8px;
      padding-left: 315px;
    }

    /* 滚动条在内部 */
    .main-view .column {
      float: left;
      position: relative;
      height: 100%;
    }

    .iplat-aside, .iplat-main {
      box-sizing: border-box;
    }

    .iplat-aside {
      background-color: #D3DCE6;
      color: #333;
      line-height: 50px;
      padding: 0 20px;
      overflow-y: scroll;
    }

    .iplat-aside b {
      font-size: 16px;

```

```

    }

    #aside-left {
        width: 300px;
        margin-left: -300px;
    }

    #aside-right {
        width: 250px;
        margin-right: -250px;
    }

    .iplat-main {
        background-color: #E9EEF3;
        color: #333;
        line-height: 200px;
        padding: 20px;
        font-size: 24px;
        text-align: center;
        width: 100%;
        overflow-y: scroll;
    }
</style>

<div class="row">
    <div class="container">
        <div class="main-view">
            <aside id="aside-left" class="iplat-aside column">
                <b>Aside Menu</b><br>
                HTML<br>
                CSS<br>
                JavaScript<br>
                高度自适应<br>
                高度自适应<br>
                高度自适应<br>
                HTML<br>
                CSS<br>
                JavaScript<br>
                高度自适应<br>
                高度自适应<br>
                高度自适应<br>
                HTML<br>
            </aside>
        </div>
    </div>
</div>

```

```

        CSS<br>
        JavaScript<br>
        高度自适应<br>
        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript
    </aside>

    <main id="main" class="iplat-main column">
        功能区 1<br>
        滚动条在内部<br>
        功能区 3<br>
        滚动条在内部<br>
        功能区 5
    </main>

    <aside id="aside-right" class="iplat-aside column">
        <b>Aside Menu</b><br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        高度自适应<br>
        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        高度自适应<br>
        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript
    </aside>
</div>

```

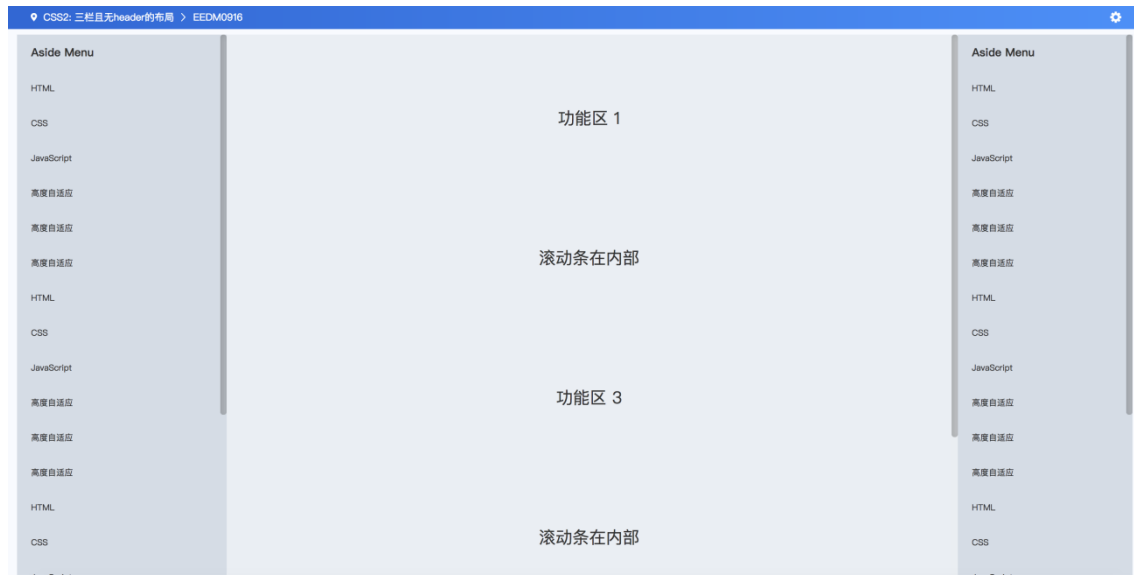
```

    </div>
  </div>

</EF:EFPage>

```

页面展示效果：



三列布局思路：

```

.main-view {
  position: absolute;
  width: 100%;
  height: 100%;
  left: 0;
  top: 0;
  padding-top: 42px;
  padding-right: 265px; // padding-right 250px for aside-right
  padding-bottom: 8px;
  padding-left: 315px; // padding-left 300px for aside-left
}

```

```

#aside-left {
  width: 300px;
  margin-left: -300px;
}

```

```

#aside-right {
  width: 250px;
}

```

```
margin-left: -250px;
}
```

3.5.7.3.2 EEDM0917 三栏且有 header 的布局

示例代码如下：

EEDM0917.jsp

```
<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<!--页面布局： 三栏且有 header 的布局-->
<EF:EFPage>
  <style>

    body {
      overflow: hidden;
    }

    .iplat-header {
      background-color: #B3C0D1;
      color: #333;
      line-height: 60px;
      padding: 0 20px;
      font-size: 18px;
      text-overflow: ellipsis;
      white-space: nowrap;
      overflow: hidden;
    }

    /* 高度自适应 */
    .main-view {
      position: absolute;
      width: 100%;
      height: 100%;
      left: 0;
      top: 0;
      padding-top: 102px;
      padding-right: 265px;
```



```
padding-bottom: 8px;
padding-left: 315px;
}

/* 滚动条在内部 */
.main-view .column {
    float: left;
    position: relative;
    height: 100%;
}

.iplat-aside, .iplat-main {
    box-sizing: border-box;
}

.iplat-aside {
    background-color: #D3DCE6;
    color: #333;
    line-height: 50px;
    padding: 0 20px;
    overflow-y: scroll;
}

.iplat-aside b {
    font-size: 16px;
}

#aside-left {
    width: 300px;
    margin-left: -300px;
}

#aside-right {
    width: 250px;
    margin-right: -250px;
}

.iplat-main {
    background-color: #E9EEF3;
    color: #333;
    line-height: 200px;
    padding: 20px;
    font-size: 24px;
    width: 100%;
}
```

```

        overflow-y: scroll;
    }

    .iplat-header, .iplat-main {
        text-align: center;
    }
</style>

<header id="header" class="iplat-header">
    页面布局：三栏且有 header 的布局
</header>

<div class="row">
    <div class="container">
        <div class="main-view">
            <aside id="aside-left" class="iplat-aside column">
                <b>Aside Menu</b><br>
                HTML<br>
                CSS<br>
                JavaScript<br>
                高度自适应<br>
                高度自适应<br>
                高度自适应<br>
                HTML<br>
                CSS<br>
                JavaScript<br>
                高度自适应<br>
                高度自适应<br>
                高度自适应<br>
                HTML<br>
                CSS<br>
                JavaScript<br>
                高度自适应<br>
                高度自适应<br>
                高度自适应<br>
                HTML<br>
                CSS<br>
                JavaScript
            </aside>

            <main id="main" class="iplat-main column">
                功能区 1<br>

```

```

        滚动条在内部<br>
        功能区 3<br>
        滚动条在内部<br>
        功能区 5
    </main>

    <aside id="aside-right" class="iplat-aside column">
        <b>Aside Menu</b><br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        高度自适应<br>
        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        高度自适应<br>
        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        高度自适应<br>
        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript
    </aside>
</div>
</div>
</div>

</EF:EFPage>

```

页面展示效果：



3.5.7.3.2.3EEDM0918 三栏且有 header、footer 的布局

示例代码如下：

EEDM0918.jsp

```
<!DOCTYPE html>
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="EF" tagdir="/WEB-INF/tags/EF" %>

<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<!--页面布局：三栏且有 header、footer 的布局-->
<EF:EFPage>
  <style>

    .i-theme-ant form.i-form {
      padding: 0;
    }

    body {
      overflow: hidden;
    }

    .iplat-header, .iplat-footer {
      background-color: #B3C0D1;
      color: #333;
      line-height: 60px;
      padding: 0 20px;
      text-overflow: ellipsis;
      white-space: nowrap;
```

```
        overflow: hidden;
    }

    .iplat-header {
        font-size: 18px;
    }

    /* 高度自适应 */
    .main-view {
        position: absolute;
        width: 100%;
        height: 100%;
        left: 0;
        top: 0;
        padding-top: 94px;
        padding-right: 250px;
        padding-bottom: 60px;
        padding-left: 300px;
    }

    /* 滚动条在内部 */
    .main-view .column {
        float: left;
        position: relative;
        height: 100%;
    }

    .iplat-aside, .iplat-main {
        box-sizing: border-box;
    }

    .iplat-aside {
        background-color: #D3DCE6;
        color: #333;
        line-height: 50px;
        padding: 0 20px;
        overflow-y: scroll;
    }

    .iplat-aside b {
        font-size: 16px;
    }

    #aside-left {
```

```

        width: 300px;
        margin-left: -300px;
    }

    #aside-right {
        width: 250px;
        margin-right: -250px;
    }

    .iplat-main {
        background-color: #E9EEF3;
        color: #333;
        line-height: 200px;
        padding: 20px;
        font-size: 24px;
        width: 100%;
        overflow-y: scroll;
    }

    .iplat-footer {
        position: absolute;
        left: 0;
        bottom: 0;
        right: 0;
        clear: both;
    }

    .iplat-header, .iplat-main, .iplat-footer {
        text-align: center;
    }

</style>

<header id="header" class="iplat-header">
    页面布局：三栏且有 header 和 footer 的布局
</header>

<div class="row">
    <div class="container">
        <div class="main-view">
            <aside id="aside-left" class="iplat-aside column">
                <b>Aside Menu</b> <br>
                HTML<br>

```

```

CSS<br>
JavaScript<br>
高度自适应<br>
高度自适应<br>
高度自适应<br>
HTML<br>
CSS<br>
JavaScript<br>
高度自适应<br>
高度自适应<br>
高度自适应<br>
HTML<br>
CSS<br>
JavaScript<br>
高度自适应<br>
高度自适应<br>
高度自适应<br>
HTML<br>
CSS<br>
JavaScript
</aside>

<main id="main" class="iplat-main column">
  功能区 1<br>
  滚动条在内部<br>
  功能区 3<br>
  滚动条在内部<br>
  功能区 5
</main>

<aside id="aside-right" class="iplat-aside column">
  <b>Aside Menu</b><br>
  HTML<br>
  CSS<br>
  JavaScript<br>
  高度自适应<br>
  高度自适应<br>
  高度自适应<br>
  HTML<br>
  CSS<br>
  JavaScript<br>
  高度自适应<br>
  高度自适应<br>

```

```

        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript<br>
        高度自适应<br>
        高度自适应<br>
        高度自适应<br>
        HTML<br>
        CSS<br>
        JavaScript
    </aside>

</div>
</div>
</div>

<footer id="footer" class="iplat-footer">
    ©上海宝信软件股份有限公司 Copyright @1998-2018 BAOSIGHT Corporation. All Rights
    Reserved
</footer>

</EF:EFPage>

```

页面展示效果：



